开发状态（随时更新）

> 本文件是**唯一权威的"当前进度"入口。**其它文档（`README.md`、`docs/0x`）描述的是 设计目标与已归档的验证结论，不反映任何未定稿的中间态。

- 仓库建立时间：2026-10-09 21:03 (+0800)
- 基线提交：`9e9aebc` `chore: 建立仓库基线（源码 + 权威输入文档 + 交付报告）`（85 文件 / 24,953 行）
- 当前 HEAD：`8bc224a`（**100** 个跟踪文件）。 ★ 本文件"当前 HEAD"一行**只写提交时能确认的既成事实**，因此它指向的通常是 **写这份文档时的上一个提交**；真正的 HEAD 请以 `git log --oneline -1` 为准。 （跟踪文件数演进：`9e9aebc` 85 → `32f9afb` 92 → `8bc224a` 100 → `faa6ba8` 103， 均可复核：`git ls-tree -r --name-only <commit> | Measure-Object`。） ★ 2026-10-10 更正：本行此前写成"`8bc224a`（92 个跟踪文件）"——**hash 换了、数字没换**， 92 是 `32f9afb` 的数字，属我自己引入的文档错误，已按实测量改正。
- 分支：`main`

---

1. 当前回归状态 —— 全绿（PASS 21 / 21）

> **★ 2026-10-12 23:52:26 干净全量复跑（v12 RTL）： `PASS 21 / 21`，21 台逐台 0 error**（`tb_enc_top` **488 s** 最长，其余 4–7 s；另跑 `tools/lint_all.ps1` = 逐文件 `xvlog` **ERROR 0 / WARNING 0**） ⇒ v12 的**遥测帧布局改动（§1.58）无功能回归**，日志 `sim/logs/SUMMARY.md` + `_synth_work/full_regression_v12.log`。 **★ v12 位流已重出并已上板**（生产版 2026-10-12 23:59:27 / ILA 版 2026-10-13 07:12:11，见 **§1.59**）⇒ **2026-10-13 12:29 烧录后读回 `0x05F2 = 0x000C`、遥测偏移 9 = `0xA2`，实机验证通过**（`0x000B` ⇒ 仍是 v11；§2 顶部现行块 + §1.59 未条）。 —— 过程中踩到一次**环境而非 RTL** 的假失败：首轮 `tb_enc_top` 报 `ERROR: [XSIM 43-3356] Could not open file xsim.dir/tb_enc_top_sim/xsim.type for writing`，根因是**上一轮被中断的 xsim/xsimk 进程仍挂着并占用该目录**（两个进程 PID 29468/28132，启动时间 = 上一次跑的时刻）； `Stop-Process` 掉这两个进程后单跑 `tb_enc_top` 立即 PASS(497 s)。⇒ **看到 `Could not open file … for writing` 先查残留 xsim 进程，不要怀疑代码。 ★ 2026-10-11 15:48:50 干净全量复跑（v7 RTL）： `PASS 21 / 21`，21 台逐台 0 error**（`tb_enc_top` **157 s** 最长，其余 3–5 s）⇒ v7 的两处常量改动（§2.8）**无功能回归**； **当时现行位流 = §2 顶部的 v11**（v7 已于 10-12 被 v9 → v10 → v11 取代，v11 又于 10-13 被 **v12** 取代； 本节"v7 RTL 无功能回归"只针对 10-11 那一轮）。 ★ 2026-10-11 15:35:20（同一 21 台）： `PASS 21 / 21`，21 台逐台 0 error **（上一版 13:31:39 是 20/20；本轮**新增 `sim/tb_loop_abz_chain.v` **（§1.39）并** 删除已被它取代的 `sim/tb_abz_chatter.v` **⇒ 21 台。删它的理由：它把 ILA 抓到的** 已滤波**角度直接回放给 `abz_gen`，实测 `edge_cnt` 净增量恒为 0…6，而硬件同一窗口是** +20 = 理想 **⇒ 是**该回放方式**（缺预热/节拍重构）不代表硬件，其 7 条断言编码了错误期望； 同一问题已由 `tb_loop_abz_chain.v` （回放**原始**角度经真实 `loop_filter`）正确复现， 见 §1.39 ⑨.4；旧日志已归档为 `sim/logs/_ORPHAN_tb_abz_chatter_superseded.log`（带来源说明头， `python tools/tidy_sim_logs.py` 复核：21 台 / 23 日志 / 无未归档孤儿）。 `tb_enc_top` 152 s **最长，其余 3–5 s）。 RTL 在 15:35 那次只新增**默认关闭**的** `parameter integer GATE_STATE = 0` **（默认 0 ⇒ 行为与改动前一致）； 15:48 那次之前又落了** v7 的三处常量**（§2.8）⇒** 改完重新回归仍是 21/21，**位流见** §2.8**。**

`sim/logs/SUMMARY.md`（2026-10-10 **16:21:05** 那次干净串行复跑 —— 文档回写后的复核， 同一套用例在 **16:03:11** 也跑过一次，两次都是 19/19；在 §1.20/§1.21 的 ILA 位序修复与 OTR 软清位、以及 §1.22 的 UART→SPI 桥之后；此前一版是 15:09:13（18/18）， 再往前是 08:37:24，含 §1.15 新增的 C19 与 T12/T12a-c 断言、§1.16 的 P1 判据统一、§1.17 的标定窗口修复、§1.18 的**幂等标定修复**、 以及 **TEST F** 与全部注释/数字同步）： **PASS 19 / 19**，19 台逐台 0 error，`REGRESS_EXIT=0`。逐台 checks： `tb_enc_top` **81**（39 → 63 → 72 → 81：TEST F 及幂等判据 F11a–h，再加 §1.21 的 **TEST D2** 端到端 OTR 软清位 9 条）、`tb_uart_scope` 94、`tb_uart_spi_bridge` **44**（§1.22 新增）、 `tb_calib_min` 54、`tb_uart_period` 27、`tb_xor_probe3` 18、`tb_uart_baud` 16。 耗时（无竞争环境下的真实值）： `tb_enc_top` **125–152 s** 最长，其余 3–5 s。

> ⚠️ **耗时数字的一个坑（我自己先踩了）**：更早一次记录里 `tb_enc_top` 写的是 **457 s**， 那是在**两个回归并发抢同一 `%TEMP%` 目录、且有进程持有 `xsim.xdbg` 锁**的情况下测到的， **不是这台测试台的真实耗时。**本次干净串行复跑的实测是 **141 s**（日志内自证： `$finish called at time : 259761710 ns`， `Simulation CPU Usage: 131593 ms`， `TEST PASSED (39 checks)` —— 该次测量在 §1.17 的 TEST F 之前，check 数当时是 39；现在同一台是 **81 checks / 125–152 s**）。 ⇒ **耗时数字只在干净串行环境下可比；并发跑的墙钟时间一律不可引用。** `tools/lint_all.ps1 -Clean` → 37 文件 **ERROR 0 / WARNING 0**（含新增的 `rtl/uart_rx.v`、`rtl/uart_spi_bridge.v`、`sim/tb_uart_spi_bridge.v`； `_tools/vlint.py` 同类静态检查 38 文件 **0 error / 3 条无害提示**： `tb_enc_top.v:130` 的 `ad_clk2`、`tb_spi_slave.v:47/:100` 的 `clr_otr` 未连接）。 `python tools/pinout_verify.py` → **PASS**（50/50 端口全部有 PACKAGE_PIN + IOSTANDARD，0 未约束；RTL 端口数 50 = 已约束 50）； `python tools/pinplan_check.py` → **PASS**； `python model/abz_uvw_model.py` → **6/6 PASS**（含第 6 项 UVW 迟滞定点自检，见 §1.4）； `python tools/to_lf.py --check` → **exit 0**（CRLF 0 / BOM 0 / .ps1 缺 BOM 0）； `python tools/scan_dangling.py` → **exit 0 / 无悬空死信号**（见 §1.5）。

★ 本轮（用户 m00001/m00058："重新进行 vivado 仿真，确保项目完全可靠，确保上板测试 不出问题。有问题一定要真实指出，不能糊弄"）把测试台从 **13 台扩到 18 台**， 补掉了 3 个**零覆盖**的模块，并因此**查出 1 个真 RTL 缺陷 + 2 个"我的测试台在空转"。** 全部逐条记在 §1.9–§1.13，**没有一条被淡化。**

新增测试台	补的覆盖	结果
`sim/tb_calib_min.v`	`rtl/calib_ctrl.v`（**原来零覆盖**）	PASS 54 checks
`sim/tb_uart_scope.v`	`rtl/uart_scope.v` 的 `uart_tx` / `uart_scope`（**原来零覆盖**）	PASS 94 checks（**v12 帧布局 v2 后为 96 checks**，见 §1.58）
`sim/tb_uart_period.v`	帧周期 / `frame_cnt` 不变式	PASS
`sim/tb_uart_baud.v`	`BAUD_DIV` 扫描 4/7/10/25 + 逐字节位周期	PASS
`sim/tb_xor_probe3.v`	帧 XOR 语义 + "帧头常量必须异或进去"（T12 把一次**我自己的**误判固化成判据，见 §1.15）	PASS

★ 回归必须在**工作区目录之外**运行 xsim（见 §1.0 的根因）。 `tools/run_sim.ps1` 已自动探测并回退，直接跑即可，不需要人工干预。**但本机实测工作区内也能跑通**， 见 §1.0 末尾的"现状澄清"。

1.0 ★ xsim 崩溃的真根因（已定位并已规避）

> **现状澄清（2026-10-10 02:23，本轮实测）**：下面这段"工作区之内必崩"的结论 **在本轮沙箱环境下不成立。**本轮 `tools/run_sim.ps1 -Rebuild` 的自动探测 **选中了工作区内的 `C:\Users\zhaoy\Desktop\FPGA\_sim_work` 并完整跑完 13 台**，没有回退到 `%TEMP%`。本机 `icacls C:\Users\zhaoy\Desktop\FPGA` 实际返回 `Everyone:(CI)(DENY)(DC)` + capability SID `S-1-4-658417623-361858394:(OI)(CI)(W,D,DC)`，与下面引用的原文（`Deny Everyone DeleteSubdirectoriesAndFiles`）**表述不同**，且 `DC`（删除子项）与 xsim 建 file mapping 之间没有已证实的因果链。⇒ **现象历史为真，但"根因是工作目录位置"这个结论未被证实**（§1.1 的 BOM 问题是 已被证实的那一类）。保留下面的原文以便对照，不要把"必须在工作区之外"当成物理定律。

现象：`xsim <snapshot> -runall` 立即崩溃，连 3 行 hello world 也一样：

```
# xsim {hs} -autoloadwcfg -runall
Could not open file mapping object (2).
Time resolution is 1 ps
Abnormal program termination (EXCEPTION_ACCESS_VIOLATION)
```

根因（实测二分，不是推测）：与本工程、RTL、Vivado 版本、内存、杀软、 残留进程/文件、临时目录、 `-autoloadwcfg`、多线程**都无关。** 唯一相关变量是——**xsim 的工作目录是否位于本工作区根目录之下**：

工作目录	xsim 结果
`C:\Users\zhaoy\Desktop\FPGA\...`（任意子目录，含 `_sim_work/`）	**崩溃**
`C:\Users\zhaoy\Desktop\probeB`（工作区**外**，同一磁盘）	正常
`C:\probeB`、`D:\...`、`%TEMP%\...`	正常

决定性对照（同一份 hello world、同一 shell、同一份 PATH）：

```
INSIDE  workspace (Desktop\FPGA\probeB)  : FAIL (file mapping)
OUTSIDE workspace (Desktop\probeB)       : OK
OUTSIDE workspace (C:\probeB)            : OK
```

工作区根目录被沙箱加了额外的 ACE（`Deny Everyone DeleteSubdirectoriesAndFiles` + 一条 capability SID 的 `Allow Write/Delete/Synchronize`），xsim 建命名 file mapping 时被拦下，且它**不报权限错、只报找不到映射对象**，所以看起来像 "环境损坏"。实际**重启机器并不能解决**（已实测：重启后 22:14 仍崩）。

已排除项（全部实测，留档以免重复排查）：非 ASCII 路径（`D:\probe\中文目录` 正常）、超长路径、文件系统（C/D 都是 NTFS）、`TEMP`/`TMP` 指向（改成 ASCII 目录 仍崩）、`-autoloadwcfg`（去掉同样崩）、多线程（`xelab` 的 slave threads 与崩溃无关）、 工作区内已有的 `xsim.dir`/`.Xil` 残留（移开仍崩）、`Stop-Process vivado -Force` 留下的内核对象（**重启后依旧崩，此推测已被证伪**）。

规避（已落地在 `tools/run_sim.ps1`）：脚本启动时先用一份 3 行 hello world 探测"当前工作目录能不能跑 xsim"，跑不动就自动把工作目录切到工作区之外（`%TEMP%\fpga_sim_work`），并打印警告。源码、日志仍留在仓库内，只有 `xelab`/`xsim` 的中间产物落在外面。也可以手动指定：`-WorkRoot <路径>`； 想强制用仓库内目录（比如在真实的非沙箱终端里）：`-NoAutoWorkRoot`。

1.1 ★ 一个由我引入并已修掉的真回归：去掉 `.ps1` 的 BOM

提交 `5ab51da` 做换行符/编码规范化时，把 `tools/*.ps1` 的 UTF-8 BOM 一并去掉了。 **这是错的**：这些脚本含中文，Windows PowerShell 5.1 对**无 BOM** 的 UTF-8 `.ps1` 按系统 ANSI 代码页解码，中文字符被拆开后有字节吃掉了字符串收尾引号：

```
At tools\run_sim.ps1:170 char:1
+ }
+ ~
Unexpected token '}' in expression or statement.
```

后果是 13 台回归一台都跑不起来（SUMMARY 全 `NO-VERDICT`）。修法：三个脚本恢复 UTF-8 BOM（保持纯 LF），`.gitattributes` 里 `*.ps1` 由 `text eol=lf` 改成 `binary`（git 不再做任何转换），并用 AST 解析器逐个验证通过。 **★ 教训：不要把"统一去 BOM"无差别套到 `.ps1` 上。**

1.2 ★ 死信号清理（全仓 `unused_ok` 扫描的产出）

新增 `tools/scan_unused.py`（去注释/去字符串字面量后按词边界计数）扫全仓。 结论：`rtl/*.v` 里已无"声明了却从未被引用"的信号。`rtl/enc_top.v` 的 `wire unused_ok` 汇聚列表 20 项**全部另有 2~6 个真实读者**，不是悬空信号。 真死信号在 `rtl/pole_counter.v`： `step_small_f` / `step_small_b` 与随之失效的 `localparam [31:0] STEP_LIM`，已删除，并把"为什么故意不定义"写进代码注释 （`step_bwd = 2^AW − step_fwd` 恒成立，若拿 `step_small_b` 当负跨零判据会与正跨零 条件恒等，一次正跨零同时报两个方向）。

1.3 ★ uvw_hyst 8192 粗栅局限 —— 已修复

`rtl/uvw_gen.v` 的迟滞基准已从旧 `SECT_W = 2^(AW-3) = 8192` 粗栅格改成 **真实扇区边界** `BND[k] = ceil(k * 2^AW / 6)`（AW=16 ⇒ 0/10923/21846/32768/43691/54614/65536）。

修复过程中定位到三个真缺陷（全部有隔离探针逐拍证据）：

#	缺陷	证据与修法
1	**冷启动基准取错**：`r_fwd = seg_lo` / `r_bwd = seg_hi`（"角度所在段的两条边界"）。开机角度落在段高端时，`seg_hi` 已是段上界，第一次前向提交要求 `eang >= seg_hi + H` 而 `seg_hi` 就是下一条边界 ⇒ 本该在 `BND+H` 提交的点**永不满足**	改为**稳态语义**：`r_fwd = BND[sec_raw+1]`、`r_bwd = BND[sec_raw]`，第一拍判据就与稳态同一门限

#	缺陷	证据与修法
2	**判据引用的基准比 `r_sec` 晚一拍**：`r_fwd` 与 `r_sec` 虽在同一 `always` 块非阻塞赋值，但 `r_fwd` 又组合参与 `off_fwd` 求值 ⇒ 提交那一拍仍用**旧基准**（正是刚被越过的那条） ⇒ `off_fwd >= H` 保持 ⇒ **下一拍再提交一次**，一个边界跳两个扇区	判据改用与 `r_sec` 同拍的组合基线 `bnd_of(r_sec+1)` / `bnd_of(r_sec)`，使"比较用基准" == "提交后生效基准"
3	**扇区 5→0 的环绕被吞掉**：`BND[6]=65536` 在 16 位角度里退化成 0，`(eang − 65536)` 恒为负 ⇒ 前向判据永不成立，扇区卡在 5	这唯一一条环绕边界写成双分支：`eang < BND[1]` 时 `off_fwd = eang`（"绕回后的推进量"），使 5→0 提交点与其它扇区的 `BND[k]+H` 同构

另有 1 处**纯观测型**修正：`r_bnd`（只作探针/文档观测量，不参与任何逻辑）在反向 提交时记为"新扇区上边界" `bnd_of(nxt_sec_b + 1)`，与前向 `bnd_of(nxt_sec_f + 1)` 几何对称 —— 两者都等于**刚跨过的那条真实边界。**

验证：`tb_uvw_gen` TEST 9 全 65536 点扫描，实测扇区边界 `1->10923 2->21846 3->32768 4->43691 5->54614`（与 `ceil(k*65536/6)` 逐项一致）； 前向/反向切换点对称落在 `BND[k] ± H`； 死区内抖动零切换。

隔离探针：`_sim_work/probe_iso.v`（模块 `probe_iso`）——只跑一个 `uvw_gen` + 一个独立写出的参考模型，逐拍比对 `r_sec` 并输出事件轨迹。**这是本轮唯一有效的调试手段**：不依赖 `sim/tb_uvw_gen.v`、不依赖 `enc_top`，单台迭代 ~10 s。

1.4 ★ Python 参考模型曾与 RTL 脱钩 —— 已修复

`model/abz_uvw_model.py` 里**还留着 RTL 已经删掉的旧迟滞实现**（固定 `SECT_W = 2^(AW-3) = 8192` 栅格、`d_fwd = (eang - r_bnd) & 0xFFFF`）， 而 RTL 已改成真实边界 `ceil(k*2^AW/6)` ⇒ 两者**不再逐位一致。** 更糟的是模型对 UVW 迟滞**一条断言都没有**（原来的 5 项 PASS 全是 ABZ 侧的）， 所以这个脱钩一直没被发现。

修法：把模型的组合段改成与 RTL 同构（真实边界 + 进位比较 `carry_hi = 2^AW - bnd_hi`）， 并新增自检 **[6] UVW 迟滞**：全 65536 点扫描 + 单调性 + 迟滞带 4 定点。 实测 `shift=6`（H=1024）提交点 `[11947, 22870, 33792, 44715, 55638] = BND[1..5] + 1024`，与 RTL `tb_uvw_gen` TEST 9 逐项一致；20 个带边界定点 0 不符。 `shift=2`（H=16384 = 1.5 个扇区宽）带重叠，提交点 `[27307, 38230, 49152, 60075]`， 单调性仍 PASS（**H ≥ 半扇区宽时单调扫描本就该跳扇区，不是缺陷**）。

1.5 ★ 全仓悬空信号扫描：`tools/scan_dangling.py`（新增工具）

`tools/scan_unused.py`（**保留**）是按文件统计词频、报告"出现次数 == 1"。它**查不到** 三类真问题：① 读者是别的模块里的哑元端口；② **空端口连接** `.port()`（词频统计 完全看不到）； ③ 整个模块的输出在顶层没连出去。为此新增 `tools/scan_dangling.py`。

做法：去注释/去字符串（**保留行列位置**）→ 抽全部 `module` 与端口方向 → 解析 `Mod #(...) u_inst ( .port(sig), .port2() );` 建**跨文件实例端口连接图** → 对每个标识符算"真实读者数 = 出现次数 − 声明处 − 纯汇聚行"。输出四张表： A（无 RTL 读者，再分 A1 带 `(* mark_debug *)` 的 ILA 探针 / A2 真死信号）、 B（空端口连接，按方向分类）、C（唯一读者是暗哑元）、D（`unused_ok` 列表逐项）。

修扫描器自己的三个 bug（写下来避免重犯）： 1. `#(...)` **参数表被当成端口表** —— `module foo #( parameter ... )( input ... );` 里 第一对括号是参数表。必须先整段跳过 `#(...)` 再取下一对括号。 2. **★ 逗号分隔的名字列表里只能用 `[ \t]`，不能用 `\s`** —— `\s` 会吃掉换行，把下一行的 方向关键字 `output` 当成端口名捞进来，表现为"**隔一个漏一个**"。实测错误形态： `'output' -> 'cfg_kp,\n output'`。修好后必须把 `input` / `output` / `inout` 从结果里 pop 掉。 3. 多行 ANSI 端口列表需要 `re.S`； `(* attr *)` 属性块要先抹掉。

扫描结果（修好后 `exit 0`）： - **A1：34 个带 `(* mark_debug *)` 的 ILA 探针**（`rtl/enc_top.v:212-215, 244-246, 299-302, 357-359, 404, 419-424, 467-473, 513-517, 599` 等）—— 有意只给 ILA 抓，**不是缺陷。** - **A2：0 个真死信号。**扫描首次跑出 1 个： `wire [17:0] ph_idx = ph[17:0];` （`sim/tb_enc_top.v:224`），全文仅此一次出现、从未被任何语句引用（`ph` 的实际 LUT 索引是 `ph[15:0]`，见 `sim/tb_enc_top.v:242-243`）⇒ **已删除**并在注释里留痕。 - **B：空端口连接 51 处，其中 `input` / `inout` 悬空 0 处**（即**没有驱动端悬空**）。 51 处**全部是 `output` 主动丢弃**：`rtl/enc_top.v` 4 处（`u_biss(.frame_total_bits())`、`u_dec_c(.dout_valid())`、`u_regs(.cfg_h2_k())`、`u_scope(.frame_cnt())`），其余 47 处在 `sim/tb_biss_c_slave.v`、`sim/tb_spi_slave.v` 的 testbench 例化里（只验接口不验全部输出， 正常写法）。 - **C：0 个**（没有"信号被接收却完全不参与逻辑"的情况）。 - **D：`rtl/enc_top.v` 的 `unused_ok` 汇聚列表 20 项全部有真实读者，充数项 0。** 真实读者数： `dir` 17、`edge_cnt[0]` 19、`wrap_neg` 16、`amp_ok` 14、`pole_pairs_eff[0]` 12、 `cfg_st_bits[0]` 9、`cfg_mt_bits[0]` 9、`cor_valid` 5、`cordic_mag[0]` 3、`biss_frame_cnt[0]` 3、 `pos_ext[0]` 3、`biss_bit_err` 2、`biss_frame_short` 2、`biss_sl_rx_valid` 2、`biss_sl_rx[0]` 2、`calib_state_dbg[0]` 2、 `amp_valid` / `abz_valid` / `uvw_valid` / `pole_valid` 各 1。 ⚠️ 其中 `amp_valid` / `abz_valid` / `uvw_valid` / `pole_valid` **只有 1 个真实读者**——不是悬空， 但确实"几乎没被用"，如实记录在此。

★ 退出码语义（避免误判）： 只有真问题才 exit 1 —— A2 真死信号、B 里 `input` / `inout` 悬空、C 非空、D 出现充数项。**A1 的 ILA 探针与 B 里 `output` 空连接不计入**（前者有意， 后者是正常写法）。

1.6 ILA 调试版：可核查的事实，以及它不能证明什么

> **★ 本节数字已由 §2 顶部记录的 2026-10-10 16:12 重建版取代** （`enc_top_ila.bit` **1,445,812 B**、`Timing constraints are not met.` / `sys_clk` **WNS −3.329 / TNS −84.745 / 26 失败 / 18010 端点**、DRC 62 checks 全 `Related violations: <none>`（0 error / 0 critical）、 LUT **8822** / FF **8544**）。下面这段保留为 **10-09 首版**的历史记录（当时的位流**早于** `calib_ctrl` / `uart_scope` / `enc_top` 三处功能修复，**已作废**）。

可核查的事实（全部可在产物里复核）： - `_synth_work/enc_top_ila.bit` 1,473,128 B + `enc_top_ila.ltx` 67,006 B； `impl_ila_drc.rpt` `0 error / 0 critical`； `place_design` / `route_design` 走完。 - 加了 ILA 之后 `Bonded IOB` 仍是 **49**（与生产版一致）⇒ `dbg_hub` 的 JTAG 走的是 专用 BSCAN 原语，**没有占用任何新增用户引脚**，接线不受影响。 - `BUFGCTRL` 1 → **2**（多出来的一个是 `dbg_hub` 的 `BSCAN` 时钟）； BSCAN 域 WNS **+27.404 ns**（928 端点，TCK 域），不挤占用户时序。 - 用户时钟 `sys_clk` WNS **+0.698 ns / 16906 端点 0 失败** ⇒ ILA 版同样收敛。 **（10-09 首版的历史数字；2026-10-10 16:12 用当前 RTL 重建后为 `sys_clk` WNS −3.329 ns / 26 失败端点 / 18010 端点，见 §2）** - 探针清单 `_synth_work/ila_probe_list.txt`： **34 条命名探针 / 总位宽 252 bit**， 每条 ≤32 bit，全部 `dbg_*` 网在 `clk_i`（50 MHz）域同拍采样。 - 探针覆盖率：SPI 状态字 `stat_byte`（`rtl/enc_top.v:588-597`）的 8 个 bit 里 **7 个可由探针还原**，另 1 个额外靠 `dbg_stat_byte[2]` 兜住：

bit	名	表达式	能否单独还原
7	res	`1'b0`	恒 0
6	adapt	`1'b0`	恒 0
5	scamp	`amp_low | amp_high`	✅ 就是 `dbg_stat_byte[5]`

bit	名	表达式	能否单独还原
4	lagfatl	`lag_fatal`	✅ `dbg_lag_fatal`
3	inclim	`limiter_active | uvw_freq_warn`	✅ 两项各有独立探针
2	laglim	`lag_limit | lag_warn`	⚠️ 只能取到 `dbg_lag_warn`，`lag_limit` 无独立探针；组合值用 `dbg_stat_byte[2]`
1	falarm	`1'b0`	恒 0
0	oflow	`cor_sat | otr_lat`	✅ 两项各有独立探针

BiSS 帧的 W 位 = `amp_low | amp_high | limiter_active`（`rtl/enc_top.v:519`）= **`dbg_stat_byte[5] | dbg_limiter_active`**，可还原。 ★ 注意 `dbg_uvw_freq_warn` **不在** BiSS 的 W 位里，它只进 SPI 状态字的 `bit3`。

★ 它不能证明什么（必须写清楚，不要含糊）： - **★ 没有上板。**上面全是"位流已生成 + 时序收敛 + DRC 干净 + 引脚未变"的静态证据， **不等于 ILA 真能抓数。**真正的验收只有一条：把 `enc_top_ila.bit` 下载进去、 用 Vivado Hardware Manager 打开 `enc_top_ila.ltx`、触发一次并看到非全 0 的波形。 这一步**只能上板做，本机无法完成。** - ILA 采样时钟固定在 `clk_i`（50 MHz）⇒ 单次捕获窗口 = 1024 / 50 MHz = **20.5 µs。** 1 kHz 的机械周期是 1 ms，**一个捕获窗口看不完一个周期**，需要多次触发或先 用 UART 遥测导出长序列。这不是缺陷，但别在报告里说"ILA 看一个完整周期"。 - 50 MHz 对 ABZ/BiSS 的**边沿细节**不够：ABZ 在 1024 线 × 4 倍频下边沿间隔约 0.5 µs（在窗口内够看几十个边沿），但 BiSS `ma` 若开到 10 MHz，一个 `ma` 周期 只有 5 个采样点 ⇒ 只能看帧级行为，**不能用 ILA 判 BiSS 的建立/保持余量。**

1.7 ★ 探针清单的编码坑（已修，根因比原先以为的深一层）

`tools/ila_debug.tcl` 原先用 `open ... w` 写 `ila_probe_list.txt`，Tcl 默认按**系统 ANSI 代码页**（中文 Windows = cp936）编码 ⇒ 中文表头写成 GBK 字节，用 UTF-8 工具 打开是乱码（`read` 直接报 `invalid UTF-8 text`）。

★ 2026-10-10 重建 ILA 时发现"已经加过 `-encoding utf-8` 却仍然是乱码"， 实测文件头两行是 `鏅㈡拡绔ˉ鐚 f 暄 = 34 / 瀛楁⁰µ: ...`（探针行本身是纯 ASCII、完全正确）。 真根因是**双重编码**：Vivado **也按系统代码页读取 `.tcl` 脚本本身**，脚本里的中文 字面量在**解析阶段**就已经被错解成乱码字符，再用 `-encoding utf-8` 写出去 ⇒ 乱码被"固化成 UTF-8 字节"。 ⇒ **定案修法：这类落盘索引一律写 ASCII**（表头改成 `probe_ports = ... / fields: ...`）， 不再依赖任何编码设置；现存文件已重写为 ASCII 表头（无 BOM、纯 LF，1,291 B）。

★ 顺手记一个自己踩的坑：修这个文件时我先用"utf-8 → gbk → cp936 → latin-1"逐个 `try` 去猜编码，结果 `latin-1` 永远成功（它能把任意字节解出来），于是把文件**二次 搞坏。不要用 latin-1 做编码探测的兜底**；正确做法是先用 `utf-8` 严格解码， 失败就说明它不是 UTF-8，再按已知来源（哪个工具写的）直接定编码。 ★ 同一类教训在别处也出现过（Tcl/PowerShell 写文件、`Out-File -Encoding utf8` 带 BOM、 `to_lf.py --check` 只扫已跟踪文件）——**凡"写文件"，都要显式指定编码与换行。**

1.8 ★ 端点数怎么读才不会自己吓自己（9474 vs 9476）

`report_timing_summary` 里**两个层级的总端点数是不同的**，别混用：

出处	`sys_clk` 端点	说明
Intra Clock Table（按时钟域）	**9474**	`sys_clk` 域内的端点，**这才是签核用的数字**
Design Timing Summary（全设计汇总）	**9476**	= 9474 + `sys_clk→adc_clk` 1 + `sys_clk→adc_clk2` 1

即全设计总表把**跨时钟域**的 2 个端点也算进来了。我一度把 9476 当成"设计变了"， 逐行读 `git diff` 后确认 `rtl/enc_top.v` 本轮改动是**纯注释**、位流大小与压缩位数 逐字节相同（741,836 B / saved 11,603,072 bits）⇒ 设计未变，只是取数取错了表。 **引用端点总数时一律注明"Intra Clock Table / 全设计汇总"。** （★ 2026-10-10 补输出侧约束后的最新一轮完全复现了这条规律：Intra Clock Table 的 `sys_clk` = **9485**， 全设计汇总 = **9487** = 9485 + `adc_clk` 1 + `adc_clk2` 1。§2.2 表中引用的 9487 是**全设计汇总。**）

1.9 ★★ 真 RTL 缺陷（本轮查出并已修）： `calib_ctrl` 的标定窗口 off-by-one

这是本轮唯一的 RTL 功能缺陷。 位置： `rtl/calib_ctrl.v` 的 `S_ACCUM` 状态。

原实现：

```
if (in_valid && (nsamp < MAX_SAMP)) begin
    S1 <= S1 + s_ext; ... nsamp <= nsamp + 18'd1; n_after_t = nsamp + 18'd1;
end
if (cycle_pulse) begin
    wcnt <= wcnt + 16'd1;
    if ((wcnt + 16'd1) >= win_n) finish_t = 1'b1;   // ← 这里少算一个周期
end
```

两个后果（都已在 `sim/tb_calib_min.v` 上实测复现）：

1. `win_cycles=N` 实际只累加 **N-1** 个完整机械周期。 实测 `win_cycles=4`、每周期 8 样本 ⇒ `nsamp=24`， **不是 32。**
2. **`win_cycles=1` 时窗口长度退化为 0**：实测 `nsamp=0 n_win=0 we_cnt=0 busy_cyc=3`， 11 字节写回**一个都没发生**，全部标定参数保持原值。 随后 `O_s = S1/0` 走"分母为 0 ⇒ 商饱和到 `48'hFFFFFFFFFFFF`"分支， **不挂死、不报错、静默失效** —— 这是最危险的一类缺陷。

修法（`rtl/calib_ctrl.v`，改成 if / else-if / else 链，每个 `wcnt` 只在一个分支赋值）：

```
if (cycle_pulse && ((wcnt + 16'd1) > win_n)) begin
    finish_t = 1'b1;                // 第 win_n+1 个周期边界，关门
    wcnt <= wcnt + 16'd1;
end else if (in_valid && (nsamp < MAX_SAMP)) begin
    S1 <= S1 + s_ext; ... nsamp <= nsamp + 18'd1; n_after_t = nsamp + 18'd1;
    if (cycle_pulse) wcnt <= wcnt + 16'd1;
end else if (cycle_pulse) begin
    wcnt <= wcnt + 16'd1;
end
```

语义变化：判据 `>=` → `>`，即"等第 `win_n+1` 个边界脉冲关门"，窗口严格覆盖 `win_cycles` 个完整机械周期。修后实测 `win_cycles=4`、P=8 ⇒ `nsamp=32`、`pulse_cnt=5`。 `rtl/enc_top.v` 用的是 `.win_cycles(16'd64)`（原语义 = 63 个周期，现语义 = 64 个周期）， 窗口实测为 `N*P-1` 个样本（32 个边界脉冲里第 1 个在窗口打开那一拍已被消耗）， 对 64 周期 × 数百样本的真实系统只差 1 个样本，**可忽略。**

判据留档（下次改这块务必先看）：理想正交输入下 `sin_dp` 必须接近 0； **若它顶到限幅 16384，一定是激励相位错，不是 RTL 错**（我在这上面白跑了一轮： "32 点正弦表"被写成周期 8 的波形， `c = table[(k+8)%32]` 于是 s 与 c 只差 45°）。

1.10 ★★ 第二个真 RTL 缺陷（本轮查出并已修）： `uart_scope` 组帧 FSM 连发两个 `tx_start`，整条遥测帧错位

这是比 §1.9 更严重的一个 —— §1.9 影响的是标定，这一条影响的是**唯一的上板观测通道。**

缺陷（`rtl/uart_scope.v` 的组帧 FSM，基线提交 `9e9aebc` 里就是这个样子的）：

```
// 原实现：只看 !tx_busy 就发下一个字节
else if (!tx_busy) begin
    tx_data_r <= frame_byte(sel);
    tx_start  <= 1'b1;          // ← tx_start 是**寄存输出**
    ...
end
```

机理：`uart_tx` 的 `busy` 在 stop 位开始的**同一拍**才拉低，而 `tx_start` 是寄存输出 （本拍赋值、下一拍才为高）。FSM 在 `busy` 刚拉低那一拍置 `tx_start`， **下一拍 `busy` 仍为 0** （`uart_tx` 要到再下一拍才把 `busy` 置起来），于是 `else if (!tx_busy)` 分支**再次命中**， `tx_start` 连高两拍：

- 第 1 个 `tx_start` 被 `uart_tx` 接受，发出当前字节；
- 第 2 个 `tx_start` 被**忽略**（此时 `uart_tx` 已不空闲），但 `tx_data_r` 已经被 **换成下一个字节**了。

⇒ **每个字节都丢一个**，线上帧变成"隔一个取一个"： `AA 12 0F BE ...`， 字段全部错位、 `XOR` 必然不符，**上位机永远解析不出合法帧。**

修法：加一个 `snd_go` 标志，强制"发一拍、歇一拍"：

```
reg snd_go;                     // 发过 tx_start 后的强制间隔拍
...
if (!sending) snd_go <= 1'b0;
if (!sending) begin
    if (latch) begin sending <= 1'b1; snd_go <= 1'b1; sel <= 8'd0; end
end
else if (snd_go) begin          // 启动/上一个字节后的强制间隔拍
    snd_go <= 1'b0;
end
else if (!tx_busy) begin
    tx_data_r <= frame_byte(sel);
    tx_start  <= 1'b1;
    snd_go    <= 1'b1;
    ...
end
```

★★ 决定性 A/B 实证（同一台 `sim/tb_uart_scope.v`，只换 `rtl/uart_scope.v` 一个文件， `xvlog --sv` + `xelab -debug typical` + `xsim -runall`，BAUD_DIV=4 / SAMPLE_DIV=100）：

	基线 `9e9aebc` （有缺陷）	**当前（已修）**
C1 帧头 `0xAA 0x55`	**FAIL**（`byte0/byte1` 不符）	PASS
C2–C10 各字段	**FAIL**，读到的序列是 `0f f1 de be 01 03 5a aa 12…`（正是"隔一个取一个"）	PASS
完整帧	错位	`aa 55 | 1234 0f0e | f1f2dead | beef0102 | 03 | 04090167` ✅

⇒ **不是我的测试台过严，是基线 RTL 真的在错帧。** 判据、位周期、波特率三项 （`sim/tb_uart_baud.v`）在两种版本下都一致通过，说明缺陷**只在组帧 FSM 的握手时序**上。

与位流的对应关系（上板必读）： - `9e9aebc` 是仓库基线，而 `rtl/uart_scope.v` 在 `9e9aebc` 之后**从未被改过** （`git log --oneline -- rtl/uart_scope.v` 只有基线和当前工作树） ⇒ **所有 10-09 生成的位流（含 741,836 B 生产版与 1,473,128 B ILA 版）都带着这个缺陷。** 拿旧位流上板，UART 遥测帧会整体错位，

`tools/uart_scope.py` 会一直报校验失败。 - **§2.1 那版新位流（10-10 02:32:17，726,924 B）是用修好的 `rtl/uart_scope.v` 生成的**， 这是第一个能正常遥测的位流。 - **★ 当前上板依据是 §2 顶部记录的 2026-10-10 16:05:29 生产版（`enc_top.bit` 804,780 B）与 16:12:01 ILA 版（`enc_top_ila.bit` 1,445,812 B）**：它们带上了 §1.9/§1.10/§1.15/§1.17/§1.18 的修正、§2.2 新增的 14 条输出侧约束、§1.20 探针位序修复、§1.21 OTR 软清位与 §1.22 UART→SPI 桥。 上板请用这两版；§2.1（02:32，726,924 B）、§2.2、§2.3（08:40，962,600 B）与 12:08 的 1,219,152 B 都保留为历史基线。 - 本轮回归与实现都在这两处 RTL 修正**之后**跑的，数字见 §1 / §2.1 / §2.2 / §2.3。

1.11 ★★ 我自己写的测试台在"空转"（两处，都已修）—— 比 RTL 缺陷更值得警惕

这两处都是 `tools/scan_dangling.py` 与本轮新台自己抓出来的，不是猜的。

(a) `sim/tb_uart_scope.v` 的线路噪声注入根本没接到接收器上。 原文 `wire rx_line = inj_mode ? inj_bit : sc_tx;` —— 这根线**被计算但从未被读**； 而接收任务（`rx_byte_sc` / `rx_frame_locked` / `rx_frame_sync` / `rx_frame_until_hdr`） 全部直接采样 `sc_tx`。于是 **[E] 段"注入 5 个垃圾字节制造线路噪声，再验证接收器能重新同步"的注入完全无效**，E0–E10 的断言实际测的是 DUT 的干净输出 ⇒ **"噪声后重新同步"这个结论原本是假的。** 修法：DUT 输出改名 `utx`，新增 `wire sc_tx = inj_mode ? inj_bit : utx;`， 接收任务一行不改就自动改走注入线路。修后实测注入 5 个各不相同、且非 `0xAA/0x55` 的 字节（`0x24 0x81 0x09 0x63 0x0d`），仍 **PASS**（当时计 93 checks；§1.15 加 C19 后现为 **94**）。 ★ 抓出它的正是 `tools/scan_dangling.py` 的 A2"其死信号"条目（`X rx_line`）—— **扫描器本来是用来查 RTL 的，这次却抓出了测试台在空转。**

(b) 同文件的 `rand_byte_safe` 让"随机噪声"退化成全 0。 原文先 `b = 8'h00;` 再 `while (((b === 8'hAA) || (b === 8'h55)) && ...)` —— **0 既不等于 `0xAA` 也不等于 `0x55`，循环一次都不进**，5 个"随机"噪声字节**全是 `0x00`** （日志实证：`injected noise byte 0x00` ×5）。全 0 噪声等于连续 break， 重同步测试被显著弱化。修法：把 `0x00` 一并列入排除集， 且判据放在**生成之后。**修后日志出现 5 个互不相同的字节。

教训（写给下一次写测试台的人）： - **"测试通过"不等于"测试跑了"。**新写的测试台必须能证明激励真的抵达被测逻辑， 否则它只是把 DUT 的正常输出又断言了一遍。 `tools/scan_dangling.py` 值得每次都跑。 - 任何"随机/压力"激励都要**把实际产生的激励打印出来看一遍**，不要只看 PASS。 - **A/B 实证比"我读了一遍代码觉得对"强得多**：本轮两个 RTL 缺陷都是靠"把基线版本 换回去、看测试台是否真的报错"来定性的（`git show <基线>:rtl/xxx.v` 换入 + 换出）。

1.12 ★★ 上板结论：旧位流都不对应现在的 RTL，必须用 §2.1 的新位流

`rtl/` 在本轮有**两处功能改动**： 1. `calib_ctrl.v` 的标定窗口 off-by-one（§1.9） 2. `uart_scope.v` 的组帧 FSM 连发 `tx_start` （§1.10）

而 `_synth_work/enc_top.bit` （741,836 B，10-09 23:17:45）与 `enc_top_ila.bit` （1,473,128 B，10-09 23:31:27）**都早于这两处改动** ⇒ **两个旧位流都带着上述两个缺陷，拿它们上板等于跑一个已知会错帧、标定会静默失效的设计。** 本轮已用修正后的 RTL 重跑生产版实现（新产物见 §2.1）。 ★ 这条是"确保上板测试不出问题"的直接前提，**不要跳过。**

1.13 ★ 上板前必须知道的四条实测事实（都不是缺陷，但会让人误判）

1. **上电默认 `uwven=0`，UVW 三相恒 0； `abzen=1`，ABZ 有输出。** `rtl/reg_file.v` 复位值 `r_output <= 8'h40` （bit6=abzen）， `rtl/reg_file.v:488-489` `cfg_abzen = r_output[6]` / `cfg_uwven = r_output[7]`， 而 `rtl/uvw_gen.v:597-599` 在 `uwven=0` 时把三相强置 0。 ⇒ **上电只看得到 ABZ，U/V/W 全是低电平，直到主机用 SPI 写 `0x0109 = 0xC0`。** 这不是坏了。 `docs/06_board_test_guide.md:275/284/495` 已如实写明。 ★ 注意 `reg_file` 复位时 `r_zero1 <= 8'h04` （`zwidth=4`），**不能是 0** —— 0 会让 `rtl/abz_gen.v` 的 `z_pulse_n` 恒 0，Z 与 UVW 一起被钉死。
2. **UART 遥测链路占用率 90.7%**：一帧 `17*(10*25+2) + 10*25 = 4534 clk = 90 µs`， 而 `latch` 节拍 `SAMPLE_DIV=5000 clk = 100 µs` ⇒ **余量只有 9.3%**， 且 `rtl/enc_top.v:741-754` 是 `.en(1'b1)` （遥测常开）。 功能正确（帧周期由 latch 决定，不会丢帧），但**没有任何富余**： 以后想把 `SAMPLE_DIV` 调小（提高帧率）或把 BAUD_DIV 调大（降波特率） 之前，必须先按这个算式复核。**这是"功能正常但裕量为零"的地方，不是缺陷。**
3. **`BAUD_DIV=25` 的波特率是精确的 2.0000 Mbaud。** 由 `sim/tb_uart_baud.v` 实测： 逐字节线上时长 `= 10*BAUD_DIV = 250 clk`，位边界偏移 `25 / 50`， 相邻 `tx_start` 间隔 `= 10*BAUD_DIV + 2 = 252 clk`； 对 `BAUD_DIV = 4/7/10/25` 四点扫描全部吻合 ⇒ **那个"+2"是与 `BAUD_DIV` 无关的固定 FSM 换字节开销**， 可以放心外推到真机。 ★ 踩过的坑：**绝不能用"相邻 `sc_tx` 跳变时间差"量位周期** —— 连续相同的位会把 跳变合并（`0xAA` 的起始位 0 与 bit0 0 连成一段 8 clk 低电平，看起来像"第一个位 区间两倍宽"）。必须用"起始位下降沿 → `busy` 下降沿 `== 10*BAUD_DIV`" 或逐拍状态 dump 来判定。
4. **`constr/enc_top.xdc` 里有一段自相矛盾的注释，别照着调参。** 第 134–136 行推导"`ad_clk` 上升沿在 `clk50` 上升沿后 10 ns"， 而第 162–165 行却宣称两者"在引脚上是**同一个物理时刻**"。 本轮用真原语对照实验已证实： **`ad_clk` 确实比 `clk50` 晚 10 ns（180° 反相）**， 即第 162–165 行的说法是错的。约束值本身偏保守（无害），但这句注释会误导后续调参。 ★ 另：**ABZ/UVW 六个输出（`a_out` / `b_out` / `z_out` / `u_out` / `v_out` / `w_out`， 走 J5 排线出去）此前没有任何 `set_output_delay`、没有驱动强度、没有端接/串阻约束** —— 50 MHz 边沿 + 长排线 + 无串阻是**真实的信号完整性风险。 本轮已补上输出侧约束与驱动强度（见 §2.2 第 10 节 + 三次迭代记录）； 但"端接/串阻"是硬件侧的事，仍未做**：上板若看到 ABZ 计数偶发多计/漏计， 第一个要怀疑这里。

1.14 ★ ODDR 仿真桩与真原语的差异（保真度记录，已量化）

`sim/xilinx_prims_stub.v` 的 ODDR 桩是 `always @(posedge C) q_r<=D1; always @(negedge C) q_r<=D2;`， 而真实原语 `D:\Vivado\2026.1\Vivado\data\verilog\src\unisims\ODDR.v` 在 `SAME_EDGE` 模式下负沿输出的是 `qd2_posedge_int` （**posedge 时采到的 D2 旧值**）。 对照实验（改名区分后同台例化， `xelab` 必须带 `glbl`）：

```
INITIAL (t=11000): q_real=0 q_stub=0
negedge t=41000:   q_real=0 q_stub=1      ← 分叉
negedge t=61000:   q_real=0 q_stub=1
negedge t=101000:  q_real=0 q_stub=1
negedge t=121000:  q_real=1 q_stub=1      ← 收敛
MISMATCH_SAMPLES=4
```

判定：因 `D1=0` / `D2=1` 是常量，**4 拍后两者完全同相位**， 稳态频率/相位/占空比一致 ⇒ 桩对功能验证有效， `ad_clk` 的 180° 反相信义成立。 差异仅在"上电后前 3 个负沿"，而这段时间 `rst_n` 仍为 0（`POR_BITS=20` ≈ 21 ms）， **无功能影响。**如实记录，不算缺陷。 ★ 教训：两次 `xvlog` 编同名 module 时**后编的覆盖先编的**， `xelab` 全程只用桩 —— 我第一次实验因此误判为"桩与真原语一致"。**必须改名区分。**

教训（写给下一次改这个状态机的人）

- 迟滞判据的**基线必须与提交寄存器同拍。**任何"寄存器存基线 + 组合比较"的写法 都会引入"提交后旧基线仍满足"的连跳，除非比较用的就是即将锁存的值。
- 冷启动基准必须**等于稳态语义下的那个值**，不能取"角度所在段的两条边界"。 前十一版反复失败的根源都在这两条上，而不是在"判据公式"上。
- `BND[6] = 65536` 是 17 位；进入 16 位表达式前必须显式处理环绕，否则它退化成 0。

1.15 ★★ 第三个"真缺陷"，而且它落在唯一的上板观测通道上：UART 帧校验和约定不一致（已定案并加守卫）

现象（我在 2026-10-10 复查主机端工具时发现）： `rtl/uart_scope.v:189-196` 组帧时把 **两个帧头常量也异或进校验和**：

```
l_chk <= 8'hAA ^ 8'h55 ^ angle[15:8] ^ ... ^ stat;      // = payload XOR ^ 0xFF
```

而主机端 `tools/uart_scope.py` 只异或 payload(off 2..16)： `xor_checksum(payload)`， `build_frame`(:78) 与 `parse_stream`(:100) 都是这个约定。⇒ 对同一帧，两边 **恒差 0xFF。**

为什么这是致命的： `parse_stream` 校验不过就 `crc_err += 1` 并继续找帧头 ⇒ 真机接上后 **每一帧都被丢弃，`frames` 恒为 0**，长程"上位机看角度/Lissajous/幅值"的整条观测链等于废掉。 而 `python tools/uart_scope.py --selftest` **照旧 PASS** —— 它自己组帧自己解析，两边用的是 同一个（错的）约定，自检根本发现不了。这正是 §1.11 那类"看着像在验证、其实在空转"的翻版。

三方定位结果（谁是对的）： - RTL（ `rtl/uart_scope.v:189-196` ）与 RTL 测试台（ `sim/tb_uart_scope.v` 的 C17/C18， 从 DUT 输出线 `utx` 上逐位采样解码）**两边一致**：实拍帧 18 字节为 `aa 55 12 34 0f 0e f1 f2 de ad be ef 01 02 03 04 5a a7`， payload XOR = `0x58`、含帧头 = `0xA7`（对应激励 `t_angle=0x1234 t_s=0x0F0E t_c=0xF1F2 t_amp=0xDEADBEEF t_pos=0x01020304 t_stat=0x5A` ）。 - **错的是主机端**（以及照抄主机端说法的 `docs/06:299` 那行）。Python 复算：工具会算出 `0x0b`， RTL 公式给 `0xf4`（另一组测试向量），差值恒为 `0xFF`，因为 `0xAA^0x55 = 0xFF`。

定案（2026-10-10）：保留 RTL 约定，改主机端与文档。 理由（不是"谁方便改谁"，是三条硬理由）： 1. 现行上板依据的位流 `_synth_work/enc_top.bit`（当时是 10-10 02:32 那一版，WNS **+0.686 ns**； 此后已被 §2.2 的 07:31:54 版取代）已按 RTL 约定 实现并验证；为一条**常量偏移**去改 RTL 会作废整个时序收敛结果，并需重跑综合/实现/全回归。 2. 含帧头常量在检错能力上**不多不少**（只是把校验值整体异或 0xFF），没有任何收益可换。 3. 主机端消费者只有 `tools/uart_scope.py` 一个文件，改动面小且可被实拍帧守卫。

已做的修改： - `sim/tb_uart_scope.v`： 新增 **C19** 断言 `fr[17] !== (payload-only XOR)`，把"含帧头常量" 钉死（谁改回纯 payload XOR，这一条立刻 FAIL）； check 数 93 → **94。** - `tools/uart_scope.py`： 新增 `frame_checksum(payload) = xor_checksum(payload) ^ HEAD1 ^ HEAD2`， `build_frame` / `parse_stream` 全部改用它；文件头文档补上约定与来龙去脉。 - `docs/06_board_test_guide.md:299` 与另一处帧表（原写"对偏移 2..16 的逐字节 XOR"） 改为" `0xAA ^ 0x55 ^` （偏移 2..16 逐字节 XOR）—— **含帧头常量**"，并加一段定案说明。 - **新增守卫（这才是防复发的关键）**： `sim/fixtures/uart_frame_rtl.hex`（**RTL 实拍帧**， 由测试台新增的机器可读行 `RTL_FRAME_HEX` 打印后落盘，含来源/激励/校验值的注释） + `tools/check_uart_frame_fixture.py`（16 checks）： 用 `tools/uart_scope.py` 的 `parse_stream` 解析这一实拍帧，要求恰好 1 帧 / 0 校验错 / 字段值全部等于激励； 再断言 `build_frame` 逐字节复现实拍帧；最后做**反事实检查** —— 把末字节换回"纯 payload XOR" 后解析必须变成 0 帧 + `crc_err>0`，证明旧约定在真机上确实是坏的。

顺带修掉的两个真问题（都在测试台里，属"我自己的错"）： - `sim/tb_uart_scope.v` 原来打印整帧的那条 `$display` **只有 16 个 `%02h` 却有 18 个实参**， `fr[16]`(stat) 与 `fr[17]`(校验和) **从来没被打印过。**旧文档里那句 `frame = aa 55 | 1234 0f0e | f1f2dead | beef0102 | 03 | 04090167` 因此**不能用来核对校验和** （它连 stat/checksum 都没打印，字段分组也对不上）。已补足 18 个格式符，并另加机器可读的 `RTL_FRAME_HEX ...` 行供 fixture 生成。**教训：格式符数量与实参数量不一致时 xsim 不报错、 也不提示，只是默默少打印 —— 这类"打印不全"会让后续所有核对都建立在残缺数据上。** - **更正一个我自己写错的"结论"**： `sim/tb_uart_scope.v` / `sim/tb_xor_probe3.v` 里曾写 "xsim 的 XOR 有求值陷阱，(a)→0x55、(c) 写法不可靠"。这是**错的**： `tb_xor_probe3.v` 自己 通过的断言已经否证它 —— T11a `(a)^0x55 == (b)`（差异**恰好**就是 (a) 少异或的那个帧头常量）、 T11c `(c) == (b)`。当时被当成"正确值 `0xFD` "的那个数，来自一段**截断的累加**：完整 payload XOR = `0x58`，只累加到 `fr[15]` 漏掉 `fr[16]=0x5A` 时 `0x58^0x5A = 0x02`， 再 `^0xAA^0x55` 得 `0xFD`； 而那次 (c) 实测得到的 `0xA7` 恰恰是正确值。 ⇒ **不是仿真器 bug，是测试台自己的累加范围写错了**（还导致我在文档里留下错误结论）。 已改注释，并在 `tb_xor_probe3.v` 新增 **T12a/T12b/T12c** 把这三个数字固化成判据。

★ **要如实记住的教训**：这一轮"补测试台"的最大产出不是发现了 RTL 缺陷，而是**发现我自己的 验证代码有两处在骗我**（§1.11 的注入没接上、§1.15 的打印少两个字节 + 一个错误结论）。 **测试台本身必须先被怀疑**，否则"全绿"只是把错误更快地固化下来。

1.16 ★★ 第四个真缺陷：粗调移位判据在两处说反了（P1，已定案取 ≤1.999）

现象（同一个算式的两套判据，互相排斥）： - RTL 用 **`tg / 2^sh <= 1.999`**： `rtl/calib_ctrl.v:117-119` `TG_SH0 = 32'd65503`（= `round(1.999*32768)`）、 `TG_SH1 = 131006`、 `TG_SH2 = 262012`， `:573-581` 用 `tg_s <= TG_SHn` 选 `res_shift_s ∈ 0..3`； `rtl/sc_correct.v:68` 注释同口径。 - 模型/文档要求 **`<= 1.0`**： `model/golden_model.py:_coarse_shift`、 `docs/01_technical_solution.md` 第 156–167 行整节"粗调移位要选到细调增益 ≤ 1.0"，论证是 "桶宽 = `2^(15-sh)/G = 1/总增益`，只有桶宽 ≥ 1 才能逐点可逆；1.2 倍时桶宽 0.833、 约 16% 整数目标取不到 ⇒ ~1 LSB 系统性误差"。 - ⇒ 同一份设计里 RTL 与模型**行为不一致**，而模型正是文档里所有精度数字的来源。

决定性实验（新增 `tools/calib_shift_judge.py`，报告 `model/calib_shift_judge_report.txt`）： 四条判据（ `rule_rtl` =1.999、 `rule_one` =1.0 按正确 Q15 单位、 `rule_one_float`、 `rule_study_buggy`） × 8 个工况（A/B/C/D/E/G + 新加 H/I 弱信号），乘加路径与 `rtl/sc_correct.v:9-10` 逐位一致： - **[A] 精度**： `rule_rtl` 与正确单位下的 `<=1.0` 判据在 **8 个工况上逐项完全相同** （e_max/e_rms/max(LSB)/径向纹波），**互有优劣的工况数 = 0 / 0 / 8 全同。** - **[B] 恒等性**：令有效乘数 `m = (g/2^15)·2^sh`。g 为偶数时 `(g, sh)` 与 `(g/2, sh+1)` 是**同一个有理数的 floor**，逐点恒等 —— 实测 `m = 0.8/1.2/1.25/1.999/3.5` 在 `x = -32768..32767` 全整数域上与 `sh=0` 的输出 **diff 全 0。** - **[C] 桶宽与可达性**：桶宽 ≥ 1 ⇔ `m <= 1` ⇔ **多对一、不可逆**； `m=1.2` 时 16.67% 的整数目标取不到（这正是旧文档说的 "~16%"）。但**校正后的矢量在本设计里是 直接送 CORDIC 使用、从不做反演**，目标不可达不产生误差：取整偏差 RMS ≈ 0.47–0.58 LSB 与 m 无关，m 为整数时甚至为 0。 - **[D] 结论由数据生成**： `<=1.0` 那条"逐点可逆"的论证**方向是反的**（可逆 = 多对一）。 两条判据的真实差别只有**细调增益量化步长**： `2^sh/32768`。 `m <= 2` 时 RTL 判据取 `sh=0` ⇒ gain 与 tg 逐位相同、**无截断**； `<=1.0` 在同区间多移 1 位 ⇒ 通道间比例误差最大 `2/32768`。 另有量级差别： `<=1.999` 时总增益上限 `1.999×8 ≈ 16×`（+24 dB），改成 `<=1.0` 掉到 `8×`（+18 dB）。

顺带查出两个真 bug —— 都是"量纲"错，让对照臂名不符实： - `tools/calib_shift_study.py:37-45` 的 `apply_rule_1p0(tg_s, tg_c)` 收到的 tg 是 **Q15 整数**（如 39043 表示 1.19 倍），却与 `1.0` 比较 ⇒ 条件永远为假、**恒返回 `sh=3`**； 它的 `:131-138` [结论] 还是**硬编码文本**，与它自己表里的数据相反。 ⇒ 该脚本连同它的报告 `model/calib_shift_study_report.txt` **已 `git rm` 删除**（由 judge 取代）。 - `model/calib_ref.py:113-118` 的 `coarse_shift_exact(tg)` 是**同一个 bug**（ `:170-171` 调用时 tg 是 Q15 ≈ 39046），所以它以前输出的"黄金 `sh=3/3` "是假的。 已改名 `coarse_shift_one(tg)`、判据改为 `(tg >> sh) <= 32768`，打印块里误导性的 "桶宽<1!" 标记换成量化步长 `2^sh/32768`。 ★ 教训：**跨语言/跨文件比较前先对齐量纲**；"结论写在代码里"比"结论由数据生成"危险得多。

定案：保留 RTL 的 `<=1.999`（RTL 逻辑一行不改，只改注释/模型/文档），理由： ① 两条判据精度实测全同（[A]），改 RTL 只会作废已验证的时序签核与位流，收益为零； ② `<=1.999` 在 `m<=2` 区间 `sh=0` 无截断，反而更细（[D]）； ③ 总增益上限 16× 是"弱信号/大间隙"卖点。 同步改动： `model/golden_model.py:189-227`（判据改 Q15 阈值 + docstring 重写）、 `rtl/sc_correct.v:80-105`（注释块重写，实测表数字更正）、 `sim/tb_sc_correct.v:24-33/:466-472`（注释与 `$display` 重写）、 `docs/01_technical_solution.md:156-169`（整节重写为"取 ≤1.999"）。

数字影响（必读）： 判据统一后 `golden_model.py` 的**工况 E 校正后 max 由 0.0220° 改善到 0.0165°（3 LSB），改善倍数 954× → 1273×**； 可标定工况里最差变成 **D 的 0.0220°（4 LSB，227×）**； F/G 不变（0.3571°/0.3131°，59×/60×）； 全部"保守口径"不变（校正后全工况最差 0.3571° = 65.0 LSB、 P=64 机械角

误差 20.1 arcsec、机械分辨率 0.309 arcsec）。 ⇒ 已把 `model/golden_model.py:405-416` 的**结论行从写死数字改成现算**（不再有"结论与表格矛盾"）， 并同步 **18 处**文档/注释数字（README 3、docs/00 1、docs/01 4、docs/02 7、docs/03 2、docs/05 1、rtl 2）。

复跑证据（本轮实跑）： `python tools/calib_shift_judge.py` → exit 0（报告 8,926 B）； `python model/golden_model.py` → 自证打印 `20.9839 deg -> 0.0165 deg，改善 1273 倍` 与 `全工况最差残余 = 0.3571 deg（F …），按最差残余计的改善 = 59 倍`； `git diff -U0 -- rtl/` 过滤后**没有任何非注释增删行** ⇒ RTL 行为未变； 回归 **18/18 PASS**（07:10:52）、 `lint_all.ps1 -Clean` **ERROR 0 / WARNING 0**、 `to_lf.py --check` **exit 0**。

> ⚠️ 遗留的判据效应： `rtl/*.v` 的 mtime 现在**晚于** §2.1 的位流（10-10 02:32），虽然改动 全是注释、逻辑未变，但按 §1.12 的严格判据（"位流必须晚于 RTL"）它已不再成立。 这一点会在 §3 的 P0-2 / P0-4 两轮重跑里一并消除（重跑后位流晚于所有 `rtl/*.v`）。

1.17 ★★ 第五个真缺陷：标定窗口永远关不上（P0-3 做集成测试时查出并已修）

为什么此前没发现： `sim/tb_calib_min.v`（54 checks）单独测 `calib_ctrl`，它**自己造** `cycle_pulse`； `sim/tb_enc_top.v`（原 39 checks）**从未触发过标定**（`key_n` 全程 1、 SPI 写列表不含 `0x0502`）。 ⇒ 顶层的 `cycle_pulse` 接法从来没有被任何测试台走过。 这就是"模块级全绿 ≠ 集成正确"的教科书例子。

现象（首轮 TEST F 实测，日志存档 `_sim_work/tb_enc_top_PRE_FIX_2026-10-10.log`）：

量	修复前	修复后
`nsamp`	**65536（=MAX_SAMP，被上限截断）**	**17192（64 个输入周期）**
`wcnt`（窗口计到的输入周期数）	**0**	**65**
`truncated`	**1**	**0**
标定耗时	64942 拍	16599 拍

根因： `rtl/enc_top.v` 把 `calib_ctrl` 的 `.cycle_pulse()` 接到了 `u_pole` 的 `wrap_pos | wrap_neg`，而 `calib_ctrl` 在标定期间会 **freeze** `loop_filter`（`enc_top.v:395 .freeze (calib_busy)`，为的是让角度静止、样本可比） ⇒ `pos_out` 恒定、角度永不跨周期边界 ⇒ **`wcnt` 恒为 0**， `win_cycles=64` 这条"按输入周期关门"的路径**永远走不到**， 窗口只能靠 `MAX_SAMP=65536` 上限截断。 生产配置下这意味着：标定窗口是 **65536 个样本**而不是设计的 **64 个输入周期**， 样本数与机械转速挂钩、 `truncated` 恒为 1，标定精度与耗时的设计口径全部不成立。

修法（`rtl/enc_top.v`，只加计数器、不动 `calib_ctrl`）：用**未被冻结**的 CORDIC 原始角度 `angle_raw`（`enc_top.v:353`，伴随有效 `cordic_valid`）做整圈折返计数 —— `angle_raw` 的 MSB 每翻转一次是半圈，两次即一个完整输入周期，正反转都成立：

```
reg [AW-1:0] calib_ang_d;
reg          calib_cyc_tgl, calib_cyc_pulse_r;
always @(posedge clk_i) begin
    calib_cyc_pulse_r <= 1'b0;
    if (cordic_valid && (angle_raw[AW-1] != calib_ang_d[AW-1])) begin
        calib_ang_d   <= angle_raw;
        calib_cyc_tgl <= ~calib_cyc_tgl;
        if (calib_cyc_tgl) calib_cyc_pulse_r <= 1'b1;  // 半圈 x2 = 一个周期
    end
end
wire calib_cycle_pulse = calib_cyc_pulse_r;   // 取代原来的 (wrap_pos | wrap_neg)
```

`MAX_SAMP` 保留为"输入不转（无激励/堵转）时"的保护上限，这两条路径从此**互不掩盖**。 同时更正了 `enc_top.v` 里一处过时注释（旧文声称"寄存器 read-back 仍显示 SPI 写入值"， 实测读回就是生效值，见下）。

顺带修掉两个测试台自身的缺陷（如实记录，都是我写的）： 1. `task chk` 的 id 声明为 `input [63:0]` ⇒ 只能装 8 个字符， `%0s` 打印的是同名的 **最后 8 个字符**（`F4b-calib-not-truncated` → `runcated`、 `D13-uvw-only-legal-codes` → `gal-codes`），日志里的失败项几乎无法辨认。 已扩到 `input [255:0]`（32 字符）。 2. 按键触发测试原来**只按 4 拍**，而 `key_n` 后面有 16 bit 去抖计数器 （`enc_top.v:175-190`，要稳定 `0xFFFF=65535` 拍 ≈1.31 ms 才认账） ⇒ 按键被去抖滤掉、第二次标定根本没发生。已改为"按住并轮询 busy 上升沿"。

修复后实测（`tools\run_sim.ps1 -Only tb_enc_top` → PASS，63 checks，133 s）：

```
F1: busy 在 0 拍后拉起（calib state=1）
F2: 0x0502 读回=0x0002 (busy=1 done=0 trunc=0)
F4: busy 落下（标定结束）：又用了 16599 拍：done=1 nsamp=17192 wcnt=65 truncated=0
F6: OFS_S=15 OFS_C=931 GAIN_S=30325 GAIN_C=29401 PHASE_K=15 SHIFT=0x0000
F10: 按键（去抖 65535 拍）在第 65540 拍触发第二次标定：17149 拍完成：
     OFS_S=-31(第一次 15) GAIN_S=32241(第一次 30325) PHASE_K=-54(第一次 15)
```

TEST F 覆盖的集成路径（这是 P0-3 的目标，逐条有断言）： SPI 写 `0x0502=0x0001` 触发 → `busy` 读回（F1/F2）→ 标定期间 `dut.angle` **必须不动**（F3，验证 `.freeze`）→ `done/truncated/nsamp`（F4/F5）→ 6 组结果与 `dut.u_calib.res_*` 逐项比对（F6，证明 `calib_we` 写回 `reg_file`）→ `dut.use_*` 等于读回值（F7，证明 `sc_correct` **取用的就是同一条路径**）→ 增益/偏置区间（F8）→ 写 `0x0502[2]` 清 `done`（F9）→ 按键路径触发第二次标定（F10）。 （`spi_rd16` 用到 TW28 的**重叠式响应**：帧 N 组包、帧 N+1 移出，见 `rtl/spi_slave.v:137-142/:154-158/:224/:242`。）

1.18 ✅ 同轮查出的第六个真问题：一键标定不可重复（2026-10-10 已修，出处是两个来源）

原始证据（修前，F10 那一行）：第二次标定（按键触发）得到 `GAIN_S 30325 → 32241`（朝单位增益 32768 **回归**）、 `PHASE_K 15 → -54`、 `OFS_S 15 → -31`。

根因一（累加的是校正后的信号）： `calib_ctrl` 累加的是 `sc_correct` 的**输出** （旧代码 `enc_top.v`： `.in_valid(cor_valid)`、 `.s_n(s_cor)`、 `.c_n(c_cor)`）， 而 `sc_correct` 的系数在第一次标定后就**已经生效**（F7 已证明写回与生效同一条路径） ⇒ 第二次标定看到的是"已经被校正过"的信号，于是把系数**往回掰**。 `GAIN_S` 从 30325 往 32768 走，正是"第二次校正把第一次的校正撤销"的直接表现。

★ 只修根因一并不够（这是本轮最有价值的一条记录）：把三个端口改成 `.s_n(s_dec) / .c_n(c_dec) / .in_valid(dec_valid)`（累加**校正前**的原始抽取样点，`sc_correct u_correct` 的输入正是 `s_dec/c_dec`）之后重跑， `tb_enc_top` 依然 **FAIL（5 error(s)/64 checks)**： 两次标定之差 `OFS_S=-37 OFS_C=-947 GAIN_S=-484 GAIN_C=432 PHASE_K=-61` （`OFS_C` 变化近千 LSB ⇒ 正交性被破坏，不是小误差）。

根因二（标定窗口不是整数个输入周期）： `calib_ctrl` 的 `S_ACCUM` 从 `start` 那一拍就开始累加，窗口 = "一段起始残缺周期 + win_cycles 个整周期"， 残缺段长度取决于触发时刻落在波形的哪个相位。两次标定的触发相位不同 （第一次经 SPI 写 `0x0502`，第二次经按键 + 65535 拍去抖）， 且 `nsamp` 与周期数对不上整周期（`nsamp=17192`、 `wcnt=65`， 而 LOG2DEC=0 时每周期 256 拍、64 整周期应为 16384） ⇒ 窗口含非整周期成分，正交性被破坏，偏置/相位估计被幅度泄漏污染。

修法（rtl/calib_ctrl.v，两处）： 1. 新增 `armed` / `arm_tmo[25:0]` / `localparam ARM_TMO_MAX = 26'd50_000_000`（1 s @50 MHz）； `S_IDLE` 的 start 分支清零；`S_ACCUM` 里**先对齐到第一个 `cycle_pulse` （输入周期边界）再开始累加**，未对齐期间一个样本都不累加； 输入不转/极慢时靠 `ARM_TMO_MAX` 超时以空窗口收尾（`truncated=1`），不挂死。 2. 关门判据随之回到 `(wcnt + 1) >= win_n` （`armed` 之前第 0 个脉冲不计数、 之后第 `win_n` 个脉冲就是整周期结束）。 ★ 细节踩了两次坑：对齐那一拍的**样本要计入**（否则 `nsamp = win_cycles*P - 1`， 实测 16383），但那一拍**不算已完成的周期**（`wcnt` 自增都加 `armed` 保护， 否则窗口少一个整周期，实测 16128 = 63×256）。两件事同时做对， 窗口才严格是 `win_cycles` 个整周期。

修后实测（`sim/logs/SUMMARY.md` 2026-10-10 08:37:24，`tb_enc_top` 72 checks)： `F4: done=1 nsamp=16384 wcnt=64 truncated=0` （= 64 × 256，整周期不变式成立）； `F6: OFS_S=7 OFS_C=7 GAIN_S=29838 GAIN_C=29838 PHASE_K=0 SHIFT=0x0000`； `F10: 第二次标定 OFS_S=7(第一次 7) GAIN_S=29838(第一次 29838) PHASE_K=0(第一次 0)`； `F11: 两次标定之差 OFS_S=0 OFS_C=0 GAIN_S=0 GAIN_C=0 PHASE_K=0`、 `两次标定窗口 nsamp=16384/16384 wcnt=64/64` ⇒ **逐位一致。** 测试台新增 `F4f` / `F11g` （窗口必须整周期）与 **`F11h` 硬回归锁** （本测试台无噪声、激励严格周期 ⇒ 两次标定六项结果必须逐位相同， 容差判据 F11b–F11f 只是工程口径；真机有 ADC 噪声时以容差为准）。 另：模块级 `sim/tb_calib_min.v` （它对 `cycle_pulse` 的相位假设正是"首个脉冲与 首个累加沿同拍"）**未做任何改动即继续通过**（`nsamp_max=32`、 `pulse_cnt=5`、 `ofs_s=ofs_c=phase_k=0`、 `gain_s=gain_c=42949`） —— 这是新语义正确的有力旁证。

影响：单次上电标定与重复标定现在都收敛到**同一个绝对结果**，与触发时刻、 与标定前的系数都无关（主机若写错系数，重新标定也能收敛回正确值）。

1.19 ★★ ADC 输入建模：注释矛盾已修，诚实约束下第一次出现负余量（2026-10-10 12:08）

背景（真实存在了半个月的两套互斥说法）： `constr/enc_top.xdc` 早期版本里， 第 130 行写着"`ad_clk` 上升沿比 `clk50` 上升沿晚 10 ns"，第 134–136 行也照这个 算了一遍引脚到引脚余量（结论"建立余量 = 3 ns"）；但后面"★★ 关键修正"块却断言 `ad_clk` 与采样它的 `clk50` 上升沿在引脚上是**同一个物理时刻**（ODDR 的 180° 相移恰好补偿了引脚到捕获寄存器的延迟），并据此把输入约束写成 `set_input_delay -clock sys_clk -max 7.000 / -min -1.000`。 真原语对照实验（§1.14）早已证实前者为真、后者为假，但**约束一直没跟着改。**

真实推导（本轮从在盘报告量出来的，写进 `constr/enc_top.xdc:168-193`）： 从 `_synth_work/impl_timing.rpt` 的 `ad_clk` 输出路径读出 `Source Clock Delay = 3.762 ns`、 `Destination Clock Delay = 3.016 ns`， 加上 ODDR 的 180°（半周期 10.000 ns）⇒ **`ad_clk` 引脚上升沿 ≈ 16.78 ns** （相对 `sys_clk` 引脚沿）；再加 AD9238 的 `tPD = 2..6 ns` 与排线 ≈1 ns ⇒ **数据到达 FPGA 引脚 ≈ 19.78 … 23.78 ns**；而捕获寄存器看到的沿 ≈ 20 ns + 时钟网络 （1.43+0.08+互连）≈ 21.6 ns（fast corner）/ ≈24.6 ns（slow corner） ⇒ **最坏情况本来就是负的**，"3 ns 余量"是漏算了两条时钟网络路径的乐观结论。

已落盘的修改： - 注释：原"★★ 关键修正"块保留（`-clock adc_clk` 导致 10 ns requirement、 WNS −4.072 / TNS −80.2 的教训），新增"★★★ 2026-10-10 重写"段逐条列出旧断言的 三处问题（与第 130 行自相矛盾 / 与真原语实验相反 / 引用的"第 86-91 行"是**悬空引用**）。 - 约束（`constr/enc_top.xdc:194-197`）： `set_input_delay -clock sys_clk -max 24.000` 与 `-min 19.500`（数据与 OTR 各两条），即"把整条发射链都算进去"的诚实值。 - 负余量的处理清单（`constr/enc_top.xdc:199-215`）： ① 上板先量 `tPD_meas` （✅ **2026-10-10 已量：≤4 ns，判据通过，见 §1.27**）； ② 功能级修复 = 捕获改 `clk_i` **下降沿**；③ 缩短排线/串 22~33 Ω； ④ 降采样率（**不能**靠改 `create_clock -period`）； ⑤ 最后才 `set_false_path`。

重跑实现的实测结果（2026-10-10 12:08，生产版）： `Design Timing Summary = WNS −3.271 / TNS −84.284 / **26 个 failing endpoints** / 9631`、 `WHS +0.041 / 0 失败`、 `WPWS 9.500 / 0 失败`，结论行 **`Timing constraints are not met.`**（本工程第一次出现）。最差路径 `ad_ch2_d[5] → u_capture/ch2_meta_reg[5]/D`： `Requirement 20.000`、 `Input Delay 24.000`、 `Data Path 0.822`（IBUF 0.479 + 走线 0.344）、 `DCD 1.592` ⇒ required 21.552 vs arrival 24.822。 DRC `Checks found: 43`（0 error / 0 critical）、布线 `8910 fully routed / 0 routing errors`、 LUT 5757 (27.68%) / FF 3528 (8.48%) / DSP 16 / IOB 49 / BUFGCTRL 1、 位流 **1,219,152 B @12:08:30**。产物已另存 `_synth_work/signoff_adc_honest/`； 采纳诚实约束前的最后一版"全约束满足"产物在 `_synth_work/signoff_0840/`（962,600 B @08:40:17）—— **两者 RTL 完全相同，只差约束值。**

加 UART→SPI 桥后的重跑（2026-10-10 16:05:29，生产版当前版）： `Design Timing Summary = WNS −3.352 / TNS −85.116 / **26 个 failing endpoints** / 10582`、 `WHS +0.033 / 0 失败`、 `WPWS 9.500 / 0 失败 / 3863`，结论行仍是 **`Timing constraints are not met.`**（`impl_timing.rpt:144`）。最差路径 `ad_ch2_d[6] → u_capture/ch2_meta_reg[6]/D`（`Requirement 20.000`、 `Input Delay 24.000`、 `Data Path 0.907` = IBUF 0.480 + 走线 0.427、 `Logic Levels 1 (IBUF)`、 `DCD 1.595`） ⇒ **26 条与 12:08 完全同源，全是 ADC 输入端口。** 资源 LUT **6010 (28.89%)** / FF **3857 (9.27%)** / DSP 16 (17.78%) / BRAM 0 / Bonded IOB **50 (20.00%)** / BUFGCTRL 1； DRC `Checks found: 43`（DPIP-1×11 + DPOP-1×16 + DPOP-2×16，**全部 `Related violations: <none>`**，0 error / 0 critical）； 位流 **804,780 B @16:05:29**（压缩 `Bitstream compression saved 11099520 bits` ⇒ 17,537,696 − 11,099,520 = 6,438,176 bit = 804,772 B，实测 804,780 B，同样的 +8 B 头部差）， 头部 `00 09 0f f0 0f f0 0f f0` 与前几版一致。 ★ 与 12:08 版对比：端点数 9631 → **10582**（多出来的就是桥的逻辑）、WNS −3.271 → **−3.352**、 LUT 5757 → 6010、FF 3528 → 3857、IOB 49 → **50**（新增 `uart_rx` = T20）； **位流大小 1,219,152 → 804,780 B 不是功能回归** —— 压缩率随实现变化（同一 RTL 历史实测 726,924 / 962,600 / 804,780 B），**判据只看资源、时序与 DRC。**桥之前的整套产物已备份到 `_synth_work/signoff_pre_uart_bridge/`。

怎么读这个负余量（不要简化成"设计坏了"）： - 26 条**全部**是"ADC 数据相对 `sys_clk` 的建立关系"，由**外部往返延迟** （ODDR 相移 + 排线 + AD9238 的 tPD）决定，**不是内部逻辑不收敛**： 内部寄存器到寄存器最差路径仍在 18–19 ns 量级、余量为正（见 §2.3）。 - `rtl/ad9238_capture.v:64-74` 对每一位都做了两级 `ASYNC_REG` 同步器 （`ch*_meta → ch*_sync`，注释写明第 1 级就是"数据有效窗口中心采样"）⇒ **亚稳态本身是被处理过的**；且 ADC 与捕获同频、相位由外部延迟固定， 稳态下同步器不会随机滑动。 - 真正剩下的物理风险只有两个：① 采样整体晚半拍/一拍（**常数延迟，无害**）； ② 若捕获沿正好落在数据跳变附近，12 位可能被"撕"成两个样本（**有害**，须实测排除）。

下一步（顺序不能颠倒）：按 `docs/07_board_checklist.md` 第 1.8 步上板量 `tPD_meas` —— ≤4 ns ⇒ 可在核对单与本节记录"实测干净"，继续跑通功能链路； ≥5 ns 或 ILA 里 `angle_raw` 出现毛刺 ⇒ 做第 ②条功能级修复： `rtl/ad9238_capture.v` 的三处 `always @(posedge clk_cap)` （:79 / :94 / :139）改 **下降沿**捕获 —— 数据在 19.78…23.78 ns 到达，晚于下降沿（≈11.6 ns）、早于**下一个** 下降沿（≈31.6 ns）⇒ 建立与保持都变成 +5…8 ns 量级；代价是全部位流与回归要重跑一遍。

→ ★ 2026-10-10 实测结果（见 §1.27）：示波器双通道量得 `tPD_meas ≤ 4 ns` ⇒ 走上面第一条分支：**采样沿不改**（三处保持 `posedge clk_cap`），26 个 ADC 输入端口 失败端点与 `constr/enc_top.xdc:150-249` 的诚实约束**保留。 唯一仍未排除的分支**：ILA/CSV 里是否出现过**真实**的 `angle_raw` 毛刺 （判据见 §1.36/§1.37；§3 item 15）。

教训：把约束改到让工具闭嘴，不等于安全；这次是**让工具说实话**， 代价是签核结论从"全部满足"变成"输入路径已知负余量"—— 但这才是能拿上板去验证、也是能在答辩里讲清楚的状态。

1.20 ★★ 第七个真缺陷落在唯一的上板观测通道上：ILA 多比特探针整条位序反转（2026-10-10 上板抓波时查出并已修）

怎么发现的：第一次上板抓波（13:39，`_synth_work/ila_cap1.csv`）里三个数字互相矛盾 —— `dbg_pole_pairs_eff` 读出 `8`，而寄存器 `0x0100` 写的是 `0x04`（极对 4）； `dbg_stat_byte` 读出 `0x80`，而 `rtl/enc_top.v:602-611` 的 bit7 `res` /bit6 `adapt` 硬接 `1'b0`； `dbg_cycle_cnt` 读出 `0x7C00` / `0xFC00` / `0x8000` / `0x0200`，位序反转后才是干净的 `62/63/64/0`。

根因：`tools/ila_debug.tcl` 把一条多比特探针的 net 列表**按 MSB→LSB** 交给 `connect_debug_port`，而该 Tcl 命令把**列表里第一个 net 接到端口 bit0** —— 于是所有多比特探针的位序整体反转（`bitrev(v, w)` 后才等于 RTL 真值）。 GUI 波形窗、 `.ltx`、导出的 CSV 三者同错，所以"用 CSV 复核 GUI"这种自证方式**查不出来。**

修法（`tools/ila_debug.tcl:126-137`，注释已写"★ 位序（实测 2026-10-10，勿改回）"）： `:132` 仍用 `lsort -integer -decreasing` 生成 `[hi:lo]` 显示名， `:137` 改用 `lsort -integer -increasing` 生成 net 列表（LSB→MSB）。**方向已由 `dbg_pole_pairs_eff` 的期望值交叉验证，但尚需重新下载位流后在波形窗看到 `4` 才能定案。**

检查项/守卫：读旧位流的 CSV 必须逐列做 `int(x,16)` + `bitrev(v,w)` + 二进制补码， `tools/ila_report.py` 已内置该解码（并在报告里打印四条位反证据）。

同轮修掉的两个会误判的文档错误（都是"按文档做会得出错误结论"）： 1. **`dbg_adc_valid` 恒为 1**（`rtl/ad9238_capture.v:163` `adc_valid <= 1'b1;`）， 文档却在 6 处建议用它做触发/"每 4 拍一个脉冲"。上升沿触发永不触发、电平触发无条件秒触发 （这解释了 13:39–13:45 的 9 次武装为何每次都在同一秒触发）。**每 4 拍一个 80 ns 脉冲的是 `dbg_dec_valid`**（probe13， `LOG2DEC=2`）。已改： `docs/07` （§1.5/§1.7）、 `docs/00_roadmap_30d.md:211`、 `docs/01_technical_solution.md:682`、 `docs/02_innovation_scoring.md:93`、 `docs/05_vivado_sim_impl_report.md:939`、 `docs/06_board_test_guide.md:219`。 2. **ILA 探针 `dbg_s_raw` / `dbg_c_raw` 是二进制补码，不是偏移二进制**：模块/引脚输出确实是 偏移二进制（ `docs/04:107`、 `docs/01:551` 讲的是引脚，正确），但 `rtl/ad9238_capture.v:130` `s_calc = ch1_ext - $signed(OFS_CODE)` （ `OFS_CODE = 2048` ）已去偏置 ⇒ 探针上 **0 V→ `000h`、−5 V→ `800h`、+5 V→ `7FFh`**。已改 `docs/07` §1.6 与速查表； 否则 1 Hz 输入时"整屏 0x000"会被误判成"时钟或跳线有问题"。

顺带澄清的两条实测事实（不是缺陷）： CSV/GUI 里的探针名是 `dbg_angle[0:15]` 这种**升序区间**名 （来自 net 名）， `_synth_work/ila_probe_list.txt` 里是 Tcl 映射后的 `dbg_angle_15_0_` 下划线名， 两者是同一根探针； `cycle_cnt` 从 64 跳回 0 并在 0 停 4 拍是 Z 清零过渡态（ `docs/07` 已注明）。

1.21 ★ 新增能力：ADC sticky OTR 可被主机软清（0x0405[0] ⇒ 1 拍 `clr_otr`），上板不再需要"重新 Program 器件"

背景（上板实测踩到的坑）： 一旦 ADC 超量程， `otr_lat` 就**永久锁存** —— `led[0]` (AB21) 常亮、 `irq_n` (K22) 常低，**把幅度降到 9 Vpp 也没用，按 KEY0 也没用。** 原因有三层： ① `otr` 是 sticky（ `rtl/ad9238_capture.v:156-160` 只有 `clr_otr | en_fall` 才清）； ② `en` 在顶层被硬接 `1'b1`， `en_fall` 永不发生； ③ `clr_otr` 原本在 `rtl/enc_top.v` 也被硬接 `1'b0`； ④ `KEY0` (W9) 只经去抖驱动 `key_fall` 做一键标定（ `rtl/enc_top.v:170-171` / `:201` ），与状态清位无关。 顶层 `rst_n = por_cnt[...]` （ `rtl/enc_top.v:117-124` ）是上电自复位，**没有外部复位端口** （ `docs/05:620` 已记 `rst_key_n` 不是 enc_top 端口）⇒ 旧 RTL 下**唯一清法是重新 Program 器件。**

改动（RTL 已落盘，方向明确）： - `rtl/reg_file.v`：新增 `output reg clr_otr`，（1 拍脉冲），在既有 `0x0405 STAT_LATCH` 的 W1C 写译码之后 `clr_otr <= we && (addr == 16'h0405) && wdata[0];` ⇒ **写 `0x0405` 的 bit0 与"清 STAT_LATCH 的 otr 位"是同一个动作**，不额外占地址。 - `rtl/enc_top.v`： 新增 `wire clr_otr_adc;`， `u_capture` 的 `.clr_otr(1'b0)` → `.clr_otr(clr_otr_adc)`， `u_regs` 新增 `.clr_otr(clr_otr_adc)`。 `clk_cap` 与 `sys_clk` 同源（ `rtl/enc_top.v:221` `.clk_cap(clk_i)` ）⇒ **单时钟域，1 拍脉冲足够，无需 CDC/拉宽。** - 语义：清位只清"锁存位 + 两级同步器"；**若 OTR 引脚此刻仍为高，下一拍就会重新锁存** —— 这是正确行为，也是上板时的判据（写后立刻又亮 ⇒ 引脚真的还是高，量 `AD1_OTR` = `J6-39` / `E19`、 `AD2_OTR` = `J6-24` / `B16` ）。

验证（两层，均已 PASS）： - `sim/tb_reg_file.v` 新增小节「6b. clr_otr 脉冲」三条判据：写 `0x0405=0x01` ⇒ `clr_otr==1`、 下一拍自清、写 `0x0405=0x04` （非 bit0）⇒ `clr_otr` 保持 0 （ `sim/logs/tb_reg_file.log:141-143` 三条 `[OK]` ）。 - `sim/tb_enc_top.v` 新增 **TEST D2**（端到端：SPI 写 → `reg_file` → `clr_otr_adc` → `ad9238_capture` ）9 条判据：OTR 引脚拉高 ⇒ `otr_lat` / `led[0]` 锁存且 `irq_n` 拉低； 引脚仍高时写 `0x0405` ⇒ 重新锁存；引脚撤掉后写 `0x0405` ⇒ 清零且 `led[0]` / `irq_n` 恢复； 非 bit0 不清位（ `sim/logs/tb_enc_top.log:147-156`，**check 数 72 → 81**）。 - 回归：**18/18 PASS**（ `tools\run_sim.ps1`， `sim/logs/SUMMARY.md` 2026-10-10 15:09:13） —— 这是**该轮（加桥前）的基线**；加 UART→SPI 桥后为 **19/19 @16:03:11**，见 §1.22。

契约与文档已同步： `rtl/SPEC_interface_contract.md:464-476` （补 `clr_otr` 端口 + 0x0405 语义）、 `rtl/reg_file.v` 文件头地址表与"关键假设"第 4 条、 `docs/07_board_checklist.md` §1.9 与 「SPI 常用包」行（新增 `0x0405=0x0001 清 OTR` ）、 `docs/01_technical_solution.md:605` （把"写 0 清"改为 **W1C 写 1 清**）。

待上板证实： ① 新位流下 `dbg_pole_pairs_eff` 显示 `4` （位序修复定案）； ② 写 `0x0405=0x0001` 后 `led[0]` 熄灭（软清 OTR 定案）。

✅ **已上板证实（2026-10-10，见 §1.23）**： ① 16:12 位流下抓到的 `_synth_work/iladata3.csv` 里 `dbg_pole_pairs_eff` 解出 **4**，位序修复定案； ② 写 `0x0405` 的软清位**确实生效**（ `0x0405=0x00FF` 把 bit2 laglim/bit5 amp 永久清掉），但 bit0(otr) 能否保持 0 取决于输入幅度 —— 10 Vpp 时毫秒级复亮， **降到 8 Vpp 后连续 5 轮 ×1 s 保持 `0x0000`**， `led[0]` (AB21) 灭、 `irq_n` (K22) 回高。

1.22 ★ 新增能力：UART→SPI 桥 —— 没有 SPI 主机也能读写任意寄存器（2026-10-10，本轮新增）

动机：整机调试现场常常没有 SPI 主机，而板上唯一现成的上位机通道就是 USB 串口（CH340E）。 需求原话："加一个 UART→SPI 桥：直接用板上现成的 USB 串口发 4 个字节就能写/读任意寄存器， 不需要任何额外硬件"。

做法：桥在 FPGA **内部**当 SPI 主机，去驱动**既有的** `rtl/spi_slave.v` 从站；顶层用 `br_ss_n` 在"桥"与"外部引脚"之间二选一（ `rtl/enc_top.v` 第 12.5 节），**不占用任何新引脚** （只是把 SPEC 里早已声明、RTL 里一直缺的 `uart_rx` 端口补上）。

新增/改动的文件： - `rtl/uart_rx.v` （**新**，8N1 接收：两级同步 + 起始位中点复采滤毛刺，±4% 波特率容差） - `rtl/uart_spi_bridge.v` （**新**，命令 FSM + SPI 主机 FSM + 应答发送 + TX 仲裁） - `rtl/uart_scope.v` （新增 `hold` / `tx_idle` 两个端口：让遥测"别启动新帧"并把发送权交出去） - `rtl/enc_top.v` （新增 `uart_rx` 端口 + 桥实例 + `u_spi` 输入二选一 + `assign spi_miso = spi_miso_w;` ） - `constr/enc_top.xdc` （新增 `uart_rx` = **T20** 引脚约束、异步输入表补 `uart_rx`，并**废止**了 原注释"设计中不存在 uart_rx、综合会报 `WARNING: [Vivado 12-584]` "） - `sim/tb_uart_spi_bridge.v` （**新**）、 `tools/uart_spi.py` （**新**，PC 端 CLI）

协议（权威版见 `rtl/SPEC_interface_contract.md` §1.12）： 命令 4 字节 = 完整 32 位 SPI 包，首字节 = `0x80 | (wm<<3) | page`； 应答 6 字节 = `5A | resp[31:24] | resp[23:16] | resp[15:8] | resp[7:0] | chk`， `chk = 0x5A ^ 前 4 字节`。**读命令连发两帧**（TW28 重叠式：命令 N 的响应在第 N+1 帧移出）、 **写命令只发一帧**（ `0x0502 CALIB_CTRL` 的 start 位写两次会触发两次标定）； 写命令的应答内容是 上一帧的，**没有语义**，只用于确认"链路通、命令已被执行"。

★★ 本轮查出的第八个真缺陷（仿真实测查出，已修）： `rtl/uart_spi_bridge.v` 原先用**同一个** 条件 `wire tx_hold = (cs==C_REPLY)||(cs==C_TX)||rp_busy;` 既当 `scope_hold` （请遥测别启动新帧） 又当 TX 二选一的选择信号。后果：一进入 `C_REPLY` （SPI 两帧刚发完，**遥测往往正发到一半**） 就把 `uart_tx` 切到空闲的应答发送器，**把正在发送的遥测帧拦腰截断** —— 首轮 TB 实测 `遥测 71 帧 / 1336 字节 / 5 帧错` （ `1336 ≠ 18×71`，字节数对不上就是断帧的铁证）。 修法 （ `rtl/uart_spi_bridge.v:429-441`，已带长注释"勿合并"）： 拆成两条 —— `tx_hold` （→ `scope_hold`，**只挡起新帧**，在发的帧照发完）与 `tx_drive = (cs==C_TX)||rp_busy` （→ `assign tx = tx_drive ? br_tx : scope_tx;`；因为 `C_TX` 之前 已经等到 `scope_idle`，切换必然发生在线路空闲=高电平时，最后一个字节的 stop 位由 `rp_busy` 兜住）。 修后遥测 **0 帧错**。教训与 §1.20 同类：**"请求让路"与"接管线路"是两件事，不能共用一个条件。**

验证： - `sim/tb_uart_spi_bridge.v`：真 `uart_scope`（`SAMPLE_DIV=1000` ⇒ 50 kHz 遥测，线路几乎满载的 最坏仲裁工况）+ 真桥 + 真 `spi_slave(USE_TW28_PACKET=1)` + 64K 寄存器阵列，并用另一个 `uart_rx` 当"上位机"解 TX 线。用例：遥测零错帧、写生效、读回新值（**两帧逻辑的唯一证据**）、`CALIB_CTRL` 只写一帧、坏首字节（`bit7=0`）被丢、半包超时后重同步、波特率慢 2%、 外部主机短占用（100 µs）/长占用（>2 ms 放弃该命令）。 **`TEST PASSED (44 checks)`**，统计行 `遥测: 80 帧 / 1440 字节 / 0 帧错; 应答: 10 条`（`sim/logs/tb_uart_spi_bridge.log`）。 - 全量回归 **19/19 PASS**（`tools\run_sim.ps1`，`sim/logs/SUMMARY.md` **2026-10-10 16:03:11**， 文档回写后又复跑一次 **16:21:05** 同为 19/19； §1.21 记的 18/18 @15:09:13 是新增这一台 TB 之前的基线）。 - 语法： `xvlog --sv -i rtl` 对 5 个相关文件 EXIT=0、无 WARNING（`_lint_tmp/` 里跑，可删）。 - `tools/uart_spi.py --selftest`： **15 checks PASS**（命令编码 + 流分流 + 坏校验和拒绝 + `0xE000` 越界报错）。 `--file` 模式已用合成抓包验证（`_sim_work/fake_cap.bin`）。

上位机用法： `python tools/uart_spi.py --port COM7 write 0x0109 0x00C0`、`read 0x05F0`、 `dump`、 `ports`； `--file` 可离线从抓包文件里找应答。需要 `pyserial`（本机现有 Python 环境**未安装**，且 pip 无法联网 ⇒ 需要用户自行 `pip install pyserial`； `--selftest` 与 `--file` 不需要它）。

待上板证实：③ 用含桥的新位流， `python tools/uart_spi.py --port COM7 read 0x05F0` 应回 `5A 01 F0 10 0C B7`（`DEVICE_ID=0x0C`、`REV=0x10`）。

> ⚠️ 上面这句预测里的第 2 字节 `01` 是**错的**：应答格式是 `5A | resp[31:24] | resp[23:16] | resp[15:8] | resp[7:0] | chk`，其中 `resp[31:24] = {page, 8'h3A}` **的读值**（`rtl/spi_slave.v:242`），读 `0x05F0` 时它 = **`0x053A` STAT_FLAGS**，会随状态变（干净时 `0x10` =caldone、OTR 锁存时 `0x18`），不是常数。

✅ **已上板证实（2026-10-10，见 §1.23）**： `python tools\uart_spi.py --port COM5 read 0x05F0` 实测回 `5A 18 F0 10 0C AE`（XOR 自洽）⇒ `DEVICE_ID=0x0C`、`REV=0x10`、当帧 `0x053A=0x18`。 写通路同批验证：写 `0x0100=0x0004` 回读一致；写 `0x0405=0x00FF` 后 bit2/bit5 永久清零。 上位机侧 `pyserial 3.5` 已装（用户站点目录）， `--selftest` 15 checks PASS。

1.23 ★ 上板判据 A/B 收口：桥实机可用、OTR 软清位生效、复亮根因 = 输入幅度（2026-10-10 实测）

判据 A（探针位序）——通过 - 数据： `_synth_work/iladata3.csv`（1024 采样 / 37 列，16:12 ILA 位流导出，2026-10-10 17:08:51）。 - 位序判定：按 MSB-first 直读时 `dbg_stat_byte` 的 bit7 违例 **0 处**、 `std(hypot(s_raw,c_raw))` = **20.04**； 按 LSB-first 位反读时违例 **1024 处**、std = **577.94** ⇒ **本版 CSV 不能再做位反**（与 08:56 旧位流相反， 见 §1.20；判定脚本 `tools/ila_report.py` 已做成自动判定）。 - **`dbg_pole_pairs_eff` = 4** ✓（配置 `0x0100=0x04`）⇒ §1.20 位序修复在板上定案。 - 同批：f_in **100.018 kHz**、半径 **2020.5 ± 20.0 LSB**（9.866 Vpp）、正交 **−90.7°**、 `angle_raw` 线性残差 RMS **0.806°**、 `angle`（环路后）**0.087°**、`angle_raw` 无非零异常步进 （倒退 0 处，非零步进 504–542 LSB ≈ 4×标称 524）⇒ **100 kHz 下无毛刺。**

判据 B（软清 OTR）——通过（带幅度前提）： `python tools\uart_spi.py --port COM5 …`，2 Mbaud，桥见 §1.22。

输入条件	写 `0x0405`（W1C）清位后 bit0(otr)	旁证
~9.87 Vpp（AFG 10 Vpp 档）	**3/3 轮毫秒级复亮**（+0.00 s 就回来）	`0x0404` bit0=1 实时、 `0x053A=0x18`、 `STAT_AMP_SQ=0x28DB4108 /2^30=0.6390`
AFG `Output` 关（源 >10 MΩ 高阻）	第 2–5 轮 **0/4**（清位后 1 s 内保持 0）	`STAT_AMP_SQ=0x00000431 /2^30=1.0e-6` ⇒ 输入确实没了
8 Vpp = 0.8 满量程	**0/5 轮 ×1 s， `0x0405` 恒 `0x0000`**	`0x0404=0x00`（无实时告警）、 `0x053A=0x10`（只剩 caldone）、 `STAT_AMP_SQ=0x1A30B9B2 /2^30=0.4097`（= (0.8×0.8082)²，标定仍生效）

根因（定量）： ADC 的 OTR 引脚本身正常且**跟随信号**， `clr_otr` 通路与 sticky 逻辑也正常； "清不掉"纯粹是**输入压在满量程上** —— 由标定增益反推原始输入 = 0.8/0.8082 = **0.990 FS**（c 路 0.983 FS； `0x0514` 读得 `GAIN_S=0x6773`、 `0x0516` 读得 `GAIN_C=0x682D`）， ILA 抓到峰值半径 **2054.8/2048 = 1.0033 FS**（波峰离轨仅 5–22 LSB）⇒ 每个周期的波峰都真的越限。 ⇒ **操作口径：本模块 10 Vpp 即满量程，上板一律用 8 Vpp**（`docs/07` 行 1.9 已写入此结论）； 10 Vpp 只用于"看满屏李萨如图"，不能用于过 OTR/告警判据。

同时证实的三点 - 桥通路收口（§1.22 的 ③）： `read 0x05F0` → `5A 18 F0 10 0C AE`（XOR 自洽）⇒ `DEVICE_ID=0x0C`、 `REV=0x10`。 - 写通路 + W1C： `write 0x0405 0x00FF` 把 bit2(laglim)/bit5(amp) **永久清掉**（`0x0025` → `0x0001` → `0x0000`）。 - 清位接线： `rtl/enc_top.v:214` 的 `clr_otr_adc` 确实由 `:753` 的 reg_file 实例输出驱动、再进 `:235` `u_capture`（不是悬空），修复已含在 16:12 位流里。

本轮附带修掉的上位机缺陷（`tools/uart_spi.py`） 1. 标定结果块名称**整体错位一格**（原把 `0x0512` 叫 `CAL_GAIN_S`、 `0x0514` 叫 `CAL_PHASE_K`）⇒ 按 `rtl/reg_file.v` 改正为 `0x0510/11=OFS_S`、 `0x0512/13=OFS_C`、 `0x0514/15=GAIN_S`、 `0x0516/17=GAIN_C`、 `0x0518/19=PHASE_K`、 `0x051A=SHIFT`。★ 正是改对之后才用增益反推出"输入 0.99 FS"这个根因。 2. `BITFIELDS[0x0405]` 位定义错（原写成 laglim/amplo/amphi/otr）⇒ 按 `rtl/reg_file.v:222-224` 改为 **bit0 otr / bit2 laglim / bit3 inclim / bit5 amp**（`0x0404` 同表）； `BITFIELDS[0x053A]` 补成 8 位 （+bit4 caldone / bit5 calbusy / bit6 crcerr / bit7 frame_active）； `0x0536-39` 更正为 angle/cycle。 3. 新增 `--retries N`（默认 3）： 2 Mbaud 下实测**偶发无应答**（约 15 条命令里 1–2 条，随机），重发即成功； 超时还会打印已收字节尾部，用于区分"完全没应答"与"应答被截断"。 4. 新增 `--verify`： 写后自动回读比对（对 W1C/自清位寄存器如 `0x0405`/`0x0502` 会报"不符"，属预期）。 5. 新诊断脚本 `_synth_work/otr_rate.py`（分析脚本，不进 git）： 清 `0x0405` → 在 +0/0.05/0.2/1.0 s 回读， 统计 bit0 复亮率 —— 上表三行就是它跑出来的； `_synth_work/otr_check.py` 从 CSV 侧独立佐证（撞轨 0 次、 峰值 1.0033 FS）。

口径核对： `docs/07` §1.6（:94）与 §1.7（:95）、 `docs/06:228-230` 对 `dbg_adc_valid` 恒 1、 `dbg_s_raw/c_raw` 是二进制补码（0 V→ `000h`）的说明已是修正后的版本，与本轮实测一致。

仍待办（本节写于 2026-10-10，以下为当时的快照，后续进展见括注）： `tPD_meas`（`docs/07` §1.8） ~~尚未量~~ → ✅ **2026-10-10 已量：≤4 ns，见 §1.27**；UVW 需写 `0x0109=0x00C0` （✅ 已在 §2.8 实测中写入）； ABZ 未验 → ✅ **已验（v6 上板 + v7 自由运行窗口，§2.8）**； BiSS-C 未验 → ✅ **已验（`docs/08` 三探针 A/B/C，v7 上复测 A1/A2）**； UART 遥测未验 → ✅ **已验（`tools/uart_spi.py` 读写/自检一直在用）**； **Z 通道**：仿真已覆盖（`tb_abz_gen` / `tb_enc_top`）， ~~板上未单独抓波~~ → ✅ **2026-10-11 已上板抓波（示波器 `J5-7`：Z 率 156 Hz @10 kHz 输入 = `f_in/64`，脉宽与 `z_width/(1024·f_in)` 相符；同时暴露并已修掉"集成路径 Z 被门控窗截成 40 ns"的缺陷，见 §1.26 板上实测块与 §1.44）**。

1.24 ★ 1 kHz 标定失效根因（窗口在半周期截断）+ CALIB_N 主机可配 + 包模式写"双字节"陷阱（2026-10-10 修复）

★ 先修正 §1.23 的 OTR 根因：本轮之后又做了两组实验，§1.23"复亮 = 输入压满量程"这个结论**已被推翻**，正确结论见 §1.25"假 OTR"。 - 有效清位后（写完立刻回读确认 bit0=0，否则重发且不计入）复亮延迟 **82 / 65 / 78 / 812 / 154 ms**（不规则亚秒级）， 不是"每个周期都报"； `_synth_work/iladata4.csv`（触发 `dbg_otr_lat` 上升沿）显示置位当拍 `s_raw = 0`、 `c_raw = 1632 = 0.797 FS`、 全窗**无撞轨**、 `cor_sat ≡ 0`、半径 1630.9 ± 2.0（7.964 Vpp）； - 用户示波器直夹模块 OTR 引脚：**静态低电平 + ~60 mV 毛刺**；两根 OTR 本来就在 §1.19 的 26 个 setup 违例端点里； - ⇒ 修法 A： `constr/enc_top.xdc:132 set_property PULLTYPE PULLDOWN [get_ports {ad_ch1_otr ad_ch2_otr}]`（证据注释在 `:124`）—— 见 §1.25。

问题（1 kHz 下"一键标定"是负优化）： `rtl/calib_ctrl.v` 的收尾判据是 `if (n_after_t >= MAX_SAMP) finish_t = 1'b1;`（**当拍关门**）⇒ 窗口停在输入周期的中途， 累加均值有偏。 `MAX_SAMPLES = 65536`； 1 kHz 时一个输入周期 **12500** 个样本， `0x0103/0x0104` 默认窗口 64 周期需要 **800000** 个样本 ⇒ 旧实现只走到 **5.24 个周期** 就被截断 （`0x0502` bit3 `truncated = 1` 实测为 1）。实测后果：圆度 RMS 从 **0.34%（未标定）** 变坏到 **3.087%**（点了"一键标定"之后）。

修法 1（`rtl/calib_ctrl.v`）—— 只在整周期边界关门 - 新增 `reg [17:0] nsamp_prev, plast_t; reg [31:0] nxt_est_t; reg cap_stop_t;`，在 `S_ACCUM` 里用"上一个周期的样本数"预测下一个周期结束时的样本数： `plast_t = nsamp - nsamp_prev; nxt_est_t = nsamp + 2*plast_t; cap_stop_t = (nsamp_prev != 0) && (nxt_est_t > MAX_SAMP);` （留 1 个周期余量 ⇒ 预报不会超上限）； - 关门判据改为 `if (armed && cycle_pulse && (((wcnt+1) >= win_n) || cap_stop_t))`，并且只有 `cap_stop_t && ((wcnt+1) < win_n)` 时才置 `truncated = 1`； 累加/不累加两条 `cycle_pulse` 分支都更新 `nsamp_prev`； - ⇒ 累加值恒为**整数个输入周期**（均值无偏），硬上限 `MAX_SAMP` 仍作兜底（输入不转时依旧能收尾并如实置 truncated）。

修法 2（`rtl/reg_file.v` + `rtl/enc_top.v`）—— CALIB_N 接到窗口 - `reg_file.v` 新增 `output wire [15:0] cfg_calib_n` + `assign cfg_calib_n = r_calib_n;`（此前 `0x0503/04` 是"仅存储"）； - `rtl/enc_top.v:573` 附近声明 `wire [15:0] cfg_calib_n;`（**必须在 reg_file 例化之前**，否则报 `ERROR: [VRFC 10-2938] 'cfg_calib_n' is already implicitly declared`），例化里 `.cfg_calib_n (cfg_calib_n),`， `.win_cycles` 由硬编码 `16'd64` 改为 `cfg_calib_n_use`。

修法 3（★ 陷阱：包模式写一次事务写 2 个字节） - `rtl/spi_slave.v:11-13 / :32 / :38`： 包模式一次传输 = 2 字节， `P_WR0` 把 `cmd_data[7:0]` 写 `addr`、 `P_WR1` 把 `cmd_data[15:8]` 写 `addr+1`（`:244-251`）； `tools/uart_spi.py:150 encode_cmd` 也总是发 16 位 data ⇒ **任何 16 位写都会顺带写 `addr+1`**。两个实例： 1. 写 `0x0513 0x0001`（OFS_C 高字节）把 `0x0514`（GAIN_S）低字节清成 0x00（`0x815A` → `0x8100`）； 2. 写 `0x0502 0x0001`（启动标定）把高字节 `0x00` 落到 `0x0503 = CALIB_N 低字节` ⇒ 窗口被清成 0。 - 这个副作用在 CALIB_N 接上之前一直看不见；接上后 `sim/tb_enc_top.v:914 spi_wr16(11'h502, 16'h0001)` 立刻把它暴露成 **TEST F4d / F4f / F11g 三项 FAIL**（`nsamp=256 / wcnt=1 / truncated=0`， 期望 `nsamp=16384 = 64 周期 × 256 样本`、 `wcnt ≥ 64`； 断言在 `sim/tb_enc_top.v:956 / :964 / :1054`）。 机制核对： `win_n = (win_cycles==0) ? 1 : win_cycles` ⇒ `win_n = 1` ⇒ 窗口 = 1 个输入周期 = 256 样本 ✓ 与实测完全一致。 - 兜底： `rtl/enc_top.v` 新增 `wire [15:0] cfg_calib_n_use = (cfg_calib_n == 16'd0) ? 16'd64 : cfg_calib_n;` （CALIB_N 复位默认 64 ⇒ 读到 0 只可能是"被 16 位写波及"，不含合法意图）； - 主机口径（已写进 `rtl/reg_file.v:90` 与 `tools/uart_spi.py` HINTS）： **先写 `0x0503` =低字节、 `0x0504` =高字节 （默认 `0x00` / `0x40` = 64），再触发 `0x0502`**； 若一定要一次 16 位写完触发，用 `0x4001`（高字节 = CALIB_N 低字节 = 0x40）。

回归（2026-10-10） - `tools\lint_all.ps1` ⇒ 合计: **`ERROR 0 / WARNING 0`**（19 个 RTL 源 + 21 个测试台）。 - 四台定向回归： **`tb_calib_min` PASS(4 s) / `tb_enc_top` PASS(145 s) / `tb_reg_file` PASS(4 s) / `tb_spi_slave` PASS(5 s)**， 合计 **PASS 4 / 共 4**，汇总 `sim/logs/SUMMARY.md`； 修法 3 之前 tb_enc_top 是 **FAIL 3 errors / 78 checks**，修完 0 errors。 - 跑法（易错）： `run_sim.ps1` 的 `-Only` 是 `[string[]]`，用 `-File` 传逗号串无效 ⇒ 必须 `powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "& '<工作区>\tools\run_sim.ps1' -Only tb_a,tb_b"`。

实测法标定（临时解，频率相关；新位流上板后需重写）： 自动标定在 1 kHz 下不可用之后，用手工扫参把圆度做到 **0.123%**（1 次谐波 0.0645% / 2 次谐波 0.0087%）： `0x0510 = 0x002B` (OFS_S=+43)、 `0x0512 = 0x015E` (OFS_C=+350)、 `0x0514 = 0x815A`、 `0x0516 = 0x8279` (GAIN_C 按闭式解 ε=0.01928 配平)、 `0x0518 = 0x0063` (PHASE_K≈0.003)、 `0x051A = 0x0000`。 诊断脚本 `_synth_work/round_diag.py`（把 r(θ) 做 1~5 次谐波最小二乘分解：1 次 = 圆心偏移、2 次 = 椭圆度/增益失配、 3 次以上 = 失真；证据 CSV `_synth_work/tele_*.csv`）。★ 这组值是**频率相关**的临时解，真正修法仍是修标定引擎。

1.25 ★ "假 OTR"定案：模块 OTR 引脚静低 + 毛刺，FPGA 侧不能也不该"修信号"（2026-10-10）

现象： 8 Vpp / 1 kHz 下 `0x0405` bit0（锁存 otr）会不定期复亮， `0x053A` bit3 = 1， `led[0]`（= `otr_lat`， `rtl/enc_top.v:813`）与 `irq_n`（`rtl/enc_top.v:818`，低有效）被拉死。

五条证据（指向"引脚上的假事件"，不是真的越量程） 1. 用户示波器**直接夹模块 OTR 引脚**： **静态低电平，放大到 20 mV 档只见 ~60 mV 毛刺** ⇒ ADC 侧根本没有真正的 OTR 断言。 2. `_synth_work/iladata4.csv`（触发 `dbg_otr_lat` **上升沿**，8 Vpp / 1 kHz，f_in = 0.980 kHz）： 置位当拍（`TRIGGER` = 缓冲第 512 拍） `dbg_s_raw = 0`、 `dbg_c_raw = 1632`（= **0.797 FS**）， 全窗两个通道**都没有撞轨**、 `dbg_cor_sat = 0`、半径 1630.9 ± 2.0 LSB（= 7.964 Vpp）⇒ 置位与信号峰值无关。 3. **有效清位后的复亮延迟不规则**（写 `0x0405` 后立刻回读确认 bit0 = 0 才算有效轮次）： 5 轮 = **82 / 65 / 78 / 812 / 154 ms**。早先记的"0.5 s / 2 s"是 0.5 s 轮询粒度限制下的旧读数，作废。 4. 两根 OTR（`ad_ch1_otr = E19`、 `ad_ch2_otr = B16`）本来就在 §1.19 诚实输入约束下的 **26 个 setup 违例端点**里。 5. 逻辑上是**两通道相或、sticky、无去抖**： `rtl/ad9238_capture.v:134 wire otr_now = otr1_sync | otr2_sync;`、 `:155-160` 清位优先 + `else if (otr_now) otr_lat <= 1'b1;`。

★ 副作用：ILA 永远触发不到 `dbg_otr_lat` 的上升沿。因为 `rtl/enc_top.v:234 .en (1'b1)` 恒 1 ⇒ `rtl/ad9238_capture.v:87 wire en_fall = en_d & (~en);` 永不成立 ⇒ `:92 wire otr_flush = clr_otr | en_fall;` 只剩"SPI 写 `0x0405[0]`"一条清路；而 sticky 置位后电平**恒定高** ⇒ 上升沿不再出现（这反过来证明当时它确实是高）。 要看这次置位，必须**先清位、再 arm 触发**（按 §1.24 的"写后回读确认"口径）。

影响面： OTR 只进 `0x0404` / `0x0405` / `0x053A` / `led[0]` / `irq_n`， **不参与角度链**（`s/c_raw → 抽取 → 校正 → CORDIC → 环路 → ABZ/UVW/BiSS`） ⇒ 不影响细分精度，但会把 `led[0]` / `irq_n` 拉死、污染 `0x0405` 的状态判读。

修法 A（已采用，本轮重综合才生效）： `constr/enc_top.xdc:132 set_property PULLTYPE PULLDOWN [get_ports {ad_ch1_otr ad_ch2_otr}]`（+ `weaken` 无关，只加弱下拉，浮空时给固定低电平）。 **修法 B（备选，未实施）**： RTL 侧"连续 8 拍确认"再置 sticky（去掉 60 mV 级毛刺），代价是需要 ADC 时钟域计数器。

1.26 ★ 更正： `z_width`（`0x0108`）的单位是 AB 边沿步，不是 50 MHz 拍

旧文档错在哪： `docs/07`（:19 / :24 / :152）、 `docs/06`（:293 / :351 / :424 / :543）多处写"Z 宽 **4 拍 = 80 ns**（@50 MHz）" ⇒ 把 `z_width` 当成 50 MHz 采样拍数。**RTL 的真实语义**（`rtl/abz_gen.v:52-56` 头注释原文： "Z 宽度：z_width 以**边沿索引步**为单位（SPEC 1.8 原文"以 AB 边沿为单位"）。Z 通道旁路 post-AB 分频 （与 TW28 一致），所以 div>1 时 Z 的宽度按原始分辨率计，折算到"输出 AB 边沿"约为 z_width/div"）：

量	值（1 kHz 输入、 `0x0101` =0 inter=256、 `0x0102` =0 div=1）
A/B 频率	256.0 kHz（周期 3.906 µs）
1 个 AB 边沿步	**977 ns**（A 周期 / 4；也是 B 相对 A 的滞后量）
`z_width` 默认 4（`0x0108` 低字节复位值 0x04）	**Z 脉宽 ≈ 3.9 µs**（不是 80 ns）； **★ v9 起 = 4.88 µs**（`z_width+1` = 5 个边沿步，按 TW28 手册口径）
`z_width = 15`	**Z 脉宽 ≈ 14.6 µs**（★ 写寄存器要用 `0x0108 = 0x400F`，见下方陷阱条）； **★ v9 起 = 15.6 µs**（16 步）
`div=8`（`0x0102 = 0x001C`）	边沿步 ×8 ⇒ `z_width=4` 时 Z 脉宽 ≈ 31 µs（时间随 div 变长，因为 Z 旁路后分频）； **★ v9 起 ≈ 39 µs**（5 步 ×8）

> **★ 2026-10-11 v9：Z 宽度语义按 TW28 手册改为 `z_width + 1`**（`z_pulse_n = {1'b0, z_width} + 5'd1`， `rtl/abz_gen.v`； `r_z_left` 由 4 位加宽到 `Z_PULSE_W = 5`）⇒ **上表三行自 v9 起为 4.88 / 15.6 / 39 µs**， 且 `z_width = 0` **不再是"无 Z"，而是 1 个边沿步（977 ns @1 kHz/inter=256）**。理由与实证见 §1.47/§1.49、 `rtl/SPEC_interface_contract.md` 的 Z 输出条。⚠ **新约束（v9 起）**： `Z_TIMEOUT = 4096 clk = 81.9 µs` 会截短过长的脉冲 ⇒ 需 `f_in ≥ 11.92 × (z_width + 1) Hz`（默认 5 步 ⇒ ≥ 59.6 Hz；16 步 ⇒ ≥ 190.7 Hz）。

> **★★ 2026-10-11 板上实证（ILA，v9 位流，`ILA_TRIG=z_out` 触发）**： `z_width = 4`（`0x0108 = 0x4004`）⇒ `z_out` 高 **240 拍 = 4.800 µs**，脉冲**起止都正好落在 A/B 边沿上**，`[start,end)` 内数到 **5 个 A/B 边沿 ⇒ 5 个边沿步** ✅（数据 `_synth_work/ila_cap_v9_z.csv`，报告 `_synth_work/ila_report_v9_z.txt` 的 `[5b]` 节）；随后写 `0x0108 = 0x4000`（`z_width = 0`）复测 ⇒ **60 拍 = 1.200 µs = 1 个边沿步** ✅ ⇒ **"`z_width = 0` ⇒ 无 Z" 的旧结论已在板上被推翻。** 与表内 4.88 µs 的差只是**输入实际 0.977 kHz vs 标称 1.000 kHz**；按**边沿步**计两者完全一致。 ⚠ **口径提醒**：`[5b]` 里用**窗口平均 f_in** 折算会得到 4.80 步（看着像"少了 0.2 步"），那是窗口平均频率与 脉冲附近瞬时边沿率的 ±5% 偏差 —— **板上判据一律以"A/B 边沿计步"为准**（`tools/ila_report.py` 已把它设为 首选判据，f_in 折算行标注为"仅作旁证"）。
>
> **★★ 2026-10-11 重大订正（Z 脉宽截断缺陷，已由 v8 修复）**：上表是**算术上的应有值**，但 v7 及以前的 RTL 在**集成路径**上根本达不到 —— `rtl/abz_gen.v` 原先把"脉冲推进 + `Z_TIMEOUT` 兜底"也放在 `if (z_window && abzen)` 分支内，而 `z_window` 在装载后 ~2 clk 就关闭（`rtl/pole_counter.v` 的 wrap 延迟 2 clk）⇒ **Z 脉宽被截成 2 clk = 40 ns，`z_width` 形同无效。**三个纯 ABZ 测试台（`sim/tb_abz_gen.v:129/307`、`sim/tb_abz_jitter.v:140`、`sim/tb_loop_abz_chain.v:119,127`）都把 `.z_window(1'b1)` **常开**，所以只在它们内部才得到上表数值；唯一使用真实 `z_window` 的 `sim/tb_enc_top.v` 实测 `Z_width_max=2`（v7/v8 两版日志一致）。详见 **§1.44。** v8 已把"装载"与"推进"分离（装载仍受窗门控，推进与 `Z_TIMEOUT` 在窗外继续），`sim/tb_enc_top.v` 的 B5 判据同时从 `[1,16]` 收紧为 `[1024,4096] clk` 以守住该修复。 ⇒ **上表数值自 v8 起在集成路径成立**；v7 及以前的实际 Z 脉宽 = 40 ns。
>
> **★★ 2026-10-11 板上实测（v8 ILA 位流 + 生产口径寄存器；示波器探 `J5-7 = z_out / M21`）**：① **Z 重复率实测 = 156 Hz** —— 与 `f_in/64` 相符（AFG 档位 10 kHz、UART 侧独立核过 `f_in ≈ 9.9994 kHz` ⇒ 期望 **156.24 Hz**）⇒ **`docs/06:543` 判据 5「每 64 个输入周期 1 个 Z」首次在板上被直证**（此前该判据只有 RTL/仿真依据，板级只有"被截成 2 clk"的缺陷记录）。 ② **Z 脉宽实测与预期相符**：当时把 `z_width` 由 4 写到 **15**（`0x0108 = 0x400F`）⇒ 10 kHz 下期望 **1.465 µs**，用户示波器读数确认"没问题、和预期类似"。 ⚠ **诚实限定**：用户只给了定性确认，**没有留下精确到小数点的示波器读数/游标截图**；若要把这条写进正式报告，建议补一张带游标的截图或一个具体数字（判据通式：v8 及以前 `宽度 = z_width / (1024 × f_in)`；**v9 起** `宽度 = (z_width + 1) / (1024 × f_in)`）。 ③ 量完已把 `0x0108` **恢复为 `0x4004`**（`z_width = 4`），回读确认 `0x0108 = 0x4004`、`0x0109 = 0x0040`（`abzen` 仍开）。
>
> **★★ 写 `0x0108` 的陷阱（真踩过，必须记）**：包模式**一次事务写 2 字节**，高字节落到 `addr+1`（`tools/uart_spi.py:167-171` 与 `rtl/spi_slave.v:244-251` 一致）⇒ 写 `0x0108 = 0x000F` 会把 **`0x0109` 写成 0x00，`abzen` 位被清掉，ABZ/Z 全部停摆**（`rtl/reg_file.v:577 cfg_abzen = r_output[6]`、`:573 cfg_zpol = r_output[2]`，而 `cfg_zwidth = r_zero1[3:0]` 在 `0x0108`）。 正确写法：**`write 0x0108 0x400F`**（z_width=15、`abzen` 保持开）/ **`write 0x0108 0x4004`**（恢复）。 与 §1.34 记的 `0x0502/0x0503` 是同一类坑（凡"低字节在自己地址、高字节在下一个地址"的寄存器都要成对写）。

旁证：`sim/tb_abz_gen.v:14` "TEST C Z 脉冲：一个输入周期恰好 1 个 Z；宽度 = `z_width` 个 AB 边沿"（**★ v9 已改为 `z_width + 1`，循环扩到 `zw = 0…8`**）、`:188` 注释"脉冲宽度 = 脉冲内 A/B 状态变化的次数（= 边沿步数）"、`:400` "受采样相位影响有 ±1~2 个边沿步的量化误差"。 **Z 的周期与机械圈无关**：`rtl/enc_top.v:470 assign z_window = (cycle_cnt[POLE_BITS-1:0] == {POLE_BITS{1'b0}});`（`POLE_BITS=6`）⇒ **每 64 个输入电周期一个 Z**（1 kHz ⇒ 15.625 Hz / 64 ms；`docs/07` 的"每机械圈 1 个"只在 "64 极对磁环转一圈 = 64 个电周期"这个模拟口径下成立）。 **读数提醒**：读 `0x0108` 会显示成 `0xC00F` 之类 —— 16 位回读窗口的高字节是**相邻寄存器** `0x0109`（=0xC0），不是本寄存器被写坏；`z_width` 只看低字节。

1.27 ★ 判据 4（`tPD_meas` 相位余量）实测通过：≤4 ns（v9）/ 3–4 ns（v11 复测）

- 用户实测（2026-10-10）：双通道示波器探 `ad_clk` 与 `D0`，读数 **≤4 ns** ⇒ 按 `docs/07` 判据"继续" ⇒ **`rtl/ad9238_capture.v` 的采样沿不改**（`:79` / `:94` / `:139` 保持上升沿捕获方案）。
- 注意口径：§1.19 的诚实输入约束下实现报告里的 `sys_clk` 域 **WNS −3.3 ns / 26 个失败端点（全是 ADC 输入端口）** 与这条实测**不矛盾** —— 报告里的违例是**静态时序模型**（`set_input_delay` 按 24 ns 数据到达 + 建立窗要求）算出来的，而"采到的数据对不对"由实测 `tPD` 与实际采样沿的相位关系决定 ⇒ **签核口径以实测为准**，时序报告作为余量警示（`docs/07` 行 4 已同步；若将来要消掉这 26 条，按 `docs/07` 行 4 的 ①→④ 顺序处理）。
- **★ v11 复测（2026-10-12 22:35，生产版 `0x000B`）**：CH1 = `J6-26`（50 MHz）、CH2 = **`J6-38`（`CH1_D11`，符号位，1 kHz 输入下 20.18 kHz）** —— 原来打算用 `CH1_D0`，但 D0 在满幅正弦下每秒跳变数百万次、5 ns/div 屏宽只有 50 ns ⇒ 必然"糊成一片"（不是故障）；D11 只在过零点翻转 ⇒ 触发稳定。触发挂 **CH2 上升沿**、5 ns/div、10× / 1 MΩ/DC ⇒ **`tPD_meas = 3-4 ns`**（手册 2…6 ns 内 ✓，与 10-10 的 ≤4 ns 一致）⇒ **判据通过，采样沿保持上升沿、`constr/enc_top.xdc` 不改。**由此 v11 建立余量 ≈ 17 − 3.5 − 1.2 = **+12.3 ns**（`tPD` =2 ns → +13.8、=6 ns → +9.8），与 XDC 推导（相对 `sys_clk` 口径 +10.2…+14.8 ns）一致 ✓。记录表见 `docs/09` §1；**带游标截图 `deliverables/figures/scope_tpd_v11.png` 待用户导出。**
- ⚠ **口径更正（2026-10-12）**：`docs/00_roadmap_30d.md:32`、`docs/05_vivado_sim_impl_report.md:22`、`deliverables/design_report.md`（§6.5(2) 第 1 条）、`deliverables/evidence_index.md:157` 等处曾把**总输入延迟 19.5…24 ns** 写成"AD9238 手册真实 `tPD`"；手册 `tPD` 是 **2…6 ns**，19.5…24 ns = 3.762（源时钟）+ 10.000（半周期相移）+ 3.016（ODDR/OBUF）+ 2…6（`tPD`）+ 1（排线）。五处已统一改写。

1.28 ★★ 第八个真问题：一键标定的 `OFS_S` 有 ~1/4 概率错 ±1800…2300 —— 根因是 `angle_raw` 的 MSB 在过零点抖动，使标定窗口变成奇数个半周期（2026-10-10 上板定位）

- **现象（新位流上电后按 KEY0 即复现）**：KEY0 自动标定给出的 `OFS_S` 时而正确（0x0037/0x0038 = 55/56，与遥测实测圆心一致），时而跳到 `0x0766`(1894) / `0x076A`(1898) / `0xF90C`(−1780) / `0xF6FE`(−2306) / `0xF6FF`(−2305)；用纯 SPI 重复标定（写 `0x0502=0x0001`）做重复性实验，`CALIB_N` = 2/4/8 各 4 轮：**OFS_S 约 1/4 轮次出错，且异常值成对出现（相邻两次几乎相同）** ⇒ 不是随机噪声，是"状态/奇偶性"效应。
- **对照指纹**：同一批实验里 `OFS_C` 全程稳定在 ±4 内（0x015C…0x0161），增益/相位只随 `O_s` 误差 同向漂 ~1% ⇒ **只有 s 通道的一阶矩坏掉**，二阶矩基本免疫。
- **机制（已定量吻合）**：`rtl/enc_top.v` 的标定周期脉冲由 `angle_raw` 的 **MSB 翻转**检测得到（注释谓之"MSB 翻转 = 半圈"）。过零点附近 s≈0、`atan2` 对 s 噪声最灵敏（实测角度噪声 ~0.2° rms，1 kHz 下约 7 个 `cordic_valid` 拍）⇒ MSB 来回抖动时 `calib_cyc_tgl` 的**奇偶性被多翻一次** ⇒ 之后的脉冲相对"θ=0 折返"错开半个周期 ⇒ 窗口变成**奇数个半周期**（非整周期），关门脉冲落在 另一种 MSB 翻转点（θ=π）上。
- **三条独立指纹（这是"定案"而非"猜测"的依据）**：
- ① 幅度：折返点 θ=0 处 s=0、c=+A（A ≈ 25920 Q1.15）；奇数 n 个半周期窗口的 sin 一阶矩 = **2A/(nπ)** ⇒ n=7（3.5 周期）→ 2357，实测 2305/2306；n=9（4.5 周期）→ 1833，实测 1894/1898/1899。
- ② 为什么只有 s 坏：同窗口下 **cos 的一阶矩恒为 0** ⇒ `OFS_C` 永远不受影响（正是实测现象）；偶数个半周期（= 整周期）窗口则无偏 ⇒ 那 ~3/4 的"好轮次"给出正确的 55/56。
- ③ 增益/相位来自方差与协方差，近似免疫（实测 `GAIN_S` 只随 `O_s` 误差同向漂 ~0.7%，与预测吻合）。
- **为什么低频最严重**：抖动张角 ∝ 1/f（≈ 6963/f 个 `cordic_valid` 拍）⇒ 板上 1 kHz 台面最坏（这也解释了更早 100 kHz 那次标定只小错）。
- **修法**：见 §1.29（去抖 + 非整周期窗口自动重标）。**注意 ILA 抓包没有直接拍到抖动**：`_synth_work/angle_msb_chatter.py` + `_synth_work/dump_wrap.py` 对 `iladata4.csv`（1 kHz、OTR 触发）的检查显示那一窗里 `s_raw` 全程 +3…+11（**同号**）、MSB 只翻 1 次 ⇒ 抖动是**间歇性**的，单窗抓不到（必须恰好在 s 的直流残差落进噪声带 ±few LSB 时抓波）。因此修复按**预防性去抖**实现，判定用板上"重复标定 20 次"的统计。

1.29 ★ 修复：非整周期窗口检测 + 自动重标（2026-10-10，已过 lint + 全量回归）

- **① 走了弯路的地方先写下来（★ 别再重复）**：第一版修法是给 `rtl/enc_top.v` 的 MSB 翻转检测加 **"先稳定再接受"**去抖（新 MSB 须连续保持 `lim = max(16, 上次间隔/64)` 个 `cordic_valid` 拍才算一次 翻转）。物理上它确实能挡抖动，但**它给脉冲加了可变延迟** `lim` ⇒ 相邻脉冲间隔 = 2H ± Δlim ≠ 整周期 ⇒ 标定窗口的样本数不再是整周期（`sim/tb_enc_top.v` 实测 `nsamp=16367 ≠ 16384=64×256`），**幂等标定的可重复性被破坏**（回归 F4f/F11 失败：两次一键标定 `OFS_S` −12 vs +7、`PHASE_K` −33 vs 0）。而且 `lim` 取大时还会死锁：进 TEST F 前输入长时间不动 ⇒ 间隔估计饱和到 2^{24} ⇒ `lim=262143` 拍 ≫ 半周期 128 拍 ⇒ **一次脉冲都不产生**，标定挂到 `ARM_TMO_MAX`（busy 不落、窗口空）。 结论：**任何改变脉冲时序的"去抖"都不要做**，脉冲间隔必须严格等于整周期。
- **② 最终修法：脉冲检测保持原样，抖动交给 `rtl/calib_ctrl.v` 的"检测 + 作废重标"。**窗口内每个 周期间隔（`plast_t` = 上一边界以来的样本数）取 min/max，收尾那一拍也计入；判据 `win_bad_t = (lim_max_t > lim_min_t + (lim_min_t >> 2))`，即 **1.25 倍**（不是 1.5 倍：半周期错位会让 窗口里恰好多出一个半周期 ⇒ 只有**一个**间隔变成 3H、其余 2H ⇒ `max/min = 1.5`，用严格 `>1.5` 的判据 会正好漏检）。判定为非整周期就**本轮作废、清累加器、重新对齐**，最多 `MAX_RETRY=3` 次；重试用尽仍坏 才置 `win_bad + truncated`（结果可能有偏，如实上报）。**抖动只要落在窗口内就必然被检出**（3H vs 2H）， 落在 arming 之前则无害（窗口仍是整数个周期，均值无偏）⇒ 这是确定性的修法。
- **③ `rtl/reg_file.v` / 主机口径**：`0x0502` 读回扩为 `[1]busy [2]done [3]truncated [4]win_bad [6:4]重试次数 [7]win_bad`（权威口径 = `rtl/reg_file.v:451`：`{8'b0, calib_bad, calib_retry, calib_trunc, r_calib_done, calib_busy, 1'b0}`； `tools/uart_spi.py` 的 `BITFIELDS` 已同步。★★ 2026-10-10 订正：本节原先写的是 `[4]win_bad [7:5]重试次数`，**是错的** —— `tools/uart_spi.py` 与 `tools/calib_repeat.py` 当时就按那个错口径解码，导致 v3 位流那 20 轮统计的 retry/win_bad 全错，见 §1.30 ①。★ 注意包模式读 16 位会把高字节 `0x0503`（CALIB_N 低字节）一起读出来 ⇒ 正常会看到 `0x400C` 这类值。
- **④ 仿真证据**：`sim/tb_calib_min.v` 新增 **T8** 定向用例——人为制造"8,7,3(半周期),11"的脉冲序列， 实测 `retry_cnt=1 / win_bad=0 / truncated=0 / 第二段整周期窗口解出 ofs_s=ofs_c=0`（62 checks 全过）； 同时新增 T3e/T4c 确认**整周期窗口不触发重标。** `tools/lint_all.ps1` = ERROR 0 / WARNING 0。
- **⑤ 旧位流的失效率基线（修复前，用于对比）**： `tools/calib_repeat.py --n 8` 在**旧位流**（无重试位域） 上实测 `OFS_S` = 56 / **1894** / **1899** / 56 / 57 / **−2305** / 56 / 56 ⇒ **好 5 / 坏 3 = 37.5%**， `OFS_C` = 352/351/349/350/351/351/351/351（**±3 稳**，印证"cos 一阶矩恒 0"的指纹）， CSV 存 `_synth_work/calib_rep_pre.csv`。
- **⑥ 待做**：重出生产版位流后，在板上跑 **20 次一键标定**（每轮 1 kHz 下 ≈ 5 ms，代价可忽略）， 判据 = `OFS_S` **20/20 收敛在好值量级**（`|OFS_S| ≤ 400`）**且** `0x0502[7]win_bad = 0`、 `0x0502[6:4]retry = 0`。修复前后对比：旧位流 37.5% 坏 ⇒ 若修复无效，20 次全洁的概率 ≈ 0。 ★ 但**本层（检测+重标）在 §1.30 ① 的实测里被证明只是安全带**，真正根治的是 §1.30 的滞回过零。

1.30 ★★★ 根治：过零参考改"带滞回的角度过零"（2026-10-10，已过 lint + 全量回归）

> 本节是 §1.28 那条"MSB 抖动根因"的**最终解法**。§1.29 的"检测 + 作废重标"在**板上实测中被证明救不了** （抖动几乎每个窗口都出现，重试只是又落进同一个抖动窗），保留它只作为安全带与诊断通道。

- **① v3 位流的板上实测（20 轮，`python tools\calib_repeat.py --port COM5 --n 20`，CSV `_synth_work/calib_rep_v3.csv`，完整 stdout `_synth_work/calib_rep_v3_stdout.txt`）**： **好 14 / 坏 6**； `OFS_S` = 57/58/58/59/**−1778**/59/**−2303**/59/**1896**/**1900**/57/**−2303**/59/60/57/**−2303**/59/58/58/58（min −2303、max 1900、极差 4203、σ=1129）； `OFS_C` = 349…353 **稳**； 坏轮 `GAIN_S` 同向偏高（33226/33337/33228/33144/33336）； `truncated=1` 全部 20 轮（正常：1 kHz 下 `MAX_SAMP=65536` 只够装 ~5 个周期）。 **★ 我的解码器当时写错了**：按错口径（`retry=(ctrl>>5)&7`、 `win_bad=ctrl&0x10`）读出的"retry 分布 {0:2,1:1,5:17} / win_bad 17 次"是假的；按正确位域（`rtl/reg_file.v:451`）反推 `0x0502` 低字节 = 17 轮 **0xBC**（`win_bad=1 + retry=3` = 重试用尽）、1 轮 **0x2C**（`retry=2`、 `win_bad=0`）、 2 轮 **0x0C**（`retry=0`、 `win_bad=0`）。⇒ 结论不变且更强：**"窗口非整周期"是真实的、且几乎每轮都发生**， 而"作废重试"最多只把 6 个坏轮里的 1 个救回来。`tools/uart_spi.py` / `tools/calib_repeat.py` / `tools/calib_repeat.py --selftest` 的位域与自检**已按 `rtl/reg_file.v:451` 订正**（自检里加了 `0x40BC` / `0x002C` 两个板上实测样本，旧口径会判它错）。
- **② 坏值量级把机理钉死（定量）**：设 A≈32304（= 采样半径 2019 LSB × 16，10 Vpp 输入），窗口含 `m+1/2` 个周期时 sin 的一阶矩 = `2A/(π(2m+1))` = `20566/(2m+1)`： **m=4 ⇒ 2285**（实测 2305/2360/2281）、**m=5 ⇒ 1870**（实测 1778/1839/1894/1899） ⇒ 窗口正好是 **4.5 / 5.5 个输入周期**（半个周期污染一阶矩）； 而 cos 的一阶矩恒为 0 ⇒ `OFS_C` 永远稳 ⇒ 这正是"MSB 多翻**奇数**次 ⇒ 脉冲奇偶性反转 ⇒ 脉冲相位错开半个周期"的指纹。 （排除过的替代解释：`rtl/calib_ctrl.v:443-460` 的 S_IDLE 启动分支**确实清** `nsamp / nsamp_prev / int_min / int_max / int_seen / retry_cnt / win_bad`（`nsamp_prev <= 18'd0` @455）⇒ 不存在"第一个间隔 被当成 2 个周期"的假阳性 ⇒ 检测器报警是**真报警**。）
- **③ 修法（`rtl/enc_top.v`）**： 把标定周期脉冲的相位参考从**"`angle_raw` MSB 翻转"**换成 **"带滞回（施密特）的角度过零"**：
- `localparam signed [AW-1:0] CAL_HYS = 16'sd2048;`（= 11.25°，**幅度无关**：只在角度域判断）
- `wire signed [AW-1:0] calib_ang_s = angle_raw;`、 `reg calib_s_side;`（1 = 已确认在 s>0 半圈）
- 每个 `cordic_valid`： `!calib_s_side && (calib_ang_s > +CAL_HYS)` ⇒ 确认进入 s>0 半圈； `calib_s_side && (calib_ang_s < -CAL_HYS)` ⇒ 确认进入 s<0 半圈（θ 越过 ±π 折返）； 每次确认才翻转 `calib_cyc_tgl`，且 `if (calib_cyc_tgl) calib_cyc_pulse_r <= 1'b1;`（**每两个半圈 = 一个输入周期发 一个脉冲**，与原来同构； `wire calib_cycle_pulse = calib_cyc_pulse_r;` 未变，仍接 `u_calib` 的 `.cycle_pulse(calib_cycle_pulse)`）。原 `calib_ang_d` 已删除。
- **为什么它不违反 §1.29 的教训（脉冲时序不能动）**： 滞回加的是**恒定角度**延迟（恒频 ⇒ 恒定时间）， 不是可变延迟；每两个相邻脉冲跨越的仍然是**整整一圈** ⇒ 脉冲间隔严格等于一个输入周期 ⇒ 窗口仍是 整数个周期。即使谐波失真让两半圈的延时不对称，**二者之和仍是整周期** ⇒ 一阶矩无偏。脉冲落在 A/B 两类事件上交替，与 `win_n` 奇偶无关。
- **余量**：θ 噪声实测 ~0.1~0.5° rms，`CAL_HYS=11.25°` 是 20σ 以上 ⇒ 抖动翻不动状态。代价是事件比 真实过零晚 3.1% 周期（1 kHz ⇒ 31 μs），arming 与关门同源 ⇒ 完全抵消。
- **④ 验证（仿真已完成，板上待复测）**： `tools/lint_all.ps1` = **ERROR 0 / WARNING 0**； `tools/run_sim.ps1` = **PASS 19/19**； `tb_enc_top` 日志： `F4: … done=1 nsamp=16384 wcnt=64 truncated=0`、 `F11: 两次标定之差 OFS_S=0 OFS_C=0 GAIN_S=0 GAIN_C=0 PHASE_K=0`、 `F11: 两次标定窗口 nsamp=16384/16384 wcnt=64/64`、 `TEST PASSED (81 checks)` ⇒ **整周期性与幂等性都保住了**（对比 §1.29 ① 那条可变延迟方案： `nsamp=16367`、F4f/F11 全挂）。 **板上判据（下一步）**： 烧 v4 位流后再跑 20 轮，要求 **20/20 的 `|OFS_S| ≤ 400`（且各轮彼此相差 ≤ 几 LSB）**，且 **`0x0502[7]win_bad = 0`、 `0x0502[6:4]retry = 0`。 ★ 2026-10-11 实测修正**：v4 板上前半句达成（20/20、σ=0.7），**后半句没有**（16/20 轮仍报警）—— 根因是单拍 ADC 错值造出的一个假周期脉冲，对结果无害，**v5 才真正让它安静**，见 §1.31 ②③。

1.31 ★★★ v5：角度毛刺保持（挡单拍杂散）+ `oflow` 长期为 1 的真相 + 一条被仿真抓住的自伤（2026-10-11）

> 起点是 §1.30 ④ 那条板上判据**没有达到**：v4 位流烧录后 20 轮标定**结果 20/20 全对**（`OFS_S` 极差 2 LSB、σ=0.7），但 `0x0502` 读回在 **16/20 轮仍是 `retry=3 + win_bad=1`**。本节先解释它， 再给修法。

- **① `oflow` 长期为 1 不是活故障，而是 sticky OTR 锁存**

- `rtl/enc_top.v:634`：状态字 bit0 `oflow = cor_sat | otr_lat`； `0x053A` bit3 = `otr`（`:111`）。
- 板上实测（COM6，`tools\uart_spi.py`）：`0x0404 = 0x2501`（`otr`=1）、`0x053A = 0x0018`（`otr` + `caldone`）；写 `0x0405 = 0x00FF` 后**立即**复读 ⇒ `0x0404 = 0x0100`（none）、`0x053A = 0x0010`（只剩 `caldone`）⇒ **W1C 清位有效**，且 `oflow` 确由 `otr_lat` 造成（若 `cor_sat` 恒真，清完不会掉）。
- **3 秒后复读 `0x0404 = 0x2101`、`0x053A = 0x0018` ⇒ 模块的 OTR 引脚自己又拉高了锁存**（8 Vpp/1 kHz 正常激励、`amplo/amphi` 恒 0 ⇒ 与幅度无关）⇒ 与 §1.25 定案一致：这是 AD9238 模块 OTR 引脚的行为，FPGA 侧不能也不该"修信号"。
- **后果与处置**：一旦锁住，`oflow` 就再没有信息量（24 份遥测 CSV 普查：`tele5s.csv` 49851 帧 全干净，而 §1.25 之后的 CSV 几乎每帧都带 `oflow`）⇒ **`tools/uart_scope.py` 与 `tools/calib_repeat.py` 现在每次采数前自动写一次 `0x0405=0x00FF`**（并丢弃该次应答），让状态字节恢复信息量。
- **② v4b 的 `retry=3/win_bad=1`（16/20）根因：单拍 ADC 错值 ⇒ 一个假周期脉冲**
- 一个被采错的 ADC 转换（半个量程）经 4 拍抽取（`LOG2DEC=2`，50 → 12.5 MSPS）后仍能改变 `0.5·A` ⇒ 角度跳 **≥22.5°** ⇒ §1.30 的滞回状态机被**翻两次**（越阈再越阈回来）⇒ `calib_cyc_tgl` 多发一个脉冲 ⇒ 某个间隔被切成 25/231 拍 ⇒ `win_bad` 报警、重试跑满 3 次。
- **F13（v5 之前那版）在仿真里精确复现了板上的签名**：`retry_cnt=3 win_bad=1 int_min=25 int_max=256`，且**6 组系数与干净输入逐位一致**（系数是 s/c 样本的函数，单个坏样本在 5 万 样本均值里只贡献 ~0.02 LSB）⇒ **§1.30 ④ 的"retry 必须为 0"这条判据口径要改**：报警可以出现，结果仍可以完全正确。
- 板上旁证：v4b 20 轮 `OFS_S ∈ [53,55]`（σ=0.7）⇒ 重试用尽后留下的窗口仍是整周期。
- **③ v5 修法：角度毛刺保持（`rtl/enc_top.v` "5b. 角度毛刺保持"）**
- `localparam signed [AW-1:0] ANG_GLITCH_MAX = 16'sd8192; // 45°`； `wire signed [AW-1:0] ang_delta = $signed(angle_raw) - $signed(ang_prev);`（16 位有符号减法 天然处理 ±180° 回绕）； `ang_glitch = ang_seen && !ang_held && ((ang_delta > 8192) || (ang_delta < -8192));`； `angle_filt = ang_glitch ? ang_prev : angle_raw`。
- `angle_filt` 同时喂 `loop_filter.ang_in`（原 `angle_raw`）与 `wire ... calib_ang_s = angle_filt;`（原 `angle_raw`）⇒ **环路与标定都不再被单拍错值踢**（`angle_raw` 原本只有这两个消费者）。
- 余量：1 kHz 输入下正常单拍步进 0.0288°，45° 门限 ≈ **1500×** 余量。
- **★ 必须"最多保持一拍"——第一版漏了这一点，被 TEST B 抓住（自伤记录）**：不限时地保持，则**真实相位阶跃**（`sim/tb_enc_top.v:475` 的 `ph = 20'd0;`）会让 `ang_prev` 冻结、而输入从 0 缓慢前进 ⇒ 差值长期 >45° ⇒ **角度被冻结约 3 万拍** ⇒ `dut.angle` 与参考最大差 **32768 = 180°**、**32732 个采样超差**（`B7-fwd-angle-tracks-phase` FAIL）。加 `reg ang_held;`（本拍保持过 ⇒ 下一拍无条件接受原始值）后恢复。真实毛刺是"一个坏样本"，挡一拍正好够，且真实阶跃只延后一拍。
- **④ 验证（全部已跑完）**
- `tools/lint_all.ps1` = **ERROR 0 / WARNING 0**； `tools/run_sim.ps1` = **PASS 19/19**； `tb_enc_top` = `TEST PASSED (99 checks)`。
- **F13**（注入 1 拍、走 `cap_stop` 路径）：`毛刺保持动作 8 次: retry_cnt=0 win_bad=0 int_min=255 int_max=256 nsamp=65280`，系数差 `OFS_S=0 OFS_C=0 GAIN_S=0 GAIN_C=0 PHASE_K=0` ⇒ **单拍杂散被完全挡住，窗口与系数逐位不变。**
- **F14**（`force dut.angle_filt` 6 拍、**绕过**保持，注入 30 次）：`retry_cnt=3 win_bad=1 int_min=1 int_max=256 nsamp=60202`，系数差 `OFS_S=+2 OFS_C=-2 GAIN_S=-1 GAIN_C=+1 PHASE_K=3` ⇒ **检测器仍然有效，且结果仍准确**（0.1% FS 量级）。
- **判据分层（写进测试台，避免下次再错）**：*"杂散被挡住 ⇒ 系数逐位一致"* 归 **F13**；*"脉冲列 真被打乱 ⇒ 只要求准确（偏移/增益 ±8 LSB、相位 ±32）"* 归 **F14**。强行要求 F14 逐位一致是错的：作废重试后选中的整周期窗口**起点会平移**（实测 `nsamp` 65280 → 60202），采样集合变了 ⇒ 系数天然差几个 LSB。
- ILA 探针：`ifdef ENC_TOP_ILA` 下新增 `dbg_ang_glitch` / `dbg_ang_delta`，为下一次 ILA 版本 **以毛刺本身作触发条件**做准备（现有 1024 深度 @50 MHz ⇒ 单次捕获仅 20.5 µs，而毛刺 ~750/s ⇒ 随机命中概率仅 ~0.5%/次）。
- **⑤ 本轮明确"不改"的东西**：`constr/enc_top.xdc:150-249` 的 `set_input_delay -clock sys_clk -max 24.000 / -min 19.500` **保持不变** —— 它是**诚实值**：数据到 FPGA 引脚 = clk 引脚 3.762 + 半周期 10.000 + ODDR Tco/OBUF 3.016 + tPD 2...6 + 排线 1 = **19.78...23.78 ns**，而捕获沿 ≈ 24.0–24.6 ns ⇒ 余量≈0。把 26 个 ADC 端点"改绿"只有两条正路：实测 `tPD_meas`（§1.27 已 ≤4 ns）或按 §1.19 / `constr/enc_top.xdc:229-245` 把捕获改成**下降沿** —— 后者等 ILA 直证毛刺确实存在后再做，本轮不动。
- **⑥ 上板判据（v5 位流烧录后）**：`python tools\calib_repeat.py --port COM6 --n 20 --csv ...` ⇒ **20/20** 且 `|OFS_S| ≤ 400`、各轮彼此 ≤ 几 LSB，**并且这次要求 `0x0502[7]win_bad = 0`、`0x0502[6:4]retry = 0`**（§1.30 ④ 当时达不到、本节 ② 已解释的那一条）；另复看 ABZ 边沿抖动（毛刺不再进 `loop_filter`）。

1.32 ★★★ v6：单通道半量程错值才是真凶（v5 的 45° 阈值根本够不着）+ 两道防线 + 可读诊断（2026-10-11）

- **① v5 上板实测：一点没改善（这就是本节的起点）** `python tools\calib_repeat.py --port COM6 --n 20 --csv _synth_work\calib_rep_v5.csv`：**结果 20/20 全对**（`OFS_S` 53...56 / `OFS_C` 354...358 / `PHASE_K` 97-98 / σ≈0.7，与 v4b 同级），**但 `retry` 分布 = {3:18, 2:1, 1:1}、`win_bad` 命中 ≈18/20 轮**，对比 v4b 的 {3:16, 2:2, 0:2} ⇒ **v5 的 45°(8192 LSB) 毛刺保持对 `win_bad/retry` 完全没有作用**，说明阈值没打在真凶身上。
- **② 根因：v5 的注入判据挑了"最猛"的情形，把真实情形漏了** §1.31 的 F13 注入的是**两个通道同时取反 ⇒ 角度跳 180°**；而板上真实出现的 ADC 错值更可能是**一个通道的 MSB 翻转**（偏移二进制里等价于 ±半量程）。此时 `atan2` 的角度误差只有 **~26° 量级**（4 拍抽取平均后单通道半量程错值仍可达 0.5·A）：**够翻 11.25°(2048) 的标定滞回 ⇒ 产生假周期脉冲；但够不着 45°(8192) 的保持 ⇒ 照旧漏过去。** ⇒ 这解释了 v5 为何"结果对、旗标照样响"。**教训：判据要按"最可能发生的错误幅度"设计，不是按"最严重的错误幅度"设计**（两者相差 7 倍时，防线会整条落空）。
- **③ v6 的两道防线（`rtl/enc_top.v`）**
- **第一道：阈值降到真实错误幅度以下。** `localparam signed [AW-1:0] ANG_GLITCH_MAX` 由 `16'sd8192`(45°) **改为 `16'sd1024`(5.6°)。**余量核算：1 kHz 输入下正常单拍步进 **0.0288°**（190× 余量）、实测角度噪声 0.1...0.5° rms（**>11σ**）。阈值**故意取在 `CAL_HYS = 2048`(11.25°) 之下** ⇒ 逻辑上保证"**凡是能翻动标定滞回的跳变，必先被毛刺保持挡住**"。
- **第二道：脉冲发生器两级化（对"任意幅度"的单拍错值免疫）。** 第一级 `side_raw` 仍用滞回比较；第二级加 `CONF_N = 4'd3` 拍确认：只有 `side_raw` 连续 3 个有效样本都与当前 `calib_s_side` 相反，才提交翻转并可能发脉冲，中途一旦相同就 `conf_cnt <= 0`。 **关键是"恒定延迟"**：提交延迟恒为 3 拍，与周期无关 ⇒ 相邻脉冲间隔仍**严格等于一个输入周期** ⇒ 标定窗口仍是整周期（这正是 §1.29 用过的教训：**任何"延迟随周期变化"的滤波都会毁掉整周期 性**，当年被放弃的 MSB 去抖就是这么坏的）。
- 两道防线**分工**：单拍错值被第一道挡住；第一道的"最多保持一拍"意味着**连续 2 拍**时第 2 拍 必然漏进 `angle_filt` ⇒ 由第二道挡住（F15 就是按这个场景设计的）。
- **④ 诊断：把"到底有没有毛刺、多大、多密"变成 UART 可读数字**（本次新增，SPI 只读）
- `rtl/enc_top.v` 新增 5c 节计数器（在 `cordic_valid` 上用 `ang_abs_delta = |ang_delta|` 分档累加） 与周期脉冲累计； `rtl/calib_ctrl.v` 新增 4 个**窗口快照**口（`dbg_int_min/dbg_int_max/ dbg_nsamp_cls/dbg_wcnt_cls`，在关门判据处锁存 ⇒ 留下的是**最后一次尝试**的窗口统计）。
- 地址表（16 位小端，低字节在前）：

地址	内容	单位/说明
0x0505/06	gl_cnt_1024	abs(Δang) > 5.6° 的累计次数（上电以来）
0x0507/08	gl_cnt_2048	> 11.25°（**能翻滞回的门槛**）
0x0509/0A	gl_cnt_4096	> 22.5°
0x050B/0C	gl_cnt_8192	> 45°
0x050D/0E	gl_hold_cnt	毛刺保持实际动作次数
0x0540/41	cyc_pulse_cnt	标定用周期脉冲累计
0x0542/43	dbg_int_min	最后一次窗口的最小脉冲间隔（拍）
0x0544/45	dbg_int_max	最后一次窗口的最大脉冲间隔（拍）
0x0546/47	dbg_wcnt_cls	关门时的周期计数
0x0548/49	dbg_nsamp_cls	关门时的样本数
0x05F2/53	rtl_build_id	**v6 = 0x0006、v7 = 0x0007、v8 = 0x0008、v9 = 0x0009（Z 宽度 = z_width+1，§1.49）；v5 及以前读 0 ⇒ 非零即 v6+**

- **用法（不怕 UART 慢/丢帧）**：读两次的**差值** ÷ cyc_pulse_cnt 的差值 = **每输入周期的毛刺次数**； int_max / int_min > 1.25 ⇒ 该窗口**不是整周期**（与 calib_ctrl 的作废判据同源）。 > 11.25° 一档为 0 ⇒ 当前激励下角度跳变**根本没到能翻滞回的幅度**（那旗标就该是别的来源）。
- tools\calib_repeat.py 已加读回段：先读 0x05F2 打印 RTL_BUILD_ID （<6 会提示"v5 或更早"）， 再打印毛刺密度、 int_max/int_min 比值与判读；整段 try/except 容错。
- **⑤ 仿真验证（v6，全部已跑完）**
- tools/lint_all.ps1 = **ERROR 0 / WARNING 0**； tools/run_sim.ps1 = **PASS 19/19**； tb_enc_top = **TEST PASSED (105 checks)**。
- **F13**（1 拍、**双通道**取反 = 180°）： 毛刺保持动作 8 次：retry_cnt=0 win_bad=0 int_min=255 int_max=256 nsamp=65280 ，系数差全 0 ⇒ 极端情形仍被第一道完全挡住。
- **F14**（force dut.angle_filt 6 拍、**绕过**保持，注入 32 次）： retry_cnt=3 win_bad=1 int_min=20 int_max=256 nsamp=63536 ⇒ **CONF_N=3 没有把检测器弄聋**（真实扰动仍被抓到， 系数差在容差内）。
- **F15（v6 的新判据，针对 ② 的真凶）**：每 8000 拍注入**连续 2 拍**的单通道 MSB 翻转（共 16 拍）： retry_cnt=0 win_bad=0 int_min=255 int_max=257 nsamp=65040 gl1024=237(+32) hold=122(+16)； 系数差 OFS_S=0 OFS_C=0 GAIN_S=+2 GAIN_C=+1 PHASE_K=+5 SHIFT=0 ⇒ **两道防线对真实情形有效。** 括号里的 **+32 / +16** 是本段自己产生的增量（计数器是上电以来累计，测试台已改成打印差值）： 8 次注入 ×（坏拍、被保持拍、以及其后两个"远离被污染前值"的干净拍）= 32 次计数 / 16 次保持， **与"每次注入在角度上留下 4 拍余波"的机理完全吻合** ⇒ 计数器行为可解释、可信。
- **判据分层再确认**：*"杂散被挡住 ⇒ 系数逐位一致"* 归 **F13**；*"脉冲列真被打乱 ⇒ 只要求 准确（偏移/增益 ±8 LSB、相位 ±32）"* 归 **F14/F15** —— 被注入的错样本虽然进不了角度脉冲路径， 但仍会进入 S1..S5 累加，**采样集合与干净那次不同** ⇒ 差几 LSB 是必然的。
- 另修一处隐患： reg [15:0] cyc_pulse_cnt; 必须声明在 u_regs 例化**之前** （否则被当成隐式 wire，lint 直接报 [VRFC 10-2938] ）；第一次 lint 就是这么抓出来的。
- **⑥ 上板判据（v6 位流烧录后）** 1. 先读 0x05F2 ⇒ **必须 ≥ 6**（v6 = 6 / v7 = 7 / v8 = 8 / **v9 = 9**；读到 0 说明烧的是 v5 或更早）。 2. python tools\calib_repeat.py --port COM6 --n 20 --csv _synth_work\calib_rep_v6.csv ⇒ **20/20**，且这次**要求 0x0502[7]win_bad = 0 、 0x0502[6:4]retry = 0**（§1.31 ⑥ 未达成）。 3. 读诊断： > 11.25° 计数应与"每周期毛刺次数"吻合；若 gl_cnt_2048 仍非 0 而 win_bad = 0 ， 说明第二道防线在起作用（这正是设计目标）；若 gl_cnt_1024 为 0，则板上**根本没有角度毛刺** ⇒ 旗标原因要另找（ int_max/int_min 与 dbg_nsamp_cls 就是为这个准备的）。 4. 顺带复看 ABZ 边沿抖动（毛刺不再进 loop_filter ）。
- **⑦ ★ 上板实测结果（2026-10-11 13:2x，烧录 v6 后实测：判定全部达成）**
- **版本确认**： 0x05F2 读回 **6** ✓（ 0x05F0 DEVICE_ID 仍 0x0C 、 0x05F1 REV 仍 0x10 ）。
- **20 轮一键标定：20/20 全部 GOOD，retry 分布 = {0:20} 、 win_bad = 0/20** ⇒ §1.31 ⑥ 遗留的"系数对但旗标脏"**彻底解决。**对照： v4b retry={3:16,2:2,0:2} / 16 轮 win_bad；v5 retry={3:18,2:1,1:1} / 18 轮 win_bad。
- **系数一致性更好**： OFS_S **53…54**（极差 1、σ=0.5，v5 是 53…56、σ≈0.7）、 OFS_C 352…359、 GAIN_S 33060…33065、 GAIN_C 33347…33354、 PHASE_K 恒 **97**、 SHIFT 0。
- trunc = 1 ：**每轮都是 1，且 v4b/v5 的 20/20 也全是 1 ⇒ 属设计行为、不是 v6 缺陷**： MAX_SAMP = 65536 、一个输入周期 ≈ 12501 样本，窗口收到 4 个周期（≈50007）时再进一个 周期就会顶上限（§1.29 的悲观预测用 2 倍间隔）⇒ cap_stop_t 主动在当前周期边界关门。
- **毛刺密度（ tools/diag_counters.py 20 s 求差， _synth_work/diag_v6.csv ）**： Δ周期脉冲 = 20038 ⇒ 输入 **≈1.002 kHz**（与 AFG 设置吻合）； Δ>5.6° = 89 、 Δ>11.25° = 0 、 Δ>22.5° = 0 、 Δ>45° = 0 、 Δ保持动作 = 89 ⇒ **0.00444 次/输入周期（≈4.45 次/秒），且全部落在 5.6°…11.25° 这一档** —— 这正是 §1.32 ② 断言的"v5 够不着、v6 正好接住"的区间，**这次是从板子上量出来的。**
- Δ>5.6° 与 Δ保持动作 **完全相等（89=89）** ⇒ 这一档的每一拍都被 5b 保持挡住； 两者之差为 0 ⇒ 该窗口内**没有出现连续 2 拍毛刺**（若出现，第 2 拍会漏进 angle_filt ， 由 CONF_N 与窗口 1.25 倍判据兜底 —— 见 ③）。
- **累计计数**（自配置起，仅 POR 清零）： >5.6° = 486 、 >11.25° = 8 、 >22.5° = 8 、 >45° = 3 ⇒ 大跳变（v5 唯一能看见的那类）**只占 0.6%**，量级差 60 倍以上； gl_cnt_8192 = 3 也直接 解释了"**为什么 v5 的 45° 阈值 18/20 轮都抓不到东西**"。
- **窗口快照**（最近一次关门）： int_min = 12498 、 int_max = 12506 ⇒ 比值 **1.0006** （判据 1.25 倍，余量极大）； wcnt = 4 、 nsamp = 50007 ⇒ 50007/12501 ≈ 4.0 **自洽**， 窗口完整。（注意：快照只在**标定关门**时更新，未标定时两次读数必然相同，不是寄存器卡死。）
- 0x053A 读回 0x18 = bit4 caldone + **bit3 otr**（模块 OTR sticky 复亮，按 §1.21 属正常现象，工具已自动清位；与累计 gl_cnt_8192 = 3 同源，是输入幅度触顶而非逻辑错误）。
- **结论：§1.32 ② 的机理判定与 ③ 的两道防线，都在硬件上被正向验证。**

1.33 ★★ ABZ 边沿抖动量化定案 + 一个"过速静默"监视盲区（2026-10-11，新增 sim/tb_abz_jitter.v，PASS 16 checks / 0 error；盲区当日已修 ⇒ 19 checks / 0 error，见 ⑥）

> 出处：用户 m02801 第④项"UVW/ABZ/BiSS/UART 遥测带着我过"里 ABZ 的"波形看起来乱"， 用户 m00598 选定"ABZ 边沿抖动复核（定量）"。目的：把"乱"变成可判读的数字， 并给出示波器要看的判据。

① 机理（为什么必然有抖动）

- `rtl/abz_gen.v:435` 的整个主时序块是 `end else if (in_valid)` 门控的 ⇒ 边沿、相位累加器、 Z 通道**只在 `in_valid` 拍上推进。**
- 生产链路里 `in_valid = filt_valid`： `rtl/enc_top.v:38` `LOG2DEC = 2`， `rtl/decim_avg.v:6` "每 2^LOG2DEC 个输入 valid 输出 1 拍 `dout_valid`"，`rtl/loop_filter.v:189-191` 的 `out_valid` 只是把 `in_valid` **打 3 拍、1:1 输出（不分频）** ⇒ `filt_valid` = 50 MHz / 4 = **12.5 MHz。**
- ⇒ 两条硬结论： 1. **AB 边沿只能落在 80 ns 的节拍栅格上**（1 节拍 = 4 clk = 80 ns）； 2. **每拍最多点火一次**（`rtl/abz_gen.v:487-499` 每拍只 ±1） ⇒ **AB 边沿率硬上限 = `filt_valid` = 12.5 MHz**，即 `inter*f_in/div ≤ 12.5 MHz` （A/B 周期频率 ≤ 3.125 MHz）。
- 理想均匀边沿间隔 = `1/(4*inter*f_in/div)`。板上实测配置 **inter=256 / div=1 / hyst=0 / ablimit=0 / zwidth=4**（`0x0101=0x0102=0x0103=0x0106=0`，`0x0108` 低字节=4）， `f_in ≈ 1 kHz` ⇒ 理想间隔 976.6 ns = **12.207 节拍（非整数）** ⇒ 实际只能在 12/13 节拍之间交替 ⇒ **峰峰抖动 = 1 节拍 = 80 ns（±40 ns，±4.1%）。**

② 量化结果（`sim/tb_abz_jitter.v`，隔离例化 `abz_gen`，理想角度斜坡 + 12.5 MHz 节拍）

相位	f_in	理想边沿率	边沿数（实测/理想）	平均间隔	间隔峰峰	与理想栅格最大偏差	limiter / lag
PH1 **生产**	1.000 kHz	1.024 MHz	10240 / 10240（10 个输入周期）	48.828 clk = 976.6 ns = 12.207 节拍	48…52 clk = **80 ns**	**78 ns（0.97 节拍）**	0 / 0
PH2 高频档	10.000 kHz	10.24 MHz	10239 / 10240（未丢沿）	97.7 ns = 1.221 节拍	4…8 clk = 80 ns	48 ns	0 / 0
PH3 过速	20.000 kHz	20.48 MHz	12500 / 20480（**丢 39%**）	80.0 ns = 1.000 节拍（钉死）	0 ns	389.6 µs（持续落后）	**当时 0 / 0（盲区，见④）⇒ 已修，见⑥**

- **PH1 明细**（`_sim_work/tb_abz_jitter/abz_edges_ph1.csv`，前 200 个边沿）：间隔是**固定 4:1 模式** `48,48,48,48,52,48,48,48,48,52,…`（= 12,12,12,12,13 节拍），误差锯齿在 0…78 ns 循环 ⇒ 抖动是 **确定性的节拍量化**，不是随机噪声。同窗口：A 上升-上升 195.24 clk（周期 3.905 µs ⇒ A/B 各 256 kHz ✓）、 A 占空 **50.02%**、A→B 与 B→A 相位间隔都是 48.83 clk = **1 个边沿间隔**（正交关系完好）、 Z 10 次/10 周期、Z 宽度 3.84 µs（v8 语义：`z_width=4` 边沿步 ⇒ 192 clk；★ v9 起同一寄存器值为 5 步 = 240 clk = 4.80 µs）。 ⚠️ **Z 宽度这一项只在"`z_window` 常开"下成立**（本测试台 `sim/tb_abz_jitter.v:140` 就是 `.z_window(1'b1)`）；集成路径上 v7 的 Z 脉宽实为 **2 clk = 40 ns**（被门控窗截断）， 已由 v8 修复，详见 §1.26 订正块与 §1.44。
- **PH2 是"乱"的真身**：10 kHz 档边沿数仍然精确（未丢沿），但理想间隔只剩 **1.221 节拍**， **1 节拍量化抖动就占 81.9%** —— 示波器上看到的"乱抖"就是这个，**不是毛刺、不是 RTL 缺陷**， 而是 `filt_valid = 12.5 MHz` 这个**配置**把量化相对量放大了。相位/占空仍正常。
- ⚠️ 与 §1.31/§1.32 治的**角度毛刺**是两回事：毛刺（5.6° 阈值 + `CONF_N=3` 确认）在**进环路前** 就被按住，密度 0.00444 次/输入周期，不会传到这里；本节的抖动是**下游输出级**的节拍量化。

③ 示波器判据（可照着量）

看什么	生产档（1 kHz）应为	说明
A（或 B）周期	**3.905 µs ±80 ns**	= 1/(inter*f_in/div)
A 高电平宽度	1.953 µs ±80 ns（占空 50%）	
相邻 AB 边沿间隔	**960 / 1040 ns 两档交替**（12/13 节拍）	峰峰 80 ns；**逐边沿交替是正常现象**
A→B / B→A 相位差	976.6 ns ±80 ns	偏离 ⇒ 正交损坏（当前配置不会）
Z 周期	1 次/输入周期，宽度 3.84 µs（★ v9 起 4.80 µs）	v8：`z_width=4` 边沿步；**v9：`z_width+1` = 5 步**；⚠️ **集成路径 v7 实测只有 40 ns**（窗截断），v8 起才成立（§1.44），v9 再补上 `+1`（§1.49）

④ ★ 本轮真发现：过速时"静默丢沿 + 无告警"（PH3）

- 20 kHz 需要 20.48 MHz 边沿 > 12.5 MHz 上限 ⇒ 实测边沿被**静默钳在 12.5 MHz** （每拍恰好一次点火，间隔恒 4 clk = 80 ns），1 ms 窗口内只出 12500 个边沿（理想 20480，**丢 39%**）。
- `lag_warn` / `lag_fatal` **全程为 0**： `pend` （`rtl/abz_gen.v:393-397`）只统计 "被 **ablimit 限频器**压住"的步数（`:487-499` 只在 `!hold_empty` 时才给 `r_slc_fwd/bwd` 加 1）， 而"**每 `in_valid` 拍最多点火一次**"这条上限根本不进限频器；多出来的步只堆在 `r_acc` 里。
- **更糟一层**： `r_acc` （`ACCW=13` 有符号）**没有饱和保护**（`:470-476` 直接 `r_acc <= acc_step_sum - acc_th`）。20 kHz 下每拍净增 `step_val - acc_th ≈ 1.6384 - 1 = 0.6384`， 约 **6400 拍（0.51 ms）**后越过 +4095 **回绕成负** ⇒ `acc_step_sum < 0` ⇒ `acc_fire_bwd` ⇒ **A/B 短暂倒转。**实测 1 ms 窗口内 **正向点火 10947 次 / 反向点火 1553 次**，而 lag 位仍全 0。
- ⇒ **实用约束**（已写进测试台输出，请照此配置使用）： 让 `inter*f_in/div ≤ filt_valid/2 = 6.25 MHz`（留一半余量）⇒ 本配置 inter=256/div=1 时 `f_in ≤ 24.4 kHz`；要在 10…20 kHz 上跑满 256 细分，就得把 `LOG2DEC` 从 2 降到 0 （`filt_valid` = 50 MHz、抖动节拍降到 20 ns，**需重综合**）或减小 `inter`。
- **可选 RTL 加固（本轮未做，留作 v7 候选，优先级 P2）**： ① `r_acc` 加减都做**饱和**； ② 把"累加器饱和/跟不上"也算进 lag 判据（例如饱和时置 `lag_warn`）。当前用户的 1 kHz 工况不受影响（PH1 全绿），故不阻塞上板。 ⇒ ✅ **2026-10-11 已做（v8 工作树），逐条实现与验证见 ⑥ 与 §3 item 16。**

⑤ 复现与自证

```
powershell -NoProfile -ExecutionPolicy Bypass -File tools\run_sim.ps1 -Only tb_abz_jitter
# -> tb_abz_jitter PASS 0 error（4 s）：**修复后 19 条断言**：
#    PH1 9 条（边沿数 / 栅格偏差 / 间隔峰峰 / A 高电平峰峰 / limiter / lag_warn / lag_fatal / Z / 无反向点火）
#    PH2 4 条（边沿数精确 / limiter / lag_warn / 无反向点火）
#    PH3 6 条（边沿数远低于理想 / 速率钉在 filt_valid / **lag_warn 必须出现** /
#             **lag_fatal 必须出现** / **无反向点火** / **limiter 必须不亮**）
# PH1 前 200 个边沿明细写 _sim_work/tb_abz_jitter/abz_edges_ph1.csv
```

> ★ PH3 原先第 3 条断言 `PH3-no-warn (KNOWN BLIND SPOT, see docs)` 是**故意编码当时已知行为**（不是期望特性），并写明"修好 lag 监视器后必须把它改成 `warn_hi > 0`"—— **2026-10-11 已照此改**（见 ⑥），断言数 16 → 19。 ★ `-Only` 运行会把 `sim/logs/SUMMARY.md` **覆盖成单台汇总**；要恢复 §1 顶部的全量结论必须重跑全量。 ★ 坑（本轮踩到）： `chk` 的 label 形参给 `[8*40-1:0]` 时，**超过 40 字符的标签会从左边被截掉**， 日志里出现 `H1-jitter…`、 `pinned to…` 这种半截标签（害得我以为断言没打出来）。 已改成 `[8*80-1:0]`； **以后新加的 chk label 保持 ASCII 且 ≤ 79 字符。**

⑥ ✅ 2026-10-11 已修（v8 工作树 / §3 item 16 ①+②）： 双向钳位 + 独立过速计数喂 lag；全回归 21/21

- **① 双向钳位**： `r_acc` 的合法区间本来就只是 `[0, acc_th-1]`（它保存的是"不够一个输出步"的**余量**， 正反向都落在这一段）⇒ 钳位后 13 位有符号数**不可能回绕成负** ⇒ `acc_fire_bwd` 反向点火**从机理上消失。** 实测 PH3 反向点火 **1553 → 0**（ `sim/logs/tb_abz_jitter.log` ）。
- **② 钳位即报（独立计数）**：钳位条件与"本拍至少丢 1 个完整输出步"**等价** （正向 `acc_rem_fwd ≥ acc_th`；反向排空后仍欠 `≥ acc_th` ）⇒ 新增**无方向** 13 位计数 `r_ovl` （饱和在 4095；每拍最多 +1 = **下界**，只偏保守、不漏报；钳位拍**不与**同拍执行相抵）， `lag_mag = |r_slc_fwd − r_slc_bwd| + r_ovl` ，再乘 `acc_th` 与阈值比较。 PH3（窗口 1 ms = 50000 clk，20 kHz 输入）：① 本项修复当时阈值还是旧的 `lag_warn_th = 2*inter` / `lag_fatal_th = 4*inter` ⇒ `lag_warn` **47601 clk** / `lag_fatal` **40205 clk**； ② **当天晚些时候按 ⑦ 重标定为 45°/180°（ `inter/2` 与 `2*inter` ）后重跑 ⇒ `lag_warn` 50000 clk（整窗） / `lag_fatal`** 47601 clk***—— 新 `lag_fatal` 计数正好等于旧 `lag_warn` 计数，因为 `4*inter` → `2*inter` 恰好是 2×， 这条*内部一致性**反过来验证了重标定只动了阈值、没动计数逻辑。两种阈值下**盲区都闭合。**
- **③ 为什么不把过速 backlog 并进 `limiter_active` （这条是实测逼出来的，不是设计洁癖）**： 第一版把钳位并进 `r_slc_fwd/r_slc_bwd` ⇒ `tb_enc_top` **`B8-limiter-within-spec` 失败** （ `limiter_hi=4194304 clk` = 整个机械圈窗口； `rtl/abz_gen.v` 尾部 `limiter_active <= (!hold_empty) || (pend_mag != PEND_ZERO)` ）。 **A/B 对照证明是本次改动引入**： `git checkout -- rtl/abz_gen.v` 后的 v7 原版，同一测试台 **PASS 且 `limiter_hi=0 clk`**；逐拍探针显示正向窗口**开始之前** `slc_b` 就已是 4， 而它唯一的衰减路径是"真的执行一个**反向**边沿" ⇒ 纯正向运行里**永不衰减** （v7 就已存在的结构陷阱，只是当时钳位不进计数器 + `ablimit=0` 时不会有点火被压住，所以碰不到）。 ⇒ 定案： `r_slc_*` 恢复 v7 语义（只统计被 `ablimit` 压住的点火）、 **`limiter_active` 语义不变** （ `rtl/SPEC_interface_contract.md:372` 的"限频保持/补发"契约）， 过速证据单独走 `r_ovl` ；并在 `sim/tb_abz_jitter.v` 新增 `PH3-limiter NOT active (overload only sets lag)` 把这条边界**钉死在测试里**（防后人又合并）。
- **验证**： `tools\lint_all.ps1 -Clean` = **ERROR 0 / WARNING 0**； 全回归（探针已撤）**PASS 21/21、exit 0** @ `2026-10-11 17:09:21` （ `sim/logs/SUMMARY.md` ）； `tb_abz_jitter` **19 ok / 0 fail**（PH3 6 条全 ok）； `tb_enc_top` `limiter_hi=0 clk`、 `[ok] B8-limiter-within-spec` 。
- **⚠ 尚未重综合**：本节改动**只存在于工作树，v7 位流（§2.8）不含**此项 ⇒ 用新位流上板前必须重跑综合签核（§3 item 16 已按此记）。
- **诚实残留（本轮不修）**： `r_slc_fwd/r_slc_bwd` 的"**单向衰减**"仍是结构缺陷 —— `ablimit>0` 且发生方向反转（或某方向被长期压住）时，残留计数会把 `limiter_active` （以及 lag 的 `|Δ|` 项）**粘住**。修法可选"任一方向执行边沿都能衰减另一方向"或给 `r_slc_*` 加泄漏， 列入 v8 后续候选（**不阻塞**当前 1 kHz 生产工况）。

⑦ ★★ 顺手查出的另一处不一致（ ✅ 2026-10-11 当天已按 (b) 重标定，见 §3 item 18）： `lag_warn/lag_fatal` 的标称角度与实现差 4×

- **原标称**（ `rtl/abz_gen.v:47-51` 、 `model/abz_uvw_model.py:36-38` ）： `45° → pend*div >= 2*inter` 、 `180° → pend*div >= 4*inter` 。
- **几何**：一个输入周期 = `4*inter` 个 AB 边沿步（RTL 头注释与 `rtl/SPEC_interface_contract.md:355` `edges_per_cycle = 4*inter` 、 `model/abz_uvw_model.py:139` `n_step = 4*inter` 三处一致） ⇒ `pend*div` 的单位就是 idx 步， `2*inter` 步 = **半个输入周期 = 180°**、 `4*inter` 步 = **360°。**
- **实测证据（旧实现， `python tools/lag_thr_probe.py` ，已入库；用模型 `AbzGen` 开大 `ablimit` 让限频器压住边沿，同时独立累加**去卷绕的真实滞后）—— inter=256 / div=1： | 角度斜率 | 首次 `lag_warn` | 首次 `lag_fatal` | |---|---|---| | 64 LSB/拍（≈0.996 idx 步/拍）| pending 512，**真实滞后 514 idx 步 = 180.7°** | pending 1024，**真实滞后 1028 步 = 361.4°** | | 256 LSB/拍 | pending 512，真实滞后 **2065 步 = 726°** | pending 1024，真实滞后 **4127 步 = 1451°** | | 1024 LSB/拍 | pending 512，真实滞后 **8269 步 = 2907°** | pending 1024，真实滞后 **16523 步 = 5809°** | ⇒ ① **标称 45°/180° 与旧实现的 180°/361° 差 4×**（慢速工况实测，不是推算）； ② 角度跑得越快，同一 pending 阈值对应的**真实**滞后越大 —— 因为 `r_slc_*` / `r_ovl` 都是 "**每拍最多 +1**"（ `rtl/abz_gen.v:487-499` 每拍至多点火一次），而真实滞后按"每拍多出的步数"累积 ⇒ 这个监视器只能当"**已经严重跟不上**"的粗警报，**不能**当精确的角度滞后指示。
- ✅ **定案与实现（2026-10-11，用户选 (b)）**：
- `rtl/abz_gen.v` ： `LAG_WARN_NUM=2` / `LAG_FATAL_NUM=4` → **`LAG_WARN_DIV=2`** （ `lag_warn_th = (inter+1)>>1` = `inter/2` idx 步 = **45°**，奇数 inter 向上取整）与 **`LAG_FATAL_MUL=2`** （ `lag_fatal_th = 2*inter` = **180°**）；比较点与 `lag_mag` 定义未动。
- `model/abz_uvw_model.py` ：常量同步为 `LAG_WARN_DIV` / `LAG_FATAL_MUL` ，判据改为 `lag_mag*div >= (inter+1)//2` / `>= LAG_FATAL_MUL*inter` ； **并补齐 item 16 的钳位与 `r_ovl`** （见下条"模型保真度"）。
- `rtl/SPEC_interface_contract.md` 行为 4b 与 `tools/lag_thr_probe.py` 同步。
- **重标定后复测（同一探针，inter=256/div=1）**： `64 LSB/拍` → warn = `lag_mag 128` 、 **真实滞后 128 idx 步 = 45.0°**，fatal = `lag_mag 512` 、**514 步 = 180.7°**（与标称一致 ✓）； `256 LSB/拍` → warn 91.1° / fatal 362.1°； `1024 LSB/拍` → warn 365.3° / fatal 1450.5° ⇒ 阈值数值正确，但"下界/粗警报"这条性质依旧（角度越快报得越晚），文档已如实写明。
- **影响面**： `lag_fatal` 进 **BiSS E 位** （ `rtl/enc_top.v:730` `.err_bit (lag_fatal | cor_sat)` ）、 `lag_warn` 进制器 `0x053A` bit2 `laglim` （ `rtl/enc_top.v:756` ）、 `led[2]` 与 `irq_n` （ `:1123` / `:1126` ）； 重标定后这些告警的灵敏度恢复为标称的 45°/180°（旧实现要到**丢掉快一整圈**才亮）。
- ⚠ 与 item 16 一样：**本次改动尚未重综合**，v7 位流（§2.8）不含，需与 item 16 同一轮重综合签核。
- **模型保真度（同批修）**： `model/abz_uvw_model.py` 原先**只有 v7 语义**（无双向钳位、无 `r_ovl` ）， 过速工况下与 RTL 分叉（其自检与表 5 仍会 PASS，因为那是打印式自检）⇒ 本次一并补上 `self.ovl` + 钳位 + `lag_mag = |pend| + ovl` ，恢复"逐位一致"（UVW 侧既有漂移见 `docs/05_vivado_sim_impl_report.md:902` 第 5 条，本轮不动）。

1.34 ★★ BiSS-C 哑主机联调：手册成文 + 三个"照着直觉接就废掉"的坑（2026-10-11，新增 `docs/08_bissc_dumb_host.md` ； RTL 未改）

背景：用户 m00838 要求把 m02801 第④项的最后一节（BiSS-C）"带着过"：用 AFG-2225 当哑主机，读 `0x053A` bit6(`crcerr`)/bit7(`frame_active`) 与 `0x053B`（16 位帧内已发位数）。本节的每一条都是逐行读 RTL 得到的，不是推断。

① 帧几何真相（与实测一致） - `rtl/enc_top.v:104-105`：`localparam integer ST_BITS_L = 22;`（= `POLE_BITS`(6) + `AW`(16)）、`MT_BITS_L = 10`；`:716-741` 例化 `biss_c_slave #(.ST_BITS(ST_BITS_L), .MT_BITS(MT_BITS_L), .TIMEOUT_CYCLES(10))`。 - ⇒ 帧长 = `1 + 22 + 10 + 2 + 6` = **41 位**：起始位(0) → ST(MSB 先) → MT(MSB 先) → E(`lag_fatal|cor_sat`) → W(`amp_low|amp_high|limiter_active`) → CRC6(MSB 先)。 - **已有实测印证**：`docs/05_vivado_sim_impl_report.md:175` 的 `[TEST E] BiSS: … frame_total_bits=41, bit_pos=36`。 - CRC6 = `x^6+x+1`、初值 0、**起始位不计入**（`rtl/biss_c_slave.v:140-146/:315/:325-330`）；实测向量见 `sim/tb_biss_c_slave.v` 头注释（帧级 `ST=0x2A5A19C,MT=0xABC,E=0,W=0 → 0x34`）。

② 三个坑（哑主机侧，全部会让人误判成"从站坏了"） 1. **`biss_sl`(`J5-12`)必须驱动，悬空必废**：XDC 里只有 `ad_ch1_otr/ad_ch2_otr` 有 `PULLTYPE PULLDOWN`（`constr/enc_top.xdc:132`），`biss_ma/biss_sl` 没有上下拉（`:299` 只给了 `set_false_path`）。而 `idle_hit` 判据是 **`ma` 与 `sl` 同时为高 ≥ `TIMEOUT_CYCLES`=10 拍 = 100 ns**（`rtl/biss_c_slave.v:340-344`）⇒ `sl` 悬空 + `ma` 1 MHz（高 500 ns）会让帧**在半路被反复复位**，`frame_cnt` 只数到 1~2；更坑的是 `frame_short_err` 是**粘滞但 SPI 读不到**的（`rtl/biss_c_slave.v:114` 定义，`rtl/enc_top.v:737` 只接到 `unused_ok` 汇聚）⇒ 没有任何读数能直接指出它。 2. **`0x053B/0x053C` 不是帧序号，是"本帧已发位数"**：`ma_fall` 时 `frame_cnt <= frame_cnt+1`（新帧起点置 1，`:317/:324`），而在 `idle_hit||done_gap` 收尾分支里被**清零**（`:349 frame_cnt <= 16'd0;`）⇒ **打完一串时钟再读必然读到 0**（这是"帧正常收尾"的证据，不是故障）。要读到 1…41，只能在帧进行中读，或让帧卡住不复位。 3. **`sl` 常低时，帧收尾只靠 `done_gap` ⇒ `ma` ≤ 2.5 MHz**：`done_gap = frame_on && (bit_pos == TOTAL) && (ma_hi_cnt >= TIMEOUT_C)`（`rtl/biss_c_slave.v:340`）⇒ 最后一拍之后 `ma` 必须保持高 ≥100 ns（实际留 200 ns 给 10 拍识别）。`ma` =1 MHz 时高电平 500 ns，余量 2.5×；**`ma` =10 MHz（高 50 ns）则帧永远收不了尾**：卡在 `bit_pos = TOTAL`、`frame_cnt` 恒 **0x0029(41)**、第 42 个下降沿走进 `:334-338` 的 else 分支 ⇒ `bit_err`/`crc_err_seen` **粘滞置位** ⇒ `0x053A` bit6=1（并且 `biss_crc_err` 进 IRQ，`rtl/enc_top.v:1124`）。 ⇒ 这同时给出**一个受控实验**：10 MHz = 故意越界，用来证明 bit6 通路可用（清法是复位/重下载，**没有软清位**；`0x0405` 是 ADC 的 OTR 软清，与 BiSS 无关）。

③ 三种探针配置（写进了手册 §3）

配置	预期
A（推荐） `sl`=GND、`ma`=1 MHz 方波	干净循环：`0x053B` 多次读落在 1…41 且**值不同**、`0x053A` bit7=1、**bit6=0**（主判据）；关掉 `ma` 后 `0x053B` 归 0（印证坑 2）
B（受控违例） `sl`=GND、`ma`=10 MHz 方波	`0x053B` 恒 = 0x0029、bit6 粘滞 = 1、`irq_n` 拉低（若 ADC 侧无其他告警）
C（反例） `sl` 悬空/由 CH2 方波驱动、`ma`=1 MHz	`frame_cnt` 只到 1~2、bit7 抖 ⇒ 说明为什么"用第二路方波当 `sl`"这种直觉接法必错（真主机在 `sl` 高电平期间是**停 `ma` 时钟**的，AFG 两通道不会自动做这件事）

④ 能力边界（如实记录） - 本版 ILA 探针表**没有 BiSS 网**：`rtl/enc_top.v` 的 `(* mark_debug *)` 共 37 处，覆盖 `dbg_s_raw/c_raw/adc_valid/otr_lat/s_dec/c_dec/dec_valid/s_cor/c_cor/cor_valid/cor_sat/angle_raw/cordic_mag/cordic_valid/ang_glitch/ang_delta/angle/cycle_` **无 `dbg_biss_*`** ⇒ 片内看不到 `ma/sl/slo`，联调只能用示波器/外接逻辑分析仪（要片内解帧需加 3 个探针并重编 ILA 位流）。 - 手册 §6/§7 给了示波器三个必看点（`ma` 波形钉死 → `ma`/`slo` 同屏看 41 位格 + 20…30 ns 更新延迟 → 帧重复性与角度相关性）与逻辑分析仪的离线解码规则（位序 + CRC6 递推 + 已实测向量）。

⑤ 顺手核到的一处 RTL/文档不一致（未改，待定夺） - `rtl/reg_file.v:86-87` 文档表 + `:286-287` 复位值把 `0x0500 ST_BITS`=26、`0x0501 MT_BITS`=12（`:463-464` 可读、`:572-573` 输出 `cfg_st_bits/cfg_mt_bits`），与实参 **22/10** 矛盾（26/12 会推出 47 位帧）。★ 但这两个信号**没有任何真实读者**（只进 `rtl/enc_top.v:1151` 的 `unused_ok` 汇聚）⇒ 属"读到会误导"，修它要动 RTL、两版位流都得重综合，**本轮不动。** - 次要不一致：`docs/01_technical_solution.md:623-624` 仍把寄存器写成 `0x500B BISS_ST / 0x500C BISS_MT`（真实为 `0x053B/0x053C`，只读）。

⑥ 同步更新的其它文档：`docs/07_board_checklist.md` 第 3.4 行加了指向手册的导引；第 4 节新增 **4.15**（把上面三个坑写进"已知坑"表）。

1.35 ★★ BiSS-C 三个探针（A 正常 / C 反例 / B 受控违例）上板全部实测通过 + 一个"掉电窗口假故障"教训（2026-10-11，RTL 未改，新增 `tools/biss_frame_probe.py`）

背景：用户选"先跑探针 A"（m01121）；接线后按台面流程实测。

① 先补的第二个工具：`tools/biss_frame_probe.py`（一键自动判读，三探针版） - 连续读 `0x053B`（`read 0x053B` **一次原子取回 16 位**帧内已发位数：应答 `5A|status|addr|reg[addr+1]|reg[addr]|chk`）+ `0x053A`，每样本打印 `frame_cnt / status / frame_active / crcerr`，并按纯函数 `verdict(samples, probe=)` **按探针给出各自的通过条件**：- `--probe A`：**A1 无 >41 / A2 覆盖锯齿（distinct≥4 且 max==41）/ A3 `crcerr` 恒 0 / A4 `frame_active` 占多数**；- `--probe B`：**B1 `frame_cnt` 全 = 41 / B2 `frame_active` 恒 1 / B3 `crcerr` 首次置位后不再回 0（粘滞语义用 `first_crc` + 尾段全 1 判定）**；- `--probe C`：**C1 数不到末位（max ≤ 2）/ C2 `frame_active` 在抖（0 < 计数 < n）/ C3 `crcerr` 仍为 0。** 另有与探针无关的定向诊断（恒 =41 ⇒ `done_gap` 不成立 / 只到 1…3 ⇒ `sl` 没真正拉低 / `crcerr` 粘滞 ⇒ 末位后仍打时钟）与各探针的结论行。CLI：`--port --baud --probe --n --interval --timeout --csv`；`--selftest` = **14 项全过**（新增 B/C 正反例，纯逻辑、无硬件，两种 python 都验过）。 - 复用了 `tools/uart_spi.py` 的 `open_port` / `Link` / `_readback(link, addr, timeout)`；两个坑：`chk()` 的返回值必须 `bool()` 再 `&=`（否则 `TypeError`），中文输出需 `$env:PYTHONIOENCODING="utf-8"`。 - `docs/08_bissc_dumb_host.md` §4 因此改为**推荐一键脚本**形态（三条命令按 A/C/B 列出、保留手工 `1..30` 循环版），端口示例改为**实测口号 COM6**；手册同时写明顺序约束（C → B，B 会污染 bit6）。

② 上板实测结果（v6 生产位流 `enc_top.bit`，`0x05F2` =0x0006 已核实） - 配置：AFG-2225 CH1 → `J5-11`(`biss_ma`) 1.000000 MHz 方波、3.3 Vpp、DC OFFSET 1.65 V、`UTIL→CH1→LoadPhase=High Z`、连续（非 Burst）；`J5-12`(`biss_sl`) 直接接 GND；AFG 地与载板共地；**`J5-13`(`biss_slo`) 是 FPGA 输出，不接。** ADC 两路**不需要任何输入**（时序只由 `ma` 边沿驱动，输入只影响帧内容）。 - 读数：`0x05F0`=0x100C（DEVICE_ID=0x0C/REV=0x10）、`0x05F2`=0x0006、`0x053A`=**0x82**（bit1 `amplo` + **bit7 `frame_active`**；**bit6 `crcerr`=0**）、`0x053B` 在 11/40/34/2/33… 间跳动。 - `tools\biss_frame_probe.py --port COM6 --n 40 --csv _synth_work\biss_probeA.csv` ⇒ **A1/A2/A3/A4 全 PASS**：40/40 `frame_active`=1、**40/40 `crcerr`=0**、`frame_cnt` 命中 27 个不同值（distinct=26 计法见工具输出）且 **max 恰好 = 41、无一 >41**；一次采样 0.23 s（单次往返 ≈5.8 ms）。 - ⇒ **一次同时证明**：① 从站在 `ma` 每个边沿逐位推进（`frame_cnt` = 本帧已发位数）；② 帧到 41 位能**自动收尾重开**（`sl` 常低 ⇒ 靠 `done_gap`，1 MHz 高电平 500 ns ≥ 100 ns，余量 2.5×），所以随机取样能覆盖整条锯齿；③ 帧尾 CRC6 与位序**无违例**。手册 §5/§10 已写入实测值与 CSV 路径。

③ ★ 一个必须记住的假故障：掉电窗口里"全部读数超时、但 COM6 还在" - 现象：接线途中板子被断电，此后**所有**读数超时（`!! 超时：没收到应答`，且**"超时期间收到 0 字节"**），而 `tools\uart_spi.py ports` 仍列出 `COM6 USB-SERIAL CH340` ⇒ 看起来像"FPGA 被 MA 过压打坏 / 串口芯片坏了"。 - 真因：**CH340 由 PC 侧 USB 供电**（照旧枚举），而 **FPGA 未配置**（掉电窗口）⇒ 全哑。DONE 灯亮、AFG 设置也确认为 `High Z` 之后**重试即恢复**，`0x05F2` 仍是 6、无需重烧。 - ⇒ 排查顺序固化为：**先看 DONE 灯 → 再关掉 AFG 输出并量 `J5-11` 静态电平（应 ≤3.3 V；AFG 若按 50 Ω 设定接高阻负载会翻倍到 ~6.6 Vpp = −1.65…+4.95 V，超出 Bank15 3.3 V 轨）→ 复查接线 → 最后才重烧位流。**这条已写进手册 §9 失败速查表。 - 顺带核实台面判活指示（`rtl/enc_top.v:1119-1124`、`constr/enc_top.xdc:304-323`）：`led[0]=otr_lat`(**AB21**)、`led[1]=amp_low|amp_high`(**AB22**)、`led[2]=limiter_active`(J21=J5-18，**板上无 LED**)、`led[3]=calib_busy`(K21=J5-19，板上无 LED)、`irq_n=~(otr_lat|amp_low|amp_high|lag_limit|lag_fatal|biss_crc_err)`(**K22=J5-20**，低有效)。

④ 探针 C 也实测通过（2026-10-11，同一 v6 位流；把 `J5-12` 的 GND 夹子取下、保持悬空） - `tools\biss_frame_probe.py --port COM6 --n 20` ⇒ **`frame_cnt` 只取 {0,1}（distinct=2，max=1）、`frame_active` 12/20 为 1（在抖）、`crcerr` 20/20 = 0**，CSV = `_synth_work\biss_probeC.csv`；单次采样耗时由 5.8 ms 涨到 **51 ms**（帧频繁被掐死重起 + 偶发重发）。 - ⇒ **完全按预测出现**：`sl` 悬空被当高电平 ⇒ `idle_hit`（`ma` 与 `sl` 同时为高 ≥100 ns）让 1 MHz 的每个高电平都把刚起的帧**半路复位**，所以 `frame_cnt` 永远只能数到 1。★ 此时 `crcerr` **仍是 0** —— 半路复位置的是 **`frame_short_err`（粘滞但 SPI 读不到）**，不是

`crc_err_seen`，这正是"这个坑没有任何读数能直接指出"的直接证据（只能靠"数不到 41"间接判定）。 - 手册 §5 的 C1 行与 §10 记录表（新增方案 C 列）已回写实测值。

⑤ 探针 B（受控违例）也实测通过 —— 三个探针全部收尾（2026-10-11，同一 v6 位流；`J5-12` 接回 GND、AFG 改 10 MHz） - `tools\biss_frame_probe.py --port COM6 --probe B --n 20 --csv _synth_work\biss_probeB.csv` ⇒ **B1 `frame_cnt` 全 = 41（20/20）、B2 `frame_active` 恒 = 1（20/20）、B3 `crcerr` 首次置位后不再回 0（粘滞）**； `0x053A` 低字节恒 = **0xC2**（`amplo` + `crcerr` + `frame_active`），单次采样 28.4 ms。 - ⇒ 机理与预期逐条吻合： `ma` 高电平仅 50 ns < 100 ns ⇒ `done_gap` 永不成立 ⇒ 帧卡在末位；第 42 个下降沿落入"CRC 后继续打时钟"分支 ⇒ `crc_err_seen` / `bit_err` 粘滞。**这一步的意义在于它是唯一能把 `0x053A` bit6 点亮的实验**（A/C 里它恒 0）⇒ 证明该位不是死位、通路可用；同时 `biss_crc_err` 进 IRQ（`rtl/enc_top.v:1124`）⇒ 此时 `irq_n`（`J5-20`/K22) 应为低 —— **2026-10-11 实测确认：趁 `crcerr` 还粘滞时示波器量 `irq_n` = 低电平**（用户 m01330 报告）⇒ 该 IRQ 通路**实测打通。** - ⇒ 由 `TIMEOUT_CYCLES=10`（=100 ns）反推**实际可用上限**： `sl` 常低时 `ma` **≤ 2.5 MHz** 才安全（1 MHz 留 2.5× 余量）。 - **清位**：无软清位（`0x0405` 是 ADC 的 OTR 软清，与 BiSS 无关）⇒ 只能**重新上电或重下载位流**；下次再跑 A/C 前必须先清，否则 bit6 读数被污染（顺序约束：C → B）。 - ⇒ **m02801 第④项（UVW / ABZ / BiSS / UART 遥测）至此全部收尾**：ABZ 抖动量化定案见 §1.33，BiSS 三探针定案见本节。手册 §5 的 B1 行与 §10 记录表的方案 B 列已回写实测值。

⑥ 剩余可选动作（都不阻塞）：① 用示波器补拍手册 §6 的三张图（`ma` 波形、 `ma` / `slo` 同屏的 41 位格、帧重复性）；② ~~B 之后做一次重新上电，跑 `--probe A` 复现"清位后 `crcerr` 回到 0"~~ —— ✅ **2026-10-11 完成（闭环）**：AFG 改回 1 MHz 后重新上电 ⇒ `0x053A` = **0x0D82**（`caldone` + `amplo` + `frame_active`，**bit6 `crcerr` = 0**）、 `--probe A --n 40` **A1–A4 全 PASS**（`crcerr` **0/40**、 `frame_active` 40/40、 `distinct=20`、 `max=41`），CSV = `_synth_work/biss_probeA_after_clear.csv` ⇒ 粘滞位"只能靠重新上电清、清完好路一起复原"这一点也实测闭环（ `status` 字节回到 `0x82`）。**★ 顺序坑（2026-10-11 实测踩到）**：**必须先把 AFG 改回 1 MHz，再重新上电** —— AFG 若仍停在 10 MHz，上电瞬间 `crcerr` 就在毫秒级被重新点着，过几秒去读仍是 `0xC2` + `0x053B` 恒 41，看起来像"重新上电没清位"（判别：清位后若 `frame_cnt` 仍恒 = 41 ⇒ MA 还是过速档，不是没清）；③ 若要在片内解帧，需给 ILA 加 `dbg_biss_*` 探针并重编 ILA 位流（见 §3 候选项）； ④ ✅ **2026-10-11：A1/A2 已在 v7 生产位流上复测通过** —— `tools\biss_frame_probe.py --port COM6 --probe A --n 80` ⇒ **A1–A4 全 PASS**（`frame_active` **80/80**、 `crcerr` **0/80**、 `frame_cnt` **distinct=36 / max=41（5/80 落在 41）/ 无一个 >41**、 `0x053A` 低字节恒 `0x82`），CSV = `_synth_work\biss_probeA_v7.csv`； A2 负对照（AFG 未接通、 `ma` 无时钟）⇒ **`0x053A` = 0x0008（仅 `otr`）、 `0x053B` = 0x0000**，证据 `_synth_work\biss_v7_a2_noclock.txt`。**★ 复测第一次是失败的**（`--n 40`： `frame_cnt` 只取 {0,1}、max=1、 `frame_active` 27/40 = **C1 签名**）⇒ 排查为 **`J5-12` 接线没到位**，复查接地后**同一位流、同一份 RTL** 全 PASS；这既证明"换 v7 位流"不是变量（`rtl/biss_c_slave.v` 自 `9c0a28e` 到 HEAD 逐字节未变、 `rtl/enc_top.v` 只差注释与 `RTL_BUILD_ID`），也证明该类失败可判可归因。**B/C 未在 v7 复测**（B 的粘滞 `crcerr` 需重下位流清位）。

1.36 ★★★ v6 ILA 版首次上板打通：一键采集脚本 + 两个"环境类"坑（2026-10-11，RTL 未改，新增 `tools/ila_capture.tcl`、`_synth_work/hw_scan.tcl`）

背景：用户 m00838 后半句"重建 v6 ILA 版"建成（§2 的 14:06:09 条目）后，本条目记录**上板。**

① JTAG 扫描（`_synth_work/hw_scan.tcl`） - `open_hw_manager` → `connect_hw_server` → 遍历 `get_hw_targets` / `get_hw_devices`，输出带 `HWSCAN` 前缀。 - 实测： `hw_target = localhost:3121/xilinx_tcf/Digilent/210512180081`、 `device = xc7a35t_0`、 `PART = xc7a35t`、 `IDCODE = 0x0362D093`。★ 连接前必须先关掉 Vivado Hardware Manager 图形界面（否则 cable 被占）。

② 坑 A（环境）： `cs_server` 要往 `%TEMP%` 解包 DLL，而沙箱只允许写工作区 - 报错原文： `ERROR: [Labtools 27-2220] Launch Error: Unable to launch local cs_server executable. + ERROR] Failed to extract VCRUNTIME140.dll: failed to open target file! fopen: Permission denied`。 - 真因： `cs_server` 是 PyInstaller 单文件程序，运行时把 `VCRUNTIME140.dll` 解包到 `%TEMP%`。 - 修法： `tools/run_vivado.ps1` 新增 **2b 段**，把 `$env:TEMP/$env:TMP` 钉到 `$root\.tmp`、 `$env:LOCALAPPDATA` 钉到 `$root\.appdata`（文件头注释加了第 5 条说明）。★ **该流程仍需一次性 `danger-full-access`； synth / impl / bitgen 完全不受影响**，所以这个症状极具误导性（看起来像位流或 Tcl 的问题）。

③ 坑 B（会复发）： `.ps1` 一律 UTF-8 带 BOM，改完必须补 - 用编辑工具改 `tools/run_vivado.ps1` 后它变成 **UTF-8 无 BOM** ⇒ PowerShell 5.1 按 ANSI/GBK 解码 ⇒ 中文注释乱码、偶发吞换行 ⇒ 报 `Unexpected token '}' in expression or statement`，且**报出的行号/行内容与文件实际内容对不上**（报"行 65 是 `} else {`"，实际行 65 是别的）。 - 修法/自检：以 `New-Object System.Text.UTF8Encoding($true)` 重写；验证 = 前 3 字节须为 `EF BB BF`，且 `[System.Management.Automation.Language.Parser]::ParseFile(<path>,[ref]$null,[ref]$errs)` 的 `$errs.Count` 必须为 0。★ 仓库里 `tools/lint_all.ps1`、 `tools/rerun_impl.ps1`、 `tools/run_sim.ps1` **原本就带 BOM**，这就是既定约定。

④ 一键脚本 `tools/ila_capture.tcl`（下载 → 载探针 → 触发 → 导出 CSV） - 环境变量： `ILA_MODE=stage1|stage2|restore`（默认 `stage2`）、 `ILA_BIT`（默认 `_synth_work/enc_top_ila.bit`）、 `ILA_LTX`（默认 `_synth_work/enc_top_ila.ltx`）、 `ILA_CSV`、 `ILA_DEPTH`（默认 1024）、 `ILA_WAIT`（默认 60000 ms）；日志带 `ILACAP` 前缀。 - **stage1 触发在 `dbg_adc_valid` 上（等价"立即触发"，用来验证链路）；stage2 触发在 `dbg_ang_glitch == 1` 上（抓真毛刺，需要 ADC 上有真实正余弦输入）**； `restore` 只 `program_hw_devices`、不碰探针，把生产 v6 位流放回（`tools/ila_capture.tcl:60-68`）。 - 三个 Vivado 细节（都踩过、都已修）：① **光设 `PROBES.FILE` 不够**，必须同时设 `FULL_PROBES.FILE` 且下载后 `refresh_hw_device $dev`（**不要**加 `-update_hw_probes false`），否则 `get_hw_probes` 返回**空表**并报 `WARNING: [Labtoolstcl 44-131] No matching hw_probes were found`（看着像"找不到触发探针"，其实位流已下好）；② **`CONTROL.TRIGGER_STATUS` 不存在**（读出空串 ⇒ 轮询永不成立 ⇒ 干等到超时，而数据其实早采到了）； ③ 真正可用的是 **`STATUS.CORE_STATUS`，实测取值 `IDLE` / `RUNNING` / `FULL`**（FULL = 采样存储已满 ⇒ 可上传）； `STATUS.TRIGGER_STATUS` 在本核上也读空串，别依赖。已改为接受 `*TRIGGERED*` / `*IDLE*` / `*FULL*`，并在前面加 3 s"确认真的武装/采集中"预检查（`tools/ila_capture.tcl:103-134`）。

⑤ stage1 实测（v6 ILA 位流首次上板，全链路通） - `$env:ILA_MODE="stage1"; $env:ILA_CSV="_synth_work\ila_cap_stage1.csv"; $env:ILA_WAIT="8000"` ⇒ 日志 `_synth_work/ila_capture_stage1d.log`： `End of startup status: HIGH`、 `Device xc7a35t is programmed with a design that has 1 ILA core(s)`、 `ila=hw_ila_1`、 `trigger probe=dbg_adc_valid`、 `TRIGGER_COMPARE_VALUE = eq1'b1`、 `The ILA core 'hw_ila_1' triggered at 2026-Oct-11 14:57:22` ⇒ **CSV `_synth_work\ila_cap_stage1.csv`（134,738 B）**。 - 分析： `tools/ila_report.py <csv>`（用运行时 python， `PYTHONIOENCODING=utf-8`）。★ **别把它的输出管进 `Select-Object -First N`** —— 管道提前关闭会让脚本 broken pipe 而 **exit code 1**（虚警，不加管子 exit=0，§[7] 逐条探针表能完整打印）。 - 结论：**位序 = 直读正确**（位反版 `hypot` std 330.34 vs 直读 1.00）； `dec_valid` / `cor_valid` / `cordic_valid` 各 256 脉冲/1024 = **1/4.00**（`LOG2DEC=2` ✓）； `TRIGGER` 落在第 512 拍（窗口 1024，与 `TRIGGER_POSITION=DEPTH/2` 一致）；当时 ADC 上**没有真实信号**（r = 19.6 ± 1.0 LSB ⇒ 0.096 Vpp 浮空小信号）、 `dbg_ang_glitch` 全程 0。

1.37 ★★★ v6 ILA stage2 抓到真毛刺：1 个 ADC 坏样本 → 角度 +9.66° → 环路仍外泄 +7.3°（2026-10-11，RTL 未改）

背景：用户按 `8 Vpp / 1 kHz / 相差 90°` 接好 ADC 正余弦输入后跑 stage2。

① 采集： `$env:ILA_MODE="stage2"; $env:ILA_WAIT="60000"; $env:ILA_CSV="_synth_work\ila_cap_stage2.csv"` ⇒ 日志 `_synth_work/ila_capture_stage2.log`： `trigger probe=dbg_ang_glitch`、 `TRIGGER_COMPARE_VALUE = eq1'b1`、 **`CORE_STATUS=FULL fired=1 elapsed=1 ms`**、 `The ILA core 'hw_ila_1' triggered at 2026-Oct-11 15:02:42`； CSV = `_synth_work\ila_cap_stage2.csv`（134,738 B）。**CSV 里 `dbg_ang_glitch` =1 恰好落在第 512 拍（= TRIGGER 行）**，共 2 次翻转（单个 4 拍宽脉冲）⇒ 触发不是空的。

② **输入质量（同时验证模拟前端标定）**：`f_in = 0.982 kHz`、半径 `r = 1623.8 ± 22.0 LSB` ⇒ **等效 7.929 Vpp**（命令 8 Vpp ✓ 交叉验证）、`s ∈ [-1571, 768]`（**768 就是那个坏样本**）、`c ∈ [416, 615]`；位序=直读正确（直读 `hypot` std 21.99 vs 位反版 562.92）。

③ **★★ 毛刺全链路（逐级拍号 + 幅度，均实测）** | 级 | 拍号 | 现象 | |---|---|---| | ADC 原始 | 488 | `s_raw` 相对 9 点中值偏 **+2307 LSB（单点**；`c_raw` 干净） | | 抽取 `decim_avg` | **490–493** | `s_dec` **+9228 LSB = 4×2307**（恰好一个抽取周期 4 clk） | | 校正 | 493–496 | `s_cor` +9228 | | 幅度监视 | 495–499 | `stat_byte` 0x01 → **0x21**（bit5 `scamp = amp_low|amp_high`） | | CORDIC | **512–515** | `cordic_mag` **−8501**、`angle_raw` **+1759 LSB = 9.66°** | - **延迟账目完全吻合**：488→490（抽取 2 clk）→493（校正 3 clk）→512（CORDIC **+19 clk = NITER 17 + 2** ✓）。 - **检测器行为正确**：阈值 `|Δangle_raw| ≥ 5.6°`（1024 LSB），+1759 命中；第 516 拍回来的 **−1764 大回跳没有被误判**（比的是"保持住的事前值"：52495 vs 52490 ✓）⇒ 只出现**一个 4 拍脉冲。**

④ **★ 但 `angle`（环路滤波后，ABZ / BiSS / UVW 用的就是它）仍然冲了** - 52464(512) → 52506(515) → 52657(516) → **53798（第 522 拍峰值）** → 53158(526)：**+1334 LSB ≈ +7.3° 过冲，约 10 clk 到峰、~0.8 µs 收回**；而标称速率只有 1.34 LSB/clk ⇒ **环路把这次事件重新注入了**，所以"角度毛刺保持"挡得住 `angle_raw`，**挡不住输出。**这也是台面"ABZ 波形乱"的机理之一。 - 同一窗口 ABZ 输出：**AB 跳变 139 次（理想 = 4×256×1 kHz×20.48 µs = 21）**、间隔中位 **4 clk**、`edge_cnt` **净 +20 但增 79 次 / 减 59 次** ⇒ 输出在抖（详见 §1.38）。

⑤ **★ 寄存器位义订正（以前记错，已核 RTL）** - `rtl/enc_top.v:748-757` 的 `stat_byte = {bit7 res=0, bit6 adapt=0, bit5 scamp=(amp_low|amp_high), bit4 lagfatl=lag_fatal, bit3 inclim=(limiter_active|uvw_freq_warn), bit2 laglim=(lag_limit|lag_warn), bit1 falarm=0, bit0 oflow=(cor_sat|otr_lat)}`（**不是**"bit0 laglim / bit5 calbusy"）。 - `rtl/reg_file.v:510-512` 的 `0x053A = {bit7 frame_active, bit6 crc_err_seen, bit5 calib_busy, bit4 r_calib_done, bit3 stat_otr, bit2 stat_amp_high, bit1 stat_amp_low, bit0 stat_lag_limit}`。 - `otr_lat` 是 `ad9238_capture` **内部**的 sticky 锁存（`rtl/enc_top.v:245 .otr(otr_lat)`；清位脉冲 `clr_otr_adc` 来自 `rtl/enc_top.v:242`，由**写 `0x0405` bit0=1** 发出，`rtl/reg_file.v:411`），与 `rtl/reg_file.v:394-411` 那个**可读回 `0x0405`** 的 W1C 锁存是**两个独立锁存器**（`rtl/SPEC_interface_contract.md:471-480`）。 - **板上交叉验证**（ILA 位流下 UART 桥照常可用）：`read 0x05F2` = **0x0006**、`0x053A` = **0x0008**（`set: otr`）、`0x053B` = 0x0000 ⇒ ILA 探针 `otr_lat` =1 与寄存器 bit3 **两条独立路径互证。**

1.38 ★★★ ABZ"抖动群"根因定案：`0x0103` AB 迟滞 = 0 是主因（四档扫描实测）+ OTR 清位真能清（2026-10-11，RTL 未改，新增 `_synth_work/abz_chatter.py`）

> ⚠️ **标题里的"迟滞是主因"已被 §1.40 推翻**（同日上板 A/B 实测 + 统一窗口复测）：真实主因是**环路 I 项增益 ×4**（`0x0302` 的 ki=3）；迟滞无效。本节保留下来的有价值结论是 ④（OTR 软清）与"迟滞放行会把累积 LSB 一次性挤出"的机理。

背景：§1.37 ④ 实测"AB 跳变 139 次 / 理想 21 次、`edge_cnt` 净对但增 79 减 59" ⇒ 顺着"迟滞寄存器没设"这条线索做成四档对比。

① **板上配置读回**（`tools\uart_spi.py --port COM6 read 0x0101 0x0102 0x0103 0x0106`）：- `0x0101` = 0x0000、`0x0102` = 0x0000 ⇒ `INTER = 0`（RTL 按 `inter_n = (inter==0) ? 256 : inter` 处理 ⇒ 实际 256）、`div_n = 0` ⇒ div = 1； - **`0x0103`（AB hyst）= 0**、`0x0104`（UVW hyst）= 0（UVW 侧按文档回退到 `UVW_HYST_SHIFT = 6` ⇒ 1024 LSB）、`0x0106`（ablimit）= 0。⇒ **A/B 通路完全没有迟滞**，`rtl/abz_gen.v` 里 `hyst_pass` 恒真，索引目标跟着角度的每一个 LSB 走。

② **输入侧证据（ABZ 的输入就是 `angle`，`rtl/enc_top.v:549`）**：同窗口 `dbg_angle` 的 1023 个逐拍步进里 **后退 440 次 / 前进 537 次 / 零 46 次、方向翻转 51 次**，而标称速率只有 +1.34 LSB/clk ⇒ **环路滤波后的角度本身就在 LSB 级来回抖**（P 通路把 CORDIC 噪声直接带到输出上）。ABZ 的索引单位 = `angle/64`（`idx = angle*inter >> 14`），抖到 64 LSB 就换一个索引台阶 ⇒ 正反成对点火。

③ **四档实测（同一 8 Vpp / 1 kHz / 90° 输入；`_synth_work/abz_chatter.py`，窗口 20.48 µs，理想 = 4×256×1 kHz×20.48 µs = 21 次）**

`0x0103` hyst	CSV	AB 跳变	`edge_cnt` 净	增/减	放大倍数	间隔中位
0（基线，stage2）	`_synth_work\ila_cap_stage2.csv`	**139**	+20	79/59	**6.9×**	4 clk
2	`_synth_work\ila_cap_hyst2.csv`	**66**	+21	43/22	**3.1×**	12 clk
4	`_synth_work\ila_cap_hyst4.csv`	86	+22	54/32	3.9×	8 clk
8	`_synth_work\ila_cap_hyst8.csv`	81	+21	51/30	3.9×	8 clk

- 四档的输入条件实测一致（r = 1593…1627 LSB ≈ 7.78…7.95 Vpp、f_in = 0.989…1.007 kHz）⇒ **可比**；四档 `edge_cnt` **净计数都对（+20…+22 ≈ 21）** ⇒ 净计数不是问题，抖才是。
- 结论：① **hyst = 0 最差**（6.9×，间隔中位 4 clk = 每个 `in_valid` 节拍都在动，示波器上就是"乱"）；② **hyst = 2 最优**（3.1×，后退点火 59→22、间隔中位回到 12 clk ≈ 真实节拍）；③ hyst 再加大**反而更差**（4/8 → 3.9×）：迟滞门放行时会把**累积的 32•hyst 个角度 LSB 一次性放出去**，形成 2…4 个边沿的"簇"，后退少了但时序更不均；④ **任何寄存器档都消不掉剩余约 3× 的边沿膨胀** ⇒ 剩余根因在 `angle` 的 50 MHz 抖动本身，要根治需 RTL 级改动（候选 v7，见 §3）。

> ⚠️ **2026-10-11 当天晚些即被 §1.40 ③ 推翻（务必先读那条）**：上面这张表的**基线行（hyst = 0，139 = 6.9×）是 stage2 毛刺触发窗口**，而 2/4/8 三行是 **stage1 立即触发窗口** ⇒ **跨窗口比较。**用统一 stage1 窗口、同一增益重测：hyst 0 ⇒ **57（2.7×）**、2 ⇒ 66（3.1×）、4 ⇒ 86（3.9×）、8 ⇒ 81（3.9×）⇒ **迟滞毫无帮助且单调变差**（上面的"hyst = 2 最优"是假象）；真实自由运行基线是 **2.7× 而不是 6.9×**，真正有效的旋钮是**环路增益 `0x0302`（ki 3→0 ⇒ 1.0×）。本节 ③ 的"放行时一次性挤出"机理仍然成立**，只是它从来不是收益。

- **与手册/规范一致**：`AB.hyst[°] = ±code×360°/2048` ⇒ code = 2 即 ±0.352°，正好对应 TW28 手册 "C1 level 8 = ±0.35°"（`docs/TW28_digest.md:254`、`docs/01_technical_solution.md:330`）；本实现 `hyst_lsb = code << (AW-11) = code×32` 角度 LSB，与之等价。`docs/04_wiring_checklist.md:334` 早就写着"AB 输出抖动 → 加大 `hyst`"，本轮给出**量化依据**（★ 但该量化结论已被 §1.40 ③ 修正：**加大 hyst 无益**；规范里的推荐只在"角度干净、抖动纯属量化"时才成立）。
- **~~推荐动作**：上电后写 `0x0103 = 0x0002` ~~ ⇒ **★ 作废（§1.40 ③）**。正确动作 = 上电后写 **`0x0302 = 0x0004`（kp=4 / ki=0 纯 P）**，见 §1.40 ②；要"开机即生效"须改 `rtl/reg_file.v:280` 的 `r_filt_k` 复位值（`0x1C` → `0x04`）⇒ 需重综合，列为 **v7 候选**（§3 item 17 ②(0)）。

④ **顺带定案：ADC 粘滞 OTR 确实能用软清位清掉（修正 §1.23 的表述）** - 连做 6 次"写 `0x0405 = 0x0001`（bit0 W1C + 1 拍 `clr_otr`）→ 立刻读 `0x053A`"：**5 次读到 0x0000、1 次读到 0x0008**（0.25 s 内又复亮）。 - ⇒ 8 Vpp 下 OTR 只是**偶发**再触发（输入峰值 7.93 Vpp ≈ 79% 满量程），**不是**"一清就毫秒级复亮"那种常亮；§1.23 里"10 Vpp 清完立刻复亮 / ≤8 Vpp 才稳"应按此修正为"偶发"。 - `0x0405` 读回 = **0x0025**（`set: otr laglim amp`）⇒ 三个锁存位历史上都被点过。两个独立锁存器与清位脉冲的关系见 §1.37 ⑤。

1.39 ★★★ ABZ"抖动群"的上游根因定案：`loop_filter` 的状态每个 clk 都在更新（`in_valid` 只有 1/4 频率 ⇒ I 项每样本增益 ×4）+ 离线全链路回放比选 v7 修法（2026-10-11，RTL 只新增了默认关闭的参数，新增 `sim/tb_loop_abz_chain.v`、`_synth_work/dump_abz_angle.py`）

① 为什么 §1.38 只治到 3× §1.38 曾认为"AB 迟滞（`0x0103`）默认 0"是主因（★ 该结论已被 §1.40 ③ 推翻：迟滞无效），且**任何寄存器档都还剩约 2.7× 的边沿膨胀** ⇒ 残留来源在送进 `abz_gen` 的那个 `angle` 上（`rtl/enc_top.v:549` `wire [AW-1:0] angle = pos_ext[ZW-1:FRAC];`）。

② 读 RTL 得到的根因（这是本轮最重要的发现） - **`rtl/loop_filter.v:158-166` 环路状态（`pos` / `integ`）每个 clk 都更新，完全没有 `in_valid` 门控**：`always @(posedge clk) begin if (!rst_n) begin pos <= {PW{1'b0}}; integ <= {PW{1'b0}}; end else begin pos <= pos_nxt; integ <= integ_nxt; end //` ★ 50 MHz 上迭代 `end` - 而它的输入 `in_valid = cordic_valid`（`rtl/enc_top.v:541`）只有 **12.5 MHz**（`LOG2DEC=2`，每 4 个采样一个有效拍）⇒ **每样本等效增益被放大 4 倍。**查移位表 `rtl/loop_filter.v:97-123`：`kp=4 → >>6`、`ki=3 → >>4`；`0x0302`（FILT_K）复位值 **`0x1C`**（`rtl/reg_file.v:74`、`:280` ⇒ `cfg_kp = r_filt_k[2:0] = 4`、`cfg_ki = r_filt_k[4:3] = 3`，`rtl/reg_file.v:553-554`）⇒ **I 项每 clk 施加 `err/16`，即每样本 `err/4`** ⇒ 环路阻尼不足。 - ⚠️ 顺带订正一处记错：`0x0105`（FILT_CFG **只存储**）**不是**环路总开关；真正的增益寄存器是 `0x0300/0x0301/0x0302`。板上默认 **kp=4 / ki=3**，环路是开着的（不是 bypass）。

③ 定量（用 1037 行 ILA stage2 CSV 直接量） - **原始 CORDIC 角** `dbg_angle_raw`（1024 点）按 12.5 MHz 节拍抽样：255 步里前进 172 / 后退 76 / 零 7，方向翻转 117，`|d|` 均值 **≈23 LSB ≈ 0.13°**；`|d| > 64 LSB`（= 1 个 ABZ 索引单位）只有 2 处（就是那个毛刺的正负跳变）⇒ **原始角本身几乎不会跨索引单位。** - **环路输出** `dbg_angle`（= 送进 `abz_gen` 的角）：50 MHz 视图 1024 拍里变化 **977 次**、节拍 `|d|` 均值 **191 LSB（≈1.0°）**、节拍方向反转 51 次（每 4 拍相位统计一致）⇒ **环路把噪声放大约 5…8 倍，越过索引单位（64 LSB）⇒ 索引目标正反横跳 ⇒ A/B 正反点火成对抵消（净计数仍对）。** - 这条链条解释了台面"波形乱"：`edge_cnt` 净增量正确，但边沿位置在抖。

④ 回放方法（全链路，可复跑） - `_synth_work/dump_abz_angle.py <csv> <out.mem> [列名]`：把 ILA CSV 的某一列导成 `$readmemh` 用的 1024 行十六进制（默认 `dbg_angle[15:0]`）；本轮导出了 `_synth_work/abz_angle_stage2.mem`（旧）、`_synth_work/abz_angle_raw_stage2.mem`（原始角，min 51818 / max 54254）、`_synth_work/abz_glitch_stage2.mem`（`dbg_ang_glitch`）。 - `sim/tb_loop_abz_chain.v`：按 **12.5 MHz 节拍**回放 `dbg_angle_raw`，**同一输入喂两条链**（A = `GATE_STATE=0` 现状、B = `GATE_STATE=1` 门控候选），每条链后接真实 `abz_gen #(.AW(16), .CLK_PERIOD_NS(20.0))`（`.inter(256) .div(3'd0) .ablimit(8'd0) .z_width(4)`）；每遍源序列加 **`NET_PASS = 1342` LSB**（= 1 kHz × 20.48 µs）拼成连续斜坡，毛刺拍按 `rtl/enc_top.v:456` 的"保持"语义处理；`NPASS=10`、**只统计最后 `MEAS=6` 遍（窗口 1536 拍 = 6 个输入周期 ⇒ 理想 125 个计数）。** - `rtl/loop_filter.v` 新增 **`parameter integer GATE_STATE = 0`**（module 头）+ `wire state_en = GATE_STATE ? in_valid : 1'b1;`，状态使能改由它门控。**默认 0 时行为与改动前一致**（`GATE_STATE=1` 才是新行为）；★ 未做综合对比，故只能说"行为等价"，**别声称与既有位流逐位相同。**

⑤ 实测比选表（理想计数 = 125；`|d|` = 滤波角节拍步进均值，单位 LSB/节拍）

相位	A 链 (kp/ki/hyst)	A:跳变	A:净	A:增/减	A:|d|	A:反转	B 链（门控）
P0-now	4/3/0	**482（3.86×）**	126 ✓	304/178	122	**816**	4/3/0：净 126 ✓、|d|115、反转 72
P1-ki1	4/1/0	**259（2.07×）**	126 ✓	192/66	12	399	2/3/0：**净 126 ✓、|d| 5、反转 0**
P2-ki0	4/0/0	**126（1.01×）**	126 ✓	126/0	5	**0**	2/0/0：净 126 ✓、反转 2
P3-ki1-h2	4/1/2	26	**26 ✗ 丢 99**	26/0	12	399	10 ✗
P4-ki0-h2	4/0/2	12	**12 ✗ 丢 113**	12/0	5	0	12 ✗
P5-ki0-h1	4/0/1	**126（1.01×）**	126 ✓	126/0	5	0	126 ✓
P6-ki1-h1	4/1/1	286（2.3×）	126 ✓	206/80	12	399	126 ✓

⑥ 结论 1. **抖动强度由 I 项增益 `ki` 主导**，三条指标（AB 跳变数 / `|d|` / 方向反转）随时间 `ki` 单调同向：`ki=3` ⇒ 3.86× 边沿、816 次反转（53% 的节拍在倒转）；`ki=1` ⇒ 2.07× / 399；**`ki=0` ⇒ 1.01×（= 理想）、0 次反转。** 2. **现状与板上吻合**：仿真 P0 = 482/125 = **3.9×**，板上 stage2 = 139/21 = **6.6×**（同量级；板上窗口只有 20.48 µs 且含毛刺余波）。 3. **只把环路状态门控（`GATE_STATE=1`）是"把增益降到 1/4"，不是"消噪"**：P0 的 B 链跳变虽降到 126，但 `|d|` 仍 115（最大 **3331 LSB ≈ 1.8°**）⇒ 环路变成"慢而大幅"的摆动，**计数对、边沿位置却在大范围漂** ⇒ **`|d|` 均值不能单独当 ABZ 质量判据，必须与跳变数一起看。** 4. **门控 + 增益重调才是正解**：P1 的 B（kp 4→2、ki 1→3）拿到全场最好指标（净 126 ✓、`|d|` 5、反转 0）；但这只是"每样本 i_inc 总量"层面的**部分**补偿（现状链每样本施加 4 次 P/4 次 I，门控后只施加 1 次）⇒ **`GATE_STATE` 必须与 kp/ki 成套重调，不能单独改。** 5. **寄存器级可立即上板验证（无需重综合）**：写 **`0x0302 = 0x0C`**（kp=4 / ki=1）或 **`0x0302 = 0x04`**（kp=4 / ki=0，纯 P）后重抓 ILA/遥测，看 AB 跳变数是否从 57 往下掉（**已实测：⇒ 22 / 20，见下条**）。**代价**：纯 P 是 I 型环路，有与转速成正比的稳态滞后（`ki=0` 时稳态误差 = 速度 ÷ P 增益），必须同时量"跟踪误差 vs 转速"才能定案。 6. **迟滞（§1.38 的推荐）在本仿真里出现反例，须先解释清**：`hyst=1` 安全（P5/P6 净 126 ✓、`ki=0+h1` 反转 0），但 **`hyst=2` 大量丢计数（P4：12/125 = 丢 90%）**，而板上那次 20 µs 窗口（hyst2 净 +21 = 理想）**看不出丢计数** ⇒ 两者矛盾。**在解释清之前不要建议 hyst ≥ 2**；板上判定必须用**长窗口**（`edge_cnt` 速率或 `tools/uart_scope.py` 遥测统计），20 µs 窗口只有 21 个理想计数，分辨不出 10% 级丢计数。

> **★ 2026-10-11 上板实测已给出答案（见 §1.40）**：`ki=0` ⇒ 边沿 1.0×、零反转；`ki=1` ⇒ 1.1×；**而 AB 迟滞用统一窗口复测后完全无效（2.7× → 3.1× → 3.9× 单调变差）**，§1.38 的"hyst=2 最优"是跨窗口比较的假象。⇒ **寄存器级的答案就是"把 ki 降下来"，迟滞这条建议作废**；剩下的 hyst 仿真/板上不一致（本条）已降级为残留问题。

⑦ v7 候选（三条，均需重综合 ⇒ 归入 §3 item 17）：① 环路状态门控（已有参数）+ kp/ki 成套重调；② 给送进 `abz_gen` 的角度加一级"只在 `in_valid` 拍更新"的整形（速率限幅/一阶低通）；③ 提高 `filt_valid`（`LOG2DEC` 2→0）让节拍栅格从 80 ns 降到 20 ns 再配强迟滞。 **⑧ 复现**：`python _synth_work\dump_abz_angle.py _synth_work\ila_cap_stage2.csv _synth_work\abz_angle_raw_stage2.mem "dbg_angle_raw[15:0]"` 然后 `tools\run_sim.ps1 -Only tb_loop_abz_chain`（约 4 s，`sim/logs/tb_loop_abz_chain.log`；`$finish at 1433830 ns`）。★ `$readmemh` 那条 `WARNING: File ... referenced on .../rtl/abz_gen.v at line 313 cannot be opened` 是**良性**（第一次相对路径失败、`../../_synth_work/` 回退成功；文件/行归属是 xsim 乱报）。

⑨ 测试台自身的三个坑（教训，怀疑数据前先怀疑统计接线） 1. **计数器接线错字**：`b_a_d <= b_b;`（应为 `b_a`）⇒ B 链跳变量虚高 **1625**（真值 67），A 链正常。**自查判据：链 A 的"跳变数"必须恒等于"增 + 减"**（P0 的 150+87 = 237 ✓、P5 的 241+178 = 419 ✓），不自洽先怀疑统计代码。 2. **方向翻转判据写反**（把"同号"当翻转）⇒ 单调序列也每步计数；正确读法：**真反转 = 非零步数 − 打印值。** 3. **`|d|` /反转计数器未受 `meas_en` 门控** ⇒ 在全部 10 遍上累计而 AB 统计只在 6 遍上 ⇒ 曾打印出"非零步 1518 / 窗口节拍 768"这种不自洽数字；另：`integer ticks_win;` 必须在使用前声明（`ERROR: [VRFC 10-3380] identifier 'ticks_win' is used before its declaration`）。 4. **前身测试台 `sim/tb_abz_chatter.v` 已删除（2026-10-11）**：它把 ILA 抓到的**已滤波** `dbg_angle` 序列直接回放给 `abz_gen`，实测 `edge_cnt` 净增量恒为 **0…6**，而硬件同一窗口是 **+20 = 理想** ⇒ **问题在回放方式**（缺预热、缺节拍重构、每遍不加 `NET_PASS` 斜坡），不在 RTL；本节的 `tb_loop_abz_chain.v` 改回放**原始** `dbg_angle_raw` 并经**真实** `loop_filter`，净计数与抖动双双对上硬件（126/125 与 3.86×）⇒ 已取代之。旧日志归档为 `sim/logs/_ORPHAN_tb_abz_chatter_superseded.log`。★ **残留未解**：为什么"直接回放 ILA 抓的 `dbg_angle` "会丢净计数（是否 ILA 抓到的 `dbg_angle` 序列不是消费者逐拍所见的那一串）—— 若要彻底钉死，需在 ILA 里加一个"`in_valid` 拍上的 `dbg_angle` 快照"探针再回放。

1.40 ★★★ 【定案】环路增益是唯一有效的旋钮：`0x0302` 上板 A/B 实测 —— ki 由 3 降到 0，AB 边沿膨胀 2.7× → 1.0× 且零反向点火，跟踪误差同时变小；AB 迟滞（`0x0103`）用统一窗口复测后确认无效（2026-10-11，RTL 未改，`tools/ila_capture.tcl` 新增 `capture` 模式 + 新增 `_synth_work/gain_ab_test.ps1`）

① 关键前提：下位流会复位整个 FPGA ⇒ 配置寄存器全部回默认值 `program_hw_devices` 一执行，`0x0302`（环路增益）、`0x0103`（AB 迟滞）、标定结果都会被复位 ⇒ **"改一个寄存器、抓一版波形"的 A/B 对比，位流只能下一次。**为此给 `tools/ila_capture.tcl` 新增 `ILA_MODE=capture`：**不重下位流**，只在已烧好 ILA 版的器件上"武装 → 触发（`dbg_adc_valid`，等于立即触发）→ 上传 → 导出 CSV"。 - ★ 这是做**寄存器 A/B 的正确姿势**；若用 `stage1`/`stage2` 反复抓，刚写的寄存器会被下位流抹掉，得到的对比表可能是**假的**（历史上若这么做过，需按本节重做）。 - 驱动脚本：`_synth_work/gain_ab_test.ps1` —— `stage1` 下一次位流并抓基线 → 写 `0x0302` 后用 `capture` 抓 → 还原 `0x001C` → `restore` 把生产 v6 位流放回。实测全程 0 失败：每次写入后回读一致，且 `capture` 之后回读**仍是写入值**（证明没被复位）。

② 实测（同一 8 Vpp / ~1 kHz / 90° 输入；三档都是"立即触发"窗口、20.48 µs；`_synth_work/abz_chatter.py`）

`0x0302`	kp/ki	AB 跳变	理想	放大	`edge_cnt` 净	增/减	间隔中位	A/B 翻转	`angle` vs `atan2` 残差 RMS
0x1C（复位默认）	4/3	57	20.8	**2.7×**	+21 ✓	39/18	12 clk	31/26	0.126°
0x0C	4/1	22	21.0	**1.1×**	+21 ✓	22/1	44 clk	10/12	0.116°
0x04（纯 P）	4/0	**20**	20.8	**1.0×**	+20 ✓	**20/0**	48 clk	10/10	**0.067°**

- 输入条件三档一致（r = 1600…1631 LSB = 7.81…7.96 Vpp、f_in = 0.990…1.013 kHz）⇒ 可比；三档 `lag_warn` / `lag_fatal` **全 0**，`stat_byte` = 0x01（`otr_lat` 锁存，8 Vpp 接近满量程的既有现象 `0x0405=0x0001` 可清，不影响本对比）。
- **纯 P（`ki=0`）在每一项指标上都最好**：边沿数 = 理想、**零反向点火**（增 20 / 减 0）、间隔几乎均匀（44/48/52 clk，理想 49.2 clk），**跟踪残差反而最小。**
- 为什么纯 P 没出现 I 型环路的稳态滞后？理论滞后 = 速度 ÷ 每拍 P 增益 = `1.34 LSB/clk ÷ (1/64)` ≈ **21 LSB ≈ 0.115°** —— 因为 P 通路在 **50 MHz** 上迭代而输入只有 12.5 MHz ⇒ **有效 P 增益被放大 4 倍，恰好抵消了"纯 P"的代价**（与 §1.39 的根因是同一枚硬币的两面）。⇒ **1 kHz 下可忽略**；它随转速线性增长（10 kHz ⇒ ≈1.15°、100 kHz ⇒ ≈11.5°）⇒ **高速档若要用 `ki=0`，必须先量跟踪误差。**
- ⇒ **建议动作：上电后写 `0x0302 = 0x0004`**（运行期配置、掉电恢复 0x1C）。要"开机即生效"须改 `rtl/reg_file.v:280` 的复位值 ⇒ 需重综合（v7）。
- **★ 这也解释了 §1.38 与 §1.37 的 139（6.9×）为什么比本节的 57（2.7×）大**：139 那个窗口是 **stage2（毛刺触发）** 抓的，里面含毛刺余波（§1.37 ③ 实测毛刺后环路过冲 +7.3°）；**自由运行的真实基线是 2.7×。**

③ ★ 订正 §1.38：AB 迟滞（`0x0103`）无效 §1.38 的"hyst = 2 最优（139 → 66，3.1×）"是**跨窗口比较的假象** —— 它的基线行用的是 **stage2（毛刺触发）** 的 `_synth_work\ila_cap_stage2.csv`（含毛刺余波 ⇒ 139），而 hyst = 2/4/8 三行是 **stage1** 立即触发窗口。用**统一的 stage1 窗口、同一增益（ki=3）**重测：

`0x0103` hyst	AB 跳变	放大	净	增/减	间隔中位
0（新基线 `ila_cap_gain1c.csv`）	**57**	**2.7×**	+21 ✓	39/18	12 clk
2（`ila_cap_hyst2.csv`）	66	3.1×	+21 ✓	43/22	12 clk
4（`ila_cap_hyst4.csv`）	86	3.9×	+22	54/32	8 clk
8（`ila_cap_hyst8.csv`）	81	3.9×	+21 ✓	51/30	8 clk

- ⇒ **迟滞毫无帮助、且随档位单调变差**（2.7× → 3.1× → 3.9×、间隔中位 12 → 8 clk）：它的机理是"把边沿攒起来再一次放出去"（放行时把累积的 `32·hyst` 个角度 LSB 一次挤出），只能让边沿**更不均匀**，治不了"角度本身在抖"这个病根。**"上电写 `0x0103 = 0x0002` "这条建议作废。**
- **残留未解（已降级）**：仿真里 `hyst=2` 丢 ~90% 计数（§1.39 ⑤ 的 P3/P4），板上同一配置净计数正确（+21 = 理想）⇒ 两侧 hyst 行为不一致。既然迟滞无用，不再投入；若日后要用，先在最小测试台上"只扫 hyst"复现（不要再用全链路回放台找它）。

④ 复现

```
# 需要一次性 danger-full-access（hw_server/cs_server 要往 TEMP 释放 VCRUNTIME140.dll）
& powershell.exe -NoProfile -ExecutionPolicy Bypass -File _synth_work\gain_ab_test.ps1
python _synth_work\abz_chatter.py --inter 256 --div 1 `
    _synth_work\ila_cap_gain1c.csv _synth_work\ila_cap_gainki1.csv _synth_work\ila_cap_gainki0.csv
```

- 产物：`_synth_work\ila_cap_gain{1c,ki1,ki0}.csv`（各 134,738 B）、日志 `_synth_work\ila_gain{1c,ki1,ki0,restore}.log`。
- 收尾状态已核对：`0x0302` = 0x001C（还原）、`0x0103` = 0x0000、生产 v6 位流已放回（`0x05F2` = **0x0006** ✓）。
- ★ `tools/ila_report.py` 管道进 `Select-Object -First N` 会因 broken pipe **exit 1**（虚警，见 §4）。

1.41 ★ 文档导出链定案：`tools/md2pdf.py`（python-markdown + Edge 无头打印）；本机没有 pandoc 的 PDF 引擎（2026-10-11，新增 `tools/md2pdf.py`）

为什么会有这一节：`docs/*.pdf` 是 `docs/*.md` 的导出件，但 2026-10-09 那批 PDF 全部早于对应 `.md`（§3 item 11 记的"交付风险"），工作区里 8 个已被删除 ⇒ 需要一个**可重复、可自证**的导出做法，不能靠"某台机器上手工点一下"。

① 本机工具链实测（别再试错）

东西	结论
`pandoc`	**在**，`E:\Anaconda\Library\bin\pandoc.exe`
pandoc 的 PDF 引擎	**一个都没有**：`xelatex` / `pdflatex` / `lualatex` / `tectonic` / `typst` / `wkhtmltopdf` / `weasyprint` 全部（none）⇒ `pandoc x.md -o x.pdf` **必失败**
LibreOffice	`soffice` / `libreoffice` 在 PATH 与两个标准安装目录下都**不存在**（DSH 运行时目录下也没有 `soffice.exe`）
`markdown`	**在**，`E:\Anaconda\python.exe` 里 `markdown 3.8`
浏览器	**在**，`C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe`（支持 `--headless=new --print-to-pdf`，中文字体正常）

② 定案做法 = `python-markdown 转 HTML → Edge 无头打印成 PDF`，脚本 `tools/md2pdf.py`： - 扩展：`tables / fenced_code / sane_lists / attr_list / toc / md_in_html`；CSS 约定写死在脚本头（A4、18 mm 边距、正文 10.5 pt 微软雅黑、代码 9 pt Consolas、表格全宽细边框、`page-break-inside: avoid` 加在表格行与代码块）。 - CLI：`& E:\Anaconda\python.exe tools\md2pdf.py docs\01_xxx.md [更多 .md]`； `--all`（docs 下全部 `.md`）、`--selftest`（小样本自检整条链路）、`--outdir`（默认 `_pdf_out/`，HTML 中间件，已进 `.gitignore`）、`--clean-html`。

③ ★ 沙箱坑（会复发，务必记住）：Edge 无头打印**要先给自己的 profile 目录授 ACL**，在**工作区沙箱下必然失败**：

```
ERROR:content\browser\network_sandbox.cc:523] Failed to grant sandbox access to network context data directory
  ...\.tmp\edge-profile\Default\Network: 拒绝访问。 (0x5)
```

表现 = `rc=2147483651`（或 `-2147483645`）、**PDF 不生成**； `--no-sandbox`、`--disable-extensions`、`--disable-features=NetworkServiceSandbox,RendererCodeIntegrity` **都无效。** ⇒ **导出动作必须带一次性** `danger-full-access` —— 与 §1.36 的 `cs_server`（要往 TEMP 释放 DLL）是**同一类坑**：*"需要提权/动 ACL 的子进程"在工作区沙箱下会以看不懂的权限错误失败*。

④ 产物与自证：2026-10-11 15:51 重导 **8 个 PDF**（`docs/00,01,02,03,04,06`、`AD9238_module_digest`、`ATK_board_digest`，与当时 `.md` 同步；`docs/05/07/08` 本来就没有 PDF）。 自证：`& <runtime-python> tools\pdf_text.py docs\00_roadmap_30d.pdf -1 _pdf_out\_check00.txt --geom` ⇒ **8 页 / 12,098 字符 / 中文可提取**（表格边框不在文本层，属正常）。

1.42 ★★ 速率假设审计（v8 规划依据）：全部常数按 `f_valid = 50 MHz` 标定、实际链路跑在 12.5 MHz —— 20 项逐条 + `GATE_STATE` 分析（2026-10-11，只读审计，未改任何 RTL）

> ★ 本节是 **2026-10-11 的只读审计**（只读 RTL 与黄金模型源码，**未改任何 RTL、未重综合、未上板**），是**选 v8 主线的依据**：它把“哪些常数已经被 4× 的有效/时钟比弄错、哪些会在 `LOG2DEC` 2→0 时再动一次”逐条摊开（共 20 项 + `GATE_STATE` 专节 + 8 条最高风险）。 与全文其它小节的关系：**§1.33 item 16** 记的“过速静默丢沿 + `r_acc` 无饱和”是同一类速率假设问题在 AB 输出上的表现；**§3 item 17** 的 (a) 环路门控 `GATE_STATE`、(b) 角度速率限幅、(c) `LOG2DEC` 2→0 三条候选，正是本节第 9/10/11 行与“`GATE_STATE` 0→1”专节直接分析的对象 —— **看 v8 候选前先看本节。**
>
> ⚠ 下面正文是该次审计的**原始英文报告，逐字保留**（20 行表格 + `GATE_STATE` 专节 + 8 条最高风险）； 表内 `文件:行` 锚点、数值、信号名均可直接 grep 核对，故**不做翻译**以免引入二次失真。

Read-only audit complete — nothing was modified. Anchor for interpreting everything below: **every constant in this RTL was numerically calibrated for `f_valid = 50 MHz` (one valid event per `clk_i`)**, while the design actually runs the chain at 12.5 MHz. So there are two classes of defect: (i) constants that are *already* off by the 4× valid/clk ratio, and (ii) constants that will move again if `LOG2DEC` 2→0.

Rate ladder: `adc_valid` = 1/clk (50 MHz, `rtl/ad9238_capture.v:172` `adc_valid <= 1'b1;`) → `dec_valid` / `cor_valid` / `cordic_valid` = 50 MHz/2^LOG2DEC = 12.5 MHz at `rtl/enc_top.v:38` → `filt_valid` = `loop_filter` `out_valid` (3 clk after each `cordic_valid`, same average rate).

Conversion rule used throughout: a counter that only advances inside an `if (in_valid)` / `else if (*_valid)` body counts **valid samples, not clocks**, so its wall-clock duration = N/f_valid. To hold a window constant when the valid rate goes f0→f1: **N_new = round(N_old × f1/f0)**; 12.5→50 MHz means **N × 4**.

Findings

#	File:line	Signal / param	Current value	Rate assumption encoded	If LOG2DEC 2→0	Class
1	`rtl/abz_gen.v:397-404` (hold decremented `:530`, `:605-610`)	`gap16 = (gap_num + NS_PER_CLK-1)/NS_PER_CLK`, `gap_num = 32'd20*(ablimit+1)*div_ext`	ablimit=0, div=1 → `gap16` =1	Comment `:45` says "20 ns/edge minimum" and converts ns→**clk** count; but `r_hold` only decrements inside `else if (in_valid)` `:530` ⇒ 1 **valid period** = 80 ns real, not 20 ns	becomes 1 valid period = 20 ns → limiter/edge-rate cap tightens 4× in time; must become 4 to preserve today's behaviour	**A + C (silent /4)**
2	`rtl/abz_gen.v:138`, used `:656`, `:669-675`	`Z_TIMEOUT = 4096 // Z 脉冲最长 clk 数`	4096	Counts only on `in_valid` ⇒ real 4096/12.5 MHz = 327.7 µs (not 81.9 µs)	4096/50 MHz = 81.9 µs ⇒ reload must be 16384 to keep the timeout	**A**
3	`rtl/uvw_gen.v:66-67,92` (`MINHALF`), `:193,507-509,520-539`	`UVW_MIN_HALF_CYCLE = 12` fed from `rtl/enc_top.v:655` `UVW_MIN_HALF = 2*f_clk/8.33 MHz`	12	`r_dwell` increments only on `in_valid` ⇒ "12 clk = 1 µs = 60° sector" is really 12 valid samples	threshold moves 2.08 MHz → 8.33 MHz; `UVW_MIN_HALF` must become 48 to keep 8.33 MHz	**A** (feeds `stat_inclim`, `rtl/enc_top.v:905`)
4	`rtl/enc_top.v:992`, FSM `:1021-1042`	`CONF_N = 4'd3` (confirmation in valid samples)	3 (+1)	Confirmation latency = CONF_N+1 **samples**, comment `:1018-1020` says constant delay in samples	4 valid periods 320 ns → 80 ns; sample-domain interval property survives, absolute window shrinks 4×	**A (time constant)**
5	`rtl/calib_ctrl.v:64-68,151,537-539,553,611-614`	`MAX_SAMPLES = 65536`, `nsamp[17:0]`, `cap_stop_t`, `plast_t`	65536	Window length is in **samples**; samples/input-cycle ∝ valid rate (comment `:530` "1 kHz → 12500 样本")	samples/cycle ×4 ⇒ `cap_stop_t` / `n_after_t` truncate ~4× sooner, usable `win_cycles` ~÷4	**A**

#	File:line	Signal / param	Current value	Rate assumption encoded	If LOG2DEC 2→0	Class
6	`rtl/enc_top.v:506-522`	`gl_cnt_1024/2048/4096/8192`, `gl_hold_cnt` (16-bit)	16-bit free-running	+1 per `cordic_valid`; wrap 65536 ticks	5.24 ms wrap (12.5 MHz) → 1.31 ms (50 MHz); documented per-input-cycle ratios need the assumed rate	**A (diagnostic)**
7	`rtl/enc_top.v:450-458,475-483`	`ANG_GLITCH_MAX = 16'sd1024`, `ang_held` 1-sample hold	1024 LSB ≈ 5.625° (comment says 5.6°), hold 1 valid sample	Threshold is angle-domain (B); hold duration is 1 valid period = 80 ns	0.029°/sample → 0.117°/sample (comment `:438-441`), still ≫ noise; hold 80 ns → 20 ns	**B threshold / A duration**
8	`rtl/decim_avg.v:22-30,48,92`	`NDEC = 1<<LOG2DEC`, boxcar gain `(1/2)·LOG2DEC` bits	LOG2DEC=2 → N=4, **1 bit (6 dB)** noise gain	Averaging window and its SNR gain scale directly with LOG2DEC	N=1, **0 bits** — the whole ADC-noise averaging gain disappears while update rate ×4 (loop must be retuned)	**A**
9	`rtl/loop_filter.v:71-79,167,173`	`GATE_STATE = 0`; `state_en = GATE_STATE ? in_valid : 1'b1`	0 (ungated, 50 MHz state updates)	Comment `:71-78` **admits** state iterates every clk while `in_valid` is 12.5 MHz ⇒ per-sample gain ×4	unchanged by LOG2DEC (still ×4) — the baked factor just moves with `filt_valid`	**C (self-documented)**
10	`rtl/reg_file.v:280-284` (cf. `:278-279`)	`r_filt_k <= 8'h04` = **kp=4, ki=0**; `r_filt_cfg=0x06`; `r_filt_lag=0x0C`	ki=0 default	Comment: loop state iterates at 50 MHz while `in_valid` = 12.5 MHz ⇒ I-term gain ×4 ⇒ board AB edge inflation 2.7× (ki=3) → 1.0× (ki=0)	the ×4 changes to ×1 if LOG2DEC=0 ⇒ ki=0 workaround becomes wrong/unnecessary, kp/ki must be retuned	**C**
11	`rtl/loop_filter.v:98-101,110-132,148-152` + `model/loop_filter_model.py:48` `FS = 50.0e6 # 单时钟域 50 MHz`	`kp_shift` {2..9}, `ki_shift` {7,6,5,4}, `INTEG_POS/NEG = ±REV/4`, `LAG_POS/NEG = ±REV/8`	as shown	Gain-per-**update**; the analytic f3dB numbers `:41-56` and `model/loop_filter_model.py:298` assume update rate = `FS` = 50 MHz	update-rate scaling changes the effective Kp/Ki; thresholds stay angle-domain (B)	**A (gain/BW) + C**
12	`rtl/pole_counter.v:207-209`, `:586` call site	`guard_cnt <= glitch_guard / guard_cnt-1`; `glitch_guard = 4'd2`	2	Decrement is **per clk** (40 ns) but `wrap_*_ev` arrive per valid sample (80 ns apart) — guard and sampling are in different time domains	guard covers only 0.5 valid sample at 12.5 MHz; at 50 MHz it covers 2 clk / 1 clk-spaced samples	**A**
13	`rtl/pole_counter.v:20-21,109,118`	`STEP_MIN = 2^AW - 2^(AW-2)` (270°), 45° windows, comment "50 MHz 采样下 … 6.25 MHz 输入频率"	270°	Max measurable input frequency is quoted per **sample** at 50 MHz	real ceiling is 1.5625 MHz today, 6.25 MHz at LOG2DEC=0 — the comment only becomes true at LOG2DEC=0	**A (limit) / B (criterion)**
14	`model/abz_uvw_model.py:65,113,485-486,653` (`clk_period_ns=10.0`) and `:362,368` (`clk_hz=100e6`) vs `rtl/abz_gen.v:88` `CLK_PERIOD_NS=20.0`	golden-model gap/threshold basis	100 MHz	Model's `step()` = one RTL clk (docstring `:6`) and gap = `20ns*(ab+1)*div/clk_period`; with 10 ns it emits **2× the RTL's** cycle count; its UVW warn (`:368`) fires at 16.66 MHz while `:356` claims 8.33 MHz	model/RTL already disagree 2×; neither matches the 12.5 MHz valid rate	**C / model drift**
15	`rtl/calib_ctrl.v:182-185,513-526`	`ARM_TMO_MAX = 26'd50_000_000`	1 s	Counted per **clk**, not gated on `in_valid`	stays a true 1 s — genuinely rate-agnostic	**B**
16	`rtl/biss_c_slave.v:92,134,187,282`	`TIMEOUT_CYCLES = 10`	100 ns	Per clk, `idle_cnt` never gated by a valid	unchanged	**B**
17	`rtl/sc_correct.v:236-282`; `rtl/amp_monitor.v:77-113`; `rtl/cordic_atan2.v:183-265`	3-stage / 2-stage pipelines; CORDIC 17 iters, 19-clk latency	—	Data path advances **every clk**; `in_valid` only rides a shift register	unchanged; latency in clocks	**B**
18	`rtl/uvw_gen.v:91` `DWELL_MAX=0xFFFF`; `rtl/abz_gen.v:140` `GAP_MAX=16'hFFFF`; `rtl/uvw_gen.v:184-186` hysteresis 1024 LSB; `rtl/loop_filter.v:98-101` ±REV/4, ±REV/8	saturating caps / angle thresholds	—	Angle- or edge-domain quantities; saturation only at unreachable rates	unchanged	**B**
19	`rtl/enc_top.v:187-206` key debounce (1.3 ms), `:48-49` `BAUD_DIV=25`, `SAMPLE_DIV=5000`, `:806-811` SPI/GAP/TO_DIV	clk_i-domain timers	—	Count `clk_i` only, no `*_valid` anywhere	unchanged	**B**

#	File:line	Signal / param	Current value	Rate assumption encoded	If LOG2DEC 2→0	Class
20	`rtl/reg_file.v:268-273`	`r_zero1 = 8'h04` (Z width = 4), comment "4 拍 @50MHz = 80 ns"	4 edge steps	Width counted in **edge steps** in `abz_gen` ⇒ rate-agnostic in edges; the comment's 80 ns is misleading (it is 80 ns only if edges come every clk)	unchanged in edge terms	**B (comment wrong)**

If `LOG2DEC` stays 2 but `GATE_STATE` 0→1 in `rtl/loop_filter.v`

Gating **only** the state block does not make the design rate-agnostic — it introduces a new inconsistency:

1. **Loop dynamics change; every gain/bandwidth number becomes wrong by the ×4 the `ki=0` workaround was hiding.** `rtl/loop_filter.v:41-56,106-132` and `model/loop_filter_model.py:48` / `:298` are all calibrated for per-clock updates. With `GATE_STATE=1` the state rate returns to the "per-sample" semantic, so `rtl/reg_file.v:280` (`ki=0`) and `r_filt_cfg` / `r_filt_lag` must be retuned together — otherwise you trade the documented AB edge inflation for an under-gained loop. `freeze` / `bypass` (`:160-163`) are outside the gate and keep working.
2. **The dominant new defect: the output valid is NOT gated.** `rtl/loop_filter.v:189-205` (`vld_sr <= {vld_sr[0], in_valid}`) still pulses `out_valid` 3 clk after every `in_valid`, so `filt_valid` stays 12.5 MHz while `pos` is frozen for 3 of every 4 clocks. `pole_counter`, `abz_gen`, and `uvw_gen` each then process **4× more valid ticks carrying an identical angle** — 3 phantom duplicate samples per real one. Every per-valid counter downstream (`abz_gen` `gap16` / `Z_TIMEOUT` / `edge_cnt` /limiter, `uvw_gen` `r_dwell` / `MINHALF`, `pole_counter` guard ratio) is then driven at an effective rate 4× the loop's state rate. Fix requires gating `out_valid` too (e.g. `out_valid <= state_en`), not just `pos` / `integ`.
3. **The AB limiter does not move, but the loop does** — `abz_gen` `r_hold` is inside its own `in_valid` body, so the limiter's minimum interval is unchanged while the closing-loop correction rate drops 4×. This is precisely the mechanism that produced the recorded "AB edge inflation 2.7×" symptom.
4. **`pole_counter` glitch guard is unaffected by `GATE_STATE` and stays wrong**: `guard_cnt` (`rtl/pole_counter.v:207-209`) still decrements per clk (2 clk = 40 ns) although valid samples are 80 ns apart — it protects less than one sample interval.
5. Nothing in the `filt_valid`-only blocks (`abz_gen`, `uvw_gen`, `pole_counter`) gets *more* correct: their `*_clk*` misnomers (items 1, 2, 3 above) remain, because their `in_valid` rate is unchanged.

> ⚠ **复核订正（2026-10-11 当晚，逐行核对代码后加注）：上面第 2 条不成立，不要把这条带进 v8 结论。** - 事实：`rtl/loop_filter.v:200-202` 是 `vld_sr <= {vld_sr[0], in_valid}; out_valid <= vld_sr[1];` ⇒ `out_valid` 是**每个 `in_valid` 恰好一个脉冲**（12.5 MHz），而 `GATE_STATE`（`:167`）只门控 `pos` / `integ` 的**状态更新**（`:173`），**完全不改变 `out_valid` 的脉冲数。** - 下游本来就是"只在 valid 拍动作"：`rtl/abz_gen.v` 的整个点火时序块在 `if (in_valid)` 内（§1.33 ⑥的逐拍探针直接看到这一点），`uvw_gen` 的 `r_dwell` 同理（本节第 3 行自己也承认）。 所以"`pos` 被冻结的那 3 个 clk"**不产生任何 valid 拍**，不存在"3 个幽灵重复样本"。 - 结论修正：`GATE_STATE=1` **不需要**额外门控 `out_valid`，是名副其实的"一行参数"； (a) 的代价与风险因此比本节原先估计的**小**。其余 4 条（增益 ×4 必须同步重调、 limiter 与环路速率解耦、 `pole_counter` guard 跨域、 `*_clk*` 命名不清）仍然成立。

Highest risk / most likely to be forgotten

1. **`rtl/abz_gen.v:397-404` `gap16`** — the AB limiter's "20 ns" is really 1 valid period (80 ns today). It is the one constant that is *wrong today and becomes right at LOG2DEC=0*, then over-limits; `lag_warn` / `lag_fatal` balancings ride on it.
2. **`rtl/uvw_gen.v:66-67,92` `UVW_MIN_HALF_CYCLE=12`** — the "8.33 MHz" warning actually trips at 2.08 MHz and feeds a status/limit bit (`stat_inclim`). Silently wrong at the current setting.
3. **`rtl/abz_gen.v:138` `Z_TIMEOUT=4096`** — a µs-scale Z-safety timer measured in samples (327.7 µs today).
4. **`GATE_STATE=1` without gating `out_valid`** (`rtl/loop_filter.v:189-205`) — creates 4× phantom valid ticks for every downstream per-valid counter. Easiest thing to miss because the parameter looks like a one-bit fix.
5. **`rtl/pole_counter.v:207-209,586`** — glitch guard counted in clk while the criterion it guards is sampled per valid; already covers <1 sample interval, and its `:20-21` "6.25 MHz" claim is only true at LOG2DEC=0.
6. **Golden-model drift** — `model/abz_uvw_model.py:65,362` use 100 MHz/10 ns vs. RTL's 50 MHz/20 ns (`rtl/abz_gen.v:88`), so the model's limiter budget is 2× the RTL's; any regression you run against it is comparing different time bases.
7. **`rtl/decim_avg.v:22-30` boxcar gain** — at LOG2DEC=0 the 1 bit (6 dB) of ADC-noise averaging vanishes; there is no floor and no compensation anywhere.
8. **Sample-domain windows that are "logically correct" but shrink silently**: `rtl/enc_top.v:992` `CONF_N`, `:450-483` one-sample glitch hold, and `rtl/calib_ctrl.v:64-68` `MAX_SAMPLES=65536` (the latter already truncating the default 64-cycle window at low input frequencies — pre-existing, not caused by LOG2DEC).

Not determined: whether any board-level symptom (beyond the recorded ki=0 / 2.7× edge inflation) is attributable to items 1, 2 and 5 — the files contain no measured AB-output-frequency or `uvw_freq_warn` data; the referenced evidence lives in `docs/DEV_STATUS.md` (§1.17, §1.28, §1.29, §1.39/§1.40 as cited in `rtl/enc_top.v:927-959` and `rtl/reg_file.v:281-284`), which I did not open.

1.43 ★★★【v8 定案】(a) `GATE_STATE=1` + kp=2/ki=3：全链路 A/B 回放比选、落地清单与对 v7 口径的取代（2026-10-11，RTL 已改、未重综合）

为什么要做：§1.42 审计把"`rtl/loop_filter.v` 的环路状态**每个 clk 都更新**、而 `in_valid` 只有 12.5 MHz（`LOG2DEC=2`）⇒ **I 项每样本增益 ×4**"列为 class A 最高风险；§1.39/§1.40 已用**只改增益**（ki 3→0）把 AB 边沿膨胀从 2.7× 压到 1.0×，但代价是**用一个欠增益的 I 型环路替换过增益的 PI 环路**（纯 P 的稳态滞后 ∝ 转速：1 kHz ≈ 0.115°、10 kHz ≈ 1.15°）。用户 2026-10-11 选定审计建议的 **(a) 方案**：**打开 `GATE_STATE`（把环路状态拉回 per-sample 语义）+ 把 kp/ki 成套重调**（而不是 (b) 只改增益、(c) 改 `LOG2DEC`）。

证据（先跑证据再改）：`sim/tb_loop_abz_chain.v` 内置 A/B 双链（`:98` `.GATE_STATE(0)` / `:103` `.GATE_STATE(1)`，其余参数相同），一次运行同时给出两链在同一激励下的 AB 跳变数（6 遍窗口、理想 125）；实测见 `sim/logs/tb_loop_abz_chain.log:150-165`：

相位	A 链（`GATE_STATE=0`）kp/ki/hyst	A：跳变 / 净 / |d|最大 / 反转	B 链（`GATE_STATE=1`）kp/ki/hyst	B：跳变 / 净 / |d|最大 / 反转
P0	4/3/0（= v6 旧默认）	482 / 126 / 122 / **816**	4/3/0	126 / 126 / **115（max 3331 LSB ≈ 1.8°）** / 72

相位	A 链（GATE_STATE=0）kp/ki/hyst	A：跳变 / 净 / |d|最大 / 反转	B 链（GATE_STATE=1）kp/ki/hyst	B：跳变 / 净 / |d|最大 / 反转
P1	4/1/0	259 / 126 / 12 / 399	**2/3/0**	**126 / 126 / 5（max 11）/ 0**
P2	4/0/0（= v7 默认纯 P）	126 / 126 / 5 / 0	2/0/0	126 / 126 / 5（max 7）/ 2
P6	4/1/1（迟滞 1）	286 / 126 / 12 / 399	4/1/1（迟滞 1）	126 / 126 / 6（max 45）/ 12

- **P0 两列直接证明根因**：只开门控（A→B 同为 kp4/ki3）能把跳变数从 482 降到 126，但 |d| 反而变大（115、max 3331 LSB ≈ 1.8°）、仍有 72 次方向反转 ⇒ **门控是"降增益"而不是"除噪"**：门控后环路的等效带宽下降，PI 系数必须同步放大/重配。
- **定案 = B-P1：`GATE_STATE=1` + kp=2 / ki=3**（跳变数 = 理想、|d| 最大仅 11 LSB、**0 次方向反转**，四项指标同时最优）。仅开门控（B-P0）、门控+kp4/ki3 都不够；B-P2（kp2/ki0）仍有 2 次反转且失去 I 项的零滞后。

落地清单（4 个文件，均已改、未重综合）：

文件	改动
`rtl/enc_top.v:532-542`	`u_filter` 例化新增 `.GATE_STATE(1)`（长注释记录理由、根因链、配套 kp2/ki3，并说明**模块默认仍保持 0**，因为 `sim/tb_loop_filter.v` 与 `model/loop_filter_model.py` 依赖"每 clk 更新"的逐位契约）
`rtl/enc_top.v:107-123`	`RTL_BUILD_ID` `16'h0007` → **`16'h0008`**，并列出 v8 三项内容（① `GATE_STATE=1`；② kp/ki 复位默认改 2/3；③ 含本工作树已改但 v7 位流不含的 lag 阈值 (b) 重标定 + ABZ 过速钳位 `r_ovl`）；顶部注释 `:33-35` 同步
`rtl/loop_filter.v:71-82`	`GATE_STATE` 参数注释整段改写：默认**保持 0**，生产例化自 v8 起显式传 1；说明门控只影响 `pos` / `integ` 状态更新、**不改 `out_valid` 脉冲数**（§1.42 复核订正）
`rtl/reg_file.v:280-294`	`r_filt_k` 复位值 `8'h04` → **`8'h1A`**（kp=2 / ki=3）+ 历史与新依据注释；`:74` 寄存器表默认值列 `0x04` → `0x1A`；`:176`、`:535` 版本注释补 v8

★ 对 v7 口径的取代（务必按 v8 读文档）：v7 的"**开机即纯 P**（`0x0302` 复位 `0x04`，kp=4/ki=0）"是 §1.40 在"门控未开"前提下的最优解，**自 v8 起被本节的 kp=2/ki=3 + 门控取代**：- `docs/04_wiring_checklist.md:334`、`docs/DEV_STATUS.md` §1.40/§3 item 17 等处"写 `0x0302 = 0x0004`"的口径：**v8 上仍可运行期写 `0x0004`（纯 P）作为对照档**，但**开机默认已是 `0x1A`**，不要再把"纯 P"当作推荐默认。- v6/v7 位流（已归档 `_synth_work/pre_v8/`）与本文档的历史实测数字**都仍然有效**——它们描述的是"门控关 + 对应增益"的配置，只是不再是默认。

⏭ 未生效 / 未做（2026-10-11 19:30 更新）：① ~~v8 位流尚未重综合~~ → ✅ **已重综合并上板**（`_synth_work/enc_top.bit` 1,075,596 B @19:11:30，已烧录、读回 `0x05F2` = 8 / `0x0302` = `0x1A`，见 §1.45）；② `model/loop_filter_model.py:48` 的 `FS = 50.0e6` 与 `rtl/loop_filter.v:41-56` 的带宽表**仍按"每 clk 更新"标定**，门控后有效更新率降到 12.5 MHz ⇒ 模型与 RTL 在此点上已知不一致（未改，属 §1.42 审计的模型漂移类，该文件已加 ⚠ 说明）；③ ~~`sim/tb_loop_abz_chain.v:20` 的注释仍写"v7 候选"~~ → ✅ **已改为"v8 已采用（B-P1）"**。

1.44 ★★★【真缺陷，v8 已修】集成路径的 Z 脉宽被 `z_window` 门控窗截成 2 clk（40 ns），`z_width` 形同无效（2026-10-11，RTL 已改、未重综合）

现象：`sim/logs/tb_enc_top.log` 的 `FWD: ... Z_pulses=1, Z_width_max=2` —— 无论 v7（`GATE_STATE=0`）还是 v8 工作树，**集成路径**上 Z 脉冲只有 **2 拍 = 40 ns**，而文档（§1.26、`docs/06:293`）声称 `z_width`（`0x0108`，复位 4 个 AB 边沿步）会给出 ≈3.84 µs。

为什么单元测试没抓到：三个"纯 ABZ"测试台都把窗**常开** —— `sim/tb_abz_gen.v:129/307`（`z_window = 1'b1`）、`sim/tb_abz_jitter.v:140`（`.z_window(1'b1)`）、`sim/tb_loop_abz_chain.v:119,127`（`.z_window(1'b1)`）⇒ 在它们内部 `z_window` 恒 1，"推进被窗截断"这条路径**根本不会出现**，`Z 宽度 vs z_width` 的用例一直是假通过。**唯一使用真实 `z_window` 的 `tb_enc_top` 从不检查 Z 脉宽**（旧判据 B5 是 `z_width_max in [1,16]`，2 拍也"通过"）⇒ 这是"文档 + 单元台都不覆盖集成路径"的典型盲区（同类盲区此前也出过：§1.17 标定窗口脉冲接错、§1.18 标定不可重复 —— 都是"模块台自己造信号，顶层从不走"）。

根因：`z_window`（`rtl/enc_top.v:615`，`cycle_cnt[POLE_BITS-1:0] == 0`）是一个**电平**，在"装载那一拍"之后 ~2 clk 就关闭（`rtl/pole_counter.v` 的 `cycle_cnt` wrap 有 2 clk 延迟）。而 `rtl/abz_gen.v` 原实现把**全部** Z 逻辑（含"按 `idx_exec` 递减 `r_z_left`"与"`Z_TIMEOUT` 兜底"）写在 `if (z_window && abzen) begin ... end else begin <清全部> end` 里 ⇒ 窗一关，`r_z_active` / `r_z_left` / `r_z_out` 立刻被清 ⇒ 脉宽只剩 2 拍。

修法（`rtl/abz_gen.v`，装载与推进分离）：- 新增组合条件（`:509-517` 一带）：`z_in_win = z_window & abzen`、`z_load = z_in_win & ~z_ph_hi & r_z_prev_hi & ~r_z_active`、`z_step_fin = r_z_active & idx_exec & (r_z_left == 4'd1)`、`z_timeout = r_z_active & (r_z_to == 16'd0)`。- "Z 通道"时序块改为：`if (z_load) <装载：active=1 / left=z_pulse_n / to=Z_TIMEOUT / out=(z_pulse_n!=0)> → else if (r_z_active) begin if (!abzen || z_step_fin || z_timeout) <清全部> else begin <idx_exec 递减 left / 递减 to> end end else <清 out/left/to>`。⇒ **装载仍受窗门控**（每 `2^POLE_BITS` 个输入周期一次，Z 的"每 64 电周期 1 个"语义不变），**脉冲推进与超时兜底在窗外继续**。- 沿检测独立出来：`if (z_in_win) r_z_prev_hi <= (z_ph_hi == 1'b1); else r_z_prev_hi <= 1'b0;`（与原实现等价）。★ 设计不变量：`r_z_prev_hi` 必须**无条件跟随** `z_ph_hi`（不能只在"非脉冲期"更新）—— 否则脉冲收尾后残留 `prev=1` 会在下一拍立刻触发第二个 Z。

验证（集成路径，`sim/logs/tb_enc_top.log`，2026-10-11 晚）：- 新增 **TEST C 的 C5** 判据（TEST C 反向段正好产生 Z 脉冲；TEST B 在本配置下**不产生** Z —— 见下）：`REV Z 脉宽: max=2042 拍` ⇒ `[ok] C5-rev-Z-pulse-width-sane`，`TEST PASSED (106 checks)`。- 理论值 = `z_width`(4) × 每边沿步拍数；本台 inter=32/div=1 ⇒ `4·inter/div = 128` 步/电周期 ⇒ 每步 `65536/128 = 512` 拍 ⇒ **2048 拍**。实测 **2042**（差 6：装载比跨零沿晚 2 拍 + 首/末边沿步的相位量化），修复前是 **2 拍**。- `sim/tb_enc_top.v` 三处同步：① 新增 `zw_rise_cyc` 记录本圈内 `z_window` 首次张开的拍并打印；② **B4 改为相位感知** —— `z_window` 每 64 电周期只开一次，若它直到测量圈最后一个电周期才张开，那一次 Z 会落到**下一圈**（v8 工作树实测 `zw_rise_cyc=4128792`，`4128792 + 65536 > REV_CLKS-1 = 4194303`）⇒ 本圈期望 **0** 个（v7 是 `zw_rise_cyc=0` ⇒ 期望 1，Z#1 在 `cyc=65560`）。**这是测试台"整圈时钟窗"的相位依赖，不是缺 Z**（RTL 侧仍每 64 个正/负跨零恰好 1 个 Z）。③ B5 从 `[1,16]` 收紧为 `if (z_pulse_cnt==1) [1024,4096] else ==0`，以守住本次修复（下界 1024 抓"被窗截断"回归，上界 4096 = `Z_TIMEOUT`）。- 顺带订正一处**日志误导**：`FWD: A_edges=4095 (1st rev 2048, expect 4096)` 里的 `2048` 是**后半圈**计数（`sim/tb_enc_top.v` 在圈末把 `mid_rev_edges` 改成差值），旧文字让人误以为丢沿；显示已改为 `(后半圈 2048, expect 2048)`。

影响面（哪些文档口径因此改变）：`docs/DEV_STATUS.md` §1.26 的"`z_width=4` ⇒ 3.9 µs"表、§1.33 ② 的 `tb_abz_jitter` "Z 宽度 3.84 µs"、`docs/06_board_test_guide.md:293`、`docs/05_vivado_sim_impl_report.md:153` 与 13 项表第 13 条 —— **均已加注"只在窗常开下成立 / 集成路径 v7 实为 40 ns，v8 起才成立"**。★ 与 §1.42 审计的关系：审计第 20 条只指出"注释里的 80 ns 会误导"，**没有发现这条截断**（因为它只看代码与注释，没有跨"单元台 vs 集成台"比对）。

⏭ 未生效：RTL 已改、**v8 位流尚未重综合** ⇒ 手上位流的 Z 脉宽仍是 40 ns。

1.45 ★★★【v8 上板首验】(2026-10-11 19:24)：生产位流烧录 + 读回校验全中 + 有效输入下的毛刺率与 OTR 同源证据

位流与烧录：`_synth_work/enc_top.bit` = **1,075,596 B @19:11:30**（v8 = §1.43 + §1.44 + item 18；综合 0 error/0 critical/52 warning 与 v7 逐字相同、DRC 0 error、Setup **26 失败端点 / WNS −3.274 ns / TNS −84.267 ns**、Hold 0 失败 / WHS +0.048 —— 对照 `_synth_work/pre_v8/impl_timing.rpt` 的 v7 = WNS −3.364 / TNS −85.065 / 26 失败 ⇒ **失败端点集合与数量不变、WNS 略好**）。烧录命令（`ILA_MODE=restore` 只下位流、不加载探针）：

```
$env:ILA_MODE='restore'; $env:ILA_BIT='_synth_work/enc_top.bit'
powershell -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 -Tcl tools\ila_capture.tcl -LogFile _synth_work\flash_v8.log -Tail 40
```

★ **必须一次性** `danger-full-access`：工作区沙箱下 cs_server（PyInstaller 单文件）往 `%TEMP%` 释放 `VCRUNTIME140.dll` 被拒 —— 实测原文 `Unable to launch local cs_server executable.` / `Failed to extract VCRUNTIME140.dll: failed to open target file!` / `fopen: Permission denied` ⇒ `connect_hw_server failed`（`_synth_work/flash_v8.log` 有完整两份日志，第一次失败、第二次提权后成功，19:24:18 `End of startup status: HIGH`）。

读回校验（POR 后默认值，`E:\Anaconda\python.exe tools\uart_spi.py --port COM6 read ...`）：

寄存器	实读	期望	判定
`0x05F2` RTL_BUILD_ID	`0x0008`	8	✅ v8 位流确在板上
`0x0302` FILT_K	`0x001A`	kp=2/ki=3	✅ 门控 + kp2/ki3 默认生效（不依赖主机写）
`0x0500` ST_BITS	22（byte0 = 0x16）	22	✅
`0x0501` MT_BITS	10（byte0 = 0x0A）	10	✅
`0x0101` / `0x0102`	0 / 0	0	✅ 无残留主机写
`0x0106`	`0x0A00`	—	读作
`0x0108`	`0x4004`	—	byte0 = 0x04 ⇒ `z_width` = 4 ✓
`0x053A` STAT_FLAGS	`0x0008`	—	仅 `otr`；`amplo`(bit1)/`amphi`(bit2)/`caldone`(bit4)/`crcerr`(bit6)/`frame_active`(bit7) **全 0**

★ **地址口径（会误读，务必按这个读）**：寄存器是**每地址 8 位**，工具按 16 位显示时低字节 = 低地址、高字节 = 高地址（`0x0500` 显示 `0x0A16` ⇒ 实为 ST 22 / MT 10 两个 8 位寄存器）。16 位量（`0x0505` 等）读**偶地址**得到的拼接值才是正确的整体值。★ **顺带结案清单里的"ST/MT 复位值 26/12"**：`rtl/reg_file.v:286-287` 已是 22/10（`:86-87` 的表同步）、`docs/01_technical_solution.md:619-620` 已写 22/10、`docs/08:204` 已记；`rtl/SPEC_interface_contract.md:433-434` 的 26/12 是 `biss_c_slave` 的**模块默认参数**，顶层 `rtl/enc_top.v:738-739` 用 22/10 覆盖 ⇒ **不需再改。**

输入状态（不是退化输入）：- 幅度：`0x0530-33` STAT_AMP_SQ = `0x26B066A0` ⇒ $/2^{30}$ = **0.6054**。用 §1.23 的锚点换算（10 Vpp 档、标定后读 0.6390、`GAIN_S` = 0x6773 = 0.8082 ⇒ 未标定等效 0.6390/0.8082² = **0.9784**）⇒ 当前输入 ≈ **7.8 Vpp**，与"8 Vpp 口径"相符；窗口 [0.36, 1.44] 内 ⇒ `amplo`/`amphi` = 0 自洽。- 标定状态：`0x0510-0x051A` 全为复位值（OFS 0 / GAIN `0x8000` / PHASE_K 0 / SHIFT 0）、`0x0502` = 0x00（未 done）、`0x053A` bit4 = 0 ⇒ **本次上电没跑标定**，读到的角度是**未标定**口径。

毛刺计数器（有效输入下的三个独立窗口，`0x0505..0x050E`）：

时刻/窗口	`0x0505` gl_1024	`0x0507` ≥11.25°	`0x0509` ≥22.5°	`0x050B` ≥45°	`0x050D` gl_hold
19:24:36.570	154	7	7	1	142
19:24:41.763（+5.19 s）	187（**+33 ⇒ 6.4 次/s**）	9	9	1	175（**+33**）
19:26:24 → 19:26:31（每 2 s 采样 4 次）	930 → 944 → 952 → 963（+14/+8/+11）	—	—	—	878 → 892 → 900 → 916（**+14/+8/+12**）

- 另有 5 个 **1 s 窗口**：`0x0505` 增量 = **+7/+7/+14/+8/+3**（第一轮）与 **+3/+9/+13/+5/+7**（第二轮）。
- ⇒ **≥5.6° 的角度毛刺在有效 8 Vpp 输入下约 6～8 次/s ≈ 0.7% 的输入周期**；≥11.25°/≥22.5° 约 0.4 次/s 量级；≥45° 几乎不出现。
- ⇒ ★ `0x050D` **gl_hold_cnt 的每次增量与 `0x0505` 完全相同**（同一读数遍内 +33/+33、+14/+14、+8/+8）⇒ "毛刺检测 → 角度保持"每次都被真正触发，检测-保持链自洽（个别 +11/+12 的 1 之差来自两寄存器读数之间的时间差）。
- ⇒ ★★ **确认 18:44 那条"1.3 k 次/s"是 `amp_low` 假象**（当时输入退化、小半径被 CORDIC 除出巨大角度噪声）；有效输入下的真值是 **6～8 次/s**，与 §1.37 的 0.00444 次/周期（≈4.4 次/秒 @1 kHz）**同一量级。⇒ §3 item 15 的 D 分支（`angle_raw` 真实毛刺）现在有了有效输入下的定量结论：真实存在、但率很低（~0.7% 周期）。**

OTR 与毛刺同源（率相关证据）：写 `0x0405 = 0x0001`（W1C + 1 拍 `clr_otr`）后**立刻读回 `0x053A` = 0x00** ⇒ 清位真的生效；随后 5 个 1 s 窗口里 **4 次在 1 s 内又置位**（`0x053A` 由 0x00 → 0x08）⇒ **OTR 事件率 ≥ ~1 次/s。** `0x0405` STAT_LATCH 锁存值 = `0x25` = **bit0(oflow/OTR) + bit2(laglim) + bit5(`scamp`)** —— 其中 **bit5 `scamp` 正是 §1.37 ③ 单样本毛刺链里置位的那一位。**★ 结论：**角度毛刺与 ADC 级事件（`scamp`/`otr`）同源**（"采样撕裂/过幅"而非纯数字伪影）这一判断，除 §1.37 的单点原始数据外，又多了**率相关**的板上独立旁证；⚠ 但这是"率同量级"，**不是逐事件同拍相关** —— 要逐拍相关仍需 ILA 或示波器同步观测。

⏭ **剩下**：① stage1 自由运行窗口（**须先建 v8 ILA 位流** —— 盘上 `_synth_work/enc_top_ila.bit` 1,164,160 B @16:22:42 是 **v7 的**，与 `pre_v8/` 归档同字节；直接跑 stage1 会把板子退回 v7 RTL）；② `0x0302` 三档 sweep（`0x1A` 默认 / `0x04` 纯 P / `0x0C` ki=1）对比 AB 跳变与 `angle` 残差；③ ~~Z 脉宽示波器实测（判据 ≈3.9 µs，见 §1.26 表 —— v8 起才成立）~~ ✅ **2026-10-11 已完成**：Z 率 156 Hz 与脉宽均与预期相符，见 §1.26 板上实测块与 §3 item 20。

1.46 ★★【v8 上板 A/B：stage1 自由运行窗口 + `0x0302` 五档 sweep（1 kHz 与 10 kHz 两轮）】(2026-10-11 19:33–19:42)

目的：完成 §3 item 19 剩下的"上板 A/B"——用自由运行窗口核对 v8 默认（`GATE_STATE=1` + kp2/ki3）下的 AB 边沿率是否与 v7 纯 P 一样接近理想，并在**同一块板、同一输入**下比较三档环路增益。

前提：v8 ILA 位流重建（19:33，`tools/ila_debug.tcl`）—— `_synth_work/enc_top_ila.bit` = **1,465,972 B**、`.ltx` = 71,337 B（36 端口 / 269 bit，与 v7 相同）；Setup **26 失败端点 / WNS −3.323 ns / TNS −84.833 ns**、Hold 0 失败 / WHS +0.057（对照 v7 ILA 版 26 / −3.338 / −84.954 ⇒ 同集合、略好）；LUT 9230 (44.38%) / FF 8950 / BRAM 7.5 / DSP 17 / IOB 50（v7 ILA：9189 / 8943 / 7.5 / 17 / 50）。

方法：`ILA_MODE=stage1`（下 ILA 位流、加载探针、按 `adc_valid` 恒 1 立即触发，窗口 1024 拍 = 20.48 µs）抓 v8 默认基线；随后 `ILA_MODE=capture`（**不重下位流** ⇒ 保留寄存器值）配合 `tools/uart_spi.py --port COM6 write 0x0302 <档>` 在**同一块板上**扫三档。所有 Vivado 连板操作都需要一次性 `danger-full-access`（cs_server 释放 `VCRUNTIME140.dll`，见 §1.45）。★ 顺手修了 `tools/ila_capture.tcl` 的一个真 bug：`write_hw_ila_data -csv` **会自己补 `.csv` 后缀**，而脚本原用 `[file size $CSV]` 报大小 ⇒ 每次抓波都在最后一步报 `no such file or directory` 并以 **exit 1** 结束（假失败）。已改为按实际存在的文件报大小。

期望边沿数（先把账算清，避免"看起来"的比值）：`dbg_edge_cnt` 统计的是 **A+B 两相总边沿**（实测 = `dbg_a_out` 翻转 + `dbg_b_out` 翻转，10 + 11 = 21）；RTL 每个电周期产生 `4 × inter_n / div` 个 AB 边沿 —— `rtl/abz_gen.v:147` `INTER_MAX = 10'd256`、`:278` `inter_n = (inter == 10'd0) ? INTER_MAX : inter`，本板 `0x0101/0x0102` 均为 0 ⇒ `inter_n = 256`、`div = 1` ⇒ **1024 边沿/电周期**（实测 993–1042，见下表最后一列，与 1024 相符）。⚠ 注意与 `sim/tb_enc_top.v:102` 的 `EXP_EDGES_REV = 4096`（**A 相单相**，= 总边沿的一半）不是一回事，别混。 ⇒ 20.48 µs 窗口内的期望边沿数 = **1024 × f_in × 20.48 µs。**

表 A：三档 sweep（两次独立抓波；第二次为 19:36:32→19:37:03 背靠背 31 s 内完成，输入更接近相同）

抓波/档	`0x0302`	f_in	半径 r / 等效幅度	`dbg_edge_cnt` 翻转	期望	比值	`angle` 线性残差 RMS	`angle` vs atan2 RMS	实测边沿/电周期
第一次 `ila_cap_v8free`	`0x1A` 默认	1.024 kHz	1586.7 / 7.75 Vpp	**21**	21.5	0.98×	**0.050°**	0.084°	1003
第一次 `ila_cap_v8_purep`	`0x04` 纯 P	0.986 kHz	1635.1 / 7.98 Vpp	**20**	20.7	0.97×	**0.009°**	0.062°	993
第一次 `ila_cap_v8_ki1`	`0x0C` kp4/ki1	0.982 kHz	1633.2 / 7.98 Vpp	**21**	20.6	1.02×	**0.029°**	0.064°	1042
第二次 `ila_cap_v8s_1a`	`0x1A` 默认	0.994 kHz	1626.8 / 7.94 Vpp	**21**	20.8	1.01×	**0.050°**	0.086°	1033
第二次 `ila_cap_v8s_04`	`0x04` 纯 P	1.018 kHz	1593.3 / 7.78 Vpp	**21**	21.3	0.98×	**0.009°**	0.077°	1006
第二次 `ila_cap_v8s_0c`	`0x0C` kp4/ki1	1.015 kHz	1594.3 / 7.79 Vpp	**21**	21.3	0.99×	**0.026°**	0.063°	1016

（每次抓波 `dbg_dec_valid` 都是 256/1024 = 1/4.00，与 `LOG2DEC = 2` 一致；`dbg_stat_byte` = 0x00 或 0x01、`lag_warn` / `lag_fatal` / `limiter_active` / `uvw_freq_warn` / `wrap_*` / `z_out` 全 0；`dbg_cordic_mag` 恒定、`hypot(s_raw,c_raw)` std ≈ 2 LSB ⇒ 位序正确、信号健康。）

结论 ①（item 19 的目标，✅ 达成）：v8 默认（`GATE_STATE=1` + kp2/ki3）在自由运行下的 AB 边沿率 **21 次 / 20.48 µs ≈ 1.0× 理想**（两次 0.98×/1.01×），与 v7 纯 P 的 21 次（§2.8 口径）**相同** ⇒ **门控 + 新的 kp/ki 默认没有破坏 AB 边沿率。**这正是要证的那一条。

结论 ②（⚠ 诚实的不利结果，必须记下来）：**在 1 kHz 自由运行下三档的边沿率完全无法区分**（都 ≈ 1.0×），而"**`angle` 线性残差 RMS"这一指标上纯 P 明显最优**：`0x04` = **0.009°**（两次独立抓波一字不差）、`0x0C` = 0.026/0.029°、`0x1A`（v8 默认）= **0.050°**（约 5.5×）。原始 `angle_raw` 三档同量级（0.041–0.061°） ⇒ 差异是环路本身的平滑能力，不是输入质量。 - ⇒ **v8 默认并不是"1 kHz 自由运行抖动"这一指标上的最优**；它取胜的依据是仿真 A/B 表里的**高速段**（`sim/tb_loop_abz_chain.v`：B-P1 链 126 跳变 / **0 次方向反转**，对照 A-P0 482 跳变 / 816 次反转）以及 **I 型环路的稳态速度滞后**（纯 P 有与转速成正比的位置滞后，`lag_warn` / `lag_fatal` 就是为它设的）。 - ⇒ ⚠ 该结论**不能**反过来判"纯 P 整体更准"：报告里的"线性残差 RMS"**先扣掉了恒定的线性滞后**，纯 P 的系统性速度滞后完全不体现在这个数里。 - ⇒ **判定 v8 默认是否真的更好，必须在更高输入频率下复做三档 sweep**（§1.40 的 PH3 = 20 kHz 工况；本板当前是 ~1 kHz）——这是本轮留下的**最优先未决项**。 - ⇒ **★★ 2026-10-11 追加（§1.48，本结论已被推翻一半）**：换成"稳态速度滞后"判据后，纯 P 在同频的滞后是 **+1.875° @1.018 kHz / +18.259° @9.947 kHz**，而 v8 默认是 **−0.006° / −0.069°（≈0）** ⇒ **"纯 P 残差 RMS 更优"是拟合扣掉滞后造成的指标假象**，v8 的收益**已在板上证实**（见 §1.48 结论 ①②）。

结论 ③（AB 方向分解：六个窗口的逆环边沿全为 0）：用 `dbg_a_out` / `dbg_b_out` 的逐拍电平重建正交序列（`tools/ab_direction.py`），把每个边沿判成"顺环/逆环"——

窗口	`0x0302`	总边沿	顺环	逆环	非法（跨 2 步）
`ila_cap_v8free` / `v8s_1a`	`0x1A` 默认	21 / 21	21 / 21	**0 / 0**	0 / 0
`ila_cap_v8_purep` / `v8s_04`	`0x04` 纯 P	21 / 21	21 / 21	**0 / 0**	0 / 0
`ila_cap_v8_ki1` / `v8s_0c`	`0x0C` kp4/ki1	21 / 21	21 / 21	**0 / 0**	0 / 0

⇒ **1 kHz 自由运行下三档都没有出现"正反成对点火"**（即 §1.37/§1.39 说的 jitter 群）——**动机现象在本工况下根本不出现**，所以这次板上 A/B **既不能证实也不能否证** v8 改动的收益。（`sim/tb_loop_abz_chain.v` 里 A-P0 的 816 次反转是**高速/大滞后工况**才有的。） ⇒ 更加说明必须做 20 kHz 档。

结论 ④（OTR 的独立旁证）：两次 `capture` 窗口里 `dbg_otr_lat` **恒 1**、`dbg_stat_byte` = 0x01 常值；而 19:35:04 的 stage1 窗口里两者**恒 0** ⇒ 那 47 s 内 OTR 又发生并**锁存**（`rtl/enc_top.v:245` 的 `otr_lat` 只在写 `0x0405` 时由 `clr_otr` 清）。期间半径 r = 1593–1635（7.78–7.98 Vpp）**远未削顶**（削顶会贴 ±2048）⇒ **OTR 不是过幅引起的**，更像 ADC 级采样撕裂/单样本跳变 —— 与 §1.37 ③（单点 +2307 LSB、`scamp` 置位）和 §1.45（`0x0405` 锁存 bit5 `scamp`）**第三次独立吻合。**

1.46 续：10 kHz 五档 sweep（19:41:27–19:42:30，`capture` 模式，同一块板同一输入）

为什么选 10 kHz：`inter_n = 256` ⇒ 1024 边沿/电周期 ⇒ 边沿率 = `1024 × f_in`。10 kHz 对应 **10.24 MHz ≈ `filt_valid` = 12.5 MHz 的 82%**，是"最能压出纯 P 的稳态滞后与反转、又不触发边沿率天花板"的工作点；天花板对应的输入上限是 **f_in ≤ 12.2 kHz**（见 §1.42 与本文件 `:1629-1640` 的既有结论）。

先核对输入频率（无需提权）：`tools/uart_scope.py --port COM6 --baud 2000000 --seconds 2 --csv _synth_work/scope_pre10k.csv` ⇒ 19,930 有效帧 / 2 s，CSV 显示每帧 `pos_full` 增量 ≈ 65535、`cycle_cnt` +1 ⇒ 每帧正好一个电周期；`rtl/uart_scope.v:132-133` `SAMPLE_DIV = 5000` ⇒ **帧率固定 10 kHz** ⇒ **f_in ≈ 9.9994 kHz**（与 10.000 kHz 差 ≈0.56 Hz，即用户已把 AFG 设到 10 kHz）。★ 副作用：f_in 恰好等于采样率时该工具的"角度步进/正负步进帧数"统计**频闪退化**，不能当抖动指标看（58/19930 帧 XOR 坏帧、圆度 RMS 0.113% 都正常）。

表 B：10 kHz 五档（含两个"验证 sim 预测"的对照档）

档 `0x0302`	f_in	半径 r / 等效幅度	`dbg_edge_cnt` 翻转	期望	比值	实测边沿/电周期	`angle` 线性残差 RMS	`angle_raw` RMS	`angle` vs atan2 RMS	逆环
`0x1A` v8 默认（门控 kp2/ki3）	9.862 kHz	1635.7 / 7.987 Vpp	**207**	206.8	**1.00×**	1024	0.086°	0.088°	0.097°	**0**
`0x04` 纯 P（kp4/ki0）	9.934 kHz	1631.7 / 7.967 Vpp	**209**	208.4	**1.00×**	1026	0.082°	0.097°	0.109°	**0**
`0x0C` kp4/ki1	9.928 kHz	1622.6 / 7.923 Vpp	**207**	208.2	**0.99×**	1017	0.092°	0.093°	0.110°	**0**
`0x1C` 门控 kp4/ki3	10.189 kHz	1593.1 / 7.779 Vpp	**213**	213.6	**1.00×**	1027	0.113°	0.115°	0.113°	**0**

档 0x0302	f_in	半径 r / 等效幅度	dbg_edge_cnt 翻转	期望	比值	实测边沿/电周期	angle 线性残差 RMS	angle_raw RMS	angle vs atan2 RMS	逆环
0x00 kp0/ki0 = bypass 直通	9.871 kHz	1631.2 / 7.965 Vpp	**208**	207.0	**1.00×**	1028	0.092°	0.092°	0.106°	**0**

（五档 dbg_dec_valid 全 = 256/1024 = 1/4.00； lag_warn / lag_fatal / limiter_active 全 0； 倒退步 **五档全 0 处**； dbg_otr_lat 五档恒 1、 dbg_stat_byte = 0x01 常值； a_out / b_out 翻转各 ≈104/103、208/209 与总边沿自洽。 0x00 是 rtl/loop_filter.v:157,162-165 的模块内 bypass （pos_nxt = ang_ext）⇒ 该档 angle 与 angle_raw 完全相等，正好做了上表"RMS 差 = 环路贡献"的**零点标定**。）

结论 ⑤（10 kHz 下依然全部 1.0×、零逆环：v8 改动在此工况也压不出差别）：五档（含"门控 + kp4/ki3"和"完全直通"两个极端）**边沿数都在理想值 ±1 之内、逆环全部 0** ⇒ 10 kHz（82% 边沿率）仍**不是**能区分环路的工况。§1.39 的 jitter 群（正反成对点火）需要"每拍角度步进 ≫ 环路带宽"的工况，本板 inter = 256 下无法达到（边沿率天花板先到）。

结论 ⑥（⚠ 与 sim 的分歧，如实记录）： sim/tb_loop_abz_chain.v 预测"**门控 + kp4/ki3**"会变成慢而大幅摆动（B-P0： |d| 最大 3331 LSB ≈ 1.8°、72 次方向反转），但板上 0x1C 档**零逆环、边沿数 1.00×**；它是五档里 angle RMS 最差的（**0.113°**，且几乎等于自身 angle_raw 0.115° ⇒ 环路在该档**几乎不做平滑**）。另一条同向的读数：1 kHz 时纯 P 的 0.009° 优势在 10 kHz 时**消失**（0.082° ≈ 直通档 0.092°）。⇒ **"环路残差 RMS"这个指标随频率强烈变化、且扣掉了稳态滞后，不能单独用来选 kp/ki**；sim 的摆动现象在板上没有以 AB 逆环形式出现。 - ⇒ **★★ 2026-10-11 追加（§1.48）**：本结论"既未证实也未否证"**已作废** —— 改用滞后判据后，10 kHz 纯 P 滞后 **+18.259°** vs v8 默认 **−0.069°**，且与解析式 1.8432e-3·f_in 吻合到 1% ⇒ **v8 默认（II 型环路）相对纯 P 的收益已在板上证实。**

★ 20 kHz 档的现实约束（原 ⏭ ① 需要修正）： inter = 256 时 20 kHz 输入要求 **20.48 MHz 边沿率 > 12.5 MHz 天花板**，板级根本跑不上去（会被静默钳在 12.5 MHz，见 :1629-1640 ），所以**不能**直接做"20 kHz 三档 sweep"来复现 §1.40 的 PH3。要在 10…20 kHz 上跑，只有两条正路：① 把 0x0101 = 0x0020 （ inter = 32 ⇒ 128 边沿/电周期，20 kHz ⇒ 2.56 MHz，余量充足）——分辨率降 8 倍但环路动力学是同一个（ rtl/reg_file.v:557-558 cfg_inter = {r_inter1[1:0], r_inter0[7:0]}、 cfg_div = r_inter1[4:2] ）；② 把 LOG2DEC 由 2 降到 0（ filt_valid 50 MHz， f_in 上限 ×4 ⇒ 24.4 kHz），但这需要重综合（属 v10 候选话题）。 ✅ **2026-10-11 两条都已按 ① 执行**（12 kHz 见 1.48 补 2、20 kHz 见 **1.48 补 3**），并且**该"静默钳位"已用 inter = 256 @20 kHz 实测到板上定量证据**：窗内边沿 256 vs 应有 422.7 ⇒ **0.606×**（= 12.5 / 20.64 MHz，丢 39.4%），与 PH3 仿真预测的" 12500/20480 、丢 39%"一致。

⏭ **下一步**：① 若要继续压环路差异，用 0x0101 = 0x0020 （ inter = 32 ）把输入推到 **12 kHz（98% 天花板）或 20 kHz** 再扫五档 + 顺环/逆环分解，并同时读 0x0505/0x0507/0x0509/0x050B/0x050D 毛刺计数器；② ~~**Z 脉宽示波器实测**——★ 判据随 f_in 变：Z 高电平 = z_width (4) 个 AB 边沿间隔 = 4 / (1024 × f_in) ⇒ **1 kHz 时 ≈3.9 µs（§1.26 表值）、10 kHz 时 ≈0.39 µs**~~ ✅ **2026-10-11 已完成**：Z 率 **156 Hz**（10 kHz ⇒ 期望 156.24 Hz）+ 脉宽与预期相符（把 z_width 临时写到 15 ⇒ 期望 1.465 µs，量完已恢复 0x4004 ）—— 判据通式 宽度 = z_width / (1024 × f_in) **（★ v9 起为 (z_width + 1) / (1024 × f_in)，见 §1.49 / §1.26 板上实证块**，本条是 v8 语义下的历史记录），详见 §1.26 板上实测块；③ 本轮证据（§1.46 表 A/表 B + tools/ila_capture.tcl 修复）提交。★ 四个分析脚本 tools/ila_report.py 、 tools/edge_per_cycle.py 、 tools/ab_direction.py 、 tools/lag_probe.py 已于 2026-10-11 **从 .gitignore 排除的 _synth_work/ 移入受版本控制的 tools/** （此前它们不进版本库，是关键分析方法的复现风险），文档引用已同步。

1.47 【规格 vs 实现，★ 2026-10-11 板上二值实验已确认 → ✅ 同日定案按手册修入 v9】Z 脉宽比 TW28 手册公式少 1 个原始边沿步（+1）

怎么发现的：2026-10-11 为了让示波器看清 Z 脉宽、把 0x0108 写到 zwidth=15 ，回头核对"宽度到底该怎么算"时，发现三处文档都按 TW28 手册写成 **(zwidth + 1) 个边沿**，而 RTL 实现是 **z_width 个边沿** —— 差 1。

出处	原文口径
docs/TW28_digest.md:365	zwidth[edges] = (ZERO1.zwidth + 1) / div
docs/TW28_digest.md:786 （第 19 条）	Z pulse width = (zwidth+1)/div AB edges
docs/01_technical_solution.md:332	「宽度 zwidth+1 个边沿」
docs/01_technical_solution.md:596	0x108 ZERO1 [3:0] zwidth ⇒ zwidth_edges = (zwidth+1)/div
本工程 RTL	rtl/abz_gen.v:507 assign z_pulse_n = (z_width == 4'd0) ? 4'd0 : z_width; , :676 r_z_left <= z_pulse_n , :515 z_step_fin = r_z_active & idx_exec & (r_z_left == 4'd1) ⇒ 整脉冲恰好跨 **z_width** 个 idx_exec 间隔

仿真证据与"实现是 z_width 步"一致： sim/logs/tb_enc_top.log 的 C5 段 Z 高电平实测 **2042 拍**，而理论值 z_width (4) × 512 拍/步 = **2048** 拍（若按手册 5 步则应为 2560）⇒ 实测支持"4 步、无 +1 "（差 6 拍是测量端点取法）。

偏差量：默认 zwidth = 4 时脉冲比手册短 **1/5 = 20%**； zwidth = 15 时短 **1/16 = 6.25%**。折算到输出单元同样少 1 个原始步（Z 旁路 post-AB 分频，见 rtl/abz_gen.v:71-74 ，所以" /div "这一层两边一致，差的只是那个 +1 ）。

功能影响（我的判断，供定夺）： - Z 只是索引/零位脉冲，**偏窄**的方向性风险 = 下游若用 A/B 边沿采样 Z，可能漏采；不会造成多计/重计。本工程 J5-7 是直接引脚输出给用户自己接的计数器，我们的脉冲更窄 ⇒ 对方需要的采样率更高。 - 但这是**规格 vs 实现的偏差**，不是"测不出来"：竞赛报告里若写"Z 脉宽 = zwidth 个边沿"就必须同时说明与手册 zwidth+1 不同；否则等于把偏差藏着。 - 反向看： zwidth = 0 时 RTL 输出无 Z （ z_pulse_n = 0 ），而手册 (0+1) = 1 个边沿 —— 0 值语义两边也不同（这一条 RTL 侧的"0 ⇒ 无 Z"是 §1.26 有意为之，因为早期版本 0 值曾导致"永不输出"）。

两条出路（★ 2026-10-11 用户已选第 1 条，见 §1.49）： 1. ✅ **已采用：按手册修（v9）**： z_pulse_n = z_width + 1 ；⚠ r_z_left 原为 reg [3:0] （最大 15）⇒ 已加宽到 Z_PULSE_W = 5 位表示 16 步；同时已改 sim/tb_abz_gen.v （判据与循环改为"宽度 = z_width + 1 个边沿"、 zw 扩到 0…8）、 sim/tb_enc_top.v （C5 注释 + **B5 收紧为 [2304,2816] 拍**，可同时拒绝 2048 与 2560 之外的截断值）、 sim/tb_abz_jitter.v 、 model/abz_uvw_model.py （跨零拍不计入步数 ⇒ 表 2b 实测 1…9 = z_width+1 ），然后重综合（v9 位流 RTL_BUILD_ID = 0x0009 ）。 2. **（未采用）声明为有意偏差**：在 rtl/SPEC_interface_contract.md 与报告里明确写"本项目 Z 宽度 = zwidth 个原始边沿步（比 TW28 手册少 1）"。

⏭ **定案（2026-10-11）**：走第 1 条 ⇒ v9 重综合 + 上板复测 Z 脉宽；v8 位流保留在板上直到 v9 位流建成并读回 0x05F2 = 9 。

✅ **2026-10-11 板上定案（用下面第 1 个测试：零宽测试，二值判据）**

- **测试条件（实测锁定）**：AFG 1 kHz（板上 UART 侧独立核对：`角度步进 +36.166°/帧` ⇒ `f_in ≈ 1004.6 Hz`，正步进 19907 / 负步进 0）、`0x0101 = 0x0000`（`inter_n = INTER_MAX = 256`）、`0x0102 = 0x0000`（`div_n = 0` ⇒ `div = 1`）、`0x0109 = 0x0040`（`abzen = 1`、`zpol = 0`）、示波器探头在 `J5-7`（`z_out`，`M21`，`constr/enc_top.xdc:267`）。
- **结果**：写 `0x0108 = 0x4000`（`z_width = 0`）⇒ **Z 线上一个脉冲都没有**；写回 `0x0108 = 0x4004`（`z_width = 4`）⇒ **15.625 Hz、3.906 µs 的窄脉冲立即回来**（`4/(1024 × 1 kHz) = 3.906 µs`）。A/B 两侧都由用户在同一探针位置上确认（`0x0108`/`0x0109` 每次写后回读校验）。
- **判读**：本实现 `z_pulse_n = (z_width == 4'd0) ? 4'd0 : z_width` ⇒ `z_width = 0` 时无 Z；而手册公式 `(zwidth + 1)/div` 在 `div = 1` 下应给出 **1 个边沿步**的脉冲。**两者语义不同，板级二值实验与仿真证据（C5 2042 拍 ≈ 4×512）一致 ⇒ 由"设计疑点"升级为"已确认的规格偏差"。**
- ⚠ **旁支（如实记录，不作证据）**：测试中在 `z_out` 读到 ~500 mV / **50.61 MHz** 的正弦，经"写回 `0x4004` 后脉冲立刻回来"确认那是**探针拾取**（频率来源是板上 50 MHz 系统时钟与 `ad_clk = B18` / `ad_clk2 = F16`，见 `constr/enc_top.xdc:91-92`），**不能用来推断板上时钟频率**；旁证：10 kHz 输入时 Z 率实测 156 Hz，若时钟真是 50.61 MHz 则应为 158.1 Hz ⇒ 与 50.00 MHz 相符。
- ✅ **用户决定（2026-10-11）**：选 **① 按手册修（v9）**，顺序上"**先做 v9 的 Z 修复**"（`inter = 32` 重扫 12/20 kHz 押后）。实施与验证记录见 **§1.49**；板上寄存器已恢复 `0x0108 = 0x4004`、`0x0109 = 0x0040`。

★ 三个判决性实测（示波器，任选其一即可定案；不需要重下位流）

> ⏭ **本节已失效（2026-10-11 21:37 补注）**：这三条是**做决定之前**的候选方案；用户已选 ① 按手册修（见上一段的"✅ 用户决定"），且 v9 的 Z 语义**已用 ILA 在板上直接量出**（`z_width = 4` ⇒ 5 个边沿步；`z_width = 0` ⇒ 1 个边沿步，见 §1.26 板上实证块与 §1.49）。保留原文只作**决策过程的记录**，其中的"本实现语义"一律指 **v8 及以前**。若还要示波器数据，只剩"给报告补一张带游标截图"的作用。
>
> 共同前提：写 `0x0108` 时**高字节 `0x40` 必须带上**（保住 `0x0109[6] = abzen`，见 §1.26 的陷阱条）；每个测试做完都写回 `0x0108 = 0x4004`。⚠ **156.25 Hz 这个读数不能用于本判定**：它是 Z 的**重复率** `f_in/64`（`POLE_BITS = 6`，见 §1.26 板上实测），与脉宽是两个互不相关的参数 —— 改 `z_width` 0…15 它都不变。它能证明的是"每 64 个输入周期出 1 个 Z"（`docs/06_board_test_guide.md:543` 判据 5）与探针位置正确。

1. **零宽测试（最干净，不依赖精确读数、不依赖带宽/时基）**：写 `0x0108 = 0x4000`（`z_width = 0`）⇒ 本实现 `z_pulse_n = 0`，**Z 应当完全消失**；若实现的是手册语义（`zwidth + 1`）则仍会出现 1 个边沿步宽的窄脉冲。**"有/无"是二值判据**，与频率误差无关。
2. **比值法（免时基/频率误差）**：同一 `f_in` 下分别量 `z_width = 15` 与 `z_width = 7` 的脉宽（依次写 `0x0108 = 0x400F`、`0x4007`）取比值 ⇒ 本实现 `15/7 = 2.143`，手册 `16/8 = 2.000`（差 7%，游标量比值即可，与绝对时基无关）。
3. **低频直读（1 kHz，游标好读）**：AFG 设 1 kHz + `0x0108 = 0x400F` ⇒ 本实现 **14.65 µs**、手册 **15.63 µs**（差 0.98 µs）。⚠ 10 kHz 下两者只差 **98 ns**（1.465 vs 1.563 µs，6.3%）——**2026-10-11 那次"脉宽读数没问题"只是定性确认，不足以区分这两者**，若要写进报告必须补具体读数或改用上面两个测试。

1.48 ★★【重要：补齐 item 19 的决定性证据】环路稳态速度滞后实测 —— 纯 P 滞后正比于 f_in（1.88° @1 kHz、18.26° @10 kHz），含 I 项 ≈ 0

为什么之前没看见：`tools/ila_report.py` 的 [4] 段本来就算了 `err = unwrap(angle) - atan2(s,c)`，但紧接着一句 `err -= np.mean(err)` 把**均值（正是滞后本身）扣掉了**，只打印残差 RMS ⇒ §1.46 表 A/表 B 用的"线性残差 RMS"判据对滞后**完全不敏感**，于是五档 sweep 得出"区分不出档位"。

方法：新增 `tools/lag_probe.py`（2026-10-11，已移入版本控制的 `tools/`）—— 把 `dbg_angle_raw` 与 `dbg_angle` 各自解卷绕成相位后取**逐拍差值的均值**（= 常数滞后），并同时给出 std 与窗内漂移以证明它是"常数偏移"而不是"还在漂"。

判据公式（可手算）：纯 P（`ki = 0`，一阶 / I 型系统）对恒速输入存在稳态滞后 `lag = Δθ_每样本 / Kp`，其中 `Δθ_每样本 = 360°·f_in / f_update`；v8 起 `GATE_STATE = 1` ⇒ `f_update = 12.5 MHz = filt_valid`，`kp = 4` ⇒ `Kp = 2^-6 = 1/64` ⇒ **`lag[deg] = 1.8432e-3 × f_in[Hz]`（= `f_in / 542.5`）**。`ki ≥ 1` 时是 II 型系统 ⇒ 恒速稳态滞后 **0**，只剩流水线/量化（实测 |lag| ≤ 0.2°）。

实测（全部来自既有 CSV，无新硬件动作；`lag_raw` = mean(unwrap(`angle_raw`) − unwrap(`angle`))，>0 = 滤波后角度滞后）

档	`0x0302`	f_in	lag_raw	std	窗内漂移	纯 P 公式预测
v8 默认（门控 kp2/ki3）	`0x1A`	0.994 kHz	**−0.006°**	0.080	−0.006	—
纯 P（kp4/ki0）	`0x04`	1.018 kHz	**+1.875°**	0.062	+0.005	1.88° ✅
kp4/ki1	`0x0C`	1.016 kHz	**−0.004°**	0.050	−0.003	—
v8 默认	`0x1A`	9.863 kHz	**−0.069°**	0.129	+0.005	—
纯 P	`0x04`	9.947 kHz	**+18.259°**	0.135	−0.076	18.33° ✅
kp4/ki1	`0x0C`	9.928 kHz	**−0.067°**	0.137	+0.000	—
门控 kp4/ki3	`0x1C`	10.184 kHz	**−0.070°**	0.148	−0.008	—
bypass（kp0/ki0）	`0x00`	9.868 kHz	**+0.213°**	0.127	+0.004	—
v8 自由运行（复位默认 `0x1A`）	—	1.025 kHz	**−0.005°**	0.081	−0.017	—
v7 自由运行（复位默认纯 P、★ 无门控）	—	0.988 kHz	**+0.474°**	0.041	−0.008	—

结论 ①（★ item 19 的决定性证据，2026-10-11）：v8 默认（`GATE_STATE=1` + kp2/ki3）的稳态速度滞后 **≈ 0**（−0.006° @1 kHz、−0.069° @10 kHz），而**纯 P 同频为 +1.875° / +18.259°**，与公式预测（1.88°/18.33°）吻合到 ~1% ⇒ **§1.46 结论 ② 里"1 kHz 下纯 P 的 0.009° 优于默认 0.050°"是指标假象**：线性拟合把 1.88° 的恒速滞后吸收进了斜率/截距。纯 P 的真实代价是**位置输出滞后真值 1.88°（1 kHz）/ 18.26°（10 kHz）**，折算到 20 bit 单圈约 **5.5k / 53k count** ⇒ "加 I 项（ki≥1）"不是可选优化，而是**速度精度必需**；v8 相对 v7 默认的收益因此**已在板上证实**（不是靠仿真）。

结论 ②（意外收获：独立证实 v8 门控生效且量级正确）：v7 自由运行是**复位默认的纯 P 且无门控**（50 MHz 更新）⇒ 预测滞后应为 v8 门控同工况的 1/4 = 0.47°，实测 **+0.474°**（对照 v8 门控纯 P 1.875° = **1/3.96**）⇒ 与"`GATE_STATE` 把有效更新率 50 MHz → 12.5 MHz、环路每样本增益 ×1/4"**定量吻合。★** 反过来说：门控本身让纯 P 的滞后变差 4 倍 —— 这正是 v8 必须把 kp/ki 成套改成 kp2/ki3（II 型）的原因，两条改动是**一套**，不能只上其中一条。

结论 ③（对照档全部 ≈0）：`0x0C`（kp4/ki1）−0.004/−0.067°、`0x1C`（门控 kp4/ki3）−0.070°、`0x00`（bypass）+0.213° @10 kHz（bypass 无环路，只剩量化与相位口径，±0.2° 量级）⇒ **滞后由"有没有 I 项"唯一决定**，与 kp（阻尼）无关，符合 I / II 型系统理论。

结论 ④（`lag_warn` 全 0 也是对的，不是漏报）：把滞后换算成边沿步 = `lag / (360°/(4·inter))`；10 kHz 纯 P = 18.26°/0.3516° = **52 步**，而阈值是 `inter/2 = 128 步`（= 45°）与 `2·inter = 256 步`（= 180°）⇒ 正确不触发 ✅（阈值以 inter 折算，任何 inter 下都仍是 45°/180°，见 §1.33/item 18）。

⏭: ① 若要在板上把 `lag_warn` 压出来：纯 P 需要 `f_in ≥ 45° / 1.8432e-3 ≈ 24.4 kHz`，而 `inter = 256` 时该频点边沿率 25 MHz **超 12.5 MHz 天花板** ⇒ 必须先用 `0x0101 = 0x0020` 把 `inter` 降到 32（128 边沿/电周期）再做（这正是下一轮 sweep 的做法）；② 下一轮 sweep 的**主判据应改成"滞后"，RMS 只作辅助**；③ 脚本 `tools/lag_probe.py` 已随本次改动纳入版本库。 ✅ **①② 已于 2026-10-11 执行**（`inter = 32`，12 kHz 见 **1.48 补 2**、20 kHz 见 **1.48 补 3**）：四点纯 P 滞后 1.875/18.259/22.380/36.884° ⇒ 归一化斜率一致到 0.4%；`lag_warn` 未在 `inter = 32` 下触发（20 kHz 纯 P 真值 36.9° < 45°），但在 `inter = 256` @20 kHz 因**丢步饱和**而 `lag_warn`/`lag_fatal` 全窗为 1（钳位判决性演示）。

1.48 补：滞后的完整分解 —— 总滞后 = 流水线延迟 + 环路速度滞后（两者都正比于 f_in）

上面量的是 `angle_raw` 与 `angle` 之差（= 纯环路部分）。`tools/ila_report.py` 的 [4] 段现在还会打印 `mean(unwrap(angle) − unwrap(atan2(s_raw,c_raw)))` —— 它包含**整条链路**（抽取/CORDIC/校正/滤波）的延迟。两项一起看，滞后可以精确分解：

总滞后（= mean(atan2 − angle)，>0 = 输出滞后于模拟真值）

采集	档	f_in	总滞后实测	拆解预测（流水线 0.52 µs + 环路项）	差
`ila_cap_v8s_1a`	`0x1A` 默认	0.994 kHz	**0.188°**	0.186 + 0	+0.00
`ila_cap_v8s_0c`	`0x0C`	1.016 kHz	**0.193°**	0.190 + 0	+0.00
`ila_cap_v8s_04`	`0x04` 纯 P	1.018 kHz	**2.072°**	0.190 + 1.876	+0.01
`ila_cap_v7free`	v7 默认纯 P、**无门控**	0.988 kHz	**0.666°**	0.185 + 0.455	+0.03
`ila_cap_v8free`	`0x1A` 默认	1.025 kHz	**0.195°**	0.192 + 0	+0.00
`ila_cap_v8k10_1a`	`0x1A` 默认	9.862 kHz	**1.850°**	1.845 + 0	+0.01
`ila_cap_v8k10_0c`	`0x0C`	9.928 kHz	**1.865°**	1.857 + 0	+0.01
`ila_cap_v8k10_1c`	`0x1C` 门控 kp4/ki3	10.184 kHz	**1.908°**	1.905 + 0	+0.00
`ila_cap_v8k10_00`	`0x00` bypass	9.868 kHz	**2.133°**	1.846 + 0	**+0.29** ⚠
`ila_cap_v8k10_04`	`0x04` 纯 P	9.947 kHz	**20.190°**	1.861 + 18.334	−0.01

- **流水线延迟 τ = 0.52 µs**（两条独立频率上解出 525 ns @0.994 kHz 与 521 ns @9.862 kHz，一致到 1%），贡献 **0.19°/kHz**：这是**所有档共有**的、与 kp/ki 无关的绝对滞后。构成估算：CORDIC 19 clk = 380 ns（`rtl/cordic_atan2.v`，主项）+ ADC 捕获/抽取/校正/毛刺保持/环路 3 级 ≈ 140 ns。★ 推论：`LOG2DEC` **2→0 的路线（审计建议 (c)，§1.48 曾暂称"v9 路线"，现改称 v10 候选以免与 v9 = Z 宽度修复混淆）只能削掉抽取的那几十 ns，对这条滞后几乎无用**（主项在 CORDIC 的 19 拍），选 (c) 的理由应是"提高 `filt_valid` 上限/修正 I 项增益"，不是降滞后。
- **环路项**：只有 `ki = 0`（纯 P，Ⅰ型）才有，`1.8432e-3·f_in`（门控后）；未门控（v7）÷4。
- ⚠ **唯一偏差行**：`0x00`（bypass）实测 2.133° vs 预测 1.846°，多 **0.29°（≈78 ns ≈ 4 clk）**。方向是"直通反而更慢"，与"少一级滤波"的直觉相反；未深究（可能是直通支路的截断/寄存路径不同），如实记录，不影响上面的结论。
- ⇒ 由此得到一句可以直接写进报告的话：**10 kHz 输入下 v8 默认档的位置输出滞后真值 `1.85°`（全部来自 0.52 µs 流水线），而纯 P 是 `20.19°`（其中 18.33° 是环路速度滞后）**；1 kHz 下分别是 `0.19°` 与 `2.07°`。

1.48 补 2：`inter = 32` 高频重扫（12 kHz）—— 滞后判据在 12 kHz 上把三档干净分开（纯 P +22.38° vs 预测 22.50°）+ `inter = 256` 天花板对照

为什么做：§1.48 的 ⏭① —— 要压出 `lag_warn`（45°）纯 P 需 `f_in ≥ 24.4 kHz`，而 `inter = 256` 时该频点的 AB 边沿需求 25 MHz **超 12.5 MHz 天花板** ⇒ 必须先把 `inter` 降到 32（128 边沿/电周期）。本轮同时回答"12 kHz 下 `inter = 256` 是否已被钳位"。

板上流程（v9 ILA 版位流，全程同一块板同一输入）：`ILA_MODE=restore` + `ILA_BIT=_synth_work\enc_top_ila.bit` 烧 ILA 版（`End of startup status: HIGH`）⇒ 写 `0x0101 = 0x0020`（回读 `0x0101` = 0x0020 / `0x0102` = 0x0000 ⇒ **inter = 32、div = 1**）⇒ AFG 由用户手动设到 12 kHz ⇒ `tools/sweep_inter32.ps1 -Tag v9k12_i32 -SettingsCsv 04,1A,00`（逐档写 `0x0302` + `ILA_MODE=capture` 抓 1024 点 = 20.48 µs）。

判据表（主判据 = 环路速度滞后；`lag_raw` = mean(unwrap(`angle_raw`) − unwrap(`angle`))，>0 = 滤波后角度滞后）

采集（`_synth_work/`）	`0x0302`	含义	f_in	lag_raw	std	窗内漂移	纯 P 公式预测	边沿（顺/逆/非法）	edges/cyc（标称 128）
`ila_cap_v9k12_i32_4.csv`	`0x04`	纯 P（kp4/ki0）	12.204 kHz	**+22.380°**	0.186	−0.266	**22.50°** ✅（差 0.12° = 0.5%）	32 / 0 / 0	128.20
`ila_cap_v9k12_i32_1A.csv`	`0x1A`	门控 kp2/ki3	11.859 kHz	**−0.081°**	0.154	+0.001	—（Ⅱ型 ⇒ 0）	31 / 0 / 0	127.60
`ila_cap_v9k12_i32_0.csv`	`0x00`	bypass	11.917 kHz	**+0.258°**	0.153	−0.003	—	31 / 0 / 0	126.93
`ila_cap_v9k12_i256_1A.csv`	`0x1A`	**inter = 256 对照**	11.961 kHz	−0.084°	0.157	+0.002	—	250 / **0** / 0	**1023.62**（标称 1024）

- **结论 ①（滞后判据在 12 kHz 上判决性成立）**：纯 P 实测 **+22.380°**（f_in = 12.204 kHz）vs 公式 `1.8432e-3 × f_in = 22.50°`，**差 0.12°（0.5%）**；Ⅱ型（0x1A）−0.081°≈0、bypass +0.258° ⇒ 三档被**干净分开**，且与 §1.48 的 1 kHz（1.875°）/10 kHz（18.259°）构成**三点线性的 f_in ∝ lag 曲线**（1.018/9.947/12.204 kHz → 1.875°/18.259°/22.380°，归一化 1.842/1.835/1.834 ×10⁻³ ⇒ 三频点一致到 0.5%）。★ 这是"v8 门控 + kp2/ki3 Ⅱ型"这条改动的第二条独立板上证据。
- **结论 ②（12 kHz 下 `inter = 256` 没有被钳位）**：`dbg_edge_cnt` 窗内翻转 **250**，期望 `1024 × 11.961 kHz × 20.48 µs = 250.7` ⇒ **1.00×**；`edges/cyc` = **1023.62**（标称 1024）；**逆环边沿 0 / 非法 0** ⇒ 尽管边沿需求 12.24 MHz ≈ **98% 天花板**（`filt_valid` = 12.5 MHz），ABZ 生成器仍一步不落。⚠ 因此"必须降 `inter`"在 12 kHz **不能**作为板上证据 —— 要演示钳位需 **20 kHz（`inter = 256` 需求 20.48 MHz ≫ 12.5 MHz）**。
- **结论 ③（★ 毛刺计数器的绝对值不可用，只能看差值）**：本会话中 `0x0505` = 26538 而 `0x0507` = 40810 —— 高档位计数**大于**低档位计数，这在 RTL 上是**不可能**的（`rtl/enc_top.v:535-538` 四个计数由同一个 `ang_abs_delta` 与递增阈值产生，必然 `C1024 ≥ C2048 ≥ C4096 ≥ C8192`）⇒ 唯一自洽解释是 **16 位回绕**（`C1024` 至少回绕 2 次、`C2048/4096` 各 1 次，对应本会话早前某段大幅度扰动/无信号期间的大量跳变）。**⇒ 绝对值不带时间戳、还会回绕，只能按两次读数**

之差使用（这正是 `rtl/enc_top.v:512-515` 与 `tools/diag_counters.py` 的设计口径）。用 `tools/diag_counters.py --port COM6 --seconds 20`（12 kHz、`0x0302` = 0x1A、inter = 32）实测稳态密度：**`gl_1024`（>5.6°）= 0.00121 次/周期**（Δ53 / Δ周期脉冲 43853）、**`gl_2048` / `gl_4096` / `gl_8192` 全 0**、`gl_hold_cnt` = 0.00121 ⇒ 跳变幅度**全部落在滞回带（11.25°）以下**，即 v6 防线（`rtl/enc_top.v:469` `ANG_GLITCH_MAX = 1024`）的区间内，与 §1.31/§1.32 的结论一致。★ 旁证：`Δ周期脉冲` 在 20 s 内也回绕过一次（25420 → 3737，实差 43853），工具按 16 位无符号求差是对的。

- **工具/坑（本轮踩到并已修）**：`tools/sweep_inter32.ps1` 的参数改为 **`-SettingsCsv`（逗号分隔的单个字符串，脚本内部 `-split '[,\s]+'`）** —— 原来的 `[string[]]$Settings` 在 `powershell.exe -File` 下**不做逗号数组解析**，`-Settings 04,1A,00` 被当成一个字符串 ⇒ `$reg` 变成 `0x004,1A,0` ⇒ `tools/uart_spi.py` 抛 `ValueError: invalid literal for int() with base 0`（**寄存器没写、却继续抓波**，是最危险的一类静默错）。★ 另一层坑：即便用 `-SettingsCsv 04,1A,00`，**PowerShell 会把裸 `04` / `00` 当数值**，传进脚本变成 `4,1A,0`（写入仍正确，因为 `0x004` / `0x000` 可解析，但 CSV 文件名丢了前导零）⇒ 调用时**给值加引号**：`-SettingsCsv "04,1A,00"`。
- ⏭ **后续**：结论②要求的 20 kHz 钳位演示**已做**，见 **1.48 补 3**。

1.48 补 3：`inter = 32` 高频重扫（20 kHz，五档）+ ★★ `inter = 256` 的钳位判决性演示（只能给出 60.6% 的边沿、丢 39.4%，`lag_warn` / `lag_fatal` 全窗为 1）

为什么做：1.48 补 2 结论②说"12 kHz 下 `inter = 256` 需求才 98% 天花板、钳不住"⇒ 要演示"降 `inter` 的必要性"必须上 **20 kHz**（`inter = 256` 时需求 `1024 × 20 kHz = 20.48 MHz ≫ 12.5 MHz`）。

板上流程（v9 ILA 版位流，全程同一块板同一输入，一次提权跑完）：`ILA_MODE=restore` 烧 `_synth_work/enc_top_ila.bit` ⇒ 写 `0x0101 = 0x0020`（inter = 32、div = 1）⇒ `tools/sweep_inter32.ps1 -Tag v9k20_i32 -SettingsCsv "04,0C,1A,1C,00" -Depth 1024 -WaitMs 30000`（**5 版全成功**，`SWEEP END ok=5 bad=[]`）⇒ 写 `0x0101 = 0x0000`（回默认 inter = 256；注意该 16 位写同时把 `0x0102` 清零 ⇒ div = 1）⇒ `-Tag v9k20_i256 -SettingsCsv "1A,04"`（2 版，`ok=2 bad=[]`）。AFG 由用户手动设到 20 kHz（实测 19.78–20.32 kHz）。

判据表（主判据 = 环路速度滞后 `lag_raw`，口径同 1.48 补 2；`edges/cyc` 标称 = `4×inter/div`）

采集（`_synth_work/`）	`0x0302`	inter	f_in	lag_raw	std	纯 P 公式预测	边沿（顺/逆/非法）	edges/cyc
`ila_cap_v9k20_i32_04.csv`	`0x04`	32	20.073 kHz	**+36.884°**	0.393	**37.00°** ✅（差 0.12° = 0.3%）	53 / **0** / 0	129.3（128）
`ila_cap_v9k20_i32_0C.csv`	`0x0C`	32	20.213 kHz	**−0.137°**	0.256	—（II 型 ⇒ 0）	52 / 0 / 0	127.5
`ila_cap_v9k20_i32_1A.csv`	`0x1A`	32	20.094 kHz	**−0.142°**	0.257	—	52 / 0 / 0	126.3
`ila_cap_v9k20_i32_1C.csv`	`0x1C`	32	19.784 kHz	**−0.138°**	0.258	—	52 / 0 / 0	125.9
`ila_cap_v9k20_i32_00.csv`	`0x00`	32	20.319 kHz	**+0.437°**	0.255	—	53 / 0 / 0	129.0
`ila_cap_v9k20_i256_1A.csv`	`0x1A`	**256**	20.152 kHz	−0.141°	0.257	—	**256** / 0 / 0	**620.32**（标称 1024 ⇒ **0.606×**）
`ila_cap_v9k20_i256_04.csv`	`0x04`	**256**	20.036 kHz	**+36.831°**	0.374	36.93° ✅	**256** / 0 / 0	**623.13**（标称 1024 ⇒ **0.608×**）

- **结论 ①（滞后判据在 20 kHz 上仍成立，且构成四点曲线）**：纯 P 滞后 **+36.884° @20.073 kHz**（预测 37.00°，差 0.3%），与 1.018 / 9.947 / 12.204 kHz 的 **+1.875° / +18.259° / +22.380°** 合并 ⇒ **四点归一化斜率 1.842 / 1.835 / 1.834 / 1.838 ×10⁻³ °/Hz，一致到 0.4%**；II 型（0x0C/0x1A/0x1C）在 20 kHz 仍 **−0.137…−0.142°≈0**，bypass `0x00` **+0.437°**（略高于 12 kHz 的 +0.258°，同为 `in_valid` 门控外的残余）。⇒ **"v8 门控 + kp2/ki3 II 型 vs 纯 P"的收益已在 1 kHz–20 kHz 全程板上证实**（item 19 的决定性证据链闭合）。
- **结论 ②（★★ 钳位判决性演示：`inter = 256` 在 20 kHz 只能给出 60.6% 的边沿、丢 39.4%）**：窗内 `dbg_edge_cnt` 翻转 = **256**（= `dec_valid` 窗内有效样本数，即**每个有效样本都点了一次火、一步都挤不出来**），而按 `4×inter_n/div × f_in × T_win` 应有 **422.7**（`1024 × 20.152 kHz × 20.48 µs`）⇒ **0.606×**；`edges/cyc` = **620.32**（标称 1024）⇒ 同一比例 0.606×。★ 这两条同时说明**上限就是 `filt_valid` = 12.5 MHz**：需求 = `1024 × 20.152 kHz = 20.64 MHz`，`12.5 / 20.64 = 0.606` **恰好等于实测比例** ⇒ 生成器已在满载输出、丢的 39.4% 是**结构上限**而非实现缺陷（`rtl/abz_gen.v:435` 的整个输出时序块被 `if (in_valid)` 门控 ⇒ 每个有效样本最多点火一次）。**这与 §1.28 PH3 的 RTL 仿真预测完全一致**（该表写"20.000 kHz / 20.48 MHz / `12500 / 20480`（**丢 39%**）"）⇒ 仿真模型对过速档的预测已在板上得到定量确认。★ 同期 `dbg_lag_warn` = **1（1024/1024 拍）**、`dbg_lag_fatal` = **1（全窗）** ⇒ 过速/丢步被 `r_ovl` → 滞后量测**如实报出**；`dbg_limiter_active` = 0、**逆环 0 / 非法 0** ⇒ 饱和期间方向环仍合法（不会伪造反向边沿）。对照 `inter = 32`：窗内 52–53 边沿 vs 期望 52.6–53.3 ⇒ **1.00×**，`lag_warn` / `lag_fatal` / `limiter_active` **全 0**、逆环 0 ⇒ **不钳位。**
- **结论 ③（工程口径）**：`inter_n = 256`（复位默认）时硬上限 `1024 × f_in ≤ 12.5 MHz` ⇒ **`f_in ≤ 12.2 kHz`**；20 kHz 必须 `inter ≤ 32`（`128 × 20 kHz = 2.56 MHz`，仅占天花板 20%）。这也是 §1.48 ⏭① 里"20 kHz 三档 sweep 在原配置下做不了"的**板上证据。**
- **★★ 工具缺陷（真问题，本轮踩到并已修）**：`tools/lag_probe.py` 与 `tools/ila_report.py` 的滞后量都由**各自独立**的 `np.unwrap` 求得、首拍不带偏置 ⇒ 抓波窗口首拍若跨 0°/360° 相位边界，真值会被显示成"**真值 − 360°**"：**20 kHz 纯 P 真值 +36.884° 曾被报成 −323.116°**（12 kHz 抓波恰好没踩到）。修法：两处都归一到 `(-180, +180]`（`(x + 180.0) % 360.0 - 180.0`），并在发生回绕时打印"原始均值 … 相差 360°×整数 ⇒ 已按 mod 360 归一"；`tools/lag_probe.py` 另加 `def wrap180(x)` 与这段实测案例的注释。★ 交叉验证：`ila_report` 的 `mean(angle − atan2)` 在 20 kHz II 型档 = **−3.785…−4.373°**（同样曾被报成 +319.233° = −40.767°）⇒ 折算 `3.785°/360 / 20213 Hz = 0.52 µs`，与 §1.48 补 解出的**流水线延迟 τ = 0.52 µs** 一致（`0.19°/kHz × 20 kHz ≈ 3.8°`），纯 P 档 −40.767° = 环路 36.9° + 流水线 3.8° ✓。

1.49 ✅ v9（2026-10-11）：Z 宽度按 TW28 手册改为 `z_width + 1` 个原始 AB 边沿步 —— 实施 / 手册自洽性 / 验证

决定：用户 2026-10-11 选定 §1.47 的两条出路中的 **①按手册修**，并定顺序"**先做 v9 的 Z 修复**"（`inter = 32` 重扫 12/20 kHz 押后）。

★ 手册两条陈述的自洽解读（报告要按这个口径写，别再当成矛盾）：`docs/TW28_digest.md:365` 写 `zwidth[edges] = (ZERO1.zwidth + 1) / div`，而同文件 `:372`（以及总表 `:783` 第 16 条）明确"**Z 输出旁路 post-AB 分频**"。二者是同一条规则的两面：**以原始 AB 边沿步计，宽度就是 `zwidth + 1`**；只有以"后分频之后的边沿"为单位才需要 `/div`。⇒ 本工程 `z_pulse_n = z_width + 1` **且 Z 继续旁路 `div`**（`rtl/SPEC_interface_contract.md` 行为 3"Z 通道旁路后分频（与 TW28 一致）"保持不变）就是手册语义，v8 及以前少的正是那个 `+1`。

RTL 改动（`rtl/abz_gen.v`，全部完成）：

位置	改动
`rtl/abz_gen.v:151`	新增 `localparam integer Z_PULSE_W = 5;`（`z_width+1 ∈ 1…16`，4 位装不下）
`rtl/abz_gen.v:196`	`reg [Z_PULSE_W-1:0] r_z_left;`（原 `reg [3:0]`）
`rtl/abz_gen.v:262`	`wire [Z_PULSE_W-1:0] z_pulse_n;`

位置	改动
rtl/abz_gen.v:519-520	assign z_pulse_n = {1'b0, z_width} + 5'd1; （取代 (z_width == 4'd0) ? 4'd0 : z_width）
rtl/abz_gen.v:528	z_step_fin = r_z_active & idx_exec & (r_z_left == 5'd1);
rtl/abz_gen.v:547,692,701,705,713	清零/装载/自减统一用 5 位常量（5'd0 / 5'd1）
rtl/abz_gen.v:690-696	★ **语义变化**：装载拍直接 r_z_out <= 1'b1、r_z_left <= z_pulse_n ⇒ z_width = 0 现在出 **1 个边沿步**（v8 是"无 Z"）。装载拍自身的 idx_exec 不参与递减（递减在 else if (r_z_active) 内），故脉冲高电平期间恰好发生 z_width + 1 次边沿步
rtl/enc_top.v	RTL_BUILD_ID → **16'h0009**（0x05F2 应读 9）
rtl/reg_file.v	r_zero1 注释更新（默认仍 8'h04 = 5 步；v9 起 z_width = 0 不再"钉死 Z 为 0"）

同步的模型/测试台：sim/tb_abz_gen.v（TEST C 说明与判据改为 z_width + 1，循环由 zw = 1…8 扩到 **zw = 0…8**）、sim/tb_enc_top.v（B5 判据收紧为 **[2304, 2816] 拍** = 2560 ±10%，可同时拒绝 v8 的 2048 与窗截断的几十拍；C5 打印文本更新）、sim/tb_abz_jitter.v（注释/打印标注 v9 语义）、model/abz_uvw_model.py（**跨零拍不计入步数**：装载拍走 if (z_load) 分支，该拍 idx_exec 不递减 —— 顺带修掉旧模型在"跨零拍正好有边沿"时比 RTL 少 1 步的**既有模型偏差**）。

验证：- **全回归 21/21 PASS**（tools\run_sim.ps1 -Rebuild，sim/logs/SUMMARY.md）。- sim/logs/tb_abz_gen.log TEST C：z_width = 0,1,2,…,8 ⇒ 实测宽度 **1,2,3,…,9** 步，全部 OK（= z_width + 1）；inter=64/div=3 行亦 OK（Z 旁路后分频）。- sim/logs/tb_enc_top.log：REV Z 脉宽：max=2558 拍（理论 5 × 512 = 2560，v8 为 2042）⇒ 集成路径确认 +1；B3/B4/B5 全 ok，TEST PASSED (106 checks)。- model/abz_uvw_model.py 表 2b（inter=64/div=1）实测 **1…9 步 = z_width+1**，内部自检全 PASS。

诚实记录的边界与未做项：- ⚠ **新增截断约束**：Z_TIMEOUT = 4096 clk = 81.9 µs 是兜底 ⇒ 需 (z_width+1)/(1024·f_in) ≤ 81.9 µs，即 **f_in ≥ 11.92 × (z_width+1) Hz**（默认 5 步 ⇒ ≥ 59.6 Hz；16 步 ⇒ ≥ 190.7 Hz）。低于此值读到的脉宽会小于名义值，**属兜底行为、不是新缺陷**（已写进 rtl/abz_gen.v:687-689 与 SPEC）。- ⚠ 手册另有"zwidth **必须小于** INTER"的约束（docs/TW28_digest.md:366），**RTL 不做检查**；默认 z_width = 4 / inter = 256 远满足，但若把 inter 降到 ≤ 16（下一轮重扫用 inter = 32，仍满足）需自行注意。- ✅ **v9 位流已重综合并上板（2026-10-11 21:04）**：- **归档先行**：_synth_work/pre_v9/（15 文件 3,252,744 B + ila_v8_evidence/ 33 文件 1,734,553 B）冻结了 v8 的两版位流与全部上板证据，enc_top.bit SHA256 162209D5…4C19、enc_top_ila.bit SHA256 EA97881B…CAC7，说明见该目录 _README.txt（遵循"重综合前先建 pre_vN/"的规矩）。- **重综合**（tools\impl_check.tcl，日志 _synth_work/impl_build_v9.log）：Bitgen Completed Successfully；生产位流 _synth_work/enc_top.bit **1,226,648 B**，SHA256 2F51DCD332178B56EC4124F87AC8D3317BD181939857444CAD0E1CA776124A43（压缩省 7,724,576 bit ⇒ 9,813,120 bit = 1,226,640 B + 8 B 头 ✓）。- **时序/资源**（impl_timing.rpt、impl_util.rpt）：WNS **−3.274** / TNS **−84.283** / **26** 失败端点 / 11,304 端点，WHS +0.024，WPWS 9.500；LUT **6360** (30.58%) / FF **4086** (9.82%) / BRAM 0 / DSP 16 (17.78%) / IOB 50。对照 v8（−3.274 / −84.267、LUT 6297 / FF 4085）：**同一临界路径、同 26 个失败端点（全在 ADC 输入口，见 §1.40 的诚实约束）**，Z 逻辑只多 63 LUT / 1 FF，未引入新违例；DRC 0 error / 0 critical，布线 0 error。- **烧录**（ILA_MODE=restore，_synth_work/flash_v9.log）：End of startup status: HIGH ⇒ DONE。- **寄存器读回**（tools\uart_spi.py --port COM6 read，21:05）：0x05F2 = 0x0009 ✅ v9、0x0302 = 0x001A ✅ kp=2/ki=3、0x0500 = 0x0A16（byte0 = 0x16 = 22）✅、0x0501 = 0x000A ✅、0x0101 = 0x0102 = 0x0000（inter_n = 256、div = 1 ⇒ 1024 边沿/电周期）✅、0x0108 = 0x4004（z_width = 4）、0x0109 = 0x0040（abzen 开）✅；0x0405 STAT_LATCH = 0x0025（otr + amplo + laglim 粘滞，上电后无稳定输入时正常，非本版引入）。- ⚠ **重综合进程偶发自旋**：本次 vivado 在打印 Bitgen Completed Successfully 之后**不再退出**（CPU 持续满载、17 分钟无任何新输出/新文件）。核对 enc_top.bit 与五份 impl_*.rpt 时间戳（20:42–20:43）与位流哈希均正常后手工终止进程，作业因此报 exit 1 —— 属**假失败**（与 §1.36 / 教训④同类）。下次流程：先按时间戳+哈希确认真写完，再收尾。- ✅ **v9 的 Z 脉宽已在板上闭环（ILA 实测，2026-10-11 21:24–21:37）**：- **ILA 位流已建成**：_synth_work/enc_top_ila.bit **1,432,868 B @21:24:56** + enc_top_ila.ltx 71,337 B + impl_ila_{timing,util,drc}.rpt（36 端口 / 269 bit；**dbg_z_out = probe34、dbg_z_window = probe35** ⇒ ILA 能直接量 Z）。⚠ **新踩的坑（必须记）**：第一次跑（受限沙箱）在 [ILA_STEP 7] opt_design 展开 debug core 时**挂死** —— 子日志 .Xil\Vivado-<PID>-ROG\dc_drv.0\dc\vivado.log 里是 rundef.js(122, 5) JavaScript runtime error: Access denied，两个 OOC 综合 run（u_ila_synth_1 / dbg_hub_synth_1）的 vivado 子进程 **0% CPU 死等 11 分钟**；⇒ **ILA / 生产位流构建一律用全权限（danger-full-access）跑**（§1.36 教训①的新症状），换全权限后**同脚本一次通过**。另：Bitgen Completed Successfully 之后包装脚本仍偶发不返回（本次确认位流与报告时间戳 21:24:43–21:24:56 正常后手工收尾，属**假失败**）。- **① z_width = 4（默认）**：ILA_TRIG=z_out 触发（21:35:12），数据 _synth_work/ila_cap_v9_z.csv（134,738 B）⇒ z_out 高 **240 拍 = 4.800 µs**、单脉冲、**起止都正好落在 A/B 边沿上**，[start,end) 内数到 **5 个 A/B 边沿 ⇒ 5 个边沿步** ✅（报告 _synth_work/ila_report_v9_z.txt 的 [5b] 节）。- **③ 零宽测试 0x0108 = 0x4000（z_width = 0）**：用 capture 模式（**不重下位流**，配置保住）抓 _synth_work/ila_cap_v9_z0.csv ⇒ **60 拍 = 1.200 µs = 1 个边沿步** ✅（ILA_ZWIDTH=0 时报告判 ✅ 与 v9 的 z_width+1 一致）⇒ **§1.47 的"z_width = 0 ⇒ 无 Z"在板上被推翻**。- ★ **口径订正（工具）**：tools/ila_report.py 的 [5b] 新增**首选判据"A/B 边沿计步"**（直接数 [start,end) 内 dbg_a_out / dbg_b_out 的边沿数，**不依赖 f_in**），旧的"窗口平均 f_in 折算"标注为**仅作旁证** —— 两者实测差 5%（4.80 步 vs 5.00 步），根因是窗口平均频率 ≠ 脉冲附近的瞬时边沿率，**别再被 4.80 误读成"少了 0.2 步"**。- **收尾**：量完把 0x0108 写回默认并**把生产版 v9 位流烧回板上**（_synth_work/flash_v9_prod_restore.log，End of startup status: HIGH），回读 0x05F2 = 9 / 0x0108 = 0x4004 / 0x0302 = 0x001A / 0x0500 = 0x0A16 / 0x0501 = 0x000A ✅（ILA 版位流已从板上撤下）。- 工具侧说明：tools\ila_capture.tcl 的 restore 模式提示文本已同步为"应读回 0x0009（v9）"，并新增 ILA_TRIG 覆盖触发探针（set TRIG "" 参与 env 循环，if {$TRIG ne ""} { set want $TRIG }）。- ✅ **② inter = 32 重扫 12 / 20 kHz 已完成**，见 **§1.48 补 2**（12 kHz 三档）与 **§1.48 补 3**（20 kHz 五档 + inter = 256 钳位判决性演示）。- ◐ **③ D 分支**：angle_raw 毛刺**已在 v9 上重新取证**（4 版 ang_glitch 触发窗口 + 环路再注入对比 + v6 复算，见 **§1.51**；**+ 19.979 kHz 频率判别 / 180 s 事件间隔统计 / 位级定位 D11+D8 / 512 格点几何假设，见 §1.52**）；crcerr **仍是开放项** —— 板上 ma 无主站时钟 ⇒ 0x053A bit7/bit6 恒 0（平凡结果），要真正激励须按 docs/08_bissc_dumb_host.md 的方案 A / B 复测（**需用户改接线**）。- ⏭ **仍剩下的量测**：① **示波器**复测 Z 脉宽（探 J5-7 = z_out / M21）并留下带游标的量化截图 —— ILA 侧已闭环，示波器现在只作**旁证/报告配图**（补 §1.26 的定量缺口）；② D 分支成因判别：**频率判别已完成（§1.52 一）**、**幅度判别已完成（§1.53：7.6 Vpp ⇒ 事件整体归零）**、**幅度反向检验也已完成并证伪了 §1.53 的预测（§1.54：8.2 Vpp ⇒ 事件不归零，率升到 13.65 次/s、相位由 −71.4° 移到 −141.2°、坏码由 0x300 变 0x400）** ⇒ 现行最稳不变量是"坏样本真值总落在 S 通道引脚码 0x200 整数倍边界上"（主进位亚稳/误码），但率对幅度的陡度与"选中哪一档边界"仍**未闭合**；剩下可选：(A) 8.2 Vpp 再抓 2–3 版窗看相位复现性、(B) 8.4/8.6/9.0 Vpp 阶梯率曲线（不提权）、(C) **示波器看 ADC 数据线 D11 / D10 与 s 输入**（判 ADC 内部误码 vs 板级完整性）。- ⚠ 命名提醒：§1.48 提到的"LOG2DEC 2→0 的 **v9** 路线"是**当时的假设编号**（审计建议 (c)），与本次的 v9 = Z 宽度修复**不是一回事**，已在该处改称 **v10 候选**。

1.51 D 分支取证（2026-10-11 21:50–22:00，v9 ILA 位流）：angle_raw 毛刺逐级链、定点码 0x300、环路再注入改善 ≈8× —— 以及"相位锁定但率不随 f_in"的未解问题

条件：板上 = v9 **ILA 位流**（_synth_work/enc_top_ila.bit）；0x0302 = 0x1A、0x0101 = 0x0102 = 0（inter = 256、div = 1 ⇒ 1024 边沿/电周期）、0x0108 = 0x4004；AFG = **20 kHz**（用户设定，我无法远程改；幅度按本次 ILA 窗口的瞬时半径 **1606 ± 15 LSB / 2048 满量程 ≈ 78%** 折算 ⇒ 约 **7.8 Vpp**（模块满量

程 10 Vpp），与 §1.45 的 8 Vpp 条件一致）。新工具 **`tools/glitch_chain.py`**（独立脚本，内嵌 `tools/ila_report.py` 的位序判定与解码约定 —— 后者是顶层脚本无法 import），逐毛刺打印逐级链。 **★ 口径（本轮踩过）**：20 kHz 下 `angle` 固有旋转 **26.6–27.3 LSB/拍**，算环路过冲必须**先按事件前的线性趋势去趋势**，否则会得到 ≈ **+7° 的假过冲**（我最初就是这么算错的）。

取证一：4 版以 `dbg_ang_glitch` 触发的窗口（`ILA_MODE=capture` + `ILA_TRIG=ang_glitch`，DEPTH 1024，一次全权限命令内循环 4 次）

产物	内容
`_synth_work/ila_cap_v9_glitch{1..4}.csv`（各 134,738 B）+ `ila_glitch{1..4}.log`	`CORE_STATUS=FULL fired=1`、`TRIGGER_COMPARE_VALUE = eq1'b1`、触发 21:56:09 / 21:56:20 / 21:56:31 / 21:56:41

- **四版结果完全一致**：每窗 1 个事件、`dbg_ang_glitch` 脉冲宽 **1 拍**；尖峰 = **`s_raw` 单样本实测为固定码 `0x300`（768）**（⇒ 相对 ±16 拍滑动中值的偏差 **+2304 LSB = 384 × 标称步长**，标称 6.0 LSB/拍；"覆盖成固定码"还是"叠加固定偏移"的分辨见取证三）；`s_dec` Δ **9308–9316** / `c_dec` Δ 216–276。
- **逐级链（glitch1，`_synth_work/_inspect_spike.py` 复核第 480–499 拍）**：第 **485** 拍 `s_raw` = 768（前后 −1537 / −1536）→ `s_dec` 第 **490–493** 拍下陷（−15348，基线 −24748，Δ ≈ +9400，**抽取输出保持 4 拍 = `dec_valid` 周期**）→ `stat_byte` 第 **495–498** 拍 0x14 → **0x34**（+ bit5 `scamp`）→ `angle_raw` 第 **511** 拍单拍跳变 **+1856…+1884 LSB = +10.20…+10.35°**（`cordic_mag` 偏差 −8498…−8534）。
- **★ 比例自洽**：`rtl/enc_top.v:271-274` 先把 12bit 补码左移 4 位成 Q1.15（`s_q15 = s_ext <<< 4`），`rtl/enc_top.v:284-294` 的 `decim_avg(DW=16, LOG2DEC=2)` 输出 4 点均值 ⇒ **`s_dec` = 16 × 近 4 点原始均值**；故单个原始样本 +2304 ⇒ `Δs_dec` = 16 × 2304/4 = **+9216**，实测 **+9400**（差 <2%，其余来自参与平均的另外 3 点也在谷底附近起伏）。
- **`cor_sat` / `otr_lat` 在整个窗口都没有置位**；`stat_byte` 的静态部分是 **0x14**（bit2 `laglim` + bit4 `lagfat1` = inter=256 @20 kHz 的正当饱和告警，见 §1.48 补 3）。
- **去趋势后的环路再注入**：`angle` 残余过冲 **−0.58…−0.92°**（对照 v6 的 +7.38°，见取证三）。
- **波形相位锁定（★ 已订正口径）**：用**未被污染的值**算（坏路取邻域中值、好路取尖峰拍实测）⇒ `atan2` = **−71.3…−71.6°**（v9 四版 −71.5 / −71.4 / −71.4 / −71.3°，v6 复算 −71.6°），瞬时半径 **1620–1622 LSB**（与健康窗半径均值 1606 一致 ⇒ 同一圆周点，该处 `|s|` ≈ 0.95 × 峰值）；坏路真值逐版一致到 ±1 LSB（v9 `s_raw` −1536/−1537、v6 −1539）⇒ **事件在正弦波上的位置确实可复现**，不是随机噪声。
- ⚠ **口径坑（本轮踩到并已修工具）**：`tools/glitch_chain.py` 原先直接打印 `atan2(s[best], c[best])` = **+55.9…+56.2°** —— 那是**伪值**（= 尖峰固定码与另一路实测值取 atan2，与波形相位无关，必然恒定）。已改为"坏路取邻域中值"的真值相位，并把伪值单独标注为"不能用来判相位锁定"。

取证二：率（`tools/diag_counters.py --port COM6 --seconds 20`，日志 `_synth_work/glitch_rate_v9k20_i256.txt`）

- Δ`gl_cnt_1024` = **36 / 20.0 s = 1.8 次/s**；Δ`gl_cnt_2048` = Δ`gl_cnt_4096` = Δ`gl_cnt_8192` = **0**（⇒ 跳变幅度全在 11.25° 滞回带以下）；Δ`gl_hold_cnt` = **36**（与 `gl_cnt_1024` 相同 ⇒ 检测-保持链自洽）。
- ⚠ 该脚本打印的"次/周期"**不可用**：分母 `0x0540 cyc_pulse_cnt` 实测 Δ7549 / 20 s = **377/s**，与 20 kHz 的电周期率不符 ⇒ **只用 Δ/秒。**
- ★ 与 §1.45 的"1 kHz / 8 Vpp 下 6–8 次/s"对照：**量级相近但不随 `f_in` 成正比**（20 kHz 反而只有 1.8 次/s）—— **尚未解释**，见下方开放问题。

取证三：v6 / 1 kHz 旧证据复算（`_synth_work/ila_cap_stage2.csv`，即 §1.37 ③ 的原始证据）

- 尖峰**同样是固定码 `s_raw = 0x300`（`c_raw = 0x200`）**、波形相位 **−71.6°**（修正口径；被污染码的伪值 +56.3° 见取证一）、偏差 **+2307**（= +2304 + 3 LSB，恰是邻值 −1539 vs −1536 之差）⇒ **倾向"样本被覆盖成定点码 `0x300`"**；⚠ 诚实说明：本批数据里坏路的**真值几乎恒定**（−1536…−1539，正因为相位锁定），所以"**覆盖成固定码**"与"**叠加固定偏移 2304**"两种解释**只差 3 LSB**，**当前证据不足以唯一定论** —— 这正是下方开放实验 ②（改 AFG 幅度/频率）要区分的问题。
- `ang_glitch` 检测器脉冲**同样是 1 拍** ⇒ §1.37 记的"4 拍脉冲"其实是 `stat_byte` / `scamp` 的宽度，不是毛刺检测器的宽度。
- 该版 `s_dec` Δ 9228、`cor_sat` 未见、`otr_lat` 在 486–499 拍为 1（当时板上 OTR 本就锁存）、`stat_byte` 0x01→0x21→0x01、`angle_raw` 最大单拍跳变 **−1764 LSB = −9.69°**（第 515 拍，`cordic_mag` −39）。
- **★ 环路再注入对比（同一去趋势口径）**：v6 = **+1343 LSB = +7.38°（34 拍到峰、6 拍收回）**，v9 = **< 0.92°** ⇒ **v8 起的角度域毛刺保持 + II 型环路把再注入压了约 8 倍。**

排除项（诚实列举） - **不是 ADC 采样撕裂**：尖峰落在正弦谷附近，原始码每拍只动 ≈7 LSB，撕裂最多给出"真值 ± 十几 LSB"；实测偏差 2304 LSB 远超该量级（与 §3 item 15 的 `tPD_meas ≤ 4 ns` 结论一致）。 - **不是饱和/超量程**：`cor_sat`、`otr_lat` 全窗未置位，且码值 0x300 不是轨道码（0x000 / 0xFFF）。 - **不是随机噪声**：码值与相位跨 **两版位流（v6 / v9）× 两个频率（1 kHz / 20 kHz）** 精确复现到 ±3 LSB。 - 工程含义：现有防线是**角度域**的检测-保持（`rtl/enc_top.v:456` 起），尖峰已经穿过 CORDIC（`angle_raw` 跳 10°）才被吃掉；若要在原始域消灭它，可在 `s_cor` / `c_cor` 之后、CORDIC 之前加 **3 点中值/尖峰抑制**（代价 2 拍延迟 + 比较器）。**本轮未实施，仅作建议**（改动需重综合 + 全回归，且会动 §1.48 补 的流水线延迟口径）。

crcerr 侧：前置条件缺失，不能算"已排查" - 读数：`0x053A` = **0x0000**（bit7 `frame_active` = 0、bit6 `crcerr` = 0）、`0x053B` = 0 ⇒ 这只是"**`ma`（`J5-11`）没有主站时钟 ⇒ 帧从未开始**"的平凡 0，**不能说明 CRC 正确。** - 真实证据链在 **`docs/08_bissc_dumb_host.md`**（★ 本文件里**没有** §1.50，先前口头引用有误）：A1（主判据）/ A2（负对照）已在 **v7** 通过，B1（故意 `ma` 10 MHz 让帧卡死并置粘滞 `crc_err_seen`）/ C1 **只在 v6** 验过。 - ILA 侧**没有任何 `biss_*` 探针**（`_synth_work/ila_probe_list.txt` 共 36 端口 / 269 bit）⇒ CRC / 帧活性**只能用寄存器观测**；要真正激励需把 AFG 接到 `ma` 并按 `docs/08` 方案 A/B 复测（**需用户改接线**）。

开放问题（未解决；下列判别实验需要用户改 AFG / 接线） - 为什么事件**相位锁定在 −71.4° 却只有 1.5–1.8 次/s、且率不随 `f_in` 变化**？（★ 原写"≈ +56°"是**伪值**，见取证一的口径坑。）**判别实验 ①（改频率到 19.979 kHz）已在 §1.52 完成** ⇒ 相位与码值完全不变、"栅格差频"假设未被支持；成因的下一步是**改幅度**（见 §1.52 六）。 - 判别实验：① AFG 频率 19.7/19.979 kHz ⇒ ✅ **已完成（§1.52 —）**：相位仍 ≈ **−71.4°**、码值仍 `0x300` ⇒ **与信号/前端相关**，不是栅格差频；② AFG 幅度降 3–5% ⇒ **待用户做**（三个可区分结局见 §1.52 六）；③ 示波器同时看 s 通道模拟输入与 ADC 数据线（`D11` / `D8` / `D9`）⇒ 区分"模拟侧尖峰"与"数字总线位错"（**未做**；§1.52 五已从位级把"相邻样本混码"排除，示波器的目标收窄为"看 D11/D8 是否真的被外来干扰置高"）。 - ⇒ 本节的 D 分支结论只到"**现象精确刻画 + 排除三类成因 + 量化环路再注入改善**"，**成因仍未定位。**

产物清单：`_synth_work/ila_cap_v9_glitch{1..4}.csv` / `ila_glitch{1..4}.log`、`glitch_chain_v9.txt`（工具完整输出）、`_inspect_spike.py` / `_inspect_spikes.py`（原始码表复核）、`glitch_rate_v9k20_i256.txt`、对照 `ila_cap_stage2.csv`（v6）与 `ila_cap_v9_dbg_state.csv`（20 kHz 健康窗口，报告 `rep_v9_dbg_state.txt`）。工具 `tools/glitch_chain.py` 已入库。

1.52 D 分支续查（2026-10-11 22:07–22:40，v9 ILA 位流）：19.979 kHz 判别窗口、180 s 事件间隔统计、离群码全窗扫描、位级定位（D11+D8）与 512 格点几何假设

条件：板上 = v9 ILA 位流；`0x0302` = 0x1A（kp=2/ki=3）、`0x0101` = `0x0102` = 0（inter = 256、div = 1）、`0x0108` = 0x4004；AFG 频率由 **20 kHz 改为 19.979 kHz**（用户手改）。**先证实频率真的变了**：`ILA_MODE=capture` + `ILA_TRIG=''`（默认 `adc_valid` 触发）抓 `_synth_work/ila_cap_v9_freqcheck.csv`（日志

`ila_freqcheck.log`），`tools/ila_report.py` 解析出 **`f_in = 19.979 kHz`**（预设闸门 19.3–20.2 kHz 通过）⇒ 频率改动 0.9%，**远小于**"栅格差频"假设生效所需的量级。

一、19.979 kHz 的 4 版 `ang_glitch` 触发窗口（`ILA_TRIG=ang_glitch`，DEPTH 1024；`_synth_work/ila_cap_v9_g197_{1..4}.csv` 各 134,738 B + `ila_g197_{1..4}.log`，触发 22:07:52 / 22:08:02 / 22:08:14 / 22:08:25；分析输出 `_synth_work/glitch_chain_g197.txt`）

量	四版实测	与 20 kHz 那批（§1.51）对比
尖峰拍 / `s_raw`	487 / 486 / 485 / 486；**0x300（4/4）**	完全相同（0x300）
相对 ±16 拍滑动中值偏差	**+2304 / +2305 / +2305 / +2306 LSB**（384.0–384.33 × 标称 6.0 LSB/拍）	相同
波形真值相位（坏路取邻域中值）	**−71.4 / −71.6 / −71.5 / −71.4°**	相同（−71.3…−71.6°）
瞬时半径	1620–1622 LSB	相同
`c_raw`	518 / 512 / 513 / 516	相同量级
`s_dec` Δ	9256–9376	相同量级
`angle_raw` 单拍跳变	**+1862…+1875 LSB = +10.23…+10.30°**	相同
去趋势后 `angle` 残余过冲	**−0.63…−0.80°**（斜率 +26.2…26.7 LSB/拍）	相同（≈8× 优于 v6）

- **★ 与 20 kHz 那批唯一差别**：本次 `otr_lat` = 1 **覆盖事件附近（约 483–498 拍）**、`stat_byte` 静态值 **0x15**（oflow+laglim+lagfatl）→ **0x35**（+scamp）发生在 495–498 拍；20 kHz 那批是 0x14→0x34 且 `otr_lat` 从未置位。`cor_sat` **仍然全窗未见。**⇒ 机制未变，OTR 只是本次运行期间另有锁存（OTR 是粘滞锁存，与事件本身无因果关系）。
- **⇒ 判别实验 ①（改频率）的结论**：相位仍锁在 **−71.4°**、码值仍是 **0x300** ⇒ **"与采样栅格/总线时序相关的差频机制"未得到支持**，事件与**波形相位/前端**强相关。

二、率（同一条件下两种方法） - `tools/diag_counters.py --port COM6 --seconds 20`（日志 `_synth_work/glitch_rate_g197.txt`）：Δ`gl_cnt_1024` = **30 ⇒ 1.5 次/s**；Δ2048/4096/8192 = **0**；Δ`gl_hold_cnt` = **30**（与 1024 一致）。 - **180 s 长测**（新工具 `tools/glitch_intervals.py`，日志/CSV `_synth_work/glitch_iv_20k_180s.{txt,csv}`）：轮询 12474 次 / 180.63 s = **69.1 Hz**（7 次丢应答）；**计数器总量 57024 → 57322 = 298 ⇒ 真率 1.65 次/s**（脚本按"观测到的变化次数"报 237 次 / 1.31 次/s，**偏低**，因为一窗内多次自增只记 1 次 ⇒ 以计数器总量差为准）。 - **⇒ 20 kHz 1.8 次/s → 19.979 kHz 1.5–1.65 次/s**：频率变动 0.9% 内率基本不变。**§1.45 的"1 kHz / 8 Vpp 下 6–8 次/s"是不同方法/位流测得，与本节不可直接比**（要判定 `f_in`–率关系必须在同一方法下重测 1 kHz）。

三、事件间隔统计（180 s，298 事件） - ⚠ **必须先过滤非单调值**：`t=81.565`（57153 → **31047**）与 `t=172.148`（57307 → **12195**）两处**非单调回退**（陈旧/异址应答被 `find_reply` 当成有效帧）⇒ 各产生 1 个假事件，并把随后一次 Δ 记成 20424 / 26106 / 39430 / 45112，还造出一个 0.003 s 的假最小间隔。**教训：轮询计数器脚本必须显式丢弃"小于上次值"的读数**（`tools/glitch_intervals.py` 已在统计前过滤，但 Δ 记账仍会受影响 ⇒ 看率时一律用计数器总量差）。 - 过滤后：**median 0.487 s、mean 0.753 s、max 4.491 s、p10 0.116 / p90 1.550、变异系数 CV ≈ 0.87** ⇒ 间隔近似**指数分布（Poisson 过程）**，**没有固定恢复时间 / 死区**（这一点对成因很关键：排除"每次事件后电路要恢复若干 ms"的锁存类机制）。 - ⚠ **但不能断言"完全没有簇"**：同一轮询窗（≤14.5 ms）内出现 2–4 次自增的窗约 **47 个**，而按实测率 λ=1.65/s、窗宽 14.5 ms 的泊松期望只有 **~4–7 个**（高 7–13 倍）；且 9 个 ILA 窗口（各 20.48 µs）**每窗只有 1 个坏样本** ⇒ "短时簇"这一结论**受 14.5 ms 分辨率和偶发无效应答限制，只能记为待确认**（要定论需要 RTL 加时间戳或更深 ILA）。

四、窗口内有没有"亚阈值连续谱"？（新脚本 `_synth_work/_outlier_codes.py`，`import glitch_chain.load` 复用位序判定） - 对 `ila_cap_v9_glitch1.csv` / `ila_cap_v9_g197_1.csv` / `ila_cap_stage2.csv` 逐样本算"相对 ±16 拍滑动中值的偏差"：**每窗只有 1 个 ≥32 LSB 的离群（就是 0x300 尖峰），`c_raw` 0 个。** - 另被标出的拍 1022/1023（+32…+35 LSB）是**窗口边缘中值窗截断伪影**，不是真实事件 ⇒ **窗口内没有亚阈值连续谱**：事件是"要么不发生、要么一次到位"的**离散翻转**，不是"误差逐渐变大"。

五、位级定位（本节的硬结论） - 引脚域 vs RTL 域：`rtl/ad9238_capture.v:5-6` 是 `s_raw = ch1_d − 2048` ⇒ ADC 输出是**偏移二进制**、模块减 2048 变补码；ILA 探针 `dbg_s_raw` 是**减完之后**的补码值（`rtl/enc_top.v:237 wire signed [11:0] s_raw,c_raw；:247-248` 引出 `dbg_s_raw`/`dbg_c_raw`）。 - 坏样本实测为 RTL 域 **0x300 = +768**，真值 ≈ **0x9FF/0xA00（−1536/−1537）** ⇒ 在**引脚域**坏字是 **0xB00**、真字是 **0x200** ⇒ **D11 与 D8 两条线被额外置 1**（+0x900 = 补码域 +2304 ✅ 与实测 +2304…+2306 误差 ≤2 LSB）。 - **★ 这条直接排除 FPGA IOB 的 setup/hold 混码**：事件附近两个真实样本（RTL 域 0x9FF / 0xA00）**D11 都是 1**，而坏样本 D11 = 0 ⇒ **任何"相邻样本的位被混合/部分捕获"都做不出这个字**（相邻两字 D11 相同）。§1.51 排除项里"不是 ADC 采样撕裂"由此**从量级论证升级为位级论证。** - 工程含义：坏字是"真值 + 0x900（引脚域）"，即**两条数据线被同时置高**；D8 属于低 9 位那组正在整组翻转的线，D11 则是**完全没有翻转**的线 ⇒ 两类线同时被置错，**不是单纯"漏采一个翻转"。**

六、512 格点几何假设（候选触发机制，本轮提出，可证伪） - 观测到的相位（−71.4°）对应瞬时半径 r ≈ 1620，而 **512•√10 = 1619.67**；格点 **(s, c) = (−3•512, +1•512) = (−1536, +512)** 恰好落在这个半径的圆上 ⇒ 事件瞬间**两个通道的码同时处在 512 的整数格点上**，各自总线都在做**10 位纹波进位**（s: 0x9FF↔0xA00，XOR = 0x3FF；c: 0x1FF↔0x200，XOR = 0x3FF）⇒ 联合同时开关噪声（SSN）在此刻最大。 - 一致性检查：`c` 的实测值 512–520 可**完全由几何解释**（c = √(r² − 1536.5²) = 513 ± 8，r = 1620 ± 15）⇒ **不需要为 `c` 单独假设一个触发条件**，`c` 的出现位置是 `s` 条件的几何推论。 - ⚠ **该假设目前不能解释全部**：S 通道其它 512 倍边界（尤其 −1024.5 = 11 位翻转、SSN 更差）在 9 个窗口里**从未出现**，且 `gl_cnt_2048/4096/8192` 恒 0 ⇒ "任意大进位都会触发"的纯 SSN 模型**被削弱**；**"为什么只有 −1536.5 这一个格点"仍未解决。** - 该模型的一个自然推论：危险样本率只由"波形落在危险相位窗内的频率"决定（= 与 `f_in` **无关**），与实测 20 kHz / 19.979 kHz 都是 1.5–1.8 次/s 相符。 - **★ 下一步判别实验（需要用户拧 AFG 幅度 —— 我无法远程改；三个可区分结局）**：把幅度**降 3–5%**（8 Vpp → 7.6 Vpp）后先 20 s 数率、再（可选）抓 1 版 `ang_glitch` 看相位/码值： - (a) **联合格点模型** ⇒ r 离开 512√10 ⇒ **事件整体消失**（最近的格点半径是 512√8 = 1448.2 = −10.6% 与 512√13 = 1846.1 = +14%，**中间没有可用格点**）； - (b) **仅 S 通道边界模型**（只要 |s| 能到 1536.5）⇒ 事件**保留但相位向峰移动**（asin(1536.5/1539) ≈ 86.5°），且再降一点（r < 1536.5）就消失； - (c) **取值映射模型**（坏值 = −真值/2）⇒ 相位**不变**、**坏码改变**（r = 1782 时约 0x399）。 - 判据说明：20 s 计数的泊松 σ ≈ ±18%（1.6 次/s ⇒ 32 ± 6）⇒ **"归零"与"相位/码值变化"是决定性判据**，单纯 ±20% 的率变化不是。 - ⇒ **已执行（用户把 AFG 改为 7.6 Vpp / 20 kHz）**：结果见 **§1.53。**

七、工具与教训 - 新增 **`tools/glitch_intervals.py`**（高频轮询 `0x0505`，输出事件表 + 间隔统计 + CV + 直方图 + CSV；`--addr` 可换地址）：默认走 `uart_spi.Link._once`（**实测 69–78 Hz**）；`--fast` 小 chunk 自读路径**实测反而更慢（18.8 Hz）** ⇒ 不默认。首次跑因超时设成 0.15 s + `tries=1` 报 `!! 首次回读超时`（同一条读用 `tools/uart_spi.py read 0x0505` 正常）⇒ 超时统一用 **0.4 s / tries=2。** - 毛刺计数语义（读 `rtl/enc_top.v:469-503` 确认）：`ANG_GLITCH_MAX = 16'sd1024`；`ang_delta = angle_raw − ang_prev`；`ang_glitch = ang_seen && !ang_held && (|ang_delta| > 1024)`；`angle_filt = ang_glitch ? ang_prev : angle_raw`；更新块在 `if (cordic_valid)` 内、`ang_held <= ang_glitch` ⇒ **一个坏样本恰好产生 1 次 `gl_cnt_1024` + 1 次 `gl_hold_cnt`**（坏样本的"退出跳变"被 `ang_held` 压掉）⇒ 计数器差 = 坏样本个数，**不需要除以 2**（这也解释了为什么 `gl_1024` 与 `gl_hold` 永远相等）。 - 本节结论只到**"现象精确刻画（码值/相位/幅度分布/间隔统计）+ 位级定位到 D11+D8 + 一个可证伪的几何触发假设"**；**成因仍未定位**，下一步取决于用户是否做上面的幅度实验。

产物清单：`_synth_work/ila_cap_v9_freqcheck.csv` + `ila_freqcheck.log`、`ila_cap_v9_g197_{1..4}.csv` + `ila_g197_{1..4}.log`、`glitch_chain_g197.txt`、`glitch_rate_g197.txt`、`glitch_iv_20k_180s.{csv,txt}`、`_iv_smoke.csv`、`_outlier_codes.py`。工具 `tools/glitch_intervals.py` 已入库。

1.53 ★ D 分支成因：幅度判别实验（2026-10-11 22:20–22:30）—— 事件整体归零，`512` 格点联合共振 得到支持、仅 `S` 通道跨边界 被否决

> ⚠ **本节的"支持/否决"结论已被 §1.54 部分推翻**（幅度 +2.5% 的反向检验**没有**归零）：请把本节当作"7.6 Vpp 归零 + 幅度双验证 + 圆参数拟合方法"的记录，**成因判别结论以 §1.54 为准。** ⚠ **追加更正（§1.55 ⑦，2026-10-11 22:39）**：本节判据里的"7.6 Vpp 下 `max|s| = 1544.2 > 1536.5` ⇒ S 码仍跨 −1536.5"是**符号错误** —— `max|s| = |O_s| + A` 是**正侧**极值；**负侧**极值 = `O_s − A` = **−1524.3**（Kasa）/ −1514.8（正弦），**并没有跨到 −1536.5** ⇒ 本节的"7.6 Vpp 归零"有平凡解释（码没到边界）、**不能再用作判别证据。**

条件：板上状态**未动**（v9 ILA 位流、`0x0302` = 0x1A、inter = 256、`0x0108` = 0x4004）；用户只把 AFG 幅度由 **8.0 Vpp 改为 7.6 Vpp**（−5%），频率仍 20 kHz。

① 率：20 s 内毛刺计数全部为 0（`tools/diag_counters.py --port COM6 --seconds 20`）

计数器	t0	t1	Δ
`0x0505 gl_1024_gt5.6deg`	44126	44126	**0**
`0x0507 / 0x0509 / 0x050B`（2048/4096/8192）	39777 / 48047 / 44453	同	**0**
`0x050D gl_hold_cnt`	16919	16919	**0**
`0x0540 cyc_pulse_cnt`	30446	37994	7548 ✅ 计数器活着

- 对照：同条件（v9 ILA、inter=256、`0x0302` =0x1A、20 kHz）在 8.0 Vpp 下 20 s 是 **30–36 次**（§1.51 取证二、§1.52 二）⇒ 泊松下零计数 $p \approx e^{-30}$，**不是统计波动。**
- ⚠ 旁注：`0x053A` = 0x0008（bit3 `stat_otr` 粘滞）—— 推测是用户拧幅度过程中 AFG 输出的瞬变触发了一次 OTR 锁存，与事件机制无关（OTR 是粘滞位，不会自行清除）。

② 输入仍然健康且幅度确实只降了 ≈5%（ILA 健康窗 `_synth_work/ila_cap_v9_amp76.csv`，134,738 B，22:25:58；工具 `tools/ila_report.py`） - 位序自检：直读 `stat_byte` bit7 违例 **0 处**（位反 1024 处）⇒ 解码正确；`f_in = 20.076 kHz`；半径 `r` 均值 **1529.6 ± 19.2**（min 1500.9 / max 1560.9）。 - `angle_raw` 线性残差 RMS **0.369°**、`angle` 与 `atan2` 残差 RMS 0.183°、恒定滞后 **−3.760°**、`[5]` 毛刺专项 **0 处**、`stat_byte` 全窗 **0x15。**

③ 圆参数拟合（新工具 `tools/arc_fit.py`）—— 本节的定量核心 - 方法：ILA 窗口只覆盖 **147.9°** 的弧，故不采用"到原点距离"的粗糙口径，而是用**已知** `f_in` 做线性最小二乘拟合 `x[n] = s + j·c = O + B·exp(j·σ·ω·n·Δt)`（`Δt` = 20 ns；σ = ±1 两个旋向都拟合、取残差小的 ⇒ 本数据旋向为 **σ = −1**），并用**不依赖频率的 Kasa 代数圆拟合**交叉校验；干净窗口拟合残差 RMS **2.2–4.0 LSB**（≈噪声量级）。 - 结果（`max|s| = |Os| + A` = S 通道码能达到的最大绝对值）：

窗口	A（Kasa / 正弦）	圆心 O	`max|s|`	距格点 (−1536,+512) 的距离
8.0 Vpp `ila_cap_v9_dbg_state.csv`	**1609.8 / 1608.4**	(−8.7, +21.5)	**1618.5**	**5.7 LSB**
7.6 Vpp `ila_cap_v9_amp76.csv`	**1534.3 / 1530.1**	(+10.0, +29.2)	**1544.2**	**85.4 LSB**

- 幅度比 **1534.3 / 1609.8 = 0.953** ⇒ −4.7%，与 7.6/8.0 = 0.95 **吻合** ⇒ 用户改动被独立证实。
- ★ 关键：新幅度下 **`max|s| = 1544.2 > 1536.5`（余量 +7.7 LSB）** ⇒ S 通道码**仍然会跨过 −1536.5** 这个 10 线翻转边界（跨边界发生在 s 极值 ±5.7° 处）。

④ 判别结论（诚实版） - **(b)「只要 S 通道自己跨 −1536.5 就触发」⇒ 被否决**：码模式（引脚域 0x200↔0x1FF、10 条数据线同时翻转）与幅度无关、`max|s|` 仍有 +7.7 LSB 余量 ⇒ (b) 预测率**基本不变**，而实测 **0 / 20 s。** - **(a)「两通道的 10 线纹波进位必须在同一拍附近同时发生（联合 SSN）」⇒ 得到支持**：8.0 Vpp 时轨迹离格点仅 **5.7 LSB**，而轨迹速度 ≈ A·ω = 1609.8 × 2π × 20076 ≈ **4.1 LSB/拍** ⇒ 两通道进位时刻相差 **≈1.4 拍**（可落在同一拍）；7.6 Vpp 时距离 **85 LSB ≈ 21 拍** ⇒ 联合进位消失。且**最近的其它格点半径是 512√8 = 1448.2（−10.6%）与 512√13 = 1846.1（+14%）**，中间没有替代共振点 ⇒ 幅度一动 5% 就"无共振可用"，与观测到的**干净归零**一致。 - **(c)「坏值 = 真值 × (−1/2) 之类的取值映射」**：它是对**坏值形态**的假设，作为**触发**机制同样预测事件继续 ⇒ 一并被否决（但"覆盖成固定码 vs 叠加固定偏移"的形态分辨**仍未解决**，本轮没有事件可看）。 - ⚠ **本实验排除不了的**：任何**随幅度单调变化**的机制（SSN 幅值、转换速率 dv/dt、裕量变化）—— 不过这类机制只会把率**压低**，很难把 1.5 次/s 干净打成 0。 - ~~★ **决定性的反向检验**：把幅度调大约 2.5%（8.2 Vpp）。(a) 预测**同样归零**（圆移到格点另一侧，距离 ≈ 41 LSB）~~ ⇒ ❌ **已被 §1.54 证伪**：8.2 Vpp 实测事件**不归零**、率升到 **13.65 次/s**（且坏码与事件相位都变了）。该预测的漏洞见 §1.54 ④：只考虑了单一格点，漏掉同一半径 512√10 的另外 6 个格点。

产物清单：`_synth_work/ila_cap_v9_amp76.csv` + `ila_amp76.log`；新工具 `tools/arc_fit.py`（已入库）；20 s 计数为终端输出（未落盘）。 ⚠ 工具接口备注：`tools/glitch_chain.py` 的 `load(P)` 接受**单个路径**、返回 **6 元组** `(df, D2, tag, viol, rstd, rev)`，`D2` 以**列基名**为键；`tools/arc_fit.py` 依赖这一点，且**旋向取 σ = −1**（`x = O + B·e^{-jωt}`）。

1.54 ★★ D 分支：幅度反向检验（8.2 Vpp，2026-10-11 22:31）—— §1.53 的预测被证伪：事件不归零，而是换相位、换坏码、增 9 倍

条件：板上状态**未动**（v9 ILA 位流、`0x0302` = 0x1A、inter = 256、`0x0108` = 0x4004）；用户只把 AFG 幅度由 **8.0 Vpp 改为 8.2 Vpp**（+2.5%），频率仍 20 kHz。

① 率：不但没归零，反而升到 13.65 次/s（`tools/diag_counters.py --port COM6 --seconds 20`）

计数器	t0	t1	Δ
`0x0505 gl_1024_gt5.6deg`	48484	48757	**273 ⇒ 13.65 次/s**
`0x0507 gl_2048_gt11.25deg`	5016	5289	**273 ★★ 每个事件都 >11.25°**（8.0 Vpp 时该档**恒 0**）
`0x0509 gl_4096_gt22.5deg`	24743	24743	0
`0x050B gl_8192_gt45deg`	22589	22589	0
`0x050D gl_hold_cnt`	33403	33676	273（与 `gl_1024` 一致 ⇒ 每坏样本 1 计，§1.52 口径）
`0x0540 cyc_pulse_cnt`	58832	844（16 位回绕）	7548 ✅ 计数器活着

对照：8.0 Vpp ⇒ 30–36 次/20 s（1.5–1.8 次/s，§1.51/§1.52）；7.6 Vpp ⇒ **0**（§1.53）。⇒ 幅度只动 +2.5%，率在 **0 → 1.5 → 13.65 次/s** 之间跨了三个数量级。

② 幅度确实只涨了 2.2%（健康窗 `_synth_work/ila_cap_v9_amp82.csv`，134,738 B，22:31:31）

项	8.0 Vpp	8.2 Vpp
`tools/ila_report.py` 半径 r	1606.1 ± 14.5	**1641.7 ± 13.6**（min 1623.7 / max 1675.2）
`f_in`	20.15 kHz	**20.266 kHz**
`tools/arc_fit.py` 半径 A	1609.8（正弦）/ 1608.4（Kasa）	**1645.0 / 1669.6**
圆心 O / `max|s|`	(−8.7, +21.5) / 1618.5	(+13.9, +11.5) / **1658.9**
线性残差 RMS / 恒定滞后	0.180° / −3.785°	0.180° / −3.785°（同量级）

A 比 **1645.0 / 1609.8 = 1.022**（+2.2%）⇒ 用户改动被独立证实，且**波形健康**（位序直读正确、`angle_raw` 残差 0.279°、`stat_byte` 全窗 0x15、`[5]` 无异常）。

③ 两版毛刺窗（`_synth_work/ila_cap_v9_g82_{1,2}.csv` + `ila_g82_{1,2}.log`，22:31:41/22:31:52）—— 相位与坏码全变了

项	8.0 Vpp（4 版，§1.51/§1.52）	8.2 Vpp（2 版，本节）
坏样本**真值**（邻域中值）	s ≈ −1536…−1539	s ≈ **−1025**
坏码（RTL 有符号 / 引脚域）	**+768** = 0x300 / 0xB00	**+1024** = 0x400 / 0xC00
相对中值偏差	+2304…+2306	**+2049**
波形相位（真值算，非伪值）	**−71.4°**	**−141.2°**
瞬时半径	1620–1622	1635–1637
抽取级 s_dec Δ	9256–9376	7972–8040
`angle_raw` 单拍跳变	+1862…+1884（10.2–10.3°）	**+3162…+3179（17.4°）**
去趋势后 `angle` 过冲	−0.63…−0.80°	**−0.39…−0.53°**（到峰 49–52 拍）
`otr_lat` / `cor_sat`	未置位 / 未见	483–498 拍置位 / 仍未见

- ✅ **系统级好消息**：`angle_raw` 跳变大了 70%，**滤波后 `angle` 的污染反而更小**（v6 同口径对照 +7.38°）⇒ v8 的门控（`ANG_GLITCH_MAX` = 1024 LSB）+ `ang_held` 单拍保持**在更大量级扰动下仍然有效。**
- 量级自洽（定量）：坏样本过 4 抽 1 平均后等效 Δs ≈ 2049/4 = 512（8.2 Vpp）/ 2304/4 = 576（8.0 Vpp）；角度灵敏度 ∂θ/∂s = c/(s²+c²)：8.2 Vpp 用 |c| = 1274、r = 1635 ⇒ **4.77e-4 rad/LSB**，8.0 Vpp 用 |c| = 512、r = 1620 ⇒ **1.95e-4** ⇒ 预测跳变 **14.0° / 6.4°**（实测 17.4° / 10.3°，线性化偏小 ≈1.5×，方向与倍数都合理）⇒ **同一量级坏样本，因事件相位的 |c| 不同而跨过/不跨过 11.25° 档**，这正是 `gl_2048` 由"恒 0"变成"273"的原因。

④ ★ 我上一节的预测被证伪（诚实记录，不许糊弄） - §1.53 结尾写的是"(a) 预测**同样归零**（圆移到格点另一侧，距离 ≈ 41 LSB）" ⇒ **错。**实测事件不归零，率还升到 13.65 次/s。 - 错在哪里（自我更正）：我只算了**单一格点**（−1536,+512）和最近的两个格点**半径**（512√8 = 1448.2、512√13 = 1846.1），**漏掉了同一半径 512√10 的另外 6 个格点。**半径 512√10 的 8 个格点 = (±512, ±1536) 与 (±1536, ±512)，相位 **±18.4° / ±71.6° / ±108.4° / ±161.6°。** - 逐点重算"圆到格点距离 = |\|O→L\| − A|"（O = (+13.9,+11.5)、A = 1645）：(−1536,−512) = **9.1 LSB**（相位 **−108.4°**）、(−512,−1536) = **10.6 LSB**（**−161.6°**）、(−1536,+512) = 16.3 LSB（−71.6°），其余 19–43 LSB。 - ⇒ 修正后的格点模型应预测事件出现在 **−108.4° 或 −161.6°**；**实测 −141.2°**，离这两个预测点分别 33° / 20° ⇒ **格点相位预测同样未获证实。** - ⚠ **追加更正（§1.55 ⑤，2026-10-11 22:39）**：上面这条是 **512 格点 + 正弦拟合**下的结果；改用 **256 格点 + Kasa 拟合**后，两个有实测的窗口（8.0 / 8.2 Vpp）预测的最近格点**与实测事件点完全一致**（2/2 命中）。⇒ 该结论应改为**悬置**（取决于格点粒度与拟合选择），不能算"已证伪"。 - ⚠ 更要紧：8.0 Vpp 离格点 **5.7 LSB** ⇒ 1.5 次/s；8.2 Vpp 离格点 **9.1–10.6 LSB** ⇒ **13.65 次/s** ⇒ **"越近越易发"的单调律不成立** ⇒ §1.53 里支持 (a) 的那条推理（85 LSB ⇒ 归零）**本身就不是充分证据**，"到格点的距离"很可能**根本不是控制变量。**

⑤ 本轮唯一稳定的不变量：坏样本的真值总落在 S 通道引脚码 0x200 整数倍的边界上 - 8.0 Vpp：真值 s = −1536 ⇒ 引脚码 **0x200**（`0x1FF ↔ 0x200`，低 10 位同时翻转）； - 8.2 Vpp：真值 s = −1025 ⇒ 引脚码 **0x3FF ↔ 0x400**（低 10 位 + 位 10，该通道量级最大的一次进位）。 - ⇒ 两个幅度下"S 通道正在做 0x200 整数倍边界进位"都成立；幅度变化改变了**轨迹扫到哪个 0x200 倍数**（−1536 = −3×512 → −1024 = −2×512），相位随之从 −71.4° 移到 −141.2°。这与"**ADC 主进位瞬间的亚稳/误码（major-carry sparkle）**"这一经典失效模式一致，也能解释坏码与真值之间**看不出简单位关系**（0x300 与 0x400 与真值的 XOR 分别是 0x900 与 0xFFF）。 - ⚠ 三条**不闭合**（如实记录）：(i) 8.0 Vpp 时轨迹同样扫过 0x400 边界（s = −1024），那一档为什么不产生 >5.6° 的大毛刺（可能只产生**未被计数器记下**的小毛刺，当前数据无法分辨）；(ii) 8.2 Vpp 为什么选中较**快**的 −1024 档（|c| = 1274 ⇒ ≈3.2 LSB/拍）而非更慢的 −1536 档（|c| ≈ 588）——"斜率越慢越易亚稳"的直觉被这批数据打掉；(iii) 率对幅度 **+2.2% ⇒ ×9** 的陡度没有任何定量模型解释（格点距离、SSN、dv/dt 都只能单调缓变）。 - ⚠ 口径澄清：§1.53 对 (b)「仅 S 通道跨 −1536.5」的**否决依然成立**（7.6 Vpp 下 `max|s| = 1544.2` 仍跨该边界而率为 0），但既然"距离"不是控制变量，该否决的**理由要弱化**为"码域条件不是充分条件"，而非"距离够远"。

⑥ 下一步候选（成本递增，待选） - **(A) 同幅度复现性**：8.2 Vpp 再抓 2–3 版 `ang_glitch` 窗（一次提权）⇒ 判"相位是否被幅度确定性地选择"（2/2 相同若 8 档等概率则 p ≈ 1/64，需更多样本）。 - **(B) 幅度阶梯率曲线**（只需计数器、**不提权**）：8.4 / 8.6 / 9.0 Vpp 各 20 s ⇒ 看率是"陡壁"还是"渐变"，以及 `gl_4096` / `gl_8192` 何时出现。 - **(C) 模拟侧示波器**：探 AD9238 的 D11/D10 与 s 通道输入 ⇒ 唯一能判"**ADC 内部误码 vs 板级信号完整性**"的手段（§1.51 已排除"FPGA 侧采样混码"：坏码 D11 = 0/1 与相邻样本相反，混码不可能改变 D11）。

产物清单：`_synth_work/ila_cap_v9_amp82.csv` + `ila_amp82.log`；`_synth_work/ila_cap_v9_g82_{1,2}.csv` + `ila_g82_{1,2}.log`；分析输出 `_synth_work/glitch_chain_g82.txt`；20 s 计数为终端输出（未落盘）。

1.55 ★★★ D 分支：8.0 Vpp 对照复现 + 坏码的"高位被抬高"结构 + 两处自我更正（2026-10-11 22:39）

条件：板上状态未动（v9 ILA 位流、`0x0302` = 0x1A、inter = 256、`0x0108` = 0x4004）；用户把 AFG 由 8.2 Vpp **设回 8.0 Vpp**（频率仍 20 kHz）。目的：判"13.65 次/s 是幅度造成的还是时间漂移"。

① 对照复现：率回到 2.3–2.6 次/s，`gl_2048` 回 0 ⇒ 幅度因果成立

计数器（两个连续 20 s 窗）	窗口 1（t0→t1）	窗口 2（t1→t2）
`0x0505 gl_1024_gt5.6deg`	45771 → 45823 = **Δ52（2.6 次/s）**	45823 → 45869 = **Δ46（2.3 次/s）**

计数器（两个连续 20 s 窗）	窗口 1（t0→t1）	窗口 2（t1→t2）
`0x0507 gl_2048_gt11.25deg`	17896 → 17896 = **Δ0** ✅ **回到 0**	Δ0
`0x0509 gl_4096` / `0x050B gl_8192`	Δ0	Δ0
`0x050D gl_hold_cnt`	26681 → 26733 = Δ52	26733 → 26779 = Δ46
`0x0540 cyc_pulse_cnt`	Δ7555 ✅	Δ7546 ✅

⇒ 与 §1.51/§1.52 的 1.5–1.8 次/s 同量级；8.2 Vpp 的 **13.65 次/s 归因于幅度**，不是时间/温度漂移。（1.5–1.8 与 2.3–2.6 的差别见 ② 的幅度轴不确定度，不当判据。）

② 四窗圆拟合总表（`tools/arc_fit.py`）—— 幅度轴有 ±1% 不确定度

窗口	f_in（传入）	正弦 O / A / RMS	Kasa O / A / RMS	实测率
`_synth_work/ila_cap_v9_amp76.csv`（7.6 Vpp）	20.076 kHz	(+15.3, +27.2) / 1530.1 / 4.00	(+10.0, +29.2) / 1534.3 / 2.62	**0**
`_synth_work/ila_cap_v9_dbg_state.csv`（8.0 Vpp 首轮）	20.106 kHz	(−13.0, +20.0) / 1606.5 / 3.91	(−8.7, +21.5) / 1609.8 / 2.17	1.5–1.8 次/s
`_synth_work/ila_cap_v9_amp80_ctrl.csv`（8.0 Vpp 对照，本节）	19.877 kHz	(+6.2, +7.0) / 1625.6 / 3.91	(+2.7, +12.3) / 1620.9 / 1.93	2.3–2.6 次/s
`_synth_work/ila_cap_v9_amp82.csv`（8.2 Vpp）	20.266 kHz	(+13.9, +11.5) / 1645.0 / 8.63	(−0.8, +41.1) / 1669.6 / 2.01	**13.65 次/s**

- 对照健康窗（`ila_amp80_ctrl.log`，22:39:13）：位序直读正确（bit7 违例 0 / 位反 1024；std(hypot) 7.03 vs 577.22）、**f_in = 19.877 kHz**、`ila_report` 半径（**到原点**的距离，非拟合半径）1626.0 ± 7.0、`angle_raw` 线性残差 RMS 0.169°、angle vs atan2 0.177°、恒定滞后 −3.724°、`[5]` 毛刺专项 0 处（256 个 `dec_valid` 更新点）、`stat_byte` 全窗 0x15。
- ⚠ **两套拟合的 A 差 ~1.5%（25 LSB）**（149° 短弧下 A 与圆心强相关）：7.6 Vpp 4 LSB / 8.0 首轮 3.3 / 8.0 对照 4.7 / 8.2 Vpp 24.6。Kasa 在四窗 RMS 全胜（2.62/2.17/1.93/2.01 vs 4.00/3.91/3.91/8.63）。
- ⚠ 对照窗 A = 1620.9 比"同样名义 8.0 Vpp"的首轮 1609.8 **高 0.8%** ⇒ AFG 同档位不可完美复现 ⇒ **1.7 vs 2.4 次/s 不是有意义差异**，幅度轴的绝对不确定度 ≈ ±1%。

③ ★ 坏码的结构（新）：坏码 = 真值 + 0x800 + δ，即"正常进位 + 位 11 被抬高 + 一个低位被抬高"

幅度	真值（引脚码）	坏码（引脚码）	差	δ
8.0 Vpp	s = −1536 ⇒ **0x200**	+768 ⇒ **0xB00**	0x900 = 2304	**0x100（位 8）**
8.2 Vpp	s = −1025 ⇒ **0x3FF**	+1024 ⇒ **0xC00**	0x801 = 2049	**0x001（位 0）**

- 两个坏码的**低 8 位全为 0**，且都可读成"过渡后的正常码 + 位 11 抬高 + 一个低位抬高"：8.0 Vpp = (0x1FF→0x200 进位) + 位 11 + 位 8；8.2 Vpp = (0x3FF→0x400 进位) + 位 11。
- **★ 排除"竞争/部分更新（race）"**：8.0 Vpp 事件前后 s 的相邻真实码是 0x1FF / 0x200、8.2 Vpp 是 0x3FF / 0x400 —— 这**四个码的位 11 全是 0** ⇒ 坏码里的位 11 **既不属于旧码也不属于新码** ⇒ 不是"旧/新码逐位混合"，而是**该数据线被瞬态抬到高电平（glitch）**。这也再次印证 §1.51"FPGA 侧采样混码被排除"（混码不可能凭空造出位 11）与 §1.54 ⑤"XOR 看不出简单位关系"（0x900 / 0xFFF 正是"被抬高的位"这个视角下的产物）。
- 未定：这个抬高发生在 **ADC 输出级 / PCB 走线（板级 SI）** 还是 **ADC 内部**——只有示波器（下一步 (C)）能分开。

④ 256 格点判据（引脚域低 9 位整组翻转）—— 无拟合的观测事实

- 8.0 Vpp 真值 (s, c) = (**−1536, +512**) ⇒ 引脚 (0x200, 0xA00)；8.2 Vpp 真值 (s, c) = (**−1025, −1274**) ⇒ 引脚 (0x3FF, 0xB06)。
- 到 0x100 整数倍的距离：s = **0 / 1 LSB**，c = **0 / 6 LSB** ⇒ **两个通道在同一拍都处在"进位到 0x100 倍数"的瞬间**（跨 0x100 倍数时引脚码低 9 位整组翻转，≥9 条数据线同时跳变，两条总线联合 ≥18 条）。
- 这是对 §1.54 ⑤ 的**升级**：由"仅 S 通道位于 0x200（512）边界"细化为"**双通道同时位于 0x100（256）边界**"（512 格点是 256 格点的子集，故不冲突）。命中概率上：两批事件共 4 个坐标，3 个精确落在 0x100 倍数上、第 4 个差 6 LSB ⇒ 不是巧合。

⑤ 拟合判据（较弱，逐项如实列出）：新建 `tools/lattice_scan.py`（256 格点共振预测器：`nearest_lattice(Os,Oc,A,step,ab_lim)` 返回（圆到格点距离，Ls，Lc，观测相位 `atan2(Ls,Lc)`，圆心距离）；`--table` 验证模式、`--o --a` 扫描模式）用两套 (O, A) 分别找"最近的 256 格点"：

窗口	Kasa 预测最近格点	正弦预测最近格点	实测事件点
7.6 Vpp	(−256, +1536) 4.2 LSB @ −9.5°	(+256, +1536) 2.2 LSB @ +9.5°	（无事件）
8.0 Vpp 首轮	**(−1536, +512) 5.7 LSB @ −71.6°** ✅ **完全一致**	(+512, +1536) 2.2 LSB @ +18.4° ✗	(−1536, +512) @ −71.6°
8.0 Vpp 对照	(+1536, −512) 0.4 LSB @ +108.4°	(−1536, −512) 1.6 LSB @ −108.4°	（未抓窗）
8.2 Vpp	**(−1024, −1280) 1.4 LSB @ −141.3°** ✅ **完全一致**	(−1280, +1024) 2.0 LSB @ −51.3° ✗	(−1024, −1280) @ −141.3°

- ⇒ **2/2 命中只在 Kasa 拟合下成立**；换成正弦拟合两个窗口都不命中。所以：既不能宣称"格点相位预测已被证实"（换拟合就翻），也不能像 §1.54 ④ 那样宣称"已被证伪"（那是在 **512 格点 + 正弦拟合**下得出的）——**并列两套结果，结论悬置。**

⑥ 距离不是控制变量（扫描证明）：以 O = (1, 26)（四窗 Kasa 圆心均值）扫 A = 1590…1710 步 10，最近 256 格点的距离 = 4.1 / 5.2 / **0.1** / **0.4** / 1.6 / 3.5 / 4.8 / 0.2 / 9.8 / 13.6 / 3.6 / 4.9 / 3.3 LSB ⇒ **13 个采样幅度里有 9 个都有格点落在 6 LSB 内**（"擦过格点"是普遍事件，不是稀有事件），而**距离最小的两处（0.1 / 0.4 LSB）对应的恰恰是低率（1.7 / 2.4 次/s）**，距离 1.4 LSB 处是 13.65 次/s、距离 4.2 LSB 处是 0 ⇒ 与 §1.54 ④ 一致：**"到格点的距离"不是率控制变量。**

⑦ ★★ 自我更正两处（诚实记录） - **§1.53 的"7.6 Vpp 仍跨边界"符号错**：`max|s| = |O_s| + A` 是**正侧**极值；**负侧**极值 = `O_s - A` = +10.0 − 1534.3 = **−1524.3 LSB**（Kasa）/ +15.3 − 1530.1 = −1514.8（正弦）⇒ **7.6 Vpp 时 S 码根本没到 −1536.5**，那一档"归零"因此有平凡解释（码压根没到边界）、**失去判别力。** §1.53 对 (b)"仅 S 通道跨 −1536.5"的否决**只剩一条理由**：8.2 Vpp 时 −1536 交叉仍存在（负侧 −1670.4），事件却移到 −1024 格点。 - **§1.54 ④ 的"格点相位预测未获证实"结论过强**：它是 512 格点 + 正弦拟合下的结果；换 256 格点 + Kasa 拟合后 2/2 命中（见 ⑤）。⇒ 该条改为"结论悬置，取决于拟合与格点粒度"。 -（§1.54 ④ 关于"我的 8.2 Vpp 归零预测是错的"这一点仍然成立，不回改。）

⑧ 未闭合（更新）：(i) 率对幅度 **+2.2% ⇒ ×9** 的陡度仍无定量模型（格点距离已排除；dv/dt、SSN、码跳变量级都只能缓变）；(ii) 为什么 8.2 Vpp 选中较**快**的 −1024 档（≈3.2 LSB/拍）而 8.0 Vpp 落在较慢的 −1536 档（≈1.2 LSB/拍）——"越慢越易亚稳"的直觉被打掉；(iii)"位 11 + 一个低位"具体由哪条线的感应造成，需要示波器。

⑨ ★ **一条被"复活"的候选修法**：坏码既然是**数据线被瞬态抬高**（而非相邻样本竞争），§1.51 用"坏样本量级 +2307 LSB 远超竞争可能"来**否决"改下降沿/延迟采样"的理由不再成立**（glitch 是整位翻转，量级论证不适用）⇒ **换沿或延迟采样**重新成为廉价可测的缓解候选：只需改采样边沿、重综合、再用 `0x0505` / `0x0507` 计数直接判定有效/无效，并同步核对 `constr/enc_top.xdc` 的 ADC 输入口时序余量。

⑩ **下一步候选（成本递增，待选）**：(A) 同幅度复现性（8.2 Vpp 再抓 2–3 版 `ang_glitch` 窗，一次提权）；(B) 幅度阶梯率曲线（只需计数器、**不提权**）；(C) 模拟侧示波器探 AD9238 的 D11/D10 + s 通道输入（唯一能分"ADC 内部误码 vs 板级 SI"的手段）；(D) ★ 新：换沿/延迟采样的计数器判据实验（需一次综合 + 烧录）。

产物清单（本节）：`_synth_work/ila_cap_v9_amp80_ctrl.csv` + `ila_amp80_ctrl.log`；`tools/lattice_scan.py`（新增）；`tools/arc_fit.py`（§1.53 新增，本节复用）；20 s 计数为终端输出。

1.56 ★★★★ 会话重启（2026-10-12 11:00–）：板上位流是生产版（无 ILA），但 A/B/A 实测把"位流"这条解释否掉了——同场 A–率散点平（0.05–0.5 次/s，与幅度无关）

① 会话事实（用户答复 + 板上读取）

- 板**断电**过、AFG **今早重新开机并重设**（用户答）⇒ 昨天 22:40 之后的计数器归零有了平凡解释，不是硬件异常。
- 计数器上电初值：`0x0505/07/09/0B` = **3 / 3 / 3 / 3**、`0x050D` = 1（昨天末值为 45869 / 17896 / … / 26779）；`0x0540` 仍在跑（`0x05F2` 读 **9**）⇒ FPGA 从 flash 自举。
- 掉电后复位默认（11:34 读）：`0x0302` = **0x1A**（与 `rtl/reg_file.v:282 r_filt_k <= 8'h1A;` 的 kp=2/ki=3 一致 ✅）、`0x0405` = 0x0001（otr 粘滞）、`0x053A` = 0x0000、`0x0101` = `0x0102` = 0、`0x0108` = 0x4004（z_width 4）、`0x0109` = 0x0040（abzen）。

② ★ 位流来源判定：flash 里那份是生产版（无 ILA）——新增只读工具 `tools/ila_probe.tcl`

```
Labtools 27-1434: Device xc7a35t (JTAG device index = 0) is programmed with a design
                  that has no supported soft debug core(s) in it.
ILAPRB   refresh_err=<none>   ilas = 0   vios = 0
```

- 该脚本**绝不下载位流、绝不武装触发、绝不写寄存器**（无 `set_property PROGRAM.FILE`、无 `program_hw_devices`、无 `run_hw_ila`），只用 `refresh_hw_device` + `get_hw_ilas/get_hw_vios` 读回。用户确认"我（或队友）用 Vivado 烧过 flash"。
- ⇒ **flash 里那份 = 无 ILA 的 v9 生产版。**（残留不确定：flash 的 .mcs/.bin 生成不在本仓库日志里，**无法校验它是否就是盘上 `_synth_work/enc_top.bit`（20:43:01）那一份编译结果**；⑦ 的 A/B/A 顺带验证二者行为是否一致。）
- ⚠ `0x05F0/0x05F1` = DEVICE_ID 0x0C / REV 0x10（⇒ 常读 `0x100C`），**与版本无关**，不能用来区分位流，故必须靠 JTAG 探测。

③ ★★ 时间线：§1.51–§1.55 的全部计数都取自 ILA 版位流

时刻	事件（`_synth_work/ila_*.log`）	板上位流
10-11 20:43:01	生产版 v9 定版（`enc_top.bit`，1,226,648 B）	—
10-11 21:04:26	`flash_v9.log` = `ILA_MODE=restore` + `enc_top.bit` ⇒ **JTAG 下载生产版**（名字叫 flash，其实不是烧 flash）	生产版
10-11 21:24:56	ILA 版 v9b 定版（`enc_top_ila.bit`，1,432,868 B；`.ltx` 21:24:45）	—
10-11 **21:35:12**	`ila_cap_v9_z.log` = `stage1` ⇒ 下载 ILA 版（抓 Z 脉宽）	ILA 版
10-11 **21:37:20**	`flash_v9_prod_restore.log` = `restore` + `enc_top.bit` ⇒ **生产版放回**	生产版
10-11 **21:41:21**	`flash_v9_ila_sweep.log` = `restore` + `enc_top_ila.bit` ⇒ **ILA 版再上**	**ILA 版**
10-11 21:43–22:39	之后**每一个** log 都是 `mode=capture`（不重下位流）：21:43–21:49 增益/插值 sweep、21:54 `dbg_state`、21:56 `glitch1..4`、22:07–22:08 `g197`、22:25 `amp76`、22:31 `amp82`、22:39 `amp80_ctrl`	**ILA 版（至会话结束）**
10-12 11:37–11:42	本节 ⑥ 的同场扫描	**生产版（flash 自举）**

- ⇒ **1.5 / 2.3–2.6 / 13.65 次/s 三档全部产生于 ILA 版位流**（21:41:21 之后再没有下载动作，只有 `capture`），今天的 0.17–0.50 次/s 产生于生产版位流。⇒ **必须用 A/B/A 判别**；该判别已在 ⑦ 完成，结论是**位流因素被否决**（同一份 ILA 位流今天只有 0.05 次/s）。
- ★ **本仓库工具链从不写 SPI flash**（全部 `_synth_work/*.log` 里没有 `write_cfgmem` / `program_hw_cfgmem` / `create_hw_cfgmem`，只有 `program_hw_devices`）⇒ flash 里那份是**手工/图形界面烧的**，其内容与本仓库的校验记录无关，只能靠 ② 的 JTAG 探测定性（无 ILA + `0x05F2` = 9 ⇒ 一份 v9 生产版）。
- 两版时序报告对比（`_synth_work/impl_timing.rpt` 20:42:50 vs `impl_ila_timing.rpt` 21:24:43）：生产版 Setup **26 failing / WNS −3.274 ns / TNS −84.283 ns**、Hold 0 failing / WHS +0.024；ILA 版 Setup **26 failing / WNS −3.323 ns / TNS −84.533 ns**、Hold 0 failing / WHS +0.048。⇒ **ADC 输入侧失败端点数量与最差裕量几乎相同（差 0.05 ns）**，"ILA 把输入裕量压坏了"这条解释**得不到时序报告支持**（仍不能排除 ILA 改变了具体路径/地弹分布，见 ⑥ 的 A/B/A 判据）。

④ `0x0540` 语义订正：它就是 `f_in`（长窗要处理 16 位回绕）

- RTL：`rtl/enc_top.v:1048-1077`，`calib_cyc_tgl` 每半圈翻转、`:1063 if (calib_cyc_tgl) calib_cyc_pulse_r <= 1'b1;` ⇒ **每个电周期 1 拍。**
- 本节实测：Δ = **10943–10952 / 30 s** ⇒ 加 **9 次** 16 位回绕 ⇒ `f_in = (9×65536 + Δ)/30` = **20.03 kHz**（与 §1.52–§1.55 的 19.877–20.266 kHz 同档 ✅）。⇒ 旧口径"该计数器不能当电周期率"应改为"**长窗（>3.28 s）有回绕歧义、1–3 s 窗无歧义**"。
- ⚠ 反面教训保留：遥测 CSV（`uart_scope.py --csv`）的 `idx` 是**工具侧帧计数**、CSV 无时间基准 ⇒ **不能用遥测的 `cycle_cnt` 算 f_in**，必须用本寄存器的短窗 Δ。

⑤ 测量纪律（今天确立，此后一律执行）

- 幅度轴**只用同时刻遥测实测半径**（`uart_scope.py` 的"校正后半径 r" **÷16 = 原始码口径**），**不看 AFG 显示值**：今天 A 在 11:08–11:36 之间于 1609.9–1654.6 之间漂了 2.3%，AFG 同档位不可复现（§1.55 ② 已见 0.8% 差）。
- 每个"率"点必须与**同时刻**的 A 配对测量（窗口前后各夹一次 1 s 遥测）。
- 测率期间用户不得碰 AFG/板/接线：11:23–11:29 用户拧旋钮期间遥测半径塌到 **310.3**（min 281.8，正常 ≈26000）、状态位 `scamp`、`oflow` 7677、毛刺计数器伪像 500–1000 次/s ⇒ **那 6 分钟数据作废。**

⑥ 同场 A–率散点（生产版位流，`_synth_work/avr_scan.txt`，新工具 `tools/avr_scan_report.py`）

完整 60 个 30 s 窗（11:37:11–12:10:26，AFG 名义 8.1 Vpp / 20 kHz，用户未碰任何东西）：

量	结果
A（遥测半径 ÷16）	**1622.7 … 1650.9**（跨度 28.2 LSB = **1.72%**，33 分钟内）
`gl_1024` 率	**0.000 … 0.500 次/s**（57/60 窗非零，中位 ≈0.2 次/s）
`gl_2048` 率	**逐窗与 `gl_1024` 相等**（少数窗差 1 个计数）⇒ 今天**每一个**毛刺都落在 **11.25°–22.5°** 带内
`gl_4096` / `gl_8192`	**全 0（0/60 窗）**
OTR（`0x0405` = 0x01）	**17/60 窗**
Pearson(A,率) / Spearman(A,率)	**−0.179 / −0.212** ⇒ 在 1.72% 幅度跨度内**率与幅度无关**（甚至微负）
零率窗的 A 区间	**1631.5 … 1643.2**（完全落在非零窗的 A 区间内部）⇒ 零率不是"幅度不够"
f_in	20.03 / 20.33 kHz 两簇（↑④；两簇差 = 30 s 窗长误差，非真实频率变化）

⇒ 今天在生产版上 A ≈ 1623–1651 时率恒为 **0.05–0.5 次/s**，而昨天 ILA 版在**同一 A 区间**为 2.3–13.65 次/s。注意 ⑤ 的幅度轴：今天 A 的上沿 1650.9 已**超过**昨天 13.65 次/s 那一窗（1641.7，原点半径口径）⇒ **幅度陡壁不能解释今天的平坦与低率。**

⑦ ★★ 位流 A/B/A 判别（12:15–12:17 实测完成）： 位流因素被排除****

相位	时刻	A_mean（遥测÷16）	`gl_1024` /s	`gl_2048` /s	`gl_4096` /s	`0x0405`
prod（= ⑥ 的 60 窗）	11:37–12:10	1622.7–1650.9	0.000–0.500	同左	0	17/60 窗 OTR
ila（`ILA_MODE=restore` + `enc_top_ila.bit`）	12:15:09	**1649.7**	**0.050**	0.050	0	0x00
prod（回烧 `enc_top.bit`）	12:17:03	**1648.5**	**0.050**	0.033	0	**0x01**

- 用的 `_synth_work/enc_top_ila.bit` 与昨晚 21:24:56 那份是**同一个文件** ⇒ 昨晚出 13.65 次/s 的位流，今天在 A = 1649.7（比昨晚那窗 1641.7 **还高 0.5%**）下只出 **0.05 次/s**，**差 ≈270×。**
- ⇒ **"ILA 版位流把 ADC 输入裕量压坏"这条解释被实测否决**（生产版与 ILA 版今天都在 0.03–0.50 次/s；③ 的时序报告对比本来也不支持它）。③ 的时间线事实仍然成立（昨晚的幅度序列确实全在 ILA 版上测的），但它**不能**用来解释两天的差异。
- 工具：`tools/bitstream_ab.py`（`--phase prod|ila|none`，先 `ILA_MODE=restore` 下载再采一个点，追加 `_synth_work/bitstream_ab.csv`，每点另存逐帧遥测 CSV 到 `_synth_work/ab_pts/` 供 `arc_fit` 拟合残余圆心）。

⑧ 未闭合（更新）：既然**位流**（⑦）与**幅度**（⑥ 无相关性；今天 A 上沿 ≥ 昨天 13.65 次/s 那一窗）在今天都被排除，控制变量只剩**我们仪器量不到的外部条件**：

1. **AFG 输出状态**：今天 AFG 被重新开机；今天 A 在 33 分钟内自行摆动 **1.72%**（11:08–11:36 更达 2.3%，其中 11:23–11:29 是用户拧旋钮）。这种幅度抖动不像温漂（不单调、双向）。
2. **接线/连接器接触**（今天板/线被重新插过）。
3. **板温 / ADC 自身状态**（掉电重启后标定重跑）。 - ⇒ §1.53/§1.54 的"幅度因果"与 §1.52 的"1.5/2.3–2.6/13.65 次/s 序列"**在生产版上不可复现**，在报告里只能作为"那一晚那一套设备状态下的观测"引用，**不能写成 RTL/幅度决定的规律。** - 待判别的外部因素入口（成本从低到高）：① 用户用示波器直接看 AFG 输出（有无失真/毛刺/幅度抖动）；② 重插 BNC/排线后再跑一次 60 窗扫描；③ 把 AFG 换一个频率（如 10 kHz 或 40 kHz）看率是否随 f_in 变化；④ 用 ILA 抓今天稀有的毛刺（`ILA_TRIG=ang_glitch`，0.05 次/s ⇒ 平均 20 s 一次）与昨晚的原始码形态逐位对比。

⑨ ★★ 事后更正（2026-10-12 13:00，依据见 §1.57）：本节 ⑤ 的"幅度轴 = 遥测校正后半径 **÷16 = 原始码口径**"**不成立**——2 s 遥测窗口里 s/c 的相角只覆盖 **8/36 格（约 80°）**，而"校正后半径 r"是**到原点的距离**（`tools/uart_scope.py:198 r = np.sqrt(s*s+c*c)`），弧段位置一漂它整体就抬/压。⇒ ⑥ 的"1.72% 幅度跨度"与 ⑦ 的 A 列**都不是幅度**，本节那两小时"跨会话 15× / 250× 幅度差"的解释**全部作废**（既不是位流、也不是幅度）。⇒ 幅度轴改用 **ILA 原始码 + 自由频率拟合**（`tools/arc_fit_free.py`）。另：⑧ 列的外部因素作为**触发条件**已被 §1.57 ④ 排除（事件位置与幅度无关、坏码逐位结构固定）。

产物清单（本节）：`tools/ila_probe.tcl`（新增，只读探测）、`tools/avr_scan_report.py`（新增，A–率散点解析）、`tools/amp_point.py`（新增，单点采集，`--csv-dir` 存逐帧遥测）、`tools/bitstream_ab.py`（新增，位流 A/B/A）、`_synth_work/avr_scan.txt`（60 窗扫描原始日志）、`_synth_work/bitstream_ab.csv`（A/B/A 结果）、`_synth_work/ab_pts/`（各点逐帧遥测 CSV）、`_synth_work/ila_probe.log`；时序对比取 `_synth_work/impl_timing.rpt` / `impl_ila_timing.rpt`。

1.57 ★★★★★ D 分支毛刺逐位归因：单拍、只错 D11–D8、只在"两通道高位组同拍翻转"的 256 格点上发生；并用采样相位判别实验（v10）判定"接口 vs ADC 内部"（2026-10-12 12:20–）

① 方法论更正：§1.56 ⑤⑥ 的"幅度轴"作废

- 新工具 `tools/arc_check.py` 检查 2 s 遥测 CSV（`_synth_work/arc_check.csv`，19906 帧）：遥测 s/c 的相角覆盖只有 **8/36 格 ≈ 80°**（28 格空）；s ∈ [−20764, +8908]、c ∈ [+16204, +26508]（×16 单位）。
- ⇒ `tools/uart_scope.py:198 r = np.sqrt(s*s+c*c)` 是**弧段上的原点到点距离**，弧段漂移就使它整体变化 ⇒ **它不是幅度。**§1.56 ⑥ 的 1.72% 跨度、⑦ 的 A/B/A 表都建立在错误的幅度轴上，**其"幅度差 15×/250×"结论作废。**
- 口径（把 §1.56 ⑤ 的条件写清）：板上 `0x0510/12 = 0`、`0x0514/16 = 0x8000`、`0x0518/1A = 0` ⇒ `rtl/sc_correct.v` 的六参数校正当前是**单位变换**；`rtl/enc_top.v:271-274`（`s_ext = {{4{s_raw[11]}}, s_raw}; s_q15 = s_ext <<< 4`）的 12→16 位对齐使遥测值 = **原始码 ×16** ⇒ "÷16 = 原始码"**只在系数为单位值时才成立**；一旦真标定（`rtl/calib_ctrl.v:67 REF_Q15 = 26214`）它变成"相对标定时刻的比值"。
- 上电自动标定**没有写入任何系数**（`0x053A` = 0、`0x0502` = 0x4000）⇒ 校正路径对毛刺行为**没有影响** ⇒ D 分支不是校正算法问题（另见 b16/§1.56 ⑨）。

② 可信的拟合方法：新工具 `tools/arc_fit_free.py` 把频率当**自由参数**做抗差 5 参拟合（Kasa/正弦皆可，3 轮剔野点），与 Kasa 一致到 ~1 LSB。旧 `tools/arc_fit.py` 的**固定频率正弦**拟合在 f_in 偏 0.1% 时会把 A/O 拉偏（同窗 RMS 8.17 vs 3.09、A 差 ~8 LSB）⇒ 本节一律用自由频率拟合 / Kasa 口径。

③ "率 ∝ 到格点距离"被否掉（撤回 §1.55 ④ 的相关性表述）

窗口（ILA 版）	自由频率/ Kasa 拟合 A / O	到最近 256 格点距离	同时刻率
10-11 `amp76`	1534.3 / (+10.0,+29.2)	53.9 LSB	0 次/s
10-11 `dbg_state`	1609.8 / (−8.7,+21.5)	5.67	1.65
10-11 `amp80_ctrl`	1620.9 / (+2.7,+12.3)	4.20	0（20 s 窗 Δ0）
10-11 `amp82`	1669.6 / (−0.8,+41.1)	1.40	13.65
10-12 12:30 `ila_cap_g12_raw`	1648.0 / (+1.3,+40.1)	8.78	0.05
10-12 12:59	1620.0 / (+26.9,+25.1)	14.28	0.05

⇒ 距离 0.44 LSB 的窗只有 2.45 次/s、而 4.20 的是 0 ⇒ **单调关系不成立。**

④ ★★ 逐位归因（新工具 `tools/glitch_bits.py`）——本分支最硬的证据

方法：自由频率拟合当参考轨迹（抗差后残差 RMS ≈ 3 LSB），列出 |残差| > 200 LSB 的拍，报 Δs / Δc、归到最近位权、**以及出错那一拍受影响通道的真值。**跑遍 10 个抓包（`glitch1..4`、`g197_1..4`、`g82_1..2`）：

- **每一次事件都只有 1 拍宽（20 ns），且永远在 s 通道**；c 通道干净（|Δc| ≤ 7 LSB，就是正常的 2–4 LSB 步进）。⇒ 单通道、单拍。
- 原始码对照（±4 拍）：

抓包	事件前	事件拍	事件后	真值应当是
`ila_cap_v9_glitch1.csv`	`s=0x9FF` (−1537)	**`s=0x300` (+768)**	`s=0xA00` (−1536)	`0xA00`
`ila_cap_v9_glitch2.csv`	`s=0x9FF` (−1537)	**`s=0x300` (+768)**	`s=0xA00` (−1536)	`0xA00`
`ila_cap_v9_g197_1.csv`	`s=0x9FF` (−1537)	**`s=0x300` (+768)**	`s=0x9FF` (−1537)	`0xA00`
`ila_cap_v9_g82_1.csv`	`s=0xBFF` (−1025)	**`s=0x400` (+1024)**	`s=0xBFA` (−1030)	`0xC00`

（同拍 c 通道：`0x1FF→0x206`、`0xAFF→0xB03`，即正常步进。导出脚本 `_synth_work/dump_raw.py`。）

- ★ 三条结构性事实： 1. **坏拍的低 8 位是对的，错的只有高 4 位（D11–D8）**：真值正在跨 `0x9FF→0xA00`（低 8 位同拍全翻转）与 `0xBFF→0xC00`，坏拍读到的低 8 位正好等于跨过之后的 `0x00`。 2. **坏码是"整齐"的固定值**：同一簇里每次都是同一个码（8 个事件全 `0x300`、2 个全 `0x400`）；`glitch_bits` 报的 Δs 在 2306.3–2307.8 间变，正是为了让坏码恒为 **768** ⇒ 不是"叠加固定偏移"，而是**采样值被一个固定码替换。** 3. **不是"前后两码的亚稳态混合"**：`0x9FF` (1001 1111 1111) 与 `0xA00` (1010 0000 0000) 的 bit11/bit10/bit9 都是确定的（1/0/0 与 1/1/0），而坏码 `0x300` 的 bit11 = 0、bit9 = 1 **与两者皆不符** ⇒ 混合不可能产生它。
- 事件**位置相位锁定**在两簇：`s ≈ −1539, c ≈ +518`（= 格点 (−6,+2)·256）与 `s ≈ −1022, c ≈ −1280`（= (−4,−5)·256）——正是**两个通道的高位组同拍翻转**（两通道 256 边界同时被跨过）的瞬间。

⑤ 机制候选（都能解释"只错高位"，需实验区分）

- **(a) ADC→FPGA 捕获接口裕量不足**：`rtl/enc_top.v` 头注与 `constr/enc_top.xdc` 都写着"建立余量约 3ns"。v9 相位下 `ad_clk` 比 `clk_i` 晚 10 ns 变高 ⇒ ADC 在 clk 下降沿后 10 ns 采样、`tPD_meas = 2..6 ns` 出数据 ⇒ 下一个 `clk_i` 上升沿捕获：理论裕量 4..8 ns，扣掉 tSU(~1.2 ns) 与报告里的时钟 skew（DCD − SCD = **1.591 ns**，见 `_synth_work/impl_timing.rpt`）只剩 **1.2–5.2 ns**；两通道 24 根数据线在 256 格点**同拍翻转**时，地弹/串扰最容易把最慢的几位拖过阈值。
- **(b) ADC 内部管线/SSN**：AD9238 是流水线 ADC，粗判级在同拍大翻转时出错会只污染高位 ⇒ 同样能产生"低 8 位对、高 4 位错"。
- (c) AFG 输出 / 接线接触等外部因素**作为触发条件已被排除**（事件位置与幅度无关、逐位结构固定）。
- **(d) 候选缓解（与根因判定无关，尚未实现/未验证）**：坏样本的形状是"**恰好 1 拍、单通道**"，而 `rtl/decim_avg.v` 的 4 点平均只把 ±2300 LSB 稀释到 ±576 LSB（≈17°，仍超过 `angle_raw` 的 1024 LSB 毛刺阈值 ⇒ 仍会触发检测-保持并造成角度冲激）。⇒ 若在 `decim_avg` **之前**对 s/c 各加一级**3 点中值**（或"|Δ| > 阈值则用邻拍均值替换"），单拍坏样本可被**完全剔除**（20 kHz 正弦在 50 MSPS 下相邻拍只差 ~4 LSB，中值不会损伤正常信号）。代价 = 1 拍延迟 + 少量比较器资源；**与 v10 相位修复不冲突，可叠加。**是否实现取决于 §1.57⑥ 的相位判别结果（若根因在接口，则相位修复是主手段，中值是冗余保险）。

⑥ ★ 相位判别实验（v10）

- RTL 改动（`rtl/enc_top.v`）：两路 ODDR 由 `D1=0/D2=1` 改 **`D1=1/D2=0`** ⇒ `ad_clk` 上升沿与 `clk_i` 上升沿**同相** ⇒ ADC 在 t=0 采样、t+2..6 ns 出数据，仍在 t=+20 ns 的 `clk_i` 上升沿捕获 ⇒ **捕获裕量 4..8 ns → 14..18 ns（+10 ns）**；`RTL_BUILD_ID` = **0x000A**，除相位外不改任何逻辑/参数。
- 判据：**改相位后坏码率剧变 ⇒ 根因在 ADC→FPGA 捕获接口（本相位即修复）；率不变 ⇒ 根因在 ADC 内部，相位不是控制量。**
- **基线（v9 相位、ILA 版在场，2026-10-12 13:08–13:13，300 s 窗，`_synth_work/phase_base.txt`）**：`0x05F2` = 0x0009（确认 v9 ILA 版在场）；`0x0505` Δ = **2**、`0x0507` Δ = 2、`0x0509/0x050B` = 0、`0x050D` Δ = 2 ⇒ **0.0067 次/s**（比当天中午 11:37–12:10 的 0.05–0.50 次/s 又低 1–2 个量级）；`0x0540` Δ = 46087（+91 次 16 位回绕）⇒ **f_in ≈ 20.03 kHz**；`0x0405` 读回 0x0000。
- 设计与判据：**A/B/A**（v9 → v10 → v9，各 600 s；脚本 `_synth_work/phase_aba.ps1`、日志 `_synth_work/phase_aba.txt`、出表工具 `tools/phase_aba_report.py` 带泊松检验）。取 A/B/A 的理由：**同一会话内率本身在下降**（11:37→12:10 窗序趋势），单次顺序比较会被时间漂移污染；两个 v9 窗夹住 v10 窗可以界定漂移量。
- 附带发现（图见 `deliverables/figures/ila_glitch.png`）：坏拍经 `rtl/decim_avg.v` 的 4 点平均被**稀释并展宽** —— `s_raw` −1537 → 坏拍 768，`s_dec` 出现约 5 拍平台在 (−1537×3+768)/4 = **−960**（与 4 点均值完全吻合）；而 `ang_glitch` 在坏拍后 **27 拍**（≈540 ns）才置 1 ⇒ 检测-保持支路的响应延迟 ≈27 拍。
- **★ v10 若被采纳，`constr/enc_top.xdc` 必须同步改两处**（否则签核口径与硬件相位不一致）： ① `constr/enc_top.xdc:157-158` 与 `:162-163` 的两个 `create_generated_clock ... -invert` 必须**去掉** **`-invert`** —— ODDR 改 `D1=1/D2=0` 后 ad_clk 引脚沿与 `clk50` 引脚沿**同相**，再声明成反相生成时钟就错了； ② `constr/enc_top.xdc:205-208` 的 `set_input_delay` 要按新相位重算：ad_clk 引脚沿 ≈ **3.762**（Source Clock Delay，见 `_synth_work/impl_timing.rpt`）+ **0**（不再是半周期 10 ns）+ **3.016**（ODDR Tco + OBUF）= **6.78 ns**，加 ADC `tPD` 2..6 ns 与排线 ≈1 ns ⇒ 数据到达 ≈ **9.78 … 13.78 ns**；捕获寄存器时钟沿 ≈ 24.0–24.6 ns ⇒ **真实建立余量 +10.2 … +14.8 ns** ⇒ 诚实约束值应为 `-max 14.000 / -min 9.500`（v9 反相时是 24.000 / 19.500）。 ③ ⇒ 本次 v10 实验构建**故意不动 XDC**（唯一变量 = ODDR 相位），所以它的时序报告仍会报同样的 26 个失败端点与 ≈−3.3 ns —— **那不是 v10 的真实裕量。**采纳 v10 后的签核动作 = 改上面两处 + 重跑实现 + 用新报告核对 26 个失败端点消失。
- **执行方式改为"生产版 A/B/A"（2026-10-12 13:25 起）**：本实验只需要**读计数器**（`0x0505…0x050D`），不需要 ILA ⇒ 用生产版位流即可，可绕开调试核综合。

- ⚠ 原计划的 v10 **ILA 版**构建（`pwsh-79`，13:08:30 起，`tools/ila_debug.tcl` → `_synth_work/ila_build_v10phase.log`）**在 `Starting Logic Optimization Task` → `Phase 1.1.1 Generate And Synthesize Debug Cores` 卡死**：日志与两个 vivado 进程的 CPU 于 **13:10:22/13:10:34 双双冻结**（PID 27072 CPU 恒 86.78 s；OOC 综合子进程 PID 26236 起始 13:10:22、CPU 恒 15.34 s），`.Xil\Vivado-27072-ROG\dc_drv.0\dc\prj_ip.runs\{dbg_hub,u_ila}_synth_1\runme.log` 均停在 `*** Running vivado ... -source dbg_hub.tcl`、零后续输出 ⇒ **判定为调试核 IP 生成子进程挂起，非 RTL 问题**（synth_design + DRC 已在同一次运行中通过：`[Project 1-461] DRC finished with 0 Errors`）。已 kill，改用生产版重建（`tools/impl_check.tcl`，13:25 启动 ⇒ `_synth_work/build_v10prod.log`）。**ILA 版 v10 留待需要逐位复核时再重试。**
- 位流备份（防止重建覆盖）：`_synth_work/enc_top_v9.bit`（= 原 `enc_top.bit`，SHA256 `2F51DCD3…4A43`）与 `_synth_work/enc_top_ila_v9.bit` + `enc_top_ila_v9.ltx`（SHA256 `BF3A37DC…2381`）。
- 脚本 `_synth_work/phase_aba_prod.ps1`（ASCII-only，ParseFile 0 错，需一次性 `danger-full-access` 才能下板）：`A1_v9`（下 `enc_top_v9.bit`）→ `B_v10`（下 `enc_top.bit`）→ `A2_v9`；每段 = 下位流 → 读 `0x05F2` 校验版本（9 / 0x000A）→ 静置 12 s → 清 `0x0405` → `diag_counters --seconds 600` → 读 `0x0405`；全部落 `_synth_work/phase_aba_prod.txt`，出表 `python tools\phase_aba_report.py _synth_work\phase_aba_prod.txt`（含泊松检验）。
- ⚠ 生产版无 ILA ⇒ **本 A/B/A 只能给"率"**；若率有显著变化、需要复核坏码的逐位形态，再重建/下载 ILA 版抓 `ang_glitch`。
- **★ 结果（2026-10-12 17:07–17:37 跑完；日志 `_synth_work/phase_aba_prod.txt`，出表 `python tools\phase_aba_report.py _synth_work\phase_aba_prod.txt`）**：

相位	窗	事件数	率
A1 = v9（下 `enc_top_v9.bit`，`0x05F2` = 0x0009 ✅）	600 s	3	0.0050 次/s
B = v10（下 `enc_top.bit`，`0x05F2` = 0x000A ✅）	600 s	**0**	**< 0.0017 次/s**
A2 = v9（回烧 `enc_top_v9.bit`，`0x05F2` = 0x0009 ✅）	600 s	7	0.0117 次/s

- v9 合计 **10 事件 / 1200 s = 0.0083 次/s**；泊松下尾检验 **$P(K \le 0 \mid \lambda = 0.0083 \times 600 = 5.0) = 6.74\times10^{-3} < 0.01$** ⇒ 按本节预设判据：**改相位后率确实下降 ⇒ 根因落在 ADC→FPGA 捕获接口一侧（候选 (a) 得到支持；候选 (b)「ADC 内部管线」不被支持）。**
- 三窗 `f_in` 一致（`0x0540` 回绕搜索后 ≈ 20.0 kHz；`tools/diag_counters.py` 打印的"次/秒"仍不可靠，只能用 Δ/实测秒数）；`gl_2048` 与 `gl_1024` 逐窗相等（毛刺全落在 11.25°–22.5° 带内），`0x0509/0x050B` = 0。
- ⚠ **效力边界必须说清**：本 A/B/A 只证明"**v10 把率压到 600 s 内 0 个**"，**效应量只有上界**（< 0.0017 次/s）；而且 v9 的两个窗自己就从 3 漂到 7（同会话内 2.3× 漂移）⇒ 这是"方向 + p 值"级的判定，**不是**"率降到几分之一"级的判定。要给出效应量，必须在**热工作点**（率 ≥ 5 次/s）重做同样的 v9/v10 对照（见 ⑦）。
- ⚠ **ILA 版 v10 重建（用户 m03406 要求）今天四次尝试全部卡在同一处**（`Phase 1.1.1 Generate And Synthesize Debug Cores` → `Processing IP xilinx.com:ip:ila:6.2 for cell u_ila_CV`）：13:10（PID 27072/26236）、13:45（24756/22732）、18:00（29404/24632，子进程 CPU 40 s 只涨 0.1 s ⇒ **阻塞**而非空转）三次在 workspace-write 沙箱下；第四次（20:43 起，`_synth_work/ila_build_v10phase_retry3.log`）**提权（`danger-full-access`）后仍在推进**——其 `.Xil\Vivado-28748-ROG\dc_drv.0\dc\prj_ip.runs\u_ila_synth_1\runme.log` 持续增长、已过 Cross Boundary 优化 ⇒ **不是死锁，而是极慢的调试核 IP 综合。**许可证竞争已排除（`D:\Vivado\Xlnx_2026_1.lic` 全为 `permanent uncounted HOSTID=ANY`、无 `SERVER` 行 ⇒ 节点锁定）。⇒ 经验：**Vivado 的调试核 OOC 综合要提权运行，并且要给它十几分钟到半小时**（10-11 那次成功构建也是这个量级）。**★ 第四次（20:43 起，提权）已成功**：`_synth_work/enc_top_ila.bit` = **1,203,448 B @2026-10-12 20:46:48**、SHA256 `EA36E38C731C42206F1AFC486C658B6C0B3F7AAFD659F2730CA4D8CA2DF45375`、`enc_top_ila.ltx` = 71,337 B @20:46:38（v9 ILA 版 1,432,868 B 已备份为 `_synth_work/enc_top_ila_v9.bit`）⇒ 该位流可作为 **v10 的 ILA 复核手段**（在热工作点抓 v9/v10 两窗原始码做对比）。

⑦ ★ 幅度轴的新证据：旋钮是极陡开关（0 → 37.8 次/s）（原始记录 `_synth_work/sweep_rate.csv`、`_synth_work/ladder.csv`；工具 `tools/sweep_rate.py`、`tools/amp_point.py`）

- 背景：① 把 §1.56 的"幅度轴"作废后，"率究竟是不是幅度控制的"一度无解。2026-10-12 17:38–17:45 让操作者**只拧 AFG 幅度旋钮**（频率不动、位流不动、同一会话），同时用 `tools/sweep_rate.py` 每 5 s 读一次 `0x0505/0x0507`：

静止平台时段	率
17:38:44–17:40:10（拧之前）	**0 次/s**
17:40:35–17:41:05	**37.8 次/s**
17:41:30–17:41:55	1.4 次/s
17:42:25–17:42:50	0.8 次/s
17:43:20–17:43:45	9.5 次/s
17:44:50–17:45:15	16.4 次/s

- 拧动瞬间的采样点是**伪像**（每 5 s 上万、最大 62509，与 11:23 那次"输入被扰动"同一签名）⇒ 只有**静止平台**能用作数据。
- ⇒ 在同一位流 / 同一频率 / 同一会话内，**幅度本身确实是控制变量**，而且开关极陡（跨 4 个数量级）。因此 ③ 的表**不能**用来否认"幅度/格点依赖"：那张表跨了三个会话、两种拟合口径，且被"同一会话内率自己漂 2.3×"污染；③ 只否掉了"**率 = 到最近 256 格点距离的单调函数**"这一具体表述，**不等于"幅度无关"。**
- ⚠ 我在本条里踩过一个量纲坑并当场更正：20 s 窗读到 `0x0540` Δ = 7548 时曾当成 377 Hz，实际要按 16 位回绕解（n = 6）⇒ 400764/20 s ≈ **20 kHz**；`tools/amp_point.py` 的 `fin_hz`（带回绕搜索）给出 20490 / 20653 ⇒ **频率从未塌陷。**教训：`0x0540` 只能看回绕修正后的值。
- ⚠ **AFG 显示值不能当幅度轴**：同一个"8.1 Vpp"档，上午实测（遥测半径，原始码口径）是 1616–1654；下午 17:47–17:48 三次 `tools/amp_point.py` 记录 **A_mean = 1582.8 / 1587.3 / 1587.5，率全 0**；操作者"拧完了"并没有落在热区 ⇒ 复现热点只能靠**遥测半径的实时读数**，不能靠旋钮刻度/显示值。
- **待做（下一步的决定性实验）**：请操作者按**最小步长**逐档上调（每档停 ~30 s）并回报档位序列 ⇒ 用平台率定出"档位 → 率"曲线 ⇒ 在**率 ≥ 5 次/s** 的档位**固定不动**，重做 **v9 → v10 → v9**（每窗 30–60 s）⇒ 拿到相位效应的**倍数**（而不只是方向）。判据同 ⑥（幅度绝不能动，否则不再是单变量）。

产物清单（本节）：`tools/arc_check.py`（新增，遥测弧段覆盖检查）、`tools/arc_fit_free.py`（新增，自由频率抗差拟合）、`tools/glitch_bits.py`（新增，逐位归因）、`tools/raw_circle.py`（新增，遥测反解原始码；SCALE=16）、`tools/sweep_rate.py`（新增，每 N 秒记录 `0x0505/0x0507` + 可选遥测半径）、`_synth_work/dump_raw.py`（新增，事件前后原始码导出）、`_synth_work/arc_check.csv`、`_synth_work/phase_base.txt`、`_synth_work/phase_aba_prod.txt`（A/B/A 原始日志）、`_synth_work/sweep_rate.csv`、`_synth_work/ladder.csv`、`_synth_work/ila_build_v10phase{,retry,retry2,retry3}.log`。

1.58 ★★★ v12：遥测帧布局 v2 —— ABZ/UVW 六路电平进遥测通道（2026-10-12，RTL 已改、测试台/工具/夹具全绿；位流已于 10-12 23:59 / 10-13 07:12 重出，见 §1.59）

- **动机（路线 C，用户 m00124："要上位机实时看 ABZ/UVW"）**：原先要 ABZ/UVW 只能上示波器（排针 J5-5..J5-10）或 ILA（单窗 20.5 µs、要 JTAG），遥测通道里**一个电平都没有。**v12 把六路电平 + 方向 + 告警塞进**原 18 字节帧**，链路占用与帧率**一字不变。**
- **帧布局 v2**（帧长仍 18 字节 = 180 bit，`BAUD_DIV=25` ⇒ 90 µs/帧，`SAMPLE_DIV=5000` ⇒ 10 kHz）：

off	旧（v11 及更早）	v12（现行）
0..7	`AA 55 + angle + s_n + c_n`	**不变**
8	`amp_sq[31:24]`	**`io_state`**：bit0..5 = `a b z u v w` 六路输出电平，bit6 = `dir`，bit7 = `limiter_active`
9	`amp_sq[23:16]`	**`0xA2` 帧布局标记**（`FRAME_TAG_IO`）
10	`amp_sq[15:8]`	**`io_flags`**：bit0 `lag_warn`、bit1 `lag_fatal`、bit2 `z_window`、bit3 `uvw_freq_warn`、bit4 `cfg_abzen`、bit5 `cfg_uwven`、bit6 `cor_sat`、bit7 `amp_bad`(= `amp_low|amp_high`)
11	`amp_sq[7:0]`	**`0x00`**（`FRAME_RESV` 保留，参与校验）
12..17	`pos_full[31:0]` + `stat`	**不变**

- **★ `0xA2` 是无歧义的判别位（不是"约定"，是可证明的）**：旧布局 off9 = `amp_sq[23:16]`，而 `rtl/amp_monitor.v:90` 的 `amp32 = (sq_r > SAT32_MAX) ? AMP_SAT : sq_r[31:0]`，`AMP_SAT` 使该字节能取到的最大值 ≤ `0x80` ⇒ **旧位流永不产生 off9 = 0xA2。**上位机 `tools/uart_scope.py` 就按 off9 分派 v1/v2 解析（旧位流会自动退回 v1，只报"无电平列"）。
- **★ 丢掉 `amp_sq` 是无损的（这是敢删的依据）**：同帧已经带着 `s_n = s_cor`、`c_n = c_cor`，`amp_sq = sat32(s_cor² + c_cor²)`；`int16` 满幅时 $s^2 + c^2 \le 2^{31}$ **够不着 `SAT32_MAX` 饱和**（$16000^2 + 16000^2 = 5.12\times10^8 \ll 2^{31}$）⇒ PC 端 `(s*s + c*c) & 0xFFFFFFFF` **逐位**等于 RTL 的 `amp_sq`。旧列在工具里**照旧输出**（只是改由 `s_n/c_n` 重算）。
- **★ 为什么坚持 18 字节而不是加到 20**：20 字节 @2 Mbaud = 100 µs，**正好等于** `SAMPLE_DIV=5000` 的采样周期，再加上组帧 FSM 每字节的强制间隔拍（§1.10）⇒ 链路过载、帧率必掉。保 18 字节则 `tools/uart_spi.py`（`TELEM_FRAME_LEN = 18`，只用来跳过遥测帧找 `0x5A` 应答）也无需改。
- **改动面（全部已落盘）**：`rtl/uart_scope.v`（端口 `amp_sq[31:0]` → `io_state[7:0]`/`io_flags[7:0]`，锁存、`frame_byte` 8..11、校验和 `:219-226` 把 `io_state ^ 0xA2 ^ io_flags ^ 0x00` 一并异或进去）、`rtl/enc_top.v`（`:1150-1163` 例化按位拼接，`RTL_BUILD_ID` 由 `16'h000B` 升为 **`16'h000C`**，见 §1.49/`docs/01` 表）、4 个测试台（`tb_uart_scope` / `tb_uart_baud` / `tb_uart_period` / `tb_uart_spi_bridge` 端口与期望字节）、`tools/uart_scope.py`（v1/v2 双布局解析 + 新报告段 + CSV 新列 + 3×2 图）、`tools/check_uart_frame_fixture.py`（v2 化，30 checks）、`sim/fixtures/uart_frame_rtl.hex`（重生成）。
- **新夹具帧**（激励 `angle=0x1234 s=0x0F0E c=0xF1F2 io=0x6D tag=0xA2 flag=0xA6 pos=0x01020304 stat=0x5A`）：`aa 55 12 34 0f 0e f1 f2 6d a2 a6 00 01 02 03 04 5a ec`（末字节 `ec`）。校验定义 = `0xAA ^ 0x55 ^ 全部 payload 字节`（= `rtl/uart_scope.v:219-226`）。
- ⚠ **顺带修掉一个守卫自身的错**：`sim/tb_uart_spi_bridge.v` 的期望遥测校验原本写的是"纯 payload 异或" `0xA7`，与 RTL（含帧头 `AA^55`）**不一致** ⇒ v12 重算后暴露，已改为 `0xEC`。也就是说：**旧夹具/旧测试台的校验期望值本身是错的**，这与 §1.15 记的"校验和约定不一致"是同一族问题。
- **上位机新能力（`tools/uart_scope.py`）**：报告新增「ABZ/UVW 电平快照」段（占空比/边沿数、UVW 状态分布并对 `000`/`111` 直接告警、Z 脉宽提醒），CSV 新增 `layout/io_state/io_flags/a..w/dir/limiter` 列，PNG 改 3×2：**第 3 行是等效采样图**（按电角度 64 分箱重画）。★ 这一行是本次的关键设计：`A/B` 边沿率 = `4×inter/div×f_in`（默认 inter=256@1 kHz ⇒ 1.024 MHz，20 kHz ⇒ 20.5 MHz 被 `filt_valid` 限频）**远高于 10 kHz 帧率**，按时间轴看必然混叠 —— 只有按角度分箱（每箱 ≥3 帧）才能真实还原波形形状，与速度无关。
- **★ 重综合已做（2026-10-12 23:59 / 2026-10-13 07:12，见 §1.59）**：生产位流 `_synth_work/enc_top.bit` **1,042,208 B @23:59:27**、ILA 调试版 **1,428,064 B @07:12:11**，两版时序均 `All user specified timing constraints are met.`。剩下的**边界只有一条**：**板上是否已换版以读回为准** —— `0x05F2` = **`0x000C`** ⇒ 板上已是 v12（遥测里会出现 `io_state`），`0x000B` ⇒ 仍是 v11（遥测 off8..11 还是 `amp_sq`，帧里没有电平）。上板步骤与判据见 §1.59 末条、`docs/07` 2.7b。
- **本轮验证状态（全部已跑完）**：`tools/lint_all.ps1` = **ERROR 0 / WARNING 0**；全量回归 **`PASS 21 / 21`**（2026-10-12 23:52:26，`tb_enc_top` 488 s 最长）⇒ `tb_uart_scope` 由 94 → **96 checks**、`tb_uart_spi_bridge` 仍 **44 checks**、`tb_uart_baud`/`tb_uart_period` 不变；夹具守卫 `tools/check_uart_frame_fixture.py` = **30 checks PASS**（打印行「payload XOR(=off2..16) = 0x13；含帧头常量应为 0xEC；实拍末字节 = 0xEC」）；`tools/uart_scope.py --selftest` = **PASS**（含"破坏一字节后 XOR 校验成功捕获"、"边界值 `0/65535/-32768/io=0xFF,0xFF/0xFFFFFFFF/-1` 回读正确"、"v1 旧布局仍可解析；off9 判别无歧义"三条 v2 相关用例）。

产物清单（本节）：`rtl/uart_scope.v`、`rtl/enc_top.v`、`sim/tb_uart_scope.v`、`sim/tb_uart_baud.v`、`sim/tb_uart_period.v`、`sim/tb_uart_spi_bridge.v`、`sim/fixtures/uart_frame_rtl.hex`、`tools/uart_scope.py`、`tools/check_uart_frame_fixture.py`；文档同步：`docs/06` §4.2/§5、`docs/07` 2.4/2.7b、`docs/01` `RTL_BUILD_ID` 表、`README.md`、`deliverables/`。

1.59 ★★★★ v12 位流重出（生产版 + ILA 版）—— 遥测帧布局 v2 走上板通道（2026-10-12 23:59 / 2026-10-13 07:12）

- **触发**：用户指令 **"重出位流"**（路线 C 的直接后续，接 §1.58）。命令 `tools/rerun_impl.ps1`（内含 `tools/impl_check.tcl` 生产版与 `tools/ila_debug.tcl` ILA 版），**两版全部产出**，源树 `RTL_BUILD_ID` = **`16'h000C`**。
- **生产版（无 ILA）**：`_synth_work/enc_top.bit` = **1,042,208 B @2026-10-12 23:59:27**、SHA256 **`D93DEBF92034FF2329117CDE59EB83E0D95620306E7A944C6B0402E8C3AE7725`**。
- `_synth_work/impl_timing.rpt` @**23:59:16**：结论行 **`All user specified timing constraints are met.`**；`sys_clk` **WNS +0.959 ns / TNS 0.000 / 0 个失败端点（共 11,275）**、WHS **+0.075**（0 失败 / 11,275）、WPWS **9.500**（0 失败 / 4,080）；最差路径 `u_regs/r_ablimit_reg[2]/C` → `u_abz/r_hold_reg[0]/R`（数据路径 18.615 ns = logic 7.797 + route 10.818、28 逻辑级、Requirement 20.000 ns）。
- `_synth_work/impl_util.rpt` @**23:59:14**：Slice LUT **6298**/20800 = **30.28%**、Slice Registers **4074**/41600 = **9.79%**、Block RAM **0**/50、DSP **16**/90 = 17.78%、Bonded IOB **50**/250 = 20.00%、BUFGCTRL **1**/32 = 3.13%。
- `_synth_work/impl_drc.rpt` @**23:59:18** = **0 Errors**（`DRC finished with 0 Errors`）；`impl_route.rpt` / `impl_clkint.rpt` @23:59:18。
- 压缩算术自检：`Bitstream compression saved 9200096 bits` ⇒ 17,537,696 − 9,200,096 = 8,337,600 bit = 1,042,200 B + 8 B 头 = **1,042,208 B** ✓（与文件实际大小逐字节吻合）。
- 与 v11 比：WNS **+0.821 → +0.959**、端点 **11,310 → 11,275**、LUT **6369 → 6298**、FF **4088 → 4074**、位流 **1,086,600 → 1,042,208 B**。相对 v11，RTL 只多了 §1.58 的遥测端口/组帧改动与 `RTL_BUILD_ID` 一行（`16'h000B` → `16'h000C`），**角度/ABZ/UVW/BiSS 行为不变。**
- **ILA 调试版**：`_synth_work/enc_top_ila.bit` = **1,428,064 B @2026-10-13 07:12:11**、SHA256 **`5CC779EE6D00FFA60AD371F9A87F82A35E74C8B6E477F45323B475A299BAEC52`**；`_synth_work/enc_top_ila.ltx` = **71,337 B @07:12:00**。

- `_synth_work/impl_ila_timing.rpt` @**07:11:57**：判据行同为 **`All user specified timing constraints are met.`**；**WNS +0.332 / TNS 0.000 / 0 个失败端点（共 20,165）**、WHS **+0.050**（0 / 20,149）、WPWS **8.870**（0 / 9,858）；最宽松的 slack 27.235 ns 落在 `dbg_hub/.../bscan_switch/state_reg[0]/C`（TCK 域）。
- `_synth_work/impl_ila_util.rpt` @**07:11:57**：LUT **9234** = **44.39%**、FF **8937** = **21.48%**、Block RAM **7.5** = **15.00%**、DSP **17** = 18.89%、Bonded IOB 50、BUFGCTRL **2** = 6.25%；探针 `_synth_work/ila_probe_list.txt` = **36 条 / 269 bit**（与 v11 相同）；`impl_ila_drc.rpt` @07:12:00 = **0 Errors**；压缩算术 11,424,448 bit = 1,428,056 B + 8 = **1,428,064 B** ✓。
- 本轮 ILA 版在**一次性** **`danger-full-access`** 下跑（07:07 起，约 5 分钟到 step 10 —— IP 已缓存，远快于历史 2.5 h 墙钟）；`deliverables/README.md:61` 记的"调试核 OOC 在受限沙箱下阻塞"这一次没有复现。
- **v11 归档（不要删）**：`_synth_work/enc_top_v11prod.bit`（1,086,600 B @2026-10-12 21:15:37、SHA256 `FA1308647D776F78E2497DF44E92F06942749567800338 0A8D78D40F5B7C2594`）、`_synth_work/enc_top_ila_v11.bit`（1,458,744 B @21:29:54、SHA256 `5E049FD3B3EAB51C79D62BEA35709D6CB922820064B316FE1C644DCB97B3210B`）+ `enc_top_ila_v11.ltx`（71,337 B）。
- ⚠ **"Vivado 写完位流不退出"这次两轮都复现（务必记住）**：两轮都在日志出现 `INFO: [Vivado 12-1842] Bitgen Completed Successfully.` 之后**再也没有退出**，也没有 tcl 末尾应有的 `BITSTREAM_WRITTEN …` / `IMPL_CHECK_DONE`（生产版）与 `ILA_BITSTREAM_WRITTEN …` / `ILA_CHECK_DONE`（ILA 版）行；生产版进程实测 **CPU 稳定约 1 核、WorkingSet 14.5 GB / 私有内存 16 GB**，挂满 50 分钟后人工终止；ILA 版 07:12 产出位流后一直挂到 11:59（约 4.7 h）才 `job_kill`（job `pwsh-473` / `pwsh-495`，事后确认无残留 `vivado` 进程）。 ⇒ **判"成功"必须靠产物核对（四项，已逐条做）**：① 压缩位数算术 = 文件字节数；② `.bit` 的 `d` 段日期时间与日志一致；③ `impl_drc.rpt` = 0 Errors；④ `impl_timing.rpt` / `impl_util.rpt` 已落盘。**不要因为日志缺 `*_DONE` 就重跑**（§1.57 ⑧ D 段早记过同一现象；v11 轮也曾出现，见 §2 顶部历史块）。
- **★ 上板验证已完成（2026-10-13 12:29–12:45，串口 COM3，用户接板）**：用 `tools\ila_capture.tcl` 的 **`ILA_MODE=restore`** 把 `_synth_work/enc_top.bit` 烧进 `xc7a35t`（cable = `Digilent/210512180081`，日志 `_synth_work/dl_v12_restore.log`，`End of startup status: HIGH` / `ILACAP DONE` / exit 0），随后串口逐条读回：
- **`0x05F2 = 0x000C` ⇒ 板上已是 v12**（`0x05F3 = 0x0000`）；同批复核 `0x0302 = 0x001A`、`0x0500 = 0x0A16`(22)、`0x0501 = 0x000A`(10)、`0x0109 = 0x0040`（复位默认，仅 `abzen`）。
- 默认配置 5 s 抓帧：**全部是 v2 帧**（`v2 49815 + v1 2`），A/B 各 **50.0%** 占空、各 **49,632** 次跳变，`dir` = 1、`limiter` = 0；`laglim`/`lagfatl` 全窗为 1 ⇒ 正是 `inter` = 256 撞 12.5 MHz 边沿率上限的已知现象（§1.33 ⑥）。
- 写 `0x0101 = 0x0020`（inter = 32 / div = 1，顺带把 `0x0102` 写成 0）、`0x0109 = 0x00C0`（`abzen` + `uwven`）后 5 s 抓帧：**A 50.7% / B 49.3% / Z 0.4%**，**UVW 六种合法状态全覆盖**（`001:7439 010:9257 011:10634 100:7480 101:7427 110:7533`），**无 `000`/`111` 非法态**，状态位"全部正常（无告警）"。
- **30 s 抓帧（298,747 帧 v2）**：A 49.8% / B 49.8% / Z 0.1%，UVW 六态均衡，**等效采样填充率 64/64 箱（每箱 ≥3 帧）** ⇒ `A/B` 正交、`UVW` 六步、`Z` 每电周期一次在等效采样图里全部可见（PNG 第 3 行）。
- **归档**：`deliverables/figures/telemetry_abz_uvw.png` + `deliverables/data/telemetry_abz_uvw.csv`（先入 5 s 版；**2026-10-13 晚改用 30 s 版覆盖，见下条**）。
- ✅ 边角已修：30 s 抓帧里有 **1 帧被判为 v1**（18 字节帧只有 1 字节 XOR 校验，挡不住个别多比特误码，`298747:1` ⇒ 误码误判而非旧位流）。当天即把 `tools/uart_scope.py:307-321` 的判据改成**双条件**：v1 帧数 ≥ `max(3, 0.1%)` 才打"★ 混流！检查位流是否重烧"，否则降级为普通行"…%d 帧被判 v1（占比 … %，XOR 校验漏网的多比特误码，非旧位流）"；两条分支都已用构造流验证（10/300 ⇒ 告警、1/3000 ⇒ 不告警），`--selftest` 仍 **TEST PASSED。**
- **★ 演示脚本 `tools/demo_abz_uvw.py`（2026-10-13 新增，一条命令做完 P2-7）**：`python tools\demo_abz_uvw.py --port COM3 --seconds 30 --archive` 一次跑完 **① 版本门禁**（读 `0x05F2`，须 `0x000C`；读到 `0x000B` 直接打印 restore 烧录命令并退出）**② 寄存器自配**（`0x0101 = 0x0020`、`0x0109 = 0x00C0`，写后回读确认，已符合则跳过）**③ 抓帧 + 每秒滚动统计**（`A 52.0% (53 边沿) B 46.8% (40 边沿) UVW=006 angle 32.5°`）**④ 全量解析 + 逐项判定**（占空 30–70%、UVW 六态全覆盖、等效采样 ≥32/64 箱、无非法态/无同步丢失，Z **只记录不判失败**——Z 脉宽远窄于 100 µs 采样周期）**⑤ 出 CSV/PNG**，退出码 0 = `DEMO PASS`。
- **实机结果（COM3、30 s、`--archive`）**：`DEMO PASS`（**9/9 项** ✅）；**298,749 帧、0 帧被判 v1**、1114 处 XOR 校验失败、20,241 个同步丢弃字节；A **50.0%**/1405 边沿、B **49.9%**/1340、Z **0.1%**/488、`dir` 恒 1、`limiter` 恒 0；UVW `001:50853 010:49248 011:49188 100:49168 101:51021 110:49271`（六态均衡、无非法）；**等效采样 64/64 箱**；`状态位 全部正常`。
- **归档（覆盖旧版）**：`deliverables/figures/telemetry_abz_uvw.png` = **120,686 B @2026-10-13**、SHA256 **`CF8D3B1985B0670319E0E9BEE268E3048C8556BD070019A570CEB9AD49EA9C95`**；`deliverables/data/telemetry_abz_uvw.csv` = **25,237,925 B（30 s 原始帧，20 列）**、SHA256 **`1EEC84B077AE8EAAE671E1D85528AB1478C9DD62D21DCE0424696A77D40E8F19`**（5 s 版 4,123,606 B 被覆盖）；CSV 行已写入 `deliverables/data/SHA256SUMS.txt`（清单 47 → **48 行**，`48/48` 实测通过；`.csv` 无行尾规则、逐字节入库故哈希克隆后稳定），`deliverables/data/README.md` §2.5 记录列含义，清单不收的 9 个 v12 位流/报告副本理由不变。
- 该脚本 `--selftest` 覆盖 4 例（正常合成流 PASS / `uwven` 关 FAIL / 纯 v1 旧布局 FAIL / `LiveScan` 非对齐分块与全量解析帧数一致）。

产物清单（本节）：`_synth_work/enc_top.bit`、`_synth_work/enc_top_ila.bit`、`_synth_work/enc_top_ila.ltx`、`_synth_work/impl_timing.rpt`、`impl_util.rpt`、`impl_drc.rpt`、`impl_route.rpt`、`impl_clkint.rpt`、`impl_ila_timing.rpt`、`impl_ila_util.rpt`、`impl_ila_drc.rpt`、`ila_probe_list.txt`；归档 `_synth_work/enc_top_v11prod.bit` / `enc_top_ila_v11.bit`(+ `.ltx`)。

1.60 ★★★★ 上位机（Windows GUI）S1 落地：串口层 + 连接界面，真机跑通（2026-10-13 13:00–13:20）

- **触发**：需求沟通完成、`docs/10_host_requirements.md` v1.0 冻结后，用户批准 **"可以，开始做 S1（推荐）"**。S1 = 工程骨架 + 串口层 + M1 连接界面。
- **新增 `host/`（全新目录，不改动 `tools/` 下任何既有脚本，它们仍被文档与测试引用）**：
文件	作用
`host/link.py`	通信层核心：`class SerialWorker(threading.Thread)` 独占串口，遥测帧（18 B，`AA 55`）与寄存器应答（6 B，`5A`，XOR=0）**无损分流**；等应答时路过的遥测帧照样送进解析器
`host/ports.py`	枚举串口、按 `USB VID:PID=1A86:7523` / `CH340` 认目标板、格式化 `COM3 — USB-SERIAL CH340`
`host/theme.py`	淡蓝主题色板 `C` + 全局 QSS `app_qss()`（需求 v1.0 冻结配色）
`host/main_window.py`	三段式主窗口（连接条 / 设备信息·通信统计·实时数据卡片 / 日志），`QTimer` 100 ms 刷新
`host/app.py`	入口；`--smoke PNG [--port COM3]` 可无人值守建窗+自动连接+截图退出
`host/selftest_link.py`	无界面自检 4 步：列端口 → 读身份证 → 收 N 秒帧 → 写回测试
`host/README.md`、`host/run_host.bat`	使用说明与双击启动脚本（`chcp 65001` 防中文乱码）
- **复用而非重写协议**：`link.py` 直接 `import uart_scope / uart_spi`（`tools/` 已加入 `sys.path`），帧校验/解码与命令编码沿用已验证实现。**★ 明确不用 `tools/uart_spi.py` 的 `class Link`**：其 `_once()`(:271) 每次操作前 `ser.reset_input_buffer()`，会把正在流入的 10 kHz 遥测帧全部丢掉（CLI 适用、界面致命）。
- **真机自检通过（COM3，v12 位流）**：`python host\selftest_link.py --port COM3 --seconds 5` ⇒ COM3 = `USB-SERIAL CH340 (USB VID:PID=1A86:7523)`；`0x05F0 -> 5A 00 F0 10 0C B6`（DEVICE_ID `0x0C`、REV `0x10`）、`0x05F2 -> 5A 00 F2 00 0C A4`（`0x000C` = v12）；**应答第 2 字节地址回显实测存在**（`F0`/`F1`/`F2`）⇒ `VERIFY_ADDR_ECHO=True` 可安全启用；写通路原值写回 `0x0101`/`0x0102` 回读一致；5.04 s 收 49,949 帧 = **9,920 帧/秒、179,954 字节/秒**（板上 10 kHz × 18 B ≈ 180 kB/s = 2 Mbaud 链路 90% 占用）。

- **界面截图验证（`--smoke`）**：连接态显示 设备型号 `0x0C` ✓ / 固件 v12（帧布局 v2）/ 帧率 9,799 帧/秒 / 数据率 178 kB/s / 电角度 251.043° / 位置 2,798,109 / 幅值 r 25,956 / 电平 `A0 B1 Z0 U1 V0 W1` / 方向 正转。
- **★ 修掉的两个 bug（都会误伤后续里程碑，记牢）**： 1. **毫秒当秒比**：`_flush_batch` 里 `if not force and (now - self._last_emit) < self.batch_ms: return`，而 `time.monotonic()` 差值是**秒** ⇒ `0.098 < 50.0` 恒真，帧永远发不出去（症状：界面只收到 "open"，`worker.batch` 一路涨到 3 万条）。修法：加 `self.batch_s = max(0.001, batch_ms/1000.0)` 并用它比较。 2. **速率虚高 10.7% 的测量陷阱**：用「累计帧数 ÷ 本窗口时长」算帧率，会把**连接阶段**（读身份证 ~0.5 s）缓冲的帧算进来，实测虚高到 **10,947 帧/秒**（> 板上 10 kHz，物理不可能）。修法：取窗口起点的计数基准做增量。**任何“帧率/字节率”显示都必须用增量。**
- **★★★★ ~0.38% XOR 校验错已归因 = 链路固有，不是上位机缺陷（对照实验）**：同一条链路、同样 90% 占用，`python tools\uart_scope.py --port COM3 --seconds 6` ⇒ 输入 **1,079,776 B**、有效帧 **59,756**、**XOR 校验失败 212**、同步丢弃 **3,940 B**、**179,963 B/s**；上位机通信层 5.04 s ⇒ 49,949 帧、**校验错 192**、重同步 **3,606 B**、**179,954 B/s**。两者数值一致 ⇒ CH340 + Windows 驱动在 90% 占用下的偶发丢字节。坏帧错位分布 **单比特 19 / 多比特 193（212 中）** ⇒ 多为丢/插字节而非位翻转。**结论：不重传、不影响显示**（丢一帧不可见），界面按“已知链路特性”提示而非报故障。
- **产物清单（本节）**：`host/link.py`、`host/ports.py`、`host/theme.py`、`host/main_window.py`、`host/app.py`、`host/selftest_link.py`、`host/README.md`、`host/run_host.bat`。

1.61 ★★★★★ 上位机 S2 落地：转速与位置（M3）+ 倍频面板（M2），并定案三个协议/量纲陷阱（2026-10-13 13:30–14:10）

- **触发**：用户 **"现在开始 S2"**（m00285）。S2 = 转速与位置卡（M3）+ 倍频倍率面板（M2），验收 = 对真机改 `inter` 回读生效、转速与已知输入频率一致。
- **新增 `host/` 文件**：`ui_kit.py`（共享界面零件，避免面板↔主窗口循环 import）、`encoder.py`（转速/位置估算，**纯逻辑无 Qt/串口**）、`regs.py`（寄存器位域编解码，纯逻辑）、`panel_speed.py`（卡片⑤ 转速与位置）、`panel_interp.py`（卡片⑥ 倍频倍率 + 高级参数折叠区）、`selftest_s2.py`（S2 无界面验收）；`main_window.py` 重写（`TaskRunner` 带 tag、`QScrollArea` 容纳 9 张卡、把每 tick 的帧喂给估算器）；`app.py` 增 `--height`（截长图）。
- **真机验收通过**：`python host\selftest_s2.py --port COM3 --seconds 6` ⇒ **exit=0 PASS**：收 60,099 帧 / 6.033 s；输入频率**独立路径 20,158.94 Hz**、**主口径 20,084.35 Hz**（`pos_full` 增量 ÷ 帧数时钟）、墙上时间口径 19,911.04 Hz、只看 16 位角度 **−0.1018 Hz**；两条独立路径差 74.6 Hz（容差 403 Hz）；倍频写回读 `inter` 64 → 256 → 2 三次全部**同帧回读一致**，结束恢复 `0x0101=0x0020` / `0x0102=0x0000`；旁证 AB 边沿率 = 4×32×20084 = **2.571 MHz**（与工程早先实测 AB 满速 2.559 MHz 同量级）。
- **★★★★★ 陷阱①：输入信号真的是 ~20 kHz，16 位 `angle` 在 10 kHz 采样下已混叠。** 早期上位机算出 **0.095 Hz**（比真值低 5 个数量级）就是因为只用了 `angle`。决定性证据（连收 15,352 帧 / 1.532 s）：`cycle_cnt` 每帧步长直方图 `(2, 14805) / (-62, 481) / (6, 56) / (4, 6)`，只有 32 个不同值且**全是偶数**，每 64 回绕一次；`angle` 每帧只抖 ±6 LSB、净漂移 **−0.10 Hz**（就是混叠残差）；旁证 inter=32 时 AB 边沿率 4×32×20000 = **2.56 MHz**，与早先记录一致。⇒ **修正：`cycle_cnt` 只有 6 位（POLE_BITS=6），`pos_full = {cycle_cnt[5:0], angle}` 是 22 位单圈位置、每 64 个电周期回绕，解回绕必须按模 2^22**（`host/encoder.py` 的 `unwrap_pos()`；`MAX_STEP=2^21` ⇒ 量程 ±320 kHz。★ 同时否定了"用寄存器 `0x0536-39` 拼 32 位位置做频率交叉验证"这条路：四个独立命令 + 每 64 周期回绕，20 kHz 时读出来的值必然撕裂（曾算出 −2.4 Hz / −20 Hz 全是垃圾），只在 <10 Hz 时才可比。
- **★★★★ 陷阱②：计时基准要用板子的帧时钟，不是上位机墙上时间。** 板子每 100 µs 出一帧 ⇒ `收到的帧 + 校验失败丢掉的帧` × 100 µs = 板上经过时间。USB（CH340）批量到达会抖 ~8 ms，1 s 窗就能让墙上时间口径差 **1.5%**（实测同一时刻：帧数时钟 20,150 Hz vs 墙上时间 19,860 Hz）。`SpeedEstimator.feed(..., n_bad=)` 收 worker 的 `crc_err` 增量做丢帧补偿，`snapshot()` 同时给 `f_in`（主口径，帧时钟）、`f_wall`（对照）、`f_ang`（混叠对照）。离线自检补了用例 8：每收 1 帧丢 1 帧（`n_bad=1`）时仍必须读出 20 kHz。
- **★★★★★ 陷阱③：写命令按"发一帧"是永远拿不到应答的，重同步还必须认应答头。** TW28 重叠响应使命令 N 的应答要到第 N+1 帧才移出 ⇒ 写帧自己那一帧的应答根本不会出现（旧实现故报"写失败（超时）"，而寄存器其实已写好）。修法两处：(a) `write_reg()` 固定发 `[写, 读, 读, 读]`，按**地址回显**收下应答，第 1 个是写结果、第 2 个是**同帧回读值**（返回值即回读值，正好满足需求“应用后自动回读比对”）；(b) `SerialWorker._find_sync()`：重同步时**同时**认遥测帧头 `AA 55` 与合法应答头 `5A`（原来只 `buf.find(b"\xaa\x55")`，会把夹在中间的 6 字节应答一起跳过 ⇒ 症状就是"写生效了却报无应答"）。修完真机连写 4 条命令全部同帧回读成功。
- **★★★★ 陷阱④：读应答是"两个字节"，高字节属于 `addr+1`。** 现象：写 `0x0102=0x0001` 后反复读 `0x0101` 稳定返回 `5A 00 01 01 00 5A`（值 `0x0100`），而 `r_inter0` 明明是 `0x00`。定案：应答 = `{寄存器[addr+1], 寄存器[addr]}`，一律只取低字节。旁证一串：`0x0100 → 0x2004` = {`r_inter0` =0x20, `r_uvw_cfg` =0x04}；`0x0106 → 0x0A00` = {`r_zero0` =0x0A, `r_ablimit` =0x00}；`0x0108 → 0xC004` = {`r_output` =0xC0, `r_zero1` =0x04}；`0x05F0 → 0x100C` = {REV=0x10, DEVICE_ID=0x0C}。★ 因此**此前把 `0x0108` 读回的 `0xC004` 解释成"仅存储高位"是误读**——`0x0108` 自己只有低字节 `0x04`（zwidth=4），`0xC0` 是 `0x0109`（OUTPUT，ABZ+UVW 使能）的值。`host/regs.py` 的解码全部按低字节写，故不受影响。
- **界面截图（`host/preview_s2.png`，真机连接态）**：输入频率 **20084.314 Hz**、机械转速 **301264.7 RPM**（极对数 4）、累计圈数 42814.283 圈、电周期数 171257.130、方向 正转、提示“单看 16 位角度已混叠（只有 −0.10 Hz），本值由多周期位置增量解出；每帧平均走 2.0 个电周期”；倍频卡 等效倍率 ×32.000、AB 边沿率 2,570,792/秒（单通道 642.7 kHz）、板上现值 `0x0101=0x0020 0x0102=0x0000 ⇒ inter=32 div=1`；通信统计 10,072 帧/秒、183 kB/s、校验错 297 (0.36%)、丢批/丢帧 0/0。
- **离线自检**：`python host\encoder.py --selftest` 8 条全 PASS（含 20 kHz 混叠用例、±320 kHz 量程告警、接收慢告警、丢帧补偿）；`python host\regs.py --selftest` 全 PASS（`inter` 2…256 × `div` 1…8 **2040 组编解码往返一致**、位域写不改同寄存器其它位）。
- **产物清单（本节）**：`host/ui_kit.py`、`host/encoder.py`、`host/regs.py`、`host/panel_speed.py`、`host/panel_interp.py`、`host/selftest_s2.py`、`host/preview_s2.png`，以及 `host/main_window.py` / `host/app.py` / `host/link.py` / `host/README.md` 的更新。★ 期间的一次性诊断脚本（`host/_diag_freq.py`、`host/_diag_rd.py` 等）已删除，不留临时文件。

1.62 ★★★★★ 上位机 S3 落地：输入波形 + 告警（M4/M7），并把频率口径修成「整数+小数」合成（2026-10-13 14:20–15:40）

- **触发**：用户 **"开始进行计划S3"**（m00748）。S3 = 输入信号波形卡（M4：`s_n` / `c_n` 时域 + 李萨如圆 + 幅值/OTR）+ 告警卡（M7：中文 + 颜色 + 清锁存按钮），验收 = 真机能看出波形与幅值、告警能翻译、锁存能清。
- **新增 `host/` 文件**：`signal.py`（幅值与告警逻辑，**纯逻辑无 Qt/串口**：环形缓冲统计 `r` 均值/范围/圆度、`r` 门限判定 `0.60…1.20` 满量程、14 行告警中文翻译）、`panel_alarm.py`（卡片⑦ 实时位 + 锁存位 + “清除锁存告警”按钮、“读状态”按钮）、`panel_wave.py`（卡片⑧ matplotlib 嵌入 Qt：1500 帧时域快照 + 李萨如圆、自适应重绘间隔 100…500 ms、“暂停刷新”）、`selftest_s3.py`（S3 无界面验收）；`ui_kit.field()` 增 `row/col` 以支持网格排布；`main_window.py` 接上两卡与 `task_read_status()` / `task_clear_latch()`。
- **真机验收通过**：`python host\selftest_s3.py --port COM3 --seconds 5 --clear` ⇒ **exit=0 PASS**：`signal.py` 离线 20 项通过（门限下限 0.60 FS `r_low=19660.8`、上限 1.20 FS `r_high=39321.6`）；真机 40,226 帧 / 4.022 s ⇒ **r 均值 25,863.7（min 25,427.9 / max 27,892.8）、圆度偏差 9.531%、削顶 0**；`0x0520` 门限与复位值一致；`0x0530 STAT_AMP_SQ` 换算的 r 与上位机 `sqrt(s²+c²)` 一致（偏差 <2%）；告警翻译 14 行全部中文且实时位/锁存位分别判定；`--clear` 写 `0x0405=0x00FF` 实测可清。
- **强制告警实测（`--force-fault`）**：把 `0x0106 ABLIMIT` 调到 200 制造“限频” ⇒ 遥测 `stat` bit2/bit3 置起、`0x0404 STAT_VAL=0x08`、**`0x0405 STAT_LATCH=0x08` 锁存**，恢复 `0x0106` 后实时位归 0、锁存位仍在，再写 `0x0405=0x00FF` ⇒ 两位都清（**先消除原因再清**，板子留在干净状态）。定案依据：`rtl/reg_file.v:411-

`416` 的 `else` 分支每一拍都用实时状态重新锁存 ⇒ W1C 清位后故障若仍存在会立刻再锁上。

- **★★★★★ 修正：输入频率口径 =「整数电周期数 + 小数残差」合成（此前偏高 0.8%）。** S3 复核界面读数时发现 S2 留下的 `20 084~20 158 Hz` 仍偏高：板子每 100 µs 恰好一帧（`rtl/uart_scope.v:151 SAMPLE_DIV=5000`，:186 `div_cnt == SAMPLE_DIV-1` ⇒ `latch`；:303 `if (latch && !hold)`），但 CH340 链路**实测丢 ~0.75% 的帧**——0.30~0.38% 被 XOR 校验抓到（`crc_err`）+ **~0.37% 被重同步吞掉**（`resync_bytes` 3,940→4,470 字节 / 18 ≈ 220~248 帧，以前没算进“帧时隙”）。丢帧让“位置增量 ÷ 收到的帧数”这个商整体抬高同样比例。四种口径同场实测（`host\_diag_alias.py`，临时脚本已删）：**A 墙上时间 23,975 Hz（被队列积压污染，不可用）、B 帧数时钟 20,155.9 Hz、C 帧数+重同步折算 20,090.6 Hz、D 粗+精 19,999.9 Hz**（整数 2 + 小数 −0.000010 个电周期/帧 × 10 kHz）。定案：`f_in = (round(每帧位置增量) + 每帧角度残差模 1) × 10000 Hz` —— 整数部分抗丢帧、小数部分（实测 −0.66 LSB = **−0.10 Hz**）不受丢帧影响；`f_clock`（含 `lost_frames = crc_err + resync_bytes//18` 补偿）与 `f_wall` 降为交叉校验，两者与主口径差 >2% 时自动改用帧数时钟口径并告警。**⇒ 板上真实输入就是 20.000 kHz**（−0.1 Hz = 50 MHz 晶振约 ±5 ppm）。`link.counters()` 新增 `lost_frames` 字段；`selftest_s2.py` 第 ③ 步判据改成“主口径 vs 帧数时钟口径（1% 容差）”，墙上时间口径明确标注“不作判据”。
- **界面截图（`host/preview_s3.png`，真机连接态，1752×2451）**：输入频率 **19999.897 Hz**、机械转速 **299998.5 RPM**（极对数 4）、累计圈数 38,168.310、方向 正转，提示“单看 16 位角度只有 −0.10 Hz（高速混叠），本值由「整数电周期数 + 角度残差」合成；每帧平均走 2.02 个电周期；链路丢帧约 0.87%，已按帧时隙折算”；倍频卡 等效倍率 ×32.000、AB 边沿率 2,559,987/秒（单通道 640.0 kHz）；⑦告警卡 14 行中文（灰=正常/待测、蓝=提示、橙=警告、红=故障）；⑧波形卡 r 均值 25510、范围 25436…25568、圆度偏差 0.52%、板上 r 26105 / 上位机 25494、刷新 1 ms（10 次/秒），**时域曲线看着是两条缓变直线**——这是 20 kHz 输入被 10 kHz 帧率采样的必然混叠，卡片上已写明“左侧时间轴波形已混叠，只看右侧李萨如圆的半径/圆度”。
- **产物清单（本节）**：`host/signal.py`、`host/panel_alarm.py`、`host/panel_wave.py`、`host/selftest_s3.py`、`host/preview_s3.png`，以及 `host/main_window.py` / `host/ui_kit.py` / `host/encoder.py` / `host/link.py` / `host/selftest_s2.py` / `host/panel_speed.py` / `host/README.md` 的更新。★ 期间的一次性诊断脚本（`host/_diag_alias.py`、`host/_crop.png`、`host/_s3_gui.png`、`host/_s2.txt`、`host/_s3.txt`、`host/__pycache__/`）已全部删除；`host/` 下 `.py` / `.md` 经核对全部 LF、无 BOM（`run_host.bat` 有意保持 CRLF，`.bat` 不在编码关卡扫描范围内）。

1.63 ★★★★ 上位机界面稳定性打磨：把“随数据变化的布局输入”全部变成常量（2026-10-13 15:45–16:30）

- **触发**：用户反馈 **“窗口还是有些许波动现象，显示位置不太稳定，先进行UI界面优化，然后再进行计划S4”**（m01180）。
- **抖动根因（四条，逐条对应真机现象）**：① 数值标签用比例字体，位数一变宽度就变（`19999.899 ↔ -0.10`）；② 提示/备注文本在 1~3 行之间跳 ⇒ 卡片高度变 ⇒ 整行上下挪；③ 长文本（`v_amp` 的门限串、`v_flags` 的告警名列表）把卡片撑宽 ⇒ 该行重新分配宽度；④ matplotlib 坐标轴随数据 `set_xlim(x[0], 0.0)` 自动缩放（x[0] 每帧都在变）；另有滚动条时隐时现、画布高度与卡片高度互相反馈两个放大器。
- **改法（全部落盘）**：`theme.py` 增 QSS 动态属性规则 `QLabel[mono="true"] { font-family: <等宽>; }`（**必须用动态属性**，因为 QSS 里写 `font-family` 会覆盖 Python 侧 `setFont()`）；`ui_kit.py` 增 `fix_lines(lab, lines)`（按 `fontMetrics().lineSpacing()*lines` 固定高度）、`hint_label(text, lines)`、`make_card(..., hint_lines)`、`field(..., vmin=150, mono=True)`；所有数值标签改等宽 + 最小宽度（`v_rpm` 190 px 等）；每张卡片设最小宽度（②③ 300、④ 400、⑤ 400、⑥ 540、⑦ 400、⑧ 560）；时域图横轴写死 `-150…0 ms`（标题里标明帧数与“已混叠，只看幅度”）；画布 `setFixedHeight(250)`；`QScrollArea` 竖直滚动条常驻、水平滚动关闭；`_update_live()` 的 flags 只显示前 3 个告警名 + `…`（完整列表进 tooltip）；状态 chip 文字写短（锁存位写成 `已锁存 b0/b2/b4`，完整来源进 tooltip），chip 固定宽 108 px。
- **量化验证**：临时脚本 `host/_stab.py`（**已删**）连真机后每 250 ms 采样 9 张卡片 + 16 个关键标签的 `geometry()`，打印哪些键变过 ⇒ **30 个采样点内“所有卡片与关键标签 geometry 完全没变”**。
- **顺带修掉本次自造的 bug**：`panel_wave._update_readouts()` 里删掉局部 `st` 后残留一句 `sq = st.get("stat_amp_sq")` ⇒ 每个刷新周期抛 `NameError`，`v_board` / `v_refresh` 两行读数停更；改为 `sq = (self.status or {}).get("stat_amp_sq")`，复跑后无异常。
- **截图复核**：`host/preview_s3.png` 已用新版重截（1752×2496 = 逻辑 1168×1664 @1.5×）：输入频率 **19999.899 Hz**、机械转速 **299998.5 RPM**、AB 边沿率 2,559,987/秒（单通道 640.0 kHz）、r 均值 26169、圆度偏差 0.43%、⑦卡 14 行 chip 全部完整显示（`ABZ+UVW 开` 不再被截断）、⑧卡提示行含“幅值门限 r 19661…39322”。
- **产物（本节）**：`host/theme.py`、`host/ui_kit.py`、`host/main_window.py`、`host/panel_speed.py`、`host/panel_interp.py`、`host/panel_alarm.py`、`host/panel_wave.py`、`host/signal.py`、`host/preview_s3.png`，以及 `host/README.md` 新增的“事实 7：界面按固定几何排布”。离线自检复跑全绿：`signal.py --selftest` 20 项、`encoder.py --selftest` 8 项、`regs.py --selftest` 2040 组。

1.64 ★★★★★ 上位机 S4 落地：ABZ/UVW 等效采样与六状态数据表（M5），并定案 Z 的 RTL 语义与“采样简并”现象（2026-10-13 16:40–18:20）

- **触发**：用户 **“界面正常了，现在开始S4”**（m01406）。S4 = 需求 M5（ABZ/UVW 六路电平 + 等效采样波形 + 六状态数据表）。
- **新增文件**：`host/abz.py`（纯逻辑、无 Qt，481 行：按电角度分箱累积、A/B 正交与占空比、UVW 六状态统计、Z 观测/理论口径、`synth_frames()` 造帧与 15 条离线自检）、`host/panel_abz.py`（卡片⑨，matplotlib 双图 + 六状态数据表 + 分箱选择 + 清空累积）、`host/selftest_s4.py`（无界面真机验收，12 项）。
- **改动**：`host/main_window.py` 新增 `cfg_from_regs(raw)`（原始地址字典 → 解码配置）、卡片⑨接线（`set_regs` / `set_output` / `set_f_in` / `feed`，倍频或高级参数写完后自动清空重来）、标题改 “S4 · ABZ/UVW 等效采样”；`host/encoder.py` 主口径改为**恒用「整数电周期数 + 角度残差」合成**（不再在两种口径打架时回退到帧数时钟，只告警）。
- **RTL 依据（决定了分箱口径）**：`rtl/abz_gen.v:118-121`（`inter` 0 视为 256、`div` 寄存器值 = div−1、ablimit 最小边沿间隔 `20 ns×(ablimit+1)×div`）、`:507-529`（`z_pulse_n = z_width + 1`、`z_in_win = z_window & abzen`、`z_load = z_in_win & ~z_ph_hi & r_z_prev_hi & ~r_z_active`、`Z_TIMEOUT = 4096 clk`）、`:673-722`（**v8 修正：只有“装载”受窗口门控，脉冲推进与超时兜底在窗外继续** ⇒ Z 允许出现在 `z_window` 标志之外，否则集成路径里提前 2 clk 关窗会把脉冲截成 40 ns）、`:726-727`（`a_out = r_div_idx[1] ^ apol`、`b_out = (r_div_idx[1] ^ r_div_idx[0]) ^ bpol`）；`rtl/uvw_gen.v:483-495`（6 扇区表）；`rtl/enc_top.v:114`（`ST_BITS_L = POLE_BITS + AW` = 22）、`:643-644`（每 `2^POLE_BITS` = 64 个输入周期出一次 Z 索引）。
- **等效采样原理**：把每帧的 `(电角度, 六路电平)` 累加进对应角度的箱，箱内均值 = 该相位下的高电平比例 ⇒ 从 10 kHz 帧流里重建 128 边沿/周期的 AB 方波与 UVW 六扇区。**Z 的“帧观测占空比”要乘 `f_in/帧率`**（本板 ≈ ×2）：Z 只占 `(z_width+1)/(4·inter·div)` ≈ 3.9% 的 AB 周期，欠采样下帧看到 z=1 的比例被放大。
- **真机验收（`python host\selftest_s4.py --port COM3 --seconds 10`）⇒ 12 项全 PASS**：104,561 帧 / 10.04 s = **9,922 帧/秒**；输入频率 **19,999.901 Hz**（帧数时钟 20,004.1、墙上时间 19,995.9、只看角度 −0.099 Hz 三口径对照）；**A/B 占空比 50.1% / 49.5%、A→B 相位差 −90.0°（偏差 0.0°）**；箱覆盖率 **256/256**（AB 与 UVW 都是）；六状态各 ≈16.5%（最大偏差 3.13%）、非法状态 1 帧（重同步残片，阈值 0.01%）；Z 观测 121 帧（0.1147%）、`z_window` 2246 帧、每帧平均 2.02 个电周期 ⇒ 理论 3125 个脉冲、捕获率 5.4%；dir 稳定（105,527/105,528）；limiter 恒 0。
- **★★ “采样简并”现象（重要，写进 README 事实 8 与排障表）**：本板输入 ≈ **20 000 Hz = 2.000× 帧率**，帧落在输入周期内的相位几乎冻结（每次运行只漂 ~0.1 Hz）⇒ Z（3.9% 脉宽、每 64 周期一次、每 10 s 应有 ~5 个脉冲）可能连续几十秒一帧都抓不到：实测 0 帧、16 帧、121 帧、123 帧都出现过。**这不是板子或软件的缺陷**；想稳定观察 Z 就把信号源频率偏离帧率整数倍（如 20 100 Hz，等效采样 1 秒内即可扫完整个周期）。判定脚本里因此把“Z 抓到与否”写成提示而不是失败项。

- **界面**：卡片⑨ 读数 = 板上配置（inter/div/极对数）、A/B 占空比、**A→B 相位差（≤25° 显示绿色）**、箱覆盖率、AB 边沿率（理论）、Z 脉冲观测/理论；下方六状态表按偏离 1/6 的幅度变绿/橙/红；分箱可选 64…1024；改倍频自动清空重来。
- **产物（本节）**：`host/abz.py`、`host/panel_abz.py`、`host/selftest_s4.py`、`host/panel_wave.py`（同批 UI 常量）、`host/main_window.py`、`host/app.py`、`host/encoder.py`、`host/README.md`（事实 8 + 排障 2 行 + 进度表）、`host/preview_s4.png`（真机全页 1752×3315）。离线自检：`abz.py --selftest` 15 项、`encoder.py --selftest` 8 项、`signal.py --selftest` 20 项、`regs.py --selftest` 2040 组全绿；19 个 `host/*.py` 全量 `py_compile` 通过。

1.65 ★★★★★ 上位机 S5 落地：CSV 记录与 PNG 导出（M6）+ 整体打磨，并修掉两个“时间”真 bug（2026-10-13 19:20–21:20）

- **触发**：用户 **“现在开始S5”**（m02224）。S5 = 需求 M6（开始/停止记录、CSV 含原始 18 字节帧 + 解析列 + 上位机时间戳、一键导出波形 PNG、显示本次时长/帧数/校验错帧数/丢帧估计）+ 整体打磨（配色/中文提示/异常处理）；验收方式 = **30 分钟连续运行 + 验收标准 1~8 逐条过。**
- **新增文件**：`host/recorder.py`（纯标准库，无 Qt/串口依赖：一行一帧写 CSV、自动分卷、写失败不崩；10 条离线自检）、`host/export_png.py`（无 Qt，用 matplotlib 把波形/ABZ 重画成 1950×1080 与 1950×990 大图并把板配置与读数写进图里；10 条离线自检）、`host/panel_record.py`（卡片⑩ 记录与导出：状态、六项读数、保存位置、两个导出按钮、“连上就自动开始记录”）、`host/selftest_s5.py`（真机验收 8 项 + `--soak-minutes` 长稳；CSV 流式回读，不把上千万行读进内存）。
- **改动**：`host/link.py` `_pump()` 给每帧附 `f["raw"] = raw`（原始 18 字节进 CSV，事后不靠上位机也能解帧）；`host/main_window.py` 卡片⑩接线（`feed` / `tick` / `set_connected` /自动记录）、**底部常驻状态条**（帧率 | 丢帧 | 校验错 | 已运行 h:mm:ss）、**清除锁存告警二次确认**（`QMessageBox`，取消则记日志返回）、标题改 “S5 · 记录与导出”、日志卡片编号 ⑤→⑪；`host/panel_abz.py` 补 `analyzer_snapshot()` / `theory(f_in)`；`host/app.py` 新增 `--record` / `--record-dir` / `--export`（无人值守长稳用），`--smoke` 退出前停表并打印记录统计。
- **关键设计决定（同步写进 README 事实 9）**：① 一行一帧，**原始 18 字节 hex + 解析列 + 上位机时间戳**三样都写；② 时间戳按帧时隙回推：一批帧的**末帧 = 本批到达时刻**，批内每帧 0.1 ms，但**不许早于上一批的末帧**；链路卡 3 秒就在时间轴上留 3 秒台阶，**不凭空补帧**；③ 时间列写 **4 位小数（0.1 ms）** —— 写 3 位会把同一批的 10 帧写成同一时刻；④ **自动分卷**（默认 100 万行 / 256 MB，`…_part%03d.csv`，序号跨卷连续）：10 kHz 下一分钟约 40 MB、六分钟就满 100 万行，不分卷会产出 Excel 打不开的巨文件；⑤ `utf-8-sig` + CRLF（Excel 双击不乱码、能分列）；⑥ 写失败（磁盘满/被占用）只给中文提示并自动停表，绝不把界面带崩。
- **★★修掉的两个“时间”真 bug**：① `feed()` 原来把**单调时钟**当墙上时间写进“上位机时间”列 ⇒ CSV 打开是 **1970-01-02**（Windows 上单调值≈开机秒数）；改为 `t_start_wall + rel` 换算，并给自检 [5] 加断言（原自检只查相对时刻，所以这个 bug 溜过去了）。② 界面每 100 ms 才把攒下的帧交一次，一次 `feed()` 常含多个 50 ms 批；按“末帧=到达时刻”往前回推会让**本批头部落到上一批之前** ⇒ CSV 相对时刻非单调（真机自检直接报“相对时刻不是单调递增”）；改为 `t_lo = max(t_end − n·0.1 ms, 上一批末帧)`、`step = (t_end − t_lo)/n`，并新增自检 **[10] 合并批单调**（0 处倒流；380 ms 卡顿在时间轴上留台阶）。
- **打磨**：底部常驻状态条满足需求“状态栏显示帧率/丢帧/校验错/已运行时长”；清除锁存这类写板操作加二次确认；修掉未开记录时 `--smoke` 退出码 1 的崩溃（`RecordPanel.stats()` 返回 `None` ⇒ `app.py` 调 `.get` 抛 `AttributeError: 'NoneType' object has no attribute 'get'`；改为返回 `{}`，调用处 `or {}`）。
- **真机验收（`python host\selftest_s5.py --port COM3 --seconds 12`）⇒ 8 项全 PASS**：板上 inter=32 div=1 极对数=4；**119,147 帧 / 12.0 s / 18.2 MB / 9,915 帧每秒 / 丢帧估计 0.83%**；CSV **24 列**、时间跨度 12.01 s、墙上时间 `2026-10-13 15:13:00.635 → 15:13:12.647`、抽 3 行原始帧独立解回一致；导出 PNG：波形 **156.0 kB / 1950×1080**、ABZ/UVW **150.3 kB / 1950×990**；输入频率 19,999.900 Hz。离线自检：`recorder.py --selftest` **10 项 PASS**（写入 20.3–21.1 万行/秒）、`export_png.py --selftest` **10 项 PASS。**
- **★★★★★ 30 分钟连续运行（官方长稳：`python -u host\selftest_s5.py --port COM3 --soak-minutes 30 --dir host\_s5_soak`，2026-10-13 16:54:12 → 17:24:12）⇒ 14/14 PASS、exit code 0**：记录 **17,858,785 帧 / 1800.0 s / 2.74 GB / 9,922 帧每秒 / 18 个分卷 / 丢帧估计 0.76%**；帧率**中位 9,847、最低 9,440、最高 10,397 帧/秒**（最低/中位 0.96、最高/中位 1.06，判据 0.70~1.30 全过）；**内存起 100 MB、终 86 MB、峰 100 MB（增长 −13.9 MB，不涨）**；链路已收 17,859,261 帧、**丢批 0、丢帧 0**；CSV 逐卷回读 **18 卷 × 17,858,785 行 × 24 列**、跨度 1,799.96 s、墙上时间 `2026-10-13 16:54:12.391 → 17:24:12.353`、抽 3 行原始帧独立解回一致；导出 PNG：波形 156.8 kB / 1950×1080、ABZ/UVW 152.9 kB / 1950×990。
- **验收标准 1~8 逐条取证**：① 列端口/连接/读身份证 = `host/selftest_link.py --port COM3` PASS；② 端口列表与刷新 = `host/ports.py` + `refresh_ports`；③ inter 64→256 写回读一致 = `selftest_s2.py` PASS（末次 62,606 帧 / 6.006 s / 10,424 帧每秒，主口径 19,999.8999 Hz）；④ 转速与已知输入一致（19,999.9 Hz）；⑤ 波形连续刷新 ≥10 次/秒 = `host/main_window.py:51 UI_PERIOD_MS = 100` + ⑧卡 `v_refresh` 实测；⑥ ABZ/UVW 正确 = `selftest_s4.py --seconds 10` PASS（12 项 + `abz.py` 15 项）；⑦ 30 分钟不崩 + 记录文件可打开 = 上条长稳 + `--verify-dir` 逐卷回读；⑧ 拔插恢复 = `--reconnect` **2/2 PASS**（连上收 19,946 帧 → 空窗 1 s → 重连收 19,919 帧；正常关端口的等效验证，异常拔线的报错路径由 `link.py` 异常兜底 + S1 自检覆盖），并修掉“拔线后界面卡在断开态、必须先点一次『断开』才能重连”的真问题（`_tick()` 的 `closed` / `error` 分支检测线程已死则 `QTimer.singleShot(0, self.disconnect_port)`）。S3 复跑 PASS（含 `signal.py` 20 项）。
- **产物（本节）**：`host/recorder.py`、`host/export_png.py`、`host/panel_record.py`、`host/selftest_s5.py`、`host/main_window.py`、`host/app.py`、`host/link.py`、`host/panel_abz.py`、`host/README.md`（事实 9 + 排障 3 行 + 文件表 + 使用说明第 10 条）、`host/preview_s5.png`（真机全页 1752×3801 = 逻辑 1168×2534，卡⑩显示“● 记录中”与本次时长/帧数/体积/丢帧/校验错/写入速率）；`host/*.py` **23 个**全量 `py_compile` 通过。

1.66 ★★★★★ 上位机 S1–S5 全面自查：101 项离线自检（+倍频 2043 组往返）+ 真机全套复跑，修掉 26 处真缺陷（2026-10-13 21:30 起）

- **触发**：用户要求 **“先全面自查一遍”**（m03503）。自查方式：把 `host/` 的 23 个文件交给 6 个子代理分片审计（`main_window.py` | `abz.py` + `export_png.py` | `encoder.py` + `signal.py` + `regs.py` | `recorder.py` + `panel_record.py` | `link.py` + `app.py` | `panel_*.py` + `theme.py`），每条结论都要求给出**文件:行号 + 触发条件 + 后果**，再由本人逐条复核并修复；**子代理报告只作线索，判据以复现/真机为准。**
- **★★修掉的真缺陷（按严重度）**：
- **`encoder.py` 丢帧率口径是假的**：原来 `loss_pct = |mean_pos|/|int_part| − 1` 只是**取整残差** —— 零丢帧的 12345 Hz 报 22.52%、14000 Hz 报 40%（面板还染告警色），而真丢 35% 只报 7.69%。改为按链路时隙真值 `1 − dn/(clock−k0)`（新增 `slots` / `resid_pct` 键），并给口径切换加**迟滞**（1.5%/3%）+ `clock_ok` 先验，`span` 在切换时用对应时钟，自检加 **[9] 零丢帧不报丢帧、[10] 丢帧率口径 = 50.0%**（8 条 → **10 条**）。
- **`regs.py` 读不到现值就按 0 覆盖同字节**：`cur or 0` 会让 `0x0108` 读失败时写 `zwidth` **清掉板上 bit14**（实测板上 `0x0108 = 0x4004`）、`0x010C` 读失败时 `zphase` / `uvwph` 互相清位。改为 `adv_field_writes()` **读不到现值就返回 `None`**（=要求先点『读回』），`panel_interp.py` 逐字段检查，任一字段缺值就 `adv_blocked` 整批不发并给中文提示；自检 2040 → **2043 条。**
- **`link.py` 命令执行固定 0.502 s + 丢命令**：老的判据 `want = max(1, len(payloads)−1)` 在板子对每条命令立即回一个应答时永远凑不齐（连发把应答挤掉）⇒ 每次写寄存器固定睡满 0.5 s 超时；`self._cmd_req = None` 还会静默抹掉执行期间提交的新命令。改为**每发一帧收一轮**（`_collect()` 带 6 ms 安静窗口）、应答

进 `_replies` 队列（原来是单槽 `_pending_reply`，多应答只留最后一条）、`_take_cmd()` / `_cmd_done(req)` 只清自己那条、`transact()` 的等待移到 `_cmd_lock` **外**。真机计时：读寄存器 **0.001 s**、写 **0.002 s**（`0x010C` 0.052 s），**原来是固定 0.502 s**；`reply_ok=24 reply_timeout=0`。

- **`disconnect_port()` 关闭顺序丢尾巴**：原来“先排空、再 stop”，在“排空返回空 → stop 生效”之间串口线程还会推进 1~2 批（≈1000 帧），那几批永远进不了 CSV。改为 **`w.stop()`（内部 join）后再 `_drain_events()`**，真机核对里“关窗时队列 2 条 → 关窗后 0 条”，`记录器行数 = 界面取帧数` **差 0**。
- **`abz.py` Z 脉宽/周期口径自相矛盾**：`z_width_deg` / `z_duty` 按 `÷div` 折算，而同文件 `synth_frames` 按原始 idx 算（与 `div` 无关）⇒ 同一快照两个说法。按 `rtl/abz_gen.v:704`（每个 `idx_exec` 递减 `r_z_left`）+ `rtl/abz_gen.v:41-42/358/422`（每 `div` 个原始边沿才点火一次）改 ∝ **div**，`ab_period_deg(inter, div=1)` / `edge_step_deg(inter, div=1)` 加 `div` 参数，`theory()` 与 `synth_frames` 同源；docstring 记明与 `docs/01` 的口径分歧**待真机 ILA 定案**。
- **`abz.py` 假上升沿 + `bins=0` 崩**：`_rising()` 把“上一箱无样本”当低电平，导致没有 AB 跳变也报出相位差（单帧 A=B=0 报 0.00°）——改为 `None` 不算跳变；`bins=0` 会让 `snapshot` 除零、`feed` 索引越界——新增 `clean_bins()`。自检 **15/15 PASS**。
- **`export_png.py` 三处**：① 自检 [5] 的 `r5` 在 try 内赋值、try 外引用 ⇒ 真抛异常时整个自检以 `UnboundLocalError` 崩掉（已给初值）；② `_save` 用调用方 path 判成功（无后缀时 matplotlib 写成 `path+".png"` ⇒ 报 `ok=False`/WinError 2；目标是目录时写到同级 `<目录>.png` 却报 `ok=True`）——改为先判 `isdir`、无后缀自动补 `.png`、**用最终路径**算 bytes/px，且 `px` 与 `_save` 统一用 `int()`（dpi=133 时与 `round()` 差 1 像素，会让 `selftest_s5` 假失败）；③ `_meta_lines` 的 `if v` 丢 `0` / `False`、键类型混排时 `sorted()` 崩——改 `v is not None` + `sorted(key=str)` + 非 dict 兜底。自检 10 → **11 条**（新增 **[11] 路径/像素契约**）。
- **`recorder.py` 自检补齐 3 条**（[11] 同名不覆盖、[12] 写盘失败自收场、[13] 上限/时钟钳位 ⇒ **13/13 PASS**）；★教训：自检用例文件名不能重复 —— [10] 用 `e.csv`、[11] 用 `f.csv` 与 [7]/[8] 撞名，而 `start()` 会用 `_unique_path()` 自动改名，“读回构造出来的路径”读到的是上一个用例的文件，报出“跨度 0.000 s”的**假失败**；改用独立名并读 `rec.files[0]` 后正常。临时目录清理也改成**不带 `ignore_errors` 的 rmtree + 顺延编号目录**（上一轮句柄没释放会删不掉，残留旧文件会被下一轮读到造成假失败）。
- **`selftest_s4.py` 判据过严**：① AB 等效采样填箱速度由 `f_in` 与 `FRAME_HZ` 的可通约性决定（板上 `f_in≈1006 Hz`、帧率 10000 Hz 时 2 s 只填 34/256 箱）——新增 `--max-seconds`（默认 `max(4×--seconds, 0)`）**填不够自动延长**并复用已收帧；② Z 逐帧命中数从**失败判据降为提示**：Z 常态只有几个 `idx_exec` 拍（≈1 µs）而帧周期 100 µs，RTL 的 `Z_TIMEOUT=4096 clk`（≈82 µs）超时兜底又会改变脉宽，同一工况实测命中 0/88/107/197/369 帧都出现过（§1.64 记过 0/16/121/123），Z 的“存在性”由「z_window 标志有出现」+「Z 脉冲稀疏」+ 离线 `abz.py` 15 项负责。
- **其余已修**：`app.py --seconds` 非有限/越界 ⇒ 中文报错 + exit 2（原来 `--seconds -1` 直接崩）、`--export` 缺 `--port` ⇒ exit 2、`--record-dir` 失败 ⇒ exit 2、截图/连接失败记 `_FAIL` 并 `return 1`、`--smoke --seconds 1 --export` 会在导出前退出（`wait_ms` 现在 ≥3500）；`panel_record.py` `finish_export` 不再 KeyError、`_refresh_buttons(self._conn)` 不再把按钮硬顶成可用、`set_dir` 兼收 `ValueError`；`panel_alarm.py` 告警 chip 由定宽 108 px 改 `min 108 / max 230`（“0x053A b0 置位”实测要 164 px，定宽会两端裁字）；`panel_speed.py ok=False` 时把 `v_f/v_rpm/v_turns/v_cycles/v_dir` 一并清成“—”（原来挂着上一次的转速）；`ports.py` 新增 `board_rank()`（VID:PID `1A86:7523`/CH340/CH341 = 0 → 含 USB-SERIAL = 1 → 其它 = 2，`preferred_port` 取 rank 最小），原来任何 “USB-SERIAL” 设备都会把真正的 CH340 顶掉；`link.py _emit()` 队列满挤掉 batch 时补记 `dropped_batches` / `dropped_frames`；`main_window.py` 重连时 `interp.reset()` 清掉上一连接的寄存器快照（避免拿旧快照做读-改-写）；`selftest_link.py` 不再无界累积 `frames`（`--seconds 60` 时原来要吃 60 万条 dict）。
- **真机复跑证据（2026-10-13 21:40–22:10，COM3 @2 Mbaud，板上 v12）**：`selftest_link.py` **PASS exit 0**；`selftest_s2.py` PASS；`selftest_s3.py` PASS；`selftest_s4.py --seconds 8` **12/12 PASS**（自动延长到 16.05 s：AB 箱 **214/256 = 83.6%**、UVW **256/256**、A 占空比 50.5%、A→B 相位 **−90.0°**、非法 UVW **0 帧**、六状态均衡最大偏差 **1.02%**、Z 占空比 0.0552%、`z_window` 置位 2489 帧、方向位 159402/159402、限幅器未激活）；`selftest_s5.py --seconds 12` PASS 8/8、`--reconnect` **2/2 PASS**；`selftest_ui.py` 离线 **30/30 PASS exit 0**、`--real-port COM3 --seconds 6` **PASS exit 0**（三判据全 0 差：`记录器行数 = 界面取帧数`、`文件行数 = 记录器 rows`、`关窗后事件队列 0` 条）。
- **`signal.py evaluate(r=None)` 会崩**：门限读到了、但这一轮 r 还没算出来（缓冲区刚清空/只读了状态）时，`"r=%.0f 正常" % None` 抛 `TypeError: must be real number, not NoneType` —— 在 Qt 槽里就是整条刷新链断掉。改为 `_r_ok` 判定（r 与两端门限都非 None 才带数值），并补 2 条自检（**20 → 22 项**）。
- **`link.py read_size` 是死参数**：`_read_available()` 原来只用 `in_waiting`，构造参数 `read_size=4096` 从未生效。改为**单次读取上限**（积压大就多读几轮；10 kHz 一帧 18 B、50 ms 一批 ≈ 9 kB ⇒ 4096 读 3 轮），真机 `selftest_link.py --port COM3 --seconds 5` 复跑 **PASS exit=0**。
- **`regs.py decode_inter_div(None, …)` 早退用例**：核对确认它对 `None` 已有早退（不会去 `int(None)`），补 2 条自检把它钉住。
- **3 处文档/口径澄清（不是行为缺陷）**：① `signal.py` 的 `VAL_BITS` b4 译成“相位滞后致命”，但 `rtl/reg_file.v:248-249` 把 b4 硬接 0（地址表 `rtl/reg_file.v:79-80` 明写 “b1/b4/b6/b7 恒 0”），所以 `0x0404/0x0405` 永远看不到这一位 —— 代码保留该分支（RTL 将来接出来不用改主机）并**加注释写明真相**，真正的 `lag_fatal` 只从遥测 `io_flags` bit1 看；② `signal.py` 的 `t` 注释写“以最新一帧为 0”，实际存的是帧计数时间、由 `panel_wave.py:195` / `export_png.py` 减 `t[-1]` 才变相对值 —— 注释改正；③ `selftest_link.py` 打印的“参照工具 212 帧校验错”是 **2026-10-13 那次参照测量**的数字，改成明确标注为历史参照，本次运行只看“本窗口 N 帧校验错”。
- **空间清理**：删掉自查用的 `host/_probe_tmp/`（探针脚本与日志）、`host/_probe_e2e.py`（临时端到端探针，其覆盖的场景已进 `selftest_ui.py`）、`host/_sb.png`；保留 `host/_s5_out/`（`selftest_s5.py:59` 的默认输出目录，每次跑都重生成）。
- **离线自检现行条数（重数过）**：`encoder.py` **10**、`abz.py` **15**、`export_png.py` **11**、`recorder.py` **13**、`signal.py` **22**、`selftest_ui.py` **30** ⇒ 合计 **101 项**，另有 `regs.py` **2043 组**倍频编解码往返 + 5 条边界/缺值用例 —— 全部 PASS（`exit=0`，`python -m compileall host` 亦 exit=0）。
- **30 分钟长稳（`link.py` 重写后）**：第一次跑到 **1320 s / 13,101,391 帧 / 2.03 GB / 丢帧 0.76% / 内存 82→86 MB / 14 卷** 时被我误判中断 —— 沙箱里 `Get-PSDrive C` 报 “free 0.0 GB”，而实际上 `System.IO.DriveInfo` 与 Python `shutil.disk_usage` 当时都还有 **29 GB**（**教训：本沙箱里别信 `Get-PSDrive`，要用 `DriveInfo` / `shutil.disk_usage`**）。顺手把这个坑变成能力：`host/selftest_s5.py` 在开记录前新增**盘空间预检**（按 10 kHz ≈ 1.5 MB/s、2 MB/s 保守估 + 20% 余量，不够就中文提示并 `exit 1`）⇒ **S5 判据 8 项 → 9 项。**（第二次完整 30 分钟结果见下面一行。）
- **30 分钟长稳复跑（`link.py` 重写后，2026-10-13 19:17:36 → 19:47:37，日志 `host/_s5_soak_run2.txt`）⇒ 15 项全 PASS**：**1800.0 s / 17,859,119 帧 / 2.77 GB / 18 卷 / 9,922 帧每秒 / 丢帧 0.76%**；CSV 24 列、跨度 1799.96 s、抽 3 行原始帧独立解回一致；长稳判据：帧率中位 **9841**（最低 9421 / 最高 10391，最低/中位 0.96、最高/中位 1.06）、**内存 起 125 MB → 终 86 MB / 峰 125 MB（增长 −38.9 MB，18 卷全程不涨）**、链路共收 17,859,596 帧没死、**丢批 0 / 丢帧 0**、记录文件可打开；导出 PNG 波形 447.9 kB（1950×1080）、ABZ/UVW 152.2 kB（1950×990）。与 `link.py` 重写前的基线（17,858,785 帧 / 2.74 GB / 18 卷 / 丢帧 0.76%）**逐项一致 ⇒ 协议层重写没有引入长跑回归**；另 `--seconds 12` 复跑 **9 项 PASS**（新增的“磁盘空间够本次记录”判据实测「剩 26.43 GB / 需 0.02 GB」）、`--reconnect` 复跑 **2/2 PASS**（19,944 帧 → 拔插 → 19,957 帧）。
- **仍留的零碎项（已知、当前无害，不阻塞验收）**：`signal.py:40` 的 `LATCH_CLEAR_VALUE = 0x00FF` **决定不收窄**——审计建议收窄到实际会被锁存的位（`0x2D`：b0/b2/b3/b5，因 `rtl/reg_file.v:248-249` 把 b4 恒接 0），但 `rtl/reg_file.v:412-413` 是整字节 W1C，对永不置位的 b1/b4/b6/b7 写 1 无副作用；反过来说，若将来 RTL 把 b4 接出来，`0x2D` 就清不掉它了 ⇒ 保持 `0x00FF`（清掉所有可能位）更稳，已在代码注释里写明这个取舍；`abz.py` 的 Z 脉宽口径（∝ `div`）待真机 ILA 与 `docs/01:596` 定案；`ports.py` 的 FTDI 分支（rank=1，含 `USB-SERIAL` 但不含 CH340）无法在本机复现，逻辑只做过单元级核对。

1.67 ★★★★ 上位机 S6 落地：PyInstaller 单文件 exe（208.2 MB）+ 4 个"单文件陷阱"改造 + 查清"沙箱里 exe 起不来"（2026-10-13 20:10–20:45）

- **触发**：用户 **"先做 1 再做 2"**（m00051）：先提交 S1–S5 成果防丢（§1.66 末条那次提交的由来），再做 **S6 = PyInstaller 单文件 exe。**需求 `docs/10_host_requirements.md:22` = 「交付形式 = 源码 + 最终打包 exe（两个都要）」、`:147` = 「PyInstaller 单文件 exe，双击即用，目标机不装 Python」、`:184` = S6 验收「单文件 exe，在目标机上双击运行验证」。
- **构建环境**：`PyInstaller 6.22.3` 装到**用户站点目录**（`C:\Users\zhaoy\AppData\Roaming\Python\Python313\site-packages\PyInstaller\__init__.py`）—— Anaconda 的 site-packages 不可写（`pip` 以 workspace-write 装包时 `[Errno 13] Permission denied: …\Temp\dsh-…\pip-unpack-…\pyinstaller-6.22.3-py3-none-win_amd64.whl`），且 `…\Python313\Scripts` **不在 PATH ⇒ 一律 `python -m PyInstaller`**。顺带装上 `altgraph 0.17.5` / `pefile 2024.8.26` / `pyinstaller-hooks-contrib 2026.8`；走清华镜像（`-i https://pypi.tuna.tsinghua.edu.cn/simple`，直连 pypi.org 要 10.7 s）。本机 `pwsh` 不在 PATH ⇒ 脚本一律 `powershell -NoProfile -ExecutionPolicy Bypass -File …`（Windows PowerShell **5.1**）。
- **产物（首轮打包，`_exe_build2.log`）**：`host\dist\host_app.exe` = **218,344,843 B（208.2 MB）@2026-10-13 20:14:33**，用时 **143 s**，SHA256 `3CCFA9852680A864588C96FBDC3E1930D9176B63886C97167B42448FA0DCC446`；修好烟测脚本后重打（`_exe_build3.log`）**218,344,428 B @20:40:32 / 135 s / SHA256 0E0FE007…**`（★ 字节数与 SHA256 **每打一次都不同**，PE 头里嵌构建时间戳 ⇒ 不能当回归判据，与 §2 位流"字节数不能当判据"同理）。
- **新增/改动（共 3 个新文件 + 4 个 host 模块）**：
- `host/host_exe.spec`（新）：`SPEC_DIR = os.path.abspath(SPECPATH)`、`REPO_ROOT/TOOLS_DIR/ENTRY` 全部从 spec 目录派生；`pathex=[SPEC_DIR, TOOLS_DIR, REPO_ROOT]`；`hiddenimports=["uart_scope","uart_spi","matplotlib.backends.backend_qtagg","serial.tools.list_ports"]`；`excludes` 去掉 tkinter/PyQt5/PyQt6/PySide2/shiboken2/pandas/scipy/sklearn/sympy/numba/sqlalchemy/IPython/jupyter/notebook/pytest/sphinx/docutils/PIL.Ima 的 tkagg·pdf·svg·ps·webagg 后端/numpy.f2py·testing·distutils；`EXE(name="host_app", upx=False, console=False)`（不用 UPX，压缩过的 exe 更容易被杀软误报）。
- `tools/build_host_exe.ps1`（新，**UTF-8 with BOM**，5575 B）：环境自检（`python` / `PySide6` / `serial` / `matplotlib` / `PIL` / `PyInstaller` 逐个 import，再查 `tools/uart_scope.py`、`tools/uart_spi.py` 在位）→ 清理 `host/_exe_work` + `host/dist` → `python -m PyInstaller --noconfirm --clean --workpath … --distpath …` → 打印路径/字节数/时间/SHA256/用时。本轮新增参数 **`-SmokeSeconds`（默认 6）** 与 **`-NoSmoke`**。
- `host/link.py`：新增 `_exe_dir()`（`getattr(sys,"frozen",False) or hasattr(sys,"_MEIPASS")` ⇒ `os.path.dirname(os.path.abspath(sys.executable))`，否则 `None`），`_ensure_tools_on_path()` 按 **exe/tools → `_ROOT/tools` → `_HERE/tools`** 顺序找（`host/link.py:31-59`）。
- `host/recorder.py::_load_tools()`：同判据，依次把 `exe/tools`、`repo/tools` 加进 `sys.path` 再 `import uart_scope`。
- `host/export_png.py`：新增 `_writable_dir()`（frozen ⇒ `tempfile.gettempdir()`），自检临时目录改用它。
- `host/selftest_ui.py`：新增 `_tmp_dir(name="_ui_selftest_tmp")`（同判据），两处 `os.path.join(_HERE, "_ui_selftest_tmp")` 改用它。
- `.gitignore` 追加 `host/_exe_work/`、`host/dist/`、`*.spec.bak`（exe 是产物不入库）。
- **★★踩坑 1（真踩，已修）：`.ps1` 缺 UTF-8 BOM ⇒ Windows PowerShell 5.1 按 GBK 解码中文，整段脚本崩。**报 `The ampersand (&) character is not allowed. The & operator is reserved for future use`、`The string is missing the terminator: "` 等一串 ParserError。**根因是中文注释尾字节吃掉了行尾换行**（GBK 双字节把 `\n` 吞进去 ⇒ 下一行代码变成上一行注释/字符串的一部分）。仓库既有 `tools/run_sim.ps1`、`tools/lint_all.ps1` 都是 UTF-8 with BOM（`tools/to_lf.py:87-96` 把"缺 BOM 的 .ps1"当问题报，并且**只对 .ps1 保留/补回 BOM**）。修法：`[System.IO.File]::WriteAllText($p,$t,[System.Text.UTF8Encoding]::new($true))`，再用 `[System.Management.Automation.Language.Parser]::ParseFile` 验 0 语法错误。★ 同类坑当天又踩了一次：临时探针脚本 `_probe_exe.ps1` 用 write 工具写（无 BOM），里面 `Get-ChildItem 'host\dist\*.exe'` 因为路径字符串被 ANSI 解码而匹配不到，**报"no exe found"空转**；补 BOM 后立刻正常。
- **★★踩坑 2（真踩，已修）：`SPECPATH` 已经是 spec 所在目录**（PyInstaller 在 `PyInstaller/building/build_main.py:1140` 做 `CONF['specpath'], CONF['specnm'] = os.path.split(CONF['spec'])`、`:1189` 注入 `'SPECPATH': CONF['specpath']`）⇒ 首版写成 `os.path.dirname(SPECPATH)` 得到的是**仓库根**，报 `ERROR: script 'C:\Users\zhaoy\Desktop\FPGA\app.py' not found`（`_exe_build1.log`，exit 1，用时 2 s）。
- **★★踩坑 3（真踩，已修）：GUI 子系统 exe 在 PowerShell 里"看起来立刻退出"。**产物 `Subsystem=2`（`console=False`），`& host_app.exe --smoke …` **不等待窗口进程**就返回 ⇒ 脚本拿到的退出码/截图全是空气，首轮烟测**误报"烟测没生成截图"**。改成 `Start-Process -PassThru` + `WaitForExit($SmokeSeconds*1000)` + 读 `_smoke.out.txt` / `_smoke.err.txt` 才算判得准。（`-SmokeSeconds` 默认 6 s：单文件每次启动都要先把包解到临时目录，208 MB 冷启动 3~10 s，杀软扫描时更久。）
- **★★★★踩坑 4（本轮最费时，结论：不是产物问题，是 DSH 沙箱受限令牌）**：**在 DSH 沙箱里，单文件 exe 一律起不来**，stderr 只有两行：`[PYI-18532:ERROR] Failed to extract Lerc.dll: failed to open target file!` / `fopen: Permission denied`（后跟 `Failed to extract entry: Lerc.dll.` / `failed to extract files!`）。
- **根因（已用 icacls 坐实）**：PyInstaller 引导器创建 `%TEMP%\_MEIxxxxxx` 时用**显式安全描述符**（`LOADER: initializing security descriptor from string: D:(A;;FA;;;S-1-5-21-…-1001)`）只给"当前用户 SID"完全控制；而 DSH 沙箱进程的写权限是靠**额外令牌 SID**（`icacls <仓库根>` 里的 `S-1-4-658417623-361858394:(OI)(CI)(W,D,DC)`）授的，`whoami /groups` 显示当前进程 `Mandatory Label\\Low Mandatory Level`。引导器**不会**把令牌 SID 写进新目录 ACL ⇒ **它自己刚建的目录自己写不进去**（`icacls` 实测那个 `_MEI…` 目录 = `ROG\\zhaoy:(F)` + `Mandatory Label\\Low Mandatory Level:(I)(OI)(CI)(NW)`，**没有令牌 SID**）。
- **排除了"大包/某个 DLL"这条岔路**：把 `console=True`、`debug=True` 的诊断 exe（`host/_diag.spec` + `host/_diag_entry.py`，逐步落盘日志）跑起来，日志直接给出上面那两行；再做**最小复现** —— 8,440,297 B 的 `--onefile` 单文件 exe（`_mini_dist\_mini.exe`，只有一行写文件）报**同样**的 `Failed to extract LIBBZ2.dll … fopen: Permission denied`。⇒ 与包大小、依赖、`console=True/False` 全都无关。
- **试过但绕不过**：① 预建 `_MEI00000001` 目录 + 给令牌 SID 重赋权（`icacls` 对 `S-1-4-…` 报 `No mapping between account names and security IDs`，而 DSH 新建的目录又不带令牌 SID，加不上去）；② `--runtime-tmpdir` 指到仓库内（`_exe_rt`、`_rt`）—— 新目录一建出来仍是那份只有用户 SID 的 ACL；③ 提升到 `danger-full-access` 重跑，同样失败。
- **另一条证据（超时不是"慢"，是引导器的错误弹窗在等点击）**：`Start-Process` 6 s/180 s 都不退出、进程 WorkingSet 只有 12 MB（远小于 208 MB 的包），`_MEI…` 目录里 0 个文件 ⇒ 引导器 `FatalError()` 后的 MessageBox 把进程挂住了。
- ⇒ **沙箱内的用法**：只跑 `powershell -File tools\build_host_exe.ps1 -NoSmoke`（出产物、exit 0），**烟测与真机验证放到沙箱外的普通终端/双击**（`host\dist\host_app.exe --smoke 图.png --seconds 4`、`--port COM3`）。这条已写进脚本 `.Notes` 与 `host/README.md` 的"六个坑"。
- **★★★★补充（2026-10-13 22:30，第三轮打包）：界面优化批次 1/2/3 落地后重打，并顺手修掉"exe 里参数记忆失效"这个真 bug。**
- **起因**：用户手里的 `host\dist\host_app.exe` 是 **20:42:50 / 218,345,402 B** 那版 —— 解包核对 `PYZ-00.pyz` 发现**没有 `ui_prefs` 模块**，`main_window` 里也没有 `wheelOwn` / `清空日志` / `显示级别` / `CardMax` / `_install_wheel_router` ⇒ 它在批次 1/2/3 **之前**，用户 m06024 那六条抱怨（滚轮、示波器量程、记录定格、日志刷屏、卡片放大）正是旧包的行为。用户 m07281 要求重打。
- **核对手法（不必运行 exe，沙箱里就能验）**：`from PyInstaller.archive.readers import ZlibArchiveReader; z = ZlibArchiveReader(r"host\_exe_work\host_exe\PYZ-00.pyz")`，`z.toc` 看模块在不在；再 `marshal.dumps(z.extract(mod))` 后按 **UTF-8 子串**搜字符串常量与方法名（★ 要按子串搜：完整字面量是 `'满屏 %g ms'` 这类，方法名只出现在 co_names 里，只有全量字节搜才可靠）。
- **★ 新发现真 bug（已修）：`host/ui_prefs.py` 的 `path()` 原来用 `os.path.dirname(os.path.abspath(__file__))`** ⇒ 单文件 exe 里 `__file__` 指向 `%TEMP%\_MEIxxxx`（**退出即删**的临时解包目录），开窗读/关窗写都落在那里 ⇒ **exe 里"参数记忆"等于失效**（还可能写不进去）。修法与另外四处冻结改造同

款：新增 `_home_dir()`（`host/ui_prefs.py:44-63`，`getattr(sys, "frozen", False) or hasattr(sys, "_MEIPASS")` ⇒ `os.path.dirname(os.path.abspath(sys.executable))`，否则 `_HERE`），`path()` 改用它；`--selftest` 由 **13 条 → 14 条**（新增第 ⑦ 组「冻结时参数文件落在 exe 旁边（不是临时解包目录）」）⇒ `python host\ui_prefs.py --selftest` **14 项通过 exit 0**、`python host\selftest_ui.py` **73 项通过 0 失败**（无回归）。

- **产物（第三轮，最终交付包）**：`host\dist\host_app.exe` = **218,385,419 B（208.3 MB）@2026-10-13 22:30:46**，用时 **112 s**，SHA256 `72493676E73D3BA1D400BCE8777FD3808CA93A516D95D6B54F00414A0A819038`（中间还有一轮 22:24:48 / 218,385,414 B / 129 s / SHA256 `C2F3F190…`，是"改了 `ui_prefs` 之前"的包，已被这轮覆盖）。★ 字节数/SHA256 每打一次都不同（PE 头嵌时间戳）⇒ **不能当回归判据。**
- **包内逐项核对（第三轮，全过）**：`ui_prefs` 模块在位；子串命中 `_install_wheel_router` / `wheelOwn` / `清空日志` / `显示级别` / `CardMax` / `⛶ 放大` → `main_window`，`已停止` → `panel_record` + `recorder`，`导出当前视图` / `满屏 %g°` / `scope_view` → `panel_abz` + `panel_wave` + `export_png`，`ui_prefs` → `main_window` + `panel_abz` + `panel_wave`，`Ctrl+滚轮` → `ui_kit` +两面板，以及新增的 `冻结时参数文件落在 exe 旁边` / `_MEI` 命中 `ui_prefs` 模块自身。
- **沙箱内烟测再次确认不可能**（复现踩坑 4）：`Start-Process host\dist\host_app.exe --smoke … --seconds 2` 既不退也不产图（引导器模态错误框挂着），只能 `Stop-Process` 清理；⇒ 烟测/双击/连板必须在沙箱外做。
- **文档同步**：`host/README.md` 的事实 14（补"exe 版落在 exe 旁边"）、文件表 `ui_prefs.py` 行（13 → 14 条 + 冻结说明）、「进度」S6 行换成第三轮数字与核对结论。
- **文档同步**：`host/README.md` 新增整节 **「## 打包成单文件 exe（S6）」**：打包/验证命令、**六个坑**（① `SPECPATH` 已是目录；② `tools/` 必须进 `hiddenimports`；③ 单文件下 `__file__` 是临时解包目录，点名 `host/link.py::_ensure_tools_on_path`、`host/recorder.py::_load_tools`、`host/export_png.py::_writable_dir`、`host/selftest_ui.py::_tmp_dir` 四处；④ `matplotlib.use("QtAgg")` 要显式声明后端；⑤ GUI exe 不等待；⑥ `.ps1` 里不能出现中文文件名）+ **DSH 沙箱里 exe 起不来的完整根因与规避** + "exe 是产物不入库、脚本会打印大小/时间/SHA256"。
- **验证状态（务必如实）**：✅ 打包链路完整跑通（环境自检 → 分析 → PYZ → PKG → EXE → `Build complete!`，两次成功，135/143 s）；✅ 源码模式未被打包改动破坏（`python host\selftest_ui.py` = 通过 30 项 / 失败 0 项，`$LASTEXITCODE=0`）；⏳ **真机验证（S6 验收要求：目标机双击可用 / 连板收帧 / 记录 / 导出 PNG）尚未做 —— 这个沙箱做不了，必须在沙箱外终端做。**
- **待办**：① `host/README.md:254` 附近的 S6 进度行仍写 `待做`，要等真机验证通过后连同实测数字一起改；② 真机验证通过后把本节的产物大小/SHA256/验证结论补齐；③ 提交 `host/host_exe.spec`、`tools/build_host_exe.ps1`、`host/README.md`、4 个 host 模块的冻结路径改动、本文件。

> ★★★★★ **现行位流 = v12**（2026-10-13 定版；**已于 2026-10-13 12:29 烧入并读回 `0x05F2 = 0x000C`，实机验证通过**，见 §1.59 末条）：生产位流 `_synth_work/enc_top.bit` **1,042,208 B @2026-10-12 23:59:27**、SHA256 `D93DEBF92034FF2329117CDE59EB83E0D95620306E7A944C6B0402E8C3AE7725`；ILA 调试位流 `_synth_work/enc_top_ila.bit` **1,428,064 B @2026-10-13 07:12:11**、SHA256 `5CC779EE6D00FFA60AD371F9A87F82A35E74C8B6E477F45323B475A299BAEC52`（`enc_top_ila.ltx` 71,337 B @07:12:00）。 **v12 = v11 + §1.58 的 UART 遥测帧布局 v2**（`rtl/uart_scope.v` 端口 `amp_sq[31:0]` → `io_state[7:0]` / `io_flags[7:0]`、`rtl/enc_top.v` 例化处按位拼接）+ `RTL_BUILD_ID` → `16'h000C`；角度/ABZ/UVW/BiSS 行为与 v11 **完全一致。 ★ 两版时序都收敛**：生产版 `_synth_work/impl_timing.rpt` @23:59:16 = `All user specified timing constraints are met.`（`sys_clk` **WNS +0.959 ns / TNS 0.000 / 0 个失败端点（共 11,275）**、WHS +0.075、WPWS 9.500）；ILA 版 `impl_ila_timing.rpt` @07:11:57 = 同一结论行（**WNS +0.332 / 0 失败 / 共 20,165**、WHS +0.050、WPWS 8.870）。资源（生产版 `impl_util.rpt` @23:59:14）：Slice LUT **6298**（30.28%）/ Slice Registers **4074**（9.79%）/ Block RAM **0** / DSP **16** / Bonded IOB 50 / BUFGCTRL 1；`impl_drc.rpt` @23:59:18 = **0 Errors。 板上判据（已实测通过）**：**2026-10-13 12:29 烧入 `_synth_work/enc_top.bit` 后读回 `0x05F2 = 0x000C`**（v11 为 `0x000B`）、遥测**偏移 9 = `0xA2`**（v2 帧标记）；`0x0108 = 0x4004`、`0x0109 = 0x0040` 与版本号无关（v11/v12 相同）。 ✅ **板上已确认是 v12**：三次实机抓帧（含 30 s、等效采样 64/64 箱）与逐条寄存器读回见 **§1.59 末条。** ⚠ 位流体积受压缩率影响极大（v12 1,042,208 B 比 v11 1,086,600 B **小**，是同 RTL 下布线/压缩率变化所致，**不是资源变少**）⇒ 位流字节数**不能当回归判据。** ⚠ **对照用位流**（拍摄日 §D.6 的"修复前"镜头）：`_synth_work/enc_top_v9.bit`（0x0009）、`_synth_work/enc_top_v10prod.bit`（0x000A）、`_synth_work/enc_top_ila_v9.bit`，以及本轮新归档的 `_synth_work/enc_top_v11prod.bit` / `enc_top_ila_v11.bit`（0x000B）；**v11 生产版回下命令见 §1.49 的 `restore`。**
>
> ★★★★★ **【历史，已被 v12 取代】板上曾是 v11**（生产位流 `_synth_work/enc_top.bit` **1,086,600 B @2026-10-12 21:15:37**，SHA256 `FA1308647D776F78E2497DF44E92F0694274956780033 80A8D78D40F5B7C2594`；该文件已归档为 `_synth_work/enc_top_v11prod.bit`；ILA 调试位流 `_synth_work/enc_top_ila.bit` **1,458,744 B @21:29:54**，SHA256 `5E049FD3B3EAB51C79D62BEA35709D6CB922820064B316FE1C644DCB97B3210B`，已归档为 `enc_top_ila_v11.bit`）。 **v11 = v10 的采样相位**（`rtl/enc_top.v` 两路 ODDR 由 `D1=0/D2=1` 改为 `D1=1/D2=0` ⇒ `ad_clk` 与 `clk` 同相）+ **配套修正的 ADC 输入约束**（`constr/enc_top.xdc` 两处 `create_generated_clock` **去掉 `-invert`**；`set_input_delay` 由 **24.000/19.500** 改为 **14.000/9.500**）+ `RTL_BUILD_ID` → `16'h000B`（**commit `991c001` 对 `rtl/` 的唯一改动就是这一行）。** 完整论证（相位判别实验 v10、热点 v9/v11 双向夹逼、70 °C 热态复测、ILA 逐拍现场）见 **§1.57 ⑧ A–I。 ★ 当时时序首次收敛**：`_synth_work/impl_timing.rpt` @21:15:26 的结论行 = `All user specified timing constraints are met.`（`sys_clk` **WNS +0.821 ns / TNS 0.000 / 0 个失败端点（共 11,310）**、WHS +0.058、WPWS 9.500）⇒ 2026-10-10 起那 **26 个 ADC 输入端口负余量已随相位修正整体消失**，不再是"诚实保留的残留"。 当时板上读回校验：`0x05F2` = **0x000B**、`0x0108 = 0x4004`、`0x0109 = 0x0040`。 当时资源（v11 生产版 `impl_util.rpt` @21:15:25）：Slice LUT **6369**（30.62%）/ Slice Registers **4088**（9.83%）/ Block RAM **0** / DSP **16** / Bonded IOB 50 / BUFGCTRL 1；DRC **43 checks**（DPIP-1×11 + DPOP-1×16 + DPOP-2×16）全 Warning、**0 error / 0 critical。** ⚠ 该轮位流体积反常（1,086,600 B 比 v10 生产版 781,716 B **大**，同 RTL 新相位下布线更不规整所致，**不是资源变多**）。
>
> ★★★★★ **【历史，已被 v11 取代】板上曾是 v9**（`_synth_work/enc_top.bit` **1,226,648 B**，SHA256 `2F51DCD332178B56EC4124F87AC8D3317BD1819398574 44CAD0E1CA776124A43`，2026-10-11 **20:43:01 定版**（同一轮实现：`impl_util.rpt` 20:42:48 / `impl_timing.rpt` 20:42:50 / `impl_drc.rpt` 20:42:52 / `impl_build_v9.log` 20:43:01），**21:04:26 经 JTAG 下载**（`_synth_work/flash_v9.log`；此前本节文字里的"21:03 重综合"与产物时间戳不符，已按文件时间戳订正）；已读回校验 `0x05F2` = 9 / `0x0302` = 0x1A / `0x0500` = 22 / `0x0501` = 10 / `0x0108` = 0x4004，见 §1.49）。 v9 = v8（§1.43 的 `GATE_STATE=1` + kp=2/ki=3 + §1.44 的 Z 脉宽截断修复 + §3 item 18 的 lag 阈值重标定）+ **§1.49 的 Z 宽度 `z_width + 1`**；v8 两版位流与全部上板证据已冻结在 `_synth_work/pre_v9/`（回退点）。 ⇒ **§2.0–§2.8 的 v6/v7 数字都是历史基线，不要再拿它们的位流当上板依据。 v7 已归档**（`_synth_work/pre_v8/`：生产位流 770,600 B、ILA 位流 1,164,160 B + `.ltx` + `impl_timing.rpt` WNS −3.364 / TNS −85.065 / 26 失败端点 + `ila_cap_v7free.csv` 自由运行窗口证据，见该目录 `_README.txt`）；**v8 的 ILA 版已重建**（`_synth_work/enc_top_ila.bit` 1,465,972 B、`.ltx` 71,337 B @19:33，Setup 26 失败 / WNS −3.323，见 §1.46）。 **★ 22:00 时板上装的是 v9 ILA 版位流**（`_synth_work/enc_top_ila.bit` 1,432,868 B @21:24:56，**21:41:21 烧录**，日志 `flash_v9_ila_sweep.log`）：Z 闭环（§1.49）、12/20 kHz 重扫（§1.48 补 2/补 3）与 D 分支取证（§1.51）都在它上面完成；其间 21:37:20 曾用 `flash_v9_prod_restore.log` 把**生产版 v9** 烧回板，随后为 sweep 又烧回 ILA 版。 要用生产版请按 §1.49 的 restore 命令重下 `_synth_work/enc_top.bit`。 ⚠ 板上 v8 位流是**未标定**口径（`0x0510-0x051A` 全复位、未跑标定，见 §1.45），与 §1.23/§2.8 里"标定生效"的数字**不能直接对比。** §2.0–§2.7 全部是历史基线，**不要再拿它们的位流当上板依据**（v6 那一版文件已被覆盖且未另存，见 §2.8 末尾）。
>
> ★★ **2026-10-10 重跑原因（务必读）**：`rtl/calib_ctrl.v` 在本轮查出并修掉了 **标定窗口 off-by-one**（§1.9），`rtl/uart_scope.v` 同段亦有改动，之后又修掉 **`rtl/enc_top.v` 的标定窗口脉冲接错**（§1.17）。 旧位流（10-09 23:17:45，741,836 B）**早于**这些 RTL 改动 ⇒ **旧位流不对应现在的 RTL，不能直接拿去上板。** 本轮已用修正后的 RTL 重跑生产版实现，新产物见下面 **§2.1 / §2.2**；下面 §2.0 的 10-09 数字保留为历史基线。 回归同步扩到 **18 台**（见 §1）。

✅ **ILA 版位流已在本轮用当前 RTL 重建（2026-10-10 16:12:01**，含 §1.18 幂等标定修复 + §1.20 探针位序修复 + §1.21 OTR 软清位 + §1.22 UART→SPI 桥）： `_synth_work/enc_top_ila.bit` **1,445,812 B**、`.ltx` 67,006 B @16:11:50、 `impl_ila_timing.rpt` @16:11:48、`impl_ila_util.rpt` @16:11:48、`impl_ila_drc.rpt` @16:11:50、 `ila_probe_list.txt` @16:10:41（表头已改 ASCII，见 §1.7）； 实测 `Timing constraints are not met.`（`impl_ila_timing.rpt:144`）—— `sys_clk` 域 `WNS -3.329 / TNS -84.745 / 26 个失败端点（全是 ADC 输入端口）/ 18010 端点`、 `WHS +0.050`、 `WPWS 8.870 / 0 失败 / 8939 端点`； 整体 `WNS -3.329 / TNS -84.745 / 26 失败 / 19161 端点`。 最差路径 `ad_ch1_d[10] → u_capture/ch1_meta_reg[10]/D`（Input Delay 24.000）。 ★ 这不是 ILA 引入的缺陷：16:12 版构建于 §1.19 的**诚实 ADC 输入约束**之后，与生产版 16:05 的 `WNS -3.352 / 10582 端点 / 26 失败` 同源；DRC 62 checks **0 error / 0 critical**（全为 Warning，逐条 `Related violations: <none>`）、 `Router Completed Successfully`、 探针仍是 **252 条** `MARK_DEBUG` **网 / 34 条端口 / 252 bit**， 资源 LUT 8822（42.41%）/ FF 8544（20.54%）/ BRAM 7.5（15.00%）/ DSP 17 / IOB 50、 压缩 `saved 5971264 bits` ⇒ `17537696-5971264 = 11566432 bit = 1,445,804 B`（+8 B 头部差）。 更早的 **15:14:21（1,435,240 B，含位序修复与 OTR 清位但早于 UART→SPI 桥）**已移入 `_synth_work/signoff_pre_uart_bridge/`； 08:56:25（1,423,056 B，早于位序修复，在 `_synth_work/signoff_ila_bitrev_old/`）、10-09 23:31:27（1,473,128 B）与 07:44:11（1,148,904 B） **全部作废。** ⚠️ 该脚本同样有"位流写完但脚本挂住（`ILA_EXIT=-1`）"的现象（见本节末尾说明）； **判据永远以产物为准**，本次已按产物核对完整。 **§3 第 9 条已更新：ILA 版位流已于 2026-10-10 13:39 首次上板**（当时用的是 08:56 版，因此波形窗把 `dbg_pole_pairs_eff` 显示成 8）； 16:12 版的核验判据是显示 **4**、可用 `0x0405=0x0001` 清 OTR、 并可用 `tools/uart_spi.py read 0x05F0`（应回 `5A 01 F0 10 0C B7`）验证 UART→SPI 桥。

✅ **★★ 2026-10-11 00:39:22 再次重建 ILA 版（= 当前这一版，含 §1.28/§1.29/§1.30 全部三处标定修复）**： `_synth_work/enc_top_ila.bit` **1,482,840 B @00:39:22**、`.ltx` **67,006 B @00:38:31**、 `ila_probe_list.txt` @00:32:39（34 端口 / 252 bit）、 `impl_ila_{timing,util,drc}.rpt` @00:38。 探针集**与 16:12 版完全一致**（同名同序同宽度， `.ltx` 连字节数都相同 67,006 B）⇒ 现有触发设置与 CSV 读法继续适用。实测 `Timing constraints are not met.`： `WNS -3.298 / TNS -84.420 / 28 失败端点 / WHS +0.038 / THS 0.000`（端点总数 19285/19269 = ILA 版口径）； 最差路径 `ad_ch1_d[9] → u_capture/ch1_meta_reg[9]/D`（同 §1.19 的 ADC 输入端口族， `Input Delay 24.000`）。 资源：LUT **8939**(42.98%) / FF **8579**(20.62%) / BRAM **7.5**(15.00%) / DSP **17** / IOB 50 / BUFGCTRL 2； DRC **62 checks**（DPIP-1×24 / DPOP-1×17 / DPOP-2×17，全 Warning）**0 error / 0 critical。** 压缩 `saved 5675040 bits` ⇒ 17,537,696 − 5,675,040 = 11,862,656 bit = **1,482,832 B**，实测 **1,482,840 B** （**+8 B** 头部差 ⇒ 与 §2.1/§2.2/§2.3/§2.4/§2.5 同口径，自校验吻合）。 ★ 本次**首次用** `danger-full-access` **+ 后台作业跑完全程**： `write_bitstream` 段 `8 Infos, 0 Warnings, 0 Critical Warnings and 0 Errors`，末尾正常打印 `ILA_BITSTREAM_WRITTEN` / `ILA_PROBES_WRITTEN` / `ILA_CHECK_DONE`， **退出码 0 —— 没有再出现历史上的** `ILA_EXIT=-1` **挂尾。** 16:12:01 那版（1,445,812 B）与它的 `.ltx`/探针清单已备份到 `_synth_work/pre_ila_v4/`，留作历史基线。 ⚠️ **两个位流必须成对区分**：生产版 = `_synth_work/enc_top.bit`（**v6 见 §2.7**，不含 `dbg_hub`）， 调试版 = `_synth_work/enc_top_ila.bit`（见下条）；两者 RTL 同源，探针只在后者里。

✅ **2026-10-11 12:27:05 又一次重建 ILA 版（含 §1.32 的 5c 诊断计数器，探针索引已变）**： `_synth_work/enc_top_ila.bit` **1,409,344 B @12:27:05**、`.ltx` **71,337 B @12:26:56**、 `ila_probe_list.txt` @12:25:53（`probe_ports = 36, total_bits = 269`，比 00:39 那版的 34 端口 / 252 bit 正好多 **+2 端口 / +17 bit =** `dbg_ang_delta` **(16) +** `dbg_ang_glitch` **(1)** ⇒ 与 §1.32 新增的逻辑吻合）： `probe2 dbg_ang_delta_15_0_(16)`、 `probe3 dbg_ang_glitch(1)`、 `probe5 dbg_angle_raw_15_0_(16)`、 `probe4 dbg_angle_15_0_(16)` ⇒ **探针索引与旧版不同，必须按名字找映射，不要再背旧索引。** 资源 LUT **9104**(43.77%) / FF **8775**(21.09%) / BRAM 7.5(15%) / DSP 17 / IOB 50 / BUFGCTRL 2； 时序仍是 `Timing constraints are not met.`（WNS −3.319 / WHS ≈ +0.067，同 §1.19 的 ADC 输入端口族）； DRC 62 checks 全 Warning、0 violation。压缩 `saved 6263008 bits` ⇒ 17,537,696 − 6,263,008 = 11,274,688 bit = **1,409,336 B**，实测 1,409,344 B（**+8 B** 头部差，同口径）。 ⚠️ 该脚本"位流写完但脚本挂住"的老现象这次**又出现了**（产物已全部落盘、父进程卡在子 vivado 管道收尾） ⇒ 已按产物核对完整后 `job_kill`； **判据永远是产物。** 10-11 00:39 那版（1,482,840 B）与其 `.ltx` 已备份到 `_synth_work/pre_ila_v5/`； 更早的 16:12 版在 `_synth_work/pre_ila_v4/`。

✅ **★★ 2026-10-11 14:06:09 用当前（v6 定版）RTL 重建 ILA 版 —— 这一版才是 v6 定版 ILA 版**（用户 m00838 第 ② 项）： **★ 已被 v7 ILA 取代**（2026-10-11 16:22:42， `1,164,160 B`、 `WNS -3.338 / 26 失败端点 / 20,158 端点`， 探针仍是 36 端口 / 269 bit —— 见 **§2.8** 的 ILA 表；本节数字保留为 v6 时代的基线）： `_synth_work/enc_top_ila.bit` **1,429,272 B @14:06:09**、`.ltx` **71,337 B @14:05:58**、 `ila_probe_list.txt` @14:04:48（日志 `ILA: connected 36 named probe ports, total 269 bits` —— 与 12:27 版**完全一致** ⇒ 探针集未变，12:27 那条里的按名字映射继续适用）、 `impl_ila_{timing,util,drc}.rpt` @14:05:56–14:05:58。 实测 `Timing constraints are not met.`： **WNS −3.368 / TNS −84.979 / 26 失败端点**（端点总数 20155/20139； `WHS +0.048 / THS 0.000`，**无保持失败**）。**★ 失败端点数与生产版（−3.330 / 26 / 11253）、与 12:27 版 （−3.319 / 26）完全相同 ⇒ 就是 §1.19 那 26 个 ADC 输入端口，v6 的 RTL 改动没有新增时序失败**； ILA 版相对生产版只多 −0.038 ns WNS（插 `dbg_hub` 的代价），与历次口径一致。 资源 LUT **9189**(44.18%) / FF **8942**(21.50%) / BRAM **7.5**(15%) / DSP **17**。 压缩 `saved 6103584 bits` ⇒ 17,537,696 − 6,103,584 = 11,434,112 bit = **1,429,264 B**，实测 **1,429,272 B** （**+8 B** 头部差，与 §2.1–§2.7 同口径 ⇒ 位流有效） 命令： `powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 -Tcl tools\ila_debug.tcl -LogFile _synth_work\ila_build_v6.log -Tail 80`（一次性 `danger-full-access`，后台作业跑）。 ⚠️ "位流写完但父进程挂住"的老现象**又出现**（日志已打印 `INFO: [Vivado 12-1842] Bitgen Completed Successfully.`， 位流/`.ltx`/三份报告全部落盘且自校验吻合）⇒ 按产物核对完整后 `job_kill`； **判据永远是产物。 ★ 与上面 12:27:05 那版的关系（务必分清）**： 12:27 版**已经包含** §1.32 的探针（+2 端口 / +17 bit）， 但它**早于** `rtl/calib_ctrl.v`（**12:37:53**）与 `rtl/reg_file.v`（**12:38:40**）的 v6 收尾改动 ⇒ **它不是 v6 定版 ILA 版**；两版位流字节数不同（1,409,344 → 1,429,272 B）、资源差 **+85 LUT / +167 FF** （9104/8775 → 9189/8942）**证实这中间的 RTL 确实变了 ⇒ 重建有意义**（与 §2.7 里"v6 新增寄存器"的量级吻合）。 12:27 版已归档到 `_synth_work/pre_ila_v6/`（含 `_README.txt` 说明），可与定版对照或回退。 ⇒ **§2.7 ⑥ 与 §3 第 14 条清单里的"v6 的 ILA 版尚未建"已由本条收口**；剩下的是把它**上板**（下载 + 加载 `.ltx` + 按名字找探针 + 触发 `dbg_ang_glitch == 1`）。

⑧ ★★★ 热点上的相位对照给出决定性结论，并据此采纳 v10 相位 + 修正配套约束（v11，2026-10-12 20:40–21:20）

背景：⑦ 的幅度轴证据（0 → 37.8 次/s 的极陡开关）说明"热点"是被测点的一个真实工作点， 但此前每次做相位对照都要碰运气等热点，而**拧旋钮本身会造伪像**（第一次命中的 A1 窗 4780 个事件就是这么废掉的）。

A. 锁定一个稳定热点（新工具 `tools/sweep_rate.py`， `--seconds 600 --step 5 --csv _synth_work/sweep3.csv`**）** - 用户把 AFG 幅度调到显示 **8.03 Vpp / 20 kHz** 后不再触碰；记录（120 行，21:01:04–21:10:59）显示率**自行爬升**： 21:01:19 起 4.6 → 9 次/s，21:02–21:06 稳定 **12–18 次/s**，21:06:49 后 **25–30 次/s**，21:10 段 **30–55 次/s**； 10 分钟累计 `gl_1024` **5011** 个， `gl_2048` / `gl_1024` ≈ **13%**（⇒ 13% 属 ④ 的 ≥11.25° 大尖峰，其余是 5.6–11.25° 小尖峰）。 - ⇒ **热点不依赖拧旋钮**，是固定档位下可持续的工作点（与 10-11 的 13.65 次/s、10-12 下午的 37.8 次/s 平台同量级）。

B. 热点上的 v9 / v10 / v9 相位对照（ `_synth_work/hot_ab.ps1`**， 30 s 窗，全程无人触碰 AFG；日志** `_synth_work/hot_ab.txt`**）**

窗	板上位流（读回 `0x05F2`）	`gl_1024` Δ	率	`gl_2048` Δ	`0x0405` 粘滞位
A1_v9 ≈21:07:42–21:08:13	`_synth_work/enc_top_v9.bit`（0x0009）	**907**	**30.2 次/s**	93	0x21 = otr+amp
B_v10 ≈21:08:35–21:09:06	`_synth_work/enc_top.bit`（0x000A）	**0**	**0**	0	0x0000（无）
A2_v9 ≈21:09:28–21:09:59	回到 `enc_top_v9.bit`（0x0009）	**681**	**22.7 次/s**	146	0x25 = otr+laglim+amp

每窗同时用 `tools/uart_scope.py --seconds 1` 记录遥测半径：A1 **26248.6** / B **25882.1** / A2 **25781.1** （AFG 仍在缓慢漂移，约 −1.8%/2.5 min）； `cyc_pulse_cnt` Δ ≈ 10,950/30 s ⇒ f_in 始终 20 kHz。

C. 四条独立证据 ⇒ 结论 1. **B 夹在两个高它三个数量级的 v9 窗之间**；若是时间/幅度漂移，A2 不可能恢复到 22.7 次/s。 2. 同时段独立运行的 5 s 记录器显示：v10 期间计数器连续 ≥45 s 恒为复位值（`sweep3.csv` 21:08:34–21:09:14 三行）。 3. 换回 v9 后**立即**恢复（21:09:19 起 5 s 内 17 个事件）⇒ 不是热漂移。 4. 附带：v9 两窗的 `otr`（超量程）粘滞位被置位、v10 窗全清 ⇒ 与 ④ 的"坏码造成假超量程"一致。 ⇒ **ADC→FPGA 的采样相位就是 D 分支毛刺的根因，v10 相位是修复；热点上抑制 >600×。** ⑥ 里"效果量只有上界（v10 < 0.005 次/s）"的保留意见就此**撤销。**

D. 落地：采纳 v10 相位 + 修正配套约束（本轮 RTL 版本号升为 v11） - `constr/enc_top.xdc`：① 两处 `create_generated_clock -name adc_clk/adc_clk2` **去掉** `-invert`（同相生成时钟， 发射沿由 +10 ns 变 0）； ② `set_input_delay -clock sys_clk` 由 **24.000/19.500** 改为 **14.000/9.500**（推导：3.762 + 0 + 3.016 + tPD 2..6 + 排线 1 ns）； ③ 文件头 ODDR 片段与说明同步改成 `D1=1/D2=0`。 - `rtl/enc_top.v`： `RTL_BUILD_ID` 由 `16'h000A` 升为 **`16'h000B`**（板上可读回区分"v10 相位 + 旧约束"与"v11 定版"）。 - **★ 实测：时序首次收敛** —— 2026-10-12 21:15 用上面改过的 XDC/RTL 重建生产版（命令 `powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 -Tcl tools\impl_check.tcl -LogFile _synth_work\build_v11prod.log -Tail 30`）：`_synth_work/impl_timing.rpt` @21:15:26 的 Design Timing Summary = **WNS +0.821 ns / TNS 0.000 / 0 个失败端点（共 11,310）**、 WHS +0.058 / WPWS 9.500 / 0 失败 ⇒ 报告结论行**首次**变为 **`All user specified timing constraints are met.`**（对比 10-10 12:08 的 WNS −3.271 / TNS −84.284 / **26 个失败端点** —— 那 26 条正是被旧相位与 24 ns 输入延迟"如实"描述的边界状态）。 位流 `_synth_work/enc_top.bit` = **1,086,600 B @21:15:37**、SHA256 **`FA1308647D776F78E2497DF44E92F069427495678003380A8D78D40F5B7C2594`**（日志 `Bitstream compression saved 8844960 bits` ⇒ 17,537,696 − 8,844,960 = 8,692,736 bit = 1,086,592 B + 8 B 头部 = 实测 1,086,600 B ✓）。 ⚠ 位流比 v10（781,716 B）**大**：只因为新相位下布线更不规整、压缩率变低，**不是**资源变多（资源量级同前）。 ⚠ "位流写完但父进程挂住"的老现象**再次出现**（Bitgen 已打印成功、位流 + 三份报告全部落盘）⇒ 按产物核对完整后 `job_kill`。 - v10 生产版已另存 `_synth_work/enc_top_v10prod.bit`； v11 上板复测结果见下方"上板收口"条。

E. 遗留 - 热点上"按需抓现场"的 ILA 流程：`tools/ila_capture.tcl` 的 `stage1` 只负责下位流，必须再 `write 0x0302` 布防、 然后 `ILA_MODE=capture`； 本次直接 `capture` 的两窗（`_synth_work/ila_hot_v9_{a,b}.csv`）里没有毛刺 —— 与概率相符（1024 点 = 20.48 µs，× 30 次/s ≈ 0.0006）。 - 示波器实测 tPD_meas（§1.19 清单第 1 条）仍未做；v11 上板后应在热点复测（预期 0）。

F. ★ 夹逼复核（2026-10-12 21:19:39–21:23:30，`_synth_work/bracket_v11.ps1`，日志 `_synth_work/bracket_v11.txt`）

为堵住"时间/幅度漂移让率自然归零"这最后一个漏洞，做了 v11 → v9 → v11 的**反向夹逼**（每窗 60 s，同一环境、同一 AFG 档位 8.03 Vpp / 20 kHz、无人触碰）：

窗	位流	60 s `gl_1024` Δ	率	`gl_2048` Δ	`0x0405` 粘滞	校正后半径
W1	v11（手中位流）	**0**（3→3）	**0 次/s**	0	0x0000（无）	25,653.5
W2	v9 生产版（下回）	**3,320**（565→3,885）	**55.3 次/s**	450	0x0025 = otr+laglim+amp	26,078.0
W3	v11（下回，板留在 v11）	**0**（3→3）	**0 次/s**	0	0x0024 = laglim+amp（**无 otr**）	26,196.2

- **★ 幅度在这 4 分钟里单调上升**（25,653.5 → 26,078.0 → 26,196.2，+2.1%），而 W2 的 v9 夹在**中间**反而打出 3,320 个毛刺、两侧的 v11 都是 **0** ⇒ 漂移若真能让率归零，只会让 W3 比 W2 更差，实测相反 ⇒ **幅度/时间漂移被彻底排除，唯一在变的自变量就是位流。**
- ★ 抑制倍数：热点上 **>3,320×**（受 60 s 窗分辨率所限，只能给下界）； 与 30 s 窗 A/B/A 的 907 → 0 → 681（>600×）、冷点 1200 s 的 10 → 0（$p = 6.7\times10^{-3}$）三个尺度互相印证。
- **★ `otr`（超量程粘滞位）只在 v9 窗出现**（W2 = 0x0025），v11 两窗一次都没有（0x0000 / 0x0024 = 仅 laglim+amp） ⇒ 与"捕获侧坏码造成假超量程"一致（角速度 `laglim` 在热工作点上本来就会置位，属正常环路行为）。
- 上板收口： `_synth_work/verify_v11.ps1`（提权）日志 `_synth_work/verify_v11.txt` = 换版前 v9 30 s 窗 **868 个 = 28.9 次/s**、换 v11 后 **120 s 窗 `gl_1024` 恒为 3（Δ=0）**， `0x05F2` 由 0x0009 → **0x000B** ✓；两次测量 f_in 都 ≈20.0–20.1 kHz（`cyc_pulse_cnt` 按 16 位回绕折算）⇒ 修复没有改变频率通路。
- **结论（D 分支闭环）**：根因 = **ADC→FPGA 捕获接口裕量不足**（v9 反相相位只剩 1.2–5.2 ns）； 修复 = **v11**（同相采样时钟 + 配套 XDC），已在同一热点上以 v9/v11/v9 双向夹逼验证；板卡当前停在 **v11**。

G. ★★★★★ D 分支机制定案：ILA 抓到坏样本的逐拍现场**（2026-10-12 21:35–21:40）**

条件：板上 = **v9 ILA 位流**（`_synth_work/enc_top_ila_v9.bit` + `.ltx`），工作点 = 热点 **8.03 Vpp / 20 kHz**（该点 v9 生产版率 ≈68 次/s），ILA 深度 1024、触发挂 `ang_glitch`、触发点固定在第 512 拍。 数据：`_synth_work/ila_cmp_v9_glitch.csv`（另有未触发的对照窗 `_synth_work/ila_cmp_v9_warmup.csv`）。

观测	实测值	依据
坏样本（原始域）	`s_raw` 第 **486** 拍 = **+768**，两侧 485 = −1537、487 = −1536 ⇒ **单拍 +2305 LSB ≈ 半量程（符号级翻转）**	CSV 第 485–487 拍
同拍另一通道	`c_raw` = −512（两侧 −517 / −509）⇒ **只有 s 通道坏**	同上
全窗唯一性	原始码一阶差分 `|Δ| > 40 LSB` **只在 485→486 与 486→487 出现**	1024 拍全扫描
探针→角度流水线延迟	**26 拍**（坏样本拍 486 → 角度异常/触发拍 512；两端各自落在自己的 4 拍更新节拍上）	`_synth_work/align_check.py`（事件直接对齐）。窗口整体最小二乘对齐给 27–29 拍，受斜率/偏置退化影响，**不作为结论**
角度域后果	`angle_raw` 第 511 拍 −19,837 → **第 512–515 拍 −21,484**（恰好 4 拍 = 一个角度更新周期）→ 第 516 拍 −19,632 ⇒ **−1647 LSB = 9.05°**，与 `ang_glitch` 触发/计数同拍	CSV 第 511–516 拍
角度更新节拍	**每 4 拍一次**： `ang_delta` 非零只出现在 `i ≡ 0 (mod 4)`（0, 4, 8, …, 512, …，本窗 259 个非零）⇒ 坏值被保持 **4 拍 = 80 ns**（50 MHz 采样、抽取 4）	`_synth_work/align_check.py`
抽取发布规律	`dec_valid` 每 4 拍一次（相位 `i mod 4 == 2`： 2, 6, 10, …, 486, …）； **第 i 拍发布的 `s_dec` = 16 × mean(s_raw[i−5 … i−2])**	两处独立验证：i=486 发布 mean(481–484) = −1528 ⇒ −24,448 ✓；i=490 发布 mean(485–488) = −961 ⇒ −15,376 ✓
抽取域后果	坏样本 486 落在 **i=490 发布的窗口（485–488）** ⇒ `s_dec` = **−15,376**（若 486 正常应 ≈ −24,592）⇒ **+9,216 LSB（= 2305/4 × 16，均值只稀释 4×）并保持一个抽取周期（4 拍 = 80 ns）**	CSV 第 490–493 拍

- **★ 与 10-10/10-11 抓到的"单拍、只改高位、抽取输出被替换"形态逐位吻合**： −24,592 = 0x9FF0 → −15,376 = 0xC3F0（16 位补码下高 4 位 0x9→0xC）⇒ 早期记录并没有记错，**坏样本的源头就是原始域的单拍错锁**，不是角度运算自身的问题。

- ★ **完整链条（端到端闭环；同一个源头，两条支路）**：采样相位裕量不足（v9 ≈1.2–5.2 ns）→ 单个 ADC 样本被错锁（s 通道、+2305 LSB）。此后分成两条： ① **原始角度支路**（`angle_raw` = 用**未校正原始码**算的角度，即 `gl_1024` / `gl_2048` / `otr` 监控的那一支）：坏样本经 **26 拍**流水线到达角度输出，落在一次 **4 拍更新**上 ⇒ **9.05° 跳变保持 80 ns** ⇒ 计数器 +1、`otr` 置假超量程。 ② **抽取支路**（喂环路/输出的那一支）：坏样本落在某一次 4 点均值的窗口内 ⇒ 抽取输出偏 **+576 原始 LSB（×16 后 +9,216）**、保持一个抽取周期（80 ns）⇒ 这是**唯一可能漏进输出角**的路径，也正是"检测-保持"防线要压的对象（实测外泄 <0.92°）。 两条支路的**外显条件都取决于坏样本落在哪一拍/哪个格点** ⇒ 这正是"事件相位锁在 256 格点、率对幅度尖锐敏感"的来源。 → **v11**（同相采样时钟 + 配套 XDC，14–18 ns 裕量）去掉源头后，同一热点上率降到 **0**（见下一节 H）。
- ⚠ **两条硬约束（写给后续任何 ILA 工作）**： 1. **任何"原始码 ↔ 角度"对照必须减去 26 拍**（且注意角度每 4 拍才更新一次）。本轮第一次按 0 拍对齐时曾误判为"原始码干净、角度域自己跳"，减 26 拍后立刻看到 +768 的坏样本。 2. **4 点均值不构成抑制**（只稀释 4×）。若要再加一层与相位无关的保险，应在 `rtl/decim_avg.v` **之前**对 s/c 各插入 3 点中值 —— 单拍离群会被中值直接剔除（§1.57⑤(d) 的候选缓解，未实施，列为 **v13 备选** —— `v12` 号已被 §1.58 的遥测帧布局 v2 占用；有了本次现场数据，这一条从"可能有用"变成"有明确靶点"）。

H. ★★★ 同一热点上的 v11 对照（2026-10-12 21:44–21:47，`_synth_work/capture_v11_clean.ps1`，日志 `_synth_work/capture_v11_summary.txt`）

板上换成 **v11 ILA 版**（`_synth_work/enc_top_ila.bit` 1,458,744 B @21:29:54、SHA256 `5E049FD3B3EAB51C79D62BEA35709D6CB922820064B316FE1C644DCB97B3210B`，与 v9 现场**同一个 AFG 档位、同一个热点、全程无人触碰**）：

测项	v9（现场）	v11（对照）
换版前/后的 30 s 计数窗	1,978 个 / 30 s（≈66 次/s）	**0**（换版前 30 s = 0、换 v11 ILA 后 30 s = 0）
ILA 立即触发窗（1024 拍，触发挂 `dbg_adc_valid`）	——	`dbg_ang_glitch` **命中 0 拍**；原始码一阶差分 **无任何 |Δ| > 40 LSB** 的样本；raw→角度重构残差落在 ~30 LSB 的量化底噪上（v9 同口径：1 个命中、+2305 LSB 的坏样本、9° 跳变）
触发改挂 `dbg_ang_glitch`、上限 60 s	立即命中（本窗命中 1 拍）	`fired=0 elapsed=60,114 ms` **⇒ 60 s 一次都没触发、无 CSV 产出**
收尾	——	`restore` 回 v11 生产版，读回 `0x05F2 = 0x000B` ✓

- ★ 这是"修复"最强的单条证据：**在 v9 每 30 s 出近 2000 个毛刺的同一工作点上，v11 连触发条件都凑不出来**（60 s 上限空转退出），且原始码层面找不到一个离群样本。
- 图：`deliverables/figures/ila_glitch_v9_vs_v11.png`（左 = v9 坏样本现场、右 = v11 同一热点；三行分别原始码 / `s_dec` 与 `angle_raw` / `ang_glitch` 标志），由 `tools/compare_glitch_figure.py` 生成。

I. ★★ 温度复测：热态夹逼 v11 → v9 → v11（2026-10-12 21:59–22:15，AFG 全程 8.03 Vpp / 20 kHz 未动，记录器 `tools/sweep_rate.py`）

方法：没有温度计 ⇒ 用**同一吹法、同一热态**下的相对比较代替绝对温度（用户定性档："能按住 1–2 秒"≈55–63 °C，本次吹到"很烫"≈**70 °C 估计**）。吹风机低档、风口 ≥20 cm、来回移动；每窗之间提权换位流并核 `0x05F2`。

时刻（窗）	位流	温度	`gl_1024` 计数	率	`gl_2048` 占比	遥测半径	CSV
21:59:47–22:00:12	v9	常温	3,993 → 5,133	39–48 次/s	2.6%	25,507	`temp_v9_amb.csv`
22:00:30–22:03:25（180 s）	v9	加热中 → "很烫"	6,014 → 15,164	**48 → 59 次/s**	**3.2% → 6.1%**	25.5k–26.3k	`temp_v9_heat.csv`
22:03:38–22:06:33（180 s）	v9	停风后仍热	15,934 → 25,911	47 → **64 次/s**	**6.4% → 8.9%**	25.7k–26.3k	`temp_v9_cool.csv`
22:07:45–22:10:10（150 s）	**v11**	同吹法（用户确认已加热）	3 → 3（**Δ=0**）	**0.000 次/s**	**0%**	25.5k–26.3k	`temp_v11_heat.csv`
22:10:43–22:12:38（120 s）	**v9**	"很烫"≈70 °C	6 → 3,338	**14.2 → 50.0 次/s**	**≈30%**	25.7k–26.3k	`temp_v9_hotb.csv`
22:13:11–22:15:06（120 s）	**v11**	"很烫"（收口）	3 → 3（**Δ=0**）	**0.000 次/s**	**0%**	25.7k–26.3k	`temp_v11_hotb.csv`

- **判读**：热态夹逼 `v11热(0，150 s) → v9热(14→50 次/s，120 s) → v11热(0，120 s)` ⇒ 同一"很烫"热态下 **v11 两窗共 270 s 绝对 0**，v9 同态 15–64 次/s。即 v11 的余量**热到 70 °C 量级仍未被吃穿。**
- **温度的指纹是"占比"而不是"总率"**：`gl_2048/gl_1024` 从常温 ~3% 涨到最热窗的 **30%**（总率只涨十几 %）。机理一致 —— 温度升 ⇒ `tPD` 升 ⇒ 余量变小 ⇒ 坏样本振幅变大，事件由 5.6–11.25° 小尖峰**迁向** ≥11.25° 大尖峰。全部窗遥测半径 25.5k–26.3k 正常抖动、无单调漂移 ⇒ **幅度未漂，率的变化不是 AFG 幅度漂移伪像。**
- ⚠ **为什么不用"冷却窗"作证据**：22:03:38–22:06:33 停风后率与占比**继续上升**（47→64 次/s、6.4%→8.9%），无法区分 ① 整板热惯性（PCB 比芯片热 ⇒ 结温滞后）与 ② 工作点自漂移（同一档位 21:01–21:10 无人碰旋钮时率也自己从 4.6 爬到 30–55 次/s）。∴ 判据只能用"热态下换位流"，不能用"加热前 vs 加热后"。
- **R2 剩余（器件批次）**：无第二块 AD9238 模块 ⇒ 本次**未做**，报告如实标注。
- 操作与记录表：`docs/09_scope_measure_log.md` §3（含防烫/防碰排线注意事项）。
- ⚠ 工具侧修复：`tools/sweep_rate.py` 原来读子进程输出用系统默认 GBK，子进程吐 UTF-8 时会崩（`UnicodeDecodeError: 'gbk' codec can't decode byte 0x8e in position 8: illegal multibyte sequence`，21:59 首次跑就撞上）⇒ 两处 `subprocess.run` 已加 `encoding="utf-8", errors="replace"`。

1.68 ★★★★★ 输入健康度复测收口（毛刺率 0 / `crcerr` 根因 = `biss_sl` 悬空）+ 李萨如 P1-1 素材落地（2026-10-13）

- **触发**：用户 **"开始做 1、2"**（m00600）——1 = 清输入健康度 `amp_low` 并排查 `crcerr`（收口 §3 item 15 的前置条件）；2 = 演示视频分镜与 P0 素材。**RTL 未改**；本节全部是板上实测 + 新增工具 + 素材归档。

A. 设备条件与环境陷阱（决定"哪些结论可信"）

部件	设定 / 事实
AFG-2225	Waveform → **F2 正弦**；`UTIL` → `CH1` → `LoadPhase` → **`High Z`**（★ 按 50 Ω 设会让幅度翻倍）；`FREQ` **1 kHz**；`AMPL` **8.0 Vpp**；`DC OFFSET` **0 Vdc**；`OUTPUT` 开、CH1 → 载板 `J?`（sin/cos 进 ADC）
示波器	探 `J5-7`（Z）与 `J5-11`（MA），1 kHz 下确认 Z 每电周期一次、MA 电平
串口	**COM3 @ 2 Mbaud**（端口号随插拔变：`python tools\uart_spi.py ports`）

部件	设定 / 事实
板上位流	**v12 生产版**：`0x05F2` 读回 **`0x000C`**（02:xx 与 03:xx 两次都读到 12）
★ 沙箱陷阱	上一轮遗留的 `_exe_rt_LEFTOVER_sandbox_MEI_delete_me\`、`_rt_LEFTOVER_sandbox_MEI_delete_me\` 里的 `_MEI*` 目录**删不掉**（`Remove-Item` / `icacls /reset` / `rmdir /s /q` 全部 `Access is denied`，重命名父目录可以成功）——根因同 §1.67 的引导器显式安全描述符问题。**需用户在资源管理器里手工删。**

B. 结论 ①：`amp_low` 前置条件满足，§3 item 15 的 10^4 量级读数作废

- **`0x053A` 全清**（读回 **`0x0000`**：`amp_low` = 0、`crcerr` = 0、`otr` = 0、`calib_busy` = 0、`frame_active` = 0）。AFG 真正 8 Vpp 进 sin/cos，`stat_amp_sq` 幅度比落在复位门限带 0.60–1.20 内。
- **`0x0505`（`gl_cnt_1024`）连续监控窗增量 = 0**：两段独立窗口（总计约 **36 s**）内 `Δgl_cnt_1024 = 0` ⇒ **毛刺率 ≈ 0 次/s**。对照历史：输入退化时同一寄存器给出 **10^4 量级/s**；输入健康时 §1.37 测得 **0.00444 次/周期 ≈ 4.4 次/s** ⇒ **10^4 量级那批读数确实只是"输入不健康"的伪像，不能当真实毛刺率**（这正是 §3 item 15 当时的诚实限定）。
- ⚠ **必须区分"求差读数"与"累计计数"**（本轮自己踩到的坑，写给下一次）：`tools/calib_repeat.py` 结束时会**顺带打印**一行累计量 —— 实测 `>5.6°=22`、`>11.25°=54431`、`>22.5°=26319`、`>45°=15786`、保持动作 `=14726`、周期脉冲累计 `=38174`，并算出"每输入周期 >5.6° 0.0006 次"。**这是上电以来的累计事件数，不是当前毛刺率**（计数器无饱和、溢出回绕）⇒ 它既不能证明"有毛刺"，也不能否定 B 段的求差结论。★ 判据只能来自"已知短间隔的两次读数求差"，与 §3 item 15 的原口径一致。

C. 结论 ②：`crcerr` 根因 = `biss_sl` 悬空（未接地），已在板上复现并消掉

- `crcerr`（`0x053A` bit6）是本轮开始时唯一还亮着的问题位。按 `docs/08_bissc_dumb_host.md` §1 的引脚表复查接线，发现 **`biss_sl`（FPGA `M20` = `J5-12`）没有接 GND**。
- 接上 GND 后：`crcerr` 清零、`0x053A = 0x0000`；随后 **30 s 窗口内 `crcerr` 全程 0**。
- **v12 上的 A 探针复测（已归档）**：`tools/biss_frame_probe.py --port COM3 --probe A --n 200 --interval 0.002 --csv deliverables/data/biss_probeA_v12.csv` ⇒ **A1–A4 全 PASS**：`frame_cnt` **min 0 / max 41**、**distinct 39**、命中 `=41` **3/200**（帧位递增的锯齿完整看到了）、`frame_active` **198/200**（仅 2 次为 0 = 帧间停顿，属正常）、**`crcerr` 0/200**；`RTL_BUILD_ID(0x05F2)` = **12**；耗时 7.93 s。统计另存 `deliverables/data/biss_probeA_v12_summary.csv`；原始 CSV SHA256 `B301A5EF2ACC4F581F9B7ABB507F5CD1A35F42AF4B52E4D1BCA300E300803CE9`（3,412 B）。另有一次同样通过的 200 点快跑（未归档）：`distinct 41 / max 41 / crcerr 0 / frame_active 196/200`。
- **机理与手册一致**：`biss_sl` 悬空 = 非 0 电平 ⇒ 破坏从站状态机 ⇒ `crc_err_seen` 粘滞；`docs/08` 的"方案 C（`sl` 悬空）"反例正是这条。带 AFG 时动到 `biss_ma` / `biss_sl` 排线是它的诱因。
- ⚠ **两条诚实限定**：① `crcerr` 是**粘滞**位（置位后不自动清），所以 `0/200` 只证明"本窗口内未再置位"，不等于"历史从未错过"；② 探针 A 只覆盖"正常接线"这一路，**探针 B/C（故意违例：`ma` 10 MHz / `sl` 悬空）在 v12 上仍未复测** ⇒ 这仍是 `deliverables/photo_shot_list.md` 的 **P0-5 剩余缺口**（B 必须放最后做：它会把 `crcerr` 粘滞置 1，污染后续 bit6 读数）。

D. 李萨如 P1-1 素材落地：把"斜椭圆 → 正圆"变成可复现（不用动 AFG 档位）

- **关键发现**：`rtl/enc_top.v:366-371` 的校正级**始终**使用 `cfg_*` 寄存器（`use_ofs_s = cfg_ofs_s` …），一键标定只是把解算结果写回这些寄存器（写口见 `rtl/reg_file.v:327-340`，地址同 `0x0510…0x051A`）⇒ **往 `0x0510…0x051A` 写值就能人为注入畸变**，"斜椭圆→正圆"全程可由串口复现，不需要动 AFG 的偏置/相位档位。
- **新增 `tools/calib_snapshot.py`**（236 行）：`snapshot`（读 `0x0502/0x0510/0x0512/0x0514/0x0516/0x0518/0x051A/0x051B` 存 JSON）/ `restore`（写回并回读校验）/ `inject`（`--offset`、`--offset-c`、`--gain-mismatch`、`--quad-deg`，`phase_k = round(-32768·sin Δφ)`，并把 shift 归零避免粗调移位放大 2×/4×）。★ 系数口径见 `rtl/sc_correct.v:6-29`：`gain` 是 Q1.15（1.0 = 32768）、`phase_k = sin(Δφ)`、`ofs_*` 是直接减到 ADC 码上的 LSB 单位。
- **改 `tools/lissajous_capture.py`**：`--data` 收到的是**文件名**而非目录时，旧代码会静默 `makedirs` 出一个同名**目录**（summary 与逐点 CSV 全被塞进这个伪目录）⇒ 现在退回其父目录并显式告警（`resolve_data_dir`）。
- **★★ 一次"结果不可信"的自我更正（工具真 bug，已修）**：第一次注入实验采到的圆度是 **17.829 %**，但复测时同一注入变成 **10.056 %**、半径 mean 44,212（贴近 46,341 上限）、`oflow` 满窗 —— 于是逐个核对寄存器，抓到 `tools/calib_snapshot.py` 两个真 bug：1. **`0x051A` 只有 4 位**（`rtl/reg_file.v:104 = [1:0]shift_s [3:2]shift_c`，读回见 `rtl/reg_file.v:507 = {4'b0000, r_shift_c, r_shift_s}`）⇒ 写 `0x00FF` 读回 `0x000F`、写 `0x00F3` 读回 `0x0006`，而 `uart_spi.do_write` 拿"回读 == 写入的全 16 位"判校验 ⇒ 对 `0x051A` 一律误报 `!! 未通过回读校验`。2. **`do_write` 的哨兵路**（旧值 == 目标时先写哨兵 `0x00FF` 验证地址通路）在 `0x051A` 上就是**事故**：本意"把 shift 归零"，实际写下 `0x00FF` ⇒ `shift_s = shift_c = 3` ⇒ **粗调 ×8**，把校正后半径顶到 44k、`oflow` / `scamp` 满窗。所以那次 17.829 % 的量是**被 ×8 污染的读数**（`0x051A` 实测读到 `0x000F` 就是铁证）。⇒ **修复**（`write_pairs` 重写）：`0x051A` 按 4 位裁剪后再比（位宽截断不再误报）；**跳过"目标值 == 当前值"的写**（从根上避开哨兵路）；`gain_reg()` 增加 **1.999 上界**硬拒绝（`rtl/sc_correct.v:9,68-73` 的定点要求，本工具把 shift 写 0，无法靠粗调移位补救）。修复后 `restore` / `inject` 都是 **失败 0 条**。
- **实测（干净复测，COM3、v12、1 kHz / 8 Vpp，三次一致）**：

阶段	校正后半径 mean	圆度偏差 RMS	状态位触发
健康（原系数 `ofs_s 41 / ofs_c 357 / gain_s 33014 / gain_c 33309 / phase_k 97 / shift 0`）	**26,214.4**	**0.101 %**	`OK(no-alarm)`（正/负步进 `49,767 / 0`）
注入后（`--offset 3000 --offset-c -3000 --gain-mismatch 1.25 --quad-deg 3`，写入 6 条失败 0 条）	26,773.9	**17.786 %**	`oflow=4978, scamp=4975`
一键标定后（写 `0x0502 = 0x0001`）	**26,213.8**	**0.101 %**	`OK(no-alarm)`

⇒ **17.786 % → 0.101 %，改善约 176×**；收敛系数 `ofs_s=42 / ofs_c=359 / gain_s=33013 / gain_c=33310 / phase_k=97 / shift=0`（与注入前原值同量级、逐项正确）。另一次独立复测：注入后 **17.795 %**、标定后 **0.102 %**；`lissajous_capture` 那次 **17.852 % → 0.146 %**。**失真检出后的半径峰峰 17,594.5–33,496.6**（健康态 26,094–26,318）也如实记在 `deliverables/data/lissajous_summary.csv`。 - **标定系数快照/复原已逐值验证**：实验前原系数 `ofs_s=41`（恢复后曾读到 42/44，属标定解算的正常抖动）/ `ofs_c=357` / `gain_s=33014` / `gain_c=33309` / `phase_k=97` / `shift=0`，`restore` 后逐值复原且 `失败 0 条`（★ 每轮注入实验结束**必须** restore，否则板上带的是被注入的系数，后续所有素材/测量作废）。 - **一键标定重复性复测（`tools/calib_repeat.py --port COM3 --n 12 --csv deliverables\data\calib_repeat_summary.csv`）**：**12/12 全 GOOD**；`OFS_S` min 40 / max 42 / mean 41.2 / 极差 2 / 标准差 0.6（判据 `GOOD_OFS_S=400`）；`OFS_C` 357–364；`GAIN_S` 33,011–33,016、`GAIN_C` 33,307–33,312；`PHASE_K` 97–98；`SHIFT` 0；`retry` **0 次**；`win_bad` **0 次**；`secs` 0.0–0.45；★ **工具默认端口**

是 COM5，必须显式传 `--port`。未解释项（如实记录，不当作故障）：CSV 的 `trunc`（`0x0502` bit3）**12/12 轮都是 1**、`stat`（`0x053A` 高字节）第 9/12 轮为 1 —— 工具本身给出的判据是"`OFS_S` 每轮收敛 ⇒ GOOD"，该语义下 12/12 通过；`trunc` 是粘滞状态位，本次**未单独定案**（`secs` 同时为 0 说明轮次判定走的是"读到 `done` 即停"的路径，没有实测到 busy 期）。判据以工具verdict为准。（工具有内置 `--selftest`，本次未跑。） - **归档（`deliverables/`，全部为本次干净复测的产物）**： - 图：`figures/lissajous_before_distortion.png`（106,991 B，SHA256 `32F9398F31EF1C71…`）、`lissajous_after_distortion.png`（114,257 B，`DEF7ECDB6F6DA190…`）、`lissajous_before.png`（108,225 B，`7B1288352A7008CD…`）、`lissajous_after.png`（114,342 B，`BB1C5A63980861D2…`） - 数：`data/lissajous_before_distortion.csv`（4,241,395 B，`3398F4F1F74B0459…`）、`lissajous_after_distortion.csv`（4,153,320 B，`F663B80EB5AA9FCE…`）、`lissajous_before.csv`（4,174,509 B，`85B37517996CD246…`）、`lissajous_after.csv`（4,157,602 B，`CE3AF847625CEBDE…`）、`lissajous_summary.csv`（794 B，`E5675DF3F14540F6…`）、`calib_repeat_summary.csv`（606 B，`166048418AFDD049…`） - **★ 文件名到语义的映射（`lissajous_capture.py` 的命名是"抓图时刻的屏上状态"）**：`*_distortion` = **板上带注入失真时**抓的（`before` = 未标定、`after` = 一键标定后）；无后缀 = **恢复健康系数后**抓的（两组都健康 ⇒ `before 0.101 %` / `after 0.101 %`，这组只作"健康基线"用）。 - **★ 两点诚实提醒**：① 更早归档的 `lissajous_before.png` 是**输入退化为椭圆时**抓的，构图与"斜椭圆"相近、**值得保留**，但它不是本轮注入实验的产物；**视频第 3 段建议拍注入版**（`P1-1b`：17.8 % → 0.101 % 的视觉反差远大于普通标定）。② P0-4 的"Z 脉宽示波器带游标截图"、P2 的板卡/接线照片、P2-6 真实传感头片段**都还没拍**（需人工 + 相机/示波器动作）。

- **本条对应的素材清单改动**：`deliverables/photo_shot_list.md` 的 P1-1 行改为 🟢，并新增 **P1-1b**（注入失真 → 一键标定，推荐作视频第 3 段）、**P1-1c**（一键标定重复性）；D.2 新增 **1-7** 步（四条命令 + "必须 restore" + `0x051A` 位宽陷阱的提醒）。

1.69 ★★★ 频率阶梯 P1-3 起步（1 kHz 归档点）+ 抓到一个误读寄存器的真 bug：`0x0534` 不是 `io_flags`（2026-10-13）

- **触发**：用户 **"先做这个 B/C 探针，然后做这个频率阶梯 P1-3"**（m01213），随后因不便动板子，改为 **"先去跑频率阶梯"**（m01228）⇒ 本节只做**不需要改接线/改 AFG** 的那一部分。**RTL 未改。**

A. ★ 真 bug：`0x0534` = `stat_cycle_cnt[7:0]`，不是 `io_flags`（已修 `tools/input_health.py`）

- 现象：`tools/input_health.py` 一直把 `0x0534` 打印成 `io_flags` 并解码成告警位，于是本轮先后"读出"过 `lag_fatal`（`0x0026`）、`uvw_freq_warn + lag_warn`（`0x0009`）等**根本不存在的告警。**
- 铁证：连读 18 次，`0x0534` 值是**规律锯齿** `0x0039 → 0x0033 → … → 0x0003 → 0x003D …`，每轮约 1 s 一个周期 —— 那不是标志位，那是**在跑的周期计数。**
- 源头核对：`rtl/reg_file.v:522-523` 明确写 `16'h0534: rdata = stat_cycle_cnt[7:0]; / 16'h0535: … [15:8]`；`io_flags` **在整个可回读寄存器地图里根本不存在**（`rtl/reg_file.v:481-547` 全部命中已逐条核对），它只活在**遥测帧**里（`rtl/uart_scope.v:16-36` 帧偏移 10、`rtl/enc_top.v:1164-1166` 组装，位序 `{amp_bad, cor_sat, uwven, abzen, uvw_fw, zwin, lagf, lagw}`）。
- 用正确来源（`tools/uart_scope.py --csv` 的 `io_flags` 列）复测：3 s / **29,864 帧**里只有两种取值 —— **`0x30`（`cfg_abzen + cfg_uwven`）29,396 帧**、**`0x34`（再加 `z_window`）468 帧** ⇒ **`lag_warn` / `lag_fatal` 从来没置过**，之前那几次"lag 告警"全部是误读。
- 修复：`tools/input_health.py` 把该行改为 **`0x0534` 周期计数低字节** 的正确标签，并在注释里写清"要看 `io_flags` 请用 `uart_scope.py --csv`"；寄存器表名 `io_flags` → `stat_cycle_cnt_lo`。`--selftest` 22 项仍全 PASS。
- **教训**：本工具一路把"地址对应的名字"硬编码在表里，**表错一次就会连续多轮报出假告警。**凡是"从寄存器读出告警位"的结论，都应当与 `rtl/reg_file.v` 的地址行逐条对照后再写进文档。

B. 频率阶梯 P1-3 的 1 kHz 归档点（★ 后续各点同表追加）

- **AFG 现值确认**：`stat_amp_sq` 幅度比 **0.8005**（低阈 0.6000 / 高阈 1.2000，`amp_low` / `amp_high` 均未置位）⇒ 幅度在位；`cyc_pulse_cnt` 的 Δ 与秒数吻合 ⇒ **AFG 确实在 1 kHz**（`0x0540` Δ = 10,038 / 10.0 s）。
- **`0x0101 INTER_L` 当前 = `0x0020`（inter = 32，不是 256）** —— 是之前拍 P2-7（ABZ/UVW 遥测）按清单要求设下、一直没改回来的。对本任务反而是好消息：过速钳位约束 `inter × f_in / div ≤ 6.25 MHz`、`div` = 4（`0x0102` = 0）⇒ **inter = 32 时 f_in 上限 ≈ 195 kHz**，比 inter = 256 的 24.4 kHz 宽 8 倍。**50k / 100k 两点预计无需改寄存器；200k / 500k 需要把 inter 再降到 16 / 8**（写入前先记下原值，跑完复原）。
- **采样点（15 s 窗，`--no-wait`）**：`tools/freq_ladder.py --port COM3 --freqs 1k --seconds 15 --no-wait --with-diag --out deliverables\data --figures deliverables\figures` ⇒ **半径 mean 26,214.4 / min 26,092.5 / max 26,337.7、圆度偏差 RMS 0.101 %、`OK(no-alarm)`、有效帧 149,363（≈9,957 帧/s）、`crc_err` 571（链路固有 ~0.38 %，见 §1.58）**。`diag_counters` 同窗：窗口 `int_min=12,501 / int_max=12,503 / 比值 1.0002 正常`，`wcnt=4 / nsamp=50,008`（自洽）；**四档角度跳变计数 Δ 全 0**（毛刺率 0，与 §1.68 结论一致）；`RTL_BUILD_ID` = 12。
- **归档**：`deliverables/figures/freq_1k.png`（111,819 B，SHA256 `1D2F0C47226724AD…`）、`deliverables/data/freq_1k.csv`（**15000 行摘录**，1,247,689 B，`304E122B2B3CAA33…`；全量 149,363 行 / 12,571,691 B / `756523806D4502DB…` 留在 `.gitignore` 的 `_synth_work\freq_1k_full.csv`，不入库）、`deliverables/data/freq_ladder_summary.csv`（465 B，`D3E5364FC7D052AE…`，后续各点追加同一文件）。
- ⚠ **诚实限定**：① 本点是**在 AFG 频率正好是 1 kHz 的巧合下**拿到的，**不是"我把 AFG 设成 1 kHz 再采"** —— 阶梯的其余各点（10k/50k/100k/200k/500k）**必须在用户逐点改 AFG 时才能采**；② `lag_deg`（环路滞后）列本次为**空**：`tools/ila_report.py` 的滞后值来自 ILA 抓包，而 `--ila` 需要下 ILA 位流（要放宽权限）并且现场并非 ILA 版，故 1 kHz 这一点只有半径/圆度/状态，**频率-误差-延迟表的"延迟"列仍缺**（对应 `deliverables/photo_shot_list.md` 的 P1-4）；③ 摘录 CSV 只是"落盘体积控制"，全量数据的统计值以上面一行数字为准。

Open objectives: 频率阶梯 P1-3 其余 5 点（10k/50k/100k/200k/500k，需用户改 AFG）；BiSS 探针 B/C 在 v12 上复测（P0-5 剩余）；演示视频分镜与 P0 素材（m00600 的 "2"）；S6 单文件 exe 通过目标机（沙箱外）验证（m00051）。

1.70 ★★★★ 上位机界面优化批次 1 落地：全界面滚轮顺畅滚动 + 记录「随动随止」+ 通信日志降噪（2026-10-13，界面自检 30 → 42 项）

- **触发**：用户自查完成后提出 6 条界面优化/修复需求（m06024），逐条确认后整理成 `docs/12_host_ui_requirements.md`（R1–R6）；用户拍板「批次 1 → 2 → 3」，本条 = **批次 1（R1 滚轮 + R4 记录定格 + R5 日志降噪）**。硬约束：不新增第三方依赖、界面全中文、离线自检与真机 S1–S5 保持全绿、`export_png.py` 的 1950×1080 / 1950×990 像素契约不变。
- **R4 记录卡片「随动随止」**（`host/panel_record.py`）：原来「校验错帧数」显示的是**实时链路计数器**（主窗口每 0.5 s `set_crc()` 喂一次），与记录是否在跑无关 ⇒ 停止后还在涨。改为 `_frozen` + `_freeze(st)`（记录停止瞬间把**时长/行数/文件大小/丢帧估计/校验错帧数/帧率**六项一起打包）+ `_show_frozen()`；`stop_record()` 之后 `tick()` 走冻结分支（状态灯「已停止」），`start_record()` 清空冻结并重置 `_crc_base` ⇒ **再开记录从 0 重算。**
- **R1 全界面滚轮**（`host/main_window.py`）：新增 `_install_wheel_router()`（`QApplication.installEventFilter`）+ `eventFilter()` / `_wheel_is_ours()`（`self.isAncestorOf(obj)`）/ `_wheel_owner()`（沿 `parentWidget()` 找 `wheelOwn` 白名单）/ `_wheel_zoom_target()`（为批次 3 预留 `wheelZoom`）/ `_route_wheel()` / `_scroll_page()`；`WHEEL_PX_PER_UNIT = 0.6`（1/8 度 → 像素）。**除白名单（日志框）外，任何位置的滚轮都转成页面的像素级滚动**，下拉框/数字框不再被滚轮误改值（改值走键盘/点击）。`closeEvent()` 里 `removeEventFilter`。

- **R5 通信日志降噪**（`host/main_window.py`）：新增 `LOG_RANK`（fault 3 / warn 2 / ok 1 / info 1 / debug 0）、`LOG_COLOR`、`LOG_FILTERS`（全部（含调试）/ 信息及以上 / 警告及以上 / 只看故障）、`LOG_FILTER_DEFAULT = "info"`；`log_line(text, level="info")` 保留最近 `LOG_MAX_LINES = 600` 条记录，按级别过滤后再 `_append_log()`（warn/fault 上色）并刷新「显示 N / 共 M 行」；日志卡片新增「显示级别」下拉 + 「清空日志」按钮。每 2 s 的告警寄存器轮询改为按 `_status_signature()`（ok / val0404 / latch0405 / flags053a / flags053b）**只在变化时以「信息」打一条**，没变化降为「调试」不可见 —— ★ 签名**故意不含幅值字段**（`amp_low_sq` / `amp_high_sq` / `stat_amp_sq` 每 2 s 都在小幅波动），否则等于没降噪。
- **验收（离线界面自检 30 → 42 项，全 PASS exit 0）**：新增 ⑪ 记录定格（停止后人为把 CRC 从 5 涨到 905、`tick()` 三次，六项读数一字不变；状态灯「已停止」；重新开表后校验错=0、帧数从 0 重算）、⑫ 日志级别（状态没变时轮询行不可见，可见 1 → 1 条；切「全部（含调试）」变 2 条；切回「信息及以上」回 1 条；清空按钮生效）、⑬ 滚轮（端口下拉框上滚轮 ⇒ 页面 0 → 72 px 且选中项不变、数字框 `inter` 64 → 64 不变、日志框上滚的是日志自己且页面 144 → 144）。
- **★ 踩坑（已修，写进 docs/12 供批次 2/3 参考）**：滚轮事件在 Qt 里是发给**滚动区视口**（`QPlainTextEdit.viewport()`）的，发给控件本体不会滚动它自己 —— 界面自检第一次跑出 41 PASS / 1 FAIL（`日志滚动条 0、页面 144 → 144`），改发视口后 42/42。
- **回归证据**：`python host\selftest_ui.py` **42/42 PASS（exit 0）**；离线模块自检全绿（encoder 10、abz 15、export_png 11、recorder 13、signal 22、regs 2043 组 + 5 用例）；`python -m compileall host` exit 0；真机 `selftest_ui.py --real-port COM3 --seconds 6` **PASS exit 0**（取帧→落盘 60,927 = 60,927 差 0、文件行数 64,616 = 记录器 rows 差 0、关窗后队列 1 → 0 条）；真机截图核对 `python host\app.py --smoke host\_b1_ui.png --port COM3 --seconds 8 --height 1800` exit 0（1166×2456，日志卡片新增「显示级别」下拉 + 「清空日志」按钮 + 右侧计数；该 PNG 为一次性核对用，看完已删）。
- **文档**：`docs/12_host_ui_requirements.md` 勾掉 R1/R4/R5 并新增「批次 1 实施记录」表；`host/README.md` 自检条数 `selftest_ui.py` 30 → **42 项**、日志级别说明、记录随动随止说明、新增第 12 条「滚轮」用法。

Open objectives: 界面优化**批次 2（R6 每张卡片可放大到全软件界面）已完成，见 §1.72；批次 3（R2 输入波形的示波器式可调 + R3 ABZ/UVW 等效采样的示波器式可调，含 Ctrl+滚轮缩放、参数记忆 `host/ui_prefs.json`、「按当前视图导出」按钮）仍未做**（m06024）。

1.71 ★★★ 频率阶梯 P1-3 第二个点（10 kHz）+ 查清遥测 `amp_sq` 的标度与"坏帧污染 max/status"（2026-10-13）

- **触发**：用户把 AFG CH1+CH2 都改成 10 kHz 后报 **"CH1+CH2 都改成 10 kHz了"**（m01429）。**RTL 未改**；本节只加采集与统计工具。
- **改频前的口径确认（回答用户"两通道是否必须同步改"）**：必须同步改。板上 sin/cos 是同一激励的 90° 正交两路，两通道频率不一致 ⇒ 李萨如变斜、`stat_amp_sq` 掉出 0.60–1.20 门限带 ⇒ 该点数据作废。AFG-2225 有双通道耦合菜单（`UTIL → Dual Chan`，可对频率/幅度/相位做耦合，见 AFG-2225 系列手册 · Coupling、AFG-2225 英文手册 PDF），但**无法确认"改 CH1 是否自动跟 CH2"**，故仍要求逐点核对 CH2 显示与板上幅度比。
- **★ 独立测频（不采信 AFG 旋钮）**：脚本不读 AFG，唯一能"发现用户改没改"的手段是用板上 `cyc_pulse_cnt`（`0x0540`）反测 —— 1 kHz 时 Δ = 5,006 / 5.01 s ⇒ **998.85 Hz**；10 kHz 时同一手法得 **9,990.8 Hz** ⇒ 改频生效。（当 `--seconds` 太短而 Δ 跨过 65,536 回绕时，工具会把回绕差当秒率，例如报出 "200 Hz" 的假值 —— 短窗一律用回绕感知的差法。）
- **采点（15 s 窗，`--no-wait`）**：`tools/freq_ladder.py --port COM3 --freqs 10k --seconds 15 --no-wait --with-diag --out deliverables\data --figures deliverables\figures` ⇒ 有效帧 **149,416**（≈9,961 帧/s）、`crc_err` **500（≈0.33 %，链路固有）**、工具报 `半径 mean=26222.6 min=26093.1 max=32357.1 rms=0.125% 状态="inclim=1, laglim=1, res=1"`。
- **改频瞬态（不是缺陷）**：改频后首次读数 `0x053A` = **0x0018** = `caldone + otr`，`0x0404` = `0x2D01`、`0x0405` = `0x002D`（`scamp + laglim + otr`）。写 `0x0405 = 0x0001`（W1C）后 `0x053A` 回 **0x0010**、`0x0404` = `0x2D00`，随后 4.8 s 内 OTR **不再复现** ⇒ 改频瞬间的瞬态过幅，非持续。`stat_amp_sq` 幅度比 **0.8004**（健康带内）、`RTL_BUILD_ID` = 12、`0x0101` = 0x0020（inter = 32）。
- **★ 关键技术发现（跨全部频率点通用）**：遥测帧的 `amp_sq` 就是**半径的平方**，即 `半径 = sqrt(amp_sq)`（单位与同帧 `(s, c)` 相同）。
- 实证：全量 149,416 帧的 `sqrt(amp_sq)` 中位 = **26,220.9**，与 `uart_scope.py` 按 `hypot(s, c)`（`tools/uart_scope.py:271 r = np.sqrt(s*s + c*c)`）报的 mean **26,222.6** 吻合到 **0.007 %**；逐帧 `|hypot(s,c) − sqrt(amp_sq)| / sqrt(amp_sq)` 中位 **0.0000 %**、p99 **0.0000 %**。
- ⚠ 我先后误试过两个标度并已在实测中排除：`/2**30` ⇒ 0.8（差约 3 万倍）、`/1000` ⇒ 829（差 31.6 倍）。**旧文档/注释里把遥测 `amp_sq` 标成 "Q2.30" 是不成立的**，只有 `rtl/amp_monitor.v` 内部的门限比较用 Q2.30。
- **★ 真数据问题：链路坏帧会污染 `max` 与 `status`。**10 kHz 全量 149,416 帧里有**恰好 1 帧**异常（`idx` = 41376）：该帧 `s` / `c` 被破坏 ⇒ `hypot(s,c)` = 32,357（把工具报的 `radius_max` 拉飞）、`sqrt(amp_sq)` = 52,163.7（偏离中位 ≥99 %），且它的 `stat` / `io_flags` / `limiter` 三个字段**同时为空**（`stat` 读到 0x8c = 140 是错位高字节）——而工具报的 `status = "inclim=1, laglim=1, res=1"` 恰好就是 0x8c 按 `tools/uart_scope.py:97` 的 `STAT_BITS` 位表解出来的三位（bit3 `inclim` + bit2 `laglim` + bit7 `res`）⇒ **那条状态告警完全来自这 1 个坏帧，板上并没有告警。**故障机理与 §1.58 的链路固有 XOR 校验错（≈0.38 %）同源：个别坏帧侥幸过检。1 kHz 那一点 15,000 行摘录里 **0 个**坏帧。 ⇒ **归档口径从此固定**：一律同时给"中位/p0.1–p99.9 分位数"与"坏帧数"，**不允许只留 `radius_max`**。
- **新工具 `tools/robust_radius.py`**（本节新建）：对 `freq_*.csv` 做稳健统计并落盘 `deliverables/data/freq_robust_summary.csv`（字段 `file / frames / median / p0.1% / p1% / p99% / p99.9% / min / max / bad_frames / dev_median_pct`）；判据 `DEV_TOL = 0.02`（偏离中位 >2 % 记坏帧）。用法 `python tools\robust_radius.py <csv>... [--csv out.csv]`。**不改 `freq_ladder.py`**（避免动不可测的硬件路径），把稳健口径放在事后分析里。
- **归档数字（基于 15,000 行摘录，`freq_robust_summary.csv`）**：

点	稳健半径 中位 / min / max	圆度（`uart_scope` RMS / 本文分位口径）	坏帧	XOR 失败
1 kHz	26,213.9 / 26,093.9 / 26,321.1	0.101 % / 0.627 %	0 / 15,000	571（0.38 %）
10 kHz	**26,215.2 / 26,141.2 / 26,282.4**	**0.125 % / 0.430 %**	**0 / 15,000**	500（0.33 %）

（`uart_scope` RMS 用 `std(r)/mean(r)`，在全量帧上算；"本文分位口径" = `(p99.9 − p0.1) / 中位`，在 15,000 行摘录上算，对坏帧免疫。两个口径都列出来是为了不让读者把"换了个算法"误读成"改进了"。）

- **归档产物**：`deliverables/data/freq_10k.csv`（**15,000 行摘录**，1,278,779 B，SHA256 `EC4B3A18204ECF0E4B7A532F2051A7F335673E534002ED6E24FAEB4C26C00CF5`；全量 149,417 行 / 12,631,178 B / `EF4D6CED4B98868ED3AEE07A62B77D092C9F044CFCB120D7F4FA0034DAE0E4E5` 移入 `.gitignore` 的 `_synth_work/freq_10k_full.csv`）、`deliverables/figures/freq_10k.png`（112,673 B）、`deliverables/data/freq_robust_summary.csv`、`deliverables/data/freq_ladder_summary.csv`（追加 10k 行；★ 该行的 `radius_max` / `status` 是被坏帧污染的**原始口径**，引用时必须连同本节说明一起用）。
- ⚠ **诚实限定**：① 10 kHz 的"AFG 频率"是用户手工改的，脚本读不到 AFG，只有板上反测值（9,990.8 Hz）背书；② **`lag_deg`（环路滞后）仍为空** —— 它只能由 ILA 抓窗 + `tools/ila_report.py` 提取，而下 ILA 位流需放宽权限（沙箱内做不了）⇒ P1-3 那张"频率–误差–延迟"表的**延迟列仍缺**；③ 50k/100k 预计无需改寄存器，**200k/500k 需先把 `0x0101` 从 32 降到 16/8** 再采、跑完复原；④ 圆度这两个点（0.215 % vs 0.313 %）的差异**不构成趋势**（口径、窗口、坏帧剔除方式都还在变），要等全部 6 点同窗同口径复算后才能写进报告。

Open objectives: 频率阶梯 P1-3 其余 3 点（50k/100k/200k/500k，需用户改 AFG；200k/500k 还需临时降 inter）； `lag_deg` 列仍需 ILA 位流（沙箱外或有权限时补）； BiSS 探针 B/C 在 v12 上复测（P0-5 剩余）； 演示视频分镜与 P0 素材（m00600 的 “2” ）； S6 单文件 exe 通过目标机（沙箱外）验证（m00051）。

1.73 ★★★★ 频率阶梯 P1-3 第三个点（50 kHz）+ 查清 `stat` bit3 `inclim` 的真实语义（不是输入过速）（2026-10-13）

- **触发**：用户把 AFG CH1+CH2 改成 50 kHz 后报「已改成 50 kHz，开始采」（ask_user_question id=`next_freq`）。**RTL 未改。**
- **改频后独立测频**：`0x0540 cyc_pulse_cnt` 0.5 s 窗 Δ = 25,163 ⇒ **f_in = 49,988.3 Hz**； `stat_amp_sq` = 685,822,617 (0x28E0D299) ⇒ 幅度比 **0.7992**（健康带 0.60–1.20 内）； `0x0101` = `0x0020`（inter = 32，本点上限 ≈195 kHz，**无需降 inter**）； `0x05F2` = 12（v12 生产位流）。
- **改频瞬态（不是缺陷）**：读到时 `0x053A` = 0x0018 = `caldone + otr`、 `0x0404` = 0x2D09、 `0x0405` = 0x002D。写 `0x0405 = 0x0001`（W1C 清粘滞）后 `0x053A` 回 0x0010，随后 3.2 s 内只在 **0x0010 / 0x0090** 之间跳（**0x90 = `caldone` + 高字节来自 `0x053B frame_active`**，不是告警），OTR 不再复现 ⇒ 与 10 kHz 同类的改频瞬态。
- **采点**：`tools/freq_ladder.py --port COM3 --freqs 50k --seconds 15 --no-wait --with-diag --out deliverables\data --figures deliverables\figures`（日志 `_synth_work/_freq50k_run.txt`，exit 0） ⇒ 有效帧 **149,356**、 `crc_err` 355、工具报 `半径 mean=26203.3 rms=0.116% 状态=inclim=149356 滞后=-`。
- **★ 本节的主要结论：`stat` bit3 `inclim` 的语义是复合位 —— 它在 50 kHz 恒亮的是 `uvw_freq_warn`，不是输入过速。**
- `rtl/enc_top.v:798-807` 的 `stat_byte`： `bit3 inclim = (limiter_active | uvw_freq_warn)`（`rtl/enc_top.v:803`）。
- **`limiter_active` 实测全程为 0**：遥测帧 `io_state` 的 bit7 就是 `limiter_active`（`rtl/enc_top.v:1149` / `rtl/uart_scope.v:33`），50 kHz 全量 **149,356 帧里 bit7 置位帧数 = 0（0.00 %）**； `limiter` 列也全 0。
- ⇒ 恒亮只能来自 **`uvw_freq_warn`**（`rtl/uvw_gen.v`）。该位语义 = 「一个 **60° 电扇区**（电半周期）占用的 clk 拍数 < `UVW_MIN_HALF_CYCLE`」，即 UVW 基波 > 8.33 MHz： `rtl/uvw_gen.v:65` 门限 12 拍 @50 MHz、 `:92 MINHALF`、 `:514-515 warn_set = uwven && r_init && sec_changed && (r_dwell < MINHALF)` / `warn_clr = sec_changed && (r_dwell >= MINHALF)`、 `:595` 粘滞（要一个 ≥12 拍的扇区才清）。
- **报告/答辩必须写明**：50 kHz 点看到 `inclim` 时**不能**读成"输入过速被限频"；它是 UVW 输出的频率告警，且 `0x0534` 也不是 `io_flags`（勘误见 §1.69）。1 kHz / 10 kHz 的摘录里该位（遥测 `io_flags` bit3）分别在 **227 / 15,000** 与 **235 / 15,000** 帧出现（1.5 %），是同一位在低速下的偶发。
- **★ 用 Vivado xsim 直接复现该告警的通路（本节新增临时测试台，`_sim_work` 下，不入库）**：把 `rtl/uvw_gen.v` 按板上真实参数（`pairs = 4`、 `MINHALF = 12 @50 MHz`、 `uwven = 1`）单独跑，输入是"每 N 拍进一个角步长"的斜坡：

每采样步长	更新周期	warn 占比	最短 `r_dwell`
262 LSB	1000 拍	**0.00 %**	10000
5 LSB	1000 拍	**0.00 %**	65535
6586 LSB	1000 拍	**84.83 %**（上升沿 488 次）	1
5462 LSB	1000 拍	**19.25 %**	1
21846 LSB	1000 拍	**99.99 %**	1
8200 LSB	1000 拍	0.00 %	未切换（呆在扇区里不动）
10923 LSB	1000 拍	0.00 %	未切换（呆在扇区里不动）

- 结论 A：门限本身是**被真实触发的**——只要一个采样内跨过半/整个扇区（≥5462 LSB），`sec_changed` 就会在切换拍看到 `r_dwell < 12` ⇒ 告警置位。所以「`uvw_freq_warn` 会被点亮」不假。
- 结论 B：`pairs = 4`（2 对极）时 **50 kHz 机械输入对应的电频率只有 100 kHz**，60° 电扇区 ≈ 3333 拍 ≫ 12 ⇒ 正常斜坡**不可能**点亮它（表中 262 LSB 那行就是这个工况，实测 0.00 %）。
- ⇒ **机理仍未收口**：板上 50 kHz 恒亮说明 `uvw_gen` 看到的**有效电角速率远高于**"50 kHz 机械 × 4 极对数"的推算。怀疑方向：① `angle` 喂进 `uvw_gen` 的实际更新率/相位图与转速模型不一致；② 扇区边界的 ±1 LSB 抖动使 `sec_changed` 连发（`dut.sec_changed` 是组合量，会按拍翻转）； ③ 电角度映射（`rtl/uvw_gen.v:156-162 mult = angle * pairs_eff`、 `eang` 取低 AW 位）与本推算是另一套口径。**要收口需要 ILA 抓 `angle/eang_x/r_sec/r_dwell` 的连续拍序列**（沙箱内下不了 ILA 位流），或用真实采样的角度序列跑 replay 测试台。
- **归档（本节产物）**：
- `deliverables/data/freq_50k.csv` = **15,000 行摘录 / 1,328,942 B / SHA256** `67297FA185E50E58C681125F864B5EE642DC5F84090EC0C184E0400220D456A9`（全量 149,356 行 / 13,474,462 B / `8647A9C341EFA8A03C75139AC7BC71292E81CC580C57E665C9C26E7B8B55CFBB` 已移入 `.gitignore` 的 `_synth_work/freq_50k_full.csv`）
- `deliverables/figures/freq_50k.png`（128,999 B）
- `deliverables/data/freq_robust_summary.csv` 追加 50k 行（现 SHA256 `A01A7D61CB587BD356A3A6C1756F3F4DF26C6605B2C5E3DD5178208E9595329A`）

点	稳健半径 中位 / min / max	圆度（`uart_scope` RMS 全量 / 本文分位口径摘录）	坏帧	XOR 失败
1 kHz	26,213.9 / 26,093.9 / 26,321.1	0.101 % / 0.627 %	0 / 15,000	571（0.38 %）
10 kHz	26,215.2 / 26,141.2 / 26,282.4	0.125 % / 0.430 %	0 / 15,000	500（0.33 %）
50 kHz	**26,210.0 / 26,078.7 / 26,289.5**	**0.116 % / 0.620 %**	**0 / 15,000**	562（0.38 %）

⇒ 三点**同量级、无趋势**（圆度 0.43 %–0.63 % 的差别由摘录窗口与坏帧剔除方式主导），**角度数据在 50 kHz 下依然干净**，但**不能把这三点的差异当成性能趋势写进报告。** - ⚠ **诚实限定**：① 圆度仍是"每点一个 15 s 窗、摘录 15,000 行"的口径，**没做长时间统计**；② `lag_deg`（环路滞后）在 50 kHz 仍为空（需 ILA）； ③ `uvw_freq_warn` 恒亮的机理**未收口**（见上）； ④ 用户手工改频，脚本读不到 AFG，只有板上反测值背书。

1.74 ★★★ 频率阶梯 P1-3 第四个点（100 kHz）+ `0x0404/0x0405` W1C 语义实证 + `uvw_freq_warn` 的边界抖动复现（2026-10-13）

- **触发**：用户答 ask_user_question（id=`next_freq100k`）选「改到 100 kHz，我接着采」。**RTL 未改。**
- **改频后独立测频**（★ 100 kHz 下 `0x0540` 每 0.655 s 就回绕一次，**必须用 <0.2 s 的短窗**，否则算出差值）： 0.08 s 窗三次 ⇒ **100,046.3 / 100,058.1 / 100,051.1 Hz**（Δ = 8,302 / 8,316 / 8,349）。★ 反面教材：同一寄存器用 1 s 长窗会因多次回绕报出 `-30652.1 Hz` 的假值。
- **健康检查**： `stat_amp_sq` = `0x3228FF09` ⇒ 幅度比 **0.8853**（健康带 0.60–1.20 内）； `0x05F2` = 12（v12 生产位流）； `0x0101` = `0x0020`（inter = 32，本点上限 ≈195 kHz，**无需降 inter**）； `0x0102` = `0x0000`（div = 4）。

- **采点**：`tools/freq_ladder.py --port COM3 --freqs 100k --seconds 15 --no-wait --with-diag --out deliverables\data --figures deliverables\figures`（日志 `_synth_work/_freq100k_run.txt`，exit 0） ⇒ 有效帧 **149,381**、`crc_err` 539、工具报 `半径 mean=26197.1 rms=0.093% 状态=inclim=149381 滞后=-`。
- **★ 实证 `0x0404/0x0405` 的 W1C 语义（不是写失败）**：改频后读到 `0x053A` = 0x0018（`caldone + otr`）、`0x0404` = 0x2D09、`0x0405` = 0x002D；写 `0x0405 = 0x0001` 后 `0x0404` 读到 **0x2D08**（bit0 `otr` 已清、**bit3 `inclim` 依在**）。
- 这不是清位失败：`rtl/reg_file.v:79-83` 的寄存器表明写 `0x0404 STAT_VAL` 是**只读实时组合**、`0x0405 STAT_LATCH` 是**只读 + W1C**（写 1 清对应位，写 0 的位保持不变；写 bit0 同时发 1 拍 `clr_otr` 清 ADC 内部 sticky）。
- ⇒ 由于 `inclim = (limiter_active | uvw_freq_warn)` 且 `uvw_freq_warn` 粘滞仍成立，**`0x0404` / `0x0405` 的 `inclim` 位根本不可能被"清掉"**，这不是缺陷。
- **★ 边界抖动复现（把 50 kHz 那条"未收口"的怀疑方向做成了可复现实验）**：在 §1.73 的 xsim 扫描台上加一列「初始角度 `ang0`」（`_sim_work/_tb_uvw_replay3.v`，仍不入库），按板上参数（`pairs = 4`、`MINHALF = 12 @50 MHz`、`uwven = 1`、每 1000 拍进一个角步长、4,000 个采样）扫"停在扇区边界附近"：

每采样步长	`ang0`	warn 占比	上升沿	最短 `r_dwell`
100 LSB	4096 / 4000 / 16384 / 16300 / 0	0.00 %	0	16,008 / 17,008 / 27,000 / 3,008 / 27,000
1 000 LSB	16384	0.00 %	0	2,000
1 000 LSB	**16 000**	**0.07 %**	**1**	**8（< 12 ⇒ 置位）**
20 / 50 / 1 LSB	16384	0.00 %	0	65,535 / 54,000 / 65,535

- 结论：**门限的触发条件比 §1.73 更宽松**——不必"一个采样跨半个扇区"，只要**跨边界那一拍的 `r_dwell < 12` 就置位**；`ang0` 离边界的位置决定第一次切换要等多久（`16000` 距电边界 `16384` 只差 384 LSB ⇒ 步长 1000 时第一步就跨界 ⇒ `r_dwell` 只剩 8）。
- ★ 但同时**证伪了"纯边界抖动 = 恒亮"**：即便把初始角度精确放在边界（`ang0 = 16384`）且步长小到 1 LSB，占比仍是 **0.00 %**（因为一次置位后需要一个 ≥12 拍的扇区才能清，而抖动场景下 `dwell_nxt` 在跨界拍被重置为 1 ⇒ 无法提供"长扇区" ⇒ 粘滞位**会一直亮**——这一条要在 ILA 连续拍数据上才能判定，见下）。
- **★ 诚实结论：`uvw_freq_warn` 在 50/100 kHz 恒亮的机理仍未收口。** 现在有两条互相矛盾的证据：
- 支持"真告警"：xsim 显示门限容易被跨边界那一拍触发，而粘滞位一旦点亮**只能靠一个 ≥12 拍的完整扇区清除**；如果角速率的相邻扇区都不足 12 拍，就会恒亮。
- 反对"真告警"：`pairs = 4` ⇒ 100 kHz 机械对应电频率仅 200 kHz ⇒ 60° 电扇区理论 ≈ 1667 拍 ≫ 12；且 §1.73 的 262 LSB 斜坡工况实测 0.00 %。
- **下一步唯一能收口的做法**：用 ILA 抓 `angle / dut.eang_x / dut.r_sec / dut.r_dwell / dut.sec_changed` 的**连续拍序列**（沙箱内下不了 ILA 位流），然后把真实序列灌进 `_tb_uvw_replay3.v` 做 replay。**报告里目前只能写「该位是 UVW 输出的频率告警、实测不是输入过速」，不能写"已定位机理"。**
- **归档（本节产物）**：
- `deliverables/data/freq_100k.csv` = **15,000 行摘录 / 1,342,133 B / SHA256 `5246B9972ADD4B2043BB475AEC3104F700EFE5A620434832B707E7970B33E644`**（全量 149,381 行 + 表头 = 149,382 行 / 13,475,522 B / SHA256 `9467AC9376C6ABF57AD2D0C2D10846A5E99BF783ADC4AF483F86FDEB13D11A35`，已移入 `.gitignore` 的 `_synth_work/freq_100k_full.csv`）
- `deliverables/figures/freq_100k.png`（130,930 B / SHA256 `E55CF5C6894E9150062D3C2DA849A6FB327B0B6F672492086BF844DD6DE73227`）
- `deliverables/data/freq_robust_summary.csv` 追加 100k 行（现 SHA256 `FA50E90B6B1EDFC6C795557D852D1E4248FD0FABB4F48CC4EF11E3B0B22F24B6`）

点	稳健半径 中位 / min / max	圆度（`uart_scope` RMS 全量 / 本文分位口径摘录）	坏帧	XOR 失败
1 kHz	26,213.9 / 26,093.9 / 26,321.1	0.101 % / 0.627 %	0 / 15,000	571（0.38 %）
10 kHz	26,215.2 / 26,141.2 / 26,282.4	0.125 % / 0.430 %	0 / 15,000	500（0.33 %）
50 kHz	26,210.0 / 26,078.7 / 26,289.5	0.116 % / 0.620 %	0 / 15,000	562（0.38 %）
100 kHz	**26,197.0 / 26,083.2 / 26,284.5**	**0.093 % / 0.573 %**	**0 / 15,000**	539（0.36 %）

⇒ 四点**同量级、无趋势**；★ 圆度最好的恰恰是频率最高的 100 kHz 点 ⇒ **这个口径量的是"采集窗口内的半径一致性"，不是编码器精度**，不能拿它做"频率越高越好/越差"的结论。 - ⚠ **诚实限定**：① 每点仍是"一个 15 s 窗 + 摘录 15,000 行"，**没做长时间统计**；② `lag_deg`（环路滞后）在 100 kHz 仍为空（需 ILA）；③ `uvw_freq_warn` 恒亮机理**未收口**（见上）；④ 用户手工改频，脚本读不到 AFG，只有板上反测值背书；⑤ `crc_err` 是工具在全量帧上数的，与摘录行数不同口径。

1.75 ★★★★ 频率阶梯 P1-3 第五个点（200 kHz，需临时降 `inter`）+ 查清"单次读 `0x0530` 不是幅值"这个测量陷阱（2026-10-13）

- **触发**：用户答 ask_user_question（id=`next_step_200k`）选「改 AFG 到 200 kHz，我同时降 inter」。**RTL 未改**，只临时改了 `0x0101`。
- **临时改 `0x0101` 与复原（全过程有回读）**：
- 改前快照：`0x0101` = **`0x0020`**（inter = 32，上限 ≈195 kHz）、`0x0102` = `0x0000`（div = 4）、`0x0100` = `0x2004`（pairs = 4）、`0x05F2` = 12。
- 写 `0x0101 = 0x0010`（inter 32 → 16，上限 ≈390 kHz），回读确认 = `0x0010`。
- **跑完立即复原**：写回 `0x0101 = 0x0020`，回读确认 = `0x0020`、`0x0102` = `0x0000`、`0x05F2` = 12，`0x053A` = `0x0010`（无告警）。
- **改频后独立测频**（短窗，200 kHz 下 `0x0540` 每 0.328 s 回绕）：0.028 s 窗四次 ⇒ **199,983.7 / 200,282.3 / 200,617.9 / 200,132.5 Hz**（均值 ≈200,004 Hz）。复原 `0x0101` 后再测 200,395.7 Hz（AFG 仍在 200 kHz）。
- **采点**：`tools/freq_ladder.py --port COM3 --freqs 200k --seconds 15 --no-wait --with-diag --out deliverables\data --figures deliverables\figures`（日志 `_synth_work/_freq200k_run.txt`） ⇒ 有效帧 **149,356**、工具报 `半径 mean=26204.2 rms=0.143% 状态=adapt=2, inclim=149356, lagfatl=1, laglim=37, oflow=1, res=1, scamp=1 滞后=-`。
- **★ 逐帧统计戳破"状态栏里的告警"**（`csv.DictReader` 全量 149,356 帧）：
- `stat`：`8`（`inclim`）**149,318 帧** / `12`（`inclim|laglim`）**36 帧（0.024 %）** / `200`（`inclim|adapt|res`）**1 帧** / `125`（`oflow|laglim|inclim|lagfatl|scamp|adapt`）**1 帧**。
- `limiter` 列全 0；遥测 `io_state` bit7（`limiter_active`）**0 / 149,356 = 0.00 %** ⇒ 输入限频器**全程未动**。
- ⇒ 工具状态栏里那一长串 `adapt / lagfatl / laglim / oflow / scamp` **全部来自那 1 个链路坏帧**（`flags = oflow|laglim|inclim|lagfatl|scamp|adapt`，`sqrt(amp_sq)` = 52,168.8、`hypot(s,c)` 偏差 34.56 %）。剔除后**板上真正的告警只有 `inclim`**（同 50/100 kHz，来自 `uvw_freq_warn`），加上 36 帧（0.024 %）的 `laglim` 偶发。
- ⇒ **200 kHz 下环路仍然锁定**：`lag_warn/lag_limit` 只占 0.024 %、`limiter_active` 恒 0 ⇒ 比"过速跟不上"的表象好得多。
- **归档（本节产物）**：

- `deliverables/data/freq_200k.csv` = **15,000 行摘录 / 1,332,640 B / SHA256** `E9CA4DC61C886273BE65D617833C423061488BD6DEF9438D3FA3BBE7AE8766A6`（全量 149,356 行 / 13,458,389 B / `E29EB328E06B6FA6206B9C8F0C63B440E616E31535592FD3722869EE27714D62` 已移入 `.gitignore` 的 `_synth_work/freq_200k_full.csv`；全量里有 1 个坏帧，摘录内 0 个）
- `deliverables/figures/freq_200k.png`（147,934 B / SHA256 `213AFF1BDC6BDDD7273D87A18379E78821436A407B15DA632B038145010D17ED`）
- `deliverables/data/freq_robust_summary.csv` 追加 200k 行（现 **499 B / SHA256** `99EBEEB4C5DB7829A8DDC12324491CD1D5EA684CDE3C564C00F3008A38BF2C41`）

点	稳健半径 中位 / min / max	圆度（`uart_scope` RMS 全量 / 本文分位口径摘录）	坏帧	XOR 失败
1 kHz	26,213.9 / 26,093.9 / 26,321.1	0.101 % / 0.627 %	0 / 15,000	571（0.38 %）
10 kHz	26,215.2 / 26,141.2 / 26,282.4	0.125 % / 0.430 %	0 / 15,000	500（0.33 %）
50 kHz	26,210.0 / 26,078.7 / 26,289.5	0.116 % / 0.620 %	0 / 15,000	562（0.38 %）
100 kHz	26,197.0 / 26,083.2 / 26,284.5	0.093 % / 0.573 %	0 / 15,000	539（0.36 %）
200 kHz	**26,201.3 / 26,059.3 / 26,304.2**	**0.143 % / 0.665 %**	**0 / 15,000**	555（0.37 %）

⇒ 五点**同量级、无趋势**（中位 26,197–26,215 的漂移 < 0.1 %），**200 kHz 下角度数据依旧干净**；★ 再把口径钉一次：这个量测的是"采集窗口内的半径一致性"，**不是编码器精度。** - **★★ 本节最有价值的副产物：查清"单次读 `0x0530` 不是幅值"这个测量陷阱**（此前 §1.74 与历史多轮都踩过）： - 现象：改 `inter` 到 16 后单次读 `0x0530` 得到 `sqrt(amp_sq)` = 0.436（Q2.30 口径），**看起来 `amp_low` 被点亮**；但同一时刻 `uart_scope.py` 的平均口径（`hypot(s,c)` 中位）= **26,199.9**，幅度完全正常。 - 机理（逐行读 `rtl/amp_monitor.v`）： `amp_sq` = **单个采样** 的 `s_n² + c_n²`（`rtl/amp_monitor.v:66-72`： `s_sq = s_n*s_n`、`c_sq = c_n*c_n`、`sq_sum = s_sq + c_sq`），是**瞬时量**； `s_n = A·sinθ`、`c_n = A·cosθ` ⇒ `amp_sq = A²`（与 θ 无关），但**定点里的 `s_n/c_n` 是包络上的逐点样本**，所以 `0x0530` 的真实含义是"最近一次采样点的 s²+c²"，**逐次读值会随采样相位大幅起伏。** - 实测坐实（`0x0530` 连读 40 次，板载 200 kHz）： `sqrt(amp_sq)` 的 **min 9,308 / p10 14,859 / 中位 23,940 / p90 31,238 / max 32,552**，`min/max = 0.29` ⇒ 单次读值**不可用于判断幅度健康。** - ⇒ **口径更正**：判断输入幅度健康只能用 ① `uart_scope.py` 的**平均口径**（`hypot(s,c)` 中位或 `stat_amp_sq` 的窗口平均），或 ② 连读多次取分位；**绝不能拿单次 `0x0530` 读数当 `amp_low` 的依据**（§1.74 里那句"单次读 0.4359 ⇒ amp_low"其实就是这个陷阱的实例，`0x053A` bit1 实测始终为 0）。 - **★ 状态位复核（排除"改频瞬态未清"的误判）**： `0x0404 STAT_VAL` = **`0x0808`**（低字节只有 bit3 `inclim`）、`0x0405 STAT_LATCH` = **`0x0008`**、`0x053A` = `0x0010`（只有 `caldone`）——三者**连续 6 次读数完全稳定**，说明 §1.73/§1.74 记的 `0x2D09`/`0x2D` 确实是**改频瞬间的瞬态**且已被 W1C 清掉，**没有残留粘滞位。** - ⚠ **诚实限定**：① 每点仍是"一个 15 s 窗 + 摘录 15,000 行"；② `lag_deg` 在 200 kHz 仍为空（需 ILA）； ③ 本点为了过 200 kHz **临时把 `inter` 从 32 降到 16**（已复原），**因此本点的 `inter` 与其余四点不同**，引用时须写明； ④ `uvw_freq_warn` 恒亮机理仍未收口（见 §1.73/§1.74）； ⑤ 用户手工改频，只有板上 `0x0540` 短窗反测值背书。

1.76-频率阶梯 ★★★★ 频率阶梯 P1-3 第六个点（500 kHz）——六点扫描收官；出现第一条"频率越高越差"的弱趋势信号（2026-10-13）

> 编号说明：本节的"1.76"与下方 `### 1.76 上位机界面优化批次 3`（另一路并行会话所写）**编号撞车**，为避免误改他人内容，本节保留 `1.76-频率阶梯` 后缀；两节内容互不相干（本节 = 频率阶梯 P1-3，那节 = 上位机 UI）。

- **触发**：用户答 ask_user_question（id= `afg_500k_ready`）选「已改到 500 kHz」。**RTL 未改**，只临时改了 `0x0101`。
- **临时改 `0x0101` 与复原（全过程有回读）**：
- 改前快照： `0x0101` = `0x0020`（inter = 32，上限 ≈195 kHz）、`0x0102` = `0x0000`（div = 4）、`0x0100` = `0x2004`（pairs = 4）、`0x05F2` = 12。
- 写 `0x0101 = 0x0008`（inter 32 → 8，上限 ≈780 kHz），回读确认 = `0x0008`。
- **跑完立即复原**：写回 `0x0101 = 0x0020`，回读确认 = `0x0020`、`0x0102` = `0x0000`、`0x05F2` = `0x000C`、`0x053A` = `0x0010`（无告警）、`0x0404` = `0x0808`（低字节只有 bit3 `inclim`）⇒ **无残留粘滞位。**
- **改频后独立测频**（短窗，500 kHz 下 `0x0540` 每 **0.131 s** 就回绕一次 ⇒ 必须用 ~15–20 ms 窗）：0.018 s 窗五次 ⇒ **500,273.2 / 500,826.2 / 501,145.5 / 501,415.2 / 501,173.9 Hz**。复原 `0x0101` 后再测 **500,658.8 Hz**（AFG 仍在 500 kHz）。
- ★ 插曲：第一次去测时端口被另一个残留的 `python` 进程（pid 10020，起于 22:10:26）占着，`open_port` 报 `PermissionError(13, '拒绝访问。')`；等它退出后重试即通。**教训：采点脚本报"端口被占"时先 `Get-Process python` 看有没有上一轮没退干净的进程，不要怀疑板子。**
- **采点**： `tools/freq_ladder.py --port COM3 --freqs 500k --seconds 15 --no-wait --with-diag --out deliverables\data --figures deliverables\figures`（日志 `_synth_work/_freq500k_run.txt`） ⇒ 有效帧 **149,380**、工具报 `半径 mean=26112.9 rms=0.211% 状态=adapt=4, falarm=3, inclim=149379, lagfatl=4, laglim=1, oflow=2, res=2, scamp=2 滞后=-`。
- **★ 逐帧统计（口径同 §1.75）**： `stat` = `8`（`inclim`）**149,375 帧**，其余 5 个值各 1 帧（`122` / `59` / `64` / `216` / `223`）； `flags` 里只有 **4 个坏帧**带告警串； `limiter` 列全 0；遥测 `io_state` bit7（`limiter_active`）**0 / 149,380 = 0.00 %。** ⇒ 工具状态栏那一长串 `adapt / falarm / lagfatl / laglim / oflow / scamp` **同样全部来自链路坏帧，板上真实告警仍然只有 `inclim` 一项**（即 `uvw_freq_warn`），**500 kHz 下没有 `laglim`/`lag_fatal` 持续告警**（200 kHz 那 36 帧 0.024 % 的 `laglim` 在本点只剩 1 帧量级）⇒ 环路**仍锁定。**
- **链路坏帧特征（六点一致，可直接当判别式）**：坏帧的 `sqrt(amp_sq)` = **52,168.8**、`hypot(s,c)` 偏离中位 34–71 %、且 `stat/flags/io_flags/limiter` 几列同时出现"本不该并存"的组合。来源是 §1.58 的链路固有 XOR 校验错（≈0.35–0.38 %）里**侥幸过检**的帧。
- **归档（本节产物）**：
- `deliverables/data/freq_500k.csv` = **15,000 行摘录 / 1,338,373 B / SHA256** `E32F6C5111C06A5B61D59CED2DB207BF22530307930CD5DE49D8EE8AC52F9598`（全量 149,380 行 / 13,468,924 B / `B842ABCB18745953413DA113ADE0181142CC9CEE620C1F0235A4B5476BADA7EC` 已移入 `.gitignore` 的 `_synth_work/freq_500k_full.csv`）
- 摘录稳健统计：中位 **26,117.3** / p0.1% 26,018.8 / p1% 26,041.3 / p99% 26,170.1 / p99.9% 26,184.8 / min 25,994.9 / **max 52,168.8（就是那个坏帧）** / **坏帧 1（全量里 4）** / 分位圆度 **0.636 %**
- `deliverables/figures/freq_500k.png`（169,566 B / SHA256 `FF0FC6C9E44DD864797EE2B18DB9B20F305957592D7738845FA3111718DFAB7D`）
- `deliverables/data/freq_robust_summary.csv` 追加 500k 行后（现 **584 B / SHA256** `12AE1A3476360C367C0C33E02765B73BDA4232E7BC271C887350BC94F1594BC9`）
- `deliverables/data/freq_ladder_summary.csv`（现 **2,376 B / SHA256** `EBA11E257C204CEF313ED7F29B389605E1E00BF221EBD89AD5E76F181DADFA08`）的 500k 原始行： `500k,26112.9,15144.0,35994.6,0.211,"adapt=4, falarm=3, inclim=149379, lagfatl=4, laglim=1, oflow=2, res=2, scamp=2",149380,545,,2026-10-13 22:11:15,...`（★ 该行 `radius_max` = 15,144.0 是 `uart_scope` 的另一套口径，与稳健口径的 52,168.8 不是一回事，引用时勿混）
- **★★ 六点扫描汇总（`freq_robust_summary.csv` 口径，摘录 15,000 行）**：

点	稳健半径 中位	分位圆度	`uart_scope` RMS（全量）	坏帧（摘录/全量）	`inter`
1 kHz	26,213.9	0.627 %	0.101 %	0 / 0	32
10 kHz	26,215.2	0.430 %	0.125 %	0 / 0	32
50 kHz	26,210.0	0.620 %	0.116 %	0 / 0	32
100 kHz	26,197.0	0.573 %	0.093 %	0 / 0	32
200 kHz	26,201.3	0.665 %	0.143 %	0 / 0（全量 1）	**16（临时）**
500 kHz	**26,117.3**	**0.636 %**	**0.211 %**	**1 / 4**	**8（临时）**

- 前五点中位半径在 26,197–26,215 之间漂（< 0.1 %），**无趋势**；**500 kHz 的中位比其余五点低约 0.36 %（26,117 vs 26,210）**，且 `uart_scope` **RMS 圆度 0.211 % 是六点最大值**（其余 ≤ 0.143 %）。
- ⇒ 这是**第一条可能的"频率越高越差"的弱趋势信号。**但它**不能单独当结论**，因为：① 本点 `inter` = 8（与 1k–100k 的 32 不同，`inter` 越小 ⇒ 更新率越高 ⇒ 环路相位裕度与滤波量都变了）；② 该点 `filter` 之后的角噪声与 `lag` 都需要 ILA 的 `lag_deg` 才能归因；③ 只有一个频点偏低，无法排除夹具/接线在改频时的偶然变化。
- ⇒ **下一步归因必须做**：跑 ILA 版拿 `lag_deg`（六点都缺这一列），或在 500 kHz 下把 `inter` 分别设 32/16/8 各采一次做"同频不同 `inter`"对照。
- ⚠ **诚实限定**：① 每点仍是"一个 15 s 窗 + 摘录 15,000 行"，**未做长窗统计**；② `lag_deg` 六点**全部为空**（需 ILA 位流，沙箱内下不了）；③ 本点为了过 500 kHz **临时把 `inter` 从 32 降到 8**（已复原），故 `inter` 与 1k–100k 四点不同，**与 200 kHz 的 16 也不同**；④ `uvw_freq_warn` 恒亮机理仍未收口（§1.73/§1.74）；⑤ 用户手工改频，只有板上 `0x0540` 短窗反测值背书；⑥ 所有"圆度"量测的都是**采集窗口内的半径一致性**，**不是编码器精度。**

2.0 历史基线（10-09 23:17 / 23:31，对应旧 RTL，勿再引用为上板依据）

★ 上一轮（用户 m08979 第 1 项要求"重跑一次回归 + 两版实现"）**已重跑并复核**： - 生产版：`_synth_work/enc_top.bit` 时间戳 **10-09 23:17:45**、`impl_*.rpt` 为 23:17:33–23:17:37 - ILA 版：`_synth_work/enc_top_ila.bit` **23:31:27**、`.ltx` 23:31:17、`impl_ila_*.rpt` 23:31:14–23:31:17

两者都**晚于** `rtl/uvw_gen.v`（21:49，真实边界修复提交 `23116ff` 之后仅换行符转换）与 `rtl/enc_top.v`（23:16，本轮纯注释改动）⇒ 位流确实对应已定稿的真实边界 RTL。 下表数字为**重跑后实测**，与上一轮**逐项一致**（含位流字节数与压缩位数完全相同）。

项	生产版	ILA 调试版
bitstream	`_synth_work/enc_top.bit` **741,836 B**	`_synth_work/enc_top_ila.bit` **1,473,128 B** + `.ltx` 67,006 B
`sys_clk` WNS	**+0.544 ns**（0 失败 / **9474** 端点，Intra Clock Table）	**+0.698 ns**（0 失败 / 16906 端点）
`sys_clk` WHS	+0.079 ns	+0.042 ns
WPWS	9.146 ns（3483 端点）	8.870 ns（8556 端点）
Slice LUTs	5653 / 20800 = **27.18%**	8466 / 20800 = **40.70%**
Slice Registers	3475 = 8.35%	8161 = 19.62%
DSPs	16 = 17.78%	17 = 18.89%
Block RAM	0	7.5 = 15.00%
DRC	43 checks，0 error / 0 critical	62 checks（全 Warning），0 error / 0 critical
压缩	`saved 11603072 bits` ⇒ 压缩后 5,934,624 bit	`saved 5752736 bits` ⇒ 压缩后 11,784,960 bit

2.1 上一轮重跑的生产版（2026-10-10 02:32，对应 §1.9/§1.10 修正后的 RTL —— 已被 §2.2 取代，保留为历史基线）

命令：`tools/run_vivado.ps1 -Tcl tools\impl_check.tcl -LogFile _synth_work\impl_build.log`

项	新值（10-10 02:32）	与 §2.0 对比
bitstream	`_synth_work/enc_top.bit` **726,924 B**，时间戳 **10-10 02:32:17**	741,836 B → **-14,912 B**
`sys_clk` WNS	**+0.686 ns**（0 失败 / **9478** 端点）	+0.544 → **+0.686 ns（变好）**
`sys_clk` WHS	**+0.082 ns**（0 失败）	+0.079 → +0.082 ns
WPWS	9.146 ns（3484 端点）	3483 → 3484
Slice LUTs	**5665** / 20800 = 27.24%	5653 → **+12 LUT**
Slice Registers	**3476** = 8.36%	3475 → **+1 FF**
DSPs	16 = 17.78%	不变
Block RAM	0	不变
Bonded IOB	**49**	不变（接线未动）
BUFGCTRL	1	不变
DRC	43 checks，**0 error / 0 critical**（仅 DSP 流水线 DPIP/DPOP Warning）	一致
布线	8771 nets，**fully routed，routing errors 0**	—
压缩	`saved 11722368 bits`	11603072 → +119,296 bit
时序判据	`All user specified timing constraints are met.`	一致

这些数字与 §1.9 的改动自洽（不是"设计被改了却没人发现"）： 新增的窗口判据（`wcnt+1 > win_n` 的 17 位比较）与 `n_after_t` 的更新路径 ⇒ **+12 LUT / +1 FF / +4 端点**，量级完全吻合；位流反而**小了 14,912 B** 是因为布局布线不同导致压缩后位图不同（`saved` 位数增加），与 LUT 数无关。

位流产物复核（不依赖脚本退出码）： - 头部 `00 09 0F F0`，Xilinx 同步字 `AA 99 55 66` 位于**偏移 260**（两版一致） - 尾部 `20 00 00 00` - `report_drc` 的 `write_bitstream` 前置 DRC： `DRC finished with 0 Errors`

> ⚠️ **`PROD_EXIT=-1`，但构建是成功的。** 本次与 §3 记载的 ILA 版一样出现 **"位流已写完（`Bitgen Completed Successfully`）后脚本挂住"**：最后两条 `puts`（`BITSTREAM_WRITTEN` / `IMPL_CHECK_DONE`）都没打印，vivado 进程空转 45 min cpu 时间、无任何新文件写出，被手动终止。 **判据永远是产物本身**（时间戳 + 报告 + 位流头/sync 字），不是脚本退出码。 这条对生产版和 ILA 版都适用。

两版都打印 `All user specified timing constraints are met.`； `adc_clk` / `adc_clk2` 两版均为 WNS +6.965 / WHS +0.965。 **★ 时序签核一律以生产版 `_synth_work/impl_timing.rpt` 为准**，ILA 版只作调试用途。 ★ 端点数**必须注明取自哪张表**（见 §1.8）： `sys_clk` = 9474 是 Intra Clock Table； 全设计汇总表给的是 **9476**（多算 `sys_clk→adc_clk` 与 `→adc_clk2` 各 1 个跨域端点）。

其它	值
回归	`sim/logs/SUMMARY.md` **PASS 18 / 18**（2026-10-10 02:23:16， `REGRESS_EXIT=0` ）
引脚	`python tools/pinout_verify.py` → PASS（49/49；载板插 **J6**，输出侧 **J5**）
引脚方案	`python tools/pinplan_check.py` → PASS
模型自检	`python model/abz_uvw_model.py` → 6/6 PASS
lint	`tools/lint_all.ps1 -Clean` → 30 文件 0 ERROR / 0 WARNING
悬空扫描	`python tools/scan_dangling.py` → **exit 0**（A 真死 0 / B input 悬空 0 / C 0 / D 充数项 0）
编码关卡	`python tools/to_lf.py --check` → exit 0

两版位流头部均为 `00 09 0f f0`，Xilinx 同步字 `AA 99 55 66` 位于偏移 **260**。

2.2 ★★ 本轮（P0-2 + §1.17）重跑的生产版：输出侧约束首次纳入签核（2026-10-10 07:31，已被 §2.3 取代，保留为历史基线）

命令同 §2.1： `tools\run_vivado.ps1 -Tcl tools\impl_check.tcl -LogFile _synth_work\impl_build.log`

为什么必须重跑：① `constr/enc_top.xdc` 新增第 10 节，给此前**完全没有输出侧约束**的 14 个输出补了 `set_output_delay` + `SLEW/DRIVE`； ② `rtl/enc_top.v` 修掉了标定窗口 （§1.17）。两者都改了签核输入，所以位流必须重生成。

输出预算的三次迭代（如实记录，包括失败的那一次）：

迭代	输出预算	WNS	失败端点	位流	说明
①	`-max 12.000` / `-min 1.000`，全部 14 个	**−3.736 ns**	**14**	923,944 B（07:14）	**不满足时序。**报告存档 `_synth_work/impl_timing_p02_max12_FAIL.rpt`； 最差路径 `u_amp/amp_low_reg/C -> led[1]`，端口数据 11.701 ns 才稳定。教训：Vivado 里 `required = 20 − output_delay − uncertainty`， **`-max` 越大留给内部越少**，12.000 在数学上不可能满足
②	`-max 6.000` / `-min 0.500`，全部 14 个	+0.097 ns	0	909,736 B（07:23）	全约束满足，但全局最差路径变成**状态灯** `u_capture/otr_reg/C -> led[0]`（数据路径 9.092 ns，其中 OBUF+布线 4.667 ns）⇒ 余量只剩 0.097 ns，太脆
③	**位置类 `-max 6.000`**（a/b/z/u/v/w_out + biss_slo + spi_miso）/ **指示类 `-max 3.000`**（led[*]、uart_tx、irq_n）	**+0.512 ns**	**0**	**731,612 B（07:31:54）**	最差路径回到**内部**（见下），输出约束不再是瓶颈 ⇒ 这才是交付版

交付版（迭代③）实测数字：

项	值	对比 §2.1（02:32）
bitstream	`_synth_work/enc_top.bit` **731,612 B**，时间戳 **2026-10-10 07:31:54**	726,924 B → **+4,688 B**
整体 WNS / TNS / 失败端点	**+0.512** / 0.000 / **0**（9487 端点 = **全设计汇总**；Intra Clock Table 的 `sys_clk` 为 9485，见 §1.8）	+0.686 → +0.512（引入输出约束的代价，见下）
整体 WHS / THS / 失败端点	**+0.075** / 0.000 / **0**（9487 端点，同上）	+0.082 → +0.075
WPWS / 失败端点	9.146 ns / **0**（3484 端点）	不变
时序判据	**`All user specified timing constraints are met.`**	一致
全局最差路径	**内部路径**： `u_regs/r_inter0_reg[3]/C -> u_abz/r_hold_reg[0]/R`， Data Path Delay **18.977 ns**（logic 9.535 / route 9.442）， Slack +0.512	之前是输出端口路径
Slice LUTs / Registers	**5656** = 27.19% / **3476** = 8.36%	5665 → −9 LUT（布局不同）
DSPs / Block RAM / Bonded IOB / BUFGCTRL	16 / 0 / **49** / 1	不变
DRC	`Checks found: 43`（`DPIP-1` ×11 / `DPOP-1` ×16 / `DPOP-2` ×16，全 Warning），**0 error / 0 critical**	一致
布线	**8767** nets fully routed，**routing errors 0**	8771 → 8767
压缩	`saved 11684864 bits` ⇒ 17,537,696 − 11,684,864 = 5,852,832 bit = **731,604 B**，实测 **731,612 B**（同样的 **+8 B** 头部差，与 §2.1 的自洽口径一致）	11684864 vs 11722368
上板判据	`INFO: [Vivado 12-1842] Bitgen Completed Successfully.`（脚本照旧在 bitgen 后挂住 ⇒ `PROD_EXIT=-1`，**判据仍是产物**）	一致

为什么 WNS 从 +0.686 掉到 +0.512 是"应该的"而不是"退步"： 新增的 14 条 `set_output_delay` 把**输出端口也纳入 setup 检查**（此前工具直接忽略它们， 所以数字好看但没有意义）。+0.512 ns 是在**把输出裕量也算进去之后**的余量， 而且全局最差路径已经回到内部逻辑 ⇒ 约束生效且不冒进。 上板用示波器/逻辑分析仪

实测边沿后，可以再收紧这两个预算并重跑一次。

§1.12 的"位流必须晚于 RTL"判据已重新成立：`rtl/` 里最新的 `enc_top.v` 是 **07:19:25**，位流 **07:31:54** ⇒ 位流晚于全部 `rtl/*.v`（§2.1 那一版曾被 §1.16 的 纯注释改动"追尾"，本轮一并消除）。

仍未做的事（如实登记）：XDC 第 10 节只做了**约束 + 驱动/压摆率**， **没有加源端串阻**（那要动硬件：`docs/04:286`、`docs/06:20` 建议 J5 排线 ≤5 cm、 必要时串 22~33 Ω）。`SLEW FAST` + `DRIVE 8 mA` 只是降低 di/dt， 真正的振铃解法仍是串阻；上板若见 ABZ 偶发多计/漏计，先怀疑这里。

2.3 ★★ 本轮（§1.18 幂等标定修复）重跑的生产版（2026-10-10 08:40；已被 §2 顶部 16:05:29 版与 §2.4 取代，本节的"全约束满足"只对这一版成立）

> ★ **本节整体保留为历史轮次**：08:40 这一版构建在 §1.19 的诚实 ADC 输入约束**之前**， 也没有 §1.20 探针位序修复 / §1.21 OTR 软清位 / §1.22 UART→SPI 桥。 **现行生产版签核以 §2 顶部"加 UART→SPI 桥后的重跑（2026-10-10 16:05:29）"为准** （`enc_top.bit` 804,780 B、 `sys_clk` WNS −3.352 / 26 个 ADC 输入负余量端点 / 10582 端点）。
>
> ★ 2026-10-10 12:08 之后的口径变化（见 §1.19）： ADC 输入约束按真实模型重写为 `-max 24.000 / -min 19.500` 后重跑，得到 **WNS −3.271 / 26 个 failing endpoints （全部是 ADC 输入端口）**、`Timing constraints are not met.`， 那一版产物在 `_synth_work/signoff_adc_honest/` （`enc_top.bit` 1,219,152 B @12:08:30）。 **两版 RTL 完全相同、只差约束值**：本节的 08:40 版是"全约束满足"的最后一版， 12:08 版是"把已知边界项显式化"的版本。上板烧哪一版都可以（功能一致）， 但报告与答辩里**必须按 §1.19 的口径说明 ADC 输入路径的实际余量状态。**

命令同 §2.1：`tools\run_vivado.ps1 -Tcl tools\impl_check.tcl -LogFile _synth_work/impl_build.log` （脚本照旧在 `Bitgen Completed Successfully.` 后挂住 ⇒ `PROD_EXIT=-1`， **判据永远是产物时间戳与报告**）。

为什么必须再重跑：本轮修 §1.18 动了两个 RTL 文件的**功能**（不是注释）—— `rtl/enc_top.v` 的 `calib_ctrl` 改为累加校正前的 `s_dec/c_dec` （`.in_valid(dec_valid)`）、 `rtl/calib_ctrl.v` 新增 `armed` / `arm_tmo` / `ARM_TMO_MAX` 对齐逻辑与关门判据。 按 **§1.12 的"位流必须晚于全部 `rtl/*.v`"**判据，位流必须重生成。

项	值（本节实测，历史轮次）	对比 §2.2（07:31）
bitstream	`_synth_work/enc_top.bit` **962,600 B**，时间戳 **2026-10-10 08:40:17**	731,612 B → **+230,988 B**（新增 26 bit 超时计数器 + 比较器 + 对齐 FSM）
整体 WNS / TNS / 失败端点	**+0.923** / 0.000 / **0**（**9631** 端点 = 全设计汇总；Intra Clock Table 的 `sys_clk` 为 **9629** = 9631 − `adc_clk` 1 − `adc_clk2` 1，同 §1.8 的规律）	+0.512 → **+0.923**（对齐逻辑没有拖累时序；布局不同）
整体 WHS / THS / 失败端点	**+0.066** / 0.000 / **0**（9631 端点）	+0.075 → +0.066
WPWS / 失败端点	**9.500 ns** / **0**（3534 端点）	9.146 → 9.500
时序判据	`All user specified timing constraints are met.`	一致
全局最差路径	**内部路径**：`u_regs/r_inter0_reg[3]/C -> u_abz/r_hold_reg[10]/R`，Data Path Delay **18.634 ns**（logic 9.802 / route 8.832），27 级逻辑（CARRY4×11、DSP48E1×1…），**Slack (MET) +0.923**	仍是内部路径（输出端口不是瓶颈）
Slice LUTs / Registers	**5763** = 27.71% / **3528** = 8.48%	+107 LUT / +52 FF（对齐逻辑的代价）
DSPs / Block RAM / Bonded IOB / BUFGCTRL / BSCANE2	16（17.78%）/ 0 / **49** / 1 / 0	不变（ILA 版才有 BSCANE2）
DRC	`Checks found: 43`（DPIP-1×11 / DPOP-1×16 / DPOP-2×16，全 Warning），**0 error / 0 critical**	一致
布线	**8904** nets fully routed，**routing errors 0**	8767 → 8904
压缩	`saved 9836960 bits` ⇒ 17,537,696 − 9,836,960 = 7,700,736 bit = **962,592 B**，实测 **962,600 B**（同样的 **+8 B** 头部差，口径与 §2.1/§2.2 一致）	11684864 → 9836960
位流头	`00 09 0f f0`（与 §2.1/§2.2 相同）	—
上板判据	日志含 `INFO: [Vivado 12-1842] Bitgen Completed Successfully.`	一致

§1.12 判据复核：位流 **08:40:17** 晚于全部 `rtl/*.v`（最新是 `rtl/calib_ctrl.v` 与 `rtl/enc_top.v`，均在 08:37 之前） ⇒ 位流对应当前源码。

仍未做的事（不变）： 源端串阻 22~33 Ω 依旧没有加（见 §2.2 末尾那段，理由与风险相同）； 输出侧预算（位置类 6.000 / 指示类 3.000 ns）仍是**预算**，上板实测后可收紧再重跑。

2.4 ★★★ 历史基线（10-10 22:10:14）： §1.29 非整周期窗口检测 + 自动重标（已被 §2.5/§2.6/§2.7 取代）

> **为什么必须再重跑**：§1.28 查明 KEY0 一键标定的 `OFS_S` 有 ~37.5~50% 概率错 ±1800…2300 （`angle_raw` MSB 抖动 ⇒ 奇数个半周期的标定窗口），§1.29 在 `rtl/calib_ctrl.v` 里落了 **"窗口周期间隔 max/min 判据（1.25 倍）+ 作废重标（最多 3 次）"**、`rtl/reg_file.v` 扩了 `0x0502` 读位域。**动了功能 ⇒ 按 §1.12 判据位流必须重生。** 本节位流**取代 21:06 的 v2（`_synth_work/pre_v3/enc_top_v2_2106.bit`，729,948 B，含本缺陷）**， 也是**用户需要重新烧录的那一版。**

命令： `powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 -Tcl tools\impl_check.tcl -LogFile _synth_work\impl_build_v3.log -Tail 20`

项	值（本轮实测）	对比 16:05 v2（pre_v3/impl_build_v2_2106.log）
bitstream	_synth_work/enc_top.bit **825,256 B**，时间戳 **2026-10-10 22:10:14**	729,948 B → **+95,308 B**（布局布线不同 + 新增重标计数器/比较器）
整体 WNS / TNS / 失败端点	**−3.378** / −84.740 / **26**（Route 35-57 Estimated Timing Summary，WHS +0.048 / THS 0.000）	−3.353 / −85.415 / 26 → **同一口径，未变差**
时序判据	Timing constraints are not met.（**与 §1.19 口径一致：26 个失败端点全是 ADC 输入端口**）	一致
全局最差路径	ad_ch1_d[7] → u_capture/ch1_meta_reg[7]/D（Input Delay 24.000，slack −3.377 ns）	相同路径、相同 Input Delay 24.000 ⇒ **不是本轮改动引入的**
次差路径	u_cordic/g_iter[15].z_p_reg[16][0]/C → u_cordic/g_iter[16].z_p_reg[17][5]/D（内部 CORDIC 迭代，历史同项）	一致
Slice LUTs / Registers	**6119** = 29.42% / **3892** = 9.36%	新增判据的代价（量级自洽）
DSPs / BRAM / BUFGCTRL	16（17.78%）/ 0 / 1	不变
DRC	Checks found: 43（DPIP-1×11 / DPOP-1×16 / DPOP-2×16，全 Warning），**0 error / 0 critical**	一致
布线	**9525** nets fully routed，**routing errors 0**	—
压缩	saved 10935712 bits ⇒ 17,537,696 − 10,935,712 = 6,601,984 bit = 825,248 B，实测 **825,256 B**（同样是 **+8 B** 头部差，口径与 §2.1/§2.2/§2.3 一致）	10935712 vs v2 的 11484992
上板判据	日志含 INFO: [Vivado 12-1842] Bitgen Completed Successfully.	一致

"新逻辑真的进了网表"的证据（不靠推断）：综合日志里出现 WARNING: [Synth 8-6014] Unused sequential element lim_min_t_reg was removed. [rtl/calib_ctrl.v:544]、 … lim_max_t_reg … [rtl/calib_ctrl.v:546]、 … win_bad_t_reg … [rtl/calib_ctrl.v:553] —— 正是 §1.29 新增的组合量；这类"*_t 临时量被当成未用寄存器删掉"的 Warning 在 cordic_atan2.v:209（y_p_reg[17]）、 uvw_gen.v:366/367/524、 biss_c_slave.v:216/273、 spi_slave.v:132/133、 calib_ctrl.v:463/464/526/527/528（finish_t / n_after_t / plast_t / nxt_est_t / cap_stop_t） 上**早就存在** ⇒ 是本仓既有的良性写法，不是本轮引入的问题。

同轮仿真与检查：tools/lint_all.ps1 = **ERROR 0 / WARNING 0**； tools/run_sim.ps1 = **PASS 19/19**（含 tb_enc_top 的 F4: done=1 nsamp=16384 wcnt=64 truncated=0、 F11: 两次标定之差全 0、 TEST PASSED (81 checks)，见 §1.29）。

待做的事（v3 —— ★ 已被 §2.5 取代，以下保留为历史）：① 用户重新烧录 _synth_work/enc_top.bit； ② python tools\calib_repeat.py --port COM5 --n 20 --csv _synth_work\calib_rep_v3.csv ⇒ 判据 **20/20 收敛**（|OFS_S| ≤ 400）且 0x0502[7]win_bad = 0、 0x0502[6:4]retry = 0 （★ 位域 2026-10-10 订正，见 §1.29 ③；本节曾误写 [4]win_bad / [7:5]retry）。 **★ 实测结果（2026-10-10，板上）**：**好 14 / 坏 6** ⇒ 本版（检测+重标）判为**不充分**，已由 §1.30 的滞回过零取代，见 §2.5。 ③ ILA 调试版**仍是 16:12 那一版**（早于 §1.28/§1.29），若需要抓新波形要重建（需一次 danger-full-access）。

2.5 ★★★★ 历史基线（v4）：§1.30 滞回过零根治（2026-10-10 22:28:24；已被 §2.6/§2.7 取代）

> **为什么必须再重跑**：v3（§2.4）在板上实测 20 轮 **好 14 / 坏 6**，证明 §1.29 的"检测 + 作废重标" **救不了**（抖动几乎每个窗口都发生，见 §1.30 ①）。§1.30 把 rtl/enc_top.v 的标定周期脉冲相位参考 换成**带滞回的角度过零**（新增 CAL_HYS / calib_s_side，删除 calib_ang_d）⇒ **动了功能 ⇒ 位流必须重生。** 本节位流 **取代 §2.4 的 v3（22:10:14，825,256 B）**，是**用户需要重新烧录的那一版。**

命令： powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 -Tcl tools\impl_check.tcl -LogFile _synth_work\impl_build_v4.log -Tail 20

项	值（v4 实测）	对比 v3（§2.4）
bitstream	_synth_work/enc_top.bit **767,116 B**，时间戳 **2026-10-10 22:28:24**	825,256 B → **−58,140 B**（MSB 比较器 + §1.29 去抖残留寄存器换成一个小状态机）
整体 WNS / TNS / 失败端点	**−3.368** / −84.834 / **26**（WHS +0.074 / THS 0.000）	−3.378 / −84.740 / 26 ⇒ **同一口径，未变差**
时序判据	Timing constraints are not met.（**26 个失败端点仍全是 ADC 输入端口**，§1.19 口径）	一致
全局最差路径	ad_ch1_d[7] → u_capture/ch1_meta_reg[7]/D（slack −3.368 ns）	同一路径 ⇒ **不是本轮引入的**
Slice LUTs / Registers	**6122** = 29.43% / **3892** = 9.36%	6119 / 3892
DSPs / BRAM / IOB / BUFGCTRL	16（17.78%）/ 0 / **50** / 1	一致
DRC	Checks found: 43（DPIP-1×11 / DPOP-1×16 / DPOP-2×16，全 Warning），**0 error / 0 critical**	一致
布线	**9527** nets fully routed（internally 6466），**routing errors 0**； Number of Failed Nets = 0	v3: 9525
压缩（自校验）	saved 11400832 bits ⇒ 17,537,696 − 11,400,832 = 6,136,864 bit = **767,108 B**，实测 **767,116 B**（**+8 B** 头部差，与 §2.1/§2.2/§2.3/§2.4 同口径 ⇒ 位流有效）	同口径 ✓
上板判据	日志含 Generate Bitstream complete + INFO: [Vivado 12-1842] Bitgen Completed Successfully.，write_bitstream 段 **0 Warnings / 0 Critical / 0 Errors**，末尾 BITSTREAM_WRITTEN + IMPL_CHECK_DONE，退出码 **0**（整轮 47 min；更早的综合段那 112 个 Warning 全是既有良性同类）	一致

"新逻辑真的进了网表"的证据：`impl_build_v4.log` 里 §1.29 那批 `*_t` 良性删除警告仍在 （`calib_ctrl.v:463/464/526/527/528/544/546/553`），而**原 MSB 检测用的 `calib_ang_d` 已完全消失** （随 §1.30 删除）⇒ 网表与 `rtl/enc_top.v` 一致。

同轮仿真与检查：`tools/lint_all.ps1` = **ERROR 0 / WARNING 0**；`tools/run_sim.ps1` = **PASS 19/19**； `tb_enc_top`： `F4: done=1 nsamp=16384 wcnt=64 truncated=0`、 `F11 差值全 0`、 `TEST PASSED (81 checks)` （详见 §1.30 ④）。

待做（v4）：① 用 Hardware Manager 烧录 `_synth_work/enc_top.bit`； ② `python tools\calib_repeat.py --port COM5 --n 20 --csv _synth_work\calib_rep_v4.csv` ⇒ 判据 **20/20**（`|OFS_S| ≤ 400` 且各轮彼此相差 ≤ 几 LSB）**且** `0x0502[7]win_bad = 0`、 `0x0502[6:4]retry = 0`（滞回奏效 ⇒ 间隔检测应完全安静）； ③ ILA 调试版**已于 2026-10-11 00:39:22 用同一份 RTL 重建完毕**（`enc_top_ila.bit` **1,482,840 B** + `enc_top_ila.ltx` 67,006 B，含 §1.30 滞回过零；16:12 那版已备份到 `_synth_work/pre_ila_v4/`， 详见 §2 顶部 00:39:22 那条）。（原备注"需一次 `danger-full-access` "仍成立：本次就是这样跑完的。）

2.6 ★★★★ 历史基线（v5）：§1.31 角度毛刺保持（2026-10-11 11:50:54；已被 §2.7 取代，且 §1.32 证明其 45° 阈值无效）

为什么必须再重跑：v4（§2.5）板上 20 轮标定**结果 20/20 全对**，但 `0x0502` 读回在 **16/20 轮** 仍是 `retry=3 + win_bad=1`。§1.31 ② 定位为"单拍 ADC 错值 ⇒ 一个假周期脉冲"（F13 在仿真里 精确复现签名），③ 的修法改的是**功能路径**（`angle_filt` 同时喂 `loop_filter.ang_in` 与标定滞回） ⇒ **位流必须重生。**本节位流**取代 §2.5 的 v4（767,116 B @ 22:28:24）**，是**用户需要重新烧录的那一版。** v4 生产位流与日志已另存 `_synth_work/pre_v5/` （`enc_top_v4_2228.bit + impl_build_v4.log`）。

命令： `powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 -Tcl tools\impl_check.tcl -LogFile _synth_work\impl_build_v5.log -Tail 20`

项	值（v5 实测）	对比 v4（§2.5）
bitstream	`_synth_work/enc_top.bit` **849,124 B**，时间戳 **2026-10-11 11:50:54**	767,116 B → **+82,008 B**（**不是逻辑变多**：裸位流是器件固定帧数 17,537,696 bit，差异全在压缩率，见下行）
整体 WNS / TNS / 失败端点	**−3.318** / −84.873 / **26**（`WHS +0.078 / THS 0.000`）	−3.368 / −84.834 / 26 ⇒ **略好，同一口径**
时序判据	`Timing constraints are not met.`（**26 个失败端点仍全是 ADC 输入端口**，§1.19 口径，本轮**没有**改输入约束）	一致
全局最差路径	`ad_ch1_d[2] → u_capture/ch1_meta_reg[2]/D`（slack −3.318 ns）	v4 是 `ad_ch1_d[7] →…[7]` ⇒ **同一类路径（ADC 输入），不是 v5 引入的**
Slice LUTs / Registers	**6168** = 29.65% / **3910** = 9.40%	6122 / 3892 ⇒ **+46 LUT / +18 FF**
DSPs / BRAM / IOB / BUFGCTRL	16（17.78%）/ 0 / **50** / 1	一致
DRC	`Checks found: 43`（DPIP-1 / DPOP-1 / DPOP-2 全 Warning），**0 error / 0 critical**	一致
布线	16,088 logical nets、**9581** routable nets fully routed，`nets with routing errors = 0`	v4: 9527 ⇒ **+54 nets**
压缩（自校验）	`saved 10744768 bits` ⇒ 17,537,696 − 10,744,768 = 6,792,928 bit = **849,116 B**，实测 **849,124 B**（**+8 B** 头部差，与 §2.1–§2.5 同口径 ⇒ 位流有效）	v4 压到 35.0%、v5 压到 38.7% ⇒ **+82 KB 是压缩率效应**
上板判据	日志含 `Generate Bitstream complete` + `INFO: [Vivado 12-1842] Bitgen Completed Successfully.` + **`8 Infos, 0 Warnings, 0 Critical Warnings and 0 Errors encountered.`** + `write_bitstream completed successfully`，末尾 `BITSTREAM_WRITTEN` + `IMPL_CHECK_DONE`，退出码 **0**（Vivado 12:18:46 退出）	一致（v4 当时是 `0 Warnings / 0 Critical / 0 Errors`，本轮多了 8 条 Info）

"新逻辑真的进了网表"的证据（比 §2.5 更直接）： - `_synth_work/impl_timing.rpt` 的脉冲宽度检查表里有 **`ang_held_reg/C`（`SLICE_X21Y118`）** —— 这正是 v5 新增的"最多保持一拍"寄存器，**实现后的网表里确实存在。** - 三个独立数字互相吻合：**+18 FF** = `ang_prev[15:0]` + `ang_seen` + `ang_held`（正好 18）、 **+46 LUT**（16 位减法器 + 两个 16 位比较器 + 选择逻辑）、**+54 可布线网。**

同轮仿真与检查：`tools/lint_all.ps1` = **ERROR 0 / WARNING 0**；`tools/run_sim.ps1` = **PASS 19/19** （`tb_enc_top 158s`）； `tb_enc_top` = **`TEST PASSED (99 checks)`**，其中新增 **F13**（单拍 ADC 取反被毛刺保持挡住 ⇒ `retry_cnt=0 win_bad=0`、系数逐位不变）与 **F14**（`force` 绕过保持 ⇒ `retry_cnt=3 win_bad=1`、系数仍准确 ±8 LSB/±32）——详见 §1.31 ④。

构建耗时口径（记录一下，避免下次误判"卡死"）：本轮 `synth_design` 1:20 / `opt_design` 0:09 / `place_design` 0:27 / `route_design` 0:20 —— **流程本体只有约 3.5 min**（全部报告在 11:50:4x 落盘）； 慢的是 `write_bitstream`： **`cpu 27:20 / elapsed 28:00`、峰值内存 15,994 MB**（v4 那次更慢： `cpu 31:25 / elapsed 47:18`）。整轮 **11:48:19 → 12:18:46 ≈ 30.5 min**，**期间日志会长时间不更新 （`Bitgen Completed` 之后还要等 ~27 min 才打印 `write_bitstream: Time (s)`），不要当成 hang。**

签核尾部（已落盘）： `8 Infos, 0 Warnings, 0 Critical Warnings and 0 Errors encountered.` / `write_bitstream completed successfully` / `BITSTREAM_WRITTEN` / `IMPL_CHECK_DONE` / `Exiting Vivado at Sun Oct 11 12:18:46 2026`，退出码 **0** ⇒ **v5 生产版可烧。** （注意： `enc_top.bit` 的**文件时间戳是 11:50:54**——bitgen 早早就把文件写出， 之后 27 min 是 Vivado 自己的收尾；核对时间戳时以 **11:50:54** 为准。）

待做（v5）：① 用 Hardware Manager 烧录 `_synth_work/enc_top.bit`（**烧前先核对时间戳 = 2026-10-11 11:50:54**）； ② `python tools\calib_repeat.py --port COM6 --n 20 --csv _synth_work\calib_rep_v5.csv` ⇒ 判据 **20/20** 且 **`0x0502[7]win_bad = 0`、 `0x0502[6:4]retry = 0`**（§1.31 ⑥）； ③ ILA 版需按同一份 RTL 重建（新增 `dbg_ang_glitch` / `dbg_ang_delta` 两条探针，探针总数与 `probeN` 索引会变 ⇒ 上板时**按名字**找，映射以重建后的 `_synth_work/ila_probe_list.txt` 为准）； ④ 烧 v5 后复看 ABZ 边沿抖动（毛刺不再进 `loop_filter`）。

★ v5 上板结论（2026-10-11 实测，写在 §2.6 末尾以免误读）：用户烧录 v5 后跑 20 轮， **结果 20/20 仍全对**（`OFS_S` 53…56、 `OFS_C` 354…358、 `PHASE_K` 97-98）， **但 `0x0502` 的 `retry={3:18,2:1,1:1}`、 `win_bad` ≈18/20 轮，与 v4b 相比毫无改善** ⇒ 45° 阈值没打在真凶身上，根因与 v6 修法见 **§1.32。v5 位流保留为可回退版本** （`_synth_work/pre_v6/enc_top_v5_1150.bit`，849,124 B @11:50:54）。

2.7 ★★★★ 历史基线（v6，已被 §2.8 v7 / v9 / v10 / v11 / §2 顶部 v12 取代）：§1.32 毛刺阈值 5.6° + `CONF_N=3` 确认 + 可读诊断计数器（2026-10-11 12:53:03）

> **为什么必须再重跑**：§2.6 的 v5 上板后 `win_bad/retry` **一点没改善**（见上），§1.32 ② 定位为 "真凶是**单通道半量程错值**（角度只偏 ~26°，落在 11.25° 与 45° 之间）"，③ 的修法动的是 **脉冲发生器的判定结构**（两级化 + `CONF_N=3`）与**角度保持阈值**（8192 → 1024），并新增 5c 诊断计数器与 `reg_file` 只读段 ⇒ **位流必须重生**。本节位流**取代 §2.6 的 v5 （849,124 B @ 11:50:54）**，是**用户需要重新烧录的那一版**；v5 已另存 `_synth_work/pre_v6/`。

命令：`powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 -Tcl tools\impl_check.tcl -LogFile _synth_work\impl_build_v6.log -Tail 20`

项	值（v6 实测）	对比 v5（§2.6）
bitstream	`_synth_work/enc_top.bit` **905,008 B**，时间戳 **2026-10-11 12:53:03**	849,124 B → **+55,884 B**（仍是压缩率差异，见下行）
整体 WNS / TNS / 失败端点	**−3.330** / −84.299 / **26**（`WHS +0.080 / THS 0.000`，端点总数 11253）	−3.318 / −84.873 / 26 ⇒ **同一口径**
时序判据	`Timing constraints are not met.`（**26 个失败端点仍全是 ADC 输入端口**，§1.19 口径，本轮**没有**改输入约束）	一致
全局最差路径	**`ad_ch2_d[5]`**（`Input Delay 24.000` 家族，slack −3.330 ns）	v5 是 `ad_ch1_d[2]` ⇒ **同一类路径（ADC 输入），不是 v6 引入的**
Slice LUTs / Registers	**6250** = 30.05% / **4076** = 9.80%	6168 / 3910 ⇒ **+82 LUT / +166 FF**（= 5 个 16 位毛刺计数 + 16 位脉冲计数 + 4 个窗口快照寄存器 + 16 位版本号 + 确认逻辑，量级吻合）
DSPs / BRAM / IOB	16（17.78%）/ 0 / **50**	一致
DRC	`Checks found: 43`（**11 DPIP-1 + 16 DPOP-1 + 16 DPOP-2**，全 Warning），**0 error / 0 critical**	一致
布线	16,519 logical nets（6740 无需布线）、**9779** routable nets fully routed，`nets with routing errors = 0`	v5: 9581 ⇒ **+198 nets**
压缩（自校验）	`saved 10297696 bits` ⇒ 17,537,696 − 10,297,696 = 7,240,000 bit = **905,000 B**，实测 **905,008 B**（**+8 B** 头部差，与 §2.1–§2.6 同口径 ⇒ 位流有效）	压到 41.3%（v5 38.7%）
上板判据（**先读这一条**）	日志含 `INFO: [Vivado 12-1842] Bitgen Completed Successfully.`；**≥ 12:52:5x 的报告全部落盘**；`impl_route.rpt` 显示 `0 routing errors`	一致

> ⚠️ **2026-10-11 15:51 的 v7 构建已把 `_synth_work/enc_top.bit` 覆盖为 770,600 B，且覆盖前 没有另存 ⇒ 本节这一版 v6 生产位流已不在磁盘上**（`_synth_work/` 在 `.gitignore` 内）。 要行为等价重建见 §2.8 末尾（改回三处常量，不保证逐位相同）。

同轮仿真与检查（v6）：`tools/lint_all.ps1` = **ERROR 0 / WARNING 0**；`tools/run_sim.ps1` = **PASS 19/19**（`tb_enc_top 150s`）；`tb_enc_top` = **`TEST PASSED (105 checks)`**，其中 **F13**（180° 极端情形仍被第一道 挡住、系数逐位不变）、**F14**（`force` 绕过保持 ⇒ 检测器照样报警，证明 `CONF_N=3` 没把检测器弄聋）、 **F15（v6 新增，针对真凶）**（连续 2 拍单通道 MSB 翻转 ⇒ `retry=0 win_bad=0`、系数差在 ±8 LSB/±32 内、 且诊断计数增量 `+32/+16` 与机理吻合）——详见 §1.32 ⑤。

构建耗时口径：本轮 `synth_design` → `route_design` 全部在 **12:50:3x → 12:52:55 ≈ 2.5 min** 内落盘（比 v5 更快），`write_bitstream` 的 `Time (s)` 行会**再晚约 27 min** 才打印（v5 同现象）⇒ **判据用产物（`enc_top.bit` 与 `impl_*.rpt` 的时间戳），不要等脚本退出。**

★ 上板实测（2026-10-11 13:2x，用户烧录 v6 后完成；判定全部达成）：

判据	要求	v6 实测	v5 对照	结论
`0x05F2`	= 6	**6**	0（无该寄存器）	板上确实是 v6
20 轮标定	20/20	**20/20 GOOD**	20/20 GOOD	一致
`0x0502[6:4]` retry	**= 0**	**{0:20}**	{3:18,2:1,1:1}	★ 达成（§1.31 ⑥ 未达成项）
`0x0502[7]` win_bad	**= 0**	**0/20**	18/20	★ 达成
`OFS_S`	稳定	**53…54**（极差 1、σ=0.5）	53…56（σ≈0.7）	更紧
毛刺密度	应落在 5.6°…11.25°	**0.00444 次/周期，`Δ>11.25° = 0`**	45° 档 18/20 轮全 0	★ 机理被硬件证实
窗口间隔比	≤ 1.25	**1.0006**（int 12498/12506）	——	余量极大

- 逐项读数、`diag_counters.py` 输出与结论见 **§1.32** ⑦（含 `trunc = 1` 属设计行为的证明、 `Δ保持动作 = Δ>5.6°`、累计分档 486/8/8/3 等）。
- 新增工具 `tools/diag_counters.py`（两次采样求差 ⇒ 毛刺密度 + 窗口健康度，**不依赖 ILA**）， `--selftest` = **全部通过**（10 项：差值、16 位环绕、除零、缺读、比值）。
- **对本次 RTL 的注释修正**：`rtl/enc_top.v` 5c 节头注释里的地址表原先写成 `0x0505/0x0506/0x0507/0x0508/0x050F`（漏了高字节配对、把 `0x0506` 当 11.25° 档、脉冲计数写成 `0x0509`），实测后按 `rtl/reg_file.v` 的真实映射改为 `0x0505/06`(>5.6°)、`0x0507/08`(>11.25°)、`0x0509/0A`(>22.5°)、`0x050B/0C`(>45°)、`0x050D/0E`(保持)、`0x0540`(脉冲)、`0x05F2`(build id) —— **纯注释，不改逻辑， 故 §2.7 的位流仍然对应当前 RTL，无需重综合。**

剩余（v6）：⑤ ~~复看 ABZ 边沿抖动~~ —— ✅ **2026-10-11 完成（量化定案，见 §1.33）**： 新增 `sim/tb_abz_jitter.v`（16 checks PASS），生产档 1 kHz 的抖动实测 **峰峰 80 ns = 1 个 12.5 MHz 节拍**（12/13 节拍交替，误差锯齿 0…78 ns），10 kHz 档相对抖动放大到 **81.9%**（这才是"波形乱"的真身）； 并查出 **过速时 lag_warn 全 0 且 `r_acc` 无饱和 ⇒ A/B 会短暂倒转**的监视盲区（P2；**本 v7 位流不含，已于 2026-10-11 在工作树修好，见 §1.33 ⑥ / §3 item 16**）； ⑥ ~~ILA 版若仍需要，按同一份 RTL 重建~~ —— ✅ **2026-10-11 14:06:09 已用当前 RTL 重建完毕**（`enc_top_ila.bit` **1,429,272 B**；探针 36 端口 / 269 bit，与 12:27 版一致；时序 **26 失败端点** / `WNS -3.368`，与生产版同源 ⇒ **未新增失败**；详见 §2 顶部 14:06:09 那条）。**剩下的是上板**（下载 + 加载 `.ltx` + 按名字找探针 + 触发 `dbg_ang_glitch == 1`）； ⑦ ~~进入 m02801 的 BiSS-C 哑主机联调~~ —— ✅ **2026-10-11 完成（三个探针全部上板实测通过，见**

§1.35)： `--probe A`（`s1`=GND + MA 1 MHz）40/40 `frame_active`=1、`crcerr`=0、`frame_cnt` 覆盖 1…41（20 项 B/C 判据另计）； `--probe C`（`s1` 悬空）`frame_cnt` 只到 {0,1}、`frame_active` 抖、`crcerr` 仍 0（半路复位置的是读不到的 `frame_short_err`）； `--probe B`（MA 10 MHz 受控违例）`frame_cnt` 恒 = 41、`crcerr` 粘滞 = 1 ⇒ 证明该位通路可用。**剩下的是示波器补拍手册 §6 三张图**（素材性）。

2.8 ★★★★ 历史基线（v7，已被 v9 / v10 / v11 / §2 顶部 v12 取代）：环路增益默认改纯 P（`0x0302` 复位 `0x1C → 0x04`）+ ST/MT 复位值订正为真实帧宽（2026-10-11 15:51:26）

> **为什么必须再重跑**：§1.40 把 ABZ"抖动群"的旋钮唯一化了 —— 板上 A/B 实测 `0x0302` 的 ki 由 3 降到 0，AB 边沿膨胀 **2.7×** → **1.0×**、反向点火 18 → **0**、1 kHz 跟踪残差同时变小（0.126° → 0.067°）； **唯一缺陷是掉电回默认 `0x1C`** ⇒ v7 把它做成**默认值**。 同一轮把 §2.7 遗留的 ST/MT 复位值 26/12 订正成真实帧宽 **22/10**（纯"读回/校验"口径： `biss_c_slave` 根本不消费这两个信号，帧长由综合期参数决定）。本节位流**取代 §2.7 的 v6 （905,008 B @12:53:03）**，是**用户需要重新烧录的那一版。**

命令： `powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 -Tcl tools\impl_check.tcl -LogFile _synth_work\impl_build_v7.log -Tail 30` （**不需要提权**）

项	值（v7 实测）	对比 v6（§2.7）
bitstream	`_synth_work/enc_top.bit` **770,600 B**，时间戳 **2026-10-11 15:51:26**	905,008 B → **−134,408 B**（**仍是压缩率差异**，见下行；v5→v6 的 +55,884 B 同性质）
整体 WNS / TNS / 失败端点	**−3.364** / −85.065 / **26**（`WHS +0.078 / THS 0.000`，端点总数 11,247）	−3.330 / −84.299 / 26（11,253）⇒ **同一口径**
时序判据	`Timing constraints are not met.`（**26 个失败端点仍全是 ADC 输入端口**，§1.19 的诚实约束；本轮**没有**改约束）	一致
全局最差路径	**`ad_ch1_d[7]`**（`Input Delay 24.000` 家族）	v6 是 `ad_ch2_d[5]` ⇒ **同一类路径**
Slice LUTs / Registers	**6315** = 30.36% / **4074** = 9.79%	6250 / 4076 ⇒ **+65 LUT / −2 FF**（见下注）
DSPs / BRAM / IOB	16（17.78%）/ 0 / **50**	一致
DRC	**0 error / 0 critical**（提示级违例行全是 `Related violations: <none>`）	一致
压缩（自校验）	`saved 11372960 bits` ⇒ 17,537,696 − 11,372,960 = 6,164,736 bit = **770,592 B**，实测 **770,600 B**（**+8 B** 头部差，与 §2.1–§2.7 同口径 ⇒ 位流有效）	压到 **35.2%**（v6 是 41.3%）
上板判据（**先读这一条**）	日志含 `INFO: [Vivado 12-1842] Bitgen Completed Successfully.`；`impl_util/timing/drc/route/clkint.rpt` 全部落盘（@15:51:14–15:51:18）； `impl_route.rpt` 报 0 routing errors	一致

> **注（LUT +65 的归因）**：v6→v7 的 RTL 只差**三处常量**（下条），+65 LUT / −2 FF 属 **常量折叠与布局的编译期差异**（1.04%），并叠加 §1.39 新增的 `parameter integer GATE_STATE = 0` （默认 0 ⇒ 行为等价）； **失败端点族、IOB、BRAM、DSP 全部不变** ⇒ 不是功能性差异。 要更严的口径可以把 `GATE_STATE` 显式钉成 0 重跑，**竞赛截止前不做。**

RTL 改动（三处常量 + 注释，逐条可核对）：

1. `rtl/reg_file.v:280`： `r_filt_k <= 8'h1C;` → **`r_filt_k <= 8'h04;`**（kp=4 / ki=0 = 纯 P）+ 4 行原因注释（引 §1.39/§1.40）。
2. `rtl/reg_file.v:286-287`： `r_st_bits <= 8'd26;` / `r_mt_bits <= 8'd12;` → **`8'd22` / `8'd10`**（= `rtl/enc_top.v:104-105` 的 `ST_BITS_L/MT_BITS_L`）+ 3 行注释。
3. `rtl/enc_top.v:107-112`： `RTL_BUILD_ID = 16'h0006` → **`16'h0007`**（顶部注释 :33-35 同步列出 v7 两项内容）； 文档表 `rtl/reg_file.v:74`（FILT_K 默认 `0x04`）、`:86-87`（ST/MT 22/10）与注释 `:176` / `:535`（v7 = 7）同步。
4. 测试台同步： `sim/tb_reg_file.v:183` 默认 `cfg_ki` 3 → **0**、`:190` `cfg_st_bits` 26 → **22**、`:191` `cfg_mt_bits` 12 → **10**（全仓确认无其它硬编码； `sim/tb_biss_c_slave.v:26-27` 用自己的 26/12 做**模块级**通用测试，**有意为之，不动**）。
5. 工具版本提示放宽为"≥6"： `tools/biss_frame_probe.py:242`、`tools/diag_counters.py`（自检夹具 6→7）、`tools/ila_capture.tcl:23` / `:72`（`tools/calib_repeat.py:298` 本来就是 `>= 6`）。

同轮仿真与检查（v7）： `tools/lint_all.ps1 -Clean` = **ERROR 0 / WARNING 0**； `tools/run_sim.ps1` = **PASS 21 / 21 @2026-10-11 15:48:50**（`tb_enc_top` 157 s，其余 3–5 s）。

构建耗时口径与一个复现现象： `synth → route` 在 15:51:18 前落盘， `write_bitstream` 于 **15:51:26** 完成； **父进程在 `Bitgen Completed Successfully.` 之后仍挂住不动**（16:08 仍 running） ⇒ 已 `job_kill` 释放。**与 §1.36/§2 里 ILA 构建的挂死是同一现象 ⇒ 判据永远是产物时间戳， 不要等脚本退出。**

ILA 版（v7，2026-10-11 16:22:42）： 用同一份 v7 RTL 重建完成 —— `_synth_work/enc_top_ila.bit` **1,164,160 B @16:22:42**、`.ltx` **71,337 B @16:22:33**、`ila_probe_list.txt` @16:21:22 （**36 端口 / 269 bit**，与 v6 定版一致 ⇒ 探针布局没变）、`impl_ila_{timing,util,drc}.rpt` @16:22:30–16:22:33。

项	v7 ILA 版	v6 ILA 版（14:06:09）
整体 WNS / TNS / 失败端点	**−3.338 / −84.954 / 26**（端点总数 20,158； `WHS +0.053`、`WPWS 8.870 / 0 失败 / 9,864`）	−3.368 / −84.979 / 26（20,155）
bitstream / 压缩自校验	1,164,160 B； `saved 8224480 bits` ⇒ 17,537,696 − 8,224,480 = 9,313,216 bit = **1,164,152 B** + 8 B = 1,164,160 ✓	1,429,272 B（同口径）
资源	LUT **9189**（44.18%）/ FF **8943**（21.50%）/ BRAM **7.5**（15%）/ DSP **17** / IOB 50	9189 / 8942 / 7.5 / 17 —— **基本一致**

项	v7 ILA 版	v6 ILA 版（14:06:09）
DRC	62 checks，**全是 Warning**（DPIP-1 24 + DPOP-1 17 + DPOP-2 17 + PDCN-1569 3 + RTSTAT-10 1），0 error / 0 critical	一致

- 构建同样**父进程挂死**（`Bitgen Completed Successfully.` 之后不再输出）⇒ 已 `job_kill`； **判据是产物时间戳。** `tools\ila_debug.tcl` 必须一次性 `danger-full-access`（§1.36）。
- **v6 时代两版 ILA 位流已另存在** `_synth_work/pre_ila_v6/`（1,409,344 B @12:27 与 1,429,272 B @14:06 + `_README.txt`）。

上板实测（v7，2026-10-11 16:2x，已做）：用 `ILA_MODE=restore` 把 v7 生产位流烧上板 （`_synth_work\ila_restore_v7.log`： `End of startup status: HIGH`、`ILA_EXIT=0` @16:23:21）， 随后逐条读回（`tools/uart_spi.py`）：

读	结果	判据
`0x05F2`	**0x0007**	= 7 ⇒ **板上确实是 v7** ✓
`0x0302`	**0x0004**	纯 P（kp=4 / ki=0）⇒ **开机即生效、不靠主机写** ✓（v6 上同样读回 0x04 是运行期写的、掉电即失）
`0x0500`	byte0 = **0x16 = 22**	ST 位宽订正生效 ✓
`0x0501`	**0x000A = 10**	MT 位宽订正生效 ✓
`0x05F0`	**0x100C**	DEVICE_ID 0x0C / REV 0x10 未变 ✓

> 读回帧的 `status` 字节是 **0x08 = `otr`**（8 Vpp 接近满量程的既有现象，`0x0405 = 0x0001` 可清， 见 §1.23/§1.38 ④，不影响本节判据）。

★ 自由运行窗口实测（v7，2026-10-11 16:26:21，已做完 —— 这一条把 §1.40 的"运行期写"结论升级成"默认值"结论**）**： `$env:ILA_MODE="stage1"` 烧 `_synth_work/enc_top_ila.bit`（**烧完即全复位 ⇒ 下面这版是在 "没有任何主机写寄存器"的纯净状态下抓的**），触发 `dbg_adc_valid`（立即触发）⇒ `_synth_work/ila_cap_v7free.csv`（134,738 B）、日志 `_synth_work/ila_cap_v7free.log`。 **烧录后立刻读回默认值**：`0x05F2` = **7**、`0x0302` = **0x0004**、`0x0103` = **0**、 `0x0101` / `0x0102` = **0** ⇒ **纯 P 确实是开机默认，不依赖主机写**（这正是 v7 相对 §1.40 的全部意义）。

量	v7 默认纯 P（本次，自由运行）	v6 + 运行期写 `0x0302=0x04`（§1.40，`ila_cap_gainki0.csv`）	v6 默认（ki=3，`ila_cap_gain1c.csv`）
输入	r = **1636.9 ± 2.2 LSB（7.99 Vpp）**，f_in **0.987 kHz**	1616.1 ± 2.2（7.89 Vpp），0.993 kHz	1593…1631（7.78…7.96 Vpp），≈1 kHz
理想 AB 跳变	20.7	20.8	20.8
AB 实测跳变 / 放大倍数	**21 / 1.0×** ✅	20 / 1.0×	**57 / 2.7×**
`edge_cnt` 净 / 增-减	**+21（21 / 0）** ✅ 零反向点火	+20（20 / 0）	+21（39 / 18）—— 正反成对抵消
间隔中位 / min / max	48 / 48 / 52 clk（直方图 `{48:14, 52:6}`，理想 48.83 ⇒ **纯 1 拍量化**）	48 / 44 / 52（`{48:12, 52:6, 44:1}`）	12 / 4 / 76
角度输入侧抖动	`|Δangle|>64 LSB` **0/1023**、方向翻转 **0**	0/1023、0	**151 处**、翻转 **51 次**
跟踪精度	`angle` 线性残差 RMS **0.009°**（max −0.023°）、vs `atan2` 残差 RMS **0.055°**； `angle_raw` RMS 0.040°	0.012° / —	0.126°
`dbg_ang_glitch` / `lag_warn` / `lag_fatal` / `limiter`	**全 0**	全 0	全 0

⇒ **v7 的纯 P 默认值在"全复位后自由运行"条件下把 AB 边沿膨胀从 2.7× 压到 1.0×、反向点火归零、 间隔只剩 1 拍量化，跟踪残差同时更小**（0.126° → 0.009°）。复现命令见 §4.y。

上板判据复核（v7，`tools/uart_spi.py`）： `read 0x05F2` = **7** ✅、`read 0x0302` = **0x04** ✅ （**开机即纯 P**，不用再写）、`read 0x0500` = **22** ✅、`read 0x0501` = **10** ✅ —— 见上面那张读回表； **上板前若换了板子/位流，照这四条先复核对。**

> ⚠️ **一个不可逆的小损失（如实记录）**：**v6 生产位流（905,008 B @12:53:03）在本次构建前 没有另存就被覆盖**，而 `_synth_work/` 全在 `.gitignore` 内 ⇒ 磁盘与 git 里都不再有这一版 （全工作区搜 905,008 B 的 `.bit` = **0 个**）。要**行为等价**重建需把上面第 1–3 条常量改回， 且**不保证逐位相同**（v6 之后 `rtl/loop_filter.v` 多了默认 0 的 `GATE_STATE`）。 v5 生产位流仍在 `_synth_work/pre_v6/`（849,124 B @11:50:54）。 **下一次重综合前先建 `_synth_work/pre_v8/` 并把它复制进去再动手。**

1.72 ★★★★ 上位机界面优化批次 2 落地：每张卡片都能「⛶ 放大」独占整窗，还原后几何逐值复原（2026-10-13，界面自检 42 → 52 项）

- **触发**：用户 6 条界面需求（m06024）中的第 6 条「每一个小窗口部分都可以点击进行放大成全软件界面，以显示更加详细的内容」； 本轮 = `docs/12_host_ui_requirements.md` 的 **批次 2（R6，窗口内独占）**。硬约束同批次 1（不新增第三方依赖、界面全中文、离线/真机自检保持全绿、`export_png.py` 的 1950×1080 / 1950×990 像素契约不变）。
- **做法（不重新 parent、不动布局结构）**： `host/ui_kit.py` 新增 `MAX_BTN_IDLE = "⛶ 放大"` / `MAX_BTN_ON = "⛶ 还原"` / `MAX_BTN_WIDTH = 74`、 `_TitleDblClick`（标题双击事件过滤器）、`_find_title(lay, title)`（**递归**找到标题标签所在的那层布局与序号）、`_has_stretch(lay)`、 `install_maximize(card)`（返回 objectName `CardMax` 的 `QToolButton`）； `make_card()` 末尾自动安装该按钮。标题分布有两种情形都要管：① 标题直接在卡片竖向布局里（`make_card()` 造的卡片）⇒ 新建「标题 + 弹簧 + 按钮」水平行插回原位；② 标题已在一条水平标题行里（如波形卡片的「暂停刷新」那行）⇒ 该行尾补弹簧 + 按钮。事件过滤器实例挂在 `card._max_watcher`，否则被 GC 后双击标题失效。 `host/theme.py` 新增 `QToolButton#CardMax` 样式（透明底、hover 变主色描边）。
- **独占实现**（`host/main_window.py: _setup_card_maximize()` / `_enter_card_max()` / `_restore_card_max()` / `_toggle_card_max()` / `resizeEvent()` + `_card_canvases()` / `_card_title()`）： 进入时先记下每张卡片的 `(isVisibleTo(self.scroll), minimumHeight, maximumHeight, sizePolicy)` 与卡内每个 matplotlib 画布的 `(min, max, policy)`； 然后**藏其它卡片**、把本卡 `setMinimumHeight(max(420, 可视高-120))`、把画布那 250 px 固定高临时解开

（`setMaximumHeight(16777215)` + `Expanding` 策略，初始高 `max(240, (可视高-260)*0.46)`）；按钮文本变 `⛶ 还原`，状态栏提示「按 Esc 或再点「⛶ 还原」回到整页」。还原时按记下的值**逐项恢复**（画布 `updateGeometry()` + `draw_idle()`）。`Esc` 用 `QShortcut(QKeySequence(Qt.Key_Escape), self)` + `setContext(Qt.ApplicationShortcut)`；窗口缩放时 `resizeEvent()` 只在本卡独占时按新视口高度重算最小高。

- **验收（离线界面自检 42 → 52 项，全 PASS exit 0）**：新增第 ⑭ 节共 10 条 —— 11 张卡片标题行**全部**有「⛶ 放大」按钮（缺按钮：无）；放大 ⑧ 波形卡后可见卡片 = **1**（仅本卡）、卡片高 **814 px / 可视 878 px**、画布 **250 → 657 px**、按钮文本 `⛶ 还原`；按 Esc 后 **11 张全回来**、按钮回 `⛶ 放大`、**卡片几何（可见性/最小高/最大高/实际高）与放大前逐值一致**、画布 **657 → 250 px**；双击 ⑨ ABZ 标题等效于点按钮（放大的是 ⑨）。
- **★ 踩坑（已修）**：首版 `install_maximize()` 只扫卡片布局的**直接 item**，而 panel 自造卡片的标题在嵌套的水平标题行里 ⇒ 波形卡片拿不到按钮（`wave btn = False None`）；改成递归 `_find_title()` + 两种情形分开处理后 11 张全覆盖。另一处：`QShortcut` 在 **`PySide6.QtGui`** 而非 `QtWidgets`，写错报 `ImportError: cannot import name 'QShortcut' from 'PySide6.QtWidgets'`。
- **回归证据**：`python host\selftest_ui.py` **52/52 PASS（exit 0）**；`python -m compileall host` exit 0；批次 1 的 ⑪ 记录定格 / ⑫ 日志级别 / ⑬ 滚轮三节未受影响（仍全绿）；`export_png.py` 像素契约与 S5 判据未动。
- **文档**：`docs/12_host_ui_requirements.md` R6 标题改「✅ 批次 2 已实现」、批次表勾掉批次 2、新增「批次 2 实施记录」表与「实现要点与踩坑」；`host/README.md` 自检条数 `selftest_ui.py` 42 → **52 项**、新增界面说明第 13 条「放大任意一张卡片」（⛶/双击标题/Esc）、并在「事实 7 固定几何」里补一段说明：放大是唯一会临时改变几何的操作，退出后逐值复原（由界面自检 ⑭ 断言）。

1.76 ★★★★ 上位机界面优化批次 3 落地：波形/ABZ 图变成"示波器式可调" + Ctrl+滚轮 + 参数记忆 + 按当前视图导出（2026-10-13，界面自检 52 → 73 项）

- **触发**：用户 6 条界面需求（m06024）的第 2、3 条（「输入信号波形」「ABZ/UVW 等效采样」都要能像真实示波器一样自由调量程、并把横纵轴与单位标清楚）。本轮 = `docs/12_host_ui_requirements.md` 的 **批次 3**：R2 + R3 + Ctrl+滚轮缩放 + `host/ui_prefs.json` 参数记忆 + 「按当前视图导出」。硬约束同批次 1/2（不新增第三方依赖、界面全中文、离线/真机自检保持全绿、`export_png.py` 的 1950×1080 / 1950×990 像素契约不变）。
- **共享交互（新增 `host/ui_kit.py::ScopeNav`）**：把 `canvas.installEventFilter(self)` + `canvas.setProperty("wheelZoom", True)`（让 `main_window` 的全局滚轮过滤器对图表**让路**）封成一个类；面板按**鸭子协议**提供视图状态 `nav_axes()/x_span()/set_x_span()/x_center()/set_x_center()/reset_auto()`（可选 `y_pan()/set_y_pan()`）。Ctrl+滚轮以**光标为中心**缩放（`ZOOM_STEP = 0.8`，`factor = 0.8 ** (dy/120)`，先改跨度再回推中心，保证光标下的数据点不动）、左键拖动平移、双击图内 `reset_auto()`；`_px_ratio()` / `_frac()` 用 `ax.bbox` 与画布尺寸换算（高 DPI 下逻辑像素 ≠ 显示像素）；整个 `eventFilter` 包 try/except，交互出错绝不把异常冒进 Qt 事件循环。
- **R2（`host/panel_wave.py`）**：`X_SPANS_MS = (0.5, 1.0, 2.0, 5.0, 10.0, 20.0, 50.0, 100.0, 200.0, 300.0)`（原来最小 5 ms，20 kHz 输入会显示 100 个周期 —— 正是用户抱怨"方波太密"）、`Y_FULLS = (1.0, 0.5, 0.25, 0.1)`、`AUTO_CYCLES = 2.0`。控件行：`cmb_tdiv`（自动（跟随输入）/ 各档满屏 ms）、`cmb_vdiv`（自动 / ±x.xx FS）、`cmb_layout`（叠加/分离）、`btn_auto`（恢复自动）、`btn_export_view`（导出当前视图）+ `lab_view`（"量程：…（手动）"）。`_auto_span_ms()` 按 `1000/f_in × 2` 算完**量化到档位**（否则轴标签每帧都在跳）；`_y_lim()`：手动 = `FS×y_full`、自动 = `FS×1.12`；分离排布把 `s_n` 上移、`c_n` 下移 `0.55×lim`；`reset_auto()` / `set_x_span()` 都 `up.put("wave", …)` 落参数。轴标题改「时间 (ms，0 = 最新帧；左负 = 更早)」「幅值 (Q1.15，满刻度 32768)」。
- **R3（`host/panel_abz.py`）**：`X_SPANS_DEG = (10, 30, 90, 180, 360)`、`GAP_BASE = 1.1`、`SPACING_RANGE = (0.6, 2.5)`；控件 `cmb_xspan`（自动（整圈 360°）/ 各档满屏 °）、`cmb_layout2`、`sp_gap`（通道间距）、`btn_auto2`、`btn_export_view2` + `lab_view2`。`_chan_offsets()`：`gap = GAP_BASE×spacing×(2.0 if split)`、`oa/ob/oz = 2×gap/gap/0`、`ylim = (-0.3+y_off, 2×gap+1.2+y_off)`；`_init_axes()` 按窗口设 `xlim`、把 y 刻度标签设为 A/B/Z（UVW 图 W/V/U）、x 轴标签「电角度 (°)」、**UVW 的 60° 分隔线只在整圈（自动）时画**（手动放大到 10° 时它会全在画面外）；`_redraw()` 不再提前 return（无数据也把窗口与量程画出来，否则空数据时量程控件看起来"没反应"）。
- **参数记忆（新模块 `host/ui_prefs.py`，自带 `--selftest` 13 项 PASS）**：`DEFAULTS` 四段 `window{w,h}` / `log{level}` / `wave{x_mode,x_span_ms,y_mode,y_full,layout}` / `abz{x_mode,x_span_deg,center_deg,layout,spacing}`；API `load()/save()/data()/section()/raw()/number(lo,hi,default)/choice(allowed,default)/put()/window_size()/_merge()`（丢弃未知键、错类型回落、数值夹取、NaN 判掉）；`put()` 只改内存，落盘由主窗口关窗时做。窗口尺寸夹取 `MIN 900×600 / MAX 6000×4000`。`host/main_window.py` 开窗 `up.window_size()` + 日志级别、`_save_prefs()`（**先 `_restore_card_max()`** 再存 `w/h` 与日志级别，保证放大状态下存的是整页几何），异常只记一条 `debug` 日志。`host/selftest_ui.py` 把 `up._path_override` 指到临时目录，**自检绝不读写用户的 `host\ui_prefs.json`**。
- **按当前视图导出**：`host/export_png.py` 的 `export_wave_png/export_abz_png/export` 新增 `view=` 参数（wave 用 `view["xlim"]/["ylim"]` 并按 `ylim` 半宽自动判"分离"排布做上下平移；abz 用 `view["offsets"]/xlim/ylim`，60° 分隔线只在 `xlim` 宽度 ≥359° 时画），meta 加「量程」行、标题与文件名带「（当前视图）」（`suggest_name` 同步）；`host/panel_record.py::finish_export()` 认 `*_view` 并写「（当前视图）」；`host/main_window.py` 把 `wave.export_view_requested` / `abz.export_view_requested` 接到 `_on_export("wave_view"/"abz_view")`，在 `_on_export()` 里对 `*_view` 取该面板的 `scope_view()` 传下去。
- **验收（离线界面自检 52 → 73 项，全 PASS exit 0）**：新增第 ⑮ **示波器式量程 11 条**（自动横轴 20 kHz → 满屏 0.500 ms；「满屏 20 ms」→ `xlim=-20…0`；「±0.25 FS」→ `ylim=-8192…8192`；Ctrl+滚轮 20 → 16 ms 且**页面滚动条不动**；双击回自动；ABZ「满屏 90°」→ `xlim=135…225°`；`sp_gap=2.0` → A/B/Z 基线 4.40/2.20/0.00、纵轴 -0.30…5.60；分离 → -0.30…10.00；ABZ Ctrl+滚轮 90 → 72°；双击回 360°）、⑯ **参数记忆 5 条**（改量程/间距/日志级别/窗口后重开窗口逐值恢复 `wave 50 ms / abz 90° / 间距 1.6 / 日志 debug / 窗口 1000×700`；参数文件写坏 ⇒ 回落 `wave=auto log=info abz=overlay` 且不崩）、⑰ **按当前视图导出 5 条**（`_on_export("wave_view")` 落盘 `编码器波形当前视图_*.png` 201 kB / 1950×1080、`abz_view` → `编码器ABZ-UVW当前视图_*.png` / 1950×990、全貌 `wave` 仍不带「当前视图」、记录卡片提示含「当前视图」）。
- **回归证据**：`python host\selftest_ui.py` **73/73 PASS（exit 0）**；`python -m compileall host` exit 0；离线模块自检全绿 —— encoder 10、abz 15、**export_png 12**（新增 [12]「按当前视图导出」：像素契约与全貌一致、用非白像素数证明视图生效 1307277 → 1311492 / ABZ 763489 → 763810、空数据 + 视图不崩）、recorder 13、signal 22、regs 2043 组 + 5 用例、ui_prefs 13；真机 `selftest_link.py --port COM3 --seconds 5` PASS、`selftest_s3.py --port COM3 --seconds 5` PASS、`selftest_ui.py --real-port COM3 --seconds 6` PASS（界面取 60 954 帧 = 记录器写 60 954 行、文件 65 105 行 = rows、队列 1 → 0）、`selftest_s4.py --port COM3` PASS（11 项通过 + 1 项跳过，见下）；批次 1/2 的 ⑪⑫⑬⑭ 四节未回归。
- **★ 一个真 bug（被自检抓到）**：`host/export_png.py` 里 `ylim` 原来只在 `if n:` 分支里赋值 ⇒ 空数据 + `view=` 导出抛 `cannot access local variable 'ylim' where it is not associated with a value`；已把 `ylim` 的计算提到分支外。
- **★ 真机 S4 的 A→B 相位差在 100 kHz 信号源下不可测（不是相位算法回归）**：频率阶梯跑到 §1.75 后信号源是 200 kHz（本次实测 99 999.5 Hz / 199 998.97 Hz），`inter=32` ⇒ AB 输出 3.2~6.4 MHz（周期 312~156 ns ≪ 帧周期 100 µs）：探针把每箱 A/B 平均值打出来是 **256/256 箱全为 0.50**（正常方波几乎全为 0/1），而 UVW（= f_in 量级）仍能分辨。同一份 `host/abz.py`（mtime 18:30，未随批次 3 改动）在 19 999.901 Hz 下实测 **−90.0°**、在 1 kHz 下也过关，离线自检 15 项里那条「A→B 相位差 ≈±90°」也照旧 PASS。**新增能力**：`host/abz.py` 的 `snapshot()` 增加"电平与角度箱相关"判据 `ab_muddy`（被填

箱里 A/B 平均值落在 0.05~0.95 的比例；不相关时 `phase_ab_deg` 置 None 并给中文提示），`host/selftest_s4.py` 把相位项在该工况下**明确跳过**（结果行打印「另有 1 项因采样简并跳过」），不再"测"出一个假的 0.0° 相位差。要看相位请把信号源调回 20 kHz 以内。

- **★ 顺手修掉的误导性报错**：`host/link.py::_open_error_text()` 原来先匹配 `"could not open port"`，于是端口被占用（`PermissionError(13, '拒绝访问。')`）被报成「找不到这个串口（板子被拔了？换一个口试试）」——本轮真踩过（上一脚本句柄未释放），差点去查硬件。现在按**内层 errno** 分派（errno 13 / permissionerror / already open / busy ⇒ "被别的程序占着"；errno 2 / filenotfounderror ⇒ "找不到这个串口"；只认得 `could not open port` 时给折中文案），并且 `SerialWorker.run()` 打开失败**重试 3 次（间隔 0.6 s）**，覆盖"上一个脚本刚退出、句柄还没释放"的常见情形。
- **文档**：`docs/12_host_ui_requirements.md` R2/R3 标题改「✅ 批次 3 已实现」、批次表勾掉批次 3、新增「批次 3 实施记录」与「实现要点与踩坑」（含 S4 简并说明与串口报错修复）；`host/README.md` 自检条数 `selftest_ui.py` 52 → **73 项**、`export_png.py` 11 → **12 条**、文件表新增 `ui_prefs.py` 行、界面说明第 8/9 条补"像示波器一样调"与 Ctrl+滚轮/拖动、第 10 条补「导出当前视图」、第 12 条去掉"随批次接入"、新增第 14 条「界面参数会记住」；`.gitignore` 加 `host/ui_prefs.json`。

1.77 ★★★★★ 单文件 exe 交付：查清"沙箱里 exe 起不来"的根因（受限令牌 / `_MEI` ACL）+ 揪出并修掉"打包版启动即崩"的 `signal` 遮蔽标准库 bug + onedir 版在沙箱内端到端验证通过（2026-10-13）

- **触发**：用户 m07281「我现在用 tools\build_host_exe.ps1 重新打包一版」→ 用户 m07427「将引导器建的 `_MEI*` 目录 ACL 带入沙箱令牌 SID，需要什么权限和我申请即可」→ 用户 m07468 选择「要，现在打 onedir 并验证」。
- **发现旧包是"界面优化之前"的**：`host\dist\host_app.exe`（2026-10-13 20:42:50、218,345,402 B）的 PYZ 里**没有** `ui_prefs` 模块，`main_window` 也不含 `wheelOwn` / 清空日志 / 显示级别 / `CardMax` ⇒ 用户 m06024 的六条抱怨正是旧 exe 的行为。核对手法（沙箱内不必运行 exe）：`PyInstaller.archive.readers.ZlibArchiveReader(r"host\_exe_work\host_exe\PYZ-00.pyz")` 看 `toc`，再 `marshal.dumps(z.extract(mod))` 按 **UTF-8 子串**搜字符串常量（方法名只在 co_names 里，必须全量字节搜）。
- **`ui_prefs` 冻结路径 bug（本轮修）**：`path()` 原用 `__file__` ⇒ 单文件 exe 里落在 `%TEMP%\_MEIxxxx`（退出即删），参数记忆等于失效。改为 `_home_dir()`（`sys.frozen` / `sys._MEIPASS` ⇒ `os.path.dirname(sys.executable)`），自检 13 → **14 项**（新增「冻结时参数文件落在 exe 旁边」）。
- **★★ 真 bug：打包版启动即崩（`host/signal.py` 遮蔽标准库 `signal`）**。onedir 版首次运行 stderr：`File "main_window.py", line 376, in _build_ui / File "panel_alarm.py", line 66, in __init__ / AttributeError: module 'signal' has no attribute 'ALARM_ROWS'`。源码模式下 `app.py` 把 `host/` 插到 `sys.path[0]` 才碰巧 import 到自己的 `signal`；PyInstaller 的 PYZ 里同名模块只有一份，**标准库赢**（核对：`PYZ-00.pyz` 的 `signal` 含 `valid_signals`、不含 `ALARM_ROWS`）⇒ 单文件 exe 同样崩。修法：`git mv host\signal.py host\alarm.py` + 6 处 `import alarm as sg`（`host/panel_wave.py:21`、`host/panel_alarm.py:18`、`host/main_window.py:48`、`host/export_png.py:58`、`host/selftest_s3.py:38`、`host/selftest_s5.py:46`）+ 显示文字/注释同步（`host/alarm.py`、`host/selftest_s3.py:9` / `:68`、`host/export_png.py` 三处）。回归：`python host\alarm.py --selftest` **22 项 PASS**、`python host\selftest_ui.py` **73 项 0 失败**、`python -m compileall host` exit 0。
- **onedir 版（新增 `host/host_onedir.spec`）**：与 `host/host_exe.spec` 同 `HIDDEN` / `EXCLUDES` / `pathex`，`EXE(..., exclude_binaries=True)` + `COLLECT(..., name="host_app")`；`python -m PyInstaller --noconfirm --clean --workpath host\_exe_work_onedir --distpath host\dist_onedir host\host_onedir.spec`（约 66 s，产物目录 556.8 MB）。**运行时零解包、不建 `_MEI` ⇒ 沙箱内可跑**，于是"打包产物"第一次能在沙箱内端到端验证：
- 测试 1（离线）：`--smoke host\_onedir1.png --seconds 4` ⇒ 截图成功：…（1166x2597）、81,651 B，且 **exe 旁边生成 `ui_prefs.json`**（冻结路径修复端到端生效）。
- 测试 2（连板）：`--port COM3 --smoke host\_onedir2.png --seconds 10 --record --record-dir host\_onedir_rec --export` ⇒ stdout `8.8 秒 / 89,896 帧 / 13.9 MB（10235 帧每秒）/ 1 个文件 / 丢帧估计 0.77%`、截图 124,142 B；记录目录 `编码器记录_20261013_224117.csv` 14,549,677 B、`encoder_wave_20261013_224117.png` **1950×1080**、`encoder_abz_uvw_20261013_224118.png` **1950×990** ⇒ frozen 下的 `tools/uart_scope`、记录器、导出与像素契约全部通。
- **★ "沙箱里单文件 exe 起不来"的根因（用户问的权限问题）**：用 `ctypes` + `advapi32!ConvertStringSecurityDescriptorToSecurityDescriptorW` + `kernel32!CreateDirectoryW` 做 ACL 实验 —— 用户 SID `S-1-5-21-1767783162-4259348043-3762101017-1001`；工作区 ACL 上的沙箱能力 SID `S-1-4-658417623-361858394:(W,D,DC)`（会话 TEMP 上是**另一个** `S-1-4-190198983-876675268-1:(OI)(CI)(W,D,DC)`）。用引导器同款 SDDL `D:(A;;FA;;;<用户SID>)` 建目录后写入 **`errno=13`**；追加能力 SID `W,D,DC`（`0x0013009F`）⇒ 写入成功；`D:(...)(A;;WD;;;sandbox)` / `Everyone` 都不行 ⇒ **沙箱令牌是写受限（write-restricted）令牌，DACL 里只认能力 SID。**而引导器给的 DACL 会**阻断继承**（`D:(A;;FA;;;user)` 与 `D:P(...)` 都不带出父目录 ACE），目录名还是随机的 ⇒ **没有任何文件夹级授权能救**：`--runtime-tmpdir` 指到工作区仍 0 文件；`_MEIPASS2` 复用解包目录在引导器字节里不存在（`find(b'_MEIPASS2')` 无命中）。**唯一解法是令牌层豁免**（以非写受限/免沙箱令牌启动该命令）—— `danger-full-access` 只改文件白名单、令牌照旧受限，故无效。⇒ 日常流程保持"沙箱内 `-NoSmoke` 出产物 + 用 onedir 版做产物级验证 + 单文件 exe 在沙箱外烟测"。
- **本轮产物**：`host\dist\host_app.exe`（单文件）与 `host\dist_onedir\host_app\`（目录版）均已用**改名后**的源码重打；单文件包内逐项核对通过（`ui_prefs` 在位、`_install_wheel_router` / `CardMax` / 显示级别 / `scope_view` / 导出当前视图 / 已停止 / 满屏 `%g ms` / 满屏 `%g°` / 冻结时参数文件落在 `exe` 旁边 等子串命中）。
- **沙箱杂项教训**：① 沙箱里**没有 `pwsh`** ⇒ 只能用 `powershell -ExecutionPolicy Bypass -File …`；② `Add-Type` 编译 P/Invoke 会因本机 `LIB` 指向不存在的 VS/Win SDK 路径而失败 ⇒ 用 `ctypes`；③ `tempfile.mkdtemp()` 建出的目录**既写不进也删不掉**（`PermissionError: [Errno 13]`，`Remove-Item` / `icacls /grant` 都无效）⇒ 临时目录一律在 `host\` 下自建编号目录；④ **`Get-PSDrive C` 在本沙箱谎报可用空间 0.0 GB**（真值用 `New-Object System.IO.DriveInfo 'C'` 或 Python `shutil.disk_usage`），我据此误杀过一次 30 分钟长稳。
- **文档**：`host/README.md` 的「打包成单文件 exe（S6）」一节新增第 7 条坑（模块名不得与标准库重名）、补写"受限令牌 write-restricted + 能力 SID"的根因与 onedir 验证路线（含实测数字）、进度表 S6 行换成最新包的数字与结论；文件表 `alarm.py` 行写明改名原因、`ui_prefs.py` 行写明 exe 旁落盘；`docs/12` 与 sect 1.76 的 `signal.py` 提法在本节统一为 `alarm.py`（历史小节保留原名）。

1.78 ★★★★ 单文件包"双击打不开"：给打包版装上启动日志与现场诊断助手（用户机器上可自证）+ 重建单文件包（2026-10-13）

- **触发**：用户 m08093（原文）：「这个软件可以打开 `C:\Users\zhaoy\Desktop\FPGA\host\dist_onedir\host_app\host_app.exe`，这个打不开 `host\dist\host_app.exe`，我已经删除」——目录版可用、单文件包**双击无任何反应**且已被用户删除。
- **为什么不能在我这边复现**：单文件包在 DSH 沙箱里必然"打不开"，根因已在 §1.77 定案（受限令牌 write-restricted，引导器自建 `_MEI*` 的 DACL 只给用户 SID、不给沙箱能力 SID ⇒ `Failed to extract … Permission denied`，文件夹级授权无解）。所以本轮不再纠结复现，而是**让用户机器自己说出卡在哪一层。**
- **`host/app.py`：打包版启动日志（本轮新增）**。冻结时日志落 `<exe 旁>\host_app_run.log`（`_home_dir()` = `os.path.dirname(sys.executable)`，与 `ui_prefs.py` 同规则；>256 KB 轮转 `.1`），内容：启动时刻/argv/exe 路径/是否冻结/解包目录/CWD/Python/系统版本、三处目录的**真写盘测试**（`_write_probe()` 写一个 tmp 再删）、**逐个 import 的结果**（`PySide6.QtWidgets`、`QtCore`、`recorder`、`theme`、`main_window`、`alarm`、`uart_scope`）、建窗耗时、未捕获异常与 traceback。写日志失败一律吞掉（绝不因日志把程序弄崩）。

- **`--diag` 子命令（在 import Qt 之前拦截）**：`if "--diag" in sys.argv[1:]: _diag_report(); raise SystemExit(0)` —— Qt 装不上时这条命令最有价值；源码模式会把 `<repo>\tools`、冻结模式把 `<exe 旁>\tools` 插进 `sys.path`（否则源码模式必误报 `import uart_scope` 失败，这是本轮踩到的坑）。
- **启动失败不再静默**：Qt/`recorder`/`theme`/`main_window` 的 import 全部包 `try/except BaseException` ⇒ 写日志 + 弹窗 + `SystemExit(3)`；`win.show()` 之后写一行「已建窗显示（耗时 %.1f s）」；`__main__` 再包一层运行期异常处理（写日志 + 弹窗 + exit 4）。弹窗优先 Qt `QMessageBox`，Qt 不可用时退到 Win32 `MessageBoxW`，再不行 print。⇒ **以后用户报"打不开"，一行读日志就能定位层次**：
- 日志文件**根本不存在** ⇒ 没跑到 Python，问题在解包/加载层（权限、杀软、TEMP、文件不完整）。
- 有「启动」没有「已建窗显示」⇒ 卡在解包或依赖加载（配合日志末尾的 import 清单）。
- 有「已建窗显示」⇒ 程序已起来，问题在后面的运行期（日志里有 traceback）。
- **诊断助手（新增 `host/diag.py` + `host/host_diag.spec`）**：一个只用标准库的小 exe（**8,593,678 B / 8.2 MB**，2026-10-13 23:35:52，SHA256 `FD10C7ECCD088D518096A9FA244FAD2464D1DF6FEAB7D9C537233D7B8D5E69B3`，console=True 带黑窗，`python -m PyInstaller --noconfirm --clean --workpath host\_exe_work_diag --distpath host\dist host\host_diag.spec`，约 30 s）。它做：目录可写性（exe 旁/TEMP/TMP/CWD 真写测试）、磁盘余量（`shutil.disk_usage`）、**Mark of the Web**（读 `path:Zone.Identifier`）、exe 完整性（MZ 头 + 大小；主程序按 `APP_MB = 208.3` 当期望值，偏小或头不对就标出"多半没复制完整"）、扫 `%TEMP%\_MEI*`（**目录在但 0 文件 = 解包被挡住的签名**）、最后 `subprocess.run([同目录的 host_app.exe, "--diag"], timeout=180)` 试跑真程序并**回读那份启动日志尾部 40 行**（默认读**被查 exe 旁边**的日志，找不到才退回自己旁边 ⇒ 诊断助手放别处也能用，`--app` 指定被查程序），全部写进 `诊断报告.txt`。⇒ 用户只需把两个 exe 放同一目录、双击诊断助手、把报告发我。
- 实测（本沙箱）：`mei_report()` 立刻列出 `_MEI000016002`、`_MEI000028882` **两个 0 文件目录**，与 §1.77 的受限令牌结论完全对上 ⇒ 这条判据可信。
- **★★ 顺手又揪出一个真 bug（同类"编码把工具弄崩"）**：拿一个**带 UTF-8 BOM** 的日志喂诊断助手时，它打印日志第一行的 `\ufeff` 撞上 GBK 管道 ⇒ `UnicodeEncodeError: 'gbk' codec can't encode character '\ufeff' in position 1488: illegal multibyte sequence` ⇒ **工具自己崩掉、连 `诊断报告.txt` 都写不出来**（"诊断工具本身出故障"是最不能接受的一种故障）。修法两处：① `host/diag.py` 新增 `fix_streams()`，`main()` 一进来就把 `sys.stdout/stderr` 重设为 `encoding="utf-8", errors="replace"`（真控制台上 Python 用 `_WindowsConsoleIO`，本来就是 UTF-8 ⇒ 中文照样正常；被重定向到管道时输出 UTF-8 而不是本地码页）；② 读日志改用新增的 `read_tail()`（`encoding="utf-8-sig"`，BOM 直接被吃掉）。同一坑在 `host/selftest_ui.py` 已经踩过一次（GBK 控制台打 ⍟ U+246A），本轮把它在 `host/app.py` 也堵上：新增 `_say()` 安全打印（流为 `None` 或编码不配合都只放弃这一次打印），`_alert()` 的兜底 print 与 `_diag_report()` 的 `print(text)` 都改走 `_say()`。
- **诊断助手的目录版（新增 `host/host_diag_onedir.spec` ⇒ `host\dist_diag\host_app_诊断\`，18.8 MB / 59 个文件）**：单文件诊断包自己也是单文件包 —— 万一故障正是"解包被挡"，它自己也会解不开，用户就什么报告都拿不到。目录版零解包，最坏情况照样能出报告（`python -m PyInstaller --noconfirm --clean --workpath host\_exe_work_diag_onedir --distpath host\dist_diag host\host_diag_onedir.spec`，约 25 s）。
- **沙箱内端到端验证了整条诊断链路（这是本轮唯一能做的验收）**：用目录版诊断助手查 `host\dist\host_app.exe` ⇒ 报告如实写出三件事：`_MEI000016002`/`_MEI000028882` 两个 **0 文件**解包目录；试跑 `--diag` **等满 180 s 未退出**（窗口版引导器弹了模态框）；`host_app_run.log` **两个位置都没有**。回归测试里换成一个伪装的单文件 exe 后，报告里直接抓到了引导器的原话：`fopen: Permission denied` + `[PYI-21468:ERROR] Failed to extract entry: LIBBZ2.dll.` ⇒ **"解包被挡"这类故障能被这份报告钉死。**
- **回归（app.py/diag.py 改动后全跑一遍）**：`python -m compileall host` exit 0；`encoder`/`abz` 15 项/`export_png`/`recorder`/`alarm` 22 项/`regs`/`ui_prefs` 14 项/`selftest_ui` 73 项**全部 exit 0**；`python host\diag.py --no-run` exit 0 且写出 `诊断报告.txt`。
- **顺手修掉一个真问题**：`host/selftest_ui.py` 的 `main()` 补 `sys.stdout/stderr.reconfigure(encoding="utf-8", errors="replace")`（其他自检脚本早有此约定）。GBK 控制台（chcp 936）下打印标题里的 ⍟（U+246A）会 `UnicodeEncodeError: 'gbk' codec can't encode character '\u246a'` 直接打断自检；修后 `QT_QPA_PLATFORM=offscreen python host\selftest_ui.py` = **73 项 0 失败 exit 0**。
- **重新打包（含启动日志）**：`powershell -ExecutionPolicy Bypass -File tools\build_host_exe.ps1 -NoSmoke` exit 0 ⇒ `host\dist\host_app.exe` **218,401,905 B / 208.3 MB @2026-10-13 23:40:11**、SHA256 `315CD23E3C5CCB3335CEC0359F1566D2692848E9B29452A95E3F0696609DFD3C`、**222 s**（含最后那次 `_say()` 安全打印修复的重打）；引导器 `runw.exe`（console=False）、manifest 已嵌。★ 该包**尚未**在沙箱外验证（沙箱内必然起不来，见 §1.77）。
- **静态核对包内容（沙箱内唯一可行的验收手段）**：`host/_check_pyz.py`（临时脚本）用 `ZlibArchiveReader(PYZ-00.pyz)` 查 680 个模块 —— `alarm`/`ui_prefs`/`main_window`/`panel_alarm`/`panel_wave`/`panel_abz`/`export_png`/`recorder`/`theme` 全部在位，`signal` 是标准库那份（`valid_signals` 命中、`ALARM_ROWS` 不命中），`main_window` 含 `_install_wheel_router`/`CardMax`/`显示级别`/`清空日志`。**★ 踩到一个方法坑**：**入口脚本不在 PYZ 里**（PYZ 的 `toc` 里既没有 `app` 也没有 `__main__`），它在 **PKG(CArchive)** 里、名字就是文件名 `app`（诊断助手是 `diag`）⇒ 要核对入口脚本新增的字符串必须用 `PyInstaller.archive.readers.CArchiveReader(r"host\dist\host_app.exe")` 再 `extract("app")`。实测：主包入口脚本 18,868 B、含 `host_app_run.log`/`_diag_report`/`已建窗显示`/`--diag`/`_write_probe`/`_env_text`；诊断助手入口脚本 16,693 B、含 `诊断报告.txt`/`_MEI`/`Zone.Identifier`/`host_app.exe --diag`。（不要拿源码里的 `raise SystemExit(3)` 这种**语句文本**去搜字节码 —— 编译后只剩常量 `3`。）
- **给用户的三条"这不一定是 bug"**：① 单文件包**首次**双击要解包 200 MB+，慢 3~10 s（SSD 上；老盘/杀软扫描下更久），期间"像卡住"；② 杀软/Win11「受控文件夹访问」拦引导器往 `%TEMP%` 解包（典型现场 = `%TEMP%` 里留下 **0 文件**的 `_MEI*`）；③ 复制不完整（该有 208.3 MB，实际偏小就是没拷全）。
- **文档**：`host/README.md` 新增「★ 双击了却"没反应"时怎么查」小节（日志判读规则 + `--diag` 用法 + 诊断包构建命令 + 上面三条"不是 bug"），文件表新增 `diag.py` 行、改写 `app.py` 行，坑数由"六个"改"七个"；`.gitignore` 增补 `host/_exe_work_onedir/`、`host/_exe_work_diag/`、`host/dist_onedir/`、`host/host_app_run.log(.1)`、`host/诊断报告.txt`。
- **待办**：等用户在**沙箱外**跑一次诊断助手/新包，用 `诊断报告.txt` 或启动日志定位"打不开"的真实层次；若确属杀软/权限，备选交付是**目录版 + ZIP**（用户已确认目录版能开）。

补充（2026-10-13 深夜）：用户真机证据到齐 —— 当时判为"受限令牌"（★ 该结论已被下面「补充 2」推翻，保留原文以存档推理过程） - 用户反馈（m08504/m08505/m08506）：双击 `host\dist\host_app.exe` 弹出标题 `Error`、正文 **`Could not create temporary directory!`** 的对话框；把**目录版**诊断助手放到 `host\dist_diag\host_app_诊断\` 里双击，**跑通了**并写出 `诊断报告.txt`；「还是只有这一个能打开 `host\dist_onedir\host_app\host_app.exe`」。 - **这份报告（32 行，2026-10-13 23:43:56）就是判据**：`冻结: True`、`TEMP = TMP = C:\Users\zhaoy\AppData\Local\Temp`；`exe 目录 … 可以写`、**`TEMP … 失败: [Errno 13] Permission denied: …\host_app_`**`写测试.tmp`、`工作目录 … 可以写`；exe 目录里只有 1.7 MB 的诊断助手（MZ ok）；`%TEMP%` 里 **21 个 `_MEI*`，每个 101 文件 / 28.4 MB**。我核对 `%TEMP%` 的 ACL：`NT AUTHORITY\SYSTEM`、`BUILTIN\Administrators`、`ROG\zhaoy` 三条**完全控制**、owner `ROG\zhaoy` ⇒ **有完全控制却写不进去 = 令牌被限制**，不是文件夹权限问题。 - **时间线判据**：`%TEMP%` 里最新的 `_MEI*` 是 **2026/10/11 10:55 与 10/12 11:07**（每个 101 文件、owner `ROG\zhaoy`），**当天（10/13）一个都没建成** ⇒ 10/11–10/12 那几次单文件启动**解包是成功的**（当时进程身份正常），今天每次启动都死在"建解包目录"这一步 —— 同一份 exe 行为随"谁启动它"而变。 - **我在沙箱里复现了同样的错误**（临时 `host\_probe_temp.py`，已删）：我的 `TEMP/TMP` 是会话目录 `…\Temp\dsh-6061qh`（可写），但往**用户 TEMP 根**写同名测试文件得到**逐字相同**的 `PermissionError: [Errno 13] Permission denied`，建目录得到 `[WinError 5] 拒绝访问`；`whoami /user` = `rog\zhaoy`，SID `S-1-5-21-1767783162-4259348043-3762101017-1001`。 - **`host/diag.py` 新增「运行身份」判据**（本轮代码）：`dir_probe(d)`（`os.mkdir("_MEI_建目录测试")` 再删 —— 正是引导器卡的第一关）、`restricted_sid_count()`（`OpenProcessToken(TOKEN_QUERY)` + `GetTokenInformation(TokenRestrictedSids=11)` **只读那个 DWORD 计数**，不碰 SID 数组指针）、`token_info()`（`whoami /user` 取 SID、`whoami /groups` 扫 `S-1-4-…` 能力 SID 与 `S-1-16-…` 完整性级别，受限数 >0 或存在能力 SID ⇒ 判词

「★ **受限令牌**（write-restricted）…单文件包必报 `Could not create temporary directory!`」，否则「普通用户令牌（未受限）」）、`parent_chain(pid=None, depth=5)`（Toolhelp32 `CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS)` / `Process32FirstW` 打启动链）、`_run_quiet()`（`CREATE_NO_WINDOW`）。报告新增段落「--- 运行身份（谁启动的? 能不能写 %TEMP%? ）---」与 `TEMP 建目录 / exe 目录 建目录` 两行。★ 踩坑：第一版手撕 `TOKEN_GROUPS` 的 SID 数组（`GetSidSubAuthorityCount/GetSidSubAuthority` 未声明 prototype ⇒ 返回指针被截断）导致进程 `0xC0000005`（exit `-1073741819`）—— 改成"只读计数 + 显式 `argtypes/restype`"后正常。实测本沙箱输出：`令牌完整性：S-1-16-4096`、`受限 SID 数：4（★ >0 = 令牌被限制…）`、`能力 SID：无`（whoami 看不到，它们只在 `TokenRestrictedSids` 里）、`判断：★ 受限令牌`。 - ★ **`--runtime-tmpdir` 实验（临时 spec `host/host_diag_tmpdir.spec`，已删）**：`runtime_tmpdir="."` ⇒ `_MEI000052d42` 确实建在**当前工作目录**里（以 `host\dist_tmp` 为 CWD 跑就在那儿生成了），但解包**依旧失败**：`[PYI-21204:ERROR] Failed to extract LIBBZ2.dll: failed to open target file! + fopen: Permission denied + Failed to extract entry: LIBBZ2.dll.` ⇒ **受限令牌新建的目录自己也继承不到写权限**（引导器显式 DACL + 不继承父 ACE），所以 `runtime_tmpdir` /文件夹级授权/换盘都救不了；**目录版不需要建新目录，所以只有它能开。**★ 结论定稿：**exe 本身没问题，问题在"谁启动它"**；正解是**用普通身份启动**（开始菜单开 `cmd` /PowerShell 敲完整路径、`Win+R` 输入完整路径、或在自己开的资源管理器里双击）—— 修好后诊断助手判词应变成「受限 SID 数：0 / 普通用户令牌（未受限）」。 - 附带：这次实验在**工作区内**留下一个删不掉的 `host\dist_tmp\_MEI000052d42`（及空父目录 `host\dist_tmp`，`Remove-Item` 报「对路径的访问被拒绝」），与先前 9 个沙箱垃圾目录一并请用户在沙箱外删。 - **重建含运行身份判据的两个诊断助手**：`host\dist\host_app_诊断.exe` **8,601,288 B @2026-10-13 23:48:14** SHA256 `50852D6EEFBCBE16047D53BAAE040AA53C1D55E5C15C79179A40CF1523584C7A`；`host\dist_diag\host_app_诊断\host_app_诊断.exe` **1,819,753 B @23:48:39** SHA256 `E2A03657FE23A97DA81660E8B558B7E29C1D4285913EF8F4EB4596B7DADEB89E`。主程序未变（23:40:11 那版）。 - **文档**：`host/README.md` 排查小节新增「★ 最先怀疑"谁启动的它"：受限令牌」（含两条铁证、判词、三种"用普通身份启动"的做法、`--runtime-tmpdir` 为什么救不了）；§1.77 那节的沙箱说明补上 GUI 版的原话 `Could not create temporary directory!` 与"用户手上就是这台机器"的对应关系。

补充 2（2026-10-13 23:5x，终审）：★ 推翻"受限令牌"说 —— 是这台机器的 `%TEMP%` 对本程序不放行；交付改为"本地解包版 + 目录版" - **转折点**：我问"你双击单文件包时那个资源管理器窗口是从哪儿打开的"，用户答（m08633）「从任务栏/桌面图标打开的」—— 也就是说**双击的进程不是 DSH 沙箱启的。**随后用户用**目录版诊断助手**（含刚加的「运行身份」判据）重新出了一份报告（m08632 截图，`host\dist_diag\host_app_诊断\诊断报告.txt`，2026-10-13 23:52:18），关键的几行是：`账户：zhaoy 用户 SID：S-1-5-21-1767783162-4259348043-3762101017-1001`、`令牌完整性：S-1-16-4096`、`受限 SID 数：0`、`能力 SID（S-1-4-…，沙箱标志）：无`、`判断：普通用户令牌（未受限）`、`启动链：host_app_诊断.exe(33452)`（只有自己一个进程 ⇒ 父进程已退出，正是资源管理器双击的样子）。 - `exe 目录 … 可以写`、`TEMP … 失败：[Errno 13] Permission denied: 'C:\Users\zhaoy\AppData\Local\Temp\host_app_写测试.tmp'`、`TMP 同样失败`、`工作目录 … 可以写`、`TEMP 建目录 … 失败：[WinError 5] 拒绝访问。: '…\Temp\_MEI_建目录测试'`、`exe 目录 建目录 … 可以建子目录`、磁盘 `24.1 GB / 487.1 GB`、21 个 `_MEI*`（各 101 文件 / 28.4 MB，最新仍是 10/12 11:07）。 ⇒ **普通用户令牌、零受限 SID，`%TEMP%` 依旧写不进也建不了目录，而 exe 所在目录两样都行** ⇒ 与"谁启动它"无关，**是这台机器上 `%TEMP%` 对本程序不放行。** - **我把 ACL 之外的可能性逐条排除**（都干净）：`icacls %TEMP% = NT AUTHORITY\SYSTEM / BUILTIN\Administrators / ROG\zhaoy 三条 (I)(OI)(CI)(F)`；`Get-Acl` **无 Deny**、全部 inherited、Owner = `ROG\zhaoy`；`C:\Users\zhaoy`、`…\AppData`、`…\AppData\Local` 同款；ACL 里那个 SID 经 `[SecurityIdentifier]::Translate` 核实 = `S-1-5-21-…-1001` = 当前用户；`fsutil reparsepoint query` → `Error 4390: The file or directory is not a reparse point.`（不是链接/VHD 重定向）；`attrib` 无特殊属性、无 `Encrypted` 属性；目录里**没有**我们探测留下的 `*写测试*` / `*建目录测试*` / `_dsh_probe*`；TEMP 共 463 个条目；`Get-CimInstance Win32_LogicalDisk` 空、`fsutil quota query C:` → `Error 5: Access is denied`（要管理员，**未决**）；文件系统过滤驱动**只有微软自带**（UCPD/Filetrace/WdFilter/FileInfo/Wof/FileCrypt/UnionFS/CldFlt/WIMMount/FltMgr/storqosflt/bindflt/FsDepends/bfs/wcifs/luafv），未发现第三方杀软进程；`Get-MpComputerStatus` / `Get-MpPreference` 在沙箱令牌下**拒绝访问** ⇒ 受控文件夹访问等 Defender 设置**查不了**，需要用户本人在"Windows 安全中心 → 病毒和威胁防护 → 保护历史记录"里看。 - **那 21 个 `_MEI*` 不是我们的**：`_MEI61242` 里是 `AgentCore-Light-BLE-Helper.exe`（9,351,765 B）+ `pywin32_system32` / `win32` / `winrt` / `yaml` / `web_ui` ⇒ 另一个 PyInstaller 单文件程序留下的；最新时间戳 10/12 11:07 ⇒ **10/12 之后这台机器上没有任何单文件程序解包成功过。** - ★ **`runtime_tmpdir` 语义实测（决定交付形态的那一句）**：给 `host/host_diag.spec` 加 `RUNTIME_TMPDIR = os.environ.get("HOST_DIAG_RUNTIME_TMPDIR") or None` 与 `EXE_NAME = os.environ.get("HOST_DIAG_NAME") or "host_app_诊断"`（`host/host_exe.spec` 加同款 `HOST_EXE_RUNTIME_TMPDIR` / `HOST_EXE_NAME`），用 `HOST_DIAG_RUNTIME_TMPDIR="."` 打出 `诊断_tmpdot.exe`（8,601,894 B）后**把 CWD 设成不可写的 `C:\Windows` 再运行** ⇒ `[PYI-32220:ERROR] Could not create temporary directory!`，**exe 旁边什么都没建** ⇒ **相对路径的 `runtime_tmpdir` 是相对"当前工作目录（CWD）"，不是 exe 所在目录**（PyInstaller 把它作为 OPTION 项 `pyi-runtime-tmpdir .` 原样写进 CArchive，见 `PyInstaller\building\api.py:562-563`）。所以：**资源管理器双击（CWD = exe 所在目录）时"本地解包"成立**；从别的 CWD 启动、或快捷方式"起始位置"设成别处，都不成立。附带核实：该 exe 二进制里有 ASCII 串 `_MEIPASS`、`pyi-runtime-tmpdir`，但**没有** `_MEIPASS2` ⇒ "用 `_MEIPASS2` 复用已解包目录"这条旁路在 PyInstaller 6 单文件包里不存在。 - **据此新增交付（本轮产物）**：`host\dist_local\host_app_本地解包.exe` **218,401,916 B @2026-10-13 23:59:58**（`$env:HOST_EXE_RUNTIME_TMPDIR="."; $env:HOST_EXE_NAME="host_app_本地解包"; python -m PyInstaller --noconfirm --clean --workpath host\_exe_work_local --distpath host\dist_local host\host_exe.spec`）与 `host\dist_local\host_app_诊断_本地解包.exe` **8,602,367 B**（同款变量、`host_diag.spec`）。主程序 `host\dist\host_app.exe`（23:40:11 那版）与目录版 `host\dist_onedir\host_app\`（用户已确认能开）**未动。这两个本地解包版未在沙箱外验证**（沙箱里本地解包同样写不进去，见 §1.77）。 - **待办**：① 请用户在自己开的 cmd 里跑 `mkdir %TEMP%\_t1 && echo hi > %TEMP%\_t1\a.txt && rmdir /s /q %TEMP%\_t1` —— 能建 = 只有本程序被拦（疑杀软/受控文件夹访问），不能建 = 整机 `%TEMP%` 坏了；② 查安全中心"保护历史记录"；③ 沙箱里删不掉的残留目录累计约 12 个（含 `host\dist_tmp\_MEI000052d42` 及其父目录）需在沙箱外删。 - **文档更正**：`host/README.md` 把"最先怀疑谁启动的它：受限令牌"整段改写为「★ "解包这一步就死了" —— 先怀疑 `%TEMP%` 写不进去」（含两条互相印证的事实、ACL 排除清单、"受限令牌曾被当成根因、后来被推翻"、自证 cmd 命令、CWD 语义、本地解包版构建命令），`:57` 的判读条目、`docs/DEV_STATUS.md` 的 §1.77 段「用户手上就是这台机器」一句、进度表 S6 行同步更正与补数字。 - **★★ 交付形态定案（用户 m08788：「不管了，就用 `C:\Users\zhaoy\Desktop\FPGA\host\dist_onedir` 里面的软件吧，不搞其他的了」）**：正式交付 = **目录版** `host\dist_onedir\host_app\host_app.exe`（556.8 MB 目录、零解包、用户亲测能开；整个 `host_app\` 目录一起拷，双击里面的 exe）。`host\dist\`（单文件）、`host\dist_local\`（本地解包版）、`host\dist_diag\`（诊断包）降级为**备用/实验产物。** `%TEMP%` 对本程序不可写一事**按用户意愿不再追查**（可自测的 cmd 判据与"安全中心保护历史记录"检查已写进 `host/README.md` 留档）。

1.79 ★★★★ 上位机界面优化批次 4a 落地：放大态"排版 / 滚轮 / 挡位步进 / 快捷键 / 全屏"（2026-10-14，界面自检 73 → 95 项）

- **需求与拍板**：用户 `m08815` 提三条（放大后的界面要重排、放大后不需要"全界面滚轮滚页"、放大后设置挡位要更细更顺手），并明确**先沟通需求、确认后再做。**我先做现状取证（离线 `offscreen` 1600×900 实测）、把方案整理进 `docs/12_host_ui_requirements.md` 的「## 批次 4」并五问挑一；用户 `m08918` 五项**全部选"推荐"**：Q1 = **B 打底 + A 放大专属**、Q2 = **滚轮缩放横轴 + `Ctrl`+滚轮 微调**、Q3 = **只做"挡位加密 + ◀▶ 步进按钮 + 键盘快捷键"**（无级微调/自定义显示、数字直填、显示方案预设本轮不做）、Q4 = **全屏与放大态换卡片都要**、Q5 = **收起提示文字 + 读数加大字号。**
- **现状取证（放大态的实测数字，也是用户抱怨的实质）**：卡片确实独占整窗（内容高 878 = 视口高 878 ⇒ 滚动范围 **0**，无残留滚动条，R6 契约成立），但 `host/main_window.py` 的 `_route_wheel()` 仍**无条件** `return True` 吃掉滚轮 ⇒ **放大态滚轮等于失效**（实测整页时裸滚轮发到波形画布 页面 0 → 72 px，放大 ⑧ 后三种发法都是 0 → 0）；读数格 2×3 被"拉宽铺满"（左列 x≈30…140、右列 x≈798…910 ⇒ 中缝 ≈660 px）；⑧ 两子图 ≈630/≈430 且**中缝 ≈240 px**；控件全挤左上（卡片内 x=53…596）而「导出当前视图」孤零零在 x=1431；挡位是粗档 `QComboBox`（时基 10 档 / 幅值 4 档 / 角度 5 档），无步进按钮、无快捷键、无微调。
- **落地（改 5 个文件，全部走既有自检）**：

- `host/theme.py`：新增 `QToolButton#StepBtn` 样式（含 `:hover` / `:pressed` / `:disabled`）。
- `host/ui_kit.py`：新增 `STEP_BTN_WIDTH=24` + `step_button(text, tip)`（`objectName="StepBtn"`、`setFocusPolicy(Qt.NoFocus)` 免得抢方向键）；`ScopeNav` 新增 `FINE_STEP=0.98` 与 `set_plain_wheel_zoom(on)`（放大态裸滚轮 = `ZOOM_STEP=0.8` / 120 单位，`Ctrl+滚轮 = 0.98` 微调）。
- `host/panel_wave.py`：`X_SPANS_MS` 10 → **17 档**（0.5/1/1.5/2/3/5/7.5/10/15/20/30/50/75/100/150/200/300）、`Y_FULLS` 4 → **7 档**（1.0/0.75/0.5/0.35/0.25/0.15/0.1）；新增 ◀▶（时基）与 `-+`（幅值）四个 `step_button` 夹住两个 combo、`step_x()/step_y()`（整档走档位表，`fine` 走 `FINE_STEP=0.9`）、`set_y_full()`、`kbd_pause()/kbd_export()`、`_sync_step_btns()`（到端点置灰）；`lab_hint.setProperty("maxHide", True)`；读数格 `setColumnStretch(2, 1)`；子图改 `fig.add_gridspec(1, 2, width_ratios=(2.0, 1.0))`。
- `host/panel_abz.py`：`X_SPANS_DEG` 5 → **11 档**（5/10/20/30/45/60/90/120/180/270/360）；◀▶ 步进按钮；`sp_gap` 改 `setDecimals(2)` + `FINE_GAP=0.05` 微调；`step_x()/step_y()`；读数格 `setColumnStretch(2, 1)`、六状态表 `tbl.setColumnStretch(6, 1)` + `setColumnMinimumWidth(3, 36)`（两组紧挨、中间 36 px）。
- `host/main_window.py`：新增 `_setup_keys()`（17 条 `QShortcut`，`Qt.WindowShortcut`）、`_on_focus_changed()`（焦点在 `QLineEdit` / `QAbstractSpinBox` / `QPlainTextEdit` / `QTextEdit` 时**全部让键**）、`_kbd_panel()/_kbd_note()/_kbd_step_x()/_kbd_step_y()/_kbd_reset()/_kbd_pause()/_kbd_export()`、`_toggle_fullscreen()`（记住 `isMaximized()`，退出还原）、`_cycle_card_max(delta)`（先 `_restore_card_max()` 再 `_enter_card_max(下一张)`）；`_route_wheel()` 重写成"放大态图表让路 / 放大态非图表吃掉 / 普通页原契约"；`_enter_card_max()/_restore_card_max()` 增加 R11（提示折叠 + 读数 34/27/14 px，逐值存 `_max_ui_saved` 并逐值退回）与 R8（开关 `_plain_zoom`）。
- **验收证据**：`python host\selftest_ui.py` 新增第 ⑱ 节 **22 项 ⇒ 95/95 PASS（exit 0）**（原 73 项全部保持，R1 滚轮契约与 R6 几何逐值复原契约均未回归）；`python -m compileall -q host` exit 0；离线自检全绿：`encoder` PASS、`abz` 15、`export_png` PASS、`recorder` PASS、`alarm` 22、`regs` PASS、`ui_prefs` 14；`python host\app.py --smoke host\_b4_smoke.png --seconds 3` ⇒ 截图成功 1586×2597（88,100 B）。⑱ 里的关键数字：档数 17/7/11、步进按钮 6 个且样式类 `StepBtn`、`10 ms → 7.50 ms`、微调 `9.000 ms`、端点 `◀=False`、`1.0 → 0.75 FS`、`←` ⇒ 7.50 ms 且状态栏「时基：横轴 7.5 ms（手动）· 纵轴 手动」、`Shift+←` ⇒ 6.750 ms、`R` ⇒ `x=auto y=auto`、数字框聚焦时按 `←` 量程不动、放大态裸滚轮 `10.00 → 8.00 ms`、`Ctrl+滚轮 ×0.9800`、滚轮落在视口上量程与页面都不动、`PgDn` ⇒ 放大卡片变「⑨ ABZ / UVW 等效采样」、`F` 进/出全屏、⑧ 子图 `(2.0, 1.0)`、⑨ 六状态表 `columnStretch(6)==1`。**版式实测（离屏 1600×900，放大卡片 1554×814）**：⑧ 读数从"中缝 660 px"变为全部落在 x≈14…580、子图 834/417 px 且中缝 240 → **138 px**；⑨ FieldName 列起点从 `[30, 370, 790, 1130]` 变为 `[14, 254, 280, 320, 378]`。
- **★ 本轮新增的"取证方法"教训（下次别再踩）**：① **量放大态几何必须让事件循环跑一轮（600 ms）再量**——在 `_toggle_card_max()` 同一次调用里立刻 `repaint()+grab()+量几何`，量到的是**重排前的中间态**（ABZ 卡片 y=1052、滚动范围 0，截图看着像"放大后错位"的假 bug）；② **别用 `mapTo(win, ...)` 量嵌套卡片里的控件**（`btn_export_view` 报 x=2878，而卡片只有 1554 宽），以**卡片内** `geometry()` 为准（实测 x=1431 正常）；③ 读数 `objectName` 有 `BigValue`（34 px）/ `BigValueSmall`（27 px）两档，只判前者会漏；⑨ 六状态表是 `QGridLayout` **不是** `QTableWidget`，自检只能 `findChildren(QGridLayout)` 后看 `columnStretch(6)`；④ 方向键必须"让键"而不是"吞键"（否则用户在数字框里按 `←` 会同时改量程和光标位置）。
- **文档**：`docs/12_host_ui_requirements.md` 追加 R7–R11 条目 +「### 批次 4a 实施记录」表 + 实现要点与踩坑；`host/README.md` 第 8/9/12/13 条改写、新增「### 键盘快捷键一览」表、文件说明的 `main_window.py` / `ui_kit.py` / `theme.py` 行、进度表新增「界面·批次 4a（✅ 已完成）」与「界面·批次 4b（⏳ 待做）」。
- **待做**：**批次 4b** = R7-A（放大态把控制条 + 读数竖排进右侧固定 280 px 一列，图表占满左侧剩余宽高）；串口/板子在位时补一轮真机 S3/S4 复验。

1.80 ★★★★ 上位机界面优化批次 4b 落地：放大态版面重排成"左图右控"（2026-10-14，界面自检 95 → 105 项）

- **需求**：批次 4 的 Q1 选的是「B 打底 + **A 放大专属**」，4a 只做了 B（读数不拉伸、子图 2:1、状态表分组）；4b 做 A —— 放大态把「控制条 + 量程行 + 读数」整块搬到**右侧固定 300 px 一列**，图表独占左侧剩余宽高。用户诉求原文见 `m08815`（"放大后的界面UI需要进行优化，调整布局排版"）。
- **落地（4 个文件）**：
- `host/panel_wave.py`：顶层 `lay` 由 `QVBoxLayout` 改 **`QGridLayout`**（`host/panel_wave.py:77` 起，margins 14/12/14/12、hSpacing 10、vSpacing 8、`setColumnStretch(0, 1)`）；新增模块常量 `SIDE_MAX_W = 300`、`READOUT_CELLS`（普通态六个读数的格子位置）；控制控件按组登记 `self._ctrl_groups = (grp_tdiv, grp_vdiv, grp_layout, grp_auto, grp_export)` 并塞进 `self._ctrl_grid`；新增容器 `self.side = QWidget()`（竖排 `_ctrl_grid` → `lab_view` → 读数格）；新增 `_fill_ctrl(vertical)`（横排 = 一行、竖排 = 每组一行，每次都先 `takeAt(0)` 清空）与 `set_max_layout(on)`（`layout.removeWidget(side/chart)` 后搬到 `(2,1)/(2,0)` 或 `(2,0,1,2)/(3,0,1,2)`，同步改 `setRowStretch`）。
- `host/panel_abz.py`：同样改造（控制组 `(grp_xspan, grp_layout, grp_gap, [btn_auto2], [btn_export_view2])`；`lab_states` 与六状态表留在底部整宽 `(4,0,1,2)` / `(5,0,1,2)`）。
- `host/main_window.py`：`_enter_card_max()` 加 `getattr(card, "set_max_layout", None)(True)`、`_restore_card_max()` 加 `...(False)`（鸭子型，与 `set_plain_wheel_zoom` 同一套写法）。
- `host/ui_kit.py`：`field()` 里把整行布局挂到返回值上（`v._row_lay = row_lay`），供放大态把读数从 2×3 改**一行一条**的竖排（右列 300 px 放不下两列：名字 76 px + 数值 `setMinimumWidth(150)` ⇒ 两列要 ≈464 px，实测会被截成"板上 r / 上位"）。`STEP_BTN_WIDTH` 24 → **26**；`host/theme.py` 的 `QToolButton#StepBtn` 改成 15 px 加粗、24×24 起（原来 13 px 太细，`-` / `+` 在放大态看不清）。
- **验收证据**：`python host\selftest_ui.py` 新增第 ⑲ 节 **10 项 ⇒ 105/105 PASS（exit 0）**（原 73 + ⑱ 22；R1 滚轮契约与 R6「还原后卡片几何与放大前逐值一致」均未回归 —— 见下面的踩坑）；`python -m compileall -q host` exit 0。**离线实测（1600×900 offscreen，放大卡片 1554×811）**：⑧ 图表 `1216×759 @x14` + 右列 `300×759 @x1240`、`lab_hint` 不可见；⑨ 图表 `1216×675` + 右列 `300×675`；还原后右列宽回 897（⑧）/1526（⑨）、图表回其下方且高度回 250/240 px。截图（用 `QT_QPA_FONTDIR=C:\Windows\Fonts` 才出中文）见 `host\_b4b_5波形放大.png`、`host\_b4b_6ABZ放大.png`。
- **★ 踩坑（本轮最值得记的一条）**：`_reflow_readouts(False)` 第一版写的是"按目标格子 `itemAtPosition(r, c)` 把旧项删掉再放新项"，**在放大态会摘错行** —— 放大态是竖排，目标格子里放的是**别的**读数行，于是有的行被摘掉、有的行留下残影，普通态布局比基线高 32 px，自检 ⑭ 的 R6 契约直接 FAIL（103/104）。改成 `idx = g.indexOf(v._row_lay)` 按值反查再 `removeItem` 后逐值复原（`图 y=195` 与基线一致）。**教训：两态重排要"按控件反查"而不是"按格子删"，并且每次先清空再重排（保证幂等）。**
- **其它教训**：① 右列 300 px 容不下两列读数 ⇒ 放大态必须竖排；② `QGridLayout` 里搬同一个 `side` 容器比拆装 `QVBoxLayout` 稳，但**必须同步改 `setRowStretch()`**，否则图表长不高；③ `host/recorder.py --selftest` 的性能项见下一条（已改成同机自校准）。
- **顺手修掉 `recorder` 性能自检的"假警报"（2026-10-14，同一轮）**：第 8 项原来要求 `rate > 150000` 行/秒（历史记录 20.3–21.1 万行/秒）。这一项量的是**纯 CPU 的逐行字符串格式化**：本机实测 `_row()` 单行 11.8 µs、整条记录路径 3.7–5.6 万行/秒，且 `time.process_time()` 与墙钟几乎相等（⇒ 进程拿到 100% CPU，不是被别的进程抢），同机裸盘顺序写仍有 203–218 MB/s ⇒ **不是回归，是固定绝对门槛在这台机器上不成立**（同一份未改动的 `host/recorder.py`）。改法（`host/recorder.py:666-695`）：先量同机"纯 `_row()` 格式化上限"（实测 5.4–9.3 万行/秒），要求记录器 **≥ 上限的 35%**（三轮实测比值 0.60 / 0.81 / 0.68），并保留 **2 万行/秒的绝对硬底线**（实机 10 kHz 只要 1 万行/秒 ⇒ 2 倍余量）；自检会打印三个数（记录器行/秒、上限、比值）与门槛。**已知局限（如**

实记）：本机单是格式化就要 12 µs/行 ⇒ "2 倍以内的格式化退化"此项测不出（试验：每行多拼 512 字符 ⇒ 3.3 万行/秒、比值 0.64，仍 PASS）；它能抓住的是"每行 fsync/flush 之类的落盘退化"（比值掉到 0.1 以下）与"≥2.7 倍的格式化退化"（跌破 2 万底线）。

- **文档**：`docs/12_host_ui_requirements.md` R7 改为"4a 打底 + 4b 增强，两小批全部落地"、新增「### 批次 4b 实施记录」表与踩坑、版式对照表补两行；`host/README.md` 第 13 条补"放大后版面重排"段、进度表「界面·批次 4b」改 ✅、文件说明的 `panel_wave.py` / `panel_abz.py` / `ui_kit.py` 行更新。
- **交付包同步重打（2026-10-14 00:58）**：`python -m PyInstaller --noconfirm --clean --workpath host\_exe_work_onedir --distpath host\dist_onedir host\host_onedir.spec` ⇒ `host\dist_onedir\host_app\host_app.exe` **7,988,975 B**、SHA256 `0ABF35DF8799CDCC598225D1DA8C9A421075AC02DC67FA1970E74E95675AD19B`、目录 426 文件 / 583,869,114 B（556.8 MB）。**包内复核**（`CArchiveReader(exe).extract("PYZ.pyz")` → `ZlibArchiveReader`，680 模块）：`panel_wave` 命中 `set_max_layout` / `_reflow_readouts` / `SIDE_MAX_W`，`main_window` 命中 `set_max_layout` / `_setup_keys` / `_plain_wheel_zoom`，`ui_kit` 命中 `_row_lay` / `step_button`，`recorder` 命中 `fmt_rate` / `ratio`（同机自校准那道门槛）⇒ 本轮所有改动确实进包。★ 本轮**没能在沙箱里跑这个包**（`Start-Process -Wait` 卡住且不产截图，沙箱对 GUI 进程的启动/等待不可靠）—— 交付前仍建议用户在机器上直接双击一次。
- **待做**：串口/板子在位时补一轮真机 S3/S4 复验；批次 4 的 ③④⑤（无级微调 + 自定义显示、数字直填、显示方案预设）用户明确本轮不做。

1.81 ★★★★★ 用户接上板子后的实机全面复查：S1–S5 全 PASS + 10 分钟长稳 + GUI 跑图，修掉 3 个真问题（2026-10-14）

> 用户原话（`m09768`）：「我现在已经把FPGA板子接上了，可以进行实机测试了，请全面复查软件运行效果， 全面检查bug和问题」。

环境：板子挂在 **COM3**（USB-SERIAL CH340，`USB VID:PID=1A86:7523`）@ 2 Mbaud； `DEVICE_ID=0x0C`、`REV=0x10`、`RTL_BUILD=0x000C`（v12，帧布局 v2）；输入 **≈1.01 kHz**、`inter=256 div=1`、极对数 4、**`0x0109=0x0040`（只开 ABZ，UVW 输出关着）**；帧率 ≈9.9–10.4 kHz。

全套结果（全部 PASS）：

项目	结果
S1 `selftest_link.py --seconds 5`	9917 帧/秒、179,954 字节/秒（2 Mbaud 的 90%）、本窗口校验错 200（0.397%）、重同步 3680 字节、丢批/丢帧 0/0；写通路原值写回 OK（`0x0101` / `0x0102` / `0x0109`）
S2 `selftest_s2.py --seconds 6`	输入频率交叉验证 **1006.8728 Hz**（独立路径①1055.33 / 帧数时钟 1000.20 / 墙上 996.08）、机械转速 15103.09 RPM；`0x0100` =0x0004、`0x0101` = `0x0102` =0x0000（**inter=256 div=1**）、`0x0106` =0x0A00、`0x0108` =0x4004；倍频写通路 inter=64/256/2 三次写读都生效并复原
S3 `selftest_s3.py --seconds 6 --clear`	`alarm.py` 22 项 PASS；r 均值 **25899.6**（min 25348.0 / max 26305.2、圆度偏差 **3.696%**、削顶 0）、板上 r 26032.5（差 0.51%）、门限 19660.8…39321.6；`STAT_LATCH=0x24` → 写 `0x0405=0xFF` 清成 `0x00`
S3 `--force-fault`（本次补跑）	`ABLIMIT(0x0106)=0x00 → 200` ⇒ `STAT_VAL=0x08`（bit3 inclim）、`STAT_LATCH=0x08`、`0x053A=0x80`（BiSS 帧进行中）；恢复 `ABLIMIT=0x00` 后**实时 0 而锁存仍 0x08**（正是"锁存"语义）⇒ 写 `0x0405=0xFF` 清成 `0x00`，收尾 `0x053A=0x00`
S4 `selftest_s4.py`	修完后 **8 项通过 + 2 项按需跳过**（见问题 1）
S5 `selftest_s5.py --seconds 12`	119,222 帧 / 18.4 MB / 9922 帧每秒 / 丢帧 0.76%；CSV 逐卷回读 + 抽行原始帧解回一致；PNG 1950×1080（478.0 kB）与 1950×990（142.7 kB）；离线 `recorder` 13 项 + `export_png` 12 项 PASS（含新的同机自校准门槛 `190688 行/秒`、上限 321574、比值 0.59）
S5 `--soak-minutes 10`	600 s / **5,952,554 帧 / 937.6 MB / 9921 帧每秒 / 丢帧 0.77% / 6 个分卷**；帧率中位 9895（最低 9428、最高 10415）；**内存起 163 MB、终 84 MB（峰 163）⇒ 不涨**；丢批 0、丢帧 0；15/15 PASS
S5 `--reconnect`	2/2 PASS：关端口→重开，前后各收 19,973 / 19,923 帧
`selftest_ui.py --real-port COM3 --seconds 6`	界面取 60,501 帧 = 记录器写 60,501 行（**差 0**）、文件 64,650 行、链路解析 73,614 帧、丢帧 592、关窗后队列 0 条
GUI 真机跑图（offscreen + `--record --export`）	13.9 s / **135,226 帧 / 20.9 MB（9696 帧每秒）/ 丢帧 0.79%**；`编码器波形_*.png` 503,448 B、`编码器ABZ-UVW_*.png` 147,616 B、CSV 21,898,950 B；运行日志只有「启动：…」「已建窗显示（耗时 0.6–0.7 s）」、**无 BOM、无异常**
写通路真机核对（临时脚本）	`0x0100/0101/0102/0103/0104/0106/0108/0109/010A/010B/010C` 共 11 个寄存器**写回原值 + 回读全部一致**
锁存监视（临时脚本，100 s / 19 次采样）	每次 `STAT_LATCH=0x00`、`STAT_VAL=0x00`、`0x053A=0x00` ⇒ S3 开头看到的 `0x24` 是**历史事件**，当前配置下不复现

★ 修掉的 3 个真问题：

1. **`host/selftest_s4.py:289-293`：UVW 未使能时误报 FAIL（测试脚本缺陷，不是产品缺陷）。** 两条 `check("无非法 UVW 状态…")` / `check("六个合法状态都出现")` **无条件执行**，而紧接着 `:294-299` 明明打印「板上 UVW 未使能（0x0109 bit7 = 0），跳过六状态均衡判定」—— 作者知道 UVW 可能关着，却只 gate 了"占比均衡"那一项。真机 `0x0109=0x0040` ⇒ U/V/W 恒 0 ⇒ `U-V-W-` 不属于六个合法态 ⇒ 144,218–165,094 帧全被算成"非法态"，每次这样跑都报 "未通过（FAIL），共 2 项不满足"。**修法**：把这两项连同"占比均衡"一起 gate 在 `out_bits.get("uwven")` 上，否则写进"跳过"清单并打印原因。修后真机复跑 ⇒ **PASS（8 项 + 2 跳过）**。
2. **`host/alarm.py::evaluate()` 漏译 `0x0405` 的 b5（scamp = 幅值越限锁存）。** RTL 依据：`rtl/reg_file.v:248-249` 的 `stat_val_now = {2'b00, (stat_amp_low|stat_amp_high), 1'b0, stat_inclim, stat_lag_limit, 1'b0, stat_otr}` ⇒ 锁存器只会出现 b0/b2/b3/**b5**；且 `rtl/reg_file.v:298` 的 `r_stat_ie <= 8'h38` 把 b3/b4/b5 都算进中断使能。而主机只映射 b0/b2/b3/b4（b4 在 `rtl/enc_top.v` 里硬接 0，代码里也注明了） ⇒ 真机 `STAT_LATCH=0x24`（b2 laglim + b5 scamp）、`0x053A=0x00` 时，卡片⑦显示"幅值越限 正常"，只有底部摘要 `0x0405=0x24` 能看出矛盾，用户查不到这个位是哪来的。**修法**：实时门限告警优先，只有"历史锁存、当前已恢复"才显示「已锁存 b5」；`alarm.py --selftest` 新增 3 条断言（`0x24` ⇒ amp 警告 + laglim 警告、`0x053A` 有实时位时仍以实时为准）⇒ **22 → 25 项**。
3. **`host/abz.py` 的"采样不足"提示归因说错，而且图上把假波形画得像真的（会误导用户）。** 原文案一口咬定"输入频率太高…请把输入降到 20 kHz 以内"（这在 §1.76 的 100 kHz 场景是对的），可真机 `f_in=1.008 kHz` 也是 muddy：真因是 **`bins (256) ≤ 4×inter (1024)`** ⇒ 每箱跨 ≥1 个 A/B 周期、每箱均值必然趋近 0.5（`rtl` 侧 AB 每输入周期 1024 个边沿 = 512 个周期，真周期 1.406°）。而同一份提示里另一条（"箱数太少、请提到 ≥1024"）才是对的 ⇒ 两条自相矛盾。更糟的是面板上 A/B 会画出一条**看似正常的方波**（约 7 个周期/360°，是混叠出来的），图上**没有任何"不可信"标记**，只有卡片顶部一行橙色小字。**修法**：① `host/abz.py` 按 "箱数不够 / AB 边沿率接近帧率"**分因**给诊断，并给出可行做法（`inter` 降到 ≤ 箱数/**8**；若 `8×inter ≤ 1024` 才建议提箱数；并明说 `inter=256` 这档即使 1024 箱也只有 2.0 点/周期）；② `host/panel_abz.py::_redraw()` 在 `ab_muddy` 时**就地**改子图标题为「ABZ 等效采样（采样不足：A/B 波形不可信，只看 Z 与读数）」并把 A/B 两条线 `alpha=0.30`；③ `selftest_s4.py` 的跳过说明改为直接引用分析器给出的那句（单一来源，避免两处说法不一

致）； ④ `selftest_ui.py` 新增 2 项（造 `ab_muddy=True` 的快照 ⇒ 标题含"不可信"且 A/B 变淡； 造 `ab_muddy=False` ⇒ 标题与 alpha 复位）⇒ **105 → 107 项。 ★ 同一天自查又抓出两个连带问题并修掉**（写在这里，因为都属于"信息不准"这一类）： （a）**建议阈值算错档**：判据是 `bins ≥ 2×4×inter`（反解 `inter ≤ 箱数/8`），我第一版写成 "`inter` 降到 ≤ 箱数/**4**" —— 按那个值调到 `inter=64`、 `bins=256` 时 `2×4×64=512 > 256`， **照样 muddy**，等于给了个无效建议（现改为"≤ 箱数/8，保证每箱跨 ≤0.5 个 A/B 周期"）； （b）**离线造帧器根本造不出 muddy**：`host/abz.py::synth_frames()` 原来用同一个 16 位角度 既算 A/B 电平又算遥测 `angle`（自洽 ⇒ A/B 与角度——对应、 `ab_muddy` 永远为假）， 而板子上 A/B 是由**细分相位累加器**（`rtl/abz_gen.v` 的 `idx_exec`）当场产生的、`angle` 只有 16 位遥测精度 —— 所以"采样不足"这条路径**从来没有被自检覆盖过。**新增参数 `fine_bits`： `>0` 时 A/B 用高 `fine_bits` 位的相位算、 `angle` 仍截断到 16 位（等价真实板子）， `fine_bits=0` 保持旧行为不动老用例。 `abz.py --selftest` 据此补 3 项（muddy 为真、文案按 "箱数不够"分因且含"箱数/8"、不再一口咬定"输入频率太高"）⇒ **15 → 18 项。**

顺手核过、确认不是问题的项（免得下次再查一遍）： ① 卡片③的"校验错 0.39%"与卡片⑩的 "校验错帧数"是**串口链路逐帧 XOR 校验**，与 BiSS 帧 CRC 无关；② 卡片⑩"丢帧估计 0.77%" = 校验错帧 + 重同步吞掉的帧占应到帧时隙的比例（`link.counters()` 的口径，tooltip 已写明）； ③ `host/ui_prefs.py::save()` 用 `except Exception` 吞掉写失败（放到只读目录不会崩）； ④ `host/recorder.py` 的磁盘满/路径不可写路径有专门自检（`[12]` 用假 writer 造 ENOSPC， 要求"不抛异常、 `active=False`、有中文 error、句柄关掉"）； ⑤ 2 s 一次的锁存轮询 gate 在 `self.worker is not None and not busy` 上，拔线后不会继续刷命令；⑥ 拔线/驱动的 `closed` / `error` 事件会走 `QTimer.singleShot(0, self.disconnect_port)` 彻底复位（`--reconnect` 已验证重连）。

还留着、等用户拍板的两条（不影响正确性，本轮未改）： ① UVW 使能位藏在 ⑥倍频倍率卡片折叠的「高级参数」里（`host/panel_interp.py:125-127`）， GUI 没有"一键启用 UVW"入口，而跑 S4 六状态验收需要它； ② `Ctrl+滚轮` 缩放后时基/角度组合框 按"最近档位"显示（显示"满屏 50 ms"而实际 49.152 ms），组合框与"量程："读数不完全一致 —— 用户本轮已明确"不做无级微调/自定义显示"，所以这条只是记录。

复验（改完这一轮重跑，全绿）： `python -m compileall -q host` exit 0； `host/alarm.py --selftest` **25/25**； `host/abz.py --selftest` **18/18**（自查补的 3 项在内）； `QT_QPA_PLATFORM=offscreen python host\selftest_ui.py` **107/107 PASS**；真机 `selftest_s4.py --port COM3` **PASS**、 `selftest_s3.py --force-fault --clear` **PASS。**临时取证脚本（`host/_wr_check.py`、 `host/_latch_watch.py`）与现场图（`host/_live_*.png`）、 测试记录目录（`host/_live_rec`、 `host/_s5_live`、`host/_s5_soak2`）已全部删除； 另删掉我测试留下的 `host/ui_prefs.json`（里面是 `wave.x_span_ms=49.152` 这类测试值）。

取证方法教训（本轮新增）： ① 并发跑两个占串口的脚本 ⇒ 后连接的那个必然报 `连接失败：串口没连上（COM3）`，这是**我自己的端口争用**，不是软件缺陷（第一次差点误判）； ② 量 offscreen 界面的几何/截图**必须让事件循环跑一轮再量**（第一版在"进放大"的同一次调用里 立刻 repaint+grab，量到的是重排前的中间态，看着像严重错位）； ③ `mapTo(win, …)` 对嵌套控件会给出漂移值（按钮报 x=2878 而卡片只有 1554 宽）⇒ **以控件在自己 父容器里的 `geometry()` 为准**；④ 我自己写的临时脚本也踩了"0 是合法值"的坑 （`int(v or -1)` 把合法 0 判成"读回不一致"，误报 7 个寄存器）—— 与 `host/regs.py` 当年那类 "读失败按 0 覆盖"是同一类错误的镜像；⑤ **别把一整套自检塞进同一个后台 pwsh 作业**：我把 9 条 `python host\*.py --selftest` 用内部函数 `Run(){ … *> $null }` 串在一个后台作业里跑，首个 python （PID 27528）**空转 30 分钟、CPU 100%、6 分钟内零文件改动、 `job_output` 无任何输出**，只能 `job_kill` 收场；改成**每条命令一次 pwsh 调用**后 9 条全部 ≤8.4 s 跑完（`compileall` 0.1 s、 `abz` 1.1 s、 `recorder` 1.1 s、 `export_png` 4.0 s、 `selftest_ui` 8.4 s）⇒ 批量串跑省不了时间，反而会失去进度可见性；⑥ 复核冻结包里的**中文串**必须按 UTF-8 解 `marshal.dumps(code_obj)`（`ZlibArchiveReader.extract()` 返回 code 对象而非 bytes），按 latin-1 解会**全部假阴性**——第一版就这样，差点误判"修法没进包"。

交付包随之重打（2026-10-14 16:48）：因为 `host/abz.py` 在 16:21 那次打包之后又被改过 （建议阈值改成"箱数/8"、 `synth_frames` 新增 `fine_bits=`、自检 15 → 18 项），故重跑 `python -m PyInstaller --noconfirm --clean --workpath host\_exe_work_onedir --distpath host\dist_onedir host\host_onedir.spec`（约 79 s）⇒ `host\dist_onedir\host_app\host_app.exe` **7,991,315 B @2026-10-14 16:48:10**、SHA256 `E295B0FDAFB2F1176704DEA299586F6A8F0F6043DC5F3A30C129F23EE06C60A6`、 目录 **426 文件 / 583,871,454 B。包内复核**（`CArchiveReader(exe).extract("PYZ.pyz")` → `ZlibArchiveReader`， 680 模块）： `abz` 命中 `每箱均值必然趋近 0.5` / `把 inter 降到 / 箱数/8` / `fine_bits` / `成因未判明`； `panel_abz` 命中 `采样不足` / `不可信`； `alarm` 命中 `已锁存 b5` / `门限：偏低`； `main_window` 命中 `set_max_layout` ⇒ 复查与自查的全部修法确实进包（`selftest_s4` / `selftest_ui` 是开发自检脚本，按设计不打包）。 ★ 复核脚本的坑：中文串必须按 **UTF-8** 解 `marshal.dumps(code_obj)` 再搜，按 latin-1 解会**全部假阴性**（第一版就这样，差点误判"修法没进包"）； `ZlibArchiveReader.extract()` 返回的是 code 对象，不是 bytes。 **下一次沙箱外的确认**：用户双击 `host\dist_onedir\host_app\host_app.exe`（整目录一起拷）。

1.82 ★★★★★ 回答"三个为什么"（ADC 过载锁存 / BiSS CRC 错 / ABZ 采样不足）+ GUI 一键使能 ABZ/UVW（2026-10-14 晚，界面自检 107 → 117 项）

用户原话（m10906）：「为什么ADC会过载锁存？为什么BISS会CRC错误？为什么ABZ会采样不足，AB波形不可信？GUI要分别有一键启动ABZ、UVW的入口」前两问是"解释"，第三问既解释又要改（顺带把文案里一个 2 倍错误修掉），最后一条是新功能。

1.82.1 为什么 ADC 会过载锁存（OTR）

- `rtl/ad9238_capture.v:43-45`： OTR 是 **ADC 芯片给的引脚电平**，本模块把它做成 **sticky** ——只有 `clr_otr=1`、 `rst_n=0` 或 `en` 下降沿才清，且**清优先于置位**； `:58-64` 端口 `ch1_otr/ch2_otr/clr_otr/otr`（`otr` = 两通道锁存相或）； `:98-101` `otr_flush = clr_otr | en_fall`。
- 于是"锁存"的含义是：**历史上任何一次**过载都会留灯，哪怕此刻信号已经正常。旧版顶层把 `clr_otr` **硬接 `1'b0`**（`rtl/reg_file.v:82` 与 `:133-137` 的注释写明了这件事）⇒ 一旦亮 就永久亮，只能复位 FPGA。现在写 `0x0405` bit0 会在同拍发清除脉冲（`rtl/reg_file.v:429`）， 所以面板上「清除锁存告警（写 0x0405 = 0xFF）」才是真正的复位手段。
- **"过载"不等于"真溢出"**：OTR 由模拟前端决定，前端超量程、偏置过大、增益档不匹配、 甚至上电瞬态都会打它一次。遥测里 OTR 并进 `stat bit0 oflow`（`rtl/enc_top.v:806` `(cor_sat | otr_lat)`），所以 GUI 上「输出溢出位」和「ADC 过载锁存」是同一盏灯的两种说法 （`host/alarm.py:185-200` 的告警行文案已经把这点写进 tooltip）。
- 与之相邻、但**不是**过载的那类灯是幅值门限：`rtl/amp_monitor.v:5-9` `amp_sq = s²+c²`（Q2.30）， `amp_low` 判"信号太弱"（磁环脱落/气隙过大）、 `amp_high` 判"太强"（偏置过大/前端饱和）， 门限默认 `(0.60)²` / `(1.20)²·2^30`（`rtl/reg_file.v:315-316`，寄存器 `0x0520-0x0527`），实测值 在 `0x0530-0x0533`； `rtl/enc_top.v:1178` `led[1]` 与 `:1182` `irq_n` 都含 `amp_low|amp_high`。

1.82.2 为什么 BiSS 会 CRC 错误

- FPGA 侧是 **BiSS-C 从机**： `rtl/biss_c_slave.v:89-99` `module biss_c_slave #(ST_BITS=26, MT_BITS=12, TIMEOUT_CYCLES=10, CDS_ENABLE=0)`，端口 `ma`（主机时钟，**异步，最高 10 MHz**）、 `sl/slo`、 `st_data/mt_data`、 `err_bit/warn_bit`。
- **帧间必须留 ≥100 ns 空隙**： `:74-80` 的帧复位判据是"`ma` 与 `sl` 同时为高 ≥ `TIMEOUT_CYCLES`(=10) 个时钟"，或"一帧发完后 `ma` 保持高 ≥100 ns"。主机若**背靠背连续打时钟**（空隙 <100 ns），从机就会把它判成"CRC 之后还在打时钟"的协议违例 ⇒ `bit_err` / `crc_err_seen` 置位。注释里明确写了这是"实现违例检测的必然代价"。
- `crc_err_seen` 是**粘滞**的（`:110-111`、置位在 `:337`），一路送到 `rtl/enc_top.v:759/:785/:954` → `0x053A` bit6（`rtl/reg_file.v:528`）。host 侧 `host/alarm.py` 的说明是「BiSS CRC 错 … 没接 BiSS 主机时应为 0」——**没接主机却亮**，就要怀疑 `ma` 引脚悬空/毛刺/串扰（异步时钟上的一根长线很容易收到工频与串口噪声）。

- 顺带说明：GUI 里「校验错 0.39%」是**串口链路逐帧 XOR 校验**，与 BiSS 帧 CRC **完全无关**（README 末尾专门写了这条提醒，免得把两个 0.x% 混成一个问题）。

1.82.3 为什么 ABZ 采样不足、A/B 波形不可信

- 卡片⑨ 的"等效采样"按**输入电角度**分箱（默认 256 箱），每个箱取箱内 A/B 的平均电平； 判据在 `host/abz.py:366-377`： `muddy_a/b` = 箱内高电平比例落在 0.05~0.95 的箱数， `ab_muddy = (muddy_a+muddy_b) >= cover_n`，muddy 时把上升沿箱位置成 `None`（相位不可测）。
- 真机参数： `inter=256`、 `f_in=1010 Hz`、 `bins=256` ⇒ **AB 基波 ≈258.6 kHz、边沿率 ≈1.034 MHz**（与 GUI 的"单通道 258.6 kHz / AB 边沿率 1,034,506 每秒"一致），而 `deg = d*inter/bins*360` （`:378-387`）说明 `self.inter` 就是"每输入电周期的 A/B 完整周期数" ⇒ **每箱正好跨 1.0 个完整 A/B 周期** ⇒ 箱内均值恒 ~0.5 ⇒ 与角度不再相关 ⇒ `ab_muddy` 成立（实测 97% 的箱都这样）。
- 图上那条"方波"是**混叠假象**：帧率 ~10.4 kHz，每帧电角度步进 `360°×1010/10368 ≈ 35°` （≈26 箱），远远跳着采 258.6 kHz 的 AB 基波 ⇒ 画出来像 7 个周期的方波，其实毫无意义。
- 结论与可行做法：**要么把 `inter` 降到 ≤ 箱数/8**（256 箱 ⇒ ≤32；1024 箱 ⇒ ≤128，保证每箱跨 ≤0.5 个 A/B 周期、每周期 ≥8 点），**要么别用等效采样看上 MHz 的边沿，直接上示波器。**

1.82.4 顺手修掉的两处（都是"报给用户的数字/建议"错，不影响判据）

1. **`per_cyc` 多乘了 2**： `host/abz.py:426` 原本 `per_cyc = 2.0*self.inter`（注释还写"每输入电周期 里的 A/B 完整周期数"），按 `:381` 的换算应为 `float(self.inter)` ⇒ "每箱跨 %.1f 个周期"与 "即使 1024 箱也只有 %.1f 点/周期"都**虚报一倍。**判据（ `bins < 8*inter` ）与结论不受影响， 但数字会误导人，已改并加第 19 项自检断言（出现"每电周期 256 个 A/B 周期""每箱跨 4.00 个周期"， 且不得出现"每箱跨 8.00 个周期"）。
2. **"箱数提到 ≥1024"的建议不成立**：旧文案无条件让用户加箱数，可 `inter=256` 时即使 1024 箱 也只有 4 点/周期。现在按 `need = 8*inter` 与上限 `BINS_CHOICES[-1]=1024` 分两支：够得着就说 "请把箱数提到 ≥%d"，够不着就说"需要 %d 箱，已超过本程序上限 1024 箱 ⇒ 请同时把 inter 降到 ≤%d"。

1.82.5 GUI 一键使能 ABZ / UVW（用户明确要求）

- **界面**（ `host/panel_abz.py` ）： `AbzPanel` 新增 `output_requested = Signal(dict)`，在卡片⑨ 的 网格**第 2 行整宽插入"输出使能"行** = 「ABZ 使能」「UVW 使能」两个 `setCheckable(True)` 按钮
- 一行状态标签（ `输出使能：ABZ 使能（0x0109 = 0x0040）` ，文案复用 `host/regs.py::output_text` ）。 插行导致 side/canvas/lab_states/表的行号整体 +1， `set_max_layout()` 的放大态坐标与 `setRowStretch` 同步改（3/4 行）。按钮点击**只发信号**（不在面板里发命令）； `set_output(bits, raw)` 记下 `_out_bits` 并用 `blockSignals` 同步按钮（供回读后刷新）， `restore_output()` 供主窗口在 "拒绝写入"时把按钮拨回去； `set_connected` 一并开关这两个按钮。
- **写逻辑**（ `host/main_window.py` ）：模块级纯函数 `output_write_value(cur, bits)` —— 只把 `abzen` / `uwven` 合进 `decode_output(cur)` 的结果，再 `(cur & 0xFF00) | encode_output(base)` ⇒ **同字节的 `apol/bpol/zpol/abzdir/uvwpol/uvwdir` 与高字节原样保留**（与 `panel_interp.py:263` 的读-改-写同一套路）； `cur is None` 时返回 `None`（**宁可不写，也不能按 0 把其它位覆盖掉**）。 `_on_set_output(bits)` 的三道闸： `worker is None or busy` ⇒ 提示 + 回滚按钮；还没读回 `0x0109` ⇒ 提示"先「设备自检」" + 回滚；值没变 ⇒ 只记一条日志。通过后 `_set_busy(True)` + `task.run(lambda: task_write_advanced(w, {REG_OUTPUT: val}), "adv")` 复用既有**写-回读**路径， `_done_adv` 会把回读值回灌面板（于是按钮显示的是**板上真值**，不是我们的愿望）。 三处 `self.abz.set_output(...)` 调用都补了 `raw` 实参，标签从此显示真实 `0x0109`。
- **自检**： `host/selftest_ui.py` 新增离线小节「⑨ 补2」8 项（按钮存在/checkable、断连置灰、 连上可点、 `set_output` 同步按钮与标签、两使能都开的标签、读失败标签、纯函数 5 组值含高字节保留、 无串口时点击被拒且按钮回滚） +「⑨ 补3」2 项（见 §1.82.6）⇒ **117/117 PASS**；**真机小节** （ `real_port_check` 内、取记录基线之前）做端到端：读 `0x0109` → 点「UVW 使能」→ 断言 bit7 翻转且 `& 0x007F` 不变 → 清累积、等 2 s、从快照断言"六状态出现合法态且非法态 <5%" → **再点一次复原原值**（不在用户板子上留下改动）； 板上本来就是开着的（ `uwven=1` ）时，复原后再查一次 六状态（把状态检查抽成局部函数 `_states_check()`，两个分支各调一次）。
- 离线全套复跑全绿： `compileall` / `encoder` / `abz` （19 条）/ `export_png` / `regs` / `ui_prefs` （14 条）/ `alarm` （25 条）/ `recorder` / `selftest_ui` （117 条）。
- ★ 踩坑：新增小节第一版末尾调 `w._refresh_buttons()`，把所有面板控件置灰，**打坏了后面既有用例** 「焦点在数字框里时方向键让给它（不改量程）」（读数变 7.50 ms）⇒ 改成先记 `_en0 = btn_abz_en.isEnabled()`、 测完 `set_connected(_en0)` 还原。另一处：标签用的是 `rg.output_text()` （"ABZ 使能、UVW 使能"）， 断言不能写成"仅 ABZ"。

1.82.6 ★ 真机第一次验收一键使能，当场抓出并修掉一个老 bug（ `_done_adv` / `_done_apply` 的 2 元结果被按 3 元解包）

- **现场症状**：用户关掉旧包、COM3 释放后跑 `python host\selftest_ui.py --real-port COM3 --seconds 6` ， 2 项 FAIL —— 板上 `0x0109 = 0x00C3` （ `apol+bpol+abzen+uwven` ， **UVW 本来就开着**，用户自己开过），点一次变 `0x0043` （对）， **再点一次没变回 `0x00C3`**。其中一项失败是我的断言掩码写错（ `0x00BF` 含 bit7，应为 `0x007F` ）， 另一项是真 bug。
- **原始日志**（探针脚本 `host/_click_probe.py` ，用完即删）： `[log:fault] !! 处理 写高级参数 的结果时出错：not enough values to unpack (expected 3, got 2)`。
- **根因**： `host/main_window.py::_done_adv` 写的是 `for addr, val, hexs in result["readback"].items():` ， 而任务函数 `task_apply_inter_div`（`:121-152`）/ `task_write_advanced`（`:168-182`）的返回是 `{"ok", "writes":[(addr,val,hex)], "readback":{addr:val}, "hexes":{addr:hex}, "bad":[(addr,want,got)]}` ⇒ **`readback` 是 2 元**，3 元的是 `writes` ，原始应答在 `hexes`。同一个错在 `_done_apply` 里也有一份。 异常被 `host/main_window.py::_on_task_done`（`:1193-1218`）的 `except Exception` 兜住，只剩一条 fault 日志 ⇒ **寄存器其实写进板子了、界面却从不刷新**： `interp.regs_raw` 停在旧值，于是下一次点击算出 `val == cur` ，走"输出使能已是目标值"分支、连命令都没发。
- **是老 bug 不是新 bug**： `git blame` 显示这两行来自首次入库提交 `5e9c326`（2026-10-13）； 旧 `selftest_ui.py` 里 `readback` / `_done_apply` / `_done_adv` **一次都没出现** ⇒ 从没被自检覆盖过。 受影响的是三条写路径：**倍频「应用」、高级参数「应用」、一键使能**（也就是说"改完不生效"这类 诡异现象以前就是这样来的）。
- **修法**：两处改成 `for addr, val in result["readback"].items():` + `hexs = (result.get("hexes") or {}).get(addr)` ，各留一段注释写明"曾经 3 元解包、被兜底日志吞掉"。
- **新增离线回归「⑨ 补3 写任务结果处理器」**：用真机形状的假结果走**完整分发路径** `w._on_task_done({"tag": "apply"/"adv", "result": …})` ，临时把 `w.log_line` 换成收集器， 断言 ①没有含"结果时出错"的日志 ② `w.interp.regs_raw[0x0109] == 0x00C0` 且按钮/标签同步； 末尾 `w.interp.set_regs(_regs0)` 还原。★ 两个踩坑： `dict(_regs0, **{REG_OUTPUT: 0x0040})` 报 `TypeError: keywords must be strings` （键是 int）⇒ 改成复制再赋值； `_on_task_done` 收尾会走 `_set_busy(False)` → `_refresh_buttons()`（`:1051-1065`），离线 `worker=None` 时把各面板按"未连接"置灰 ⇒ 该小节末尾显式 `w.btn_check.setEnabled(True)` 并对 `speed/interp/alarm/wave/abz/record` 逐个 `set_connected(True, True)` ，恢复"构造时可用状态"。
- **修完真机复跑 PASS**： `[OK] 真机：点「UVW 使能」→ 0x0109 bit7 跟随，同字节其它位不变 0x0109 = 0x00C3 （uwven=1）→ 0x0043` ； `[OK] 真机：再点一次恢复原值（不残留改动）0x0109 = 0x00C3 → 0x00C3` ； `[OK] 真机：UVW 本来就开着时，六状态表也是合法态主导（非法态 <5%）合法 19597 / 非法 0 （U-V-W+:3917 U-V+W-:1828 U-V+W+:2092 U+V-W-:3914 U+V-W+:3919 U+V+W-:3927` —— 六个态并非均匀， 是帧步进 ~35°、状态宽 60° 的采样偏置，**非法态 0 才是关键**）； 记录链路仍 `界面取`

61081 帧 = 记录器写 61081 行（差 0）、 文件行数 93863 = rows 93863 、链路解析 103378 帧、 丢帧 814、队列丢弃 0、关窗后队列 0 条； 真机核对 通过 (PASS)。

- **交付包**： host/dist_onedir 里的 exe 在 17:14 那次替换时还带着这个 bug（ main_window.py 之后才改）， 故 17:19 又**重打一次**（第三次）⇒ host_app.exe **7,995,302 B @2026-10-14 17:19:37**、 SHA256 63019D2D07F0CBD01F2DB46C9232910FAF09BB7D960F5DF35091D69BF2E7023E 、426 文件 / 583,875,441 B。 PYZ 复核（680 模块）确认 output_write_value / 一键设输出使能 / 输出使能未写入 / 还没读回 0x0109 / 已是目标值 / ABZ 使能 / UVW 使能 / output_requested / 箱数/8 / 每电周期 %d 个 A/B 周期 / fine_bits / 已锁存 b5 / 门限：偏低 / 采样不足 / 不可信 全在包里，并用 _done_apply / _done_adv 两个 **code 对象**里存在常量 hexes 证明 2 元解包修法确实进包 （★ 教训：**代码行不会原样出现在常量表里** —— result["readback"].items() 被拆成 'readback' / 'items'， 所以"某一行改没改"只能在对应函数的 co_consts 上验证； selftest_s4 / selftest_ui 按设计不打包）。

1.83 ★★★★★ 回答"为什么 A/B 不是标准方波" + 卡片⑨「方波可看 / 还原」一键预设（2026-10-14 晚，界面自检 129 → 133 项）

用户原话（m11491、m11509）：

> 「为什么ABZ信号波形图不是标准的方波？什么原因导致的？要怎么才能变为标准的方波？」 「加一个 「方波可看」一键预设」

1.83.1 为什么不是标准方波（三个成因，按重要性）

1. **图上画的不是采样点连线，而是"每角度箱里 A/B 为高的平均占比"**（ host/abz.py:334-336 的 hi_a[i]/n_ab[i] ）⇒ 天生是 0~1 连续量，边沿必然占一个箱宽的斜坡。板上 A/B 本身是真方波： 实测占空比 50.2%/50.2%、边沿率 4×inter/div×f_in = 4×256×1010 ≈ 1.034 MHz 、AB 周期 **1.406°**。
2. **一箱跨 inter/箱数 个 AB 周期**（ host/abz.py:378-387 的相位换算是 deg = d*inter/bins*360， 由此可知 self.inter 就是"每输入电周期的 A/B 完整周期数"）。真机 inter=256 / 256 箱 ⇒ **每箱正好跨 1.0 个周期**，箱内高低各半、均值恒 ~0.5（实测 97% 的箱落在 0.05~0.95） ⇒ 图上既没有 0 也没有 1。判据见 host/abz.py:399-416： **箱数 ≥ 8×inter。**
3. **时间轴混叠**：帧率 ~10.4 kHz，每帧 AB 相位跳 inter×f_in/帧率 = 256×0.0972 ≈ 24.9 个周期、 模 1 = 0.9 ⇒ 相邻帧细相位几乎同相、抽样被梳状抽稀，图上"约 7 个周期的方波"是**混叠幻影。** （ ab_hz > 0.3×fps 那条提示只是怀疑：实测 f_in=19 999.901 Hz + inter=32 仍得到正确的 −90.0°。）

处方（ f_in≈1010 Hz ）： **inter=32 / div=1 / 分箱 1024**（每箱跨 0.031 周期、每周期 32 点、AB 基波 32.3 kHz）； 备选 inter=8 + 256 箱；要连"边沿率 ≤0.3× 帧率"也满足则 inter=2 / div=4 + 256 箱。 改 inter 会真的改板上 AB 输出频率，测完要改回；**只把分箱提到 1024 而 inter 不动没用**（4 点/周期）。

1.83.2 实现： square_pair() + ⑨ 卡两个按钮

- host/main_window.py： 常量 SQUARE_POINTS = 16 （≈ :72 ）； 模块级纯函数 square_pair(inter, bins, points=SQUARE_POINTS, choices=None) （≈ :210 ）—— 已满足 （ inter*points <= bins ）原样返回；否则 it2 = min(inter, max(choices)//points)， 再挑**最小的** 能满足 b >= it2*points 的箱数档（箱数少 ⇒ 填箱快）； 都不行则 (max_bins, it2)。 实测： (256,256)→(1024,64) 、 (32,256)→(512,32) 、 (8,256)→(256,8) 、 (2,64)→(64,2) 、 (256,1024)→(1024,64)。
- host/panel_abz.py： 新增 square_requested / square_restore_requested 两个信号；head 行在 「暂停刷新」后加「方波可看」「还原」两个按钮； set_connected() 一并开关；新增公开方法 bins() / set_bins(bins) （校验 az.BINS_CHOICES 、 blockSignals 同步下拉框、 an.configure(bins=)）。
- 主窗口两个槽（≈ :1205-1300 ）： _on_square_preset() 走三道闸（未连/忙、没读回 0x0101 / 0x0102 、 已满足就只说"不用改"）， 先本地 set_bins()， inter 真变了才 task_apply_inter_div(...) 写板子； _on_square_restore() 把 inter 与分箱改回去。

1.83.3 ★ 真机抓出的两个问题（都已修 + 离线回归）

1. **task_write_advanced 被多传了一个实参**： _on_square_restore 里写成 task_write_advanced(w, dict(raw), "adv") —— 第三个实参本该给 task.run 的 tag，于是任务一执行就 TypeError: task_write_advanced() takes 2 positional arguments but 3 were given， 被 _on_task_done 的兜底 except 吞成一条 fault 日志 ⇒ **点"还原"什么都不写**（板上停在预设值）。 ★ 离线自检当时看不见，因为 _runf 桩只记 tag、**不调用**任务函数 ⇒ 现在离线用假串口 _FakeW **真跑**任务函数，参数错会进 _errs，并断言写入序列就是当初读到的两个原始值。
2. **任务内回读可能拿到陈旧应答**：真机出现过"写 0x0101=0x0000 之后，任务里的回读却是上一次 写进去的 0x0040 "。根因是 0x0101 在现板上是 **16 位对写**（写它等于同时写 r_inter0/r_inter1 两个字节，见 §1.83.4），一次写会带出一串应答，链路忙时容易取到旧的那条。后果不只是显示错： regs_raw 被污染，而「还原」的原值正是从它来的。修法： task_write_advanced 在回读不符时 **再读一次确认**（与 task_apply_inter_div 早就有的做法一致），并且「还原」改成**逐字节写回 当初读到的原始值**而不是按 (inter, div) 重新编码 —— decode→encode 不是恒等 （ decode_inter_div(0x0000,0x0000) → (256,1)， encode_inter_div(256,1) → (0x0000,0x0001) ）， 只有原样写回才能逐字节复原。同时「已经还原过了」的判据也改成比**原始字节**，不再只看解出的值。

1.83.4 顺带取证的寄存器语义（ 0x0101 / 0x0102 ）

- 应答帧 5A <addr_lo> <addr_hi> <b_hi> <b_lo> <xor>， host/link.py:100-110 取 value = (rep[3] << 8) | rep[4]； 实测 rep[4] 是**请求地址自身**的字节、 rep[3] 是**地址 +1** 的字节（读 0x0109 得 5A 00 09 00 40 ⇒ 0x0040）⇒ **读 0x0101 得到的 16 位值天然就是 (r_inter1 << 8) | r_inter0** （因为 0x0102 紧跟其后），这正是 regs.decode_inter_div(v101, v102) 消费的形状。
- rtl/reg_file.v： :230 reg [7:0] r_uvw_cfg, r_inter0, r_inter1; （都是 8 位）； :261-262 复位 r_inter0 <= 8'h00 （注释"INTER = 0 -> inter = 256"）与 r_inter1 <= 8'h00 （"div = 1"）； :355-356 16'h0101: r_inter0 <= wdata; / 16'h0102: r_inter1 <= wdata;； :452-453 回读； :559-560 cfg_inter = {r_inter1[1:0], r_inter0[7:0]} （10 位）、 cfg_div = r_inter1[4:2]。
- 板上实测（临时探针，用完即删）：写 0x0101=0x0000 ⇒ 两个寄存器都读 0x0000；写 0x0101=0x0140 ⇒ 0x0101 读 0x0140 且 0x0102 读 0x0001；写 0x0102=0x0004 ⇒ 0x0101 读 0x0440、 0x0102 读 0x0004 ⇒ **板上 bitstream 的 0x0101 写是 16 位对写**（比 rtl/reg_file.v 现源"更宽"，按 16 位对写实现）； 0x0102 只改自己的字节。窗口读法与直读在链路空闲时完全一致（4 次比对全同）。
- 收尾：板子已写回文档默认 **0x0101=0x0000 / 0x0102=0x0000 （inter=256、div=1）**、 0x0109=0x0040 （只开 ABZ，与用户原配置一致），无残留改动。

1.83.5 自检与真机

- 离线： compileall exit 0； host\selftest_ui.py ⇒ **通过 133 项、失败 0 项**（117 → 129 → 133）。 本轮新增 ⑨补4「方波可看 / 还原」12 项 + 「任务真跑（参数错会显形）」1 项 + 「回读第二次机会」1 项 = 14 项；⑨补4 覆盖：按钮存在/断开置灰、 square_pair 5 组值、 set_bins 非法档拒绝、 无串口点击只提示、没读回倍频配置时只提示、inter=32/256 箱时只提箱数且记下 raw、 还原逐字节写回（断言写入序列）、无 raw 时退回 apply、忙态、已满足则"不用改"、没按过只提示。 另修一处测试自身的坑： _set_busy(False) → _refresh_buttons() 在离线会把所有面板置灰， 必须在小节末尾恢复各面板的"可用状态"，否则后面的焦点用例会失败。
- 真机（ python host\selftest_ui.py --real-port COM3 --seconds 6 , exit 0）： 0x0101/0x0102 = 0x0000 0x0000 → 0x0040 0x0000 （inter 256→64、分箱 256→1024、div 与高字节不动） → 0x0000 0x0000 （原样复原）； 另加一条"窗口自检读到的值 vs 直读值一致"的检查，把"链路陈旧应答" 单独报出来而不是笼

统判失败。UVW 一键使能、六状态、记录链路判定仍全 PASS。

- 交付包：`host\dist_onedir\host_app\host_app.exe` **8,000,459 B @2026-10-14 18:18:56**、 SHA256 `FCC63C4253FBD391628CA8D0B8851B8D5C7C3E80016091B5FCCD9C38F39A39F4`、426 文件 / 583,880,598 B。 PYZ 复核（680 模块）命中 `方波可看` / `方波可看」未执行` / `还原」没做什么` / `原样写回` `0x0101=0x%04X` / `square_pair` / `task_write_advanced`（后者在嵌套 `lambda` 的 `co_names` 里， 必须按整块 `marshal.dumps` 的字符串搜）， `_done_apply` / `_done_adv` 的 `co_consts` 仍含 `hexes`。

1.84 ★★★★ 微调后量程组合框不再"就近显示"（R15，界面自检 133 → 146 项）

用户原话（m11938，此前我在报告里把它列为"最后一条待你拍板的建议"）：

> 「Ctrl+滚轮 缩放后时基/角度组合框按"最近档位"显示」

1.84.1 症状与根因

`Ctrl+滚轮` （放大态）与 `Shift+↔/→` 都是**无级微调**（`ui_kit.ScopeNav.FINE_STEP = 0.98`、 `panel_wave.FINE_STEP = 0.9`），产出的值不落在档位表里（如 49.152 ms、359.1°）。而 `panel_wave.py::_sync_controls()` 与 `panel_abz.py::_sync_controls()` 在"值不在档位表里"时用 `findData` 失败后的兜底逻辑**就近显示**：

```
idx = cmb.findData(val)
if idx < 0 and isinstance(val, float):
    idx = min(range(cmb.count()),
              key=lambda i: abs(float(cmb.itemData(i)) - val)
              if isinstance(cmb.itemData(i), float) else 1e18)
```

于是组合框写「满屏 50 ms」、提示行写「量程：横轴 49.152 ms」，两处对不上（用户就是这么发现的）。 ★ 注意两种滚轮不是一回事：**正常态** `Ctrl+滚轮` = 20% 档进（`ScopeNav.ZOOM_STEP`）， **放大态**（`_plain_zoom`，R8）才是裸滚轮 20%、 `Ctrl+滚轮` 2% 微调 —— 所以"微调值"主要来自放大态。

1.84.2 修法：`ui_kit.sync_span_combo()`

新增共享函数（`host/ui_kit.py`，紧跟 `step_button` 之后）：

- 常量 `CUSTOM_FLAG_ROLE = Qt.UserRole + 1` —— 临时档的标记（data 仍是真实数值，所以默认的 `findData()` 语义不变；标记必须往后挪一位，否则和 data 撞车）；
- `sync_span_combo(cmb, val, fmt)`：值命中档位（浮点按 `1e-9` 相对容差）⇒ 选该档，并**删掉**已有临时档； 否则在「自动」后面插一条临时档（已有就 `setItemText` / `setItemData` 原地更新），文本 `fmt % val`； 返回选中文本便于自检断言。
- ★ 关键细节：删临时档会让它后面所有下标前移一位，所以删完要**重新校正下标** （`idx - (1 if cidx < idx else 0)`），否则会选到隔壁档位 —— 这条专门有自检覆盖。

三处调用：

位置	文本格式	与读数对齐
`panel_wave.py` 卡片⑧ 时基 `cmb_tdiv`	满屏 `%.4g ms`（微调）	`_view_text()` 的 横轴 `%.4g ms`
`panel_wave.py` 卡片⑧ 幅值 `cmb_vdiv`	`±%.2f FS`（微调）	`_view_text()` 的 `±%.2f FS`
`panel_abz.py` 卡片⑨ 角度窗口 `cmb_xspan`	满屏 `%.4g°`（微调）	`_view_text()` 的 横轴 `%.4g°`

位数刻意对齐：第一版我用 `%g`（49.152）而读数用 `%.4g`（49.15），自检立刻报"组合框与读数不一致" —— 这正是想要的判据：**两处必须写同一个数字。**选回真档位或点「恢复自动」时，临时档自动收掉 （档数复原）。

1.84.3 自检与取证

- 离线（`host/selftest_ui.py` 新增 ⑨ 补5「微调后量程组合框不再『就近显示』」，12 项 + 1 项集成）：
- 档位值（50 ms）⇒ 组合框就是那一档、没有多余临时档；
- 微调 49.152 ⇒ 文本 `满屏 49.15 ms（微调）` 、**且与 `lab_view` 的「量程：」写同一个数字**、 档数只多一条；
- 再微调 46 ⇒ 原地更新（档数不变）；
- 回到真档位 100 ⇒ 临时档删除、档数复原、选中项确实是 100（下标校正）；
- 幅值微调 0.90 FS ⇒ `±0.90 FS（微调）`，回 0.50 ⇒ 复原；
- 角度窗口微调 359° ⇒ `满屏 359°（微调）`，回 270° ⇒ 复原；
- 「恢复自动」⇒ 临时档收掉、回到「自动（跟随输入）」；
- 纯函数五条路径（命中/插入/更新/回档/自动）单测 `uk.sync_span_combo()`；
- 集成：⑮ 节里真发一个放大态 `Ctrl+滚轮` 事件，断言组合框含"（微调）"且数字与「量程：」一致 （实测 `7.84 ms ⇒ 组合框='满屏 7.84 ms（微调）' / 读数='量程：横轴 7.84 ms（手动）…'`）。
- `compileall` exit 0； `host\selftest_ui.py` ⇒ **通过 146 项、失败 0 项**（133 → 146）。
- 离屏取证（临时脚本，用完即删）：卡片⑧ 放大态、 `50 ms` 先落档位，再发一次 `Ctrl+滚轮` ⇒ 组合框 `满屏 49 ms（微调）`、量程行 `横轴 49 ms（手动）`、图标题 `横轴 49 ms`，三处一致； 截图确认组合框加长后**不被裁切。**
- 交付包： `host\dist_onedir\host_app\host_app.exe` **8,001,154 B @2026-10-14 18:43:02**、 SHA256 `B6E7F5DA5A9064B66EC726CAA2E34A41FB86DB05B7F9B1BAF7813FBCEE256DF2`、426 文件 / 583,881,293 B。 PYZ 复核（680 模块）命中 `sync_span_combo` / `CUSTOM_FLAG_ROLE` / `满屏 %.4g ms（微调）` / `±%.2f FS（微调）` / `满屏 %.4g°（微调）`， `_done_apply` / `_done_adv` 的 `co_consts` 仍含 `hexes`。

---

3. 未完成事项

1. ~~修 `rtl/uvw_gen.v` 迟滞/状态机，使回归回到 13/13~~ —— **已完成（13/13）**。

2. ~~接上 `rtl/enc_top.v:662-672` 注释里的其他未接线项~~ —— **已闭合。**原注释只列了 `cfg_hyst_uvw`；本轮用 `tools/scan_dangling.py` 把**全部实例端口连接**过了一遍，发现另一处空连接 `.cfg_h2_k ()`（`rtl/enc_top.v:688`）。查明其为**有意预留**（二次谐波系数，SPI `0x051B/0x051C` 已可读写，但算法侧未接入，见 `rtl/sc_correct.v` 头注释第 3 条与 `rtl/calib_ctrl.v` 的未来方案），**不是缺陷**；已在 `rtl/enc_top.v:688` 加行内说明、并在文件末补全"未接线项"整段注释。⇒ **功能输入端口至此无悬空项。**
3. ~~全仓扫 `unused_ok` 汇聚列表，列出剩余未接线/悬空信号~~ —— **已完成**，见 §1.2 与 §1.5。结论：汇聚列表 20 项**全部有真实读者**（无充数项）；真死信号只有 `sim/tb_enc_top.v:224` 的 `ph_idx`（已删）。⚠️ 如实记录：`amp_valid` / `abz_valid` / `uvw_valid` / `pole_valid` 各只有 **1 个**真实读者，"几乎没被用"，但不是悬空。
4. ~~重跑生产版与 ILA 版实现，刷新全部文档数字~~ —— **已完成（见 §2）。**
5. ~~重跑 13 台回归~~ —— **已完成**；**本轮进一步扩到 18 台并 18/18 PASS**（干净串行复跑 2026-10-10 06:32:48，见 §1）。
6. ~~`rtl/calib_ctrl.v` 与 `rtl/uart_scope.v` 零专用测试台~~ —— **已完成**，新增 `sim/tb_calib_min.v`（54 checks）、`sim/tb_uart_scope.v`（94 checks）、`sim/tb_uart_period.v`（27 checks）、`sim/tb_uart_baud.v`（16 checks）、`sim/tb_xor_probe3.v`（18 checks）。**★ 并因此查出两个真 RTL 缺陷**（`calib_ctrl` 标定窗口 off-by-one，见 §1.9；`uart_scope` 组帧 FSM 连发 `tx_start` 导致整帧错位，见 §1.10）、**两处"我的测试台在空转"**（已修，见 §1.11）、以及**一个主机端真缺陷**（`tools/uart_scope.py` 的帧校验和与 RTL 恒差 `0xFF`，会让上板后 `frames` 恒为 0，见 §1.15）+ **一处我自己写错的结论**（"xsim XOR 陷阱"，已更正并把误判固化成 T12）。
7. ~~**【未完成 / 已知风险】ABZ 与 UVW 六个输出没有做任何输出侧约束。**~~ ✅ **已完成（2026-10-10，见 §2.2）**：`constr/enc_top.xdc` 新增第 10 节，给此前零约束的 **14 个输出**补了 `set_output_delay`（位置类 `-max 6.000`、指示类 `-max 3.000`，全部 `-min 0.500`，`-clock sys_clk`）与 `SLEW/DRIVE`（位置类 FAST/8 mA、指示类 SLOW/4 mA），并**重跑实现签核**：`WNS +0.512 / TNS 0.000 / 0` 失败端点、`WHS +0.075 / 0` 失败、`All user specified timing constraints are met.`、DRC 0 error / 0 critical、布线 0 error，位流 **731,612 B @ 07:31:54。** ★ 迭代过程（含一次真实的失败）与全部数字见 §2.2。

   > ★ 补充（本轮查过，避免误判）：**这不是"文档漏写"。**工程自己的 `docs/04_wiring_checklist.md:286` 已经给了缓解措施 ——"若波形变形/振铃严重 → 缩短线长（**建议 ≤5 cm**），或在时钟线上串 22~33 Ω 端接电阻"，且 `docs/06_board_test_guide.md:20` 同样要求"建议 ≤5 cm，最好用排线"。**仍未做的是硬件侧的源端串阻**（改 XDC 只能约束 + 降驱动，真正的振铃解法是串阻），如实登记为"未做的事"。

8. **[历史记录•2026-10-10 已处理]** **`constr/enc_top.xdc` 注释自相矛盾（原第 162–165 行 vs 第 134–136 行）** —— 现象见下，**结论与后果见 §1.19**：第 134–136 行自相矛盾**（前者说 `ad_clk` 与 `clk50` 上升沿"是同一物理时刻"，已被本轮真原语对照实验证伪：`ad_clk` 确实晚 10 ns）。约束值本身偏保守、无害，**但注释会误导后续调参，**应改。** → 2026-10-10 已处理（见 §1.19）：**注释矛盾整段重写（保留被证伪断言的历史记录 + 逐条反证），约束按真实模型改成 `-max 24.000 / -min 19.500`；重跑实现后如实得到** WNS −3.271 ns / 26 个 failing endpoints（全部是 ADC 输入端口）。**这不是"改坏了"，而是把一直存在的边界项第一次显式化——上板第 1.8 步量 `tPD_meas` 即可定论（`docs/07_board_checklist.md`）。** → ★ 2026-10-10 已量并定案（见 §1.27）：**示波器双通道实测 `tPD_meas ≤ 4 ns` ⇒ 相位余量判据通过，**采样沿不改**（`rtl/ad9238_capture.v` 保持 `posedge clk_cap`）；26 个失败端点保留、约束不改（`constr/enc_top.xdc:150-249`）。仍在的分支只剩"ILA 里 `angle_raw` 是否出现真实毛刺"（§3 item 15）。
9. ~~**【未验证 / 只能上板做】ILA 版位流从未上板**（见 §1.6）~~ —— ✅ **2026-10-10 13:39 首次上板**：`enc_top_ila.bit` 下载成功、`End of startup status: HIGH`、枚举到 1 个 ILA core、`.ltx` 加载后 34 条探针可见，并抓到两份 CSV（`_synth_work/ila_cap1.csv` 100 kHz / 10 Vpp、`_synth_work/iladata2.csv` 10 kHz / 9 Vpp）。**并因此立刻查出 §1.20 那个"观测通道本身位序反转"的真缺陷**，以及 6 处会误判的上板判据（`dbg_adc_valid` 恒为 1、ILA 探针是二进制补码而不是偏移二进制等），已全部修入文档。⇒ 本节原先那句"`docs/05_*` 里所有上板相关结论都只是静态证据"**只对 2026-10-10 13:39 之前成立。**仍待上板证实的两条：~~① 新位流下 `dbg_pole_pairs_eff` 显示 `4`；② 写 `0x0405=0x0001` 后 `led[0]` 熄灭。~~ → ✅ **2026-10-10 17:08 两条全部收口（见 §1.23）**：① 16:12 位流导出的 `_synth_work/iladata3.csv` 解出 `dbg_pole_pairs_eff` = **4**，位序修复在板上定案；② 写 `0x0405` 的软清位**生效且能保持**（8 Vpp 输入下连续 5 轮 ×1 s `0x0405` 恒 `0x0000`，`led[0]` =AB21 灭、`irq_n` =K22 回高）；

   > ★ **2026-10-11 精细化（见 §1.38 ④）**：连做 6 次"写 `0x0405=0x0001` → 立刻读 `0x053A`"，**5 次读到 0x0000、1 次在 0.25 s 内又读到 0x0008**，且测试末尾 `0x0405` 又读回 `0x0025`（`otr laglim amp`）⇒ 准确说法是"**软清位确实生效且能保持，但 8 Vpp 下 OTR 会偶发再触发**"（不是"一清就复亮"，也不是"永不复亮"）。并顺带证实 UART→SPI 桥实机可用（`read 0x05F0` → `DEVICE_ID=0x0C / REV=0x10`）。新查清的一条口径：**本模块 10 Vpp 即满量程，10 Vpp 时 OTR 每周期真越限**，上板统一用 8 Vpp。

10. ~~**【未覆盖 / 已知缺口】`sim/tb_enc_top.v` 只做 39 个 check，且完全没碰 SPI 写寄存器与标定触发路径。**~~ —— ✅ **已完成（2026-10-10，见 §1.17）**：新增 **TEST F**，check 数 **39 → 63**，逐条断言覆盖 "SPI 写 `0x0502[0]=1` 触发 → `busy` 读回 → 标定期间 `angle` 冻结 → `done/truncated/nsamp` → 6 组结果与 `dut.u_calib.res_*` 比对（证明 `calib_we` 写回 `reg_file`）→ `dut.use_*` 等于读回值（证明 `sc_correct` 取用同一条路径）→ 写 `0x0502[2]` 清 `done` → 按键（含 65535 拍去抖）触发第二次标定"。**并且立刻查出两个真问题**：§1.17 的标定窗口接错（已修 RTL）、§1.18 的"一键标定不可重复"（已留证据；**2026-10-10 已修**，见 §1.18 与本节第 13 条）。 > 如实记录：TEST F 首轮自己还暴露了两个**测试台自身**的缺陷 > （`task chk` 的 id 只有 8 字符导致失败项名字被截断；按键只按 4 拍被去抖滤掉）， > 都已修 —— 与 §1.11 同一类教训。 > 另：TEST C 里 `fwd_cycles=0 / rev_cycles=4194239` **不是缺陷** —— > 该计数器在 `sim/tb_enc_top.v:655` 用 `cyc > CYCLES_PER_REV` 过滤， > 只统计"反向段"内的方向采样，所以"全部为反向"正是预期结果。
11. **【未完成 / 交付风险】`docs/` 下 7 个 `.pdf` 全部早于对应的 `.md`**：PDF 生成于 2026-10-09 07:37–07:39，而 `docs/00`–`docs/06` 的 `.md` 最后更新在 10-09 19:00–22:46，之后又叠加了本轮 §1.9–§1.15 的多项修正 ⇒ **PDF 里的数字与结论已经过期。**这些 PDF 只是内部设计与验证文档的导出件，**不是竞赛要求的"设计报告"**；而**仓库里目前仍没有任何设计报告 / 演示视频成品（提交物 0%）。** ⇒ 11/02–11/04 写正式报告时**不要直接复用这些过期 PDF**，要从 `.md` 重新导出或重写。 > **★ 2026-10-13 07:07 更新：本条"PDF 过期"分支已闭合。** `docs/00`、`01`、`02`、`04`、`06` > 五份 PDF 已用 `tools/md2pdf.py` 重导（Edge 无头打印，5 份全 `ok` / 失败 0）， > 新时间戳 **2026-10-13 07:07:09–07:07:31**，体积 897,342 B / 1,693,965 B / 934,231 B / > 939,533 B / 1,510,721 B；抽查 `docs/01_technical_solution.pdf`（23 页 / 32,665 字符）正文含 > `WNS` ×3、`0.821` ×1、`v11` ×3 ⇒ 确为 10-12 修订后的内容。`docs/03`、`AD9238_module_digest`、 > `ATK_board_digest` 三份 PDF 本来就比对应 `.md` 新，**未动**；`docs/05` / `07` / `08` / `09` / `DEV_STATUS` > 没有 PDF（`05` 是内部验证报告，非交付件）。 > ⚠️ **本条仍成立的部分**：`docs/*.pdf` 只是内部文档导出件，**竞赛提交件在 `deliverables/`** > （设计报告 `.md` + `.pdf`、选题说明 `.md` + `.pdf`、evidence_index、photo_shot_list均在位）； > **仍未开工的只剩演示视频 `deliverables/video/`**（目录里只有 `.gitkeep`）。
12. ~~粗调移位判据分歧（RTL 的 `<=1.999` vs 模型/文档的 `<=1.0`）~~ —— **已定案：取 `<=1.999`**，RTL 逻辑未改（只改注释），模型/文档/测试台注释与全部数字已对齐（见 §1.16）。同时删除 `tools/calib_shift_study.py` 与它的报告（一个恒返回 `sh=3` 的量纲 bug + 硬编码且与自身表格相反的结论），新增 `tools/calib_shift_judge.py` + `model/calib_shift_judge_report.txt`（四条判据 × 8 工况 + 恒等性 + 可达性，结论由数据生成）。
13. ~~**【已定位 / 未修】一键标定不可重复**~~ —— ✅ **2026-10-10 已修**（本轮）。修前现象（F10）：`GAIN_S 30325 → 32241` 朝单位增益 32768 回归、`PHASE_K 15 → −54`、`OFS_S 15 → −31`。**它有两个来源，只修一个不够**：① `calib_ctrl` 累加的是 `sc_correct` 的**输出** ⇒ 改为累加校正前的原始抽取样点 `s_dec/c_dec`（`rtl/enc_top.v`）；② 标定窗口从 `start` 那一拍就开始累加 ⇒ 窗口含"起始残缺周期"，结果依赖触发相位 ⇒ `rtl/calib_ctrl.v` 增加 `armed` 对齐：**先等到第一个输入周期边界再累加**，窗口严格等于 `win_cycles` 个整周期（`nsamp = win_cycles×P`），并对输入不转的情况 加 1 s 超时保护（`truncated=1` 空窗口收尾，不挂死）。修后实测（`sim/logs/SUMMARY.md` 2026-10-10 08:37:24，`tb_enc_top` 72 checks）：`F11 两次标定之差 OFS_S=0 OFS_C=0 GAIN_S=0`

GAIN_C=0 PHASE_K=0、 窗口 nsamp=16384/16384 wcnt=64/64 ⇒ **逐位一致**； tb_calib_min 未改动即继续通过。 详见 **§1.18**（含踩坑记录：对齐拍"计入样本但不计周期"两件事必须同时做对）。

14. **【v6 复测 ✅ 2026-10-11 已完成（§2.7 / §1.32 ⑤：20/20 GOOD、retry {0:20}、 win_bad 0/20）； 本条剩余分支 = 下面的 ILA / 毛刺计数器 / 采样沿】**（详见 §1.32 与 §2.7）： v5 已实测：**结果 20/20 全对，但 win_bad 仍命中 ≈18/20 轮**（§1.32 ①）⇒ v5 的 45° 阈值 没打在真凶（单通道半量程错值）身上。判据 = 20/20 且 |OFS_S| ≤ 400、各轮差 ≤ 几 LSB， **并且这次要求 0x0502[7]win_bad = 0、 0x0502[6:4]retry = 0**； 同一轮里顺便： ① 读 0x05F2 确认 **≥ 6**（v6 = 6 / v7 = 7 / v8 = 8 / **v9 = 9**）； ② 按 §1.32 ④ 的地址表读毛刺计数与窗口快照 （读**差值** ÷ 0x0540 周期脉冲差 = 每周期毛刺次数; int_max/int_min > 1.25 ⇒ 非整周期）； ③ ~~复看 ABZ 边沿抖动（毛刺不再进 loop_filter）~~ —— ✅ **已完成，见 §1.33。**

15. **【✅ 毛刺存在已被 ILA 直证（v6，2026-10-11）； ⭐ 采样沿改动仍未获支持 —— 见下】ILA 版需要"以毛刺本身为触发条件"**： dbg_ang_glitch / dbg_ang_delta 探针已在 ifdef ENC_TOP_ILA 下加好（rtl/enc_top.v 5b 节）， 但触发条件要在 Hardware Manager 里手动设成 dbg_ang_glitch == 1 —— tools/ila_debug.tcl 只建探针与深度（C_DATA_DEPTH 1024），**不含触发设置**；不设触发就只能靠 20.5 µs 随机窗口 去撞 ~750/s 的毛刺（概率 ~0.5%/次）。 **★ v6 起还有更省事的路**：§1.32 ④ 的 0x0505..0x050E 计数器已把"有没有毛刺、多大、多密"变成 UART 可读数字 ⇒ **先用寄存器判断要不要上 ILA。若 ILA 直证毛刺确实存在** ⇒ 再按 §1.19 / constr/enc_top.xdc:229-245 把 ADC 捕获沿改成上升沿→下降沿，届时全部回归与两版位流都要重跑。

    - ✅ **2026-10-11 已做到"直证"这一步**： ILA_MODE=stage2 （触发 dbg_ang_glitch == 1） 在 v6 上抓到**真毛刺**（§1.37 ③： s_raw 单点 +2307 LSB → angle_raw +1759 LSB = 9.66° → 滤波后 angle 仍外泄 +1334 LSB ≈ 7.3°）， CSV = _synth_work/ila_cap_stage2.csv。
    - ⭐ **但"改下降沿"这条动作目前证据不支持，如实记录**： ① tPD_meas ≤ 4 ns （§1.27，判据通过）； ② 该毛刺是**单点原始样本**偏离 9 点中值 +2307 LSB（1 kHz / 8 Vpp 下每 20 ns 输入只变 ≪1 LSB，纯采集撕裂很难产生这种量级），同时 stat_byte bit5 scamp （amp_low|amp_high） 置位、板上 0x053A = **0x0008（bit3 otr）** （§1.37 ⑤ 两条独立路径互证） ⇒ 更像**ADC 自身的模拟/过幅事件**，而不是捕获沿相位问题。 ⇒ 因此**采样沿保持上升沿不变**；把"改下降沿"留作**未实施的备选实验**（要做则必须 全回归 + 两版位流重跑，见 §1.19）。
    - ✅ **2026-10-11 已读毛刺计数器（补齐上面这条待办；生产 v7 位流在板上， tools/uart_spi.py --port COM6）**：
    - ⚠ **先记两条口径（这两条决定"怎么读才有意义"，第一版读数表就是错在这里，已撤）**： ① 计数器是**16 位单调事件计数、溢出后回绕、无饱和**（rtl/enc_top.v:506-522： 在 else if (cordic_valid) 里 +1， 阈值 ang_abs_delta > 8192/4096/2048/1024； gl_hold_cnt 数 angle_glitch） ⇒ **跨秒做差无效**，必须用"已知的短间隔"测差； ② 读地址 A 返回的是 **16 位窗口 {reg[A+1], reg[A]}**（rtl/uart_spi_bridge.v:30-32 的应答格式 0x5A|resp[31:24]|resp[23:16]|resp[15:8]|resp[7:0]|chk），**低字节在低地址**（rtl/reg_file.v:476-485 是 8 位/地址） ⇒ 0x0505 一词 = gl_cnt_1024， 单独读 0x0506 得到的是**错位窗口** {reg[0x0507], reg[0x0506]}，不是计数器值。
    - **有效读数（2026-10-11 18:44:47 / 18:44:48 / 18:44:51 三次带时间戳读）**： | 寄存器 | 读数 | 含义 | |---|---|---| 0x0505 词 = gl_cnt_1024 | T0 **0x1BD7 = 7127** → T1（≤0.88 s 后）**0x74F1 = 29937** ⇒ **≥ ~2.6×10⁴ 次/s**；同段 ~10 ms 内高字节 0x74→0x76（+512）⇒ 量级 **10⁴–10⁵ 次/s**；T2（3.16 s 后）回到 **0x74F1（与 T1 逐字节相同）** = 回绕所致 | |Δangle_raw| ≥ 1024 LSB (5.6°) 事件计数（**10⁴ 量级/s**） || 0x0507 词 = gl_cnt_2048 | 0x11AE = **4526** | ≥ 2048 LSB (11.25°) || 0x0509 词 = gl_cnt_4096 | **0x0003 = 3** | ≥ 4096 LSB (22.5°) || 0x050B 词 = gl_cnt_8192 | **0x0003 = 3** | ≥ 8192 LSB (45°) || 0x050D 词 = gl_hold_cnt | 自由运行（单次读数无意义） | angle_filt 保持计数 | | 0x053A | **0x0042** = bit1 amp_low + bit6 **crcerr**（**bit3 otr = 0**） | 输入幅度仍偏低； **BiSS CRC 错标志新置位**； ADC **没有**过幅 | | 0x05F2 / 0x0302 / 0x0101·0x0102 | 0x0007（v7）/ 0x0004（kp=4, ki=0）/ 0（inter=0、div=0） | 位流版本与增益档确认 |
    - ⚠ **诚实限定（关键，两次读数都成立）**： 输入条件**仍然是退化的** —— 0x053A bit1 amp_low **依然置位**（用户 18:44 前已把 AFG 接回，但幅度仍低于下限 ⇒ 信号没有真正以 8 Vpp 进 sin/cos，或通道/电平不对），**小半径会被 CORDIC 除成巨大的角度噪声** ⇒ 上表 10⁴ 量级/s 的 gl_cnt_1024 **不能**当作"正常 8 Vpp 输入下的真实毛刺率"（§1.37 在 8 Vpp 下测到的密度是 **0.00444 次/周期 ≈ 4.4 次/s**，差 4 个数量级）。
    - → **对"是否改下降沿"的结论：证据不支持改，维持上升沿**（理由见上一段 ①②）； 同时 otr 全程未再置位 ⇒ 这些角度跳变更像"小半径放大"而不是"过幅或采集撕裂"。
    - ⏭ **要把这条彻底钉死，前置条件是"输入先健康"**： 0x053A bit1 amp_low 必须清掉 （AFG 真正 8 Vpp 进 sin/cos），然后**用已知短间隔**重读 0x0505 词（判据： 毛刺密集且 otr 从不置位 ⇒ 排除过幅； amp_low 已清而毛刺仍密 ⇒ 才值得再上 ILA 并用 dbg_ang_glitch 触发复看是不是采集撕裂）。**在 amp_low 清掉之前，本条无法收口。**
    - → ✅ **2026-10-13 收口（见 §1.68）**： amp_low 的前置条件**已满足** —— AFG 真正 8 Vpp 进 sin/cos 后 0x053A = **0x0000**（amp_low = 0），且用**已知短间隔求差**（两窗共约 36 s） 重读 0x0505 词得 **Δ = 0 ⇒ 毛刺率 ≈ 0 次/s。**∴ 表中 10⁴ 量级读数确系"输入不健康"的伪像， **本条"是否改下降沿"的结论维持不变（证据不支持改）。** ✅ **2026-10-13 复测已补**： tools/biss_frame_probe.py --probe A --n 200 在 v12 上 **A1–A4 全 PASS**（frame_cnt 0…41 单调锯齿、distinct 39、=41 命中 3/200、 frame_active 198/200、 crcerr 0/200），CSV 归档 deliverables/data/biss_probeA_v12.csv。⚠ 仍留的尾巴： **探针 B/C（故意违例： ma 10 MHz / sl 悬空）尚未在 v12 上复测**（P0-5 的剩余部分）。
    - ⚠ **本轮新发现待排查： crcerr （0x053A bit6）新置位**（此前同板读数为 0x0A82，无此位） ⇒ BiSS 帧 CRC 校验错过，很可能是接 AFG 时动到了 biss_ma/biss_sl 接线 （biss_sl 悬空 = 非 0 电平会破坏从站状态机，见 docs/08_bissc_dumb_host.md §2）； 排查法：按手册重跑 A 探针，或写 0x0109 关闭 BiSS/重下位流后观察该位是否复现。 → ✅ **2026-10-13 定案（见 §1.68 C）： 根因就是 biss_sl （J5-12）没接 GND。** 接上 GND 后 crcerr 清零、 0x053A = 0x0000，30 s 窗内 crcerr 全程 0；**并在 v12 上补做 A 探针通过**： tools/biss_frame_probe.py --port COM3 --probe A --n 200 --interval 0.002 --csv deliverables/data/biss_probeA_v12.csv ⇒ **A1 PASS**（无 >41）、**A2 PASS**（frame_cnt distinct 39、max 41、命中 41 三次）、 **A3 PASS**（crcerr 0/200）、**A4 PASS**（frame_active 198/200；仅 2 次为 0 = 帧间停顿）， 耗时 7.93 s， RTL_BUILD_ID(0x05F2) = 12。⚠ **诚实限定**： crcerr 是**粘滞**位（置位后不自动清）， 所以 0/200 只证明"本窗口内未再置位"；它同时也说明接线下 bit_pos 能完整走到 41 拍，不再有"帧中断"。 ⚠ **探针 B/C 在 v12 上的复测仍未做**（P0-5 剩余）； B 会让 crcerr 粘滞置位污染 bit6，故必须放在最后做。

16. **【✅ 2026-10-11 已修（v8 工作树；⚠ 尚未重综合 ⇒ v7 位流不含）】ABZ 过速时的"静默丢沿 + r_acc 无饱和"**（详见 §1.33 ④/⑥）：

    - **原现象**： inter*f_in/div > filt_valid （本配置 > 12.5 MHz）时边沿被**静默**钳在 filt_valid， lag_warn / lag_fatal 全程为 0；且 r_acc （13 位有符号）无饱和保护，约 **0.51 ms** 后 回绕成负 ⇒ acc_fire_bwd ⇒ **A/B 短暂倒转**（实测 1 ms 窗口内正向 10947 / 反向 1553 次）。
    - **已做（两条）**： ① r_acc **双向钳位**在 [0, acc_th-1]（回绕不可能，反向点火机理上消失）； ② 新增**独立无方向**过速计数 r_ovl（饱和 4095，每拍 +1 = 下界）， lag_mag = |r_slc_fwd − r_slc_bwd| + r_ovl 喂 lag_warn/lag_fatal。 **limiter_active 语义刻意不变** —— 第一版把钳位并进 r_slc_* 会让 tb_enc_top B8-limiter-within-spec 失败（limiter_hi = 满窗 4194304 clk；A/B 对照证明是本次改动引入， v7 原版同测试台 limiter_hi=0），详见 §1.33 ⑥ ③。
    - **证据**： tools\lint_all.ps1 -Clean = ERROR 0 / WARNING 0；全回归（探针已撤） **PASS 21/21 exit 0 @ 2026-10-11 17:09:21**（sim/logs/SUMMARY.md）； sim/logs/tb_abz_jitter.log PH3： lag_warn 47601 clk / lag_fatal 40205 clk （**旧阈值档**；同日晚些时候按 §3 item 18 (b) 重标定为 45°/180° 后重跑 = lag_warn 50000 / lag_fatal **47601** clk，两档下盲区都闭合，见 §1.33 ⑥ ②）、 正向 12500 / **反向 0**（修前 10947/1553）、 limiter 0 clk，PH3 六条判据全 ok（断言 16 → **19**）； tb_enc_top limiter_hi=0 clk、 [ok] B8-limiter-within-spec。
    - ⏭ **剩下的动作**： ① **重综合**（改动落在 AB 输出最核心的相位累加器上；新位流需按 §2.7/§2.8 那套 重新签核：时序/DRC/利用率 + 全回归 + ILA 版）； ② 诚实残留： r_slc_fwd/r_slc_bwd 的**单向衰减**是 v7 既有结构缺陷， ablimit>0 + 方向反转时会把 limiter_active/lag 粘住（修法：任一方向执行边沿都可

衰减，或加泄漏） ⇒ 列入 v8 候选。

- **使用侧规避（仍然有效，已写进 §1.33 ③/④ 与测试台输出）**：保持 `inter*f_in/div ≤ 6.25 MHz`（本配置 ⇒ `f_in ≤ 24.4 kHz`）；10…20 kHz 想跑满 256 细分则把 `LOG2DEC` 由 2 降到 0。

17. **【v7 已实施最小项，其余留 v8】ABZ 输出"抖动群"的根因与唯一有效旋钮已上板定案**（详见 §1.37 / §1.38 / §1.39 / **§1.40**；**v7 位流 = §2.8**）：
    - **一句话机理**：`rtl/loop_filter.v:158-166` 的环路状态**每个 clk 都更新**，而 `in_valid` 只有 1/4 频率（12.5 MHz）⇒ **I 项每样本增益 ×4**（板上默认 `0x0302 = 0x1C` ⇒ kp=4 / ki=3）⇒ 环路欠阻尼 ⇒ `angle` 节拍抖动 117…191 LSB（原始 CORDIC 角只有 ~23 LSB）**> ABZ 索引单位 64 LSB** ⇒ `abz_gen` 正反点火成对抵消（净计数仍对）。离线全链路回放（`sim/tb_loop_abz_chain.v`，6 遍窗口 / 理想 125 计数）：**ki=3 ⇒ 482 跳变（3.86×）/ 816 次方向反转；ki=1 ⇒ 259（2.07×）；ki=0 ⇒ 126（= 理想）/ 0 次反转。★** 板上对照：**自由运行的 stage1 基线 = 57/20.8 = 2.7×**，而 §1.37/§1.38 里的 139（6.9×）是 **stage2 毛刺触发窗口**（含毛刺余波）⇒ **2.7× 才是真实基线。**
    - **⓪ ✅ 已上板实测完成（零风险、无需重综合，见 §1.40 ②）**：同一 8 Vpp / 1 kHz / 90° 输入、同为立即触发窗口下，**`0x0302 = 0x001C`（默认 kp4/ki3）⇒ 57 跳变（2.7×）、增 39/减 18；`0x0302 = 0x000C`（ki=1）⇒ 22（1.1×）、增 22/减 1；`0x0302 = 0x0004`（ki=0 纯 P）⇒ 20 = 理想（1.0×）、增 20/减 0、间隔 44/48/52 clk 近均匀、`angle` 残差 RMS 最小（0.067°，默认 0.126°）**；三档 `lag_warn/lag_fatal` 全 0、净计数都对。 ⇒ **v6 上的动作是"上电后写 `0x0302 = 0x0004`"**（运行期、掉电恢复 0x1C）；**v7 已把该值做成复位默认（§2.8）⇒ 开机即纯 P。纯 P 是 I 型环路，高速档必须先量跟踪误差**：理论滞后 ≈ `速度 ÷ (1/64)`（每拍 P 增益因 50 MHz 迭代被放大 4 倍）= 1 kHz 时 ~21 LSB ≈ 0.115°（可忽略）、10 kHz ≈ 1.15°、100 kHz ≈ 11.5°。
    - **① 运行期治标（已实测）：AB 迟滞（`0x0103`）无效** —— 用统一 stage1 窗口、同一增益复测：hyst 0 ⇒ 57（2.7×）、2 ⇒ 66（3.1×）、4 ⇒ 86（3.9×）、8 ⇒ 81（3.9×），**单调变差**（放行时把累积的 `32·hyst` 个角度 LSB 一次挤出 ⇒ 边沿更不均匀）。**§1.38"上电写 `0x0103 = 0x0002`"那条建议作废**（原结论是拿 stage2 基线对比 stage1 扫描行的跨窗口假象）。`hyst=2` 仿真丢计数 vs 板上不丢的矛盾降级为残留（既然无用，不再投入）。
    - **② RTL 级（v7，已实施）：(0) ✅ 已完成 —— `rtl/reg_file.v:280` 的复位值已由 `8'h1C` 改成 `8'h04`**（连同 ST/MT 22/10、`RTL_BUILD_ID` → 7 ⇒ **v7 位流见 §2.8**）⇒ **开机即纯 P**，不再需要主机写寄存器；(a) 环路状态按 `in_valid` 门控（`rtl/loop_filter.v` 已加 `parameter integer GATE_STATE`，默认 0 = 现状）+ **kp/ki 成套重调**（只改门控 = 把增益降到 1/4，实测会变成"慢而大幅"的摆动：`|d|` 最大 3331 LSB ≈ 1.8°，计数对但边沿位置在大范围漂）；(b) 给送进 `abz_gen` 的角度加一级"只在 `in_valid` 拍更新"的速率限幅 / 一阶低通；(c) `LOG2DEC` 2→0 把边沿栅格从 80 ns 降到 20 ns 再配强迟滞。**(a)(b)(c) 都不为竞赛截止（2026-11-04）冒重综合风险。 ★★ 2026-10-11 当晚更新：用户已决定采用 (a)**（见 §1.43 全链路 A/B 回放定案 = `GATE_STATE=1` + kp=2/ki=3），RTL 已改、**尚未重综合**；上面这句"不为竞赛截止冒重综合风险"是当日早些时候的判断，**已被该决定取代**（(b)(c) 仍未做）。
    - **验证手段已就绪**：`_synth_work\gain_ab_test.ps1`（**一次下位流 + `ILA_MODE=capture` 反复抓**，见 §1.40 ①；★ 直接重下位流会复位 `0x0302`/`0x0103`）+ `_synth_work/abz_chatter.py <csv>`（板上 CSV 的 AB 跳变数 / `edge_cnt` 净增量 / 增-减 / 放大倍数 / 间隔中位 / 输入侧抖动）+ `tools\run_sim.ps1 -Only tb_loop_abz_chain`（全链路回放比选，结果表见 §1.39 ⑤），基线 CSV 见 §1.40 ②③。

18. ✅ **【2026-10-11 已修（v8 工作树，未重综合）】`lag_warn/lag_fatal` 的标称角度与实现差 4×**（详见 §1.33 ⑦）：
    - 原实现是 `lag_warn: lag_mag*div >= 2*inter`、`lag_fatal: >= 4*inter`，按 `4*inter` 步 = 360° 的几何 ⇒ 实际在 **180° / 360°** 才报警；而 `rtl/abz_gen.v:47-51` 与 `model/abz_uvw_model.py:36-38` 标称 **45° / 180°**。实测（`python tools/lag_thr_probe.py`，已入库；inter=256/div=1 慢速斜坡）： 首次 warn 时真实滞后 **180.7°**、首次 fatal **361.4°**。
    - 因为 `r_slc_*` / `r_ovl` 都是"每拍最多 +1"，角度跑得越快、同一阈值对应的真实滞后越大 （原实现斜率 1024 LSB/拍时 warn 已在 **2907°**）⇒ 该监视器只能当"严重跟不上"的粗警报。
    - 影响：`lag_fatal` 进 **BiSS E 位**（`rtl/enc_top.v:730`）、`lag_warn` 进 `0x053A` bit2（`:756`）。
    - **定案 = (b) 重标定**（用户 2026-10-11 选定）：`rtl/abz_gen.v` 改为 `LAG_WARN_DIV=2` → `lag_warn_th = (inter+1)>>1`（45°）、`LAG_FATAL_MUL=2` → `lag_fatal_th = 2*inter`（180°）； `model/abz_uvw_model.py` 常量与判据同步，**并补齐 item 16 的双向钳位与 `r_ovl`**（原模型只有 v7 语义 ⇒ 过速时与 RTL 分叉）； `rtl/SPEC_interface_contract.md` 行为 4b 与 `tools/lag_thr_probe.py` 同步。
    - **重标定后复测**（同一探针）：64 LSB/拍 → warn = **45.0°**（lag_mag 128）、fatal = **180.7°**（lag_mag 512） ⇒ 与标称一致；256 LSB/拍 → 91.1° / 362.1°；1024 LSB/拍 → 365.3° / 1450.5° ⇒ "下界/粗警报"这条性质仍在，已如实写进 SPEC 行为 4b 与 §1.33 ⑦。
    - ⏭ **剩下的动作**：与 item 16 一起**重综合**（重签核 + 全回归 + ILA 版）；v7 位流不含本改动。

19. ★★★**【v8 已重综合 + 已上板 + 读回校验 ✅ 2026-10-11 19:24】审计建议 (a) 落地：`GATE_STATE=1` + kp=2/ki=3（取代 v7 的"纯 P"默认）**（详见 **§1.43**、上板首验 **§1.45**；用户 2026-10-11 选定 (a)）：
    - **内容**：`rtl/enc_top.v:532-542` 的 `u_filter` 显式传 `.GATE_STATE(1)`（环路状态回到 per-sample 语义，根因见 §1.39/§1.40/§1.42）； `rtl/reg_file.v:280` 的 `r_filt_k` 复位 `8'h04` → **`8'h1A`**（kp=2 / ki=3）； `RTL_BUILD_ID` → **8**；`rtl/loop_filter.v:71-82` 参数注释改写（模块默认仍 0）。
    - **依据**：`sim/tb_loop_abz_chain.v` A/B 双链一次运行同时给出两链结果（`sim/logs/tb_loop_abz_chain.log:150-165`）—— **B 链 + kp2/ki3：跳变 126 = 理想、净 126、`|d|` 最大 11 LSB、0 次方向反转**；对照 A 链 kp4/ki3（v6 默认）482 跳变 / 816 次反转，只开门控仍 kp4/ki3 则 `|d|` 冲到 3331 LSB ≈ 1.8° / 72 次反转 ⇒ **门控必须与 kp/ki 成套重调。**
    - **取代关系**：v7"开机即纯 P（`0x0302 = 0x04`）"的推荐口径**自 v8 起作废**；v8 上板应读回 `0x05F2 = 8` 与 `0x0302 = 0x1A`（✅ 实测已中，见 §1.45）。纯 P 保留为可运行期写入的对照档。已同步： `docs/01`（`RTL_BUILD_ID` 表、`0x302` 位域 `[4:3]` 订正 + 三版默认值）、`docs/04:334`、`docs/07:151`、`docs/08:4`、`sim/tb_loop_abz_chain.v` 注释。
    - ✅ **已完成**：① v8 生产位流重综合（`_synth_work/enc_top.bit` 1,075,596 B @19:11:30；Setup 26 失败端点 / WNS −3.274 / TNS −84.267，与 v7 的 26 / −3.364 同集合）； ② 烧录 + 上板读回校验（`0x05F2 = 8` / `0x0302 = 0x1A` / `0x0500 = 22` / `0x0501 = 10`，见 §1.45）； ③ 全回归 **21/21 PASS**（2026-10-11 19:08:49，`sim/logs/SUMMARY.md`）； ④ **v8 ILA 位流重建 + stage1 自由运行窗口 + `0x0302` 五档 sweep（1 kHz 三档 + 10 kHz 五档，见 §1.46 表 A/表 B）** —— 默认档 AB 边沿 21 次（1 kHz）/ 207 次（10 kHz，上限 206.8）**均 ≈ 1.0× 理想**，逆环边沿**全 0**（与 v7 纯 P 相同 ⇒ 门控 + kp2/ki3 未破坏边沿率）。
    - ~~⚠ **该项的一个不利实测（§1.46 结论 ②⑥）**~~ ⇒ ✅ **2026-10-11 由 §1.48 定案（结论反转）**： 1 kHz 下三档边沿率不可区分（都 ≈1.0×）、`angle` 线性残差 RMS 表面上**纯 P 最优**（`0x04` = 0.009°，`0x0C` = 0.026/0.029°，v8 默认 `0x1A` = 0.050°），10 kHz 下该"优势"消失（0.082° ≈ 直通档 0.092°）⇒ 一度记为"选 (a) 的依据至今在板上既未证实也未否证"。 **★ 改用"稳态速度滞后"判据后（`tools/lag_probe.py`，§1.48）**：纯 P（Ⅰ 型）滞后 **+1.875° @1 kHz / +18.259° @10 kHz**，与解析式 `1.8432e-3·f_in` 吻合到 1%；v8 默认（kp2/ki3，Ⅱ 型）**−0.006° / −0.069°（≈0）**； v7 无门控纯 P 实测 +0.474°，正是门控后（1.875°）的 1/3.96 ⇒ **"纯 P 残差 RMS 更优"是拟合扣掉滞后的指标假象，选 (a) 的收益已在板上证实**（位置精度差 1.88°/18.26°，折算 20 bit 单圈约 5.5k/53k count）。
    - ⏭ **剩下的动作**：① **想再压环路差异只能改 `inter`**： `inter_n = 256` 时 20 kHz 输入需要 20.48 MHz 边沿率 > 12.5 MHz 天花板 ⇒ **"20 kHz 三档 sweep" 在原配置下做不了**（会被静默钳在 12.5 MHz，见 `:1629-1640`）； 正路是运行期写 `0x0101 = 0x0020`（`inter = 32` ⇒ 128 边沿/电周期，20 kHz 只需 2.56 MHz）后在 12 kHz（98% 天花板）~20 kHz 重扫五档 + 顺环/逆环分解（**主判据改用滞后**；顺带可在 ≥24.4 kHz 时把纯 P 的 `lag_warn`（45°）压出来，见 §1.48 ⏭），或走 `LOG2DEC` 2→0"的 v10 候选路线（需重综合）。 ✅ **2026-10-11 已完成**： `inter = 32` 的 12 kHz / 20 kHz 两轮五档重扫见 **§1.48 补 2 /**

补 3（四点滞后曲线一致到 0.4%、逆环全 0），并顺带在 20 kHz 用 `inter = 256` 做出**钳位判决性演示**（0.303×，`lag_warn` / `lag_fatal` 全窗为 1）；② ~~Z 脉宽示波器实测（见 item 20，注意判据随 f_in 变）~~ ✅ **2026-10-11 已完成**（Z 率 156 Hz + 脉宽相符，见 item 20）；③ 已知未跟随项：`model/loop_filter_model.py` 的 `FS` /带宽表仍按每 clk 更新（该文件已加 ⚠ 说明，权威改为 A/B 回放台）。

20. ★★★**【真缺陷，v8 工作树已修，未重综合】集成路径 Z 脉宽被 `z_window` 截成 2 clk（40 ns），`z_width` 形同无效**（详见 **§1.44**）：
    - **现象/证据**：`sim/logs/tb_enc_top.log` 两版（v7、v8）都是 `Z_width_max=2`；三个纯 ABZ 测试台都把 `.z_window(1'b1)` 常开 ⇒ 该路径在单元台里**永不出现**（假通过），集成台旧判据 B5 = `[1,16]` 又太松。
    - **根因**：`z_window` 是电平（`rtl/enc_top.v:615`）且 `pole_counter` 的 wrap 有 2 clk 延迟 ⇒ 装载后 ~2 clk 窗就关；原实现把"脉冲推进 + `Z_TIMEOUT` 兜底"也放在 `if (z_window && abzen)` 内 ⇒ 立刻被清。
    - **修法**：`rtl/abz_gen.v` 拆出 `z_in_win/z_load/z_step_fin/z_timeout`，"Z 通道"时序块改为"装载受窗门控、推进与超时不受窗门控"，沿检测 `r_z_prev_hi` 独立无条件跟随 `z_ph_hi`（否则收尾后残留 prev=1 会立刻触发第二个 Z）。
    - **验证（集成路径）**：`tb_enc_top` 新增 **C5**（TEST C 反向段才是本配置下真正出 Z 的地方）⇒ `REV Z 脉宽: max=2042 拍`（理论 2048 = 4 边沿步 × 512 拍/步，修复前 2 拍）、`TEST PASSED (106 checks)`；B4 改为相位感知（v8 实测 `zw_rise_cyc=4128792` ⇒ 该 Z 落到下一圈 ⇒ 本圈期望 0，v7 是 `zw_rise_cyc=0` ⇒ 期望 1）、B5 收紧为 `[1024,4096]`。
    - ⏭ **剩下的动作**：随 item 19 一起重综合（✅ 已完成）；~~上板后用示波器量 Z 脉宽~~ ✅ **2026-10-11 已上板实测**（示波器探 `J5-7` / `M21`）：
    - **Z 重复率 = 156 Hz**（AFG 10 kHz ⇒ `f_in/64` = 156.24 Hz 期望）⇒ `docs/06:543` 判据 5 首次板级直证；
    - **Z 脉宽与预期相符**（`z_width` 临时写到 15 ⇒ 10 kHz 下期望 **1.465 µs**，用户读数确认"没问题"）；v7 及以前该值只有 **40 ns**（被门控窗截断，差 ~36×）⇒ v8 修复在集成路径上**得到板级证实**；
    - ⚠ 用户只给定性确认、未留具体示波器数字/游标截图（要进正式报告需补一张）；
    - ★ 判据通式 `宽度 = z_width / (1024 × f_in)`（1 kHz ⇒ 3.9 µs @z_width=4）；量完已恢复 `0x0108 = 0x4004`。
    - ★ 写 `0x0108` 必须带高字节（`0x400F` / `0x4004`）：包模式一次事务写 2 字节，写 `0x000F` 会清掉 `0x0109` 的 `abzen`（详见 §1.26 陷阱条）。
21. ★**【规格 vs 实现：2026-10-11 板上二值实验确认 → ✅ 同日按手册修入 v9，已闭环】** Z 脉宽比 TW28 手册公式少 1 个原始边沿步（详见 §1.47 与 **§1.49**）**——本条现已修复**：
    - TW28 手册口径 = **`(zwidth + 1) / div`** 个边沿（`docs/TW28_digest.md:365`、`:786` 第 19 条、`docs/01_technical_solution.md:332`、`:596`）；★ 手册同时写明 **Z 旁路 post-AB 分频**（`docs/TW28_digest.md:372`、`:783`）⇒ 以**原始 AB 边沿步**计就是 `zwidth + 1`，两条陈述自洽。
    - **v8 及以前**的 RTL = **`z_width` 个边沿步**（`assign z_pulse_n = (z_width == 4'd0) ? 4'd0 : z_width - r_z_left` 为 `reg [3:0]`）。
    - 实证 ①（仿真）：`sim/logs/tb_enc_top.log` C5 段 Z 高电平 **2042 拍** ≈ `z_width`(4) × 512 = 2048（按手册 5 步应 2560）⇒ 旧实现确实是 `z_width` 步、无 `+1`。
    - 实证 ②（★ 板上二值实验，2026-10-11，AFG 1 kHz，`0x0101 = 0` / `0x0102 = 0` ⇒ `inter_n = 256`、`div = 1`，`0x0109 = 0x0040`，探头在 `J5-7`）：写 `0x0108 = 0x4000`（`z_width = 0`）⇒ **Z 上一个脉冲都没有**；写回 `0x4004` ⇒ **15.625 Hz、3.906 µs 窄脉冲立即回来。**手册在 `div = 1` 下 `zwidth = 0` 应给 **1 个边沿步** ⇒ **语义不同，偏差成立。**（★ 该"0 个脉冲"是 **v8 语义**下的观测；v9 修复后同一实验已用 ILA 复测为 **1 个边沿步**，见 §1.26 板上实证块。）
    - ✅ **v9 修复**（2026-10-11，用户选定"按手册修"）：`rtl/abz_gen.v` 的 `z_pulse_n = {1'b0, z_width} + 5'd1`、`r_z_left` 加宽到 `Z_PULSE_W = 5`、装载拍 `r_z_out <= 1'b1`；同步 `sim/tb_abz_gen.v` / `sim/tb_enc_top.v`（B5 收紧为 `[2304,2816]`）/ `sim/tb_abz_jitter.v` / `model/abz_uvw_model.py`；**全回归 21/21 PASS**，`tb_enc_top` 的 REV Z 脉宽 **2558 拍**（理论 2560）⇒ 集成路径已确认 `+1`。完整记录见 **§1.49。**
    - ✅ **已上板闭环**（2026-10-11）：归档 `pre_v9/` → 重综合（`0x05F2` = 9 读回 ✅、`0x0108` = 0x4004、`0x0302` = 0x001A）→ **ILA** `ILA_TRIG=z_out` **实测**：`z_width = 4` ⇒ 240 拍 = **5 个边沿步** ✅、`0x0108 = 0x4000`（`z_width = 0`）⇒ 60 拍 = **1 个边沿步** ✅（把上面"0 个脉冲"的 v8 观测正式反转）；示波器旁证待补（探 `J5-7`，1 kHz/`z_width = 4` 期望 **4.883 µs**）。详见 §1.26 板上实证块与 **§1.49。**

> ★ 顺手清理（本轮）：`sim/logs/tb_calib_bnd.log` 是一份**孤儿日志** —— 产生它的 `sim/tb_calib_bnd.v` 已经删除（覆盖面由 `sim/tb_calib_min.v` 承接），而 `.log` 被 `.gitignore` 忽略、删测试台**不会**连带删掉日志，于是那份 `TEST FAILED (5 error(s) / 38 checks)` 的旧失败现场（正是 §1.9 那个缺陷**修复前**的记录）就一直躺在 `sim/logs/` 里，极易被后来的人（包括我自己）误读成"当前有失败"。已归档为 `sim/logs/_ORPHAN_tb_calib_bnd_pre-fix.log` 并加了来源说明头，另新增 `python tools/tidy_sim_logs.py` 可随时体检（列出所有"源测试台已不存在"的日志）。**教训：绿/红状态的判据永远是 `sim/logs/SUMMARY.md`，不是 `sim/logs/` 里随手一份 .log。**

14. **【上板待办 / 顺序不能颠倒】** 2026-10-10 已完成 `docs/07_board_checklist.md` §1.1–§1.7（连接、烧 ILA 版、抓 100 kHz 与 10 kHz 两份波并跑通 `tools/ila_report.py`）。剩下的是：> ★ **2026-10-11 更新**：**v6 位流已烧录并实测通过**（20/20 GOOD、`retry {0:20}`、> `win_bad 0/20`、毛刺密度 0.00444 次/周期且全落在 5.6°…11.25° 档、窗口比值 1.0006）> ⇒ §1.32 ⑦ 与 §2.7 的判据**全部达成**，"烧 v6 + 20 轮 + 读诊断"三项已完成；> 下面清单里 **v6 的 ILA 版已于 2026-10-11 14:06:09 重建完成**（§2 顶部 14:06 那条；探针 36 端口 / 269 bit）

    —— 而**它已于 2026-10-11 15:00 前后上板并全通**：stage1（触发 `dbg_adc_valid`）= 位流下载 + `.ltx` 探针 + 触发 + 导出 CSV 全链路打通，stage2（触发 `dbg_ang_glitch == 1`）**抓到真毛刺并逐级定案**，见 §1.36 / §1.37（一键脚本 `tools/ila_capture.tcl`，含 `restore` 模式可把生产位流放回）；ABZ 抖动群另由 §1.38 定案（`0x0103` AB 迟滞默认 0）—— ★ **但该节的"上电写 `0x0002`"建议已由 §1.40 ③ 实测推翻（迟滞无效、单调变差），正确动作是写 `0x0302 = 0x0004`（纯 P，边沿 1.0×），见 §1.40 ②。** ★ **v7 位流（770,600 B @15:51:26）已建成、已上板、已读回校验通过**（§2.8）：`0x05F2` = **7**、`0x0302` = **0x04**（开机即纯 P）、`0x0500/0x0501` = **22/10**；v7 的 ILA 版也已重建（**1,164,160 B @16:22:42**，36 端口 / 269 bit；`pre_ila_v6/` 里存着 v6 的两版）。★ **自由运行窗口已于 2026-10-11 16:26:21 抓完**（v7 ILA 版 + `ILA_MODE=stage1`，整片复位后无任何主机写）**并写进 §2.8**：AB 跳变 **21 vs 理想 20.7 ⇒ 1.0×**、`edge_cnt` 净 +21（21/0，零反向点火）、间隔只剩 1 拍量化、`angle` 线性残差 RMS **0.009°**（对照 v6 默认 ki=3：2.7× / 0.126°）⇒ **v7 的 ABZ 判据全部达成**；抓完已 `restore` 回生产位流。★ **v6 生产位流已被覆盖且未另存**（见 §2.8 末尾）；**v7 两版位流已归档到** `_synth_work/pre_v8/`（含 SHA256，见该目录 `_README.txt`）。**BiSS-C 哑主机联调**的手册已成文（`docs/08_bissc_dumb_host.md`，见 §1.34），**并且三个探针已于 2026-10-11 全部上板实测通过**（A 正常 / C 反例 / B 受控违例，见 §1.35；CSV 在 `_synth_work/biss_probe{A,B,C}.csv`）⇒ 清单里"BiSS-C 主机对接"的**功能性判据已达成**（★ 并已于 2026-10-11 在 **v7 生产位流**上复测 **A1 与 A2** 通过：`--probe A --n 80` ⇒ A1–A4 全 PASS、`frame_active` 80/80、`crcerr` 0/80、`frame_cnt` distinct=36 / max=41；`ma` 无时钟 ⇒ `0x053A` =0x0008 / `0x053B` =0 —— 手册 §5/§10 已补 v7 行，中间那次失败是 `J5-12` 接线没到位，见 §1.35 ④；**B/C 仍只在 v6 上验过**），只剩示波器补拍三张图（手册 §6）这类素材性动作（ABZ 抖动复看已由 §1.33 收口）。

    - **§1.8 量 `tPD_meas`**：✅ **2026-10-10 已量 = ≤4 ns（§1.27）⇒ 采样沿不改。** 判据复核（`docs/07` §1.8 / §1.19）：≤4 ns ⇒ 记录"实测干净"继续；≥5 ns 或 ILA 里 `angle_raw` 出现毛刺 ⇒ 把 `rtl/ad9238_capture.v` 三处 `always @(posedge clk_cap)`（`:79` / `:94` / `:139`）改**下降沿**捕获（理由与代价见 §1.19）。★ **剩下的唯一分支**：ILA/CSV 里是否出现过**真实的** `angle_raw` 毛刺（§3 item 15）。
    - 重新下载修好位序的 `enc_top_ila.bit`，确认波形窗里 `dbg_pole_pairs_eff` = **4**（§1.20）。

- 软清 OTR：SPI 写 `0x0405 = 0x0001`， `led[0]` (AB21) 应熄灭、 `irq_n` (K22) 回高（§1.21）。
- 写 `0x0109 = 0x00C0` （ `abzen|uwven` ）后看 `u/v/w` 与示波器 `J5-8/9/10` 。
- ABZ/Z 上示波器（ `J5-5/6/7` ）：**用 10 kHz 输入**验满速 256× 插补； 100 kHz 时 `inter=256` 要求 102.4 MHz 边沿率、已超 `filt_valid` 上限 ~12.5 MHz （见 §1.13 与 `docs/07` 的频率警示），要满速须把 `0x0101` 写成 `0x10` （inter=16）。
- BiSS-C 主机对接（ `J5-11/12/13` ）、SPI 回读、UART 遥测（2 Mbaud / 10 kHz / 18 B 帧）。

> ★ `tools/ila_debug.tcl` 必须放宽一次沙箱权限（ `danger-full-access` ）， 否则 IP 生成的子 vivado 进程管道会让父进程挂死。 `tools/impl_check.tcl` 不需要。 ⚠️ 实测： `tools/run_vivado.ps1` 跑 ILA 版时，**位流已经写完（`Bitgen Completed Successfully`）但脚本仍会挂住、最终 `ILA_EXIT=-1`** （2026-10-10 07:44、08:56、15:14 与 16:12 四次重建均复现）。 这是**外围脚本的收尾问题，不是构建失败** —— 判据永远是产物本身： `enc_top_ila.bit` **1,445,812 B @2026-10-10 16:12:01** + `.ltx` 67,006 B @16:11:50 + `impl_ila_*.rpt` 时间戳已刷新（@16:11:48–16:11:50）、且报 **`Timing constraints are not met.`** （ `sys_clk` `WNS −3.329 / TNS −84.745 / 26 失败 / 18010 端点` ，与生产版 16:05 同源；**不要再拿 08:56 的 `+0.527 / 0 failing` 或 15:14 的 `−3.268` 当判据**）、DRC 62 checks 全 `Related violations: <none>` （0 error / 0 critical）、 日志含 `MARK_DEBUG = 252 / 34 probe bases / 34 named probe ports` 。遇到 `ILA_EXIT=-1` 不要直接判失败，先核对产物时间戳与报告。 **★ 2026-10-11 14:06:09 的 v6 ILA 重建再次复现同一现象**（最新 36 端口 / 269 bit 那版）： 日志已含 `Bitgen Completed Successfully.` 、位流与 `.ltx` /三份报告齐全且自校验吻合（1,429,272 B）， 父进程仍挂住 ⇒ 已 `job_kill` 。**产物齐全即判成功**，别等脚本自己退出。

3.A 【2026-10-13 补记】v11 收口证据、v12 位流事实、仍未做完的清单

> 本节由并发工作的另一路会话追加（2026-10-13 00:49 起草、07:07 定稿），**不改动上面 1–21 条的编号与结论**； 每条都带可复核的路径 / 时间戳。落笔与定稿时都确认过 **无 `xsim` / `xelab` / `xvlog` / `vivado` 进程在跑。**

1. **全量回归已在"纯 v11"上重跑通过**（补 v10/v11 改 RTL 后未重跑的欠账）： `rtl/enc_top.v` 最后改动 2026-10-12 21:13:13（v10 的 ODDR 相位 + `RTL_BUILD_ID` ），而当时 `sim/logs/SUMMARY.md` 仍停在 2026-10-11 20:39:05（v9）。做法： `git worktree add --detach _v11_verify HEAD` （HEAD `ff97456` ，树内 `RTL_BUILD_ID = 16'h000B` ）后在该隔离树跑 `tools/run_sim.ps1` ⇒ **`PASS 21 / 共 21`，exit 0，证据 `_v11_verify/sim/logs/SUMMARY.md` @2026-10-12 23:51:23**； `tb_enc_top` = PASS / 0 errors / 487 s（10-11 基线 157–159 s，慢因同期有并发 xsim/Vivado 抢 CPU）。 **结论：ODDR 相位改动对功能回归无害。** - ⚠️ **并发坑（已复现，勿再踩）**：同一工作树里并行跑两个 `run_sim.ps1` 会共用 `_sim_work/<tb>/` 与 `sim/logs/` ， `tb_enc_top` 会出 `NO-VERDICT` （日志在 `[TEST C]` 中途截断，mtime 与另一路的日志交错） —— 这就是 §1.0 记录的 xsim 文件映射崩溃模式，**不是 RTL 缺陷**；要并行必须各用独立工作树。
2. **v11 签核（当时核对）**：已被 §1.59 的 v12 取代）： `_synth_work/enc_top.bit` 1,086,600 B @2026-10-12 21:15:37； `impl_timing.rpt` @21:15:26 = `All user specified timing constraints are met.` （WNS +0.821 / TNS 0.000 / 0 失败 / 11310 端点）；读回 `0x05F2` = `0x000B` 。
3. **★ v12 位流（本项已由后续轮次补齐并背书，见 §1.59）**：生产版 `_synth_work/enc_top.bit` **1,042,208 B @23:59:27** （SHA256 `D93DEBF92034FF2329117CDE59EB83E0D95620306E7A944C6B0402E8C3AE7725` ）， `impl_util.rpt` @23:59:14 = Slice LUT **6298** （30.28%）/ Slice Registers **4074** （9.79%）/ Block RAM 0 / DSP 16 / Bonded IOB 50 / BUFGCTRL 1、 `impl_timing.rpt` @23:59:16 判据行 **`All user specified timing constraints are met.`** （**WNS +0.959 / 0 失败 / 11,275 端点**）、 `impl_drc.rpt` @23:59:18 = **0 Errors**、 `impl_build.log` 末行 `INFO: [Vivado 12-1842] Bitgen Completed Successfully.` ； 同树的回归日志 `full_regression_v12.log` 尾部 `PASS 21 / 共 21 ；非 PASS 0` （对应 `sim/logs/SUMMARY.md` @23:52:26）。 **2026-10-13 07:12 又补出 ILA 调试版**（ `enc_top_ila.bit` **1,428,064 B @07:12:11**、 `enc_top_ila.ltx` 71,337 B @07:12:00、 WNS +0.332 / 0 失败 / 20,165 端点、36 探针 / 269 bit）。 `RTL_BUILD_ID = 16'h000C` （UART 遥测帧布局 v2）； v11 已归档为 `_synth_work/enc_top_v11prod.bit` / `enc_top_ila_v11.bit` 。 **板上版本定名仍以读回 `0x05F2` 为准**（ `0x000C` = v12、 `0x000B` = v11）； 遥测偏移 9 = `0xA2` 是 v2 帧标记。
4. **仍未做完（素材与测量类；详见 `deliverables/photo_shot_list.md` §B 与 `deliverables/evidence_index.md` §8）**： - **P0**：Z 脉宽带游标示波器截图（1 kHz / `z_width=4` ⇒ 4.883 µs，目前只有定性确认）； `scope_tpd_v11.png` 导出（读数 3–4 ns 已有）； BiSS **A/B/C 三探针在现行位流上复测**（B/C 目前只有 v6 数据）。 - **P1**：~~李萨如校前/校后~~（ ✅ **2026-10-13 已落地**，含"注入失真 → 一键标定"可复现实验，见 §1.68 与 `deliverables/README.md` §2.4a）；~~频率阶梯 1k/10k/50k/100k/200k/500 kHz 的"频率–误差–延迟"表~~（**六点的"频率–误差"已齐**，见 §1.69/§1.71/§1.73/§1.74/§1.75/**§1.76-频率阶梯**与 `deliverables/evidence_index.md` §3A； ⚠ **仍缺"延迟"列** —— `lag_deg` 六点全空，必须下 ILA 位流 `_synth_work/enc_top_ila.bit` 跑 `freq_ladder.py --ila` 才能补）； 环路延迟实测（信号源过零 → A 输出跳变沿，预算 ≈830 ns）； 静态精度误差曲线。 - **P2**：板卡照片、接线详图、 `DEVICE_ID` 截图（ `0x05F0` ⇒ `5A 01 F0 10 0C B7` ）、时序报告整页截图、 **真实传感头 / 磁环片段**（答辩"送命题"的唯一解）。 - **P3**：示波器探 D11/D10 + s 通道、原始码口径的"时间–率"曲线、源端串阻 22–33 Ω（硬件侧，见上面第 7 条）。 - **演示视频** **`deliverables/video/` 仍未开拍**（目录里只有 `.gitkeep` ）； 器件批次复测做不了（无第二块 AD9238 模块， 报告已如实标注）。
5. **文档一致性（2026-10-13 00:49 已修，留痕备查）**：根 `README.md` 的 4 处 "19/19" 已改为 **21/21** （ `:69` 、 `:211` 、 `:225` 、 `:296` ，并保留 10-10 那一轮 19/19 的历史字样）， `:92` 与 `:297` 的 "硬件实测尚未做"清单已改为已完成项 + 真正剩下的项；资源数字已是 v11 口径 （LUT 6369 / FF 4088 / DSP 16 / BRAM 0 / IOB 50 / BUFGCTRL 1）； `docs/*.pdf` 过期项见第 11 条（已重导）。
6. **本机操作环境备注**： `tools/run_sim.ps1` 等脚本实际走 **Windows PowerShell 5.1** （ `pwsh` 不在 PATH）， 需先 `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force` ； `tools/md2pdf.py` 与 `tools/ila_debug.tcl` 都必须在**一次性 `danger-full-access`** 下运行（Edge 要给自己 profile 目录授 ACL / IP 生成的子 vivado 进程管道会让父进程挂死），2026-10-13 07:07 的 PDF 重导再次验证了这一点。
7. **【2026-10-15 新增】竞赛合规两项必交物已补齐**（这是当前工期的最高优先级，因为它**不做就直接扣分**， 且不需要动 RTL/位流）： - **技能包** `skills/` （**7 项**）： `README.md` + `golden-model-bit-exact.md` 、 `fixedpoint-width-safety.md` 、 `testbench-blindspot.md` 、 `adc-source-sync-constraints.md` 、 `deterministic-measurement.md` 、 `vivado-report-autocollect.md` 、 `repo-reproducibility.md` 。 每项统一六节结构，其中 **§4 失效条件** 与 **§5 提炼过程（从哪些失败或观察中总结）** 是指南点名要求的两节。 - **大模型协作记录** `docs/ai-collaboration-log.md` ： 用户提示词原话的三个权威落点 （ `docs/10_host_requirements.md:188` 附录 A、 `_synth_work/prompt_host.md` 、 `_synth_work/prompt_disp.md` ）、 **11 条带锚点的自我纠错轨迹**（每条按"用户原话 → 错误假设 → 暴露它的数字 → 修法+验证 → 可复用结论"五段式）、 智能体工作流三层结构（主智能体 / 子代理 / 确定性工具）与 4 条子代理使用约定、以及**这套方式的 3 条代价。** - 同步落点：根 `README.md` 目录树 + 新增 §2b + 提交前自查清单 3 条； `deliverables/design_report.md` **新增 §9「大模型协作记录与技能包」**（含 11 条纠错摘要表、工作流、7 项技能表、代价）； `README.md` 开发环境段由"Vivado 2020.2 或更高"更正为 **2026.1**（竞赛指南推荐版本）。 - ⏭ **仍未做**：仓库内 **52 个非 ASCII 文件名**的合规改名（核心交付物约 30 处，含 `deliverables/design_report.md` 、 `docs/00..13` 、 `rtl/SPEC_interface_contract.md` ），以及 `docs/` 下未成文的 `docs/13_display_ui_requirements.md` / `docs/dual-channel-host-feasibility.md` 的 PDF 重导。改名方案见 `skills/repo-reproducibility.md` §3.1。 （**⇒ 这两项已做完/已改名，见下面第 8、9、11 条**）
8. **【2026-10-15 新增】竞赛合规第三项：根目录 `LICENSE` + 厂商材料移出仓库**（同属"不做就直接扣分"类）： - 新根文件 `LICENSE` （MIT） + `NOTICE` （第三方材料逐条清单：每份材料的版权方、 它在本工程里被用来做什么、为什么**不**在 MIT 覆盖范围内、以及使用者该去哪里自己获取）。 - **厂商 PDF / 引脚表 `.xls(.xlsx)` 全部移出仓库**，统一放到 **`_refs/vendor/`** （已被 `.gitignore` 排除）； 文件名同步改成 ASCII （ `AD9238_carrier_pin_map.xls` 、

DLC35_io_occupancy.xlsx、 HSTAD9238_module_manual.pdf、 AD9238_carrier_schematic.pdf），两个脚本的查找路径也随之更新（tools/pinplan_check.py、tools/pinout_verify.py）。 - ★ **为什么要移出、而不是"反正在仓库里也没人看"**：iC-Haus 手册（docs/TW28_digest.md 的上游）与 MIT **不兼容**，把它放进 MIT 仓库等于对外声明"这些材料也在 MIT 之下"—— 一是法律上不成立，二是评审看到会认为是许可滥用。参考材料**可以引用结论、不能随仓库分发**。 - ★ **降级行为已实测**（原来会崩、现已修）：厂商表缺席时 tools/pinplan_check.py 的 A/B/C 三项与 tools/pinout_verify.py 的 [D] 项显示 **SKIP**，终版判据 [C2]/[D]（只依赖仓库内 constr/enc_top.xdc 与本脚本内嵌针表）照常给 **PASS**。修之前 tools/pinout_verify.py:318 会抛 TypeError: expected str, bytes or os.PathLike object, not NoneType（os.path.relpath(None, ROOT)）—— 这属于"缺输入就崩"而不是"缺输入就降级"，对**克隆仓库的评审**是硬伤，已改成显式 SKIP 文案。 README.md 新增「授权与第三方材料」一节说明这个降级是**设计好的**，不是缺陷。

9. **【2026-10-15 新增】竞赛合规第四项：仓库内文件名去中文（42 个文件）**—— 指南要求"仓库内所有目录名与文件名必须使用纯英文（小写字母、数字、下划线或连字符）"。 - **改名 42 个**：docs/00..13（含 .md 与同名 .pdf）、deliverables/设计报告* → deliverables/design_report*、 deliverables/选题说明* → deliverables/topic_proposal*、 deliverables/数据与证据索引.md → evidence_index.md、 deliverables/素材拍摄清单.md → photo_shot_list.md、 rtl/SPEC_接口契约.md → rtl/SPEC_interface_contract.md、 host/_s5_*/编码器*.png → encoder_*.png；用 git mv 保留历史。 - **交叉引用同步替换约 830 处 / 91 个文件**（含 .gitignore、 constr/enc_top.xdc、 tools/*.py 与全部文档）。脚本：_synth_work/rename_ascii.py、 _synth_work/rename_artifacts.py、 _synth_work/rename_examples.py；查残留：_synth_work/check_ascii.py。 - **判据（可复跑）**：git ls-files | ? { $_ -match '[^\x00-\x7F]' } **输出为空**，且 git ls-files --others --exclude-standard 同样为空。 - ★ **顺手做对的一件事：产物名也改 ASCII。** host/export_png.py 的 suggest_name() 原来生成 编码器波形_<时间>.png，现在生成 **encoder_wave_<时间>.png / encoder_abz_uvw_<时间>.png**（按当前视图导出差分用 _view 段，例如 encoder_wave_view_…png）；host/recorder.py 的 default_name() 生成 **encoder_record_<时间>.csv**。理由：这些文件会被写进 deliverables/data/ 与演示视频，中文名一旦跨环境（Linux CI、zip 解包、外文评审）就变乱码。 **用户机器上的保存目录仍保留中文** 我的文档\host_app记录——那不是仓库里的文件，竞赛那条约束不适用它，中文目录对用户更友好（host/recorder.py 的 default_dir() 已注明）。 改完实测：host/export_png.py 自检 **[10]/[11]/[12] 全 PASS**（encoder_abz_uvw_20231115_061320.png / encoder_wave_19700101_080000.png / _view 段判定）、 host/recorder.py 自检 13 项 PASS、 python -m compileall host tools model exit 0。 - ★ **踩到并修掉的一个真 bug（我自己写的改名脚本）**：rewrite_references() 里写了 for old, rep in sorted(REPL, …) 却又想**原地**排序，list.sort() 返回 None 使遍历直接失效、只跑掉前半张规则表 —— 结果"脚本自称改了 596 处、其实裸文件名规则一条没生效"。 教训与 skills/fixedpoint-width-safety.md 同一类：**"改完要能自己验"**，所以才补了 _synth_work/check_ascii.py 做**独立判据**，而不是相信脚本自己打印的"已改名 42 个"。 - ★ **刻意不改的地方（P2 备注，不是漏做）**：docs/DEV_STATUS.md 里 30 余处"当时真实存在过的文件名"（诊断报告.txt、编码器记录_20261013_224117.csv、 host_b4b_5波形放大.png、 host\dist_diag\host_app_诊断\…）是**历史日志的一部分**，改写等于伪造记录；它们的现役对应名见 host/README.md（已是 ASCII）。另一份未落盘的 docs/dual-channel-host-feasibility.md 已按编号规范改名为 **docs/14_dual_channel_feasibility.md**（★ 2026-10-15 更新：该文现已进 README.md §2c 文档索引；**该文的 PDF 已于 2026-10-15 补导**，见第 11 条）。 - ⏳ **待用户拍板（我没有替你删）**：git status 里有两个 D 状态的跟踪文件 host/_s5_out/编码器ABZ-UVW_20261013_194829.png 与 host/_s5_out/编码器波形_20261013_194829.png —— 它们在**本批改名开始之前就已经从工作树里消失**（更早的"空间清理"删掉了、删除未入暂存区），我没有动它们。★ **2026-10-15 已定案：git add -A -- host/_s5_out 把这两条删除入暂存**（文件在盘上确实不存在、目录都已消失，留下它们只会让 git ls-files 的 **非 ASCII 判据永远 FAIL** —— 那是竞赛硬性要求的第一条）。现 git ls-files 的非 ASCII 记录数 = **0**。⚠️ 入暂存同时也把本批 36 个改名一起暂存了（git status --short 的暂存条目 = 38），所以**下一次 git commit 会是一次大提交**；若你想拆开提交，请在提交前 git restore --staged 挑拣。 ★ **推翻了上一版的建议**：早先我写" docs/*.pdf 是否重导"由你拍板 —— 现在 docs/*.pdf 与 .md 一同被 git mv 改成 ASCII 名（docs/00_roadmap_30d.pdf 等），**PDF 内容还是改名前那一版**，所以 ① 文件名合规、② 内容里的旧名引用与新增章节 **没有**同步进去。要不要用 python tools/md2pdf.py <新名>.md 重导一遍（推荐，约 10 分钟），仍等你一句话。★ **2026-10-15 已定案并执行：用户答"重导一遍" ⇒ 已全量重导 28 个，见第 11 条。**

10. **【2026-10-15 新增】改名后的收尾核查：判据自己也修了一个 bug + "仓库外"的边界说清**—— - ★ **_synth_work/check_ascii.py 原来的正则漏判（真 bug，已修）**：旧判据用 [\w\-./\\\[\]]*\.[A-Za-z]{2,5}\b，而 **\w 不含中文 ⇒ 纯中文文件名（如 选题指南.pdf） 根本匹配不上**，"检查通过"是**假通过**。改成把 CJK 纳入路径字符集，且用 **"旧名字干白名单"**（OLD_STEMS：设计报告 / 选题说明 / 数据与证据索引 / 素材拍摄清单 / 接口契约 / 双通道上位机可行性分析 / 编码器ABZ-UVW / 编码器波形 / 诊断报告 / 本地解包 …）判定"真残留"，其余归入 **[D] 需人工过目**（不计 FAIL）—— 否则中文在正文里大量出现，报告会有上百条噪声、没人再看。**教训同 skills/fixedpoint-width-safety.md：检查器本身也要被检查。** - ★ **"仓库内"三个字只覆盖提交进去的东西**：盘上仍有非 ASCII 路径，但它们**全部已被 .gitignore / .git/info/exclude 挡住**，实测逐条确认（git check-ignore -v）：信号发生器AFG-2225/（.gitignore:52）、泰克示波器/（:55）、【正点原子】达芬奇-lite之FPGA开发指南V1.0(1).pdf（:50）、 host/_b4b_*.png（:133）、 host/dist_onedir/（:111）、 host/_exe_work_onedir/（:107）、 _pdf_out/（:27）、 _tools/（:32）、 _v11_verify/（.git/info/exclude:7）。 ⇒ **git ls-files 的非 ASCII 记录数 = 0**。但**评审若直接翻你拷过去的那个文件夹**看到的仍是中文名，所以 skills/repo-reproducibility.md 已补一段：真正的做法是**移出仓库**或**改 ASCII 名**二选一。 - ⏳ **仍在仓库里、且是"旧名引用"的只剩 3 处（都是有意保留 vs 待修，逐条说明）**：① host/README.md:124/:138/:139/:647 的 host\dist_local\host_app_本地解包.exe —— 那是 **2026-10-13 当时的真实产物名**（§1.77 的现场记录），且 host/dist_local/ 早已 gitignore；现役 **目录版** 的 name 已是 ASCII（host/host_onedir.spec ⇒ host\dist_onedir\host_app\host_app.exe）。 ⚠️ 但**盘上**还存在一个 **2026-10-13 用环境变量打出来的旧目录** host\dist_onedir\编码器上位机\（含 编码器上位机.exe / _运行日志.log，已被 :111 忽略）——**不是仓库内容、不影响合规**，但如果你要把它拷给评委，请再打一次 onedir 包（就会是 ASCII 名）； ② docs/14_dual_channel_feasibility.md:230/:232 的旧名处在 **§5.1"改名前现场快照"** 表内，已就地加 ✅ 状态说明（故意不改成新名，否则"待改"栏里写着已改好的名字，自相矛盾）； ③ docs/DEV_STATUS.md 内的历史日志行（见第 9 条尾注）。 - **顺手补齐的"文档漂移"（docs/14_dual_channel_feasibility.md §5.4 提的两处）**：rtl/SPEC_interface_contract.md:6、 rtl/enc_top.v:4、 constr/enc_top.xdc:4 的 "Vivado 2020.2+" 统一为 **2026.1（本机实跑）**；docs/00_roadmap_30d.md:75 / :254、 docs/01_technical_solution.md:731、 docs/07_board_checklist.md:223 的"降到 25 MHz" 回退项就地标注**已作废**（附 tPD_meas = 3–4 ns、WNS +0.959 / 0 失败端点的依据）。 同时给 docs/14_dual_channel_feasibility.md 的 §5.1/§5.2/§5.3/§5.4 各加 ✅ 状态说明，避免"缺口清单"被误读成"还没做"。 - **降级行为在"厂商资料缺席"下的真机复测**（把 _refs/vendor 临时移走再跑）：tools/pinplan_check.py 打 **未找到**（[A]/[B]/[C] 将标 SKIP）、 tools/pinout_verify.py 打 **未找到**（[D] 的占用冲突数将标 SKIP） + — SKIP：无板载占用表，未做冲突判定（不是 PASS，也不是 FAIL），两者 **RESULT 均为 PASS、零异常**；tools/sync_doc_numbers.py --check = 合计 0 条（未写任何文件）。

11. **【2026-10-15 新增】全部文档 PDF 重导（28 个，失败 0）**——改名后 .md 与 .pdf 文件名已同步，但 **PDF 内容还停在改名前那一版**（尤其 deliverables/design_report.pdf 是 10-12 导出的、**不含新增的 §9**）。本次全量重导消除该漂移。
    - **命令**：python tools/md2pdf.py --all（docs/ 下 26 个）+ python tools/md2pdf.py deliverables/design_report.md deliverables/topic_proposal.md（2 个）⇒ **导出 28 个，失败 0 个。**
    - ★ **必须一次性 danger-full-access**：沙箱下无头 Edge 起不来，固定报 [FAIL] … rc=2147483651 + crashpad_client_win.cc:456] OpenProcess: 拒绝访问。(0x5) + FATAL:mojo\public\cpp\platform\platform_channel.cc:187] Check failed: . : 拒绝访问。(0x5) ——这是**环境限制，不是脚本 bug**。提权后 --selftest 一次通过（_selftest.pdf 40,679 B，exit 0）。
    - **版本漂移确认已消除**（用 tools/pdf_text.py <pdf> -1 out.txt 抽全文复核）：旧 design_report.pdf = **27 页且全文搜不到"大模型协作记录"**；新 = **29 页**，含 9. 大模型协作记录与技能包、 9.2 自我纠错轨迹（摘要，共 11 条）、 9.3 智能体工作流（三层）、 9.4 技能包（skills/，7 项）。 deliverables/topic_proposal.pdf 新旧**都是 3 页**（重导未改变页数）。

- **首次产出 PDF 的 18 个**（此前从未导出、`git status` 为 `??`）：
  `docs/{05_vivado_sim_impl_report,07_board_checklist,08_bissc_dumb_host,09_scope_measure_log,10_host_requirements,11_host_selfcheck_s1_s5,12_host_ui_requirements,13_display_ui_requirements,14_dual_channel_feasibility,DEV_STATUS,DPO2024B_quick_ref,DPO2024B_user_guide,LCD_test_program_readme,LCD_test_program_requirements,LCD_touch_test_readme, RGB_LCD_wiring_check,TW28_digest,ai-collaboration-log}.pdf`； 另有 `docs/AD9238_module_digest.pdf`、`docs/ATK_board_digest.pdf` 内容更新（工作树 `M`）。
- **磁盘账**：`docs/*.pdf` 26 个 = **35.3 MB**、`deliverables/*.pdf` 2 个 = **2.4 MB**； HTML 中间件在 `_pdf_out/`（35 文件 / 2.3 MB，已 gitignore）。 ⚠️ 其中 `docs/DEV_STATUS.pdf` 单个 **11.8 MB / 88 页**（历史日志）——**不建议入仓库**； 其余 17 个新 PDF **也尚未 `git add`**（留给用户决定要不要随仓库提交）。 **本轮只重导，未改仓库跟踪状态**（后由第 12 条整体 `git add -A`）。

12. **【2026-10-15 新增】★ 纠正一个我此前说错的合规结论：厂商材料其实一直还在版本库里**—— 这是本项目上大模型协作最有价值的一次自纠错（"我说做了"与"真做了"的差别），已同步写进 `NOTICE` §1.1 与 `README.md` 自查清单。
    - **错在哪**：第 8 条我写"厂商 PDF / 引脚表 `.xls(.xlsx)` 全部**移出仓库**，统一放到 `_refs/vendor/`（已被 `.gitignore` 排除）"。实际做的只是 **`git mv` 到 `_refs/vendor/` 再往 `.gitignore` 加了一行 `_refs/`** —— 而 **`.gitignore` 对已被跟踪的文件完全无效。** 它们自最初那个基线提交 **`9e9aebc`（2026-10-09，"chore: 建立仓库基线"）** 起就一直是 **被 git 跟踪**的，所以 `git ls-files _refs` 会列出 6 个文件、 `git ls-files '*.pdf'` 里会混进厂商手册。**"文件不在仓库里"这句话当时是假的。**
    - **为什么没被发现**：我当时只验了"两个校核脚本在厂商表缺席时能降级"、 "`README` / `NOTICE` 文字写对了"，**没有跑我自己刚写进 README 自查清单的那条判据** `git ls-files "*.pdf"`。教训：**写完判据必须立刻跑一遍判据本身**（同 `skills/repo-reproducibility.md` 与 `skills/fixedpoint-width-safety.md` 的口径：工具/检查器自己也要被检查）。
    - **已做的修复**：`git rm -r --cached _refs/vendor`（**保留本地文件**）⇒ `git ls-files _refs` 现在是**空输出**、`.gitignore:31` 的 `_refs/` 规则开始生效 （`git check-ignore -v` 命中）、两个校核脚本仍 **RESULT: PASS**（本地文件还在，功能不受影响）。
    - **仍未解决的一半（已于 2026-10-15 当晚解决）**：这 6 个文件**仍在历史提交 `9e9aebc` 里**， `git ls-tree -r 9e9aebc` 能列出它们 ⇒ 当时**克隆仓库依然会下载到厂商手册。** 本仓库**没有配置任何 remote**（`git remote -v` 为空）、共 103 个提交， 因此重写历史是安全但会影响此后全部提交哈希。 `git-filter-repo` 未安装； `NOTICE` §1.1 已给出 `git filter-branch` 的可直接照抄命令。 **决定：先不重写（等用户拍板），但已把"推到 Gitee/GitHub 之前必须重写"写成显式前置条件。** ⇒ **用户随后拍板"重写历史"，已执行并验证**（见第 13 条）。
    - **本轮 `git add -A` 的结果**：暂存条目 **227**（工作树已无未暂存改动）； 新增 138 个文件 / 约 **34.35 MB**，其中 `_disp_work/`（7 寸屏那条线的 Vivado 工作目录） **36 个文件**、`docs/` 30 个（含 18 个新 PDF）、`lcd_test2/` 24、`lcd_test/` 18、 `skills/` 8（`README.md` + 7 份条目）、`tools/` 6、`host/` 5、根目录 4 （`LICENSE`、 `NOTICE`、 `7inch_UI_design_report.md`、 `fpga_ui_feasibility_report.md`）、 `disp/` 4、`deliverables/` 1、`constr/` 1、`sim/` 1。 ★ `_disp_work/` 与 `_synth_work/`（`.gitignore:8`）**性质相同**（都是 Vivado 构建工作目录） 却是**唯一一个没被忽略**的 —— 这 36 个 `.rpt` 属构建产物，建议仿照 `_synth_work/` 加进 `.gitignore` 并 `git rm -r --cached _disp_work`（**本轮未擅自处理，等用户拍板**）。 `docs/DEV_STATUS.pdf`（11.29 MB / 89 页历史日志）也已按 `git add -A` 入暂存， 如不想让它在仓库里，用 `git restore --staged docs/DEV_STATUS.pdf` 单独撤回即可。

13. **【2026-10-15 新增】★ 已执行 `git filter-branch` 重写全部历史，把 6 个厂商文件从每一个提交里清除**—— 用户对"提交前收尾"的答复是**只选**"重写历史，彻底清掉 6 个厂商手册"这一项。
    - **备份先做**： `git bundle create _refs/backup_before_filter.bundle --all` = **71.9 MB**， 路径已被 `.gitignore:31` 的 `_refs/` 排除 ⇒ **只留本机、不随仓库分发**，确认无需回退后可删。
    - **重写命令**（本仓库无 remote，安全）： `git filter-branch --force --index-filter "git rm -r --cached --ignore-unmatch _refs TW28_datasheet_D5en.pdf 'HSTAD9238模块说明书.pdf' AD9238V11标准版原理图.pdf Davinci_Lite_1V0.pdf '双AD9238标准版引脚映射表.xls' '达芬奇Lite开发板IO引脚分配表.xlsx'" --prune-empty --tag-name-filter cat -- --all` ⇒ **exit 0，103 个提交全部重写。**
    - **复核（判据全部为空/符合）**： `git rev-list --objects main | grep -iE 'TW28_datasheet|HSTAD9238|AD9238V11|Davinci_Lite|_refs/vendor|引脚映射表|IO引脚分配表'` = **空输出**； ★ 判据里不能写成宽松的 `TW28` —— 仓库里本来就有自建的 `docs/TW28_digest.md`（手册数值摘录）， 用 `TW28` 会把它一起打出来（实测 3 处命中全是 `docs/TW28_digest.md` / `.pdf`），判据就永远不空。 `git log main --oneline -- <各旧路径>` = 空；新基线 **`8f194eb`**（旧 `9e9aebc`）、 新 tip **`6a995e9`**（旧 `de59811`）； `git ls-files` 非 ASCII = **0**、跟踪文件 **414**； `.git` 体积 **135 MB → 70 MB**；工作树干净、无 remote。 **★ 判据必须限定到 `main`**：实测 `git log --all --oneline -- TW28_datasheet_D5en.pdf` **会打出旧基线 `9e9aebc`** —— 因为 `--all` 把下面那个本机 worktree 的 HEAD 也算作 可达根。 `--all` 不是"克隆会拿到什么"的正确判据， `main` 才是。
    - **★ 一个必须记住的操作教训**：我一开始额外建了 `backup-before-filter` 分支再执行 `--all`，结果 filter-branch **把这个"备份分支"本身也一起重写了**（`--all` 把它的 ref 也算进重写范围），**备份形同虚设。** ⇒ **真正的备份必须落在仓库之外**（bundle 文件或 外部副本），不能是同仓库里的一个分支/ref。同理，filter-branch 留下的 `refs/original/*` 也必须 `git update-ref -d` 逐个删掉，否则旧提交仍被 ref 钉住。
    - ⏳ **遗留一个纯本机现象（不影响克隆）**： 链接工作树 `_v11_verify`（`.git/info/exclude:7` 排除，1198 个文件）的 detached HEAD 停在**重写前**的旧提交 `ff97456`（2026-10-12）上，而 `git rev-list --all` 会把这个 worktree HEAD 也算作可达根 ⇒ 那些厂商 blob 仍留在**本机对象库**里， `git gc` 收不走。 **但这不影响任何克隆结果**（克隆只取 `main` 可达对象，已验证为空）。 不再需要那份 2026-10-12 v11 快照时，用 `git worktree remove --force _v11_verify` + `git gc --prune=now` 即可清干净。

---

4. 复现命令

```
# 回归（21 台）— 自动探测并回退 xsim 工作目录，直接跑即可
& powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_sim.ps1

# ABZ 边沿抖动量化（§1.33：16 checks，PH1/PH2/PH3 三相位，~5 s）
#   ★ `-Only` 会把 sim/logs/SUMMARY.md 覆盖成**单台**汇总，要恢复全量结论须重跑全量
& powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_sim.ps1 -Only tb_abz_jitter

# ---- 上板：ILA 一键采集（§1.36）★ 需要一次性 danger-full-access（cs_server 要往临时目录解包 DLL）
#   ILA_MODE=stage1 触发 dbg_adc_valid（等于立即触发，用来验链路）
#   ILA_MODE=stage2 触发 dbg_ang_glitch == 1（抓真毛刺，需要 ADC 上有真实正余弦输入）
#   ILA_MODE=capture 不下位流，只武装/触发/导出（§1.40 ①：做寄存器 A/B 必须用它，
#                    因为 program_hw_devices 会复位 0x0302/0x0103/标定结果）
#   ILA_MODE=restore 只下载位流、不碰探针（把生产位流放回板子）
$env:ILA_MODE='stage2'; $env:ILA_CSV='_synth_work\ila_cap_stage2.csv'; $env:ILA_WAIT='60000'
& powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 `
    -Tcl tools\ila_capture.tcl -LogFile _synth_work\ila_capture_stage2.log -Tail 20
$env:ILA_MODE='restore'; $env:ILA_BIT='_synth_work\enc_top.bit'
& powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 `
    -Tcl tools\ila_capture.tcl -LogFile _synth_work\ila_restore.log -Tail 12

# ---- ILA CSV 分析（§1.37）★ 别把输出管进 `Select-Object -First N`：broken pipe 会让它 exit 1（虚警）
python _synth_work\ila_report.py _synth_work\ila_cap_stage2.csv

# ---- ABZ"抖动群"量化（§1.38/§1.40）：AB 跳变数 / edge_cnt 净增量 / 增-减 / 放大倍数 / 间隔中位 / 输入侧抖动
#      ★ 比较时窗口类型必须一致（stage1/stage2/capture 不要混着比，§1.40 ③ 的教训）
python _synth_work\abz_chatter.py --inter 256 --div 1 `
    _synth_work\ila_cap_gain1c.csv _synth_work\ila_cap_gainki1.csv _synth_work\ila_cap_gainki0.csv `
    _synth_work\ila_cap_hyst2.csv _synth_work\ila_cap_hyst4.csv _synth_work\ila_cap_hyst8.csv

# ---- 环路增益 A/B 一键实测（§1.40）：一次下位流 + 三次 capture + 还原生产位流
#      需要一次性 danger-full-access；产物 _synth_work\ila_cap_gain{1c,ki1,ki0}.csv
& powershell.exe -NoProfile -ExecutionPolicy Bypass -File _synth_work\gain_ab_test.ps1

# ---- AB 迟滞（§1.38，★ 已由 §1.40 ③ 实测推翻：无效、单调变差，仅作记录，不要再照此设置）
& E:\Anaconda\python.exe tools\uart_spi.py --port COM6 write 0x0103 0x0002
& E:\Anaconda\python.exe tools\uart_spi.py --port COM6 read  0x0103 0x0104

# ---- 环路增益（§1.40 定案，★ v8 起默认已改，见 §1.43）：0x0302 = {ki[4:3], kp[2:0]}
#      v6 默认 0x1C = kp4/ki3；v7 默认 0x04 = kp4/ki0（纯 P）；★ v8 默认 0x1A = kp2/ki3（配 GATE_STATE=1）
#      上板实测（v6 位流）：0x1C ⇒ 57 跳变(2.7×) / 0x0C(ki=1) ⇒ 22(1.1×) / 0x04(ki=0 纯P) ⇒ 20 = 理想(1.0×)、零反向点火
#      ★ v8 上板要做的 A/B：默认 0x1A 与对照档 0x04 / 0x0C 三档 sweep（v8 位流尚未重综合）
& E:\Anaconda\python.exe tools\uart_spi.py --port COM6 write 0x0302 0x0004
& E:\Anaconda\python.exe tools\uart_spi.py --port COM6 read  0x0302

# ---- 全链路离线回放（§1.39）：先把 ILA CSV 的列导成 .mem，再喂 tb_loop_abz_chain
python _synth_work\dump_abz_angle.py _synth_work\ila_cap_stage2.csv _synth_work\abz_angle_raw_stage2.mem "dbg_angle_raw[15:0]"
& powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_sim.ps1 -Only tb_loop_abz_chain

# 只跑某几台（新增 .v 无需注册：脚本按 sim\tb_*.v 自动发现）
& powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_sim.ps1 `
    -Only tb_calib_min, tb_uart_scope -Rebuild

# 单台快速迭代
# ★★ 工作目录必须在**工作区之外**，否则 xsim 会崩（见 §1.0）。
#    在真实（非沙箱）终端里若想用仓库内目录，加 -NoAutoWorkRoot 强制不探测。
$env:Path = 'D:\Vivado\2026.1\Vivado\bin;' + $env:Path
$W = Join-Path $env:TEMP 'fpga_sim_work\my_iter'
New-Item -ItemType Directory -Force -Path $W | Out-Null
Push-Location $W
& xvlog --sv 'C:\Users\zhaoy\Desktop\FPGA\rtl\uvw_gen.v' 'C:\Users\zhaoy\Desktop\FPGA\sim\tb_uvw_gen.v'
& xelab -debug typical tb_uvw_gen -s s
& xsim s -runall
Pop-Location

# 指定 xsim 工作目录（覆盖自动探测）
& powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_sim.ps1 -WorkRoot D:\fpga_sim_work
```

```
# 引脚 / 模型自检
python tools\pinout_verify.py
python tools\pinplan_check.py
python model\abz_uvw_model.py

# UART 帧校验和约定守卫（RTL 实拍帧 -> 主机端解析，16 checks，见 §1.15）
#   改了 tools\uart_scope.py 或 rtl\uart_scope.v 的校验和约定，这一条会立刻 FAIL
python tools\check_uart_frame_fixture.py

# 上位机工具自检（不接硬件，200 帧合成数据 + 逐字段回读 + 破坏字节）
python tools\uart_scope.py --selftest

# sim/logs/ 体检：列出"源测试台已不存在"的孤儿日志（它们里的 FAILED 不是当前失败）
python tools\tidy_sim_logs.py

# RTL lint
& powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\lint_all.ps1 -Clean

# 编码关卡（CRLF / BOM / .ps1 缺 BOM 任一命中即 exit 1）
python tools\to_lf.py --check
```

> **★ 这个关卡只扫描 `git ls-files`（已跟踪文件），所以"commit 之前"跑它永远是绿的。 commit 之后必须再跑一次。** 2026-10-10 实测：`model/calib_shift_study_report.txt` 在 8bc224a 之前一直是未跟踪文件 ⇒ 从未被关卡看过；一提交进来，关卡立刻变红（worktree 里是 CRLF）。根因是 `tools/calib_shift_study.py` 用 text 模式写文件（Windows 上 `"\n"` 被翻成 CRLF），已改成 `newline='\n'` 并重跑该脚本。 注意这里有个**容易骗过人的细节**（★ 该脚本已按 §1.16 删除；取代它的 `tools/calib_shift_judge.py` 从第一行起就以 `newline` 显式指定 LF；**规则不变： 新脚本一旦被跟踪，commit 之后仍要再跑一次本关卡**）： 由于 `.gitattributes` 里 `*.txt text eol=lf`， git 在提交时已经把内容规范化成 LF 存进对象库，所以 **`git status` 是干净的、 只有 `to_lf.py --check` 会报红**（它读的是工作区原始字节）。⇒ 两者都要看。

★ 每次单台迭代**必须换新建目录**，否则复用旧 `xsim.dir` 会得到与源码不符的输出。 ★ 用 `edit` / `write` 改过 `tools/*.ps1` 之后**必须**跑 `python tools/to_lf.py` 补回 UTF-8 BOM，并用 `[System.Management.Automation.Language.Parser]::ParseFile()` 校验语法 —— 这两个工具会去掉 BOM，而 PowerShell 5.1 读无 BOM 的中文脚本会解析失败（见 §1.1）。

4.x 写 commit 信息的一个坑（2026-10-10 实测踩到并已修正）

提交信息一律**写进临时文件再 `git commit -F`**（临时文件名用 `_commitmsg*.txt`， `.gitignore` 已覆盖，提交完即删）。但**不要用 PowerShell 的 `Out-File -Encoding utf8` 生成这个文件**：Windows PowerShell 5.1 的 `utf8` 会写 **UTF-8 BOM**，BOM 会原样进 commit 主题，`git log --oneline` 里主题开头就多一个不可见的 `\ufeff`（本轮 `3810842` 就是这样，已用 `git commit --amend -F` 修正为 `6e3333e`）。 生成方式应为 `write` 工具，或 `[System.IO.File]::WriteAllText($p, $s, (New-Object System.Text.UTF8Encoding($false)))`。 复核： `git log -1 --format=%s` 取出后看首 3 字节是否为 `EF BB BF`。

4.y v7 重综合 + 文档导出（2026-10-11 新增）

> **★ v8 重综合（尚未做，见 §1.43/§1.44/item 19/20）只需把下面的 `_v7` 换成 `_v8`**， 并在烧录后核对 **`0x05F2` = 8**（v7 是 7）、 **`0x0302` = `0x1A`**（v7 是 `0x04`）、 `0x0500/0x0501` = 22/10（不变）； Z 脉宽上板判据从 40 ns 改为 `z_width` 对应值（§1.26 表）。

```
# ---- 全量回归（改了 RTL 必跑；★ 别单跑 -Only，它会把 sim/logs/SUMMARY.md 覆盖成单台汇总）
& powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_sim.ps1

# ---- v7 生产位流（不需要提权）
& powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 `
    -Tcl tools\impl_check.tcl -LogFile _synth_work\impl_build_v7.log -Tail 30

# ---- v7 ILA 位流（★ 必须一次性 danger-full-access：IP 生成的子 vivado 进程管道会让父进程挂死，见 §1.36）
& powershell.exe -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 `
    -Tcl tools\ila_debug.tcl -LogFile _synth_work\ila_build_v7.log -Tail 80

# ---- 板上确认 v7（RTL_BUILD_ID 必须 = 7；★ v8 位流这里应读到 8，且 0x0302 = 0x1A；ST/MT 读回应为 22/10）
& E:\Anaconda\python.exe tools\uart_spi.py --port COM6 read 0x05F2 0x0302
& E:\Anaconda\python.exe tools\uart_spi.py --port COM6 read 0x0500 0x0501

# ---- docs/*.md → 同名 PDF（§1.41：python-markdown + Edge 无头打印）
#      ★ 这一步也必须一次性 danger-full-access（Edge 要给自己 profile 目录授 ACL）
& E:\Anaconda\python.exe tools\md2pdf.py docs\01_technical_solution.md
& E:\Anaconda\python.exe tools\md2pdf.py --all          # 全部 docs/*.md
& E:\Anaconda\python.exe tools\md2pdf.py --selftest     # 只验链路
# 自证：导出的 PDF 能用 pdf_text.py 抽出中文（页数/字数应为非零）
& <runtime-python> tools\pdf_text.py docs\00_roadmap_30d.pdf -1 _pdf_out\_check00.txt --geom
```

显示屏本机 UI 项目（2026-10-15 起）

> 由「编码器 + 显示引擎」合并实验产生。本节独立于既有 §1.x / §4.x，不修改任何已签核结论。

U-1 合并 A/B：显示引擎的真实代价 = 0.126 ns 建立 / 0.014 ns 保持（实现后，2026-10-15）

为什么做：7 寸屏本机 UI 要与编码器共存于 50 MHz 单时钟域、1 拍 1 像素。写真正的渲染器之前必须先量出「加一套显示引擎要花掉多少时序余量、多少资源」，据此定 UI 复杂度上限——而不是写完才发现超时。

方法：同一脚本 `tools/synth_disp.tcl`、同一批 RTL（`rtl/*.v` 18 个 + `disp/*.v` 4 个）、同一 Vivado v2026.1，只切换顶层。

- 基线 A：`synth_design -top enc_top`，约束 `constr/enc_top.xdc`（50 pin）
- 实验 B：`synth_design -top enc_top_disp -verilog_define ENC_DISP_TAP`，合并约束 `constr/enc_top_disp.xdc`（88 ball）

日志与报告：A = `_disp_work/synth_encbase.log`（报告备份在 `_disp_work/prev_synth/`）；B = `_disp_work/synth_disp.log` + `_disp_work/timing.rpt` + `_disp_work/util.rpt`。

Design Timing Summary：

设计	WNS(ns)	TNS(ns)	失败端点	总端点	WHS(ns)	THS	WPWS
A 编码器	-0.119	-1.434	12	11,269	+0.037	0.000	9.500
B 编码器 + 显示引擎	-0.120	-1.446	12	11,613	+0.037	0.000	9.500
增量 (B-A)	**-0.001**	-0.012	**0**	+344	**0.000**	0	0

实现后的真实数字（2026-10-15 08:15，job pwsh-17 STEP B） —— 这才是可签核的口径：

设计	WNS(ns)	TNS(ns)	失败端点	总端点	WHS(ns)	THS
A 编码器（实现）	+0.959	0.000	0	11,275	+0.075	0.000
B 编码器 + 显示引擎（实现）	**+0.833**	0.000	**0**	11,629	**+0.061**	0.000
增量 (B-A)	**-0.126**	0	0	+354	**-0.014**	0

★★ 综合的 0.001 ns 把真实代价低估了约 126 倍：实现后的真实增量是 **-0.126 ns（建立）/ -0.014 ns（保持）**。原因是综合阶段所有网络都是 `unplaced`，显示引擎新增的布线代价根本不体现在估计里 —— **只有实现后的增量才可签核。**两个设计的失败端点都是 **0**，显示引擎没有引入任何违例。

最差路径的身份两次都没变，仍全在编码器里：建立 = `u_enc/u_regs/r_ablimit_reg[2]/C` → `u_enc/u_abz/r_hold_reg[0]/R`；保持 = `u_enc/u_cordic/g_iter[16].z_p_reg[17][5]/C` → `u_enc/u_cordic/angle_reg[1]/D`。⇒ 显示引擎至今**不在关键路径上。**

实现产物：`_disp_work/enc_top_disp.bit` 1,196,045 B（2026-10-15 07:47:53，DRC 0 Errors）；对比编码器生产位流 `_synth_work/enc_top.bit` 1,042,208 B。

实现后的资源增量：Slice LUT 6,298 → 6,457（+159，31.04%）；Slice Register 4,074 → 4,204（+130，10.11%）；Block RAM Tile 0 → 1.5（3.00%，RAMB18 = 1）；DSP 16 → 16；Bonded IOB 50 → 88（+38 = 30 LCD + 8 探针）；BUFGCTRL 1 → 2。

> 下表是**综合阶段**的数字，只用于读增量、以及与实现口径对照，不可当绝对值用（理由见 U-2）：

综合阶段的资源增量：Slice LUT 6,511 → 6,666（+155）；Slice Register 4,072 → 4,197（+125）；Block RAM Tile 0 → 1.5；DSP 16 → 16；Bonded IOB 50 → 88；BUFGCTRL 1 → 2。

结论：

1. **骨架级显示引擎在实现后花掉 0.126 ns 建立余量（占 0.959 的 13%）与 0.014 ns 保持余量（占 0.075 的 19%）。** UI 复杂度的天花板由**保持余量（只剩 +0.061 ns）**与 BRAM/LUT 预算共同决定 —— 都还够用，但**保持时间是其中最紧的一项**，每次合并后必须看。
2. 设计的最紧路径**不在显示引擎里**——两次综合的失败端点是**同一组 12 个**，路径为 `u_regs/r_inter0_reg[5]/C` → `u_abz/r_ovl_reg[0]/CE`。
3. **★ 这 12 个失败端点是先决存在的**（基线 A 单独综合同样是 12 个 / -0.119 ns），与显示引擎无关。
4. **★★ 而这 12 个失败端点在实现后并不存在**（见 U-2）——它们是纯综合阶段产物。所以 U-1 的综合数字只能读**增量**，不能读绝对值。

U-2 ★ 已定位：那 12 个失败端点在实现后不存在（纯综合阶段产物）

事实：`_synth_work/impl_timing.rpt`（编码器 v12 实现签核）为 **WNS +0.959、TNS 0.000、0/11,275 失败端点**，最差路径 `u_regs/r_ablimit_reg[2]/C` → `u_regs/r_hold_reg[0]/R`（data path 18.615 ns，logic 41.9% / route 58.1%，28 级逻辑）。而本次**综合**基线 A 在同一批 RTL、同一份已提交的 `constr/enc_top.xdc` 上给出 -0.119 ns / 12 失败。

根因（2026-10-15 实测确认）：是 **`opt_design` 重构了这条链的逻辑**，不是「综合的走线估计偏悲观」——后者已被数据推翻：综合估计的 route 是 9.363 ns，实现后的真实 route 反而**更高**（10.818 ns）。真正被削掉的是**逻辑**：综合 10.412 ns / 30 级（路径里含 `DSP48E1` 与 `MUXF7`）→ 实现 7.797 ns / 28 级（无 DSP、无 MUXF7），逻辑少 2.6 ns，足以盖过走线多出的 1.5 ns。

验证已执行（2026-10-15，job pwsh-17 STEP A）：对当前 HEAD RTL 重跑 `tools/impl_check.tcl`（`opt_design → place_design → route_design`，与 v12 签核同一流程），结果与 2026-10-12 签核**逐位相同**：WNS +0.959、TNS 0.000、**0/11,275 失败端点**、WHS +0.075、THS 0.000、WPWS 9.500；最差路径也逐字相同（`u_regs/r_ablimit_reg[2]/C` → `u_abz/r_hold_reg[0]/R`，18.615 ns，28 级，CARRY4=12 / LUT2=1 / LUT3=3 / LUT4=5 / LUT5=4 / LUT6=3）。

★ 结论：那 12 个失败端点并不存在于实现后的设计里——它们是纯综合阶段产物。 因此**综合 WNS 对本设计不是可用的绝对值代理**；§1.x 里「综合比实现乐观约 1.5 ns」的说法也不能再引用（那是拿 2026-10-08 的旧 RTL（9,424 端点）与 10-12 的新 RTL 对比得来的，两个被比较对象根本不同）。

★ U-1 的 A/B 增量（0.001 ns）不受影响：同一流程、同一阶段测量，绝对偏差相减时抵消。

存档：重跑的 A 报告在 `_disp_work/impl_enc_recheck/`；v12 签核原件已从 `_disp_work/prev_impl_signoff/` 复位回 `_synth_work/`，两者内容一致（位流同为可复现结果）。

沿用纪律：显示引擎只从**已寄存**的值抽头，绝不复用 `gap_num` / `ablimit` 一类组合链，也绝不给它们增加扇出。